

nageshnnazare/ nasm-know-hows

Contents

1.	NASM Assembly Programming - Complete Tutorial	18
	Course Structure	18
	Complete Curriculum	18
	Q&A Section - Deep Dives	26
	Progress Tracking	27
	How to Use This Tutorial	28
	Prerequisites	29
	Quick Command Reference	29
	Learning Goals	29
	Additional Resources	30
	Contributing	30
2.	Complete File Index	31
	Root Directory	31
	/topics - Core Curriculum	31
	/qna - Deep Dive Explanations	41
	File Statistics	44
	Recommended Reading Order	44
	Completion Status	46
	Finding What You Need	46
	Document Conventions	47
	Updates & Maintenance	47
	/topics - OS Internals (Level 9)	47
	/qna - Supplementary Deep Dives (OS Internals)	52
	How to Use This Collection	52
	Printable Checklist	53
3.	NASM Quick Reference Guide	56
	Build Commands	56
	Program Template	57
	Common Operations	57
	Control Flow	58
	Stack Operations	59
	Linux Syscalls (64-bit)	59
	Data Types	61
	Register Reference	61
	Memory Addressing	63
	String Operations	63
	Arithmetic (Advanced)	64

Function Calling (System V AMD64)	64
Debugging	65
Useful Tools	66
Tips & Tricks	66
Quick Bit Flags	67
Common Patterns	67
Resources	68
4. Linux x86-64 Syscall Reference	69
Quick Reference: Syscall Registers	69
Detailed Syscall Argument Table	70
Common File I/O Syscalls	75
Process Management Syscalls	78
Memory Management Syscalls	79
Time and Sleep Syscalls	81
Directory and File System Syscalls	82
Complete Syscall Number Reference	84
Error Handling	87
Practical Examples	89
Quick Tips	91
Resources	91
Related Topics	91
5. All Types of Jumps in Assembly	92
Overview	92
1. Unconditional Jumps	92
2. Conditional Jumps (Signed Comparisons)	93
3. Conditional Jumps (Unsigned Comparisons)	94
4. Conditional Jumps (Single Flag Tests)	96
5. Loop Jumps	98
6. Function Call/Return (Special Jumps)	99
7. Indirect Jumps	101
8. Jump Tables (Switch Statement)	102
9. Conditional Move (Alternative to Jumps)	105
10. Short vs Near vs Far Jumps	106
11. Jump Optimization Patterns	107
12. Complete Jump Reference Table	109
Summary	111
Practice Exercises	112
6. Instruction Size (Bytes) vs Execution Time (Cycles)	114
The Question	114
Understanding the Difference	114
Both Measurements Are Important!	115

Complete Comparison Table	116
Why Bytes Matter: Code Density	117
Why Cycles Matter: Execution Speed	118
When Each Measurement Matters Most	118
Both Matter Together: The I-Cache Effect	119
Practical Guidelines	120
How to Measure Each	121
Real-World Example: Optimizing a Loop	123
Summary	124
Related Topics	124
7. CMP vs TEST - What's the Difference?	125
Question	125
Quick Answer	125
Detailed Explanation	126
Side-by-Side Comparison	127
When to Use Each	128
Practical Examples	130
Performance Comparison	131
Common Mistakes	132
Equivalent Operations	133
Decision Tree	134
Memory Reference	135
TL;DR	135
Related Topics	136
8. Q&A: How Does fork() Actually Work?	137
The Question	137
Quick Answer	137
The Full Story	137
Common Fork Patterns	140
Why Fork Returns Twice	140
TL;DR	141
9. Q&A: How Does malloc() Actually Work?	142
The Question	142
Quick Answer	142
The Full Story	142
Assembly-Level Implementation	146
Common Bugs Explained	146
TL;DR	147
10. Instruction Encoding in x86-64	148
Overview	148

The Translation Process	148
Part 1: Instruction Format	148
Part 2: The REX Prefix - 64-bit Mode	149
Part 3: Opcodes	151
Part 4: ModR/M Byte	152
Part 5: SIB Byte (Scale-Index-Base)	155
Part 6: Complete Encoding Examples	158
Part 7: Why Instruction Size Matters	161
Part 8: Optimization Techniques	162
Part 9: Tools for Exploration	164
Part 10: Decoding Practice	165
Part 11: Advanced Topics	167
Summary Tables	168
Key Takeaways	168
Further Reading	169
Next Steps	169
11. int 0x80 vs syscall - When to Use Which?	170
Question	170
Quick Answer	170
Detailed Comparison	170
2. Register Usage - CRITICAL DIFFERENCE!	171
3. Syscall Numbers - DIFFERENT VALUES!	171
4. Performance	172
5. Practical Examples	172
6. What Happens If You Mix Them Up?	174
7. How to Detect Your Architecture	175
8. Decision Tree	175
9. Modern Best Practice	176
10. Quick Reference Card	176
TL;DR	176
12. Understanding Memory Addressing with [] Brackets	178
Question	178
The Key Concept	178
Analogy: House Addresses	178
Your Example Explained	179
Visual Representation	179
Comparison: With vs Without Brackets	180
Memory Addressing Modes	180
Practical Examples	182
Size Specifications	186
Common Patterns	187
Memory Safety Warning	188

TL;DR - The [] Summary	188
13. Push and Pop Explained	190
Question	190
Answer	190
PUSH Instruction	190
POP Instruction	190
Common Uses	191
Key Points	191
Bonus Question: Swapping EAX and EBX with Push/Pop	191
Alternative Swap Methods	193
Visual Representation	193
14. Sections (.text, .data, .bss) and XOR Optimization	195
Questions	195
The Memory Layout	195
Section Breakdown	197
Side-by-Side Comparison	201
Why This Matters	203
Summary Table: Sections vs C Code	203
Quick Answer	203
The Math	204
Detailed Comparison	204
All Zeroing Methods Compared	206
Practical Example	206
Real-World Usage	207
When XOR Can Be Confusing	208
Compiler Output Example	208
Summary Table: Zeroing Methods	209
TL;DR	209
15. Q&A: Signals, Traps, and Exception Handling	211
The Question	211
Quick Answer	211
Signal vs Exception vs Interrupt	211
How Signal Delivery Works	212
Catching SIGSEGV (Segmentation Fault)	214
Signal Lifecycle	216
Common Signals Quick Reference	216
Signal Handling Setup in Assembly	217
The Async-Signal-Safety Problem	218
TL;DR	218

16. Q&A: Virtual Memory Explained	219
The Question	219
Quick Answer	219
The Translation (One Slide Version)	219
Why Virtual Memory Exists	220
Page Tables: The 4-Level Structure	221
What a Page Fault Really Is	222
Observing Virtual Memory	223
Key Insights	223
TL;DR	224
17. gen_figures.py	225
18. Topic 1: Setup & First Program	270
Part 1: Installation	270
Part 2: The Assembly Workflow	270
Part 3: Program Structure	271
Part 4: Your First Program - Hello World	271
Part 5: Build and Run	272
Part 6: Understanding the Code	273
Part 7: 32-bit Version (Alternative)	274
Part 8: Common Mistakes & Debugging	275
Practice Exercises	275
Quick Reference: Linux 64-bit Syscalls	276
Knowledge Check	276
19. Topic 2: Registers & Data Types	278
Part 1: What Are Registers?	278
Part 2: The x86-64 Register Set	278
Part 3: Register Sizes - THE KEY CONCEPT!	279
Part 4: Critical Rule - 32-bit Zeroing	283
Part 5: Register Purposes (Conventions)	283
Part 6: Data Types in NASM	286
Part 7: Practical Examples	287
Part 8: Common Mistakes	291
Practice Exercises	292
Quick Reference Card	293
Knowledge Check	293
20. Topic 3: Basic Instructions	295
Overview	295
Part 1: The MOV Instruction	295
Part 2: Arithmetic Instructions	297

Part 3: Bitwise Instructions	302
Part 4: Shift and Rotate Instructions	305
Part 5: LEA - Load Effective Address	308
Part 6: Comparison Instructions	310
Part 7: Practical Examples	311
Part 8: Performance Tips	317
Part 9: Common Patterns	318
Practice Exercises	319
Quick Reference	321
Knowledge Check	322
21. Topic 4: Flags & Comparisons	323
Overview	323
Part 1: The RFLAGS Register	323
Part 2: Individual Flags Explained	324
Part 3: Flags and Instructions	329
Part 4: The CMP Instruction	330
Part 5: The TEST Instruction	331
Part 6: Reading Flag Combinations	333
Part 7: Practical Flag Examples	334
Part 8: Flag Manipulation Instructions	339
Part 9: Common Patterns and Idioms	340
Part 10: Debugging with Flags	342
Practice Exercises	343
Quick Reference	346
Knowledge Check	347
22. Topic 5: Conditional Jumps	348
Overview	348
Part 1: How Jumps Work	348
Part 2: Unconditional Jumps	349
Part 3: Conditional Jumps	350
Part 4: Using Conditional Jumps	353
Part 5: If/Else in Assembly	355
Part 6: Loops with Jumps	357
Part 7: Special Loop Instructions	360
Part 8: Practical Examples	361
Part 9: Optimization Tips	373
Part 10: Common Patterns	374
Practice Exercises	377
Quick Reference	380
Knowledge Check	381

23. Topic 6: Loops	383
Overview	383
Part 1: Loop Fundamentals	383
Part 2: Counted Loops (For Loop)	384
Part 3: Condition-Tested Loops	386
Part 4: The LOOP Instruction	388
Part 5: Loop Patterns	390
Part 6: Loop Control	393
Part 7: Nested Loops	396
Part 8: Loop Optimization	400
Part 9: Advanced Loop Techniques	403
Part 10: Practical Examples	405
Practice Exercises	417
Quick Reference	421
Knowledge Check	422
24. Topic 7: The Stack	423
Overview	423
Part 1: What is the Stack?	423
Part 2: Stack Pointer (RSP)	424
Part 3: PUSH Instruction	425
Part 4: POP Instruction	426
Part 5: Common Stack Patterns	427
Part 6: Stack Frames	430
Part 7: Calling Conventions (System V AMD64)	433
Part 8: Stack Alignment	434
Part 9: Practical Examples	435
Part 10: Stack Pitfalls	438
Part 11: Stack Debugging	439
Practice Exercises	440
Quick Reference	440
Knowledge Check	441
25. Topic 8: Functions & Procedures	442
Overview	442
Part 1: CALL and RET Instructions	442
Part 2: System V AMD64 Calling Convention	444
Part 3: Parameter Passing	445
Part 4: Return Values	449
Part 5: Local Variables	451
Part 6: Recursive Functions	453
Part 7: Function Pointers	455
Part 8: Variadic Functions	458

Part 9: Interfacing with C	460
Part 10: Advanced Techniques	463
Practice Exercises	465
Quick Reference	466
Knowledge Check	467
26. Topic 9: Calling Conventions	468
Overview	468
Part 1: Why Calling Conventions Matter	468
Part 2: System V AMD64 ABI (Linux/Unix x86-64)	469
Part 3: System V Examples	470
Part 4: Microsoft x64 Calling Convention (Windows)	475
Part 5: 32-bit Conventions (cdecl, stdcall, fastcall)	475
Part 6: Comparison Table	478
Part 7: Practical Example - Mixed Convention	478
Part 8: Stack Frame Layout	480
Part 9: Common Mistakes	480
Part 10: Debugging Tips	482
Practice Exercises	482
Quick Reference Card	484
Knowledge Check	484
27. Topic 10: Procedures	486
Overview	486
Part 1: The CALL Instruction	486
Part 2: The RET Instruction	488
Part 3: Simple Procedures	488
Part 4: Preserving Registers	491
Part 5: Local Variables	492
Part 6: Recursion	493
Part 7: Nested Calls	498
Part 8: Leaf vs Non-Leaf Procedures	500
Part 9: Tail Call Optimization	501
Part 10: Common Patterns	502
Part 11: Debugging Procedures	503
Practice Exercises	504
Quick Reference	506
Knowledge Check	507
28. Topic 11: Memory Addressing Modes	508
Overview	508
Part 1: The Square Brackets []	508
Part 2: All Addressing Modes	508
Part 3: Scale Factor Examples	512

Part 4: RIP-Relative Addressing (x86-64)	512
Part 5: Practical Examples	513
Part 6: Address Calculation Examples	516
Part 7: Size Specifiers	516
Part 8: Common Patterns	517
Part 9: Performance Considerations	518
Practice Exercises	518
Quick Reference	520
Knowledge Check	520
29. Topic 12: Arrays & Strings	521
Overview	521
Part 1: Arrays in Assembly	521
Part 2: Common Array Operations	522
Part 3: String Basics	523
Part 4: String Instructions	524
Part 5: String Operations Examples	526
Part 6: Multi-Dimensional Arrays	529
Practice Exercises	530
Quick Reference	531
30. Topic 13: Multiplication and Division	532
Overview	532
Multiplication Instructions	532
Division Instructions	536
Practical Applications	541
Common Pitfalls and Best Practices	547
Optimization Tips	548
Performance Characteristics	550
Practice Exercises	550
Summary	555
Next Topic	556
31. Topic 14: Shifts and Rotates (Advanced)	557
Overview	557
Logical Shifts	557
Arithmetic Shift	559
Rotate Instructions	561
Double-Precision Shifts	564
Practical Applications	566
Advanced: Barrel Shifter Emulation	572
Optimization Techniques	573
Performance Characteristics	575
Common Patterns and Idioms	575

Practice Exercises	576
Summary	581
Next Topic	582
32. Topic 15: Macros & Directives	583
Overview	583
Part 1: Simple Macros	583
Part 2: Multi-Line Macros	584
Part 3: Macro Parameters	586
Part 4: Local Labels	587
Part 5: Conditional Assembly	588
Part 6: File Inclusion	589
Part 7: Advanced Macro Techniques	590
Part 8: Practical Macro Library	591
Part 9: Complete Example	593
Part 10: Best Practices	595
Practice Exercises	595
Quick Reference	596
33. Topic 16: Linux System Calls	598
Overview	598
Syscall Mechanism	598
Common System Calls	601
Process Management	606
Memory Management	608
Time and Date	609
Process Information	610
Advanced I/O	611
Practical Example: Echo Server Skeleton	613
Error Handling	616
Performance Considerations	617
Summary	619
Practice Exercises	619
Next Topic	620
34. Topic 17: Interfacing with C	621
Overview	621
Why Interface with C?	621
Calling Conventions (Review)	622
Assembly Function Called from C	623
C Function Called from Assembly	627
Stack Alignment	630
Passing Structures	631
Inline Assembly in C	632

Creating a Reusable Library	635
Debugging Mixed C/Assembly	638
Common Pitfalls	639
Summary	641
Practice Exercises	641
Next Topic	642
35. Topic 18: SIMD Instructions (SSE/AVX)	643
Overview	643
SIMD Registers	643
SSE Instructions	645
Practical Example: Array Addition	649
Advanced SIMD Techniques	651
AVX Instructions	654
Complete Real-World Example: Dot Product	656
Integer SIMD Operations	658
Performance Considerations	659
Common Patterns	660
Summary	661
Practice Exercises	662
Next Topic	662
36. Topic 19: Performance and Optimization	664
Overview	664
CPU Architecture Fundamentals	664
Instruction Timing	665
Memory Hierarchy	666
Optimization Techniques	668
Branch Optimization	673
Data Alignment	677
Complete Optimization Example	678
Profiling and Measurement	681
Optimization Checklist	682
Common Pitfalls	683
Summary	684
Practice Exercises	684
Next Topic	685
37. Topic 20: Debugging and Tools	686
Overview	686
GDB - The GNU Debugger	686
Objdump - Disassembly Tool	691
Strace - System Call Tracer	692
Valgrind - Memory Debugger	693

Readelf - ELF File Analysis	694
Hexdump - Binary Data Viewer	695
Common Debugging Scenarios	696
Advanced Debugging Techniques	698
Reverse Engineering Basics	699
Best Practices	700
Debugging Checklist	701
Tool Summary	702
Practice Exercises	702
Summary	703
Congratulations!	704
38. Topic 21: Memory Allocation Internals	705
Overview	705
The Memory Hierarchy	705
Part 1: brk() — The Program Break	706
Part 2: mmap() — Memory Mapped Allocation	711
Part 3: How malloc() Actually Works	715
Part 4: Free List Strategies	723
Part 5: What Happens at Page Fault	729
Part 6: Thread-Local Allocation (Arenas)	730
Part 7: Memory Alignment	733
Part 8: Debugging Memory Issues	735
Exercises	738
Key Takeaways	739
Next Topic	739
39. Topic 22: Process Internals	740
Overview	740
Part 1: Process Memory Layout	741
Part 2: fork() — Process Duplication	746
Part 3: clone() — The Real Process Creation	752
Part 4: execve() — Replacing the Process Image	755
Part 5: fork + exec Pattern (Spawning a Program)	765
Part 6: Process Signals	768
Part 7: Process Groups and Sessions	771
Part 8: Pipes — Inter-Process Communication	774
Exercises	780
Key Takeaways	781
Next Topic	781
40. Topic 23: Virtual Memory & Paging	782
Overview	782
Part 1: Why Virtual Memory?	782

Part 2: Page Table Structure (x86-64, 4-Level)	784
Part 3: The TLB (Translation Lookaside Buffer)	789
Part 4: Page Sizes	792
Part 5: Page Faults in Detail	795
Part 6: Memory Protection	801
Part 7: Shared Memory	806
Part 8: Kernel Virtual Address Space	808
Part 9: ASLR (Address Space Layout Randomization)	809
Part 10: Memory-Mapped Files	812
Exercises	814
Key Takeaways	815
Next Topic	815
41. Topic 24: I/O Internals — How Print and Read Actually Work	816
Overview	816
Part 1: The write() Syscall Path	816
Part 2: File Descriptors Explained	819
Part 3: Terminal I/O — The Full Path	821
Part 4: Buffered I/O (Why printf Doesn't Write Immediately)	825
Part 5: The read() Syscall Path	829
Part 6: Implementing printf-like Formatting	835
Part 7: Standard I/O Streams and Redirection	841
Part 8: ANSI Escape Sequences	844
Part 9: Direct Hardware I/O (Framebuffer)	847
Part 10: writev() — Scatter/Gather I/O	850
Exercises	851
Key Takeaways	851
Next Topic	851
42. Topic 25: ELF Binary Format	852
Overview	852
Part 1: ELF File Structure	853
Part 2: Object Files (.o) — Relocatable ELF	858
Part 3: The Linker (ld) — Combining Object Files	861
Part 4: Program Headers — Runtime Loading	863
Part 5: Dynamic Linking	864
Part 6: How the Kernel Loads an ELF	869
Part 7: Symbol Tables	870
Part 8: Debugging Information (DWARF)	871
Part 9: ELF Inspection Tools	872
Part 10: Self-Modifying ELF Tricks	873
Exercises	875
Key Takeaways	876
Next Topic	876

43. Topic 26: Interrupts & Exceptions	877
Overview	877
Part 1: Interrupt Types	878
Part 2: The Interrupt Descriptor Table (IDT)	879
Part 3: What Happens During an Exception	881
Part 4: Hardware Interrupts (IRQs)	884
Part 5: Debugger Breakpoints (INT 3)	886
Part 6: The syscall Instruction vs INT 0x80	888
Part 7: vDSO — Virtual Dynamically-linked Shared Object	891
Part 8: Interrupt Latency and Real-Time	892
Part 9: Exception Handling in Practice	893
Part 10: Interrupt Descriptor Table Setup (Kernel Context)	899
Exercises	901
Key Takeaways	901
Next Topic	902
44. Topic 27: Context Switching & Scheduling	903
Overview	903
Part 1: What Is Process Context?	903
Part 2: The Context Switch in Detail	905
Part 3: The switch_to Macro (Assembly)	906
Part 4: FPU/SSE/AVX State Switching	908
Part 5: Thread Context Switching	909
Part 6: Scheduler Algorithms	911
Part 7: Measuring Context Switch Cost	913
Part 8: CPU Affinity and NUMA	917
Part 9: Process States	919
Part 10: Coroutines — User-Space Context Switching	923
Exercises	927
Key Takeaways	928
Next Topic	928
45. Topic 28: Synchronization Primitives	929
Overview	929
Part 1: Memory Ordering and the Problem	930
Part 2: Atomic Instructions	932
Part 3: Spinlocks	935
Part 4: The Futex Syscall	937
Part 5: Lock-Free Data Structures	943
Part 6: Read-Write Lock	946
Part 7: Semaphores	949
Part 8: Condition Variables	950
Part 9: Producer-Consumer with Ring Buffer	953

Part 10: Practical Example — Thread-Safe Counter	956
Exercises	960
Key Takeaways	961
Further Reading	961

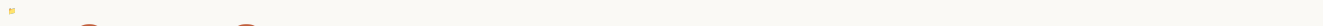


CHAPTER 1

NASM Assembly Programming - Complete Tutorial



Welcome to your comprehensive NASM assembly programming course! This tutorial will take you from beginner to advanced assembly programmer through structured, hands-on lessons.

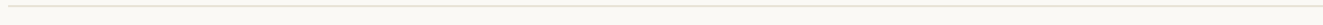


Course Structure

This course is organized into progressive levels with 20 core topics plus advanced projects.

Directory Structure

```
nasm-tutorial/  
├── README.md                # This file - course overview  
├── topics/                  # Individual topic lessons  
│   ├── topic-01-setup.md  
│   ├── topic-02-registers.md  
│   └── ...  
└── qna/                     # Common questions and deep dives  
    ├── push-pop-explained.md  
    ├── int-vs-syscall.md  
    ├── memory-addressing.md  
    └── sections-and-optimization.md
```



Complete Curriculum

LEVEL 1: Foundation (Weeks 1-2)

Topic 1: Setup & First Program

- Install NASM and linker
- Understand assembly workflow
- Write “Hello World” using syscalls
- Learn basic program structure

Topic 2: Registers & Data Types

- General purpose registers
- Register sizes (64, 32, 16, 8-bit)
- Data types: DB, DW, DD, DQ

- Register conventions

Supplementary: Instruction Encoding

- How assembly becomes machine code
- REX prefix and 64-bit mode
- Opcode, ModR/M, SIB bytes
- Why some instructions are shorter
- Optimization techniques

Topic 3: Basic Instructions

- MOV and data movement
- Arithmetic: ADD, SUB, INC, DEC, NEG, MUL, DIV
- Bitwise: AND, OR, XOR, NOT, TEST
- Shifts and rotates: SHL, SHR, SAL, SAR, ROL, ROR
- LEA for address calculation and arithmetic
- CMP for comparisons

LEVEL 2: Control Flow (Weeks 3-4)

Topic 4: Flags & Comparisons

- The RFLAGS register structure
- Six arithmetic flags: CF, ZF, SF, OF, PF, AF
- How instructions affect flags
- CMP instruction (compare)
- TEST instruction (test bits)
- Signed vs unsigned flag interpretation
- Flag manipulation and common patterns

Topic 5: Conditional Jumps

- How jumps work (RIP register)
- Unconditional jumps: JMP
- Conditional jumps based on flags
- Signed comparisons: JG, JGE, JL, JLE, JE, JNE
- Unsigned comparisons: JA, JAE, JB, JBE
- If/else statement implementation
- Loop construction
- LOOP instruction and variants

- Practical examples and patterns

Topic 6: Loops

- Loop fundamentals and structure
 - Counted loops (for equivalent)
 - Condition-tested loops (while/do-while)
 - LOOP instruction and variants
 - Loop patterns (array iteration, search, accumulation)
 - Loop control (break/continue)
 - Nested loops
 - Loop optimization techniques
 - Practical examples (bubble sort, Fibonacci, matrix multiplication)
-

LEVEL 3: The Stack (Week 5)

Topic 7: Stack Operations

- `push` and `pop` mechanics
- Stack pointer (ESP/RSP)
- Stack alignment
- Complete with C code equivalents

Topic 8: Stack Frames

- Base pointer (EBP/RBP)
 - Function prologue/epilogue
 - Local variables on stack
 - Complete with C code equivalents
-

LEVEL 4: Functions & Procedures (Weeks 6-7)

Topic 9: Calling Conventions

- `cdecl`, `stdcall`, `fastcall`
- System V AMD64 ABI (Linux)
- Microsoft x64 (Windows)
- Complete with C code equivalents

Topic 10: Procedures

- `call` and `ret` instructions
- Return values
- Recursive functions
- Complete with C code equivalents

LEVEL 5: Memory & Addressing (Week 8)

Topic 11: Memory Addressing Modes

- Direct, indirect, indexed
- Scaled indexed addressing
- RIP-relative (x64)
- SIB addressing with scale factors
- Complete with C code equivalents

Topic 12: Arrays & Strings

- String instructions (`MOVS`, `LODS`, `STOS`, `SCAS`, `CMPS`)
- Direction flag
- `rep` prefix
- Complete with C code equivalents

LEVEL 6: Advanced Operations (Weeks 9-10)

Topic 13: Multiplication & Division

- `mul` / `imul` - unsigned and signed multiplication
- `div` / `idiv` - unsigned and signed division
- 64-bit results and `RDX:RAX` usage
- Complete with C code equivalents

Topic 14: Shifts & Rotates (Advanced)

- Logical shifts: `shl`, `shr`
- Arithmetic shift: `sar`
- Rotates: `rol`, `ror`, `rcl`, `rcr`
- Complete with C code equivalents

Topic 15: Macros & Directives

- `%define`, `%macro`
 - Conditional assembly (`%if`, `%ifdef`)
 - `%include` and file organization
 - Complete with practical examples
-

LEVEL 7: System Programming (Weeks 11-12)

Topic 16: Linux System Calls

- `syscall` instruction and mechanism
- Syscall numbers and register usage (RAX, RDI, RSI, RDX, R10, R8, R9)
- File I/O: read, write, open, close
- Process management: fork, exec, wait
- Memory management: brk, mmap, munmap
- Error handling and return values
- Complete with C code equivalents

Topic 17: Interfacing with C

- Calling conventions (System V AMD64 ABI)
 - Calling C from assembly
 - Calling assembly from C
 - Stack alignment requirements
 - Using `printf`, `malloc`, and libc functions
 - Creating reusable assembly libraries
 - Inline assembly in C
 - Complete with C code equivalents
-

LEVEL 8: Optimization & Advanced (Weeks 13-14)

Topic 18: SIMD Instructions

- SSE/AVX fundamentals
- XMM/YMM registers (128-bit/256-bit)
- Packed operations on multiple data elements
- Vector arithmetic, comparisons, shuffling
- Practical examples (array sum, dot product)
- Performance gains from vectorization

- Complete with C code equivalents

Topic 19: Performance & Optimization

- CPU architecture and pipeline fundamentals
- Instruction latency vs throughput
- Memory hierarchy and cache optimization
- Loop unrolling and multiple accumulators
- Branch prediction and branchless code
- Strength reduction and induction variables
- Data alignment and SIMD optimization
- Profiling with perf and RDTSC
- Complete optimization checklist

Topic 20: Debugging & Tools

- GDB comprehensive guide (breakpoints, watchpoints, TUI mode)
- Objdump for disassembly
- Strace for syscall tracing
- Valgrind for memory debugging
- Readelf for ELF analysis
- Common debugging scenarios (segfault, infinite loop, wrong results)
- Core dumps and remote debugging
- Reverse engineering basics

LEVEL 9: OS Internals at Assembly Level (Weeks 15-18)

Topic 21: Memory Allocation Internals

- How malloc/free actually work at the assembly level
- brk() and mmap() syscalls for obtaining memory from the kernel
- Free list management, chunk headers, bins
- Chunk coalescing and fragmentation
- Demand paging and page faults
- Thread-local arenas
- Implementing a custom allocator in assembly

Topic 22: Process Internals

- How fork() duplicates a process (Copy-on-Write)
- clone() — the real syscall behind fork and threads

- `execve()` — replacing process image, ELF loading
- The initial stack layout after `execve` (`argc`, `argv`, `envp`, `auxv`)
- Signals — installation, delivery mechanism, signal handlers
- Pipes and inter-process communication
- Implementing shell pipelines in assembly

Topic 23: Virtual Memory & Paging

- 4-level page table structure (PML4 → PDPT → PD → PT)
- Page Table Entry format (Present, R/W, U/S, NX bits)
- TLB (Translation Lookaside Buffer) and performance
- Page sizes: 4KB, 2MB huge pages, 1GB gigantic pages
- Page faults: demand paging, COW, stack growth
- `mprotect()` and memory protection
- Guard pages, ASLR, shared memory
- Memory-mapped files

Topic 24: I/O Internals

- Complete path of `write()` from syscall to screen pixels
- File descriptor table internals
- Terminal I/O: TTY layer, line discipline, PTY
- Buffered I/O (why `printf` doesn't write immediately)
- Implementing `printf`-like formatting in assembly
- `read()` path: keyboard → IRQ → driver → TTY → your buffer
- Blocking vs non-blocking I/O, `poll/select`
- ANSI escape sequences, direct framebuffer access
- `writev()` scatter/gather I/O

Topic 25: ELF Binary Format

- Complete ELF file structure (header, program headers, sections)
- Creating a minimal ELF executable by hand (~170 bytes)
- Object files (`.o`): relocations and symbol resolution
- The linker: symbol resolution, section merging, address assignment
- Segments vs sections (runtime vs link-time view)
- Dynamic linking: GOT, PLT, lazy binding
- Position-Independent Code (PIC/PIE) and ASLR
- How the kernel loads an ELF (`execve` internals)
- DWARF debug information

Topic 26: Interrupts & Exceptions

- Interrupt types: exceptions, hardware IRQs, software interrupts
- Interrupt Descriptor Table (IDT) structure and entries
- CPU exception handling: page fault, GPF, divide error
- What the CPU pushes on the stack during an exception
- Hardware interrupt delivery (APIC, IRQ routing)
- Timer interrupt and preemptive multitasking
- Debugger breakpoints (INT 3) and hardware watchpoints
- SYSCALL vs INT 0x80 — why syscall is faster
- vDSO: syscalls without entering the kernel

Topic 27: Context Switching & Scheduling

- What constitutes process context (all saved state)
- The switch_to mechanism in assembly
- Voluntary vs involuntary context switches
- FPU/SSE/AVX state saving (XSAVE/XRSTOR)
- Thread switching vs process switching
- Thread Local Storage (TLS) and the FS register
- CFS scheduler: vruntime, red-black tree, time slices
- Measuring context switch cost
- CPU affinity and NUMA
- User-space context switching (coroutines)

Topic 28: Synchronization Primitives

- x86-64 memory ordering model (TSO)
- Memory barriers (MFENCE, SFENCE, LFENCE)
- Atomic instructions: LOCK prefix, XCHG, CMPXCHG
- Compare-and-Swap (CAS) patterns and retry loops
- Spinlocks: test-and-set, ticket locks
- Futex: fast userspace mutex (hybrid kernel/user approach)
- Implementing a full mutex with futex in assembly
- Lock-free data structures (Treiber stack)
- Read-write locks, semaphores, condition variables
- Producer-consumer with lock-free ring buffer

Q&A Section - Deep Dives

These files contain detailed explanations of specific questions that came up during learning:

1. **Push and Pop Explained** - Stack mechanics - LIFO behavior - Swapping values with push/pop
2. **int 0x80 vs syscall** - When to use each - Register differences - Syscall number differences - Performance implications
3. **Memory Addressing with [] Brackets** - Dereferencing explained - All addressing modes - Practical examples - Common mistakes
4. **Sections (.text, .data, .bss) and XOR Optimization** - Memory layout - Section purposes - Relation to C code - Why `xor reg, reg` is better than `mov reg, 0`
5. **CMP vs TEST - What's the Difference?** - CMP does subtraction, TEST does AND - When to use each instruction - Performance comparison - Common mistakes and decision tree
6. **Linux x86-64 Syscall Reference** - Complete syscall register usage (RAX, RDI, RSI, RDX, R10, R8, R9) - Common syscalls with examples (read, write, open, close, exit) - File I/O, process control, memory management - Complete syscall number tables - Error handling and return values - Practical examples for each syscall
7. **All Types of Jumps - Complete Reference** - Unconditional jumps (JMP) - Conditional jumps: signed (JG, JL, etc.) and unsigned (JA, JB, etc.) - Flag-specific jumps (JZ, JS, JC, JO, JP) - Loop instructions (LOOP, LOOPE, LOOPNE) - Function calls (CALL/RET) - Indirect jumps and jump tables (switch statements) - Conditional move (CMOVcc) as branchless alternative - Complete reference table with all 40+ jump variants - Performance optimization patterns
8. **Instruction Size (Bytes) vs Execution Time (Cycles)** - Why we measure BOTH bytes and cycles - Bytes = space in memory (code size, cache efficiency) - Cycles = execution time (performance, speed) - How instruction size affects L-cache performance - Real-world examples showing both measurements matter - When to optimize for bytes vs cycles - Measuring techniques for each (objdump, RDTSC, perf) - Complete comparison with benchmarks
9. **How malloc() Actually Works** - The hidden chunk header (why free knows the size) - Free list bins (fast bins, small bins, large bins) - malloc decision tree: tcache → fastbin → brk → mmap - Why free() doesn't return memory to the OS - Assembly-level implementation - Common bugs: use-after-free, double-free, heap overflow
10. **How fork() Actually Works**
 - Copy-on-Write explained (no actual memory copy!)
 - Page table manipulation during fork
 - COW page fault sequence
 - Why fork() returns different values in parent/child
 - fork+exec efficiency with COW
 - vfork for ultra-fast fork+exec

11. Virtual Memory Explained

- Virtual addresses are an illusion
- The 4-level page table walk visualized
- TLB: why translation isn't slow (99%+ hit rate)
- What page faults really are (5 cases)
- `/proc/self/maps`: observing your own mappings
- Key insights: free until touched, isolation, shared libraries

12. Signals, Traps, and Exception Handling

- Signal vs exception vs interrupt
- How the kernel delivers signals (stack modification)
- Catching SIGSEGV and resuming execution
- Signal lifecycle: generated → pending → delivered
- Common signals reference table
- Async-signal-safety: what you can/can't do in handlers

Progress Tracking

Foundation (Weeks 1-2):

- Topic 1: Setup & First Program
- Topic 2: Registers & Data Types
- Topic 3: Basic Instructions

Control Flow (Weeks 3-4):

- Topic 4: Flags & Comparisons
- Topic 5: Conditional Jumps
- Topic 6: Loops

The Stack (Week 5):

- Topic 7: Stack Operations
- Topic 8: Stack Frames

Functions & Procedures (Weeks 6-7):

- Topic 9: Calling Conventions
- Topic 10: Procedures

Memory & Addressing (Week 8):

- Topic 11: Memory Addressing Modes

- Topic 12: Arrays & Strings

Advanced Operations (Weeks 9-10):

- Topic 13: Multiplication & Division
- Topic 14: Shifts & Rotates (Advanced)
- Topic 15: Macros & Directives

System Programming (Weeks 11-12):

- Topic 16: Linux System Calls
- Topic 17: Interfacing with C

Optimization & Advanced (Weeks 13-14):

- Topic 18: SIMD Instructions
- Topic 19: Performance & Optimization
- Topic 20: Debugging & Tools

OS Internals (Weeks 15-18):

- Topic 21: Memory Allocation Internals
- Topic 22: Process Internals
- Topic 23: Virtual Memory & Paging
- Topic 24: I/O Internals
- Topic 25: ELF Binary Format
- Topic 26: Interrupts & Exceptions
- Topic 27: Context Switching & Scheduling
- Topic 28: Synchronization Primitives

Supplementary:

- Instruction Encoding (Advanced)
- All 12 Q&A deep-dive articles
- Comprehensive syscall reference

ALL 28 TOPICS COMPLETED! Over 50,000 lines of comprehensive content!

How to Use This Tutorial

1. **Sequential Learning:** Follow topics in order - each builds on previous knowledge
2. **Hands-On Practice:** Type and run every code example
3. **Complete Exercises:** Practice problems reinforce concepts
4. **Check Understanding:** Review knowledge checks before moving on



5. Reference Q&A: Deep dive into specific questions as needed

Prerequisites

- Linux system (64-bit recommended)
- NASM installed: `sudo yum install nasm`
- GCC for linking: `sudo yum install gcc`
- Text editor (vim, nano, VSCode, etc.)
- Terminal access

Quick Command Reference

```
# Assemble 64-bit
nasm -f elf64 program.asm -o program.o

# Link
ld -o program program.o

# Run
./program

# Check exit code
echo $?

# Assemble with debug info
nasm -f elf64 -g -F dwarf program.asm

# Debug with GDB
gdb ./program
```

Learning Goals

By completing this course, you will be able to:

- Write complete assembly programs from scratch
- Understand x86-64 architecture deeply
- Read and understand compiler output
- Optimize code at the lowest level
- Debug assembly with confidence

- Interface assembly with high-level languages
- Work with system calls and OS interfaces
- Apply assembly knowledge to reverse engineering and security
- Understand how malloc/free manage memory (brk, mmap, free lists)
- Know exactly what fork/exec do at the page table level
- Explain virtual memory translation (4-level page tables, TLB)
- Trace the path of a write() from syscall to screen pixels
- Parse and understand ELF binaries (headers, segments, relocations)
- Implement synchronization primitives (spinlocks, mutexes, lock-free structures)
- Understand how the OS context-switches between processes

Additional Resources

- **Official NASM Documentation:** <https://www.nasm.us/docs.php>
 - **Intel Manuals:** Intel® 64 and IA-32 Architectures Software Developer Manuals
 - **Linux Syscall Table:** <https://filippo.io/linux-syscall-table/>
 - **Practice:** [pwn.college](#), [crackmes.one](#)
-

Contributing

Found an error? Have a question? Want to add content? Feel free to modify and extend these materials!

Happy Assembly Programming!

CHAPTER 2

Complete File Index

A comprehensive index of all files in this NASM tutorial collection.

Root Directory

README.md (Main Overview)

Purpose: Course overview, complete curriculum (20 topics), progress tracking

Content: Full learning path from beginner to advanced, organized by weeks and levels

Start Here: This is your main navigation file

QUICK_REFERENCE.md (Cheat Sheet)

Purpose: Fast lookup reference for common operations and syntax

Content: Commands, templates, syscalls, addressing modes, debugging tips

Use When: You need to quickly look up syntax or common patterns

INDEX.md (This File)

Purpose: Complete file listing with descriptions

Content: Overview of every file in the tutorial

`/topics` - Core Curriculum

Sequential lessons that form the main course content. Work through these in order.

topic-01-setup.md **Completed**

Level: Foundation (Week 1)

What You'll Learn:

- Installing NASM and build tools
- Understanding the assembly workflow (.asm → .o → executable)
- Program structure (.data, .bss, .text sections)
- Your first "Hello World" program
- Debugging with GDB

Key Concepts:

- `nasm -f elf64` command

- `syscall` instruction
- Linux syscalls (`sys_write`, `sys_exit`)
- `$ - label` for calculating lengths

Exercises: 5 practice problems included

topic-02-registers.md **Completed**

Level: Foundation (Week 1)

What You'll Learn:

- The 16 general-purpose registers (RAX-R15)
- Register sizes: 64, 32, 16, 8-bit access
- Critical 32-bit zeroing behavior
- Register naming conventions and purposes
- Data types: DB, DW, DD, DQ
- Reserved vs defined data (RESB vs DB)

Key Concepts:

- RAX → EAX → AX → AH/AL hierarchy
- Writing to 32-bit register zeros upper 32 bits
- Register roles (accumulator, counter, stack pointer)
- System V calling convention (RDI, RSI, RDX...)

Exercises: 5 practice problems included

instruction-encoding.md **Completed (Supplementary)**

Level: Advanced/Supplementary

What You'll Learn:

- How assembly instructions become machine code
- Instruction format: Prefixes, Opcode, ModR/M, SIB, Displacement, Immediate
- REX prefix for 64-bit mode
- Why `xor eax, eax` is 2 bytes but `mov eax, 0` is 5 bytes
- ModR/M byte structure and addressing modes
- SIB byte for complex addressing
- Optimization techniques based on encoding

Key Concepts:

- Variable-length instructions (1-15 bytes)

- REX.W for 64-bit operations
- ModR/M byte for operand specification
- Special encodings (PUSH is 1 byte!)
- Instruction cache implications
- Decoding practice exercises

Tools Covered:

- objdump for disassembly
- NASM listing files
- Online assemblers 

When to Read: After Topic 2, or when curious about why certain instructions are “better”

topic-03-basic-instructions.md **Completed**

Level: Foundation (Week 2)

What You’ll Learn:

- MOV instruction and all its forms
- Arithmetic operations: ADD, SUB, INC, DEC, NEG
- Multiplication and division: MUL, IMUL, DIV, IDIV
- Bitwise operations: AND, OR, XOR, NOT, TEST
- Shift and rotate: SHL, SHR, SAL, SAR, ROL, ROR
- LEA for efficient arithmetic
- CMP instruction for comparisons
- Practical patterns and optimization tips

Key Concepts:

- MOV restrictions (no mem-to-mem)
- Setting up RDX:RAX for division
- Flags affected by each instruction
- LEA as a powerful arithmetic tool
- Choosing the most efficient instruction
- Common coding patterns

Practical Examples:

- Sum array elements
- Find maximum value
- Calculate factorial
- Bit manipulation

- Using LEA for complex math

Exercises: 5 practice problems with solutions

topic-04-flags.md **Completed**

Level: Control Flow (Week 3)

What You'll Learn:

- The RFLAGS register and its structure
- Six arithmetic flags: CF, ZF, SF, OF, PF, AF
- When each flag is set
- How instructions affect different flags
- CMP instruction in detail
- TEST instruction for bit testing
- Reading flag combinations
- Signed vs unsigned comparisons
- Flag manipulation instructions
- Common patterns and debugging

Key Concepts:

- CF for unsigned overflow/borrow
- ZF for zero results and equality
- SF for sign (negative numbers)
- OF for signed overflow
- CMP performs subtraction without storing
- TEST performs AND without storing
- Flag combinations for conditionals
- Which instructions affect which flags

Practical Examples:

- Zero checking
- Range validation
- Sign detection
- Overflow handling
- Multi-precision arithmetic
- Parity checks

Exercises: 5 practice problems with solutions

topic-05-jumps.md Completed

Level: Control Flow (Week 3)

What You'll Learn:

- Unconditional and conditional jumps
 - All conditional jump variants (signed/unsigned)
 - If/else and switch statement implementation
 - Practical examples with complete C equivalents
-

topic-06-loops.md Completed

Level: Control Flow (Week 4)

What You'll Learn:

- Loop patterns (for, while, do-while)
 - LOOP instruction and optimization
 - Nested loops with examples (bubble sort, matrix ops)
 - Complete with C code equivalents
-

topic-07-stack.md Completed

Level: The Stack (Week 5)

What You'll Learn:

- PUSH/POP mechanics and stack pointer
 - Stack alignment (16-byte requirement)
 - Practical examples with C equivalents
-

topic-08-functions.md Completed

Level: The Stack (Week 5)

What You'll Learn:

- Stack frames and base pointer
 - Function prologue/epilogue patterns
 - Local variables on stack
 - Complete with C code equivalents
-

topic-09-calling-conventions.md **Completed**

Level: Functions & Procedures (Week 6)

What You'll Learn:

- cdecl, stdcall, fastcall conventions
 - System V AMD64 ABI (Linux) detailed
 - Microsoft x64 (Windows) differences
 - Parameter passing in registers vs stack
 - Complete with C code equivalents
-

topic-10-procedures.md **Completed**

Level: Functions & Procedures (Week 7)

What You'll Learn:

- CALL and RET instructions
 - Return values and register preservation
 - Recursive functions (factorial, Fibonacci)
 - Complete with C code equivalents
-

topic-11-memory-addressing.md **Completed**

Level: Memory & Addressing (Week 8)

What You'll Learn:

- All 7 addressing modes explained
 - Direct, indirect, indexed, scaled indexed
 - RIP-relative addressing (x64)
 - Complete with C code equivalents
-

topic-12-arrays-strings.md **Completed**

Level: Memory & Addressing (Week 8)

What You'll Learn:

- String instructions (MOVS, LODS, STOS, SCAS, CMPS)
 - Direction flag and REP prefix
 - Array manipulation patterns
 - Complete with C code equivalents
-

topic-13-multiplication-division.md **Completed**

Level: Advanced Operations (Week 9)

What You'll Learn:

- MUL (unsigned) and IMUL (signed) multiplication
- DIV (unsigned) and IDIV (signed) division
- 128-bit results (RDX:RAX)
- Sign extension (CDQ, CQO)
- Practical examples: factorial, GCD, integer sqrt, decimal to ASCII
- Performance characteristics and optimization
- Complete with C code equivalents

Key Topics:

- Single, double, and triple operand IMUL forms
- Proper sign extension before IDIV
- Common pitfalls (division by zero, forgetting to clear EDX)
- When to use shifts instead of multiply/divide

topic-14-shifts-rotates.md **Completed**

Level: Advanced Operations (Week 9)

What You'll Learn:

- Logical shifts (SHL, SHR) for unsigned
- Arithmetic shift (SAR) for signed division
- Rotates (ROL, ROR, RCL, RCR)
- Double-precision shifts (SHLD, SHRD)
- Bit manipulation patterns
- Complete with C code equivalents

Practical Applications:

- Bit extraction and field packing
- Fast multiply/divide by powers of 2
- Byte swapping (endianness)
- Count leading zeros
- Alignment checks
- Barrel shifter emulation

topic-15-macros.md **Completed**

Level: Advanced Operations (Week 10)

What You'll Learn:

- Text substitution with %define
 - Multi-line macros with %macro
 - Conditional assembly (%if, %ifdef)
 - File inclusion and code organization
 - Complete with C code equivalents
-

topic-16-system-calls.md **Completed**

Level: System Programming (Week 11)

What You'll Learn:

- SYSCALL instruction and mechanism (user → kernel)
- Register mapping (RAX, RDI, RSI, RDX, R10, R8, R9)
- File I/O: open, read, write, close with flags and modes
- Process management: fork, execve, wait4
- Memory management: brk, mmap, munmap
- Time functions: time, clock_gettime, nanosleep
- Error handling (-errno in RAX)
- Complete echo server skeleton example
- Complete with C code equivalents

Key Concepts:

- Syscall vs int 0x80 (64-bit vs 32-bit)
 - Why R10 instead of RCX (RCX saves return address)
 - Syscall overhead and batching optimization
-

topic-17-interfacing-c.md **Completed**

Level: System Programming (Week 12)

What You'll Learn:

- System V AMD64 ABI in depth
- Calling C functions from assembly (printf, malloc, fgets)
- Calling assembly from C
- Stack alignment (16-byte requirement critical)
- Passing structures by value vs pointer

- Inline assembly in C with GCC extended asm
- Creating reusable assembly libraries
- Complete with C code equivalents

Practical Examples:

- String length function callable from C
- Using printf for debugging
- Dynamic memory allocation
- Math library (factorial, power, GCD)
- Debugging mixed C/Assembly with GDB

topic-18-simd.md **Completed**

Level: Optimization & Advanced (Week 13)

What You'll Learn:

- SIMD fundamentals (Single Instruction Multiple Data)
- XMM registers (128-bit) and YMM registers (256-bit)
- Packed operations: 4x float, 2x double at once
- SSE instructions: MOVAPS, ADDPS, MULPS, etc.
- AVX non-destructive operations
- Complete with C code equivalents

Real-World Examples:

- Array addition (4x speedup)
- Array sum with horizontal reduction
- Dot product with benchmarking
- Shuffle and broadcast operations
- Integer SIMD operations

Performance:

- Typical 3-8x speedup for vectorizable code
- Alignment requirements (16/32 bytes)
- When SIMD helps vs hurts

topic-19-performance.md **Completed**

Level: Optimization & Advanced (Week 13)

What You'll Learn:

- CPU architecture: pipeline, superscalar execution
- Instruction latency vs throughput
- Memory hierarchy (L1/L2/L3 cache, RAM)
- Cache lines and false sharing
- Complete with C code equivalents

Optimization Techniques:

- Loop unrolling (reduce branch overhead)
- Multiple accumulators (break dependency chains)
- Software pipelining (overlap operations)
- Strength reduction (LEA tricks, shifts)
- Induction variable elimination
- Branch prediction and branchless code
- Data alignment optimization

Complete Example:

- Optimized array average: 10-15x speedup
- Before/after comparison with explanation

topic-20-debugging.md **Completed**

Level: Optimization & Advanced (Week 14)

What You'll Learn:

- GDB comprehensive guide (50+ commands)
- TUI mode for visual debugging
- Breakpoints, watchpoints, conditional breaks
- Examining registers and memory
- Disassembly with objdump
- System call tracing with strace
- Memory debugging with valgrind
- ELF analysis with readelf
- Binary viewing with hexdump

Debugging Scenarios:

- Fixing segmentation faults

- Detecting infinite loops
- Correcting wrong results
- Using core dumps
- Remote debugging with gdbserver

Best Practices:

- Defensive programming patterns
- Debug vs release builds
- ? • Common bug locations checklist
- Tool selection guide

/qna - Deep Dive Explanations

Detailed answers to specific questions that arose during learning. Read these when you encounter the topic or have similar questions.

push-pop-explained.md

Question: “What does push and pop do in assembly?”

Topics Covered:

- Stack mechanics (LIFO data structure)
- How PUSH decrements RSP and writes data
- How POP reads data and increments RSP
- Common uses: saving registers, function calls, local variables
- Bonus: Swapping registers with push/pop

Related Lessons: Topic 7 (Stack Operations)

Read When: Before learning about functions and stack frames

int-vs-syscall.md

Question: “When to use int 0x80, and when to use syscall?”

Topics Covered:

- Architecture differences (32-bit vs 64-bit)
- Register conventions (EAX/EBX vs RAX/RDI)
- Different syscall numbers between 32/64-bit
- Performance comparison (~300 vs ~100 cycles)
- Decision tree for choosing the right method
- Complete working examples of both

Related Lessons: Topic 1 (Setup & First Program)

Read When: Confused about which system call method to use

memory-addressing.md

Question: "What does the [] syntax mean?"

Topics Covered:

- Memory dereferencing concept
- Bracket syntax = "go to this address"
- All 6 addressing modes with examples
- Practical array and string access patterns
- Size specifiers (byte, word, dword, qword)
- Common mistakes and memory safety warnings

Related Lessons: Topic 2 (Registers), Topic 11 (Memory Addressing Modes)

Read When: Learning about pointers and memory access

sections-and-optimization.md

Questions:

1. "What is .text .bss .data sections with respect to C code?"
2. "Why xor rax, rax is used instead of mov rax, 0?"

Topics Covered:

Part 1 - Sections:

- Complete memory layout diagram
- .text section: executable code (read-only)
- .data section: initialized variables (in file)
- .bss section: uninitialized variables (NOT in file!)
- Relation to C global/static variables
- File size optimization techniques

Part 2 - XOR Optimization:

- Why `xor rax, rax` is 2-5 bytes smaller
- CPU zero-latency optimization
- Performance benefits (dependency breaking)
- Compiler output examples
- All zeroing methods compared

Related Lessons: Topic 1 (Program Structure), Topic 3 (Basic Instructions)

Read When: Learning about program structure and optimization

cmp-vs-test.md

Question: “What is the difference between CMP and TEST?”

Topics Covered:

- CMP performs subtraction (for magnitude comparison)
- TEST performs AND (for bit testing and zero checks)
- When to use each instruction
- Flag behavior differences
- Encoding size comparison
- Common use cases and patterns
- Decision tree for choosing
- Common mistakes to avoid

Related Lessons: Topic 3 (Basic Instructions), Topic 4 (Flags & Comparisons)

Read When: Confused about when to use CMP vs TEST

all-jump-types.md

Question: “What all types of jumps exist in assembly?”

Topics Covered:

- **Unconditional jumps:** JMP (direct, indirect, register, memory)
- **Conditional jumps (signed):** JG, JGE, JL, JLE, JE, JNE
- **Conditional jumps (unsigned):** JA, JAE, JB, JBE, JE, JNE
- **Flag-specific jumps:** JZ/JNZ, JS/JNS, JC/JNC, JO/JNO, JP/JNP
- **Loop instructions:** LOOP, LOOPE/LOOPZ, LOOPNE/LOOPNZ
- **Function calls:** CALL, RET (direct and indirect)
- **Indirect jumps:** Through registers and memory
- **Jump tables:** Efficient switch statement implementation
- **Conditional move:** CMOVcc as branchless alternative
- **Short vs Near vs Far jumps**
- Complete reference table with all 40+ jump variants
- Performance optimization patterns
- Decision tree for choosing the right jump

Related Lessons: Topic 5 (Conditional Jumps), Topic 4 (Flags), Topic 6 (Loops)

Read When: Need comprehensive reference for all jump types

File Statistics

Total Files: 27 markdown files

Total Lines: ~31,000+ lines

Total Size: ~875 KB

Breakdown:

- README.md: 12 KB (curriculum overview - UPDATED)
- INDEX.md: 27 KB (this file - UPDATED)
- QUICK_REFERENCE.md: 12 KB (cheat sheet)
- syscall-reference.md: 35 KB (complete syscall reference)
- Topics: 17,651 lines / ~550 KB (20 complete topics!)
 - topic-01 through topic-12: 12 files (existing)
 - topic-13: Multiplication & Division (20KB)
 - topic-14: Shifts & Rotates (21KB)
 - topic-15: Macros (10KB)
 - topic-16: System Calls (19KB)
 - topic-17: Interfacing with C (18KB)
 - topic-18: SIMD (18KB)
 - topic-19: Performance (18KB)
 - topic-20: Debugging (17KB)
- Q&A: ~210 KB (7 detailed explanations with C equivalents)
 - push-pop-explained.md
 - int-vs-syscall.md
 - memory-addressing.md
 - sections-and-optimization.md
 - cmp-vs-test.md
 - instruction-encoding.md
 - all-jump-types.md (25KB) NEW

ALL 20 TOPICS + 7 Q&A ARTICLES COMPLETED!

Recommended Reading Order

For Complete Beginners:

1. README.md - Understand the learning path
2. topic-01-setup.md - Get environment set up
3. int-vs-syscall.md - Understand system calls
4. topic-02-registers.md - Learn about registers
5. memory-addressing.md - Understand [] syntax
6. sections-and-optimization.md - Program structure
7. topic-03-basic-instructions.md - Core instructions

- 8. [topic-04-flags.md](#) - Flags and comparisons
- 9. [topic-05-jumps.md](#) - Control flow
- 10. [topic-06-loops.md](#) - Loops and iteration
- 11. [topic-07-stack.md](#) - Stack operations
- 12. [push-pop-explained.md](#) - Stack deep dive
- 13. [topic-08-functions.md](#) - Stack frames
- 14. [topic-09-calling-conventions.md](#) - ABI details
- 15. [topic-10-procedures.md](#) - Functions and recursion
- 16. [topic-11-memory-addressing.md](#) - All addressing modes
- 17. [topic-12-arrays-strings.md](#) - Data structures
- 18. [topic-13-multiplication-division.md](#) - Math operations
- 19. [topic-14-shifts-rotates.md](#) - Bit manipulation
- 20. [topic-15-macros.md](#) - Macros and directives
- 21. [topic-16-system-calls.md](#) - OS interface
- 22. [topic-17-interfacing-c.md](#) - C integration
- 23. [topic-18-simd.md](#) - Vectorization
- 24. [topic-19-performance.md](#) - Optimization
- 25. [topic-20-debugging.md](#) - Debugging tools
- 26. [QUICK_REFERENCE.md](#) - Keep open for reference
- 27. [syscall-reference.md](#) - Syscall lookup table

For Quick Reference:

- 1. [QUICK_REFERENCE.md](#) - Common operations and syntax
- 2. [syscall-reference.md](#) - Complete syscall tables
- 3. Topic files - Specific concepts
- 4. Q&A files - Deep explanations

Completion Status

```

LEVEL 1 - Foundation (3/3 topics complete)
LEVEL 2 - Control Flow (3/3 topics complete)
LEVEL 3 - The Stack (2/2 topics complete)
LEVEL 4 - Functions & Procedures (2/2 topics complete)
LEVEL 5 - Memory & Addressing (2/2 topics complete)
LEVEL 6 - Advanced Operations (3/3 topics complete)
LEVEL 7 - System Programming (2/2 topics complete)
LEVEL 8 - Optimization & Advanced (3/3 topics complete)

```

ALL 20 CORE TOPICS: 100% COMPLETE!

SUPPLEMENTARY: 7/7 Q&A articles complete

REFERENCES: 3/3 reference documents complete

Total: 30 comprehensive files covering all aspects of NASM assembly!

Finding What You Need

I want to learn...

- **“How to set up NASM”** → [topic-01-setup.md](#)
- **“About registers”** → [topic-02-registers.md](#)
- **“How push/pop work”** → [push-pop-explained.md](#)
- **“When to use int 0x80 vs syscall”** → [int-vs-syscall.md](#)
- **“What [] brackets mean”** → [memory-addressing.md](#)
- **“About .text .data .bss sections”** → [sections-and-optimization.md](#)
- **“Why use xor instead of mov”** → [sections-and-optimization.md](#)
- **“How instructions are encoded”** → [instruction-encoding.md](#)
- **“Why some instructions are shorter”** → [instruction-encoding.md](#)
- **“Quick syntax reference”** → [QUICK_REFERENCE.md](#)

I need help with...

- **Building programs** → [QUICK_REFERENCE.md](#) (Build Commands)
- **System calls** → [QUICK_REFERENCE.md](#) (Linux Syscalls)
- **Memory addressing** → [memory-addressing.md](#) + [QUICK_REFERENCE.md](#)
- **Debugging** → [topic-01-setup.md](#) + [QUICK_REFERENCE.md](#) (Debugging)
- **Register names** → [topic-02-registers.md](#) + [QUICK_REFERENCE.md](#)

Document Conventions

Status Indicators:

- Completed - Content finished and ready
- In Progress - Content being developed
- ☐ Planned - Content planned but not started

File Naming:

- `topic-##-name.md` - Sequential curriculum topics
- `descriptive-name.md` - Q&A and reference files
- All lowercase, hyphens for spaces

Content Structure:

- Each file starts with clear title and purpose
 - Progressive complexity within each file
 - Code examples for every concept
 -  Practice exercises where applicable
 - Cross-references to related content
-

Updates & Maintenance

Last Updated: December 17, 2025

What's New:

-  Created complete directory structure
 -  Completed Topics 1-3 (Foundation complete!)
 - Added comprehensive Instruction Encoding guide
 - Added 4 comprehensive Q&A documents
 -  Created quick reference guide
 - Topics 4-20 planned
-

`/topics` - OS Internals (Level 9)

Advanced topics covering how operating system primitives work at the assembly level.

topic-21-memory-internals.md **Completed**

Level: OS Internals (Week 15) **What You'll Learn:**

- How `brk()` extends the heap (program break)
- How `mmap()` allocates independent memory regions
- Implementing `sbrk()` in assembly
- `malloc` chunk structure (headers, flags, size encoding)
- Free list strategies (fast bins, small bins, large bins)
- Chunk coalescing to fight fragmentation
- Demand paging and page faults on first access
- Thread-local arenas for multi-threaded allocation
- Memory alignment guarantees
- Debugging: use-after-free detection, double-free detection

Key Concepts: `brk`, `mmap`, `munmap`, `mprotect`, page faults, free lists, chunk headers, COW

topic-22-process-internals.md **Completed**

Level: OS Internals (Week 15-16) **What You'll Learn:**

- Process virtual address space layout
- `fork()` syscall and what the kernel duplicates
- Copy-on-Write (COW) mechanism in detail
- `clone()` — the real syscall behind `fork()` and threads
- `execve()` — how it destroys and rebuilds address space
- Initial stack after `execve` (`argc`, `argv`, `envp`, auxiliary vector)
- Signal installation and delivery mechanism
- Process groups, sessions, and daemons
- Pipes for inter-process communication
- Implementing shell pipelines (`ls | wc -l`) in assembly

Key Concepts: `fork`, `clone`, `execve`, COW, signals, pipes, `dup2`, `wait4`, process groups

topic-23-virtual-memory.md **Completed**

Level: OS Internals (Week 16) **What You'll Learn:**

- Why virtual memory exists (isolation, overcommit, sharing)
- 4-level page table structure (PML4/PDPT/PD/PT)
- Page Table Entry (PTE) bit fields (Present, R/W, U/S, NX, Dirty, Accessed)
- Complete virtual address translation walkthrough

- TLB: sizes, hit rates, flush events, PCID optimization
- Page sizes: 4KB standard, 2MB huge pages, 1GB gigantic pages
- Using huge pages via mmap (MAP_HUGETLB)
- Page fault types and kernel handler logic
- Memory protection with mprotect()
- Guard pages for overflow detection
- ASLR (Address Space Layout Randomization)
- Shared memory between processes
- Memory-mapped files (file as an array)

Key Concepts: page tables, TLB, page faults, huge pages, mprotect, ASLR, shared memory

topic-24-io-internals.md **Completed**

Level: OS Internals (Week 16-17) **What You'll Learn:**

- Complete write() syscall path (user → kernel → VFS → driver → hardware)
- What happens inside the SYSCALL instruction (MSRs, ring transition)
- File descriptor table structure (fd → file struct → operations)
- Terminal I/O path: kernel TTY → line discipline → PTY → terminal emulator
- Canonical vs raw terminal mode (ioctl TCSETS)
- User-space buffering (why printf doesn't write immediately)
- Implementing buffered I/O in assembly
- Implementing printf-like formatting (%d, %s, %x, %c)
- The read() path: keyboard IRQ → input subsystem → TTY → process
- Blocking vs non-blocking I/O (O_NONBLOCK, fcntl)
- poll() for monitoring multiple file descriptors
- Shell redirection (dup2 mechanics)
- ANSI escape sequences for terminal graphics
- Direct framebuffer access (/dev/fb0)
- writev() scatter/gather I/O ▢

Key Concepts: write, read, ioctl, file descriptors, buffering, TTY, poll, dup2, framebuffer

topic-25-elf-format.md **Completed**

Level: OS Internals (Week 17) **What You'll Learn:**

- ELF file structure overview (header, program headers, sections)
- ELF header fields (64 bytes): magic, type, machine, entry point

- Crafting a minimal ELF executable by hand (~170 bytes)
- Object files (.o): sections, symbol table, relocations
- Relocation types (R_X86_64_64, R_X86_64_PC32, R_X86_64_PLT32)
- What the linker does: symbol resolution, section merging, address assignment
- Linker scripts and custom memory layouts
- Segments vs sections (runtime loading vs link-time organization)
- Program header types (PT_LOAD, PT_DYNAMIC, PT_INTERP)
- Dynamic linking: GOT, PLT, lazy binding, dl_runtime_resolve
- Position-Independent Code (PIC/PIE) for ASLR
- How the kernel loads an ELF during execve()
- Symbol tables and visibility (global, local, weak)
- DWARF debug information
- ELF inspection tools (readelf, objdump, nm, strip)

Key Concepts: ELF, sections, segments, relocations, GOT, PLT, PIE, dynamic linking

topic-26-interrupts.md **Completed**

Level: OS Internals (Week 17-18) **What You'll Learn:**

- Interrupt classification: exceptions, hardware IRQs, software interrupts
- Exception subtypes: faults (retried), traps (next instr), aborts
- IDT structure: 256 entries × 16 bytes, gate descriptors
- Interrupt gate vs trap gate
- Complete exception table (vectors 0-31)
- Page fault (#PF) step-by-step: what CPU pushes, error code bits, CR2
- General Protection Fault (#GP) causes
- Hardware interrupt delivery: device → I/O APIC → Local APIC → CPU
- Timer interrupt and how preemptive scheduling works
- Debugger breakpoints: INT 3 (0xCC) and how GDB uses them
- Hardware debug registers (DR0-DR7) for watchpoints
- SYSCALL instruction vs INT 0x80: MSR-based fast path
- Why RCX and R11 are clobbered by syscall
- vDSO: kernel-mapped shared library for fast “syscalls” without ring transition
- Interrupt latency and real-time considerations
- Catching SIGSEGV and SIGFPE: recovery in signal handlers

Key Concepts: IDT, exceptions, IRQs, page fault, timer, INT 3, SYSCALL, vDSO

topic-27-context-switching.md **Completed**

Level: OS Internals (Week 18) **What You'll Learn:**

- Complete process context: all state that defines execution
- Voluntary switch (blocking on I/O) vs involuntary (timer preemption)
- The `switch_to` assembly: push callee-saved, swap RSP, pop, RET
- What's on each process's kernel stack when switched out
- FPU/SSE/AVX state: lazy vs eager switching, XSAVE/XRSTOR
- Thread switching vs process switching (no CR3 change = much faster)
- Thread Local Storage (TLS) via FS segment register
- CFS scheduler: vruntime, red-black tree, weight-based time slices
- Process states (RUNNING, SLEEPING, ZOMBIE, STOPPED)
- Measuring context switch cost (pipe ping-pong benchmark)
- CPU affinity: `sched_setaffinity`, pinning to cores
- NUMA awareness
- User-space context switching: implementing coroutines in assembly

Key Concepts: `switch_to`, kernel stack, CR3, XSAVE, CFS, coroutines, TLS, CPU affinity

topic-28-synchronization.md **Completed**

Level: OS Internals (Week 18) **What You'll Learn:**

- x86-64 memory model (Total Store Order)
- Memory barriers: MFENCE, SFENCE, LFENCE
- LOCK prefix: makes read-modify-write atomic + full barrier
- Atomic operations: LOCK INC, LOCK ADD, XCHG, LOCK XADD
- CMPXCHG (Compare-and-Swap): the fundamental CAS operation
- CMPXCHG16B for 128-bit atomic operations
- Spinlock implementations: test-and-set, ticket lock (fair)
- PAUSE instruction: why it matters in spin loops
- Futex syscall: FUTEX_WAIT, FUTEX_WAKE
- Implementing a full mutex using futex (3-state: unlocked/locked/contented)
- Lock-free Treiber stack with CAS
- ABA problem and solutions (version counter with CMPXCHG16B)
- Exponential backoff for contention
- Read-write locks
- Semaphores with futex
- Condition variables

- Single-producer single-consumer lock-free ring buffer
- Complete thread-safe counter example with clone()

Key Concepts: LOCK, CMPXCHG, spinlock, futex, lock-free, CAS, ring buffer, memory ordering

/qna - Supplementary Deep Dives (OS Internals)

how-malloc-works.md **Completed**

Question: How does malloc() actually work from application to physical memory? **Topics:** chunk headers, free list bins, brk vs mmap decision, why free doesn't return memory, common bugs

how-fork-works.md **Completed**

Question: How does fork() create a process copy without copying all memory? **Topics:** Copy-on-Write mechanism, page table duplication, COW page fault, return value magic, vfork

virtual-memory-explained.md **Completed**

Question: What is a virtual address and how is it translated to physical? **Topics:** 4-level translation, TLB caching, page fault types, /proc/self/maps, isolation

signals-and-traps.md **Completed**

Question: How do signals get delivered? Can I catch SIGSEGV? **Topics:** signal vs exception vs interrupt, kernel stack modification, async-signal-safety, SIGSEGV recovery

How to Use This Collection

1. **Linear Learning:** Follow README.md curriculum sequentially
 2. **Question-Driven:** Use INDEX.md to find specific answers
 3. **Reference:** Keep QUICK_REFERENCE.md open while coding
 4. **Deep Dives:** Read Q&A files for thorough explanations
 5. **Practice:** Complete exercises in each topic file
 6. **OS Internals:** Topics 21-28 explain how the OS works under the hood at assembly level
-

Printable Checklist

Foundation (Weeks 1-2):

- [] Topic 1: Setup & First Program
- [] Topic 2: Registers & Data Types
- [] Topic 3: Basic Instructions

Control Flow (Weeks 3-4):

- [] Topic 4: Flags & Comparisons
- [] Topic 5: Conditional Jumps
- [] Topic 6: Loops

The Stack (Week 5):

- [] Topic 7: Stack Operations
- [] Topic 8: Stack Frames

Functions (Weeks 6-7):

- [] Topic 9: Calling Conventions
- [] Topic 10: Procedures

Memory (Week 8):

- [] Topic 11: Memory Addressing Modes
- [] Topic 12: Arrays & Strings

Advanced Ops (Weeks 9-10):

- [] Topic 13: Multiplication & Division
- [] Topic 14: Shifts & Rotates
- [] Topic 15: Macros & Directives

System Programming (Weeks 11-12):

- [] Topic 16: Linux System Calls
- [] Topic 17: Interfacing with C

Optimization (Weeks 13-14):

- [] Topic 18: SIMD Instructions
- [] Topic 19: Performance & Optimization
- [] Topic 20: Debugging & Tools

OS Internals (Weeks 15-18):

- [] Topic 21: Memory Allocation Internals
- [] Topic 22: Process Internals
- [] Topic 23: Virtual Memory & Paging
- [] Topic 24: I/O Internals
- [] Topic 25: ELF Binary Format
- [] Topic 26: Interrupts & Exceptions
- [] Topic 27: Context Switching & Scheduling
- [] Topic 28: Synchronization Primitives

Supplementary:

- [] Instruction Encoding (Advanced)
- [] 12 Q&A Deep-Dive Articles
- [] Comprehensive Syscall Reference

Ready to learn? Start with [README.md](#)!

[← Back to Main](#)



CHAPTER 3

NASM Quick Reference Guide



A fast reference for common NASM operations and patterns.

Build Commands

```
# 64-bit Assembly
nasm -f elf64 program.asm -o program.o
ld -o program program.o
./program

# 32-bit Assembly
nasm -f elf32 program.asm -o program.o
ld -m elf_i386 -o program program.o
./program

# With Debug Info
nasm -f elf64 -g -F dwarf program.asm
ld -o program program.o
gdb ./program

# Check exit code
echo $?
```

Program Template

```
; program.asm - Description here

section .data
    ; Initialized data goes here
    message db "Hello", 10, 0
    number  dd 42

section .bss
    ; Uninitialized data goes here
    buffer resb 64
    counter resd 1

section .text
    global _start

_start:
    ; Your code here

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Common Operations

Zero a Register

```
xor rax, rax    ; Fastest, smallest (preferred)
xor eax, eax    ; Even smaller, zeros full RAX
```

Move Data

```
mov rax, 42     ; Immediate to register
mov rax, rbx    ; Register to register
mov rax, [rbx]  ; Memory to register
mov [rbx], rax  ; Register to memory
```

Arithmetic

```
add rax, 10      ; RAX = RAX + 10
sub rax, 5       ; RAX = RAX - 5
inc rax         ; RAX = RAX + 1
dec rax         ; RAX = RAX - 1
neg rax         ; RAX = -RAX
```

Bitwise

```
and rax, rbx     ; RAX = RAX & RBX
or  rax, rbx     ; RAX = RAX | RBX
xor rax, rbx     ; RAX = RAX ^ RBX
not rax          ; RAX = ~RAX
shl rax, 2       ; RAX = RAX << 2 (multiply by 4)
shr rax, 2       ; RAX = RAX >> 2 (divide by 4)
```

Control Flow

Unconditional Jump

```
jmp label        ; Always jump
```

Conditional Jumps (Signed)

```
cmp rax, rbx     ; Compare RAX with RBX
je label         ; Jump if equal
jne label        ; Jump if not equal
jg label         ; Jump if greater (RAX > RBX)
jge label        ; Jump if greater or equal
jl label         ; Jump if less (RAX < RBX)
jle label        ; Jump if less or equal
```

Conditional Jumps (Unsigned)

```
ja label         ; Jump if above (unsigned >)
jae label        ; Jump if above or equal
jb label         ; Jump if below (unsigned <)
jbe label        ; Jump if below or equal
```

Test and Jump

```
test rax, rax      ; AND without storing result
jz label           ; Jump if zero
jnz label          ; Jump if not zero
```

Loops

```
mov rcx, 10        ; Loop counter
loop_start:
    ; ... code ...
loop loop_start    ; Decrements RCX, jumps if not zero
```

Stack Operations

```
push rax           ; Save RAX on stack
pop rbx            ; Restore into RBX

; Function prologue
push rbp
mov rbp, rsp
sub rsp, 16        ; Allocate 16 bytes for locals

; Function epilogue
leave              ; Equivalent to: mov rsp, rbp; pop rbp
ret               ; Return from function
```

Linux Syscalls (64-bit)

Register Convention

```
RAX = syscall number (input) / return value (output)
RDI = arg1
RSI = arg2
RDX = arg3
R10 = arg4 (NOT RCX!)
R8  = arg5
R9  = arg6
```

Note: RCX and R11 are destroyed by syscall

Common Syscalls

```

; sys_read (0) - Read from file descriptor
mov rax, 0          ; sys_read
mov rdi, 0          ; stdin
mov rsi, buffer     ; buffer pointer
mov rdx, 64         ; max bytes to read
syscall
; RAX = bytes read (or negative on error)

; sys_write (1) - Write to file descriptor
mov rax, 1          ; sys_write
mov rdi, 1          ; stdout
mov rsi, buffer     ; message pointer
mov rdx, length     ; message length
syscall
; RAX = bytes written (or negative on error)

; sys_open (2) - Open file
mov rax, 2          ; sys_open
mov rdi, filename   ; path to file
mov rsi, 0          ; flags (0=O_RDONLY, 1=O_WRONLY, 2=O_RDWR)
mov rdx, 0644o      ; mode (permissions)
syscall
; RAX = file descriptor (or negative on error)

; sys_close (3) - Close file descriptor
mov rax, 3          ; sys_close
mov rdi, fd         ; file descriptor
syscall
; RAX = 0 on success, negative on error

; sys_exit (60) - Exit program
mov rax, 60         ; sys_exit
xor rdi, rdi        ; exit code (0 = success)
syscall
; Does not return

```

Syscall Quick Numbers

0	read	9	mmap	57	fork
1	write	11	munmap	59	execve
2	open	12	brk	60	exit
3	close	35	nanosleep	61	wait4
4	stat	79	getcwd	80	chdir
5	fstat	83	mkdir	87	unlink

File Descriptors:

- 0 = stdin
- 1 = stdout
- 2 = stderr

For complete reference see: [Linux Syscall Reference](#)

Data Types

```
section .data
    ; Define with initial values
    byte_val    db 0x42           ; 1 byte
    word_val    dw 0x1234         ; 2 bytes
    dword_val   dd 0x12345678     ; 4 bytes
    qword_val   dq 0x123456789ABC ; 8 bytes

    string_val  db "Hello", 0     ; String with null
    array_val   dd 1, 2, 3, 4, 5  ; Array of 5 ints

    buffer      times 64 db 0     ; 64 zero bytes

section .bss
    ; Reserve without initial values
    buffer2     resb 256          ; Reserve 256 bytes
    counter     resd 1            ; Reserve 1 dword
    pointer     resq 1            ; Reserve 1 qword
```

Register Reference

64-bit Registers

RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP
R8, R9, R10, R11, R12, R13, R14, R15

32-bit (lower 32 bits, zeros upper)

EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
R8D, R9D, R10D, R11D, R12D, R13D, R14D, R15D

16-bit (lower 16 bits)

```
AX, BX, CX, DX, SI, DI, BP, SP  
R8W, R9W, R10W, R11W, R12W, R13W, R14W, R15W
```

8-bit (lowest byte)

```
AL, BL, CL, DL, SIL, DIL, BPL, SPL  
R8B, R9B, R10B, R11B, R12B, R13B, R14B, R15B
```

8-bit (high byte, legacy)

```
AH, BH, CH, DH
```

Register Roles

```
RAX - Accumulator, return values  
RBX - Base pointer (callee-saved)  
RCX - Counter for loops  
RDX - Data, I/O operations  
RSI - Source index  
RDI - Destination index  
RBP - Base pointer (stack frame)  
RSP - Stack pointer (don't modify directly!)
```

Memory Addressing

```

; Direct
mov rax, [0x1000]          ; Read from address 0x1000

; Register indirect
mov rax, [rbx]             ; Read from address in RBX

; Register + displacement
mov rax, [rbx + 8]         ; Read from RBX + 8

; Base + index
mov rax, [rbx + rcx]       ; Read from RBX + RCX

; Base + index*scale + displacement
mov rax, [rbx + rcx*4 + 8] ; Read from RBX + (RCX*4) + 8

; Label
mov rax, [myvar]           ; Read from address of myvar

```

Size specifiers:

```

mov byte [rbx], 42        ; 1 byte
mov word [rbx], 42        ; 2 bytes
mov dword [rbx], 42       ; 4 bytes
mov qword [rbx], 42       ; 8 bytes

```

String Operations

```

; Set direction flag
cld          ; Forward (increment)
std          ; Backward (decrement)

; String instructions (work with RSI/RDI)
movsb        ; Move byte [RSI] to [RDI], inc both
lodsb        ; Load byte [RSI] to AL, inc RSI
stosb        ; Store AL to [RDI], inc RDI
cmpsb        ; Compare [RSI] with [RDI], inc both
scasb        ; Compare AL with [RDI], inc RDI

; With REP prefix
rep movsb    ; Repeat RCX times
repe cmpsb   ; Repeat while equal
repne scasb  ; Repeat while not equal

```

Arithmetic (Advanced)

Multiplication

```
; Unsigned multiply
mov rax, 10
mov rbx, 20
mul rbx           ; RDX:RAX = RAX * RBX

; Signed multiply
imul rbx          ; RDX:RAX = RAX * RBX (signed)
imul rax, rbx     ; RAX = RAX * RBX (truncated)
imul rax, rbx, 10 ; RAX = RBX * 10
```

Division

```
; Unsigned divide
mov rax, 100
xor rdx, rdx      ; Clear RDX (high part)
mov rbx, 10
div rbx           ; RAX = quotient, RDX = remainder

; Signed divide
mov rax, 100
cqo              ; Sign-extend RAX into RDX
mov rbx, 10
idiv rbx         ; RAX = quotient, RDX = remainder
```

Function Calling (System V AMD64)

Call Function

```
; Arguments in: RDI, RSI, RDX, RCX, R8, R9, then stack
mov rdi, arg1
mov rsi, arg2
mov rdx, arg3
call my_function
; Return value in RAX
```

Define Function

```
my_function:
    push rbp
    mov rbp, rsp
    sub rsp, 16      ; Allocate locals

    ; Function body
    ; Access args: [rbp+16], [rbp+24], etc.
    ; Access locals: [rbp-4], [rbp-8], etc.

    mov rax, result  ; Return value

    leave            ; Restore stack
    ret
```

Debugging

```
# Compile with debug info
nasm -f elf64 -g -F dwarf program.asm
ld -o program program.o

# Run in GDB
gdb ./program

# GDB commands
(gdb) break _start      # Set breakpoint
(gdb) run               # Run program
(gdb) stepi             # Step one instruction
(gdb) info registers    # Show all registers
(gdb) x/10x $rsp        # Examine stack (10 hex values)
(gdb) disassemble       # Show assembly
(gdb) quit              # Exit GDB
```

Useful Tools

```
# Disassemble
objdump -d program -M intel

# View sections
objdump -h program

# Size of sections
size program

# Trace system calls
strace ./program

# Check for memory errors
valgrind ./program

# View hex dump
hexdump -C program

# File info
file program
```

Tips & Tricks

Always Zero with XOR

```
xor eax, eax      ; 2 bytes (preferred)
mov  eax, 0       ; 5 bytes (slower, bigger)
```

Test for Zero

```
test rax, rax      ; Sets ZF if RAX is zero
jz  is_zero        ; Jump if zero
```

Swap Registers

```
xchg rax, rbx      ; Swap RAX and RBX (one instruction)
```

Load Effective Address (LEA)

```
lea rax, [rbx + rcx*4 + 8] ; RAX = RBX + RCX*4 + 8 (no memory access)
```

Sign Extend

```
movsx rax, al ; Sign-extend AL to RAX
movzx rax, al ; Zero-extend AL to RAX
```

Quick Bit Flags

```
CF - Carry Flag (unsigned overflow)
ZF - Zero Flag (result is zero)
SF - Sign Flag (result is negative)
OF - Overflow Flag (signed overflow)
PF - Parity Flag (even number of 1 bits)
```

Common Patterns

Array Iteration

```
xor rcx, rcx ; index = 0
loop_start:
  mov al, [array + rcx]
  ; process AL
  inc rcx
  cmp rcx, array_len
  jl loop_start
```

String Length

```
xor rax, rax ; length = 0
strlen_loop:
  cmp byte [rsi + rax], 0
  je strlen_done
  inc rax
  jmp strlen_loop
strlen_done:
  ; RAX contains length
```

If-Else

```
cmp rax, 10
jl else_branch
    ; if RAX >= 10
    ; ...
    jmp end_if
else_branch:
    ; else
    ; ...
end_if:
```

Resources

- **NASM Manual:** <https://www.nasm.us/docs.php>
- **Intel Manuals:** <https://software.intel.com/content/www/us/en/develop/articles/intel-sdm.html>
- **Linux Syscall Table:** <https://filippo.io/linux-syscall-table/>
- **x86-64 ABI:** https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf

[← Back to Main](#)

CHAPTER 4

Linux x86-64 Syscall Reference

Complete reference for Linux system calls in 64-bit assembly.

Quick Reference: Syscall Registers

Register Usage (System V AMD64 ABI)

Register	Purpose
RAX	Syscall number (input) Return value (output)
RDI	1st argument
RSI	2nd argument
RDX	3rd argument
R10	4th argument (not RCX!)
R8	5th argument
R9	6th argument
RCX, R11	Destroyed by syscall

Note: R10 is used instead of RCX because syscall uses RCX internally!

Basic Syscall Template

```
mov rax, <syscall_number>    ; Syscall number
mov rdi, <arg1>               ; 1st argument
mov rsi, <arg2>               ; 2nd argument
mov rdx, <arg3>               ; 3rd argument
; mov r10, <arg4>             ; 4th argument (if needed)
; mov r8, <arg5>               ; 5th argument (if needed)
; mov r9, <arg6>               ; 6th argument (if needed)
syscall                       ; Make the call

; RAX now contains return value (or negative error code)
```

Detailed Syscall Argument Table

File I/O Syscalls

Number	Syscall	RAX	RDI (arg1)	RSI (arg2)	RDX (arg3)
R10 (arg4)	R8 (arg5)				
0	read	0	int fd	void *buf	size_t count
-	-				
1	write	1	int fd	void *buf	size_t count
-	-				
2	open	2	char *pathname	int flags	mode_t mode
-	-				
3	close	3	int fd	-	-
-	-				
4	stat	4	char *pathname	struct stat *buf	-
-	-				
5	fstat	5	int fd	struct stat *buf	-
-	-				
8	lseek	8	int fd	off_t offset	int whence
-	-				
16	ioctl	16	int fd	unsigned long req	unsigned long arg
-	-				
17	pread64	17	int fd	void *buf	size_t count
off_t offset	-				
18	pwrite64	18	int fd	void *buf	size_t count
off_t offset	-				
32	dup	32	int oldfd	-	-
-	-				
33	dup2	33	int oldfd	int newfd	-
-	-				
72	fcntl	72	int fd	int cmd	unsigned long arg
-	-				
73	flock	73	int fd	int operation	-
-	-				
74	fsync	74	int fd	-	-
-	-				
76	truncate	76	char *path	off_t length	-
-	-				
77	ftruncate	77	int fd	off_t length	-
-	-				
85	creat	85	char *pathname	mode_t mode	-
-	-				
86	link	86	char *oldpath	char *newpath	-
-	-				
87	unlink	87	char *pathname	-	-
-	-				
88	symlink	88	char *target	char *linkpath	-
-	-				
89	readlink	89	char *pathname	char *buf	size_t bufsiz
-	-				
90	chmod	90	char *pathname	mode_t mode	-
-	-				

92	chown	92		char *pathname	uid_t owner	gid_t group
-	-	-				
95	umask	95		mode_t mask	-	-
-	-	-				

Process Management Syscalls

Number	Syscall	RAX		RDI (arg1)	RSI (arg2)	RDX (arg3)
R10 (arg4)	R8 (arg5)					
57	fork	57		-	-	-
-	-	-				
58	vfork	58		-	-	-
-	-	-				
59	execve	59		char *pathname	char **argv	char **envp
-	-	-				
60	exit	60		int status	-	-
-	-	-				
61	wait4	61		pid_t pid	int *status	int options
rusage *ru	-	-				
62	kill	62		pid_t pid	int sig	-
-	-	-				
102	getuid	102		-	-	-
-	-	-				
104	getgid	104		-	-	-
-	-	-				
105	setuid	105		uid_t uid	-	-
-	-	-				
106	setgid	106		gid_t gid	-	-
-	-	-				
107	geteuid	107		-	-	-
-	-	-				
108	getegid	108		-	-	-
-	-	-				
110	getppid	110		-	-	-
-	-	-				
111	getpgrp	111		-	-	-
-	-	-				
186	gettid	186		-	-	-
-	-	-				
231	exit_group	231		int status	-	-
-	-	-				

Memory Management Syscalls

Number	Syscall	RAX	RDI (arg1)	RSI (arg2)	RDX (arg3)
R10 (arg4)	R8 (arg5)				
9	mmap	9	void *addr	size_t length	int prot
int flags	int fd				
10	mprotect	10	void *addr	size_t len	int prot
-	-				
11	munmap	11	void *addr	size_t length	-
-	-				
12	brk	12	void *addr	-	-
-	-				
25	mremap	25	void *old_addr	size_t old_size	size_t new_size
int flags	void *new_addr				
26	msync	26	void *addr	size_t length	int flags
-	-				
28	madvise	28	void *addr	size_t length	int advice
-	-				

Note: mmap has a 6th argument - R9 (arg6) = off_t offset

Time and Timer Syscalls

Number	Syscall	RAX	RDI (arg1)	RSI (arg2)	RDX (arg3)
R10 (arg4)	R8 (arg5)				
35	nanosleep	35	timespec *req	timespec *rem	-
-	-				
201	time	201	time_t *tloc	-	-
-	-				
228	clock_gettime	228	clockid_t clk_id	timespec *tp	-
-	-				
229	clock_settime	229	clockid_t clk_id	timespec *tp	-
-	-				
230	clock_getres	230	clockid_t clk_id	timespec *res	-
-	-				

Directory and Path Syscalls

Number	Syscall	RAX	RDI (arg1)	RSI (arg2)	RDX (arg3)
R10 (arg4)	R8 (arg5)				
79	getcwd	79	char *buf	size_t size	-
-	-				
80	chdir	80	char *path	-	-
-	-				
81	fchdir	81	int fd	-	-
-	-				
82	rename	82	char *oldpath	char *newpath	-
-	-				
83	mkdir	83	char *pathname	mode_t mode	-
-	-				
84	rmdir	84	char *pathname	-	-
-	-				
161	chroot	161	char *path	-	-
-	-				

Socket/Network Syscalls

Number (arg3)	Syscall	RAX R10 (arg4)	RDI (arg1) R8 (arg5)	RSI (arg2)	RDX
41	socket	41	int domain	int type	int
protocol	-	-			
42	connect	42	int sockfd	sockaddr *addr	socklen_t
addrlen	-	-			
43	accept	43	int sockfd	sockaddr *addr	socklen_t
*addrlen	-	-			
44	sendto	44	int sockfd	void *buf	size_t
len	int flags	sockaddr *dst			
45	recvfrom	45	int sockfd	void *buf	size_t
len	int flags	sockaddr *src			
46	sendmsg	46	int sockfd	msghdr *msg	int
flags	-	-			
47	recvmsg	47	int sockfd	msghdr *msg	int
flags	-	-			
48	shutdown	48	int sockfd	int how	
-	-	-	-		
49	bind	49	int sockfd	sockaddr *addr	socklen_t
addrlen	-	-			
50	listen	50	int sockfd	int backlog	
-	-	-	-		
54	setsockopt	54	int sockfd	int level	int
optname	void *optval	socklen_t len			
55	getsockopt	55	int sockfd	int level	int
optname	void *optval	socklen_t *len			

Note: sendto/recvfrom have 6th argument - R9 (arg6) = socklen_t addrlen (for sendto)

Common File I/O Syscalls

read (0) - Read from file descriptor

```

; ssize_t read(int fd, void *buf, size_t count)
mov rax, 0           ; syscall number: read
mov rdi, <fd>        ; file descriptor (0 = stdin)
mov rsi, <buffer>     ; pointer to buffer
mov rdx, <count>      ; number of bytes to read
syscall
; RAX = number of bytes read, or -1 on error

```

Example:

```

section .bss
    buffer resb 64

section .text
    mov rax, 0          ; sys_read
    mov rdi, 0          ; stdin
    mov rsi, buffer     ; buffer address
    mov rdx, 64         ; read up to 64 bytes
    syscall
    ; RAX contains bytes read

```

write (1) - Write to file descriptor

```

; ssize_t write(int fd, const void *buf, size_t count)
mov rax, 1          ; syscall number: write
mov rdi, <fd>       ; file descriptor (1 = stdout, 2 = stderr)
mov rsi, <buffer>    ; pointer to data
mov rdx, <count>     ; number of bytes to write
syscall
; RAX = number of bytes written, or -1 on error

```

Example:

```

section .data
    msg db "Hello, World!", 10

section .text
    mov rax, 1          ; sys_write
    mov rdi, 1          ; stdout
    mov rsi, msg        ; message address
    mov rdx, 14         ; message length
    syscall
    ; RAX contains bytes written

```


open (2) - Open file

```
; int open(const char *pathname, int flags, mode_t mode)
mov rax, 2           ; syscall number: open
mov rdi, <filename>  ; pointer to filename string
mov rsi, <flags>      ; flags (O_RDONLY=0, O_WRONLY=1, O_RDWR=2, etc.)
mov rdx, <mode>       ; permissions (e.g., 0644)
syscall
; RAX = file descriptor, or -1 on error
```

Common flags:

- O_RDONLY = 0
- O_WRONLY = 1
- O_RDWR = 2
- O_CREAT = 64 (0x40)
- O_TRUNC = 512 (0x200)
- O_APPEND = 1024 (0x400)

Example:

```
section .data
    filename db "output.txt", 0

section .text
    mov rax, 2           ; sys_open
    mov rdi, filename    ; filename
    mov rsi, 577         ; O_CREAT | O_WRONLY | O_TRUNC (64+1+512)
    mov rdx, 0644o       ; permissions: rw-r--r--
    syscall
    ; RAX = file descriptor
```

close (3) - Close file descriptor

```
; int close(int fd)
mov rax, 3           ; syscall number: close
mov rdi, <fd>        ; file descriptor to close
syscall
; RAX = 0 on success, -1 on error
```

Example:

```

mov rax, 3          ; sys_close
mov rdi, [file_fd]  ; file descriptor
syscall

```

Process Management Syscalls

exit (60) - Terminate process

```

; void exit(int status)
mov rax, 60          ; syscall number: exit
mov rdi, <exit_code> ; exit status (0 = success)
syscall
; Does not return!

```

Example:

```

mov rax, 60          ; sys_exit
xor rdi, rdi         ; exit code 0 (success)
syscall

```

fork (57) - Create child process

```

; pid_t fork(void)
mov rax, 57          ; syscall number: fork
syscall
; RAX = 0 in child, child PID in parent, -1 on error

```

Example:

```

mov rax, 57          ; sys_fork
syscall
test rax, rax
jz child_process     ; RAX = 0 in child
; Parent process (RAX = child PID)
jmp parent_process
child_process:
; Child code here

```

execve (59) - Execute program

```

; int execve(const char *pathname, char *const argv[], char *const envp[])
mov rax, 59          ; syscall number: execve
mov rdi, <pathname>   ; program path
mov rsi, <argv>        ; argument array
mov rdx, <envp>        ; environment array
syscall
; Does not return on success, returns -1 on error

```

wait4 (61) - Wait for process

```

; pid_t wait4(pid_t pid, int *status, int options, struct rusage *rusage)
mov rax, 61          ; syscall number: wait4
mov rdi, <pid>        ; PID to wait for (-1 = any child)
mov rsi, <status_ptr> ; pointer to status variable
mov rdx, <options>     ; options (0 = block until exit)
mov r10, <rusage_ptr>  ; resource usage (NULL = don't care)
syscall
; RAX = PID of exited child, -1 on error

```

Memory Management Syscalls

brk (12) - Change data segment size

```

; int brk(void *addr)
mov rax, 12          ; syscall number: brk
mov rdi, <new_break> ; new program break address (0 = query)
syscall
; RAX = new program break address

```

mmap (9) - Map memory

```
; void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)
mov rax, 9           ; syscall number: mmap
mov rdi, <addr>      ; address hint (0 = kernel chooses)
mov rsi, <length>    ; length in bytes
mov rdx, <prot>      ; protection (PROT_READ=1, PROT_WRITE=2, PROT_EXEC=4)
mov r10, <flags>     ; flags (MAP_PRIVATE=2, MAP_ANONYMOUS=32)
mov r8, <fd>         ; file descriptor (-1 for anonymous)
mov r9, <offset>     ; offset in file
syscall
; RAX = address of mapped region, or -1 on error
```

Example: Allocate anonymous memory

```
mov rax, 9           ; sys_mmap
xor rdi, rdi        ; addr = 0 (kernel chooses)
mov rsi, 4096       ; length = 4096 bytes (1 page)
mov rdx, 3          ; prot = PROT_READ | PROT_WRITE
mov r10, 34         ; flags = MAP_PRIVATE | MAP_ANONYMOUS
mov r8, -1          ; fd = -1 (no file)
xor r9, r9          ; offset = 0
syscall
; RAX = address of allocated memory
```

munmap (11) - Unmap memory

```
; int munmap(void *addr, size_t length)
mov rax, 11         ; syscall number: munmap
mov rdi, <addr>     ; address of mapped region
mov rsi, <length>   ; length in bytes
syscall
; RAX = 0 on success, -1 on error
```

Time and Sleep Syscalls

nanosleep (35) - Sleep for specified time

```
; int nanosleep(const struct timespec *req, struct timespec *rem)
mov rax, 35           ; syscall number: nanosleep
mov rdi, <timespec_ptr> ; pointer to timespec structure
mov rsi, <remaining_ptr> ; pointer to remaining time (or NULL)
syscall
; RAX = 0 on success, -1 on error
```

Timespec structure:

```
section .data
    timespec:
        dq 1           ; seconds
        dq 500000000   ; nanoseconds (0.5 seconds)
```

Example: Sleep for 1 second

```
section .data
    sleep_time:
        dq 1           ; 1 second
        dq 0           ; 0 nanoseconds

section .text
    mov rax, 35         ; sys_nanosleep
    mov rdi, sleep_time ; timespec pointer
    xor rsi, rsi        ; don't care about remaining time
    syscall
```

time (201) - Get current time

```
; time_t time(time_t *tloc)
mov rax, 201          ; syscall number: time
mov rdi, <time_ptr>    ; pointer to store time (or NULL)
syscall
; RAX = seconds since epoch (Jan 1, 1970)
```

Directory and File System Syscalls

getcwd (79) - Get current working directory

```
; char *getcwd(char *buf, size_t size)  
mov rax, 79           ; syscall number: getcwd  
mov rdi, <buffer>     ; buffer to store path  
mov rsi, <size>        ; size of buffer  
syscall  
; RAX = pointer to buffer on success, -1 on error
```

chdir (80) - Change directory

```
; int chdir(const char *path)  
mov rax, 80           ; syscall number: chdir  
mov rdi, <path>        ; path to new directory  
syscall  
; RAX = 0 on success, -1 on error
```

mkdir (83) - Create directory

```
; int mkdir(const char *pathname, mode_t mode)  
mov rax, 83           ; syscall number: mkdir  
mov rdi, <pathname>    ; directory path  
mov rsi, <mode>         ; permissions (e.g., 0755)  
syscall  
; RAX = 0 on success, -1 on error
```

rmdir (84) - Remove directory

```
; int rmdir(const char *pathname)  
mov rax, 84           ; syscall number: rmdir  
mov rdi, <pathname>    ; directory path  
syscall  
; RAX = 0 on success, -1 on error
```

unlink (87) - Delete file

```
; int unlink(const char *pathname)  
mov rax, 87           ; syscall number: unlink  
mov rdi, <pathname>   ; file path  
syscall  
; RAX = 0 on success, -1 on error
```

Complete Syscall Number Reference

File Operations

Number	Syscall
0	read - Read from file descriptor
1	write - Write to file descriptor
2	open - Open file
3	close - Close file descriptor
4	stat - Get file status
5	fstat - Get file status by fd
6	lstat - Get file status (no follow symlink)
8	lseek - Reposition file offset
16	ioctl - Control device
17	pread64 - Read from fd at offset
18	pwrite64 - Write to fd at offset
19	readv - Read into multiple buffers
20	writev - Write from multiple buffers
32	dup - Duplicate file descriptor
33	dup2 - Duplicate fd to specific number
72	fcntl - File control
73	flock - File locking
74	fsync - Synchronize file state
75	fdatasync - Sync file data
76	truncate - Truncate file to length
77	ftruncate - Truncate file by fd
78	getdents - Get directory entries
85	creat - Create file
86	link - Make hard link
87	unlink - Delete file
88	symlink - Make symbolic link
89	readlink - Read symbolic link
90	chmod - Change file permissions
91	fchmod - Change file permissions by fd
92	chown - Change file owner
93	fchown - Change file owner by fd
94	lchown - Change file owner (no follow)
95	umask - Set file creation mask

Process Control

Number	Syscall
57	fork - Create child process
58	vfork - Create child (share memory)
59	execve - Execute program
60	exit - Terminate process
61	wait4 - Wait for process
62	kill - Send signal
96	getpriority - Get scheduling priority
97	setpriority - Set scheduling priority
102	getuid - Get user ID
104	getgid - Get group ID
105	setuid - Set user ID
106	setgid - Set group ID
107	geteuid - Get effective user ID
108	getegid - Get effective group ID
110	getppid - Get parent process ID
111	getpgrp - Get process group
186	gettid - Get thread ID
231	exit_group - Exit all threads

Memory Management

Number	Syscall
9	mmap - Map memory
10	mprotect - Set memory protection
11	munmap - Unmap memory
12	brk - Change data segment size
25	mremap - Remap memory
26	msync - Synchronize memory with storage
27	mincore - Determine if pages are in memory
28	madvise - Give advice about memory usage
29	shmget - Get shared memory segment
30	shmat - Attach shared memory
31	shmctl - Shared memory control

Signals

Number	Syscall
13	rt_sigaction - Set signal action
14	rt_sigprocmask - Set signal mask
15	rt_sigreturn - Return from signal
34	pause - Wait for signal
62	kill - Send signal to process
127	rt_sigpending - Get pending signals
128	rt_sigtimedwait - Wait for signal with timeout
129	rt_sigqueueinfo - Queue signal
130	rt_sigsuspend - Wait for signal

Time and Timer

Number	Syscall
35	nanosleep - High-resolution sleep
96	gettimeofday - Get time (deprecated)
163	settimeofday - Set time (deprecated)
201	time - Get current time
228	clock_gettime - Get time from clock
229	clock_settime - Set time for clock
230	clock_getres - Get clock resolution
222	timer_create - Create timer
223	timer_settime - Set timer
224	timer_gettime - Get timer
226	timer_delete - Delete timer

Directories

Number	Syscall
79	getcwd - Get current directory
80	chdir - Change directory
81	fchdir - Change directory by fd
82	rename - Rename file
83	mkdir - Create directory
84	rmdir - Remove directory
161	chroot - Change root directory

Networking (Socket)

Number	Syscall
41	socket - Create socket
42	connect - Connect socket
43	accept - Accept connection
44	sendto - Send message
45	recvfrom - Receive message
46	sendmsg - Send message (advanced)
47	recvmsg - Receive message (advanced)
48	shutdown - Shut down socket
49	bind - Bind socket to address
50	listen - Listen for connections
51	getsockname - Get socket name
52	getpeername - Get peer name
53	socketpair - Create socket pair
54	setsockopt - Set socket options
55	getsockopt - Get socket options

Error Handling

Syscalls return:

- **Positive value or zero:** Success (meaning depends on syscall)
- **Negative value:** Error (negate to get errno)

Common Error Codes

Code	Name	Description
-1	EPERM	Operation not permitted
-2	ENOENT	No such file or directory
-3	ESRCH	No such process
-4	EINTR	Interrupted system call
-5	EIO	I/O error
-9	EBADF	Bad file descriptor
-11	EAGAIN	Try again
-12	ENOMEM	Out of memory
-13	EACCES	Permission denied
-14	EFAULT	Bad address
-17	EEXIST	File exists
-20	ENOTDIR	Not a directory
-21	EISDIR	Is a directory
-22	EINVAL	Invalid argument
-24	EMFILE	Too many open files
-28	ENOSPC	No space left on device
-30	EROFS	Read-only file system
-32	EPIPE	Broken pipe

Checking for Errors

```

mov rax, 2          ; sys_open
mov rdi, filename
mov rsi, 0          ; O_RDONLY
syscall

; Check for error
test rax, rax
js error_handler    ; Jump if negative (sign flag set)

; Success - RAX contains file descriptor
mov [file_fd], rax
jmp continue

error_handler:
; RAX contains negative error code
neg rax             ; Convert to positive errno
; Handle error...

continue:

```

Practical Examples

Example 1: Print String to stdout

```
section .data
    message db "Hello, World!", 10
    msg_len equ $ - message

section .text
    global _start

_start:
    mov rax, 1          ; sys_write
    mov rdi, 1          ; stdout
    mov rsi, message    ; buffer
    mov rdx, msg_len     ; length
    syscall

    mov rax, 60         ; sys_exit
    xor rdi, rdi        ; exit code 0
    syscall
```

Example 2: Read from stdin

```
section .bss
    buffer resb 100

section .text
    mov rax, 0          ; sys_read
    mov rdi, 0          ; stdin
    mov rsi, buffer     ; buffer
    mov rdx, 100        ; max bytes
    syscall
    ; RAX = bytes read
```

Example 3: Open, Write, Close File

```

section .data
    filename db "output.txt", 0
    content db "File content here", 10
    content_len equ $ - content

section .bss
    fd resq 1

section .text
    ; Open file
    mov rax, 2          ; sys_open
    mov rdi, filename
    mov rsi, 577        ; O_CREAT | O_WRONLY | O_TRUNC
    mov rdx, 0644o      ; permissions
    syscall
    mov [fd], rax       ; Save file descriptor

    ; Write to file
    mov rax, 1          ; sys_write
    mov rdi, [fd]       ; file descriptor
    mov rsi, content
    mov rdx, content_len
    syscall

    ; Close file
    mov rax, 3          ; sys_close
    mov rdi, [fd]
    syscall

```

Example 4: Allocate Memory with mmap

```

; Allocate 4KB of memory
mov rax, 9          ; sys_mmap
xor rdi, rdi        ; addr = NULL (kernel chooses)
mov rsi, 4096       ; length = 4096
mov rdx, 3          ; prot = READ | WRITE
mov r10, 34         ; flags = PRIVATE | ANONYMOUS
mov r8, -1          ; fd = -1
xor r9, r9          ; offset = 0
syscall
; RAX = address of allocated memory

```

Quick Tips

1. **Always check return values** - Negative = error
 2. **Use R10 for 4th arg** - Not RCX (syscall uses it internally)
 3. **RCX and R11 are destroyed** - Save them if needed
 4. **Null-terminate strings** - For path arguments
 5. **Use octal for permissions** - 0644o, 0755o, etc.
-

Resources

- **Linux Syscall Table:** <https://filippo.io/linux-syscall-table/>
 - **Man Pages:** `man 2 <syscall_name>` (e.g., `man 2 write`)
 - **Header Files:** `/usr/include/asm/unistd_64.h`
 - **Kernel Source:** `arch/x86/entry/syscalls/syscall_64.tbl`
-

Related Topics

- [Topic 1: Setup & First Program](#)
 - [int 0x80 vs syscall](#)
-

[← Back to Main](#)

CHAPTER 5

All Types of Jumps in Assembly

Overview

Jumps are the fundamental control flow instructions in assembly. They change the instruction pointer (RIP/EIP) to alter program execution flow. This guide covers **every type of jump** available in x86-64 assembly.

```
// C equivalent concepts:
if (x > 5) { }           // Conditional jump
goto label;              // Unconditional jump
for (;;) { }             // Loop with jump
switch (x) { }           // Jump table
function();              // Function call (special jump)
```

1. Unconditional Jumps

JMP - Jump Always

Transfers control unconditionally to the target address.

```
; C equivalent:
; goto target;

jmp target                ; Always jump to target

target:
    ; execution continues here
```

Forms:


```

; Direct jump (relative)
jmp label           ; Most common - PC-relative

; Indirect jump (register)
jmp rax             ; Jump to address in RAX

; Indirect jump (memory)
jmp [table + rbx*8] ; Jump to address stored in memory

; Far jump (segment:offset)
jmp 0x08:offset     ; Change code segment (rarely used in 64-bit)

```

Example: Skip code section

```

section .text
; C equivalent:
; if (condition) {
;     goto skip_dangerous;
; }
; dangerous_code();
; skip_dangerous:
; safe_code();

cmp rax, 0
je skip_dangerous ; Conditional jump

; Dangerous code here
call dangerous_function

skip_dangerous:
; Safe code continues
mov rbx, 42

```

2. Conditional Jumps (Signed Comparisons)

Based on **signed** integer comparison results.

After `CMP a, b` (signed):

Instruction	Condition	Flags	Meaning	C Equivalent
<code>JE</code> / <code>JZ</code>	Equal / Zero	ZF=1	a == b	<code>if (a == b)</code>
<code>JNE</code> / <code>JNZ</code>	Not Equal / Not Zero	ZF=0	a != b	<code>if (a != b)</code>
<code>JG</code> / <code>JNLE</code>	Greater / Not Less or Equal	ZF=0 AND SF=OF	a > b	<code>if (a > b)</code>
<code>JGE</code> / <code>JNL</code>	Greater or Equal / Not Less	SF=OF	a >= b	<code>if (a >= b)</code>
<code>JL</code> / <code>JNGE</code>	Less / Not Greater or Equal	SF≠OF	a < b	<code>if (a < b)</code>
<code>JLE</code> / <code>JNG</code>	Less or Equal / Not Greater	ZF=1 OR SF≠OF	a <= b	<code>if (a <= b)</code>

Example:

```

section .data
    x dd -10
    y dd 5

section .text
    ; C equivalent:
    ; int x = -10, y = 5;
    ; if (x < y) {
    ;     // x is less than y (signed)
    ; }

    mov eax, [x]      ; EAX = -10 (signed)
    cmp eax, [y]      ; Compare -10 vs 5
    jl x_is_less      ; Jump if less (signed)

    ; x >= y path
    jmp done

x_is_less:
    ; x < y path (this executes)

done:

```

3. Conditional Jumps (Unsigned Comparisons)

Based on **unsigned** integer comparison results.

After `CMP a, b` (unsigned):

Instruction	Condition	Flags	Meaning	C Equivalent
<code>JE</code> / <code>JZ</code>	Equal / Zero	ZF=1	a == b	<code>if (a == b)</code>
<code>JNE</code> / <code>JNZ</code>	Not Equal / Not Zero	ZF=0	a != b	<code>if (a != b)</code>
<code>JA</code> / <code>JNBE</code>	Above / Not Below or Equal	CF=0 AND ZF=0	a > b	<code>if ((unsigned)a > b)</code>
<code>JAE</code> / <code>JNB</code> / <code>JNC</code>	Above or Equal / Not Below / No Carry	CF=0	a >= b	<code>if ((unsigned)a >= b)</code>
<code>JB</code> / <code>JNAE</code> / <code>JC</code>	Below / Not Above or Equal / Carry	CF=1	a < b	<code>if ((unsigned)a < b)</code>
<code>JBE</code> / <code>JNA</code>	Below or Equal / Not Above	CF=1 OR ZF=1	a <= b	<code>if ((unsigned)a <= b)</code>

Example:

```

section .data
    x dd 0xFFFFFFFF    ; Large unsigned value
    y dd 5

section .text
    ; C equivalent:
    ; unsigned int x = 0xFFFFFFFF; // 4,294,967,280
    ; unsigned int y = 5;
    ; if (x > y) {
    ;     // x is greater (unsigned)
    ; }

    mov eax, [x]        ; EAX = 0xFFFFFFFF (unsigned: 4,294,967,280)
    cmp eax, [y]        ; Compare unsigned
    ja x_is_greater     ; Jump if above (unsigned)

    jmp done

x_is_greater:
    ; x > y path (this executes for unsigned)

done:

```

Key Difference:

```

; Same values, different interpretation:
mov eax, 0xFFFFFFFF0

; Signed: -16
cmp eax, 5
jg is_greater_signed    ; Does NOT jump (-16 < 5)

; Unsigned: 4,294,967,280
cmp eax, 5
ja is_greater_unsigned  ; DOES jump (4,294,967,280 > 5)

```

4. Conditional Jumps (Single Flag Tests)

Test individual flags directly.

Flag-Specific Jumps:

Instruction	Condition	Flag	Meaning	C Equivalent
JZ	Zero	ZF=1	Result is zero	<code>if (x == 0)</code>
JNZ	Not Zero	ZF=0	Result is not zero	<code>if (x != 0)</code>
JS	Sign	SF=1	Result is negative	<code>if (x < 0)</code>
JNS	Not Sign	SF=0	Result is non-negative	<code>if (x >= 0)</code>
JO	Overflow	OF=1	Overflow occurred	(detect overflow)
JNO	Not Overflow	OF=0	No overflow	(no overflow)
JP / JPE	Parity / Parity Even	PF=1	Even parity	(even bits set)
JNP / JPO	Not Parity / Parity Odd	PF=0	Odd parity	(odd bits set)
JC	Carry	CF=1	Carry flag set	(unsigned overflow)
JNC	Not Carry	CF=0	Carry flag clear	(no carry)

Example: Zero Check

```

; C equivalent:
; if (x == 0) {
;     // handle zero
; }

test eax, eax      ; Set flags based on EAX
jz is_zero         ; Jump if zero flag set

; Non-zero path
jmp done

is_zero:
    ; Zero path

done:

```

Example: Negative Check

```

; C equivalent:
; if (x < 0) {
;     // handle negative
; }

test eax, eax      ; Set flags based on EAX
js is_negative     ; Jump if sign flag set

; Non-negative path
jmp done

is_negative:
    ; Negative path

done:

```

Example: Overflow Detection

```

; C equivalent:
; int result = a + b;
; if (/* overflow occurred */) {
;     // handle overflow
; }

mov eax, [a]
add eax, [b]      ; Addition sets OF if overflow
jo overflow_handler ; Jump if overflow

; Normal path
jmp done

overflow_handler:
    ; Handle overflow

done:

```

5. Loop Jumps

Special jump instructions for loop control.

Loop Instructions:

Instruction	Operation	C Equivalent
LOOP	--RCX; if (RCX != 0) jump	for (int i = n; i > 0; --i)
LOOPE / LOOPZ	--RCX; if (RCX != 0 && ZF=1) jump	Loop while equal/zero
LOOPNE / LOOPNZ	--RCX; if (RCX != 0 && ZF=0) jump	Loop while not equal/zero

Example: Simple Loop

```

; C equivalent:
; for (int i = 10; i > 0; --i) {
;     // loop body
; }

mov rcx, 10          ; Loop counter

loop_start:
    ; Loop body here

    loop loop_start ; Decrement RCX, jump if RCX != 0

; Continue after loop

```

Example: LOOPE (Loop While Equal)

```

; C equivalent:
; int i = 10;
; while (i > 0 && condition_is_true) {
;     --i;
; }

mov rcx, 10

loop_start:
    ; Check some condition
    cmp byte [flag], 1 ; Sets ZF if flag == 1
    loope loop_start   ; Continue if ZF=1 and RCX != 0

; Exit when RCX=0 or condition becomes false

```

6. Function Call/Return (Special Jumps)

CALL - Call Subroutine

Jumps to a function and saves return address on stack.

```

; C equivalent:
; function();

call function          ; Push return address, jump to function

function:
    ; Function body
    ret                ; Pop return address, jump back

```

How CALL Works:

```
; call target is equivalent to:
push rip + 5      ; Save address of next instruction (size of call = 5 bytes)
jmp target       ; Jump to function
```

RET - Return from Subroutine

Pops return address from stack and jumps to it.

```
; ret is equivalent to:
pop rip          ; Pop return address into instruction pointer
```

Forms:

```
; Direct call
call function_name

; Indirect call (register)
call rax          ; Jump to address in RAX

; Indirect call (memory)
call [function_ptr] ; Jump to address stored in memory

; Return
ret              ; Simple return

; Return and pop N bytes
ret 16           ; Pop return address, then remove 16 bytes from stack
```

Example: Function with Arguments


```

; C equivalent:
; int add(int a, int b) {
;     return a + b;
; }
; int result = add(10, 20);

section .text
global _start

_start:
    ; Call function
    mov rdi, 10      ; First argument
    mov rsi, 20      ; Second argument
    call add         ; CALL pushes return address, jumps to 'add'

    ; RAX now contains result (30)
    mov [result], rax

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

add:
    ; Function prologue
    push rbp
    mov rbp, rsp

    ; Function body
    mov rax, rdi      ; RAX = first arg
    add rax, rsi       ; RAX += second arg

    ; Function epilogue
    pop rbp
    ret               ; RET pops return address, jumps back

```

7. Indirect Jumps

Jump to an address stored in a register or memory.

Register Indirect:

```

; C equivalent:
; void (*func_ptr)(void) = some_function;
; func_ptr(); // Call through pointer

lea rax, [target]    ; Get address of target
jmp rax              ; Jump to address in RAX

target:
    ; Code here

```

Memory Indirect:

```

; C equivalent:
; void (*func_ptr)(void) = function_array[index];
; func_ptr();

section .data
    function_ptr dq target

section .text
    jmp [function_ptr] ; Jump to address stored at function_ptr

target:
    ; Code here

```

8. Jump Tables (Switch Statement)

Efficient implementation of multi-way branches.

```

; C equivalent:
; switch (x) {
;     case 0: action0(); break;
;     case 1: action1(); break;
;     case 2: action2(); break;
;     case 3: action3(); break;
;     default: default_action(); break;
; }

section .data
    jump_table dq case0, case1, case2, case3

section .text
    ; Bounds check
    mov rax, [x]
    cmp rax, 4
    jae default_case    ; Jump if x >= 4 (out of range)

    ; Jump through table
    jmp [jump_table + rax*8]    ; Each pointer is 8 bytes

case0:
    ; Handle case 0
    jmp done

case1:
    ; Handle case 1
    jmp done

case2:
    ; Handle case 2
    jmp done

case3:
    ; Handle case 3
    jmp done

default_case:
    ; Handle default

done:

```

Complete Example:

```

section .data
    msg0 db "Zero", 10, 0
    msg1 db "One", 10, 0
    msg2 db "Two", 10, 0
    msg3 db "Three", 10, 0
    msg_default db "Other", 10, 0

    jump_table dq case0, case1, case2, case3

section .text
extern printf
global main

main:
    push rbp
    mov rbp, rsp

    ; Get input value
    mov rax, 2          ; Test with value 2

    ; Bounds check
    cmp rax, 4
    jae default_case

    ; Indirect jump through table
    lea rbx, [jump_table]
    jmp [rbx + rax*8]

case0:
    lea rdi, [msg0]
    jmp print_and_exit

case1:
    lea rdi, [msg1]
    jmp print_and_exit

case2:
    lea rdi, [msg2]
    jmp print_and_exit

case3:
    lea rdi, [msg3]
    jmp print_and_exit

default_case:
    lea rdi, [msg_default]

print_and_exit:
    xor rax, rax
    call printf

```

```

xor eax, eax
leave
ret

```

9. Conditional Move (Alternative to Jumps)

CMOVcc instructions provide branchless conditional execution.

Common CMOV Instructions:

Instruction	Condition	When to Use
CMOVE	Equal (ZF=1)	<code>result = (a == b) ? x : result;</code>
CMOVNE	Not Equal (ZF=0)	<code>result = (a != b) ? x : result;</code>
CMOVG	Greater (signed)	<code>result = (a > b) ? x : result;</code>
CMOVL	Less (signed)	<code>result = (a < b) ? x : result;</code>
CMOVA	Above (unsigned)	<code>result = (a > b) ? x : result;</code>
CMOVNB	Below (unsigned)	<code>result = (a < b) ? x : result;</code>

Example: Branchless Max

```

; C equivalent:
; int max = (a > b) ? a : b;

; With conditional jump (slower if mispredicted):
mov eax, [a]
cmp eax, [b]
jg a_is_max
mov eax, [b]
a_is_max:
    ; EAX contains max

; With CMOV (faster, no branch):
mov eax, [a]
mov ebx, [b]
cmp eax, ebx
cmovl eax, ebx    ; If a < b, move b to a
                ; EAX now contains max

```

10. Short vs Near vs Far Jumps

Jump Distances:

Type	Range	Encoding	Use Case
Short	-128 to +127 bytes	2 bytes	Small loops, nearby labels
Near	±2GB (32-bit offset)	5-6 bytes	Same segment, most common
Far	Different segment	7+ bytes	Rarely used in 64-bit

NASM Automatically Chooses:

```
jmp short target      ; Force short jump (2 bytes)
jmp near target       ; Force near jump (5 bytes)
jmp target            ; NASM chooses optimal size
```

Example:

```
loop_start:
    ; Loop body (small)
    loop loop_start    ; Short jump (2 bytes)

very_far_label:
    ; Lots of code here (> 127 bytes)
    jmp very_far_label ; Near jump required (5 bytes)
```

11. Jump Optimization Patterns

Pattern 1: Remove Unnecessary Jumps

```
; BAD: Unnecessary jump
cmp eax, 0
je case_zero
jmp case_nonzero

case_zero:
    ; handle zero

case_nonzero:
    ; handle non-zero

; GOOD: Fall through
cmp eax, 0
jne case_nonzero

; handle zero (fall through)
jmp done

case_nonzero:
    ; handle non-zero

done:
```

Pattern 2: Invert Condition to Remove Jump

```
; BAD:
cmp eax, 0
je skip
; common path
skip:
    ; rare path

; GOOD:
cmp eax, 0
jne common_path
; rare path (fall through)
jmp done

common_path:
    ; common path

done:
```

Pattern 3: Use CMOV for Simple Conditionals

```
; BAD: Branch for simple selection
cmp eax, ebx
jle use_eax
mov ecx, ebx
jmp done
use_eax:
    mov ecx, eax
done:

; GOOD: Branchless with CMOV
mov ecx, eax
cmp eax, ebx
cmovg ecx, ebx      ; If eax > ebx, use ebx
```

12. Complete Jump Reference Table

All Jump Instructions Alphabetically:

Instruction	Alternative Names	Condition	Flags	Type
<code>JA</code>	<code>JNBE</code>	Above (unsigned)	CF=0 & ZF=0	Conditional
<code>JAE</code>	<code>JNB</code> , <code>JNC</code>	Above or Equal	CF=0	Conditional
<code>JB</code>	<code>JNAE</code> , <code>JC</code>	Below (unsigned)	CF=1	Conditional
<code>JBE</code>	<code>JNA</code>	Below or Equal	CF=1 OR ZF=1	Conditional
<code>JC</code>		Carry	CF=1	Flag test
<code>JCXZ</code>		CX is zero	CX=0	Special
<code>JECXZ</code>		ECX is zero	ECX=0	Special
<code>JRCXZ</code>		RCX is zero	RCX=0	Special
<code>JE</code>	<code>JZ</code>	Equal / Zero	ZF=1	Conditional
<code>JG</code>	<code>JNLE</code>	Greater (signed)	ZF=0 & SF=OF	Conditional
<code>JGE</code>	<code>JNL</code>	Greater or Equal	SF=OF	Conditional
<code>JL</code>	<code>JNGE</code>	Less (signed)	SF≠OF	Conditional
<code>JLE</code>	<code>JNG</code>	Less or Equal	ZF=1 OR SF≠OF	Conditional
<code>JMP</code>		Always	None	Unconditional
<code>JNA</code>	<code>JBE</code>	Not Above	CF=1 OR ZF=1	Conditional
<code>JNAE</code>	<code>JB</code> , <code>JC</code>	Not Above or Equal	CF=1	Conditional
<code>JNB</code>	<code>JAE</code> , <code>JNC</code>	Not Below	CF=0	Conditional
<code>JNBE</code>	<code>JA</code>	Not Below or Equal	CF=0 & ZF=0	Conditional
<code>JNC</code>	<code>JAE</code> , <code>JNB</code>	No Carry	CF=0	Flag test
<code>JNE</code>	<code>JNZ</code>	Not Equal / Not Zero	ZF=0	Conditional
<code>JNG</code>	<code>JLE</code>	Not Greater	ZF=1 OR SF≠OF	Conditional
<code>JNGE</code>	<code>JL</code>	Not Greater or Equal	SF≠OF	Conditional
<code>JNL</code>	<code>JGE</code>	Not Less	SF=OF	Conditional
<code>JNLE</code>	<code>JG</code>	Not Less or Equal	ZF=0 & SF=OF	Conditional
<code>JNO</code>		No Overflow	OF=0	Flag test

Instruction	Alternative Names	Condition	Flags	Type
<code>JNP</code>	<code>JPO</code>	Not Parity / Parity Odd	PF=0	Flag test
<code>JNS</code>		Not Sign	SF=0	Flag test
<code>JNZ</code>	<code>JNE</code>	Not Zero / Not Equal	ZF=0	Conditional
<code>JO</code>		Overflow	OF=1	Flag test
<code>JP</code>	<code>JPE</code>	Parity / Parity Even	PF=1	Flag test
<code>JPE</code>	<code>JP</code>	Parity Even	PF=1	Flag test
<code>JPO</code>	<code>JNP</code>	Parity Odd	PF=0	Flag test
<code>JS</code>		Sign	SF=1	Flag test
<code>JZ</code>	<code>JE</code>	Zero / Equal	ZF=1	Conditional
<code>LOOP</code>		Loop	RCX--, RCX≠0	Loop
<code>LOOPE</code>	<code>LOOPZ</code>	Loop Equal	RCX--, RCX≠0 & ZF=1	Loop
<code>LOOPNE</code>	<code>LOOPNZ</code>	Loop Not Equal	RCX--, RCX≠0 & ZF=0	Loop
<code>LOOPNZ</code>	<code>LOOPNE</code>	Loop Not Zero	RCX--, RCX≠0 & ZF=0	Loop
<code>LOOPZ</code>	<code>LOOPE</code>	Loop Zero	RCX--, RCX≠0 & ZF=1	Loop

Summary

Jump Categories:

1. **Unconditional:** `JMP` (always jumps)
2. **Signed Conditional:** `JG`, `JGE`, `JL`, `JLE`, `JE`, `JNE`
3. **Unsigned Conditional:** `JA`, `JAE`, `JB`, `JBE`, `JE`, `JNE`
4. **Flag Tests:** `JZ`, `JNZ`, `JS`, `JNS`, `JO`, `JNO`, `JP`, `JNP`, `JC`, `JNC`
5. **Loop:** `LOOP`, `LOOPE`, `LOOPNE`
6. **Function:** `CALL`, `RET`
7. **Indirect:** Through register or memory
8. **Conditional Move:** `CMOVcc` (branchless alternative)

Quick Decision Tree:

```

Need to jump?
├─ Always? → JMP
├─ Based on comparison?
│   ├─ Signed integers? → JG, JGE, JL, JLE, JE, JNE
│   └─ Unsigned integers? → JA, JAE, JB, JBE, JE, JNE
├─ Based on single flag?
│   ├─ Zero? → JZ, JNZ
│   ├─ Sign? → JS, JNS
│   ├─ Carry? → JC, JNC
│   └─ Overflow? → JO, JNO
├─ Loop counter? → LOOP, LOOPE, LOOPNE
├─ Call function? → CALL (with RET to return)
├─ Computed target? → JMP [register/memory]
└─ Want branchless? → CMOV (conditional move)
  
```

Performance Tips:

1. **Avoid unpredictable branches** - use CMOV when possible
2. **Put common case first** - less misprediction
3. **Minimize jump distance** - short jumps are faster
4. **Use jump tables** for multi-way branches (switch)
5. **Eliminate unnecessary jumps** - let code fall through

Practice Exercises

Exercise 1: Implement Absolute Value

```

; int abs(int x) {
;     return (x < 0) ? -x : x;
; }

; Your code here using conditional jumps
  
```

Solution

```
abs_value:
    ; Input: EDI = x
    ; Output: EAX = |x|

    mov eax, edi
    test eax, eax
    jns positive      ; Jump if not sign (x >= 0)
    neg eax           ; x = -x
positive:
    ret
```

Exercise 2: Implement Clamp

```
; int clamp(int x, int min, int max) {
;     if (x < min) return min;
;     if (x > max) return max;
;     return x;
; }

; Your code here
```

Solution

```
clamp:
    ; RDI = x, RSI = min, RDX = max
    mov rax, rdi
    cmp rax, rsi
    cmovl rax, rsi      ; If x < min, x = min
    cmp rax, rdx
    cmovg rax, rdx      ; If x > max, x = max
    ret
```

Related Topics:

- [Topic 5: Conditional Jumps](#)
- [Topic 4: Flags & Comparisons](#)
- [Topic 6: Loops](#)
- [CMP vs TEST](#)

CHAPTER 6

Instruction Size (Bytes) vs Execution Time (Cycles)

The Question

“Why is every instruction written with 2 bytes, 5 bytes, etc.? Aren’t we supposed to see the cycles and not bytes?”

Short Answer: Both matter! **Bytes** measure how much memory/space an instruction occupies, while **cycles** measure how long it takes to execute. They’re different measurements for different purposes.

Understanding the Difference

Instruction Size (Bytes) - Space in Memory

```
// C equivalent concept:
sizeof(instruction) // How much memory does it occupy?
```

What it measures: The physical size of the machine code instruction in memory.

Why it matters:

1. **Code size** - smaller programs fit in cache better
2. **Instruction fetch** - CPU must fetch these bytes from memory
3. **Cache efficiency** - more instructions fit in I-cache
4. **Jump distances** - affects whether you can use short jumps

Example:

```
; These do the same thing but have different sizes:

mov rax, 0           ; 7 bytes: REX.W + opcode + immediate (8 bytes)
xor rax, rax         ; 3 bytes: REX.W + opcode + ModRM

; Both set RAX to zero, but XOR is 4 bytes smaller!
```

Hexdump showing actual bytes:

```
48 c7 c0 00 00 00 00    ; mov rax, 0 (7 bytes)
48 31 c0                ; xor rax, rax (3 bytes)
```

Execution Time (Cycles) - How Long It Takes

```
// C equivalent concept:
time_to_execute(instruction) // How many CPU cycles?
```

What it measures: The number of CPU clock cycles required to execute the instruction.

Why it matters:

1. **Performance** - faster code = fewer cycles
2. **Latency** - when is the result available?
3. **Throughput** - how many per second can execute?
4. **Optimization** - choosing faster instructions

Example:

```
; Same result, but different execution times:

mov rax, 0          ; ~1 cycle latency
xor rax, rax        ; ~0 cycles latency (dependency breaking!)

div rbx             ; ~40-90 cycles (very slow!)
shr rax, 1          ; ~1 cycle (fast alternative for divide by 2)
```

Both Measurements Are Important!

Real-World Example: Jump Instructions

Let's look at why we care about BOTH for jumps:

```
section .text
loop_start:
    ; Loop body (10 bytes of instructions)
    inc rax
    cmp rax, 100

    ; Which jump should we use?
```

Option 1: Short Jump (2 bytes)

```
jnl loop_start      ; 2 bytes: EB F6
```

Bytes: 2 bytes (opcode + 8-bit signed offset) **Cycles:** 1-2 cycles if predicted correctly, ~15-20 if mispredicted **Range:** -128 to +127 bytes

Option 2: Near Jump (5-6 bytes)

```
jnl loop_start      ; 5-6 bytes: 0F 8C [4-byte offset]
```

Bytes: 5-6 bytes (prefix + opcode + 32-bit offset) **Cycles:** Same 1-2 cycles if predicted, ~15-20 if mispredicted **Range:** $\pm 2\text{GB}$

Why BOTH Matter:

```
; If loop_start is nearby (< 128 bytes away):
jnl loop_start      ; NASM generates: 2 bytes
                    ; Faster to fetch from memory
                    ; More code fits in cache
                    ; Same execution time

; If loop_start is far away (> 127 bytes):
jnl loop_start      ; NASM generates: 5-6 bytes
                    ; Required for large distance
                    ; Same execution time
                    ; Takes more space
```

Complete Comparison Table

Aspect	Instruction Size (Bytes)	Execution Time (Cycles)
Measures	Memory/space occupied	Time to execute
Unit	Bytes	CPU clock cycles
Affects	Code size, cache efficiency	Performance, speed
Visible in	Disassembly, hexdump	Profiler, benchmarks
Matters for	Memory, I-cache	Execution speed
Fixed?	Yes (per encoding)	No (varies by CPU)

Why Bytes Matter: Code Density

Example: Loop with Different Jump Sizes

```
; Short jumps (2 bytes each) - GOOD
section .text
    xor rax, rax
.loop:
    inc rax
    cmp rax, 10
    jl .loop      ; 2 bytes (loop is nearby)
    ret

; Total: ~15 bytes
; Fits entirely in ONE cache line (64 bytes)
; Fast instruction fetch!
```

```
; Long jumps (5-6 bytes each) - WORSE
section .text
    xor rax, rax
    jmp start      ; Force long jumps
    times 200 nop  ; Padding to force distance
start:
.loop:
    inc rax
    cmp rax, 10
    jl .loop      ; 5-6 bytes (forced long jump)
    ret

; Total: ~220 bytes
; Spans MULTIPLE cache lines
; Slower instruction fetch
; Same execution time for the jump itself!
```

Result:

- Short version: Fetches 1 cache line → faster overall
- Long version: Fetches 4 cache lines → slower overall
- Individual jump cycle count: SAME
- Total program performance: DIFFERENT (due to I-cache)

Why Cycles Matter: Execution Speed

Example: Division vs Shift

```
; C equivalent:
; unsigned int result = x / 4;

; Option 1: Division instruction
mov eax, [x]
xor edx, edx
mov ecx, 4
div ecx          ; SIZE: 2 bytes, TIME: ~40-90 cycles (SLOW!)
mov [result], eax

; Option 2: Shift instruction
mov eax, [x]
shr eax, 2       ; SIZE: 3 bytes, TIME: ~1 cycle (FAST!)
mov [result], eax
```

Comparison:

Instruction	Size (Bytes)	Cycles	Speed Difference
<code>div ecx</code>	2	~40-90	Baseline
<code>shr eax, 2</code>	3	~1	40-90x faster!

Key Point: The shift is **1 byte larger** but **40-90x faster!** Here, cycles matter WAY more than bytes.

When Each Measurement Matters Most

Bytes Matter Most When:

1. **Embedded systems** - limited ROM/Flash
2. **Cache optimization** - fitting hot loops in L-cache
3. **Code golf** - making smallest possible program
4. **Firmware** - space-constrained environments

```
; Embedded: Every byte counts!
xor rax, rax      ; 3 bytes (GOOD)
; vs
mov rax, 0        ; 7 bytes (WASTEFUL)
```

Cycles Matter Most When:

1. **Performance optimization** - hot loops
2. **Real-time systems** - predictable timing
3. **High-throughput** - processing large data
4. **Latency-sensitive** - fast response time

```
; Performance: Speed is critical!
shr eax, 2           ; 1 cycle (GOOD)
; vs
div ecx             ; 90 cycles (TERRIBLE)
```

Both Matter Together: The I-Cache Effect

Modern CPUs have separate caches for instructions (I-cache) and data (D-cache).

Instruction Cache (I-Cache)

Typical sizes:

- L1 I-Cache: 32-64 KB
- Cache line: 64 bytes

How instruction size affects performance:

```
; Example: Hot loop (executed millions of times)

; Version 1: Compact (short jumps, small instructions)
section .text
hot_loop:
    mov eax, [rsi]
    add eax, [rdi]
    mov [rdx], eax
    add rsi, 4
    add rdi, 4
    add rdx, 4
    dec rcx
    jnz hot_loop      ; 2 bytes (short jump)
    ret
; Total: ~30 bytes - fits in ONE cache line!
```

```

; Version 2: Bloated (long jumps, inefficient encoding)
section .text
    jmp start
    times 100 nop        ; Force long jumps
start:
hot_loop:
    mov eax, [rsi]
    add eax, [rdi]
    mov [rdx], eax
    add rsi, 4
    add rdi, 4
    add rdx, 4
    dec rcx
    jnz hot_loop        ; 6 bytes (long jump, forced)
    ret
; Total: ~130 bytes - spans THREE cache lines!

```

Performance Impact:

```

Version 1 (30 bytes):
- Cache line fetches per iteration: 0 (already in I-cache)
- I-cache misses: 0
- Execution time: ~10 cycles per iteration

Version 2 (130 bytes):
- Cache line fetches per iteration: 0-3 (depending on alignment)
- I-cache misses: More frequent
- Execution time: ~10-80 cycles per iteration (with cache misses)

```

Conclusion: Even though the instructions execute in the same number of cycles, Version 1 is **much faster overall** because it fits in I-cache!

Practical Guidelines

Optimize for Bytes When:

```

; Hot loop that fits in I-cache
tight_loop:
    ; Use shortest instructions
    xor eax, eax        ; 2-3 bytes
    lea rbx, [rsi+8]    ; 4 bytes (vs mov+add = 7 bytes)
    inc rcx              ; 3 bytes (vs add rcx, 1 = 4 bytes)
    loop tight_loop     ; 2 bytes

```

Optimize for Cycles When:

```
; One-time initialization (not in hot path)
initialization:
    ; Use fastest instructions, size doesn't matter
    xor rax, rax           ; Fast (zero latency)
    xorps xmm0, xmm0      ; Fast (zero latency)

    ; Avoid slow instructions
    ; BAD: div rbx (90 cycles)
    ; GOOD: shr rax, cl (1 cycle)
```

Optimize for Both:

```
; Best: Short AND fast
ideal:
    xor eax, eax           ; 2-3 bytes, 0 cycle latency
    inc ecx                ; 2-3 bytes, 1 cycle latency
    shl rax, 3             ; 3-4 bytes, 1 cycle latency
```

How to Measure Each

Measuring Instruction Size (Bytes)

Method 1: Disassembly with objdump

```
$ objdump -d program | grep -A5 "loop_start"
00401000 <loop_start>:
 401000: 48 31 c0          xor     rax,rax      # 3 bytes
 401003: 48 ff c0          inc     rax          # 3 bytes
 401006: 48 83 f8 0a       cmp     rax,0xa      # 4 bytes
 40100a: 7c f7            jl      401003       # 2 bytes (short)
```

Method 2: Hexdump

```
$ hexdump -C program | head
00000000  48 31 c0 48 ff c0 48 83  f8 0a 7c f7 c3
```

Method 3: NASM listing

```
$ nasm -f elf64 -l listing.txt program.asm
$ cat listing.txt
    1 00000000 4831C0      xor rax, rax      ; 3 bytes
    2 00000003 48FFC0      inc rax          ; 3 bytes
    3 00000006 4883F80A     cmp rax, 10      ; 4 bytes
    4 0000000A 7CF7       jl loop_start    ; 2 bytes
```

Measuring Execution Time (Cycles)

Method 1: RDTSC (CPU timestamp counter)

```
section .text
global benchmark

benchmark:
    ; Start timing
    rdtsc                ; Read timestamp
    shl rdx, 32
    or rax, rdx          ; RDX:RAX = 64-bit cycle count
    mov r12, rax         ; Save start time

    ; CODE TO BENCHMARK
    mov eax, 100
    xor edx, edx
    mov ecx, 4
    div ecx              ; Measure this

    ; End timing
    rdtsc
    shl rdx, 32
    or rax, rdx
    sub rax, r12         ; RAX = cycles elapsed
    ret
```

Method 2: perf (Linux)

```
$ perf stat -e cycles,instructions ./program

Performance counter stats for './program':

    1,234,567      cycles
    456,789       instructions          #    0.37  insn per cycle
```

Method 3: Intel Architecture Code Analyzer (IACA)

```
$ iaca -arch SKL program.o
Throughput Analysis Report
Block Throughput: 2.50 Cycles
```

Real-World Example: Optimizing a Loop

```
// C code to optimize:
for (int i = 0; i < 1000000; ++i) {
    result += array[i];
}
```

Version 1: Naive (Not optimized)

```
xor eax, eax          ; result = 0
xor ecx, ecx          ; i = 0
loop_v1:
    add eax, [array + rcx*4] ; 7 bytes
    inc ecx              ; 3 bytes
    cmp ecx, 1000000      ; 6 bytes
    jl loop_v1           ; 6 bytes (near jump, forced by size)

; Size: ~22 bytes per iteration
; Cycles: ~5 per iteration (with I-cache misses)
; Total: 1M iterations × 5 cycles = 5M cycles
```

Version 2: Optimized for Bytes AND Cycles

```
xor eax, eax          ; result = 0
mov ecx, 1000000      ; i = 1000000 (count down)
lea rsi, [array]      ; Pointer to array
loop_v2:
    add eax, [rsi]      ; 2 bytes (simpler addressing)
    add rsi, 4          ; 3 bytes
    dec ecx             ; 2 bytes
    jnz loop_v2         ; 2 bytes (short jump!)

; Size: ~9 bytes per iteration (fits in I-cache!)
; Cycles: ~2 per iteration (no I-cache misses)
; Total: 1M iterations × 2 cycles = 2M cycles
```

Improvement: 2.5x faster! (Both from bytes AND better instruction choice)

Summary

Key Takeaways:

1. **Bytes = Space** (how big is the instruction in memory)
2. **Cycles = Time** (how long does it take to execute)
3. **Both matter** for different reasons
4. **Small code** → better I-cache efficiency → faster overall
5. **Fast instructions** → fewer cycles → faster execution

When to Focus on Each:

Situation	Focus	Why
Hot loops	Both	Small code fits in I-cache, fast instructions reduce cycles
Cold code	Cycles	Executed rarely, size doesn't matter
Embedded	Bytes	Limited memory, space critical
Performance	Cycles	Speed is critical

The Best Code:

```

; Ideal: Short AND fast!
xor rax, rax      ; 3 bytes, 0 cycle latency
shl rax, 3        ; 4 bytes, 1 cycle latency
; vs
mov rax, 0        ; 7 bytes, 1 cycle latency (wasteful)
; vs
div rbx           ; 2 bytes, 90 cycle latency (slow)

```

Related Topics

- [Topic 19: Performance & Optimization](#) - Detailed cycle analysis
- [Instruction Encoding](#) - How bytes are structured
- [Sections & Optimization](#) - Why XOR is smaller
- [All Jump Types](#) - Jump sizes and distances

Bottom Line: We measure instructions in **bytes** for size/space and **cycles** for speed/time. Both measurements are critical for understanding and optimizing assembly code!

CHAPTER 7

CMP vs TEST - What's the Difference?

Question

What is the difference between CMP and TEST instructions?

Quick Answer

cmp / test set the flags; jcc reads them



test does AND, cmp does SUB — neither writes its operands, they only set flags

ASCII diagram

Instr	Operation	Primary Use
CMP	Subtraction (doesn't store)	Comparing values (equality, greater/less than)
TEST	AND (doesn't store)	Testing bits, checking zero, checking sign

Detailed Explanation

CMP - Compare (Subtraction)

```
cmp destination, source      ; Performs: destination - source
                              ; Sets flags, discards result
```

What it does:

- Subtracts `source` from `destination`
- Sets arithmetic flags based on the result
- **Does NOT store** the result (unlike SUB)

Flags set: CF, ZF, SF, OF, AF, PF (all arithmetic flags)

Example:

```
mov rax, 10
cmp rax, 5      ; Internally: 10 - 5 = 5

; Flags after CMP:
; ZF = 0 (result is not zero)
; SF = 0 (result is positive)
; CF = 0 (no borrow needed)
; OF = 0 (no signed overflow)
```

TEST - Test Bits (AND)

```
test operand1, operand2     ; Performs: operand1 & operand2
                              ; Sets flags, discards result
```

What it does:

- Performs bitwise AND of `operand1` and `operand2`
- Sets logic flags based on the result
- **Does NOT store** the result (unlike AND)

Flags set: ZF, SF, PF (logic flags), **CF and OF** cleared to 0

Example:

```

mov rax, 10          ; RAX = 0b00001010
test rax, rax        ; Internally: 0b00001010 & 0b00001010 = 0b00001010

; Flags after TEST:
; ZF = 0 (result is not zero)
; SF = 0 (bit 63 is 0)
; CF = 0 (always cleared by TEST)
; OF = 0 (always cleared by TEST)

```

Side-by-Side Comparison

The Math

```

mov rax, 10
mov rbx, 5

; CMP does subtraction
cmp rax, rbx          ; Computes: 10 - 5 = 5
                     ; Sets flags based on 5

; TEST does AND
test rax, rbx         ; Computes: 0b1010 & 0b0101 = 0b0000
                     ; Sets flags based on 0

```

Flag Behavior

```

; After: cmp rax, 10
; All arithmetic flags affected:
; - CF can be 0 or 1 (based on borrow)
; - ZF can be 0 or 1 (based on equality)
; - SF can be 0 or 1 (based on sign of result)
; - OF can be 0 or 1 (based on overflow)

; After: test rax, 10
; Logic flags affected, arithmetic cleared:
; - CF always 0 (cleared by TEST)
; - ZF can be 0 or 1 (based on AND result)
; - SF can be 0 or 1 (based on MSB of result)
; - OF always 0 (cleared by TEST)

```

When to Use Each

Use CMP when:

1. Comparing for Equality

```
cmp rax, 42
je equal      ; Jump if RAX == 42
jne not_equal ; Jump if RAX != 42
```

2. Comparing Magnitude (Greater/Less)

```
cmp rax, rbx
jg greater      ; RAX > RBX (signed)
jl less         ; RAX < RBX (signed)
ja above        ; RAX > RBX (unsigned)
jb below        ; RAX < RBX (unsigned)
```

3. Range Checking

```
cmp rax, 10
jl too_small    ; RAX < 10
cmp rax, 100
jg too_large    ; RAX > 100
```

4. Checking Against Non-Zero Values

```
cmp rax, 0      ; Check if zero
je is_zero

cmp rax, -1     ; Check if -1
je is_minus_one
```

Use TEST when:

1. Checking if Zero

```
test rax, rax      ; Better than cmp rax, 0
jz is_zero          ; Jump if RAX == 0
jnz not_zero        ; Jump if RAX != 0
```

Why better than CMP?

- Shorter encoding (2-3 bytes vs 4-7 bytes)
- Clearer intent
- Faster on some CPUs

2. Checking Specific Bits

```
test al, 0x01      ; Check bit 0
jnz bit_is_set

test al, 0x80      ; Check bit 7 (sign bit)
jnz bit7_set

test rax, 0xFF00   ; Check if any bits 8-15 are set
jnz has_high_bits
```

3. Checking Even/Odd

```
test rax, 1        ; Check bit 0
jz is_even         ; Bit 0 = 0, even
jnz is_odd         ; Bit 0 = 1, odd
```

4. Checking Sign (Negative)

```
test rax, rax
js is_negative     ; Jump if sign flag set
jns is_positive    ; Jump if sign flag clear
```

5. Multiple Bit Checks

```
test al, 0b11110000 ; Check if any upper 4 bits set
jnz has_upper_bits

test al, 0b00001111 ; Check if any lower 4 bits set
jnz has_lower_bits
```

Practical Examples

Example 1: Zero Check

```
; * Less efficient with CMP
mov rax, [value]
cmp rax, 0           ; 4-7 bytes encoding
je is_zero

; More efficient with TEST
mov rax, [value]
test rax, rax        ; 2-3 bytes encoding
jz is_zero
```

Example 2: Range Check (Use CMP)

```
; Check if RAX is in range [10, 20]
cmp rax, 10
jle out_of_range    ; RAX < 10
cmp rax, 20
jge out_of_range    ; RAX > 20
; In range!
```

Can't use TEST for this - TEST can't compare magnitudes!

Example 3: Bit Flags (Use TEST)

```
; Check status flags (bit field)
; Bit 0 = ready, Bit 1 = error, Bit 2 = done

test al, 0x01        ; Check ready bit
jz not_ready

test al, 0x02        ; Check error bit
jnz has_error

test al, 0x04        ; Check done bit
jnz is_done
```

Can't use CMP for this - CMP is for comparing values, not testing bits!

Example 4: Null Pointer Check

```
; Check if pointer is NULL

; Method 1: CMP
cmp rax, 0
je null_pointer

; Method 2: TEST (preferred)
test rax, rax
jz null_pointer      ; More idiomatic
```

Example 5: Powers of 2 Check

```
; Check if RAX is a power of 2
; Power of 2 has exactly one bit set
; Algorithm: (x & (x-1)) == 0 for powers of 2

mov rbx, rax
dec rbx          ; RBX = RAX - 1
test rax, rbx     ; RAX & (RAX-1)
jz is_power_of_two ; Zero means power of 2
```

Must use TEST - We're checking a bitwise operation result!

Performance Comparison

Instruction Encoding Size

```
; CMP encodings
cmp rax, 0          ; 7 bytes: 48 83 F8 00
cmp rax, rbx        ; 3 bytes: 48 39 D8
cmp eax, 42         ; 3 bytes: 83 F8 2A

; TEST encodings
test rax, rax        ; 3 bytes: 48 85 C0
test eax, eax        ; 2 bytes: 85 C0
test al, 0x01        ; 2 bytes: A8 01
```

Winner for zero check: TEST (smaller encoding)

CPU Performance

On modern CPUs (Intel/AMD), both are similarly fast:

- **Latency:** 1 cycle (both)
- **Throughput:** 4 per cycle (both)

But TEST has advantages:

- Smaller code = better cache usage
- More readable for bit testing
- Doesn't involve subtraction logic

Common Mistakes

Mistake 1: Using CMP for Bit Testing

```
; WRONG: Trying to test bit 0 with CMP
cmp rax, 1
je bit_is_set           ; This checks if RAX == 1, not if bit 0 is set!

; If RAX = 5 (0b0101), bit 0 is set but RAX != 1
; CMP will say "not equal"!

; CORRECT: Use TEST
test rax, 1
jnz bit_is_set          ; This correctly checks bit 0
```

Mistake 2: Using TEST for Magnitude Comparison

```
; WRONG: Trying to check if RAX > 10
test rax, 10
jg greater              ; This doesn't work! TEST does AND, not subtraction

; CORRECT: Use CMP
cmp rax, 10
jg greater
```


Mistake 3: Forgetting TEST Clears CF/OF

```
; Checking carry flag after TEST
add rax, rbx           ; May set CF
test rax, rax          ; CF cleared to 0!
jc carry_set           ; This will NEVER jump!

; If you need to preserve flags:
add rax, rbx           ; May set CF
jc carry_set           ; Check CF immediately
test rax, rax          ; Now safe to use TEST
```

Equivalent Operations

Both Can Check Zero

```
; These are equivalent for zero check:
cmp rax, 0
je is_zero

test rax, rax
jz is_zero

or rax, rax           ; Also works but less common
jz is_zero
```

Both Can Check Sign

```
; For checking if negative (signed):
cmp rax, 0
jl is_negative        ; Less than zero

test rax, rax
js is_negative        ; Sign flag set (MSB = 1)
```

Decision Tree

```

Need to check a value?
|
├─ Comparing two values?
|   └─ Use CMP
|       └─ Equality: je/jne
|           └─ Greater/Less: jg/jl, ja/jb
|               └─ Range: multiple CMPs
|
├─ Checking if zero?
|   └─ Use TEST (more efficient)
|       └─ test reg, reg; jz/jnz
|
├─ Checking specific bits?
|   └─ Use TEST
|       └─ test reg, mask; jz/jnz
|
├─ Checking sign?
|   └─ Use TEST (clearer intent)
|       └─ test reg, reg; js/jns
|
└─ Even/Odd check?
    └─ Use TEST
        └─ test reg, 1; jz(even)/jnz(odd)
  
```

Memory Reference

CMP vs TEST Quick Reference

Operation	Use CMP	Use TEST
x == y	✓	-
x != y	✓	-
x > y	✓	✓
x < y	✓	✓
x >= y	✓	✓
x <= y	✓	✓
x == 0		✓
x != 0		✓
x < 0 (negative)		✓
x >= 0 (positive/zero)		✓
Test bit N	-	
Even/Odd	-	
Multiple bits set	-	
Power of 2	-	

Legend: Works Better choice - Not suitable

TL;DR

CMP:

- Does **subtraction** (operand1 - operand2)
- Use for **comparing magnitudes** (equal, greater, less)
- Sets all arithmetic flags (CF, ZF, SF, OF, AF, PF)
- Think: "Is A bigger/smaller/equal to B?"

TEST:

- Does **bitwise AND** (operand1 & operand2)
- Use for **testing bits** and **zero checks**
- Sets logic flags (ZF, SF, PF), clears CF and OF
- Shorter encoding for zero checks
- Think: "Are these bits set?"

Golden Rules:

1. Use **CMP** when comparing two different values
 2. Use **TEST** for zero checks (`test reg, reg`)
 3. Use **TEST** for bit testing (`test reg, mask`)
 4. When in doubt for zero: **TEST is better**
-

Related Topics

- [Topic 4: Flags & Comparisons](#)
 - [Topic 3: Basic Instructions](#)
 - [Topic 5: Conditional Jumps](#) (*coming soon*)
-

[← Back to Q&A Index](#)

CHAPTER 8

Q&A: How Does fork() Actually Work?

The Question

When I call `fork()`, the documentation says it “creates a copy of the process.” But copying gigabytes of memory would be insanely slow. How does fork actually work? What’s Copy-on-Write? And why does `fork()` return different values in parent and child?

Quick Answer

`fork()` does NOT copy memory. It duplicates the process descriptor (`task_struct`) and page tables, but marks all writable pages as read-only in BOTH processes. Only when either process tries to WRITE to a page does the kernel actually copy that specific page. This is **Copy-on-Write (COW)** — making fork nearly instant regardless of process size.

The Full Story

Step 1: What Gets Duplicated (Immediately)

Created instantly on `fork()`:

- ✓ `task_struct` (process descriptor) – new PID, new scheduling state
- ✓ Page table entries (PTEs) – pointing to SAME physical pages
- ✓ File descriptor table – new table, same underlying file objects
- ✓ Signal handlers – copied
- ✓ Memory map (VMA list) – new VMAs pointing to same pages
- ✓ Credentials (UID, GID)
- ✓ Kernel stack (small, for kernel-mode execution)

NOT copied:

- ✗ Physical memory pages (shared, COW)
- ✗ Page cache data
- ✗ Open file positions (shared via file object reference)
- ✗ PID (child gets new one)

Step 2: Page Table Manipulation

```
Before fork():
  Parent PTE for page at 0x7fff0000:
    Physical page: 0x1A3000
    Permissions: RW (read-write)

After fork():
  Parent PTE for 0x7fff0000:
    Physical page: 0x1A3000 (SAME!)
    Permissions: R- (read-ONLY now!) ← Changed!

  Child PTE for 0x7fff0000:
    Physical page: 0x1A3000 (SAME!)
    Permissions: R- (read-ONLY now!)

  Physical page 0x1A3000:
    Reference count: 2 (both processes point to it)

Time taken: proportional to number of PTEs, NOT amount of data
  - 1GB process with 256K pages: copy ~256K × 8 bytes = 2MB of PTEs
  - NOT copying 1GB of actual data!
```

Step 3: Copy-on-Write in Action

```
Parent writes to 0x7fff0000:
  1. CPU tries to write → PTE says read-only → PAGE FAULT
  2. Kernel page fault handler checks:
    - Is this a COW page? (VMA says writable, but PTE says read-only = COW)
    - YES!
  3. Kernel action:
    a. Allocate new physical page (0x2B4000)
    b. Copy content from old page (0x1A3000) to new page (0x2B4000)
    c. Update PARENT's PTE: physical = 0x2B4000, permissions = RW
    d. Decrement reference count on old page (2 → 1)
    e. If ref count reaches 1: restore write permission in other PTE
  4. Return to user code – write instruction RETRIED and succeeds

Result:
  Parent PTE: 0x2B4000 (RW) – private copy
  Child PTE: 0x1A3000 (RW) – now sole owner, write restored
```

Step 4: The Return Value Magic

```
; How fork returns DIFFERENT values to parent and child:
;
; When fork() is called:
;   1. Kernel creates child task_struct
;   2. Kernel sets up child's kernel stack with saved registers
;   3. In child's saved registers: RAX = 0
;   4. In parent's execution path: RAX = child's PID
;   5. Both "return" from the syscall, but with different RAX values

; From the kernel's perspective (pseudocode):
; pid = create_child_task()
; child->saved_regs.rax = 0           ; Child will see 0
; parent_return_value = child->pid    ; Parent will see child PID
; schedule(child)                    ; Put child on run queue
; return parent_return_value          ; Return to parent
```

Step 5: Fork with exec (The Common Pattern)

fork() + execve() – why COW makes this efficient:

Without COW:

fork(): Copy entire address space (slow!)

execve(): Immediately throw it all away and load new program (waste!)

With COW:

fork(): Share all pages read-only (fast! ~microseconds)

execve(): Release shared page references (just decrement counters)

Load new program (only map what's needed)

Result: The "copy" in fork was never actually made!

Child never wrote to inherited pages → no actual copying happened!

Common Fork Patterns

Safe fork + exec

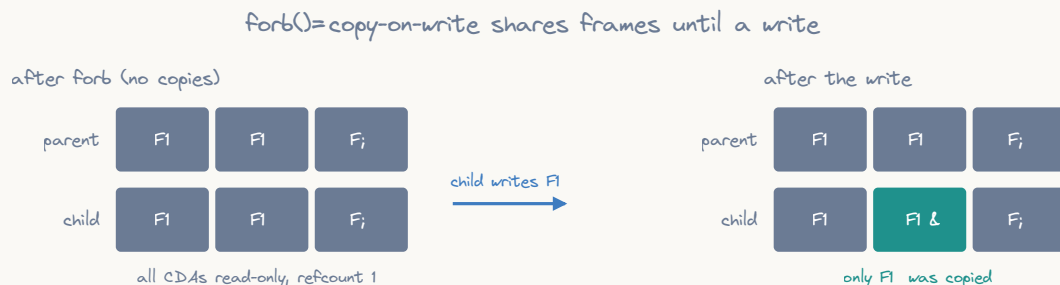
```
; Parent:
; fork() → gets child PID
; waitpid(child) → blocks until child exits
;
; Child:
; fork() → gets 0
; close unnecessary FDs
; execve(new_program)
; _exit(127) if exec fails ← NOT exit()! (would flush parent's stdio)
```

vfork (Ultra-Fast Fork)

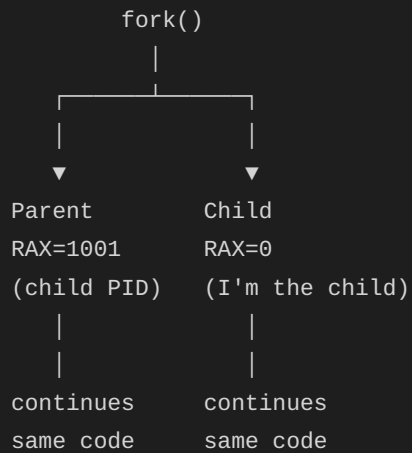
```
; vfork() – even faster than COW fork:
; - Child SHARES parent's address space entirely (no PTE copy!)
; - Parent is SUSPENDED until child calls exec or _exit
; - Child must NOT modify any memory (would corrupt parent!)
; - Only useful for immediate fork+exec pattern
;
; sys_vfork = 58
;
; Danger: If child doesn't immediately exec/exec, undefined behavior!
; Modern alternative: posix_spawn() or clone(CLONE_VFORK)
```

Why Fork Returns Twice

This confuses everyone at first. Think of it this way:



ASCII diagram



Both processes execute the exact same code after fork. The ONLY way to tell them apart is checking the return value. That's why the pattern is always:

```

mov rax, 57      ; sys_fork
syscall
test rax, rax
jz .child        ; RAX=0 → child
; RAX>0 → parent (and RAX = child's PID)
  
```

TL;DR

Question	Answer
Does fork copy all memory?	No! Just page table entries (~2MB for 1GB process)
When is memory actually copied?	Only when someone writes (Copy-on-Write)
Is fork slow for large processes?	No — COW makes it fast (~microseconds)
Why different return values?	Kernel sets RAX=0 in child's saved regs, RAX=pid in parent
What if child never writes?	No pages ever copied (fork+exec pattern)
What's shared after fork?	Physical pages (until written), open file descriptions
What's NOT shared?	PID, page tables (separate copies), parent PID
vfork vs fork?	vfork shares address space entirely (dangerous, fast)

CHAPTER 9

Q&A: How Does malloc() Actually Work?

The Question

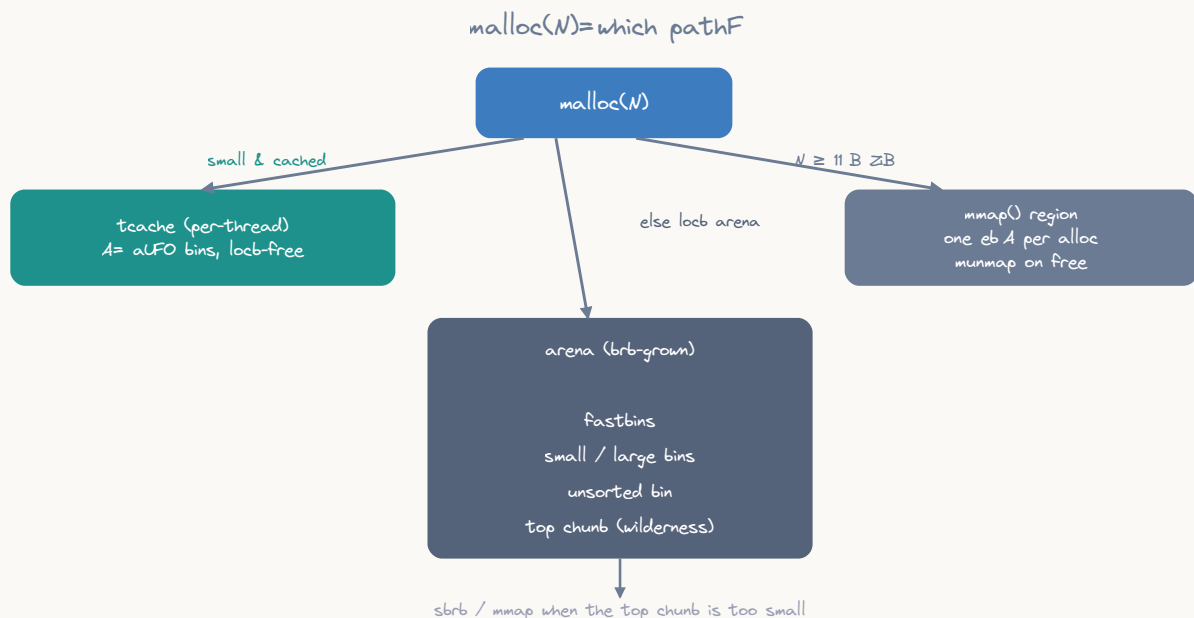
When I call `malloc(64)` in C (or implement my own in assembly), what actually happens from the application level down to physical memory? How does `free()` know how much to free? Where does fragmentation come from?

Quick Answer

`malloc(n)` does NOT call the kernel every time. It manages a pool of pre-obtained memory using a free list. Only when the pool is empty does it request more from the kernel via `brk()` (small allocations) or `mmap()` (large allocations). `free()` doesn't return memory to the kernel — it adds the chunk back to the free list for reuse.

The Full Story

Level 0: What malloc Returns



ASCII diagram

Fast bins (16-80 bytes, step 16) – LIFO, single-linked, never coalesced:

```
fastbin[0]: 16-byte chunks → □ → □ → □ → NULL
fastbin[1]: 32-byte chunks → □ → □ → NULL
fastbin[2]: 48-byte chunks → □ → NULL
fastbin[3]: 64-byte chunks → □ → □ → □ → □ → NULL
fastbin[4]: 80-byte chunks → NULL
```

Small bins (96-1024 bytes) – FIFO, doubly-linked, coalesced on free:

```
smallbin[0]: 96-byte chunks → □ ↔ □ ↔ □
smallbin[1]: 112-byte chunks → □ ↔ □
...
```

Large bins (>1024 bytes) – sorted by size, doubly-linked:

```
largebin[0]: 1024-2048 → □(1100) ↔ □(1500) ↔ □(2000)
...
```

Unsorted bin – recently freed chunks awaiting classification:

```
unsorted → □ → □ → □ → NULL
```

Level 3: malloc Decision Tree

malloc(n):

1. n < 32 bytes? → return from tcache (per-thread cache)
2. n < 80 bytes? → return from fastbin[size_to_index(n)]
3. n < 1024 bytes? → return from smallbin[size_to_index(n)]
4. Otherwise:
 - a. Consolidate fastbins into unsorted bin
 - b. Search unsorted bin for exact or close fit
 - c. Search large bins for best fit
 - d. Split the "top chunk" (wilderness)
 - e. If top chunk too small: extend heap with brk()
 - f. If n >= 128KB: use mmap() directly

Level 4: Why free() Doesn't Return Memory

After free(p):

- Chunk is added to free list (available for future malloc)
- Physical pages are NOT returned to kernel
- Virtual address space is NOT released
- Process RSS (resident set) stays the same

Why?

- brk() can only release memory at the TOP of the heap
- If any chunk above is still allocated, can't shrink
- Future mallocs will reuse freed chunks (fast, no syscall)

When IS memory returned?

- mmap'd chunks (>128KB): munmap'd immediately by free()
- glibc's malloc_trim(): explicitly release top of heap
- Process exit: kernel reclaims everything

Assembly-Level Implementation

malloc in ~30 Lines of Assembly (Simplified)

```
; Minimal brk-based allocator:
my_malloc:
    ; RDI = requested size
    add rdi, 16          ; Add header size
    add rdi, 15
    and rdi, ~15         ; Align to 16 bytes

    ; Extend heap
    push rdi             ; Save total size
    mov rax, 12          ; sys_brk
    xor rdi, rdi         ; Get current break
    syscall
    mov rbx, rax         ; Old break = chunk start

    pop rdi              ; Total size
    lea rdi, [rbx + rdi] ; New break
    mov rax, 12          ; sys_brk
    syscall

    ; Write header
    pop rdi              ; (need to recalculate)
    ; Actually, simpler version:
    mov qword [rbx], rdi ; Store size in header

    ; Return pointer past header
    lea rax, [rbx + 16]
    ret
```

Common Bugs Explained

Use-After-Free

```
p = malloc(64);    → chunk allocated, p valid
free(p);           → chunk on free list, p now DANGLING
*p = 42;           → writing to freed chunk!
                   Might corrupt free list metadata!
                   Might silently work (until next malloc uses that chunk)
```

Double Free

```
free(p);           → chunk added to free list
free(p);           → SAME chunk added AGAIN!
                    free list now has a cycle: ... → chunk → ... → chunk → ...
                    Next two mallocs return SAME pointer!
                    Both callers think they own the memory exclusively!
```

Heap Overflow

```
p = malloc(64);
memcpy(p, data, 100); → wrote 36 bytes past end of chunk!
                       Corrupted NEXT chunk's header!
                       Next free/malloc will read garbage size → crash/exploit
```

TL;DR

Question	Answer
Does malloc syscall every time?	No — only when its pool is empty
Which syscall?	brk() for small (<128KB), mmap() for large
How does free know the size?	Hidden header 16 bytes before your pointer
Does free return memory to OS?	Usually no (only for mmap'd chunks)
What's the minimum allocation?	32 bytes (header + alignment)
Why is malloc fast?	Per-thread caches, size-indexed bins, no syscall usually
What causes fragmentation?	Alternating alloc/free of different sizes

CHAPTER 10

Instruction Encoding in x86-64

Advanced Topic - Understanding how assembly becomes machine code

Overview

When you write assembly, NASM translates it into **machine code** - raw bytes that the CPU understands. Understanding this encoding helps you:

- Write more compact code
- Understand performance differences
- Debug at the lowest level
- Appreciate CPU design

Note for C Programmers: This topic covers low-level machine code encoding that happens transparently when you compile C code. C programmers don't typically need to worry about instruction encoding - the compiler handles it automatically. However, understanding it helps explain why some operations are faster or more compact than others.

The Translation Process

```
You Write:      mov rax, rbx
                  ↓
NASM Assembles:  48 89 D8      (3 bytes)
                  ↓
CPU Decodes:     "Move RBX into RAX"
                  ↓
CPU Executes:    RAX now contains value from RBX
```

Part 1: Instruction Format

x86-64 instructions are **variable length** (1-15 bytes) with this structure:

Anatomy of an x86-64 instruction



only the opcode is mandatory; most instructions are 1-15 bytes

```
mov edi,rsi; 1e; eax = 0; opcode = 0; ModR/M = 0; SIB = 0; displacement = 0; immediate = 0
```

ASCII diagram

Prefixes	Opcode	ModR/M	SIB	Displacement	Immediate
(0-4)	(1-3)	(0-1)	(0-1)	(0,1,2,4)	(0,1,2,4,8)
Optional	Required	Optional	Optional	Optional	Optional

Minimum: 1 byte (opcode only)
Maximum: 15 bytes (all components)

Component Breakdown

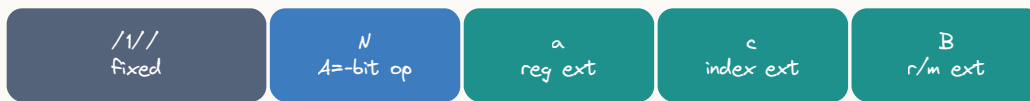
Component	Size	Purpose	Example
Prefixes	0-4 bytes	Modify instruction behavior	REX (64-bit), LOCK
Opcode	1-3 bytes	The operation to perform	0x89 (MOV), 0x31 (XOR)
ModR/M	0-1 byte	Specifies operands	Register or memory
SIB	0-1 byte	Complex addressing	base + index*scale
Displacement	0,1,2,4 bytes	Memory offset	[rbx + 100]
Immediate	0,1,2,4,8 bytes	Constant value	mov eax, 42

Part 2: The REX Prefix - 64-bit Mode

The **REX prefix** (0x40-0x4F) enables 64-bit operations and extended registers.

REX Byte Structure

The REX prefix byte



N=1 selects A=bit operands; a / c / B supply the high bit to reach rB r1/

ASCII diagram



Fixed | | | |

- Extends ModR/M r/m field (R8-R15)
- Extends SIB index field (R8-R15)
- Extends ModR/M reg field (R8-R15)
- 1 = 64-bit operand, 0 = default

Common REX values:

0x48 (01001000) = REX.W - 64-bit operation

0x49 (01001001) = REX.W + REX.B - 64-bit with extended r/m

0x4C (01001100) = REX.W + REX.R + REX.X - full extensions

When REX is Required

```
; 64-bit operations - NEEDS REX.W
mov rax, rbx      ; 48 89 D8 (3 bytes)
add rax, rcx      ; 48 01 C8 (3 bytes)

; 32-bit operations - NO REX needed
mov eax, ebx      ; 89 D8 (2 bytes)
add eax, ecx      ; 01 C8 (2 bytes)

; Extended registers (R8-R15) - NEEDS REX
mov r8, rax       ; 49 89 C0 (3 bytes)
mov rax, r9       ; 4C 89 C8 (3 bytes)

; Accessing low bytes of RSI, RDI, RBP, RSP - NEEDS REX
mov sil, 0        ; 40 B6 00 (3 bytes)
```

REX Examples

; Example 1: mov rax, rbx

48 89 D8

|

└ 0100 1000

|

└ W=1 (64-bit operation)

; Example 2: mov r9, r10

4D 89 D1

|

└ 0100 1101

||||

||| └ B=1 (r/m uses R8-R15)

|| └ X=0 (not used)

| └ R=1 (reg uses R8-R15)

└ W=1 (64-bit)

Part 3: Opcodes

The opcode identifies the operation. Can be 1-3 bytes.

Common 1-Byte Opcodes

Opcode	Operation
0x01	ADD r/m, r
0x29	SUB r/m, r
0x31	XOR r/m, r
0x89	MOV r/m, r
0x8B	MOV r, r/m
0x50-57	PUSH r (RAX-RDI)
0x58-5F	POP r (RAX-RDI)
0x90	NOP
0xC3	RET
0xEB	JMP short
0xFF	Various (INC, DEC, CALL, JMP)

2-Byte Opcodes (0x0F prefix)

Opcode	Operation
0F 84	JZ/JE (conditional jump)
0F 85	JNZ/JNE
0F AF	IMUL r, r/m
0F B6	MOVZX r, r/m8 (zero extend)
0F BE	MOVSX r, r/m8 (sign extend)

Opcode Direction

Many operations have two opcodes for different directions:

```
mov eax, ebx      ; 89 D8 (opcode 0x89: MOV r/m, r)
mov ebx, eax      ; 8B D8 (opcode 0x8B: MOV r, r/m)

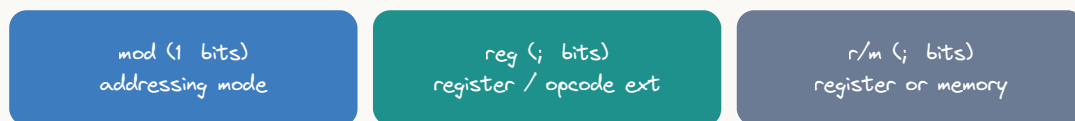
; The direction is encoded in the opcode itself!
```

Part 4: ModR/M Byte

The **ModR/M** byte specifies operands and addressing modes.

ModR/M Structure

The ModR/M byte



mod=11 → register operand; mod=0001/0 → memory with 1/-byte disp

ASCII diagram

Mod	Reg	R/M
(2 bits)	(3 bits)	(3 bits)
7	6	5
4	3	2
1	0	

Mod Field (Addressing Mode)

Mod	Meaning
00	[reg] - indirect, no displacement
01	[reg + disp8] - 8-bit displacement
10	[reg + disp32] - 32-bit displacement
11	Register direct (no memory access)

Reg Field (Register)

Reg	Without REX.R	With REX.R
000	RAX/EAX/AX/AL	R8/R8D/R8W/R8B
001	RCX/ECX/CX/CL	R9/R9D/R9W/R9B
010	RDX/EDX/DX/DL	R10/R10D/.../..
011	RBX/EBX/BX/BL	R11/R11D/.../..
100	RSP/ESP/SP/SPL	R12/R12D/.../..
101	RBP/EBP/BP/BPL	R13/R13D/.../..
110	RSI/ESI/SI/SIL	R14/R14D/.../..
111	RDI/EDI/DI/DIL	R15/R15D/.../..

R/M Field (Register/Memory)

Same encoding as Reg field, but with special meaning when Mod=00/01/10:

When Mod != 11 (memory mode):

R/M	Meaning
000	[RAX/R8]
001	[RCX/R9]
010	[RDX/R10]
011	[RBX/R11]
100	SIB byte follows
101	[RIP + disp32] or [RBP + disp]
110	[RSI/R14]
111	[RDI/R15]

ModR/M Examples

D8 binary: 11 011 000

```
| | └─ R/M = 000 (RAX)
| └─ Reg = 011 (RBX)
└─ Mod = 11 (register direct)
```

Translation: "Move RBX (reg) to RAX (r/m)"

03 binary: 00 000 011

```
| | | R/M = 011 (RBX)
| | | Reg = 000 (EAX)
| | | Mod = 00 (indirect, no disp)
```

Translation: "Move [RBX] (memory) to EAX (reg)"

43 binary: 01 000 011

```
| | | R/M = 011 (RBX)
| | | Reg = 000 (EAX)
| | | Mod = 01 (indirect with disp8)
```

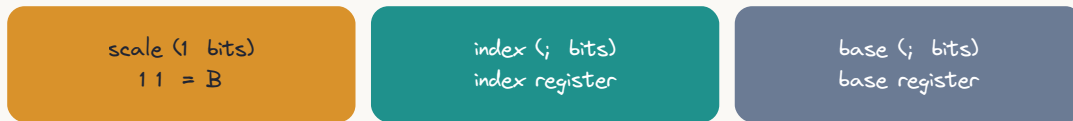
Translation: "Add [RBX + 8] to EAX"

Part 5: SIB Byte (Scale-Index-Base)

Used for complex addressing: $[base + index * scale + disp]$

SIB Structure

The SIB byte (scaled-index addressing)



present only when r/m=1//; encodes dbase (index&scale) like drbx (rsi&=e

ASCII diagram

Scale	Index	Base
(2 bits)	(3 bits)	(3 bits)
7 6	5 4 3	2 1 0

Scale Field

Scale	Multiplier
00	*1
01	*2
10	*4
11	*8

Index and Base Fields

Use same register encoding as ModR/M Reg field (000-111).

Special Cases:

- Index = 100 (RSP) means “no index”
- Base = 101 (RBP) with Mod=00 means disp32 only

SIB Examples

```
; Example 1: mov eax, [rbx + rcx*4]
; Encoding: 8B 04 8B
;           Opcode = 8B (MOV r, r/m)
;           ModR/M = 04 = 00000100
;           SIB = 8B = 10001011
```

ModR/M: 00 000 100

```

| | | R/M = 100 (SIB follows)
| |   Reg = 000 (EAX)
| |   Mod = 00 (no displacement)
```

SIB: 10 001 011

```

| | | Base = 011 (RBX)
| |   Index = 001 (RCX)
| |   Scale = 10 (*4)
```

Translation: "Move [RBX + RCX*4] to EAX"

```
; Example 2: mov eax, [rbx + rcx*4 + 8]
; Encoding: 8B 44 8B 08
;           Opcode = 8B
;           ModR/M = 44 = 01000100
;           SIB = 8B = 10001011
;           Disp8 = 08
```

ModR/M: 01 000 100

```

| | | R/M = 100 (SIB follows)
| |   Reg = 000 (EAX)
| |   Mod = 01 (disp8 follows)
```

SIB: 10 001 011

```

| | | Base = 011 (RBX)
| |   Index = 001 (RCX)
| |   Scale = 10 (*4)
```

Disp8: 08 (8 in decimal)

Translation: "Move [RBX + RCX*4 + 8] to EAX"

```
; Example 3: Array access pattern
; C code: array[i] where sizeof(int) = 4
mov eax, [rdi + rsi*4] ; RDI=array base, RSI=index
; Encoding: 8B 04 B7
```

Part 6: Complete Encoding Examples

Example 1: `nop`

Assembly: `nop`

Machine: `90`

Analysis:

`90` = Opcode for NOP (No Operation)

Total: 1 byte (shortest instruction!)

Example 2: `xor eax, eax`

Assembly: `xor eax, eax`

Machine: `31 C0`

Analysis:

`31` = Opcode (XOR r/m, r)

`C0` = ModR/M byte

`C0` = `11 000 000`

| | └─ R/M = `000` (EAX)

| └─ Reg = `000` (EAX)

└─ Mod = `11` (register)

Total: 2 bytes

Why it's short: No prefixes, no immediate, register-to-register

Example 3: `mov eax, 0`

Assembly: `mov eax, 0`

Machine: `B8 00 00 00 00`

Analysis:

`B8` = Opcode (MOV EAX, imm32) - special short form

`00 00 00 00` = 32-bit immediate (0)

Total: 5 bytes (1 opcode + 4 immediate)

Why it's long: Must encode the full 32-bit immediate value

Example 4: `mov rax, rbx`

Assembly: `mov rax, rbx`

Machine: `48 89 D8`

Analysis:

48 = REX.W (enable 64-bit)

0100 1000

└─ W=1 (64-bit operands)

89 = Opcode (MOV r/m, r)

D8 = ModR/M byte

11 011 000

└─ R/M = 000 (RAX destination)

└─ Reg = 011 (RBX source)

└─ Mod = 11 (register)

Total: 3 bytes (REX + opcode + ModR/M)

Example 5: `push rax`

Assembly: `push rax`

Machine: `50`

Analysis:

50 = Opcode (PUSH RAX)

Note: PUSH has special 1-byte encodings for each register:

50 = PUSH RAX/R8

51 = PUSH RCX/R9

52 = PUSH RDX/R10

53 = PUSH RBX/R11

54 = PUSH RSP/R12

55 = PUSH RBP/R13

56 = PUSH RSI/R14

57 = PUSH RDI/R15

Total: 1 byte (most compact!)

Example 6: `mov dword [rdi + rsi*4 + 100], eax`

Assembly: `mov dword [rdi + rsi*4 + 100], eax`

Machine: `89 84 B7 64 00 00 00`

Analysis:

89 = Opcode (MOV r/m, r)

84 = ModR/M byte

`10 000 100`

| | └─ R/M = 100 (SIB follows)

| └─ Reg = 000 (EAX source)

└─ Mod = 10 (disp32 follows)

B7 = SIB byte

`10 110 111`

| | └─ Base = 111 (RDI)

| └─ Index = 110 (RSI)

└─ Scale = 10 (*4)

`64 00 00 00` = Displacement (100 in little-endian)

Total: 7 bytes (opcode + ModR/M + SIB + disp32)

Example 7: `mov r15, [r14 + 8]`

Assembly: `mov r15, [r14 + 8]`

Machine: `4D 8B 7E 08`

Analysis:

4D = REX byte

0100 1101

||||

|||└ B=1 (R/M uses R8-R15)

||└ X=0 (not used)

|└ R=1 (Reg uses R8-R15)

└ W=1 (64-bit)

8B = Opcode (MOV r, r/m)

7E = ModR/M byte

01 111 110

| |└ R/M = 110 (with REX.B = R14)

|└ Reg = 111 (with REX.R = R15)

└ Mod = 01 (disp8)

08 = Displacement (8)

Total: 4 bytes (REX + opcode + ModR/M + disp8)

Part 7: Why Instruction Size Matters

Performance Impact

Instruction Cache:

- L1 I-cache: 32-64 KB (very fast)
- Smaller code = more instructions fit in cache
- Cache miss = ~50-100 cycle penalty

Example:

1000 instances of "xor eax, eax" (2 bytes each) = 2 KB

1000 instances of "mov eax, 0" (5 bytes each) = 5 KB

Difference: 3 KB saved = more cache-friendly

Decoding Efficiency

Modern CPUs decode multiple instructions per cycle:

- Shorter instructions = more can be decoded
- Simple instructions = faster decode

Intel/AMD can decode 4-5 instructions per cycle if:

- ✓ Instructions are short
- ✓ Instructions are simple
- ✓ No complex addressing modes

Branch Prediction

Smaller code = shorter branch distances

- Short jumps: 2 bytes (JMP rel8)
- Long jumps: 5 bytes (JMP rel32)

More code fits in prediction buffers

Part 8: Optimization Techniques

1. Choose Shorter Encodings

```
; BAD: 7 bytes
mov rax, 0           ; 48 C7 C0 00 00 00 00

; GOOD: 3 bytes (and faster!)
xor eax, eax         ; 31 C0
; Note: zeros upper 32 bits automatically in 64-bit mode
```

2. Use 32-bit When Possible

```
; If you only need 32-bit result:
xor eax, eax         ; 2 bytes (no REX needed)
; vs
xor rax, rax         ; 3 bytes (needs REX.W)

; Both zero the full 64-bit register!
```

3. Leverage Special Encodings

```
; PUSH/POP are compact
push rax           ; 1 byte
push rbx           ; 1 byte
; ... do work ...
pop rbx            ; 1 byte
pop rax            ; 1 byte

; vs saving to memory (much longer)
mov [temp1], rax   ; 7-8 bytes
mov [temp2], rbx   ; 7-8 bytes
```

4. Minimize Immediate Values

```
; BAD: 5 bytes
mov eax, 1         ; B8 01 00 00 00

; BETTER: 5 bytes total, but might be faster
xor eax, eax       ; 31 C0 (2 bytes)
inc eax            ; FF C0 (2 bytes) or 48 FF C0 (3 bytes for RAX)

; CONTEXT MATTERS: If you already have EAX=0, just INC!
```

5. Use LEA for Arithmetic

```
; Instead of multiple operations:
add eax, ebx       ; 2 bytes
add eax, 8          ; 3-6 bytes
; Total: 5-8 bytes

; Use LEA (Load Effective Address):
lea eax, [ebx + eax + 8]; 3-4 bytes
; Does (EBX + EAX + 8) in one instruction!
```

6. Avoid Unnecessary Prefixes

```
; 64-bit REX prefix adds 1 byte
; If you don't need 64-bit, don't use it

; Accessing low 32-bits:
mov eax, [rdi]     ; No REX needed if EAX is enough
; vs
mov rax, [rdi]     ; Needs REX.W
```

Part 9: Tools for Exploration

Using objdump

```
# Create test file
cat > test.asm << 'EOF'
section .text
    global _start
_start:
    xor eax, eax
    mov eax, 0
    mov rax, rbx
    push rax
    mov eax, [rbx + rcx*4 + 8]
EOF

# Assemble
nasm -f elf64 test.asm -o test.o

# Disassemble with machine code
objdump -d test.o -M intel

# Output:
# 0: 31 c0          xor    eax, eax
# 2: b8 00 00 00 00 mov    eax, 0x0
# 7: 48 89 d8       mov    rax, rbx
# a: 50            push   rax
# b: 8b 44 8b 08   mov    eax, DWORD PTR [rbx+rcx*4+0x8]
```

Using hexdump

```
# View raw bytes
hexdump -C test.o | less

# Look for the .text section
```

Online Assembler

- **Defuse Online x86 Assembler:** <https://defuse.ca/online-x86-assembler.htm>
- Type assembly, get machine code instantly
- Great for quick experiments
- **Godbolt Compiler Explorer:** <https://godbolt.org/>
- See how C code compiles to assembly
- Shows machine code bytes

NASM List File

```
# Generate listing file with machine code
nasm -f elf64 -l test.lst test.asm

# View listing
cat test.lst

# Shows line numbers, addresses, machine code, and source
```

Part 10: Decoding Practice

Exercise 1

Machine code: 48 01 C3

Decode it:

48 = ?

01 = ?

C3 = ?

Assembly: ?

Solution

48 = REX.W (64-bit)

01 = ADD r/m, r

C3 = 11 000 011

| | └─ R/M = 011 (RBX)

| └─ Reg = 000 (RAX)

└─ Mod = 11 (register)

Assembly: add rbx, rax

Exercise 2

Machine code: 89 D8

Decode it:

89 = ?

D8 = ?

Assembly: ?

Solution

```

89 = MOV r/m, r
D8 = 11 011 000
    |  |  └─ R/M = 000 (EAX/RAX)
    |  └─── Reg = 011 (EBX/RBX)
    └────── Mod = 11 (register)

```

Assembly: `mov eax, ebx` (or `mov rax, rbx` with REX)
Context determines size!

Exercise 3

Machine code: 50

What instruction?

Solution

50 = PUSH RAX (or PUSH R8 with REX.B)

Single-byte encoding for PUSH register.

Exercise 4

Machine code: 8B 04 8B

Decode it:

8B = ?

04 = ?

8B = ?

Assembly: ?

Solution

```

8B = MOV r, r/m
04 = 00 000 100
    |  |  └─ R/M = 100 (SIB follows)
    |  └─── Reg = 000 (EAX)
    └────── Mod = 00 (no disp)

```

```

8B = 10 001 011
    |  |  └─ Base = 011 (RBX)
    |  └─── Index = 001 (RCX)
    └────── Scale = 10 (*4)

```

Assembly: `mov eax, [rbx + rcx*4]`

Part 11: Advanced Topics

RIP-Relative Addressing (64-bit)

```
; Position-independent code uses RIP-relative  
mov rax, [rel myvar]    ; Encoded relative to instruction pointer  
  
; Encoding uses ModR/M with special values:  
; Mod = 00, R/M = 101 means [RIP + disp32]
```

VEX/EVEX Prefixes (AVX/AVX-512)

```
SIMD instructions use longer prefixes:  
- VEX: 2-3 byte prefix (AVX, AVX2)  
- EVEX: 4 byte prefix (AVX-512)  
  
Example:  
vaddps xmm0, xmm1, xmm2 ; VEX prefix + opcode
```

Lock Prefix

```
; Atomic operations  
lock add [counter], 1    ; 0xF0 prefix + normal encoding
```

Segment Overrides

```
; Rarely used in 64-bit mode  
mov rax, gs:[0x28]       ; TLS access (0x65 prefix)
```

Summary Tables

Instruction Length by Category

Instruction Type	Bytes	Example
Simple ops	1-2	NOP, PUSH, POP
Register-register	2-3	ADD, MOV, XOR
Register-immediate	5-7	MOV EAX, 42
Memory-register	3-8	MOV [RBX], RAX
Complex addressing	7-10	MOV [RBX+RCX*4+N], .

Component Sizes

Component	Bytes	When Present
REX prefix	1	64-bit or R8-R15
Opcode	1-3	Always
ModR/M	1	Most instructions
SIB	1	Complex addressing
Displacement	1-4	Memory with offset
Immediate	1-8	Constant operand

Key Takeaways

1. **Variable Length:** Instructions are 1-15 bytes
2. **Structured Format:** Each component has a specific purpose
3. **REX is for 64-bit:** Required for 64-bit ops and R8-R15
4. **Shorter = Better:** Saves cache space and decoding time
5. **ModR/M is Complex:** Handles registers and all addressing modes
6. **SIB for Arrays:** Essential for `base + index*scale` patterns
7. **Special Encodings:** Some instructions have compact forms (PUSH, NOP)
8. **Optimization Matters:** Understanding encoding helps write better code

Further Reading

Official Documentation

- **Intel® 64 and IA-32 Architectures Software Developer's Manual**
 - Volume 2: Instruction Set Reference
 - Chapter 2: Instruction Format
- **AMD64 Architecture Programmer's Manual**
 - Volume 3: Instruction Reference

Articles & Guides

- **Agner Fog's Optimization Manuals:** CPU-specific optimization
- **OSDev Wiki:** x86-64 instruction encoding
- **Felix Cloutier's x86 Reference:** Online instruction reference

Tools

- **Intel XED:** Intel's encoder/decoder library
- **Capstone:** Disassembly framework
- **NASM Documentation:** Assembler-specific details

Next Steps

Now that you understand instruction encoding:

1. Read disassembly output with understanding
2. Write more optimal assembly code
3. Debug at the machine code level
4. Appreciate compiler optimizations
5. Understand performance implications

Continue with: [Topic 3: Basic Instructions](#)

[← Back to Main](#) | [Q&A Index](#)

CHAPTER 11

int 0x80 vs syscall - When to Use Which?

Question

When to use int 0x80, and when to use syscall?

Quick Answer

Method	Architecture	Use When
<code>int 0x80</code>	32-bit	Writing 32-bit programs or need legacy compatibility
<code>syscall</code>	64-bit	Writing modern 64-bit programs (preferred)

Detailed Comparison

1. Architecture Requirement

`int 0x80` - Legacy 32-bit Method

- Works on: 32-bit x86 processors
- Can work on 64-bit processors running in 32-bit mode
- Been around since the 386 processor

`syscall` - Modern 64-bit Method

- Works on: 64-bit x86-64 processors only
 - Introduced with AMD's x86-64 architecture
 - **Does NOT work on pure 32-bit systems**
-

2. Register Usage - CRITICAL DIFFERENCE!

`int 0x80` (32-bit registers)

```
mov eax, 4      ; syscall number in EAX
mov ebx, 1      ; arg1 in EBX
mov ecx, msg    ; arg2 in ECX
mov edx, len    ; arg3 in EDX
mov esi, arg4   ; arg4 in ESI (if needed)
mov edi, arg5   ; arg5 in EDI (if needed)
int 0x80        ; trigger interrupt
```

`syscall` (64-bit registers)

```
mov rax, 1      ; syscall number in RAX
mov rdi, 1      ; arg1 in RDI
mov rsi, msg    ; arg2 in RSI
mov rdx, len    ; arg3 in RDX
mov r10, arg4   ; arg4 in R10 (if needed)
mov r8, arg5    ; arg5 in R8 (if needed)
mov r9, arg6    ; arg6 in R9 (if needed)
syscall         ; fast system call
```

Key Difference:

- `int 0x80`: EAX, EBX, ECX, EDX, ESI, EDI
- `syscall`: RAX, RDI, RSI, RDX, R10, R8, R9

3. Syscall Numbers - DIFFERENT VALUES!

The same system call has **different numbers** in 32-bit vs 64-bit!

Operation	32-bit (<code>int 0x80</code>)	64-bit (<code>syscall</code>)
sys_read	3	0
sys_write	4	1
sys_open	5	2
sys_close	6	3
sys_exit	1	60

Example - Exit syscall:

```

; 32-bit exit
mov eax, 1      ; exit is syscall #1 in 32-bit
xor ebx, ebx    ; exit code 0
int 0x80

; 64-bit exit
mov rax, 60     ; exit is syscall #60 in 64-bit!
xor rdi, rdi    ; exit code 0
syscall

```

4. Performance

`syscall` is MUCH faster than `int 0x80`:

- `int 0x80`: ~300 CPU cycles (slow context switch)
- `syscall`: ~100 CPU cycles (optimized fast path)

`syscall` was designed specifically to be faster by avoiding the overhead of interrupt handling.

5. Practical Examples

Complete 32-bit Program (`int 0x80`)

C Equivalent (same for both 32-bit and 64-bit):

```

#include <unistd.h>

int main() {
    const char *msg = "32-bit mode!\n";
    size_t len = 13;

    // write(1, msg, len)
    write(1, msg, len);

    // exit(0)
    return 0;
}

```

Assembly (32-bit):


```

; Build: nasm -f elf32 prog32.asm && ld -m elf_i386 -o prog32 prog32.o

section .data
    msg db "32-bit mode!", 10
    len equ $ - msg

section .text
    global _start

_start:
    ; write(1, msg, len)
    mov eax, 4          ; sys_write = 4
    mov ebx, 1          ; stdout
    mov ecx, msg        ; buffer
    mov edx, len        ; count
    int 0x80

    ; exit(0)
    mov eax, 1          ; sys_exit = 1
    xor ebx, ebx        ; status 0
    int 0x80

```

Complete 64-bit Program (`syscall`)

C Equivalent (same as 32-bit version):

```

#include <unistd.h>

int main() {
    const char *msg = "64-bit mode!\n";
    size_t len = 13;

    // write(1, msg, len)
    write(1, msg, len);

    // exit(0)
    return 0;
}

// Note: The C code is identical - only the assembly differs!

```

Assembly (64-bit):

```

; Build: nasm -f elf64 prog64.asm && ld -o prog64 prog64.o

section .data
    msg db "64-bit mode!", 10
    len equ $ - msg

section .text
    global _start

_start:
    ; write(1, msg, len)
    mov rax, 1          ; sys_write = 1
    mov rdi, 1          ; stdout
    mov rsi, msg        ; buffer
    mov rdx, len        ; count
    syscall

    ; exit(0)
    mov rax, 60         ; sys_exit = 60
    xor rdi, rdi        ; status 0
    syscall

```

x

6. What Happens If You Mix Them Up?

Using `int 0x80` in 64-bit mode

```

; This WILL work but is WRONG and SLOW
section .text
    global _start

_start:
    mov eax, 4          ; Wrong! Using 32-bit syscall numbers
    mov ebx, 1
    mov ecx, msg
    mov edx, len
    int 0x80            ; Works but uses compatibility layer (slow!)

```

Problems:

- ✗ Uses slow compatibility mode
- ✗ Only accesses lower 32 bits of registers
- ✗ Different syscall numbers may cause confusion
- ✓ Will technically work on Linux (for compatibility)

Using `syscall` in 32-bit mode

```
; This will CRASH!
mov rax, 1
syscall           ; ILLEGAL INSTRUCTION on 32-bit CPU!
```

Result: `Illegal instruction (core dumped)`

7. How to Detect Your Architecture

Check what you're running:

```
# Check if your kernel is 64-bit
uname -m
# Output: x86_64 (64-bit) or i686/i386 (32-bit)

# Check what format your program is
file ./your_program
# Output examples:
# ELF 32-bit LSB executable -> use int 0x80
# ELF 64-bit LSB executable -> use syscall
```

8. Decision Tree

```
Are you writing for 64-bit Linux?
|
├─ YES → Use `syscall`
|   • Use 64-bit registers (rax, rdi, rsi, rdx...)
|   • Use 64-bit syscall numbers
|   • Assemble with: nasm -f elf64
|   • Link with: ld (default)
|
└─ NO (32-bit or compatibility)
    → Use `int 0x80`
    • Use 32-bit registers (eax, ebx, ecx, edx...)
    • Use 32-bit syscall numbers
    • Assemble with: nasm -f elf32
    • Link with: ld -m elf_i386
```

9. Modern Best Practice

For new code in 2025:

- Use `syscall` - You're almost certainly on 64-bit
- Faster and more efficient
- Better register usage (more registers available)
- Native to modern CPUs

Only use `int 0x80` if:

- You're maintaining legacy 32-bit code
- You're learning for historical purposes
- You're targeting ancient 32-bit systems

10. Quick Reference Card

32-BIT (<code>int 0x80</code>)	64-BIT (<code>syscall</code>)
<code>nasm -f elf32</code>	<code>nasm -f elf64</code>
<code>ld -m elf_i386</code>	<code>ld</code>
<code>mov eax, 4 ; write</code>	<code>mov rax, 1 ; write</code>
<code>mov ebx, 1</code>	<code>mov rdi, 1</code>
<code>mov ecx, msg</code>	<code>mov rsi, msg</code>
<code>mov edx, len</code>	<code>mov rdx, len</code>
<code>int 0x80</code>	<code>syscall</code>
<code>mov eax, 1 ; exit</code>	<code>mov rax, 60 ; exit</code>
<code>xor ebx, ebx</code>	<code>xor rdi, rdi</code>
<code>int 0x80</code>	<code>syscall</code>

TL;DR

- 64-bit program? → Use `syscall` (faster, modern, RAX/RDI/RSI/RDX)
- 32-bit program? → Use `int 0x80` (legacy, EAX/EBX/ECX/EDX)
- Different syscall numbers! (write is 4 vs 1, exit is 1 vs 60)
- Don't mix them or you'll get confused/slow code

Since you're on a modern Linux system (`x86_64`), stick with `syscall` for all your 64-bit programs!

See Also:

- [Topic 1: Setup & First Program](#)
- Linux Syscall Table: <https://filippo.io/linux-syscall-table/>

[← Back to Q&A Index](#)

CHAPTER 12

Understanding Memory Addressing with [] Brackets

Question

In `mov rbx, array_base` and `mov al, [rbx + 5]`, what does the `[ ]` syntax mean?

The Key Concept

The **square brackets** `[ ]` mean “go to this memory address and get/put the value there” - this is called **memory dereferencing** or **indirect addressing**.

```
; WITHOUT brackets - direct value  
mov rax, 5                ; RAX = 5 (the number 5)  
  
; WITH brackets - memory access  
mov rax, [5]              ; RAX = (value stored at memory address 5)
```

Think of it like this:

- **No brackets:** “Give me the number”
 - **Brackets:** “Go to this address and give me what’s there”
-

Analogy: House Addresses

```
Without brackets:  
mov rax, 123  
"The number is 123"  
  
With brackets:  
mov rax, [123]  
"Go to house number 123 and tell me what's inside"
```

Your Example Explained

```
mov rbx, array_base    ; RBX = address of array (like "123 Main St")
mov al, [rbx + 5]      ; Go to address (RBX + 5) and load 1 byte into AL
```

Step-by-Step:

```
section .data
    array_base db 10, 20, 30, 40, 50, 60, 70, 80
    ; Memory layout (assume array_base starts at address 0x1000):
    ; Address: 0x1000 0x1001 0x1002 0x1003 0x1004 0x1005 0x1006 0x1007
    ; Value:   10    20    30    40    50    60    70    80

section .text
    mov rbx, array_base    ; RBX = 0x1000 (address of first element)
    mov al, [rbx + 5]      ; AL = value at address (0x1000 + 5) = 0x1005
                          ; AL = 60
```

What happens:

1. **RBX** contains **0x1000** (the memory address)
2. **[rbx + 5]** means “address **0x1000 + 5 = 0x1005**”
3. CPU goes to address **0x1005** and reads 1 byte
4. That byte (value **60**) is loaded into **AL**

Visual Representation

Affective address = $d \text{ base} \langle \text{index} \rangle \text{scale} \langle \text{disp} \rangle e$



→ one memory operand, computed by the CCU

`mov eax, dbx [rsi] 4` → 14e loads int element rsi of an array at rbx(14

ASCII diagram

Memory:

Address	Val	
0x1000	10	← array_base (RBX points here)
0x1001	20	
0x1002	30	
0x1003	40	
0x1004	50	
0x1005	60	← [rbx + 5] reads from here
0x1006	70	
0x1007	80	

```
mov al, [rbx + 5]
```

└─ Go to this address and read the value

Comparison: With vs Without Brackets

```
section .data
    value dq 42
    ptr    dq value           ; ptr stores the ADDRESS of value

section .text
    ; WITHOUT brackets - load the address itself
    mov rax, value           ; RAX = 0x12345... (memory address of value)
    mov rbx, ptr             ; RBX = 0x12345... (same address)

    ; WITH brackets - load the value AT the address
    mov rax, [value]         ; RAX = 42 (the actual value)
    mov rbx, [ptr]           ; RBX = 0x12345... (address stored in ptr)
    mov rcx, [rbx]           ; RCX = 42 (go to address in RBX, read value)
```

Memory Addressing Modes

The brackets support several addressing modes:

1. Direct Address

```
mov rax, [0x1000]      ; Read from address 0x1000
mov [0x2000], rbx      ; Write RBX to address 0x2000
```

2. Register Indirect

```
mov rax, [rbx]         ; Read from address stored in RBX
mov [rdi], rax         ; Write RAX to address stored in RDI
```

3. Register + Displacement

```
mov rax, [rbx + 8]     ; Address = RBX + 8
mov al, [rsi + 100]    ; Address = RSI + 100
mov [rdi + 16], rcx    ; Write to address RDI + 16
```

4. Base + Index

```
mov rax, [rbx + rcx]   ; Address = RBX + RCX
mov al, [rsi + rdx]    ; Address = RSI + RDX
```

5. Base + Index*Scale + Displacement (SIB)

```
; Format: [base + index*scale + displacement]
; Scale can be 1, 2, 4, or 8

mov rax, [rbx + rcx*4] ; Address = RBX + (RCX * 4)
mov al, [rsi + rdx*8 + 16] ; Address = RSI + (RDX * 8) + 16

; Perfect for arrays!
; array[i] where each element is 4 bytes:
mov eax, [array + rcx*4]
```

6. Label (Named Address)

```
section .data
    myvar dq 100

section .text
    mov rax, myvar      ; RAX = address of myvar
    mov rax, [myvar]    ; RAX = 100 (value AT myvar)
```

Practical Examples

Example 1: Array Access

C Equivalent:

```
#include <stdint.h>

int main() {
    uint8_t numbers[] = {5, 10, 15, 20, 25};
    uint8_t *ptr = numbers; // ptr points to numbers[0]

    // Access individual elements (two ways)
    uint8_t value1 = ptr[0];    // value1 = 5 (numbers[0])
    uint8_t value2 = ptr[1];    // value2 = 10 (numbers[1])
    uint8_t value3 = *(ptr + 2); // value3 = 15 (numbers[2]) - pointer arithmetic

    // Using index variable
    int index = 2;
    uint8_t value4 = ptr[index]; // value4 = 15 (numbers[2])

    return 0;
}
```

Assembly:

```

section .data
    numbers db 5, 10, 15, 20, 25    ; Array of 5 bytes

section .text
    global _start

_start:
    mov rbx, numbers                ; RBX = address of numbers[0]

    ; Access individual elements
    mov al, [rbx + 0]               ; AL = 5 (numbers[0])
    mov al, [rbx + 1]               ; AL = 10 (numbers[1])
    mov al, [rbx + 2]               ; AL = 15 (numbers[2])
    mov al, [rbx + 3]               ; AL = 20 (numbers[3])
    mov al, [rbx + 4]               ; AL = 25 (numbers[4])

    ; Using index register
    mov rcx, 2                      ; index = 2
    mov al, [rbx + rcx]              ; AL = 15 (numbers[2])

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: 32-bit Integer Array

C Equivalent:

```

#include <stdint.h>

int main() {
    int32_t ints[] = {100, 200, 300, 400, 500};
    int32_t *ptr = ints; // ptr points to ints[0]

    int index = 2; // We want ints[2]

    // C automatically multiplies index by sizeof(int32_t) = 4
    int32_t value = ptr[index]; // value = 300

    // Equivalent pointer arithmetic:
    // int32_t value = *(ptr + index);

    return 0;
}

```

Assembly:

```

section .data
    ; Array of 32-bit integers (4 bytes each)
    ints dd 100, 200, 300, 400, 500

section .text
    global _start

_start:
    mov rbx, ints          ; RBX = base address
    mov rcx, 2             ; index = 2 (want ints[2])

    ; Access ints[2] - need to multiply index by 4 (size of int)
    mov eax, [rbx + rcx*4] ; EAX = 300

    ; Alternative without scale:
    mov rcx, 8             ; offset = 2 * 4 = 8 bytes
    mov eax, [rbx + rcx]   ; EAX = 300

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 3: Writing to Memory

C Equivalent:

```

#include <stdint.h>

int main() {
    char buffer[10]; // 10-byte buffer
    char *ptr = buffer;

    // Write values to buffer
    ptr[0] = 'H';
    ptr[1] = 'e';
    ptr[2] = 'l';
    ptr[3] = 'l';
    ptr[4] = 'o';
    ptr[5] = '\0'; // Null terminator

    // Now buffer contains "Hello\0"

    return 0;
}

```

Assembly:

```
section .bss
    buffer resb 10          ; 10-byte buffer

section .text
    global _start

_start:
    mov rbx, buffer        ; RBX = address of buffer

    ; Write values to buffer
    mov byte [rbx + 0], 'H'
    mov byte [rbx + 1], 'e'
    mov byte [rbx + 2], 'l'
    mov byte [rbx + 3], 'l'
    mov byte [rbx + 4], 'o'
    mov byte [rbx + 5], 0   ; Null terminator

    ; Now buffer contains "Hello\0"

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Example 4: Pointer Dereference

```
section .data
    value    dq 42          ; A 64-bit value
    ptr      dq value       ; ptr stores ADDRESS of value

section .text
    global _start

_start:
    ; Method 1: Direct access
    mov rax, [value]        ; RAX = 42

    ; Method 2: Through pointer
    mov rbx, [ptr]          ; RBX = address of value
    mov rax, [rbx]          ; RAX = 42 (dereference pointer)

    ; This is like C code:
    ; int value = 42;
    ; int *ptr = &value;
    ; int x = *ptr;        // x = 42

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Size Specifications

Sometimes you need to tell NASM the size explicitly:

```
mov rax, [rbx]              ; Size inferred from RAX (64-bit)
mov [rbx], rax              ; Size inferred from RAX (64-bit)

; But what about this?
mov [rbx], 100              ; ERROR: Size ambiguous!

; Solution: Specify size explicitly
mov byte [rbx], 100         ; Store 1 byte
mov word [rbx], 100         ; Store 2 bytes
mov dword [rbx], 100        ; Store 4 bytes
mov qword [rbx], 100        ; Store 8 bytes
```

Size keywords:

- **byte** - 8 bits (1 byte)
- **word** - 16 bits (2 bytes)

- `dword` - 32 bits (4 bytes)
- `qword` - 64 bits (8 bytes)

Common Patterns

String/Array Iteration

```
section .data
    str db "Hello", 0

section .text
    mov rsi, str          ; RSI = address of string
    xor rcx, rcx          ; RCX = index = 0

loop_start:
    mov al, [rsi + rcx]    ; Load character
    test al, al           ; Check if zero
    jz loop_end           ; Exit if null terminator

    ; Process character in AL...

    inc rcx               ; Next character
    jmp loop_start

loop_end:
    ; Done
```

Stack Frame Access

```
; Function with local variables
my_function:
    push rbp
    mov rbp, rsp
    sub rsp, 16           ; Allocate 16 bytes for locals

    ; Access local variables
    mov dword [rbp - 4], 10 ; local1 = 10
    mov dword [rbp - 8], 20 ; local2 = 20

    ; Access function parameters (passed on stack)
    mov rax, [rbp + 16]    ; First parameter
    mov rbx, [rbp + 24]    ; Second parameter

    leave
    ret
```

Memory Safety Warning

```

;   SAFE - valid address
mov rbx, buffer
mov al, [rbx]

;   DANGEROUS - uninitialized pointer
mov rbx, 0
mov al, [rbx]           ; SEGFAULT! Accessing address 0

;   DANGEROUS - random value
mov al, [rax]           ; SEGFAULT if RAX has garbage value

;   DANGEROUS - out of bounds
section .data
    arr db 1, 2, 3
section .text
    mov rbx, arr
    mov al, [rbx + 1000] ; Reading past end of array!

```

TL;DR - The [] Summary

SYNTAX	MEANING
mov rax, 5	RAX = 5
mov rax, [5]	RAX = memory[5]
mov rax, rbx	RAX = RBX
mov rax, [rbx]	RAX = memory[RBX]
mov rax, [rbx + 8]	RAX = memory[RBX + 8]
mov rax, [rbx + rcx]	RAX = memory[RBX + RCX]
mov rax, [rbx+rcx*4]	RAX = memory[RBX + RCX*4]

KEY RULE:

[] = "Go to this address and access memory there"
 No [] = "Use this value directly"

Related Topics:

- [Topic 2: Registers & Data Types](#)
- Pointer arithmetic
- Array indexing

- C pointers vs assembly addressing

[← Back to Q&A Index](#)

CHAPTER 13

Push and Pop Explained

Question

What does *push* and *pop* do in assembly?

Answer

In assembly language, `push` and `pop` are fundamental instructions for working with the **stack**, which is a Last-In-First-Out (LIFO) data structure in memory.

PUSH Instruction

`push` adds data onto the top of the stack:

1. **Decrements** the stack pointer (ESP/RSP on x86/x64)
2. **Writes** the value to the memory location now pointed to by the stack pointer

```
push eax      ; Save eax register on stack
push 0x42     ; Push immediate value onto stack
```

What happens internally:

- Stack grows “downward” (toward lower memory addresses)
 - $ESP = ESP - 4$ (on 32-bit) or $RSP = RSP - 8$ (on 64-bit)
 - $Memory[ESP] = \text{value}$
-

POP Instruction

`pop` removes data from the top of the stack:

1. **Reads** the value from the memory location pointed to by the stack pointer
2. **Increments** the stack pointer

```
pop ebx      ; Restore value from stack into ebx
```

What happens internally:

- $\text{Value} = Memory[ESP]$

- $ESP = ESP + 4$ (on 32-bit) or $RSP = RSP + 8$ (on 64-bit)

Common Uses

1. Saving/Restoring Registers:

```
push eax      ; Save eax
push ebx      ; Save ebx
; ... do work that modifies eax and ebx ...
pop ebx       ; Restore ebx (LIFO order!)
pop eax       ; Restore eax
```

2. Function Calling Conventions:

```
push arg3     ; Push arguments in reverse order
push arg2
push arg1
call function ; Call function
add esp, 12    ; Clean up stack (3 args × 4 bytes)
```

3. Local Variables:

```
push ebp      ; Save old base pointer
mov ebp, esp  ; Set up new stack frame
sub esp, 16    ; Allocate 16 bytes for locals
```

Key Points

- **Order matters:** Pop must be in reverse order of push (LIFO)
- **Stack alignment:** Must keep stack balanced (every push needs a corresponding pop or stack adjustment)
- **Stack overflow:** Pushing too much can overflow the stack
- **Fast:** Stack operations are very fast (often single CPU cycle)

Bonus Question: Swapping EAX and EBX with Push/Pop

Question

How do I swap the values in `eax`, `ebx` with push & pop?

Answer

C Equivalent (conceptual stack):

```
#include <stdint.h>

// Simulating push/pop with an array
int32_t stack[100];
int32_t stack_pointer = 0;

void push(int32_t value) {
    stack[stack_pointer++] = value;
}

int32_t pop() {
    return stack[--stack_pointer];
}

int main() {
    int32_t eax = 5;
    int32_t ebx = 10;

    // Swap using push/pop
    push(eax);      // Stack: [5]
    push(ebx);      // Stack: [5, 10]
    eax = pop();    // eax = 10, Stack: [5]
    ebx = pop();    // ebx = 5, Stack: []

    // Result: eax = 10, ebx = 5 (swapped!)
    return 0;
}
```

Assembly: You can swap the values in `eax` and `ebx` using push and pop like this:

```
push eax          ; Stack: [eax]
push ebx          ; Stack: [ebx, eax]
pop  eax          ; eax now has ebx's value, Stack: [eax]
pop  ebx          ; ebx now has eax's value, Stack: []
```

How It Works (Step-by-Step)

Let's say `eax = 5` and `ebx = 10`:

1. `push eax` → Stack contains: `[5]`
2. `push ebx` → Stack contains: `[10, 5]` (10 on top)
3. `pop eax` → `eax = 10`, Stack contains: `[5]`
4. `pop ebx` → `ebx = 5`, Stack is empty

Result: **eax = 10, ebx = 5** ✓ Swapped!

Alternative Swap Methods

Using **xchg** (simpler):

```
xchg eax, ebx ; One instruction swap!
```

Using **XOR** trick (no extra memory):

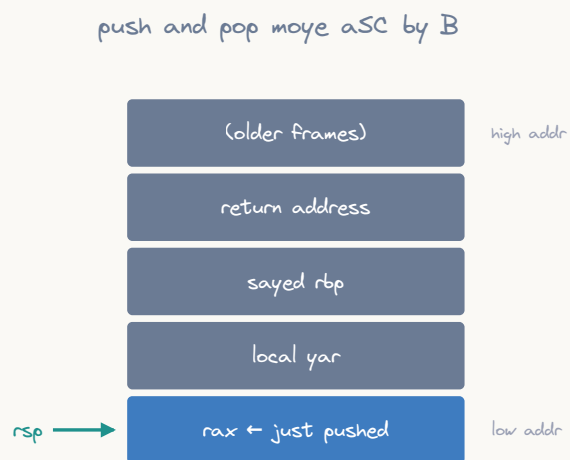
```
xor eax, ebx
xor ebx, eax
xor eax, ebx
```

Using a temporary register:

```
mov ecx, eax
mov eax, ebx
mov ebx, ecx
```

The **push/pop** method is useful when you need to understand stack operations, but **xchg** is the most practical for simply swapping two registers.

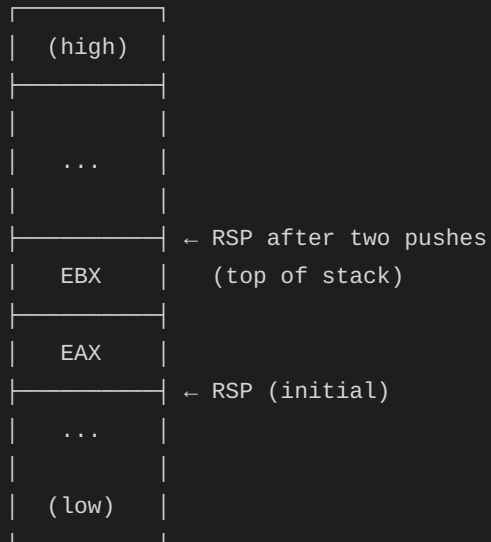
Visual Representation



push=rsb => B then store a pop=load then rsp (= B) a the stack grows downward

ASCII diagram

Stack Memory (grows downward):



Related Topics:

- Stack Frames
- Function Calling Conventions
- Stack Alignment

[← Back to Q&A Index](#)

CHAPTER 14

Sections (.text, .data, .bss) and XOR Optimization

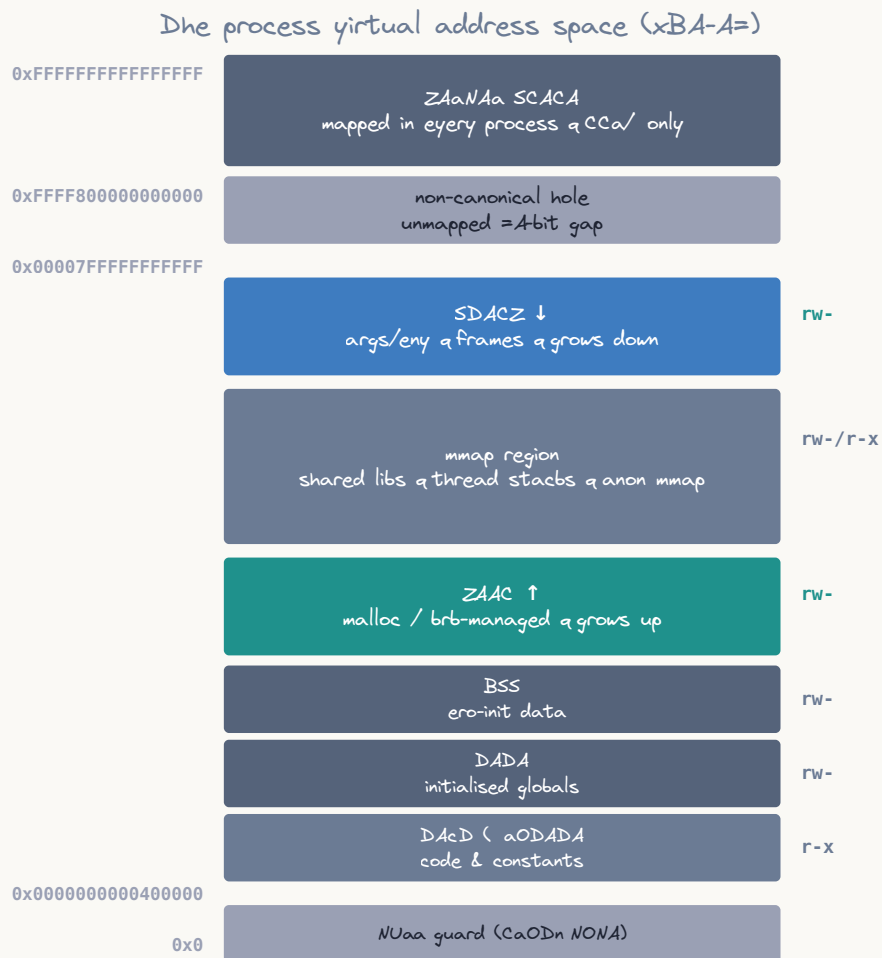
Questions

1. What is .text .bss .data sections with respect to C code?
 2. Why is `xor rax, rax` used instead of `mov rax, 0`?
-

Part 1: Understanding Sections

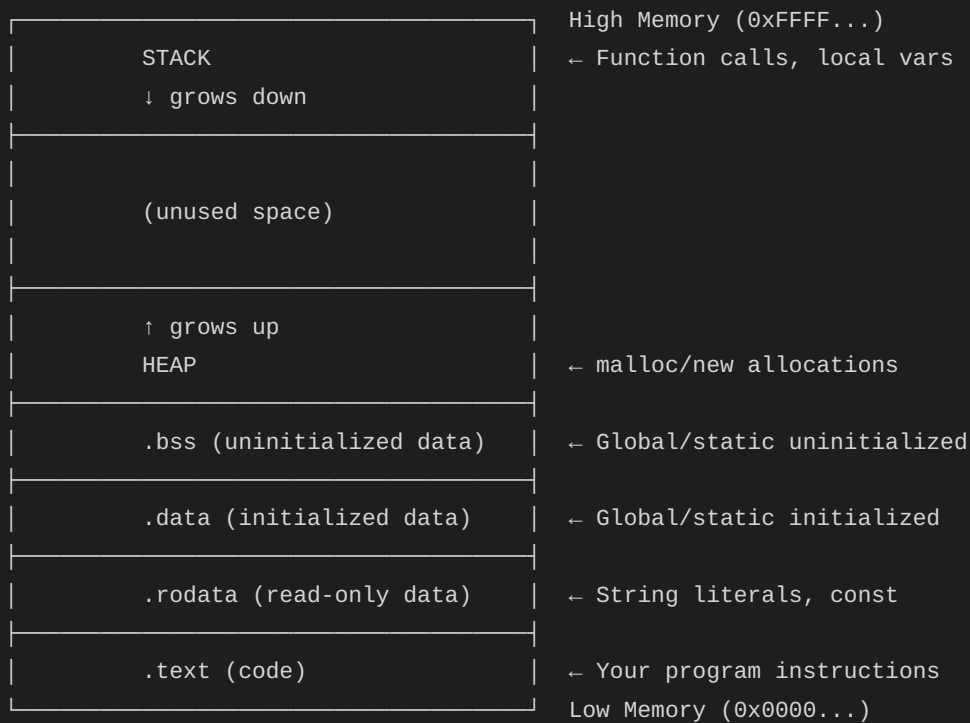
The Memory Layout

When your program runs, memory is organized into **segments** (sections):



virtual ≠ physical & memory is la y & the b b U maps pages to frames

ASCII diagram



Section Breakdown

1. `.text` Section - THE CODE

What it is: Your actual executable instructions (machine code) **Permissions:** Read + Execute (NOT writable for security)

C Code Example:

```
#include <stdio.h>

int add(int a, int b) {           // ← Function code goes in .text
    return a + b;
}

int main() {                     // ← Function code goes in .text
    int x = add(5, 10);
    printf("Result: %d\n", x);
    return 0;
}
```

Corresponding Assembly (.text section):

```

section .text
    global add
    global main

add:
    push rbp
    mov rbp, rsp
    mov eax, edi        ; a
    add eax, esi        ; + b
    pop rbp
    ret

main:
    push rbp
    mov rbp, rsp
    mov edi, 5
    mov esi, 10
    call add
    ; ... more code ...
    pop rbp
    ret

```

Key Points:

- All your functions live here
- Shared among all processes (read-only)
- Smallest possible size saves RAM

2. `.data` Section - INITIALIZED GLOBAL/STATIC DATA

What it is: Global and static variables with **initial values** **Permissions:** Read + Write **Storage:** Takes up space in executable file

C Code Example:

```
// Global variables with initial values → .data
int global_count = 100;           // .data (4 bytes)
char message[] = "Hello";        // .data (6 bytes with \0)
double pi = 3.14159;             // .data (8 bytes)

// Static variables with initial values → .data
static int file_counter = 0;      // .data

int main() {
    // Static local with initial value → .data
    static int func_calls = 1;    // .data

    global_count++;
    func_calls++;
    return 0;
}
```

Corresponding Assembly (.data section):

```
section .data
    global_count    dd 100           ; int = 100
    message         db "Hello", 0    ; char[] = "Hello\0"
    pi              dq 3.14159       ; double = 3.14159
    file_counter     dd 0             ; static int = 0
    func_calls       dd 1             ; static int = 1
```

Key Points:

- Variables initialized before `main()` runs
- Takes space in the executable file on disk
- Each process gets its own copy (writable)

3. `.bss` Section - UNINITIALIZED GLOBAL/STATIC DATA

What it is: Global and static variables **without** initial values (or initialized to zero) **Permissions:** Read + Write **Storage:** Does NOT take space in executable! (Clever optimization)

C Code Example:

```

// Global variables without initializers → .bss
int global_array[1000];           // .bss (4000 bytes)
char buffer[256];                // .bss (256 bytes)
double values[100];              // .bss (800 bytes)

// Explicitly initialized to zero → .bss (compiler optimization)
int zero_count = 0;              // .bss (not .data!)
static char null_term = '\0';    // .bss

// Static variables without initializers → .bss
static long uninitialized;        // .bss

int main() {
    // Static local without initializer → .bss
    static int cache[50];         // .bss

    // These are automatically zeroed:
    printf("%d\n", global_array[0]); // Prints 0
    printf("%d\n", zero_count);      // Prints 0

    return 0;
}

```

Corresponding Assembly (.bss section):

```

section .bss
    global_array    resd 1000        ; Reserve 4000 bytes (not in file!)
    buffer          resb 256         ; Reserve 256 bytes
    values          resq 100         ; Reserve 800 bytes
    zero_count      resd 1           ; Reserve 4 bytes
    null_term       resb 1           ; Reserve 1 byte
    uninitialized   resq 1           ; Reserve 8 bytes
    cache           resd 50          ; Reserve 200 bytes

```

Key Points:

- **Magic trick:** OS allocates and zeros this memory at runtime
- Saves executable file size (5000 bytes of zeros = 0 bytes in file!)
- Still gets memory - just not stored in the file
- All initialized to zero automatically

BSS = “Block Started by Symbol” (historical Unix term)

Side-by-Side Comparison

C Code:

```
// .data - initialized
int x = 42;
char msg[] = "Hi";

// .bss - uninitialized or zero
int y;
int z = 0;
char buf[100];

// .text - code
int add(int a, int b) {
    return a + b;
}

int main() {
    // .stack - local variables
    int local = 10;

    // .heap - dynamic allocation
    int *ptr = malloc(sizeof(int));
    *ptr = 20;
    free(ptr);

    return 0;
}
```

Assembly Equivalent:

```

section .data
    x    dd 42          ; Initialized data
    msg  db "Hi", 0

section .bss
    y    resd 1          ; Uninitialized
    z    resd 1          ; Zero (optimized to .bss)
    buf  resb 100        ; Buffer

section .text
    global add
    global main

add:
    lea  eax, [rdi + rsi] ; return a + b
    ret

main:
    push rbp
    mov  rbp, rsp
    sub  rsp, 16          ; Local var on stack

    mov  dword [rbp-4], 10 ; local = 10

    mov  rdi, 4            ; malloc(4)
    call malloc
    mov  qword [rbp-16], rax ; ptr = result

    mov  rbx, [rbp-16]
    mov  dword [rbx], 20   ; *ptr = 20

    mov  rdi, [rbp-16]
    call free              ; free(ptr)

    xor  eax, eax
    leave
    ret

```

Why This Matters

File Size Example:

```
// Option 1: Using .data (BAD for large arrays)
char buffer[1000000] = {0}; // 1 MB in executable file!

// Option 2: Using .bss (GOOD!)
char buffer[1000000];           // 0 bytes in executable file!
                                // OS allocates 1 MB at runtime
```

Verify with `size` command:

```
$ size my_program
text    data    bss     dec     hex filename
1234    1000      8    2242    8c2 program1 # data version
1234      8    1000    2242    8c2 program2 # bss version
```

Same runtime memory, but different file size!

Summary Table: Sections vs C Code

Section	Contains	In File?	Writable?
.text	Code (functions)	Yes	No
.data	Initialized globals	Yes	Yes
.bss	Uninitialized globals	No!	Yes
.rodata	const, string literals	Yes	No
stack	Local variables	No (runtime)	Yes
heap	malloc/new allocations	No (runtime)	Yes

Part 2: XOR Optimization - Why `xor rax, rax` Instead of `mov rax, 0`?

Quick Answer

Both set RAX to zero, but `xor rax, rax` is smaller and faster!

```
mov rax, 0      ; 7 bytes, slower
xor rax, rax    ; 3 bytes, faster
```

The Math

XOR truth table:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

← Same value XOR itself = 0

Applied to registers:

```
; If RAX = 0b10110101
; RAX XOR RAX:
 10110101
^ 10110101
-----
 00000000 ← Always zero!
```

Any value XORed with itself equals zero!

Detailed Comparison

1. Instruction Encoding Size

```
; 32-bit registers
mov eax, 0      ; 5 bytes: B8 00 00 00 00
xor eax, eax    ; 2 bytes: 31 C0

; 64-bit registers
mov rax, 0      ; 7 bytes: 48 C7 C0 00 00 00 00
xor rax, rax    ; 3 bytes: 48 31 C0

; Even shorter with 32-bit (zeros upper bits anyway)
xor eax, eax    ; 2 bytes: 31 C0
                ; RAX = 0x0000000000000000
```

Savings: 4-5 bytes per instruction!

2. Performance

Modern CPUs have special handling for `xor reg, reg`:

```
MOV RAX, 0:
- Needs execution unit
- Takes ~1 cycle
- Uses resources

XOR RAX, RAX:
- Recognized by CPU as "zeroing idiom"
- Often eliminated at decode stage (0 cycles!)
- Doesn't use execution port
- Breaks dependency chains
```

Zero-latency optimization:

- Modern CPUs (Intel, AMD) recognize `xor reg, reg`
- Handled by register renaming unit
- Doesn't even execute - just marks register as zero!

3. Dependency Breaking

```
; BAD - dependency chain
mov rax, [mem1]
add rax, rbx
mov rax, 0           ; Still depends on previous RAX
add rax, rcx         ; Waits for MOV

; GOOD - breaks dependency
mov rax, [mem1]
add rax, rbx
xor rax, rax         ; CPU knows: "RAX is zero, no dependency!"
add rax, rcx         ; Can start immediately
```

All Zeroing Methods Compared

```

; Method 1: MOV with immediate
mov rax, 0           ; 7 bytes, standard speed
mov eax, 0           ; 5 bytes, zeros upper 32 bits too

; Method 2: XOR (BEST)
xor rax, rax         ; 3 bytes, fastest
xor eax, eax         ; 2 bytes, fastest (zeros full RAX)

; Method 3: SUB
sub rax, rax         ; 3 bytes, fast (affects flags!)
sub eax, eax         ; 2 bytes, fast (affects flags!)

; Method 4: AND (weird but works)
and rax, 0           ; 7 bytes, slow
and eax, 0           ; 6 bytes, slow

; Method 5: IMUL (very weird!)
imul rax, 0          ; 4 bytes, slow (use XOR instead!)

```

Recommendation: Always use `xor reg, reg`

Practical Example

C Equivalent:

```

#include <stdint.h>

int main() {
    // In C, you just initialize variables to zero
    int32_t eax = 0;
    int32_t ebx = 0;
    int32_t ecx = 0;
    int32_t edx = 0;

    // The C compiler automatically uses XOR for optimization:
    // C code: int x = 0;
    // Compiles to: xor eax, eax

    return 0; // This also compiles to: xor eax, eax
}

```

Assembly:

```

section .text
    global _start

_start:
    ; Zeroing multiple registers efficiently
    xor eax, eax      ; 2 bytes
    xor ebx, ebx      ; 2 bytes
    xor ecx, ecx      ; 2 bytes
    xor edx, edx      ; 2 bytes
    ; Total: 8 bytes

    ; vs MOV version:
    mov eax, 0        ; 5 bytes
    mov ebx, 0        ; 5 bytes
    mov ecx, 0        ; 5 bytes
    mov edx, 0        ; 5 bytes
    ; Total: 20 bytes (2.5x larger!)

    ; Common pattern: clear return value
    xor eax, eax      ; return 0
    ret

    ; Exit syscall
    mov rax, 60
    xor rdi, rdi      ; exit code = 0 (FAST!)
    syscall

```

Real-World Usage

1. Function Return Zero

```

; Every C function that returns 0:
int main() {
    return 0;
}

; Compiles to:
main:
    push rbp
    mov rbp, rsp
    xor eax, eax      ; return 0
    pop rbp
    ret

```

2. Loop Counters

```
; Initialize loop counter
xor rcx, rcx           ; rcx = 0
loop_start:
    ; ... do work ...
    inc rcx
    cmp rcx, 10
    jl loop_start
```

3. Clearing Before Syscalls

```
; printf() with floating point
xor eax, eax           ; AL = 0 (no vector registers used)
mov rdi, format
mov rsi, 42
call printf
```

x

When XOR Can Be Confusing

```
; * WRONG - Thinking XOR works like MOV
mov rax, 5
xor rax, 3              ; RAX = 6 (NOT 3!)
                        ; 101 XOR 011 = 110

; Correct understanding
mov rax, 5
xor rax, rax            ; RAX = 0 (same value XORed)
```

Compiler Output Example

C Code:

```
int main() {
    int x = 0;
    int y = 0;
    return 0;
}
```

GCC Output (optimized):

```
$ gcc -O2 -S test.c -o test.s
```

```
main:
    xor    eax, eax    ; return 0
    ret
```

Even the locals `x` and `y` are optimized away! Only the return uses XOR.

Summary Table: Zeroing Methods

Instruction	Bytes	Speed	Flags	Recommendation
<code>xor eax, eax</code>	2	*****	Modified	BEST
<code>xor rax, rax</code>	3	*****	Modified	BEST
<code>mov eax, 0</code>	5	***☆☆	Unchanged	Larger
<code>mov rax, 0</code>	7	***☆☆	Unchanged	Larger
<code>sub eax, eax</code>	2	***☆☆	Modified	Use XOR

TL;DR

Sections:

- `.text` = Your code (C functions) - read-only, executable
- `.data` = Initialized variables - in file, writable
- `.bss` = Uninitialized/zero variables - NOT in file, writable (magic!)

XOR Trick:

- `xor rax, rax` is 2-5 bytes **smaller** than `mov rax, 0`
- **Faster** - CPU recognizes it as special “zeroing” operation
- **Standard practice** - all optimizing compilers use it
- **Always use it** for zeroing registers!

Related Topics:

- [Topic 1: Setup & First Program](#)
- [Topic 2: Registers & Data Types](#)

- Memory layout and program loading
- Compiler optimizations

[← Back to Q&A Index](#)

CHAPTER 15

Q&A: Signals, Traps, and Exception Handling

The Question

How do signals actually get delivered to my assembly program? What's the difference between a signal and an exception? How does SIGSEGV work? Can I catch a segfault and keep running? What about SIGINT (Ctrl+C)?

Quick Answer

Signals are the Unix mechanism for asynchronous notifications. Some come from hardware exceptions (SIGSEGV from page faults, SIGFPE from divide-by-zero), some from other processes (`kill`), and some from the kernel (SIGCHLD when a child exits). The kernel delivers signals by modifying your process's stack and registers when returning from kernel mode to user mode, effectively "injecting" a call to your signal handler.

Signal vs Exception vs Interrupt

Exception (CPU generates):

- └ #PF (Page Fault) → Kernel may handle internally (demand paging, COW)
- └ → OR convert to signal: SIGSEGV
- └ #DE (Divide Error) → Kernel converts to: SIGFPE
- └ #UD (Invalid Opcode) → Kernel converts to: SIGILL
- └ #BP (Breakpoint) → Kernel converts to: SIGTRAP
- └ #GP (General Prot.) → Kernel converts to: SIGSEGV

Interrupt (hardware generates):

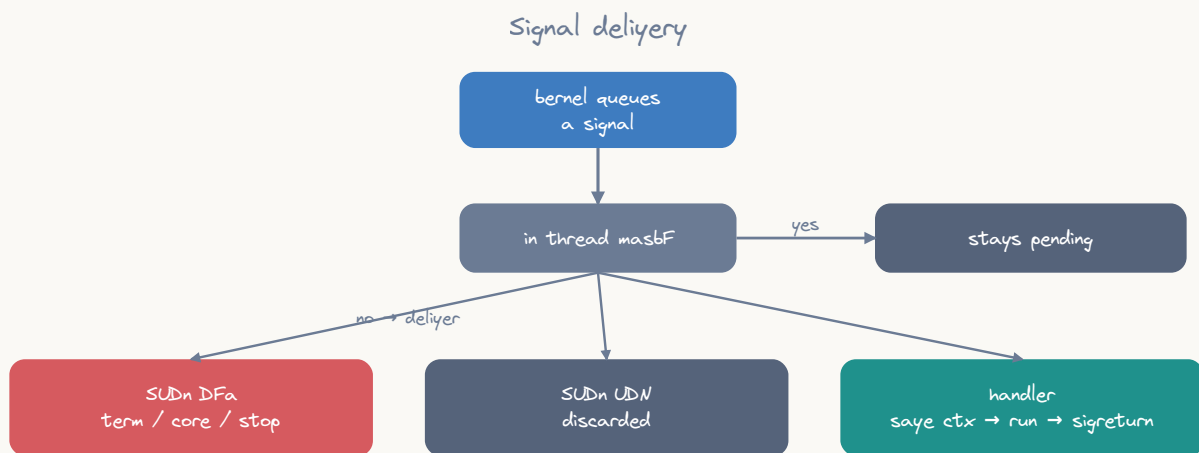
- └ Timer (IRQ 0) → Kernel handles internally (scheduling)
- └ Keyboard (IRQ 1) → Kernel feeds to TTY, may generate: SIGINT, SIGTSTP
- └ Disk (IRQ 14) → Kernel handles internally (I/O completion)

Signal (software mechanism):

- └ From exceptions → SIGSEGV, SIGFPE, SIGILL, SIGBUS, SIGTRAP
- └ From kill() → Any signal (SIGTERM, SIGUSR1, etc.)
- └ From kernel → SIGCHLD, SIGPIPE, SIGALRM
- └ From terminal → SIGINT (Ctrl+C), SIGTSTP (Ctrl+Z), SIGQUIT (Ctrl+\)

How Signal Delivery Works

The Delivery Mechanism



ASCII diagram

Your process is running in user mode:

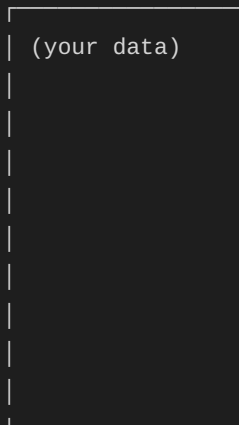
```
mov rax, [some_address]    ← executing normally
```

A signal arrives (e.g., another process called `kill(your_pid, SIGUSR1)`):

1. Signal is PENDING (marked in your `task_struct`)
 - Signals aren't delivered while in user mode!
 - They wait for the next transition through kernel
2. Next time you enter kernel (syscall, interrupt, exception):
 - Timer interrupt fires (or you call `read()`, etc.)
 - Kernel handles the event
3. On RETURN to user mode, kernel checks pending signals:
 - `do_signal()` → "hey, SIGUSR1 is pending!"
4. Kernel modifies your user-mode stack and registers:

Before:

Stack:



Registers:

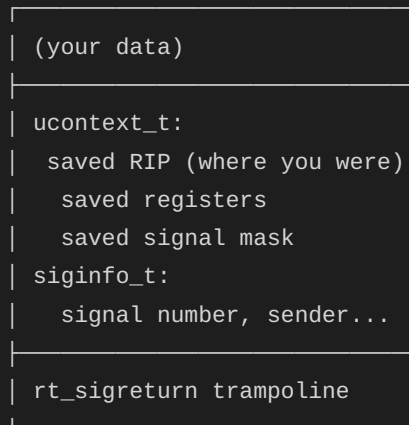
RIP = your_code_address

RSP = your_stack

RDI = (whatever)

After kernel manipulation:

Stack:



Registers:

RIP = signal_handler ← CHANGED!

RSP = (above the saved frame)

RDI = signal_number

5. SYSRET to user mode → you "magically" start executing signal handler!

6. Signal handler returns (`ret`):
 - Executes `rt_sigreturn` trampoline on stack
 - `sys_rt_sigreturn` syscall
 - Kernel restores original registers from `ucontext` on stack
 - Returns to your ORIGINAL code as if nothing happened

In Assembly Terms

```
; What the kernel effectively does to your process:

; Save current state onto your stack:
; sub rsp, sizeof(ucontext_t)
; mov [rsp + UC_RIP], original_rip
; mov [rsp + UC_RSP], original_rsp
; mov [rsp + UC_RAX], original_rax
; ... (all registers)
;
; Set up signal handler call:
; mov rdi, signal_number      ; First argument
; mov rsi, address_of_siginfo ; Second argument (if SA_SIGINFO)
; mov rdx, address_of_ucontext ; Third argument (if SA_SIGINFO)
; mov rip, handler_address    ; "Return" to handler instead of original code
;
; Place return trampoline:
; [stack]: mov rax, 15 ; sys_rt_sigreturn
;          syscall
```

Catching SIGSEGV (Segmentation Fault)

Why You Might Want To

Use cases for catching SIGSEGV:

1. Implement your own virtual memory system (JIT, GC, memory-mapped DB)
2. Graceful error reporting before crash
3. Implement stack growth (like the kernel does)
4. Guard page detection (array bounds checking without branch overhead!)
5. NaN-boxing and pointer tagging in language runtimes

The Tricky Part: Resuming After a Fault

```

; If your handler just returns, the CPU will re-execute the faulting instruction
; → Infinite loop of: fault → handle → fault → handle → ...
;
; Solutions:
; 1. Modify the saved RIP in ucontext to skip past faulting instruction
; 2. Use siglongjmp to jump to a different code path
; 3. Fix the memory (mmap the page) so the instruction succeeds on retry

; Option 1: Skip the instruction (advance RIP)
my_segv_handler:
    ; RDX = ucontext_t pointer (third argument with SA_SIGINFO)
    ; The RIP is at a known offset in the mcontext within ucontext
    ; On Linux x86-64: offset to RIP in ucontext is 0xA8

    add qword [rdx + 0xA8], 7    ; Skip 7-byte instruction (approximate!)
    ret                        ; "Return" → rt_sigreturn → resume at new RIP

; Option 3: Fix the fault (map the page)
my_segv_handler_fix:
    ; RSI = siginfo_t pointer
    ; siginfo_t.si_addr (offset 16) = the faulting address
    mov rdi, [rsi + 16]        ; Faulting address
    and rdi, ~0xFFF           ; Round down to page boundary

    ; Map the page
    mov rax, 9                 ; sys_mmap
    ; RDI already = page-aligned address
    mov rsi, 4096              ; 1 page
    mov rdx, 3                 ; PROT_READ | PROT_WRITE
    mov r10, 50                ; MAP_PRIVATE | MAP_ANONYMOUS | MAP_FIXED (0x32)
    mov r8, -1
    xor r9, r9
    syscall

    ret                        ; Return → instruction retried → page now exists!

```

Signal Lifecycle

1. Signal GENERATED:
 - `kill(pid, sig)`
 - Kernel generates on exception
 - `raise(sig) = kill(getpid(), sig)`
2. Signal PENDING:
 - Stored in `task_struct` bitmask
 - One bit per signal type (1-64)
 - Standard signals: can't queue (multiple SIGINTs = one pending)
 - Real-time signals (32-64): can queue
3. Signal BLOCKED? (check process signal mask)
 - If blocked: stays pending until unblocked
 - Use `sigprocmask` to block/unblock
4. Signal DELIVERED (on return to user mode):
 - If handler installed: call handler (stack modification)
 - If `SIG_IGN`: discard
 - If `SIG_DFL`: default action (terminate, core dump, stop, ignore)
5. After handler returns:
 - `rt_sigreturn` restores original state
 - Process continues where it was interrupted

Common Signals Quick Reference

Signal	Number	Default	Generated By
SIGKILL	9	terminate	Cannot be caught or ignored!
SIGSTOP	19	stop	Cannot be caught or ignored!
SIGINT	2	terminate	Ctrl+C (terminal)
SIGTERM	15	terminate	kill command (polite request)
SIGSEGV	11	core dump	Invalid memory access
SIGFPE	8	core dump	Arithmetic error (div by 0)
SIGILL	4	core dump	Illegal instruction
SIGBUS	7	core dump	Bus error (alignment, bad mmap)
SIGTRAP	5	core dump	Breakpoint (INT 3, ptrace)
SIGCHLD	17	ignore	Child process state change
SIGPIPE	13	terminate	Write to broken pipe/socket
SIGALRM	14	terminate	<code>alarm()</code> timer expired
SIGUSR1	10	terminate	User-defined signal 1
SIGUSR2	12	terminate	User-defined signal 2
SIGTSTP	20	stop	Ctrl+Z (terminal)
SIGCONT	18	continue	Resume stopped process

Signal Handling Setup in Assembly

```

; Installing a signal handler requires rt_sigaction:
; struct sigaction {
;     void (*sa_handler)(int);           offset 0
;     unsigned long sa_flags;           offset 8
;     void (*sa_restorer)(void);        offset 16
;     sigset_t sa_mask[1];              offset 24 (128 bytes on x86-64)
; };

; sa_flags important values:
; SA_SIGINFO (4):      Handler gets siginfo_t and ucontext_t
; SA_RESTART (0x10000000): Restart interrupted syscalls
; SA_RESTORER (0x04000000): sa_restorer field is set (required on x86-64!)
; SA_NODEFER (0x40000000): Don't block signal during handler

section .data
    align 8
    sigaction_struct:
        dq handler_func           ; sa_handler
        dq 0x04000004             ; sa_flags = SA_RESTORER | SA_SIGINFO
        dq sig_restore            ; sa_restorer
        times 16 db 0             ; sa_mask (128 bytes, all zeros = don't block)

section .text
; The restorer – kernel requires this for proper signal return:
sig_restore:
    mov rax, 15                   ; sys_rt_sigreturn
    syscall
    ; Never returns – kernel restores original context

; Install handler:
install_handler:
    ; RDI = signal number to handle
    mov rax, 13                   ; sys_rt_sigaction
    ; RDI = signum (already set)
    lea rsi, [rel sigaction_struct] ; act
    xor rdx, rdx                  ; oldact = NULL
    mov r10, 8                    ; sigsetsize (sizeof(sigset_t))
    syscall
    ret

```

The Async-Signal-Safety Problem

Inside a signal handler, you're limited to "async-signal-safe" functions!

Why? A signal can interrupt your code ANYWHERE:

- In the middle of `malloc()` → calling `malloc` in handler = deadlock/corruption!
- In the middle of `printf()` → calling `printf` in handler = buffer corruption!
- While holding a lock → acquiring same lock in handler = deadlock!

Safe in signal handlers:

- ✓ `write()` (syscall – no user-space state)
- ✓ `_exit()`
- ✓ `read()`
- ✓ Simple variable assignment (to volatile `sig_atomic_t`)
- ✓ All syscalls (they're atomic from user-space perspective)

UNSAFE in signal handlers:

- ✗ `malloc()` / `free()`
- ✗ `printf()` / `puts()`
- ✗ Any function that uses locks internally
- ✗ `exit()` (calls `atexit` handlers which may use unsafe functions)

TL;DR

Question	Answer
How are signals delivered?	Kernel modifies your stack/RIP before returning to user mode
When can signals arrive?	Only when transitioning from kernel → user mode
What's SIGSEGV?	Page fault on invalid address → kernel sends SIGSEGV
Can I catch SIGSEGV?	Yes! Install handler with <code>rt_sigaction</code>
Can I resume after SIGSEGV?	Yes: modify RIP in <code>ucontext</code> , or <code>mmap</code> the page
What can't be caught?	SIGKILL (9) and SIGSTOP (19) — by kernel design
Is my handler safe?	Only if you use async-signal-safe functions (mostly syscalls)
What's the restorer?	Trampoline code that calls <code>sys_rt_sigreturn</code> to restore context
Signals vs interrupts?	Signals = software (process-level); Interrupts = hardware (CPU-level)

CHAPTER 16

Q&A: Virtual Memory Explained

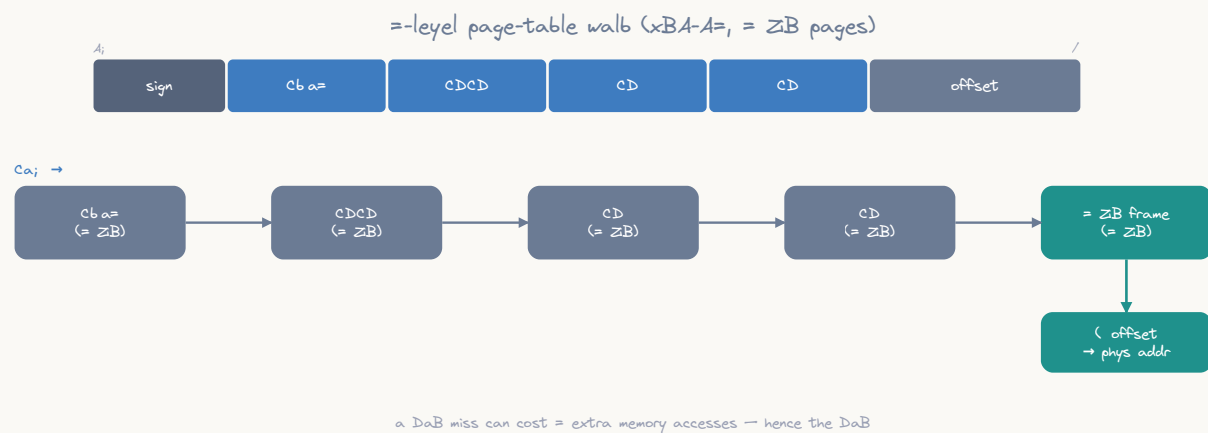
The Question

Every pointer I use in assembly is a “virtual address.” What does that mean? How does the CPU translate it to a real physical location? Why can two processes use the same address without conflict? And what’s a page table?

Quick Answer

Virtual memory is an illusion: every process thinks it has 256TB of contiguous memory starting at address 0. In reality, the CPU’s MMU (Memory Management Unit) translates every virtual address to a physical address using a 4-level page table structure. Different processes have different page tables, so the same virtual address maps to different physical locations. Pages that haven’t been accessed don’t use any physical memory at all.

The Translation (One Slide Version)



ASCII diagram

Your instruction: `mov rax, [0x7FFF7A2C123]`

CPU does (transparently, ~1ns with TLB hit):

Virtual Address: `0x00007FFF7A2C123`

PML4	PDPT	PD	PT	Offset
255	507	418	44	0x123

```

      |
      |
CR3---|
[PML4 table]
entry 255---|
[PDPT table]
entry 507---|
[PD table]
entry 418---|
[Page Table]
entry 44 → Physical Page 0x1F3A0
Physical Address: 0x1F3A0000 + 0x123 = 0x1F3A0123

```

Physical RAM byte at `0x1F3A0123`

Why Virtual Memory Exists

Without Virtual Memory (1970s)

Physical RAM: 64KB total

OS: 0x0000-0x3FFF
Program A: 0x4000-0x7FFF
Program B: 0x8000-0xBFFF
Free: 0xC000-0xFFFF

Problems:

1. Programs must be compiled for specific addresses
2. Program A can read/write Program B's memory (no protection!)
3. Can't run program that needs more memory than physically available
4. Can't run program whose address range overlaps another
5. Memory fragmentation: 20KB free but not contiguous → can't use it

With Virtual Memory (Modern)

Each process sees:

0x00000000000000 - 0x7FFFFFFFFFFFFF	256 TB for me alone!
	(Even if only 16GB RAM exists)
My code at 0x400000	
My heap growing from 0x600000	
My stack at 0x7FFFFFFFE000	

Process A and Process B can BOTH use address 0x400000

- Different page tables → different physical pages
- Complete isolation, no possible interference

Page Tables: The 4-Level Structure

Why 4 Levels?

48-bit virtual address → 256 TB address space

If we used a flat table: $2^{48} / 4096 = 2^{36}$ entries $\times$ 8 bytes = 512 GB

- Table would be LARGER than most RAM!

Solution: hierarchical (only populate entries actually used)

Typical process uses:

- ~100 MB of mapped memory → needs only ~25,000 page table entries
- Rather than 68 billion entries in a flat table!

Each level is one 4KB page with 512 entries ($2^9 = 512$, $\times$ 8 bytes = 4096)

4 levels $\times$ 9 bits = 36 bits for indexing + 12 bits for page offset = 48 bits

TLB: Why It's Not Slow

```
Full page table walk: 4 memory accesses (one per level)
→ ~400ns total (4 × ~100ns memory access)

TLB (Translation Lookaside Buffer): cache of recent translations
→ ~1ns for a TLB hit (99%+ hit rate for most programs)

TLB size: ~1500 entries
→ Covers 1500 × 4KB = ~6MB with 4KB pages
→ Covers 1500 × 2MB = ~3GB with huge pages!

TLB miss penalty: triggers hardware page table walk
→ CPU has dedicated Page Miss Handler (PMH) hardware
→ Walks page tables from CR3 automatically
→ Fills TLB entry, retries access transparently
```

What a Page Fault Really Is

```
Access to virtual address 0x12345678:

Case 1: Valid mapping, page present
→ TLB hit (fast) or TLB miss + page walk → access succeeds
→ No fault, no kernel involvement

Case 2: Valid mapping, page NOT present (demand paging)
→ Page fault exception (#PF)
→ Kernel: "this address is valid (in VMA), just not loaded yet"
→ Allocate physical page, zero it, install PTE, resume
→ This is a MINOR page fault (no disk I/O)

Case 3: Valid mapping, page on disk (swapped out)
→ Page fault exception (#PF)
→ Kernel: "this page was swapped to disk"
→ Read from swap partition, install PTE, resume
→ This is a MAJOR page fault (disk I/O, ~10ms)

Case 4: Invalid address (no VMA covers it)
→ Page fault exception (#PF)
→ Kernel: "this address doesn't belong to this process"
→ Deliver SIGSEGV → program crashes (Segmentation Fault!)

Case 5: Permission violation (write to read-only)
→ Could be COW: kernel copies page, makes writable
→ Could be actual violation: SIGSEGV
```

Observing Virtual Memory

/proc/self/maps

```
$ cat /proc/self/maps
ADDRESS_RANGE      PERMS  OFFSET  DEV    INODE  PATH
00400000-00401000   r-xp   00000000 08:01 131072 /usr/bin/cat
00601000-00602000   rw-p   00001000 08:01 131072 /usr/bin/cat
00602000-00623000   rw-p   00000000 00:00 0      [heap]
7f1234560000-7f1234720000 r-xp 00000000 08:01 262144 /lib/x86_64-linux-gnu/libc-2.31.so
7ffff7ffd000-7ffff7fff000 r-xp 00000000 00:00 0      [vdso]
7ffff7ffde000-7ffff7fff0000 rw-p 00000000 00:00 0      [stack]
```

Columns:

Address range: virtual address start-end

Perms: r(read) w(write) x(execute) p(private)/s(shared)

Offset: offset into mapped file

Dev: device (major:minor)

Inode: file inode number

Path: mapped file or special region name

In Assembly

```
; Read your own memory map:
mov rax, 2          ; sys_open
lea rdi, [rel .path]
xor rsi, rsi
syscall             ; Open /proc/self/maps
; Read and print contents...

section .rodata
.path: db "/proc/self/maps", 0
```

Key Insights

1. Virtual Memory is “Free” Until Touched

```
mmap(NULL, 1GB, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
→ Returns immediately
→ Uses ZERO physical memory
→ Only allocates page table entries (pointing to nothing)
→ Physical pages allocated ONE AT A TIME on first write (page fault)
```

2. Two Processes, Same Virtual Address, Different Data

Process A: page table entry for 0x400000 → physical page 0x1A000

Process B: page table entry for 0x400000 → physical page 0x2B000

Same virtual address, completely different physical pages.

When CPU switches between A and B, it loads different CR3 → different page tables.

3. Shared Libraries: One Physical Copy, Many Virtual Mappings

libc.so loaded at:

Process A: virtual 0x7f1234560000 → physical pages 0x100000-0x1C0000

Process B: virtual 0x7f9876540000 → physical pages 0x100000-0x1C0000

Process C: virtual 0x7f5555500000 → physical pages 0x100000-0x1C0000

Different virtual addresses, SAME physical pages!

Only ONE copy in RAM serves all processes.

TL;DR

Question	Answer
What's a virtual address?	An illusion — NOT a real RAM location
Who translates it?	MMU hardware (page table walk or TLB cache)
How fast?	~1ns with TLB hit; ~400ns on TLB miss
Why 4 levels?	Sparse coverage — only allocate tables for used regions
What's a page?	4KB unit of virtual-to-physical mapping
What's a page fault?	Address valid but page not in RAM → kernel handles it
What's SIGSEGV?	Page fault on address that ISN'T in your VMA list
What's CR3?	CPU register holding physical address of top-level page table
What happens on context switch?	New CR3 loaded → completely different address space
Can I see my own page tables?	No (kernel only), but /proc/self/maps shows mappings

CHAPTER 17

gen_figures.py

```
#!/usr/bin/env python3
"""Generate the guide's SVG figures, tuned to the htmler blue theme.

The kit's grey/purple house style is re-hued to htmler's blue-forward palette.
Because the figures are inlined as static base64 images (no page CSS reaches
them), every colour is chosen to work on BOTH the dark (#0b0d12) and light
(ffffff) themes at once. The trick: a mid-slate around luminance ~0.2 gives
roughly 4.3:1 contrast three ways – white text sitting on the fill, and the
same colour used as ink on either background.

* slate blue   #6B7B94   (neutral boxes, connectors, axes, labels)
* blue         #3E7CC0   (highlighted / "after" boxes)           + dark #2F5F98
* teal         #1F918C   (positive "result" accent)
* amber        #D9922B   (warning / spill; dark text on fill)
* red          #D65A5F   (problem callouts)
* muted        #9AA0B4   (captions)
* white        #FFFFFF   (text inside dark fills)
* 1.5pt wide rules, Aptos / system sans font stack

Run:  python3 scripts/gen_figures.py
Output: <chapter>/figures/*.svg
"""

import base64
import io
import os
import re

# — House-style constants (htmler blue theme, dual light/dark legible) —
GREY = "#6B7B94"
GREY_D = "#55637A"
PURPLE = "#3E7CC0"
PURPLE_D = "#2F5F98"
TEAL = "#1F918C"
AMBER = "#D9922B"
RED = "#D65A5F"
WHITE = "#FFFFFF"
LIGHT = "#9AA0B4"
INK_DARK = "#1F2433" # text on light (amber) fills
# Hand-drawn Excalidraw look: Virgil is embedded per-figure (see _font_face);
# 'Segoe Print'/cursive are only fallbacks if the embed ever fails.
FONT = "'Virgil','Segoe Print','Comic Sans MS',cursive"
RULE = 1.5 # pt wide rules

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
FONT_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                          "fonts", "Virgil.woff2")

_FACE_CACHE = {}

def _font_face(text):
```

```

"""Return a <style> block embedding a Virgil subset for `text`.

The figures are inlined as base64 <img> data URIs, and browsers do not
fetch external fonts for <img>-loaded SVGs – so the hand-drawn font must
travel *inside* each SVG. We subset to the glyphs actually used to keep
each figure tiny (~8-14 KB)."""
# Subset to exactly the glyphs this figure uses (plus a space) so each
# embedded font stays as small as possible.
key = "".join(sorted(set(text) | {" "}))
if key in _FACE_CACHE:
    return _FACE_CACHE[key]
try:
    from fontTools import subset as _subset
    opts = _subset.Options()
    opts.flavor = "woff2"
    opts.desubroutinize = True
    opts.ignore_missing_unICODES = True
    font = _subset.load_font(FONT_PATH, opts)
    ss = _subset.Subsetter(options=opts)
    ss.populate(text=key)
    ss.subset(font)
    buf = io.BytesIO()
    font.save(buf)
    b64 = base64.b64encode(buf.getvalue()).decode()
    face = ("<style>@font-face{font-family:'Virgil';font-style:normal;"
            "font-weight:400;src:url(data:font/woff2;base64," + b64 +
            ") format('woff2');}</style>")
except Exception as exc: # pragma: no cover - fonttools optional
    print(" ! font embed skipped:", exc)
    face = ""
_FACE_CACHE[key] = face
return face

# — Primitive builders —————
def esc(s):
    return (s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;"))

def defs():
    """Arrowhead markers in each ink colour."""
    marks = []
    for name, col in (("g", GREY), ("p", PURPLE), ("t", TEAL),
                      ("r", RED), ("a", AMBER), ("l", LIGHT)):
        marks.append(
            f'<marker id="c17-ah-{name}" viewBox="0 0 10 10" refX="8.5" refY="5" '
            f'markerWidth="4.5" markerHeight="4.5" orient="auto-start-reverse">'
            f'<path d="M0 0L10 5L0 10z" fill="{col}" /></marker>')
    return "<defs>" + "".join(marks) + "</defs>"

```



```

def rrect(x, y, w, h, fill, rx=9, stroke=None, sw=RULE, dash=None, opacity=None):
    s = (f'<rect x="{x}" y="{y}" width="{w}" height="{h}" rx="{rx}" ry="{rx}" '
        f'fill="{fill}"')
    if stroke:
        s += f' stroke="{stroke}" stroke-width="{sw}"'
    if dash:
        s += f' stroke-dasharray="{dash}"'
    if opacity is not None:
        s += f' opacity="{opacity}"'
    return s + ">"

def tspan_lines(x, cy, lines, fill, size, weight, lh):
    """Vertically centred multiline <text>."""
    n = len(lines)
    y0 = cy - (n - 1) * lh / 2.0
    out = [f'<text x="{x}" y="{y0}" fill="{fill}" font-family="{FONT}" '
        f'font-size="{size}" font-weight="{weight}" text-anchor="middle" '
        f'dominant-baseline="central">']
    for i, ln in enumerate(lines):
        dy = 0 if i == 0 else lh
        out.append(f'<tspan x="{x}" dy="{dy}">{esc(ln)}</tspan>')
    out.append("</text>")
    return "".join(out)

def box(x, y, w, h, lines, fill=GREY, tcol=WHITE, size=13, weight=600,
        rx=9, lh=16, stroke=None, sw=RULE, dash=None):
    if isinstance(lines, str):
        lines = [lines]
    r = rrect(x, y, w, h, fill, rx=rx, stroke=stroke, sw=sw, dash=dash)
    t = tspan_lines(x + w / 2.0, y + h / 2.0, lines, tcol, size, weight, lh)
    return r + t

def obox(x, y, w, h, lines, stroke=GREY, tcol=GREY, size=13, weight=600,
        rx=9, lh=16, sw=RULE, dash=None, fill="none"):
    """Outlined box (transparent fill) with coloured text."""
    r = rrect(x, y, w, h, fill, rx=rx, stroke=stroke, sw=sw, dash=dash)
    t = tspan_lines(x + w / 2.0, y + h / 2.0, lines if isinstance(lines, list)
        else [lines], tcol, size, weight, lh)
    return r + t

def text(x, y, s, fill=GREY, size=13, weight=600, anchor="middle",
        italic=False, mono=False):
    fam = ('SFMono-Regular', ui-monospace, 'JetBrains Mono', Consolas, monospace"
        if mono else FONT)
    st = "" # italics disabled: the hand-drawn font is hard to read slanted
    return (f'<text x="{x}" y="{y}" fill="{fill}" font-family="{fam}" '
        f'font-size="{size}" font-weight="{weight}" text-anchor="{anchor}"')

```

```

        f'{st} dominant-baseline="central">{esc(s)}</text>')

def line(x1, y1, x2, y2, col=GREY, sw=RULE, dash=None):
    d = f' stroke-dasharray="{dash}"' if dash else ""
    return (f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" stroke="{col}" '
            f'stroke-width="{sw}"{d}/>')

def _mk(col):
    return {GREY: "g", PURPLE: "p", TEAL: "t", RED: "r", AMBER: "a",
            LIGHT: "l"}.get(col, "g")

def arrow(x1, y1, x2, y2, col=GREY, sw=RULE, dash=None):
    d = f' stroke-dasharray="{dash}"' if dash else ""
    return (f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" stroke="{col}" '
            f'stroke-width="{sw}" marker-end="url(#ah-{_mk(col)})" {d}/>')

def path(d, col=GREY, sw=RULE, dash=None, arrow_end=False, fill="none"):
    dd = f' stroke-dasharray="{dash}"' if dash else ""
    m = f' marker-end="url(#ah-{_mk(col)})"' if arrow_end else ""
    return (f'<path d="{d}" fill="{fill}" stroke="{col}" stroke-width="{sw}" '
            f'{dd}{m}/>')

def cylinder(x, y, w, h, fill=GREY, tcol=WHITE, lines=None, size=12,
             stroke=None, sw=RULE):
    """Database / memory cylinder."""
    ry = min(h * 0.16, 14)
    st = (f' stroke="{stroke}" stroke-width="{sw}"') if stroke else ""
    body = (f'<path d="M{x} {y+ry} A{w/2} {ry} 0 0 0 {x+w} {y+ry} '
            f'L{x+w} {y+h-ry} A{w/2} {ry} 0 0 1 {x} {y+h-ry} Z" '
            f'fill="{fill}"{st}/>')
    top = (f'<ellipse cx="{x+w/2}" cy="{y+ry}" rx="{w/2}" ry="{ry}" '
            f'fill="{fill}"{st}/>')
    lip = (f'<path d="M{x} {y+ry} A{w/2} {ry} 0 0 0 {x+w} {y+ry}" '
            f'fill="none" stroke="{WHITE}" stroke-width="1" opacity="0.35"/>')
    t = ""
    if lines:
        t = tspan_lines(x + w / 2.0, y + h / 2.0 + ry / 2, lines, tcol, size,
                        600, 15)
    return body + top + lip + t

def svg(w, h, body, title=""):
    t = f"<title>{esc(title)}</title>" if title else ""
    used = "".join(re.findall(r'>([^\<]*)<', body)) + title
    face = _font_face(used)
    return (f'<?xml version="1.0" encoding="UTF-8"?>\n'
            f'<svg width="{w}" height="{h}">{t}{body}</svg>')

```

```

f'<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 {w} {h}" '
f'width="{w}" height="{h}" font-family="{FONT}">{face}{t}{defs()}'
f'{body}</svg>\n')

def write(rel_path, content):
    full = os.path.join(REPO_ROOT, rel_path)
    os.makedirs(os.path.dirname(full), exist_ok=True)
    with open(full, "w") as f:
        f.write(content)
    print("wrote", rel_path, f"({len(content)} bytes)")

# — Before/after "code card" primitives —
MONO = ('SFMono-Regular', ui-monospace, 'JetBrains Mono', Menlo,
        "Consolas, monospace")
CARD_BG = "#232A35"           # self-contained dark code card (theme-independent)
CODE_FG = "#D7DCE6"
CODE_DIM = "#8892A5"
CODE_HI = "#7FC4FF"           # changed / highlighted line
CODE_GOOD = "#83CEA3"         # added
CODE_BAD = "#E98A90"          # removed
LBL_BEFORE = "#9AA0B4"
LBL_AFTER = "#7FC4FF"
PAD = 14
LH = 19
CSIZE = 12.5
CHARW = 7.55
LABEL_AREA = 28
BOTTOM = 12
_STYLE_COL = {"n": CODE_FG, "hi": CODE_HI, "dim": CODE_DIM,
              "good": CODE_GOOD, "bad": CODE_BAD}

def _txt(ln):
    return ln[0] if isinstance(ln, tuple) else ln

def card_size(lines, label, minw=0):
    maxlen = max([len(_txt(l)) for l in lines] + [len(label) + 2])
    w = max(minw, PAD * 2 + int(round(maxlen * CHARW)))
    h = LABEL_AREA + len(lines) * LH + BOTTOM
    return w, h

def code_card(x, y, lines, label, border, labelcol, minw=0):
    w, h = card_size(lines, label, minw)
    out = [rrect(x, y, w, h, CARD_BG, rx=11, stroke=border, sw=1.75)]
    out.append(f'<text x="{x+PAD}" y="{y+15}" fill="{labelcol}" '
              f'font-family="{FONT}" font-size="10.5" font-weight="700" '
              f'letter-spacing="1.2" text-anchor="start" ')

```

```

        f'dominant-baseline="central">{esc(label)}</text>')
    cy = y + LABEL_AREA + LH / 2
    for ln in lines:
        txt, style = (ln if isinstance(ln, tuple) else (ln, "n"))
        out.append(
            f'<text x="{x+PAD}" y="{cy}" fill="{_STYLE_COL[style]}" '
            f'font-family="{MONO}" font-size="{CSIZE}" text-anchor="start" '
            f'dominant-baseline="central" '
            f'xml:space="preserve">{esc(txt)}</text>')
        cy += LH
    return "".join(out), w, h

def before_after(fname, title, before, after, op="", note_b="", note_a="",
                 blabel="BEFORE", alabel="AFTER", title2="", gap=104):
    wl, hl = card_size(before, blabel)
    wr, hr = card_size(after, alabel)
    top = 46 if not title2 else 62
    y0 = top
    maxh = max(hl, hr)
    xl = 24
    xr = xl + wl + gap
    W = xr + wr + 24
    note_h = 26 if (note_b or note_a) else 0
    H = top + maxh + note_h + 18
    b = [text(W / 2, 24, title, GREY, 15.5, 700)]
    if title2:
        b.append(text(W / 2, 44, title2, LIGHT, 11.5, 500, italic=True))
    cl, _, _ = code_card(xl, y0, before, blabel, GREY_D, LBL_BEFORE)
    cr, _, _ = code_card(xr, y0, after, alabel, PURPLE, LBL_AFTER)
    b.append(cl)
    b.append(cr)
    ay = y0 + maxh / 2
    b.append(arrow(xl + wl + 16, ay, xr - 12, ay, PURPLE, 2.0))
    if op:
        b.append(text((xl + wl + xr) / 2, ay - 13, op, PURPLE, 11, 700))
    if note_b:
        b.append(text(xl + wl / 2, y0 + maxh + 15, note_b, RED, 11, 600))
    if note_a:
        b.append(text(xr + wr / 2, y0 + maxh + 15, note_a, TEAL, 11, 600))
    write(fname, svg(W, H, "".join(b), title))

def rules_fig(fname, title, pairs, note="", lhs_hdr="", rhs_hdr=""):
    """A card of lhs → rhs rewrite rules (monospace)."""
    lw = max(len(l) for l, _ in pairs)
    rw = max(len(r) for _, r in pairs)
    x0, y0 = 24, 46
    lx = x0 + PAD
    arrow_x1 = lx + int(lw * CHARW) + 12
    arrow_x2 = arrow_x1 + 30

```

```

rx = arrow_x2 + 12
cardw = (rx + int(rw * CHARW) + PAD) - x0
rows = len(pairs)
hdr_h = 20 if (lhs_hdr or rhs_hdr) else 0
cardh = LABEL_AREA + hdr_h + rows * LH + BOTTOM
W = x0 + cardw + 24
H = y0 + cardh + (24 if note else 12)
b = [text(W / 2, 24, title, GREY, 15.5, 700)]
b.append(rrect(x0, y0, cardw, cardh, CARD_BG, rx=11, stroke=GREY_D,
               sw=1.75))
cy = y0 + LABEL_AREA + hdr_h + LH / 2
if hdr_h:
    b.append(text(lx, y0 + 16, lhs_hdr, LBL_BEFORE, 10.5, 700,
                  anchor="start"))
    b.append(text(rx, y0 + 16, rhs_hdr, LBL_AFTER, 10.5, 700,
                  anchor="start"))
for l, r in pairs:
    b.append(f'<text x="{lx}" y="{cy}" fill="{CODE_FG}" '
             f'font-family="{MONO}" font-size="{CSIZE}" text-anchor="start" '
             f'dominant-baseline="central" '
             f'xml:space="preserve">{esc(l)}</text>')
    b.append(arrow(arrow_x1, cy, arrow_x2, cy, PURPLE, 1.8))
    b.append(f'<text x="{rx}" y="{cy}" fill="{CODE_GOOD}" '
             f'font-family="{MONO}" font-size="{CSIZE}" text-anchor="start" '
             f'dominant-baseline="central" '
             f'xml:space="preserve">{esc(r)}</text>')
    cy += LH
if note:
    b.append(text(W / 2, y0 + cardh + 13, note, LIGHT, 11, 500,
                  italic=True))
write(fname, svg(W, H, "".join(b), title))

# — memory-domain helpers —————
def seg(x, y, w, h, label, color, sub=None, tcol=WHITE, size=12.5, rx=3,
        stroke=None):
    lines = [label] if sub is None else [label, sub]
    return box(x, y, w, h, lines, color, tcol=tcol, size=size, rx=rx, lh=15,
              stroke=stroke)

def addr(x, y, s):
    return text(x, y, s, LIGHT, 10.5, 600, anchor="end", mono=True)

def perm(x, y, s, col=TEAL):
    return text(x, y, s, col, 11, 700, anchor="start", mono=True)

```

```
# — root: the virtual address-space map (signature hero) —————
def fig_address_space():
    W, H = 780, 740
    bx, bw = 250, 330
    b = [text(W / 2, 28, "The process virtual address space (x86-64)",
              GREY, 16, 700)]
    rows = [
        ("KERNEL SPACE", "mapped in every process \u00b7 CPL0 only", GREY_D, 66,
         "0xFFFFFFFFFFFFFFFF", "---"),
        ("non-canonical hole", "unmapped 47-bit gap", LIGHT, 40,
         "0xFFFF800000000000", ""),
    ]
    y = 46
    for label, sub, col, h, a, pr in rows:
        b.append(seg(bx, y, bw, h, label, col, sub=sub,
                    tcol=(INK_DARK if col == LIGHT else WHITE), size=12))
        b.append(addr(bx - 12, y + 12, a))
        y += h + 6
    b.append(addr(bx - 12, y + 8, "0x00007FFFFFFFFF"))
    y += 14
    # stack (grows down)
    b.append(seg(bx, y, bw, 58, "STACK \u2193", PURPLE,
                sub="args/env \u00b7 frames \u00b7 grows down", size=12))
    b.append(perm(bx + bw + 12, y + 22, "rw-", TEAL))
    y += 58 + 8
    b.append(seg(bx, y, bw, 92, "mmap region", GREY,
                sub="shared libs \u00b7 thread stacks \u00b7 anon mmap", size=12))
    b.append(perm(bx + bw + 12, y + 30, "rw-/r-x", GREY))
    y += 92 + 8
    b.append(seg(bx, y, bw, 58, "HEAP \u2191", TEAL,
                sub="malloc / brk-managed \u00b7 grows up", size=12))
    b.append(perm(bx + bw + 12, y + 22, "rw-", TEAL))
    y += 58 + 6
    for label, sub, col, pr in [
        ("BSS", "zero-init data", GREY_D, "rw-"),
        ("DATA", "initialised globals", GREY_D, "rw-"),
        ("TEXT + RODATA", "code & constants", GREY, "r-x")]:
        b.append(seg(bx, y, bw, 40, label, col, sub=sub, size=11))
        b.append(perm(bx + bw + 12, y + 20, pr, GREY))
        y += 40 + 4
    b.append(addr(bx - 12, y + 4, "0x0000000000400000"))
    y += 10
    b.append(seg(bx, y, bw, 34, "NULL guard (PROT_NONE)", LIGHT,
                tcol=INK_DARK, size=11))
    b.append(addr(bx - 12, y + 24, "0x0"))
    b.append(text(W / 2, H - 12,
                "virtual \u2260 physical \u00b7 memory is lazy \u00b7 the MMU maps "
                "pages to frames", LIGHT, 11, 500))
    write("figures/address-space.svg", svg(W, H, "".join(b),
                                           "Virtual address space"))
```

```

def fig_roadmap():
    W, H = 940, 560
    b = [text(W / 2, 28, "A reading path through the guide", GREY, 16, 700)]
    bw, bh = 150, 46

    def chip(x, y, num, ttl, col=GREY, w=bw):
        b.append(box(x, y, w, bh, [f"{num} {ttl}"], col, size=11.5, rx=8))

    cols = [40, 240, 440, 640, 780]
    chip(cols[1], 60, "00", "fundamentals", PURPLE)
    chip(cols[1], 130, "02", "address space")
    chip(cols[0], 200, "03", "stack")
    chip(cols[2], 200, "01", "virtual memory")
    chip(cols[0], 270, "04", "heap / malloc")
    chip(cols[1], 270, "05", "mmap")
    chip(cols[2], 270, "06", "brk / sbrk")
    chip(cols[1], 340, "08", "process syscalls")
    chip(cols[0], 410, "09", "threads")
    chip(cols[2], 410, "10", "IPC & shm")
    chip(cols[1], 410, "11", "protection")
    chip(cols[1], 480, "07", "paging / swap")
    chip(cols[0], 480 - 0, "13", "advanced", GREY_D)
    chip(cols[2], 480, "12", "debugging", GREY_D)
    chip(cols[3] + 40, 340, "14", "allocators", PURPLE, w=170)
    # a few guiding arrows
    def a(x1, y1, x2, y2, dash=None):
        b.append(arrow(x1, y1, x2, y2, GREY, 1.6, dash=dash))
    a(cols[1] + bw / 2, 106, cols[1] + bw / 2, 130)
    a(cols[1] + bw / 2, 176, cols[1] + bw / 2, 200 - 0, dash="4 4")
    a(cols[1] + 20, 176, cols[0] + bw / 2, 200)
    a(cols[1] + bw - 20, 176, cols[2] + bw / 2, 200)
    a(cols[0] + bw / 2, 246, cols[0] + bw / 2, 270)
    a(cols[2] + bw / 2, 246, cols[2] + bw / 2, 270)
    a(cols[1] + bw / 2, 316, cols[1] + bw / 2, 340)
    a(cols[1] + bw / 2, 386, cols[1] + bw / 2, 410)
    a(cols[1] + bw / 2, 456, cols[1] + bw / 2, 480)
    b.append(arrow(cols[1] + bw, 363, cols[3] + 40, 363, PURPLE, 2))
    write("figures/roadmap.svg", svg(W, H, "".join(b), "Reading path"))

# — 00 fundamentals —————
def fig_hierarchy():
    W, H = 1060, 430
    b = [text(W / 2, 28, "The memory hierarchy \u2014 latency vs capacity",
        GREY, 16, 700)]
    rows = [
        ("registers", "~1 cycle", "< 1 KiB", PURPLE, 170),
        ("L1 cache", "~4 cycles / ~1 ns", "32 KiB / core", PURPLE, 250),
        ("L2 cache", "~12 cycles / ~3 ns", "256 KiB\u20131 MiB", GREY, 340),
        ("L3 cache", "~40 cycles / ~10 ns", "8\u201332 MiB", GREY, 430),
    ]

```

```

        ("DRAM", "~200 cycles / ~80 ns", "many GiB", GREY_D, 540),
        ("NVMe SSD", "~10 \u00b5s", "TiB", GREY_D, 620),
        ("spinning disk", "~10 ms", "TiB", LIGHT, 700),
    ]
    y = 52
    cx = 60
    for name, lat, cap, col, w in rows:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(box(cx, y, w, 40, [name], col, tcol=tc, size=12.5, rx=6))
        b.append(text(cx + w + 14, y + 20,
                      f"{lat} \u00b7 {cap}", GREY, 11, 600, anchor="start"))
        y += 50
    b.append(text(W / 2, H - 14,
                  "smaller & faster at the top \u2014 locality keeps you near "
                  "the top", LIGHT, 11, 500))
    write("figures/memory-hierarchy.svg",
          svg(W, H, "".join(b), "Memory hierarchy"))

def fig_cache_line():
    before_after(
        "figures/false-sharing.svg",
        "Cache line = 64 bytes (the unit of coherence)",
        [("core A writes a[0]", "n"), ("core B writes a[1]", "n"),
         ("a[0],a[1] same line", "bad")],
        [("pad to 64 B each", "good"), ("a[0] | line 0", "hi"),
         ("a[1] | line 1", "hi")], op="align",
        note_b="line ping-pongs (false sharing)",
        note_a="one line per core \u2014 no bouncing")

# — 01 virtual memory —————
def fig_page_walk():
    W, H = 1000, 460
    b = [text(W / 2, 28, "4-level page-table walk (x86-64, 4 KiB pages)",
              GREY, 16, 700)]
    # VA split bar
    bx, y = 90, 54
    parts = [("sign", 100, GREY_D), ("PML4", 120, PURPLE), ("PDPT", 120, PURPLE),
              ("PD", 120, PURPLE), ("PT", 120, PURPLE), ("offset", 160, GREY)]
    x = bx
    for name, w, col in parts:
        b.append(box(x, y, w, 40, [name], col, size=11.5, rx=4))
        x += w + 2
    b.append(text(bx, y - 8, "63", LIGHT, 9, 600, anchor="start"))
    b.append(text(x - 2, y - 8, "0", LIGHT, 9, 600, anchor="end"))
    # table chain
    ty = 150
    cx0, step, bw = 95, 195, 130
    names = ["PML4", "PDPT", "PD", "PT", "4 KiB frame"]
    for i, name in enumerate(names):

```



```

        cx = cx0 + i * step
        col = TEAL if i == 4 else GREY
        b.append(box(cx - bw / 2, ty, bw, 56, [name, "(4 KiB)"], col, size=11,
                    lh=14))

        if i > 0:
            b.append(arrow(cx - step + bw / 2, ty + 28, cx - bw / 2, ty + 28,
                          GREY, 1.8))

    fx = cx0 + 4 * step # frame column centre
    b.append(text(cx0 - bw / 2, ty - 12, "CR3 \u2192", PURPLE, 11, 700,
                anchor="start"))
    b.append(arrow(fx, ty + 56, fx, ty + 96, TEAL, 2))
    b.append(box(fx - bw / 2, ty + 96, bw, 46, ["+ offset", "\u2192 phys addr"],
              TEAL, size=11, lh=14))
    b.append(text(W / 2, ty + 180,
                "a TLB miss can cost 4 extra memory accesses \u2014 hence the TLB",
                LIGHT, 11, 500))
    write("figures/page-table-walk.svg",
        svg(W, H, "".join(b), "Page-table walk"))

def fig_tlb():
    W, H = 720, 320
    b = [text(W / 2, 28, "TLB \u2014 caching virtual\u2192physical translations",
              GREY, 15, 700)]
    b.append(box(60, 70, 150, 50, ["CPU: load VA"], GREY, size=12))
    b.append(box(285, 70, 150, 50, ["TLB lookup"], PURPLE, size=12))
    b.append(arrow(210, 95, 285, 95, GREY, 1.8))
    b.append(box(510, 60, 150, 44, ["hit \u2192 phys addr"], TEAL, size=12))
    b.append(arrow(435, 88, 510, 78, TEAL, 1.8))
    b.append(text(475, 66, "~0 cycles", TEAL, 10, 700))
    b.append(box(285, 165, 150, 50, ["page-table walk"], GREY, size=11))
    b.append(arrow(360, 120, 360, 165, RED, 1.8))
    b.append(text(372, 142, "miss", RED, 10, 700, anchor="start"))
    b.append(box(285, 250, 150, 44, ["install in TLB", "retry"], GREY_D,
              size=11, lh=14))
    b.append(arrow(360, 215, 360, 250, GREY, 1.8))
    b.append(path(f"M285 272 C180 272 180 95 285 95", GREY, 1.6,
                arrow_end=True, dash="4 4"))
    b.append(text(W / 2, H - 12,
                "flushed on CR3 reload / INVLPG / munmap / mprotect "
                "(global pages survive)", LIGHT, 10.5, 500))
    write("figures/tlb.svg", svg(W, H, "".join(b), "TLB"))

def fig_page_fault():
    W, H = 760, 430
    b = [text(W / 2, 28, "Page-fault handling (#PF \u2192 do_page_fault)",
              GREY, 15, 700)]
    b.append(box(300, 56, 160, 44, ["MMU walk \u2192 #PF"], PURPLE, size=12))

    def q(x, y, t, col=GREY, w=200):

```

```

        b.append(box(x, y, w, 40, [t], col, size=11))
    q(280, 120, "valid VMA?", GREY)
    b.append(arrow(380, 100, 380, 120, GREY, 1.8))
    b.append(box(540, 120, 150, 40, ["SIGSEGV"], RED, size=11))
    b.append(arrow(480, 140, 540, 140, RED, 1.6))
    b.append(text(512, 132, "no", RED, 10, 700))
    q(280, 190, "perms OK?", GREY)
    b.append(arrow(380, 160, 380, 190, GREY, 1.8))
    b.append(box(540, 190, 150, 40, ["SIGSEGV / SIGBUS"], RED, size=10))
    b.append(arrow(480, 210, 540, 210, RED, 1.6))
    b.append(text(512, 202, "no", RED, 10, 700))
    b.append(box(200, 270, 170, 56, ["minor fault", "alloc frame / zero page"],
        TEAL, size=11, lh=14))
    b.append(box(400, 270, 170, 56, ["major fault", "read swap / file"], AMBER,
        tcol=INK_DARK, size=11, lh=14))
    b.append(arrow(340, 230, 285, 270, GREY, 1.8))
    b.append(arrow(420, 230, 485, 270, GREY, 1.8))
    b.append(box(300, 356, 160, 40, ["fill PTE \u2192 restart"], GREY_D, size=11))
    b.append(arrow(285, 326, 360, 356, GREY, 1.6))
    b.append(arrow(485, 326, 400, 356, GREY, 1.6))
    write("figures/page-fault.svg",
        svg(W, H, "".join(b), "Page fault"))

def fig_zero_page():
    W, H = 820, 300
    b = [text(W / 2, 28, "Lazy allocation: the shared zero page + CoW",
        GREY, 15, 700)]
    xs = [40, 300, 560]
    steps = [
        ("1 mmap(anon)", ["VMA created", "no PTEs", "RSS += 0"], GREY),
        ("2 read p[0]", ["PTE \u2192 zero page", "read-only, shared", "RSS += 0"],
            PURPLE),
        ("3 write p[0]", ["CoW: new frame", "PTE now rw-", "RSS += 4 KiB"],
            TEAL),
    ]
    for x, (ttl, rows, col) in zip(xs, steps):
        b.append(box(x, 60, 220, 120, [ttl] + rows, col, size=12, lh=20,
            rx=10))
        if x != xs[-1]:
            b.append(arrow(x + 220, 120, x + 260, 120, GREY, 2))
    b.append(text(W / 2, H - 16,
        "malloc(1 GiB) succeeds on a small machine \u2014 frames attach "
        "only when touched", LIGHT, 11, 500))
    write("figures/zero-page.svg",
        svg(W, H, "".join(b), "Zero page"))

# — 02 process address space —————
def fig_elf_load():
    W, H = 780, 380

```

```

b = [text(W / 2, 28, "execve: ELF file \u2192 LOAD segments in memory",
        GREY, 15, 700)]

# ELF file
fx = 60
b.append(text(fx + 95, 58, "a.out on disk", GREY, 12, 700))
file_rows = [("ELF header", GREY_D), ("program headers", GREY_D),
             (".text (RX)", GREY), (".rodata (R)", GREY),
             (".data (RW)", PURPLE), (".bss (in headers only)", PURPLE_D)]

y = 72
for name, col in file_rows:
    b.append(seg(fx, y, 190, 40, name, col, size=11))
    y += 42

# arrows to VA
vx = 480
b.append(text(vx + 95, 58, "process VA", GREY, 12, 700))
va_rows = [("stack", PURPLE, "rw-"), ("mmap: libc, ld.so", GREY, "r-x"),
           ("heap", TEAL, "rw-"), (".bss (zero-filled)", PURPLE_D, "rw-"),
           (".data", PURPLE, "rw-"), (".text + .rodata", GREY, "r-x")]

y = 72
for name, col, pr in va_rows:
    b.append(seg(vx, y, 200, 40, name, col, size=11))
    b.append(perm(vx + 205, y + 20, pr, GREY))
    y += 42

b.append(arrow(fx + 190, 92 + 42 * 2 + 20, vx, 72 + 42 * 5 + 20, GREY, 1.6))
b.append(text((fx + 190 + vx) / 2, 194, "kernel honours", GREY, 10, 600))
b.append(text((fx + 190 + vx) / 2, 208, "LOAD headers", GREY, 10, 600))
b.append(text(W / 2, H - 14,
             ".bss has memsz > filesz \u2014 the kernel zero-fills the extra",
             LIGHT, 11, 500))
write("figures/elf-load.svg",
      svg(W, H, "".join(b), "ELF load"))

# — 03 stack —————
def fig_stack_frame():
    W, H = 640, 460
    b = [text(W / 2, 28, "A single stack frame (System V x86-64 ABI)",
            GREY, 15, 700)]
    bx, bw = 180, 300
    rows = [
        ("caller args 7,8,9\u2026", "first 6 in registers", GREY_D, 46),
        ("return address", "pushed by call", PURPLE, 40),
        ("saved RBP of caller", "push %rbp \u2190 %rbp", GREY, 46),
        ("callee-saved regs", "rbx, r12-r15 if used", GREY_D, 40),
        ("local variables", "\u2026", GREY, 60),
        ("alignment padding", "16-byte aligned \u2190 %rsp", GREY_D, 40),
    ]
    y = 56
    for label, sub, col, h in rows:
        b.append(seg(bx, y, bw, h, label, col, sub=sub, size=12))
        y += h + 5

```

```

b.append(text(bx - 14, 56 + 12, "higher", LIGHT, 10, 600, anchor="end"))
b.append(text(bx - 14, y - 20, "lower", LIGHT, 10, 600, anchor="end"))
b.append(arrow(bx - 30, 80, bx - 30, y - 24, GREY, 1.6))
b.append(text(W / 2, H - 16,
              "stack grows downward; args in rdi,rsi,rdx,rcx,r8,r9; "
              "return in rax", LIGHT, 10.5, 500))
write("figures/stack-frame.svg",
      svg(W, H, "".join(b), "Stack frame"))

def fig_stack_canary():
    W, H = 720, 360
    b = [text(W / 2, 28, "Stack canary: overflow trips the guard before return",
              GREY, 15, 700)]

    def panel(x, title, canary):
        b.append(text(x + 130, 58, title, GREY, 12, 700))
        rows = [("return address", PURPLE), ("saved rbp", GREY_D)]
        if canary:
            rows.append(("CANARY (random)", AMBER))
            rows.append(("locals / buffers", GREY))
        y = 76
        for name, col in rows:
            tc = INK_DARK if col == AMBER else WHITE
            b.append(seg(x, y, 260, 44, name, col, tcol=tc, size=12))
            if canary and name.startswith("CANARY"):
                b.append(text(x + 260 + 10, y + 22, "checked", TEAL, 10, 700,
                              anchor="start"))
                b.append(text(x + 260 + 10, y + 34, "before ret", TEAL, 10, 700,
                              anchor="start"))
            y += 48
        panel(40, "-fno-stack-protector", False)
        panel(410, "-fstack-protector-strong", True)
        b.append(text(180, H - 16, "overflow silently overwrites ret",
                      RED, 10.5, 600))
        b.append(text(540, H - 16, "overflow hits canary \u2192 __stack_chk_fail",
                      TEAL, 10.5, 600))
        write("figures/stack-canary.svg",
              svg(W, H, "".join(b), "Stack canary"))

# — 04 heap / malloc —
def fig_malloc_paths():
    W, H = 820, 400
    b = [text(W / 2, 28, "malloc(N): which path?", GREY, 16, 700)]
    b.append(box(330, 54, 160, 44, ["malloc(N)"], PURPLE, size=13))
    b.append(box(560, 130, 220, 60, ["mmap() region", "one VMA per alloc",
                                       "munmap on free"], GREY, size=11, lh=15))
    b.append(arrow(430, 98, 620, 130, GREY, 1.8))
    b.append(text(560, 112, "N \u2265 128 KiB", GREY, 10, 700))
    b.append(box(40, 130, 240, 60, ["tcache (per-thread)",

```

```

        "64 LIFO bins, lock-free"], TEAL, size=11,
        lh=15))
b.append(arrow(360, 98, 160, 130, GREY, 1.8))
b.append(text(200, 112, "small & cached", TEAL, 10, 700))
b.append(box(300, 210, 240, 150,
    ["arena (brk-grown)", "", "fastbins", "small / large bins",
    "unsorted bin", "top chunk (wilderness)"], GREY_D, size=11,
    lh=22))
b.append(arrow(380, 98, 400, 210, GREY, 1.8))
b.append(text(470, 150, "else lock arena", GREY_D, 10, 700))
b.append(arrow(420, 360, 420, 384, GREY, 1.6))
b.append(text(420, 392, "sbrk / mmap when the top chunk is too small",
    LIGHT, 10.5, 500))
write("figures/malloc-paths.svg",
    svg(W, H, "".join(b), "malloc paths"))

def fig_chunk():
    W, H = 720, 360
    b = [text(W / 2, 28, "A glibc chunk: in-use vs free", GREY, 15, 700)]

    def panel(x, title, rows, col_hi):
        b.append(text(x + 130, 58, title, GREY, 12, 700))
        y = 76
        for name, col in rows:
            b.append(seg(x, y, 260, 40, name, col, size=11.5))
            y += 44
    panel(40, "allocated", [
        ("prev_size", GREY_D), ("size | A M P", PURPLE),
        ("payload \u2190 malloc returns here", TEAL),
        ("\u2026 padding to 16B", GREY)], TEAL)
    panel(420, "free (in a bin)", [
        ("prev_size", GREY_D), ("size | A 0 P", PURPLE),
        ("fd (next free)", AMBER), ("bk (prev free)", AMBER)], AMBER)
    b.append(text(W / 2, H - 14,
        "16 B of metadata per allocation; free chunks store fd/bk "
        "inside the old payload", LIGHT, 10.5, 500))
    write("figures/chunk.svg", svg(W, H, "".join(b), "Chunk"))

def fig_bins():
    rules_fig(
        "figures/bins.svg",
        "Freelist bins by chunk size",
        [("16..1032 B", "tcache (per-thread LIFO)",
            ("16..88 B", "fastbin (LIFO, no coalesce)",
            ("16..512 B", "smallbin (exact size)",
            ("\u2265 1024 B", "largebin (size-ordered)",
            ("just freed", "unsorted bin"),
            ("top of heap", "top chunk / wilderness)]",
        note="free(p) routes the chunk to a bin by size",

```

```

    lhs_hdr="size", rhs_hdr="destination")

def fig_brk_vs_mmap():
    W, H = 760, 320
    b = [text(W / 2, 28, "Why ptmalloc uses both brk and mmap", GREY, 15, 700)]
    # brk contiguous
    b.append(text(170, 60, "brk heap (one VMA)", GREY, 12, 700))
    rows = [("in-use", GREY), ("free", LIGHT), ("in-use", GREY),
            ("top chunk", TEAL)]
    y = 78
    for name, col in rows:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(seg(60, y, 220, 40, name, col, tcol=tc, size=11.5))
        y += 44
    b.append(text(170, y + 6, "shrinks only from the top", RED, 10, 600))
    # mmap separate
    b.append(text(560, 60, "mmap chunks (many VMAs)", GREY, 12, 700))
    for i in range(3):
        b.append(seg(470, 78 + i * 60, 220, 44, f"alloc {i+1}", PURPLE,
                    size=11.5))
        b.append(text(700, 78 + i * 60 + 22, "\u2192 munmap", TEAL, 9.5, 600,
                    anchor="start"))
    b.append(text(560, y + 6, "each returnable on its own free", TEAL, 10, 600))
    write("figures/brk-vs-mmap.svg",
        svg(W, H, "".join(b), "brk vs mmap"))

# — 05 mmap —————
def fig_mmap_quadrants():
    W, H = 780, 420
    b = [text(W / 2, 28, "mmap: MAP_PRIVATE/SHARED \u00d7 anonymous/file",
        GREY, 15, 700)]
    cells = [
        (60, 70, "PRIVATE \u00b7 anon", ["zero pages, CoW",
            "malloc-style memory"], TEAL),
        (410, 70, "PRIVATE \u00b7 file", ["read file lazily",
            "writes stay local"], GREY),
        (60, 230, "SHARED \u00b7 anon", ["shared after fork",
            "(or memfd/shm)"], GREY),
        (410, 230, "SHARED \u00b7 file", ["writes go to the file",
            "shared page cache"], PURPLE),
    ]
    for x, y, ttl, rows, col in cells:
        b.append(box(x, y, 310, 130, [ttl, ""] + rows, col, size=12, lh=24,
            rx=10))
    write("figures/mmap-quadrants.svg",
        svg(W, H, "".join(b), "mmap quadrants"))

def fig_file_mapping():

```

```

W, H = 760, 340
b = [text(W / 2, 28, "File mapping: pages served from the page cache",
        GREY, 15, 700)]
b.append(text(W / 2, 60, "disk file img.bmp", GREY, 12, 700))
for i in range(3):
    b.append(seg(180 + i * 140, 74, 130, 40, f"page {i}", GREY_D, size=11))
b.append(text(W / 2, 200, "process virtual address space", GREY, 12, 700))
for i in range(3):
    x = 180 + i * 140
    b.append(seg(x, 214, 130, 40, f"p + {i}\u00d74K", PURPLE, size=11))
    b.append(arrow(x + 65, 114, x + 65, 214, GREY, 1.6, dash="4 4"))
b.append(text(W / 2, H - 14,
        "pages pulled in on demand (major fault, then cache hits are "
        "free)", LIGHT, 10.5, 500))
write("figures/file-mapping.svg",
    svg(W, H, "".join(b), "File mapping"))

def fig_mmap_lifecycle():
    W, H = 900, 240
    b = [text(W / 2, 28, "The lifecycle of an anonymous mapping", GREY, 15, 700)]
    steps = [
        ("mmap", "VMA, no PTEs", GREY),
        ("touch", "fault \u2192 frame", TEAL),
        ("madvise\nDONTNEED", "drop frames", AMBER),
        ("mprotect", "flip perms", PURPLE),
        ("munmap", "VMA gone", GREY_D),
    ]
    bw = 150
    xs = [30 + i * 172 for i in range(5)]
    for x, (ttl, sub, col) in zip(xs, steps):
        tc = INK_DARK if col == AMBER else WHITE
        b.append(box(x, 90, bw, 64, [ttl, sub], col, tcol=tc, size=12, lh=17))
        if x != xs[-1]:
            b.append(arrow(x + bw, 122, x + 172, 122, GREY, 1.8))
    write("figures/mmap-lifecycle.svg",
        svg(W, H, "".join(b), "mmap lifecycle"))

# — 06 brk —————
def fig_brk():
    W, H = 700, 340
    b = [text(W / 2, 28, "The program break (sbrk grows the heap up)",
        GREY, 15, 700)]

    def panel(x, title, brk_y):
        b.append(text(x + 130, 58, title, GREY, 12, 700))
        b.append(seg(x, 76, 260, 44, ".bss / .data", GREY_D, size=11))
        b.append(text(x - 10, 88, "start_brk", LIGHT, 9.5, 600, anchor="end"))
        b.append(seg(x, 122, 260, brk_y, "in-use heap", TEAL, size=11))
        b.append(text(x - 10, 122 + brk_y - 8, "brk(0)", LIGHT, 9.5, 600,

```

```

        anchor="end"))
    b.append(seg(x, 122 + brk_y + 4, 260, 150 - brk_y,
        "virtual hole (unused)", LIGHT, tcol=INK_DARK, size=11))
    panel(40, "before sbrk", 44)
    panel(400, "after sbrk(+page)", 96)
    b.append(arrow(310, 170, 400, 170, PURPLE, 2))
    b.append(text(355, 156, "sbrk", PURPLE, 10, 700))
    write("figures/brk.svg", svg(W, H, "".join(b), "Program break"))

# — 07 paging / swap —————
def fig_swap():
    W, H = 820, 260
    b = [text(W / 2, 28, "Anonymous page lifecycle \u2192 swap", GREY, 15, 700)]
    steps = [("created", "malloc + touch", TEAL),
        ("active", "recently used", PURPLE),
        ("inactive", "aged by LRU", GREY),
        ("swapped out", "written to disk", GREY_D)]
    bw = 165
    xs = [24 + i * 195 for i in range(4)]
    for x, (ttl, sub, col) in zip(xs, steps):
        b.append(box(x, 84, bw, 60, [ttl, sub], col, size=12, lh=17))
        if x != xs[-1]:
            b.append(arrow(x + bw, 114, x + 195, 114, GREY, 1.8))
    b.append(path(f"M{xs[3]+bw/2} 144 C{xs[3]+bw/2} 210 {xs[0]+bw/2} 210 "
        f"{xs[0]+bw/2} 144", AMBER, 2, arrow_end=True))
    b.append(text(W / 2, 200, "next access \u2192 major fault reads it back in",
        AMBER, 11, 700))
    b.append(text(W / 2, H - 12,
        "only anonymous pages swap; file pages are just dropped & "
        "re-read", LIGHT, 10.5, 500))
    write("figures/swap.svg", svg(W, H, "".join(b), "Swap"))

def fig_page_cache():
    W, H = 780, 240
    b = [text(W / 2, 28, "The page cache absorbs file writes", GREY, 15, 700)]
    steps = [("write(fd,buf,n)", "user call", GREY),
        ("copy \u2192 page cache", "marked dirty (RAM)", PURPLE),
        ("returns now", "no disk I/O yet", TEAL),
        ("writeback later", "kworker \u00b7 fsync forces", GREY_D)]
    bw = 168
    xs = [24 + i * 188 for i in range(4)]
    for x, (ttl, sub, col) in zip(xs, steps):
        b.append(box(x, 84, bw, 60, [ttl, sub], col, size=11.5, lh=17))
        if x != xs[-1]:
            b.append(arrow(x + bw, 114, x + 188, 114, GREY, 1.8))
    b.append(text(W / 2, H - 14,
        "look at MemAvailable, not MemFree \u2014 cache is reclaimable",
        LIGHT, 10.5, 500))
    write("figures/page-cache.svg",

```



```

        svg(W, H, "".join(b), "Page cache"))

def fig_buddy():
    W, H = 720, 340
    b = [text(W / 2, 28, "Buddy allocator: split & coalesce powers of two",
              GREY, 15, 700)]

    y = 60
    widths = [(1, 560), (2, 276), (4, 134)]
    labels = ["order 4 (16 pages)", "split \u2192 8 | 8", "split \u2192 4 | 4"]
    for i, ((n, w), lab) in enumerate(zip(widths, labels)):
        x = 80
        for j in range(n):
            col = TEAL if (i == 2 and j == 0) else (PURPLE if i > 0 else GREY)
            b.append(seg(x, y, w, 44, "" if n > 1 else "one 16-page block", col,
                        size=11))

            x += w + 8

        b.append(text(660, y + 22, lab, GREY, 10.5, 600, anchor="start")
                  if False else text(80, y - 8, lab, LIGHT, 10, 600,
                                      anchor="start"))

        y += 66
    b.append(text(140, y + 4, "return first 4 pages (order 2), mark used",
                  TEAL, 10.5, 600, anchor="start"))
    b.append(text(W / 2, H - 12,
                  "free coalesces buddies recursively back up the orders",
                  LIGHT, 10.5, 500))
    write("figures/buddy.svg", svg(W, H, "".join(b),
                                   "Buddy allocator"))

# — 08 process syscalls —————
def fig_process_lifecycle():
    W, H = 820, 240
    b = [text(W / 2, 28, "The life of a process", GREY, 16, 700)]
    steps = [("fork / clone", "newborn child", GREY),
              ("running", "R / S / D / T", PURPLE),
              ("zombie (Z)", "exit code held", AMBER),
              ("reaped", "parent wait()", TEAL)]

    bw = 168
    xs = [24 + i * 195 for i in range(4)]
    for x, (ttl, sub, col) in zip(xs, steps):
        tc = INK_DARK if col == AMBER else WHITE
        b.append(box(x, 88, bw, 62, [ttl, sub], col, tc=tc, size=12, lh=17))
        if x != xs[-1]:
            b.append(arrow(x + bw, 119, x + 195, 119, GREY, 1.8))
    b.append(text(xs[1] + bw / 2, 172, "execve replaces the image", GREY, 10,
                  600))
    b.append(text(W / 2, H - 14,
                  "an unreaped child is a zombie; an orphan is re-parented to "
                  "init (PID 1)", LIGHT, 10.5, 500))
    write("figures/process-lifecycle.svg",

```

```

        svg(W, H, "".join(b), "Process lifecycle"))

def fig_fork_cow():
    W, H = 820, 320
    b = [text(W / 2, 28, "fork(): copy-on-write shares frames until a write",
              GREY, 15, 700)]

    def frames(x, y, cols, tag):
        for i, c in enumerate(cols):
            b.append(seg(x + i * 62, y, 58, 40, f"F{i+1}", c, size=11))
            b.append(text(x - 10, y + 20, tag, GREY, 11, 700, anchor="end"))
        b.append(text(150, 62, "after fork (no copies)", GREY, 12, 700))
        frames(150, 80, [GREY, GREY, GREY], "parent")
        frames(150, 130, [GREY, GREY, GREY], "child")
        b.append(text(150 + 90, 186, "all PTEs read-only, refcount 2",
                      GREY, 10, 600))
        b.append(arrow(360, 140, 430, 140, PURPLE, 2))
        b.append(text(395, 126, "child writes F2", PURPLE, 10, 700))
        b.append(text(600, 62, "after the write", GREY, 12, 700))
        frames(560, 80, [GREY, GREY, GREY], "parent")
        b.append(seg(560, 130, 58, 40, "F1", GREY, size=11))
        b.append(seg(560 + 62, 130, 58, 40, "F2'", TEAL, size=11))
        b.append(seg(560 + 124, 130, 58, 40, "F3", GREY, size=11))
        b.append(text(560 - 10, 150, "child", GREY, 11, 700, anchor="end"))
        b.append(text(560 + 90, 186, "only F2 was copied", TEAL, 10, 600))
    write("figures/fork-cow.svg",
          svg(W, H, "".join(b), "fork copy-on-write"))

def fig_signal():
    W, H = 760, 300
    b = [text(W / 2, 28, "Signal delivery", GREY, 15, 700)]
    b.append(box(300, 54, 160, 44, ["kernel queues", "a signal"], PURPLE,
              size=11, lh=15))
    b.append(box(300, 130, 160, 40, ["in thread mask?"], GREY, size=11))
    b.append(arrow(380, 98, 380, 130, GREY, 1.8))
    b.append(box(540, 130, 180, 40, ["stays pending"], GREY_D, size=11))
    b.append(arrow(460, 150, 540, 150, GREY, 1.6))
    b.append(text(500, 142, "yes", GREY, 10, 700))
    outs = [("SIG_DFL", "term / core / stop", RED),
            ("SIG_IGN", "discarded", GREY_D),
            ("handler", "save ctx \u2192 run \u2192 sigreturn", TEAL)]
    for i, (ttl, sub, col) in enumerate(outs):
        x = 40 + i * 240
        b.append(box(x, 220, 210, 56, [ttl, sub], col, size=11, lh=15))
        b.append(arrow(380, 170, x + 105, 220, GREY, 1.4))
    b.append(text(300, 196, "no \u2192 deliver", GREY, 10, 700, anchor="end"))
    write("figures/signal-delivery.svg",
          svg(W, H, "".join(b), "Signal delivery"))

```

```

# — 09 threads —————
def fig_thread_memory():
    W, H = 780, 340
    b = [text(W / 2, 28, "Threads: one address space, private stacks + TLS",
              GREY, 15, 700)]
    b.append(rrect(40, 56, 700, 240, CARD_BG, rx=12, stroke=GREY_D, sw=1.75))
    b.append(text(W / 2, 84, "PROCESS (one PML4 / CR3)", LIGHT, 12, 700))
    b.append(box(70, 104, 640, 44,
                ["text \u00b7 rodata \u00b7 data \u00b7 bss \u00b7 heap \u00b7 mmap",
                 "\u00b7 fds "
                 "\u2014 SHARED"], TEAL, size=11.5))
    for i in range(3):
        x = 90 + i * 210
        b.append(box(x, 170, 180, 108,
                    [f"thread {i+1}", "", "stack", "registers / PC", "TLS"],
                    PURPLE, size=11, lh=19))
    b.append(text(W / 2, H - 14,
                  "kernel schedules tasks sharing a TGID; signal mask is "
                  "per-thread", LIGHT, 10.5, 500))
    write("figures/thread-memory.svg",
          svg(W, H, "".join(b), "Thread memory"))

def fig_thread_stack():
    W, H = 560, 420
    b = [text(W / 2, 28, "A non-main thread's stack VMA", GREY, 15, 700)]
    bx, bw = 170, 250
    rows = [("TCB (glibc)", GREY_D, 46), ("TLS data", PURPLE, 46),
            ("stack frames", GREY, 96), ("free / unused", LIGHT, 60),
            ("guard page (---p)", RED, 44)]
    y = 56
    for name, col, h in rows:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(seg(bx, y, bw, h, name, col, tcol=tc, size=12))
        y += h + 6
    b.append(text(bx - 14, 70, "high", LIGHT, 10, 600, anchor="end"))
    b.append(text(bx + bw / 2, 56 + 46 + 46 + 40, "grows down", WHITE, 10, 600))
    b.append(text(W / 2, H - 14,
                  "default 8 MiB mmap + 4 KiB PROT_NONE guard; touching the "
                  "guard \u2192 SIGSEGV", LIGHT, 10, 500))
    write("figures/thread-stack.svg",
          svg(W, H, "".join(b), "Thread stack"))

def fig_futex():
    W, H = 780, 300
    b = [text(W / 2, 28, "Futex: fast userspace path, kernel only on contention",
              GREY, 15, 700)]
    b.append(box(60, 90, 300, 90,
                ["FAST PATH (no syscall)", "atomic CAS on the futex word",

```

```

        "\u2192 1 : lock acquired"], TEAL, size=12, lh=22, rx=10))
b.append(box(420, 90, 300, 90,
            ["SLOW PATH (contended)", "FUTEX_WAIT: sleep if still locked",
             "FUTEX_WAKE: wake a waiter"], PURPLE, size=12, lh=22, rx=10))
b.append(arrow(360, 135, 420, 135, GREY, 2))
b.append(text(390, 122, "CAS fails", RED, 9.5, 700))
b.append(text(W / 2, H - 16,
            "word: 0=free, 1=locked, 2=locked+waiters \u00b7 kernel keys "
            "waiters by physical address", LIGHT, 10, 500))
write("figures/futex.svg", svg(W, H, "".join(b), "Futex"))

# — 10 IPC —————
def fig_shared_memory():
    W, H = 760, 320
    b = [text(W / 2, 28, "Shared memory: one object, two mappings", GREY, 15,
              700)]
    b.append(box(60, 90, 200, 70, ["process A", "mmap(MAP_SHARED)"], PURPLE,
                size=12, lh=18))
    b.append(box(500, 90, 200, 70, ["process B", "mmap(MAP_SHARED)"], PURPLE,
                size=12, lh=18))
    b.append(box(300, 200, 160, 70, ["shm object", "tmpfs / memfd", "frames"],
                TEAL, size=11, lh=17))
    b.append(arrow(180, 160, 340, 200, GREY, 1.8))
    b.append(arrow(580, 160, 420, 200, GREY, 1.8))
    b.append(text(W / 2, H - 14,
            "shm_open + ftruncate + mmap; or memfd_create passed over "
            "SCM_RIGHTS", LIGHT, 10.5, 500))
    write("figures/shared-memory.svg",
          svg(W, H, "".join(b), "Shared memory"))

# — 11 protection —————
def fig_defenses():
    W, H = 720, 430
    b = [text(W / 2, 28, "The user-space memory-protection stack", GREY, 15,
              700)]
    rows = [
        ("ASLR", "randomise the layout", PURPLE),
        ("NX / W^X", "data pages non-executable", GREY),
        ("stack canaries", "detect overflow before ret", GREY),
        ("RELRO (full)", "GOT/PLT read-only after load", GREY),
        ("Fortify Source", "checked memcpy/sprintf/\u2026", GREY_D),
        ("PIE", "executable base randomised too", PURPLE),
    ]
    y = 56
    for name, sub, col in rows:
        b.append(box(120, y, 480, 50, [f"{name} \u2014 {sub}"], col, size=12,
                    rx=8))
        y += 58
    b.append(text(W / 2, H - 14,

```

```

        "check with checksec --file=./a.out", LIGHT, 11, 500))
write("figures/defenses.svg",
      svg(W, H, "".join(b), "Protection stack"))

# — 13 advanced —————
def fig_numa():
    W, H = 780, 320
    b = [text(W / 2, 28, "NUMA: local DRAM is fast, remote costs ~2\u00d7",
              GREY, 15, 700)]
    for i, (x, node) in enumerate([(60, 0), (440, 1)]):
        b.append(box(x, 70, 280, 90,
                     [f"CPU socket {node}", f"cores {i*16}..{i*16+15}",
                      "L1 / L2 / L3"], PURPLE if i == 0 else GREY, size=12,
                      lh=20))
        b.append(box(x + 40, 200, 200, 60, [f"NUMA node {node}", "local DRAM"],
                     TEAL if i == 0 else GREY_D, size=12, lh=17))
        b.append(arrow(x + 140, 160, x + 140, 200, GREY, 1.8))
    b.append(path("M340 115 C390 115 390 115 440 115", GREY, 2))
    b.append(arrow(340, 115, 380, 115, GREY, 2))
    b.append(arrow(440, 115, 400, 115, GREY, 2))
    b.append(text(390, 100, "QPI / UPI", GREY, 10, 700))
    b.append(text(W / 2, H - 12,
                  "first-touch policy: a page lands on the node of the CPU that "
                  "first writes it", LIGHT, 10.5, 500))
write("figures/numa.svg", svg(W, H, "".join(b), "NUMA"))

def fig_huge_pages():
    W, H = 940, 280
    b = [text(W / 2, 28, "Huge pages: one TLB entry covers far more",
              GREY, 15, 700)]
    rows = [("4 KiB page", "walk 4 levels \u00b7 1 TLB entry / 4 KiB", GREY, 200),
            ("2 MiB page", "walk 3 levels \u00b7 512\u00d7 coverage", PURPLE, 360),
            ("1 GiB page", "walk 2 levels \u00b7 almost free TLB", TEAL, 560)]
    y = 66
    for name, sub, col, w in rows:
        b.append(box(90, y, w, 46, [name], col, size=12.5, rx=6))
        b.append(text(90 + w + 14, y + 23, sub, GREY, 10.5, 600,
                      anchor="start"))
        y += 60
    b.append(text(W / 2, H - 12,
                  "THP (automatic) or explicit MAP_HUGETLB \u2014 wins when the "
                  "working set is huge", LIGHT, 10.5, 500))
write("figures/huge-pages.svg",
      svg(W, H, "".join(b), "Huge pages"))

# — 14 allocators —————
def fig_allocators():
    rules_fig(

```

```

    "figures/allocators.svg",
    "Five allocators, five trade-offs",
    [("bump / arena", "O(1), no per-object free"),
     ("pool freelist", "fixed size, O(1) push/pop"),
     ("slab", "same-size sets, cache-aware"),
     ("freelist", "variable size, coalescing"),
     ("buddy", "power-of-2, OS page level")],
    note="real allocators = per-thread cache + size classes + page backing",
    lhs_hdr="allocator", rhs_hdr="what it is best at")

# — second pass: more diagrams —
def fig_guide_map():
    W, H = 900, 460
    b = [box(250, 46, 400, 56, ["ONE-STOP GUIDE \u00b7 LINUX MEMORY",
                                   "kernel + glibc + hardware"], PURPLE, size=13,
                                   lh=18)]
    row1 = [("CPU / MMU", "caches \u00b7 TLB"),
             ("virtual memory", "pages \u00b7 page tables"),
             ("process space", "/proc \u00b7 segments"),
             ("allocators", "malloc \u00b7 mmap \u00b7 arenas")]
    xs = [36, 262, 488, 714]
    for x, (t, s) in zip(xs, row1):
        b.append(box(x, 168, 172, 62, [t, s], GREY, size=11, lh=15))
        b.append(arrow(450, 102, x + 86, 168, GREY, 1.3))
    row2 = [("syscalls", "fork \u00b7 exec \u00b7 clone"),
             ("threads", "pthreads \u00b7 TLS \u00b7 futex"),
             ("IPC", "shm \u00b7 pipe \u00b7 socket")]
    xs2 = [110, 364, 618]
    for i, (x, (t, s)) in enumerate(zip(xs2, row2)):
        b.append(box(x, 320, 180, 62, [t, s], GREY_D, size=11, lh=15))
        b.append(arrow(xs[i] + 86, 230, x + 90, 320, GREY, 1.2, dash="4 4"))
    b.append(text(W / 2, H - 14,
                  "every topic is an elaboration of the address-space map",
                  LIGHT, 11, 500))
    write("figures/guide-map.svg", svg(W, H, "".join(b), "Guide map"))

def fig_how_to_use():
    W, H = 860, 240
    b = [text(W / 2, 28, "How to use this guide", GREY, 15, 700)]
    steps = [("Read", "the section README", GREY),
             ("Run", "edit the examples", PURPLE),
             ("Inspect", "/proc \u00b7 gdb \u00b7 strace \u00b7 perf", TEAL)]
    xs = [40, 320, 600]
    bw = 220
    for x, (t, s, c) in zip(xs, steps):
        b.append(box(x, 74, bw, 64, [t, s], c, size=12, lh=17))
        if x != 600:
            b.append(arrow(x + bw, 106, x + bw + 60, 106, GREY, 2))
    b.append(box(320, 176, 220, 44, ["sketch what happened"], GREY_D, size=11))

```

```

b.append(arrow(430, 138, 430, 176, GREY, 1.8))
write("figures/how-to-use.svg", svg(W, H, "".join(b), "How to use"))

def fig_malloc_syscalls():
    W, H = 820, 300
    b = [text(W / 2, 26, "Malloc \u2014 malloc is two syscalls in disguise",
              GREY, 15, 700)]
    b.append(box(40, 84, 190, 60, ["user", "malloc(40)"], GREY, size=12, lh=17))
    b.append(box(290, 84, 230, 60,
                ["glibc ptmalloc", "find a free chunk in a bin", "(no syscall)"],
                PURPLE, size=11, lh=16))
    b.append(arrow(230, 114, 290, 114, GREY, 2))
    b.append(text(560, 60, "no chunk fits \u2192 grow the heap", GREY, 10, 600))
    b.append(box(580, 78, 210, 46, ["small \u2192 brk(new_break)"], TEAL, size=11))
    b.append(box(580, 138, 210, 46, ["big \u2192 mmap(anon)"], TEAL, size=11))
    b.append(arrow(520, 104, 580, 98, GREY, 1.6))
    b.append(arrow(520, 124, 580, 158, GREY, 1.6))
    b.append(text(W / 2, H - 16,
                "small requests are served from the cached arena with no "
                "syscall at all", LIGHT, 10.5, 500))
    write("figures/malloc-syscalls.svg", svg(W, H, "".join(b), "malloc syscalls"))

def fig_virtual_physical():
    W, H = 800, 360
    b = [text(W / 2, 26, "Virtual \u2260 physical: same address, different frames",
              GREY, 15, 700)]
    b.append(box(40, 84, 180, 50, ["process A", "0x400000"], PURPLE, size=11,
                lh=15))
    b.append(box(40, 170, 180, 50, ["process A", "0x401000"], PURPLE, size=11,
                lh=15))
    b.append(box(580, 84, 180, 50, ["process B", "0x400000"], GREY, size=11,
                lh=15))
    b.append(box(580, 170, 180, 50, ["process B", "0x401000"], GREY, size=11,
                lh=15))
    b.append(box(330, 74, 140, 40, ["frame F1"], TEAL, size=11))
    b.append(box(330, 148, 140, 40, ["frame F2"], AMBER, tcol=INK_DARK,
                size=11))
    b.append(box(330, 224, 140, 40, ["frame F3"], GREY_D, size=11))
    b.append(arrow(220, 109, 330, 96, GREY, 1.5))
    b.append(arrow(220, 195, 330, 172, GREY, 1.5))
    b.append(arrow(580, 109, 470, 244, GREY, 1.5))
    b.append(arrow(580, 195, 470, 172, GREY, 1.5))
    b.append(text(400, 138, "shared frame", AMBER, 9.5, 700))
    b.append(text(W / 2, H - 14,
                "per-process page tables map identical virtual addresses to "
                "distinct (or shared) frames", LIGHT, 10, 500))
    write("figures/virtual-physical.svg",
          svg(W, H, "".join(b), "Virtual vs physical"))

```

```

def fig_alignment():
    W, H = 820, 320
    b = [text(W / 2, 26, "Alignment & endianness", GREY, 15, 700)]
    b.append(text(200, 66, "aligned load @ 0x1008", TEAL, 11, 700))
    b.append(box(60, 82, 280, 44, ["one 8-byte access"], TEAL, size=11))
    b.append(text(610, 66, "unaligned @ 0x1009", RED, 11, 700))
    b.append(box(475, 82, 155, 44, ["7 bytes\u2026"], GREY, size=11))
    b.append(box(632, 82, 155, 44, ["\u2026 1 byte"], GREY, size=11))
    b.append(text(610, 142, "straddles two lines \u2192 two loads", RED, 10, 600))
    b.append(text(W / 2, 202, "little-endian: uint32 0x11223344", GREY, 12, 700))
    for i, bv in enumerate(["44", "33", "22", "11"]):
        b.append(box(282 + i * 66, 218, 60, 40, [bv],
                    PURPLE if i == 0 else GREY, size=12))
    b.append(text(312, 274, "low addr", LIGHT, 10, 600))
    b.append(text(510, 274, "high addr", LIGHT, 10, 600))
    write("figures/alignment.svg",
        svg(W, H, "".join(b), "Alignment & endianness"))

def fig_va_split():
    W, H = 900, 280
    b = [text(W / 2, 28, "The 48-bit canonical virtual address", GREY, 15, 700)]
    fields = [("sign ext", "63..48", GREY_D, 130), ("PML4", "47..39", PURPLE,
        110), ("PDPT", "38..30", PURPLE, 110), ("PD", "29..21", PURPLE,
        110), ("PT", "20..12", PURPLE, 110), ("offset", "11..0", GREY,
        140)]
    x, y = 40, 64
    for name, bits, col, w in fields:
        b.append(box(x, y, w, 50, [name], col, size=11.5, rx=4))
        b.append(text(x + w / 2, y - 8, bits, LIGHT, 9, 600))
        x += w + 2
    desc = [("PML4", "index into the top-level table"),
        ("PDPT", "\u2192 page-directory-pointer table"),
        ("PD", "\u2192 page directory"),
        ("PT", "\u2192 page table"),
        ("offset", "byte within the 4 KiB page")]
    for i, (k, v) in enumerate(desc):
        b.append(text(60, 150 + i * 22, k, PURPLE, 10.5, 700, anchor="start",
            mono=True))
        b.append(text(170, 150 + i * 22, v, GREY, 10.5, 600, anchor="start"))
    write("figures/va-split.svg",
        svg(W, H, "".join(b), "VA split"))

def fig_pte():
    W, H = 960, 300
    b = [text(W / 2, 28, "Page-table entry (PTE) bits", GREY, 15, 700)]
    fields = [("NX", "63", RED, 60), ("reserved", "62..52", GREY_D, 100),
        ("PFN", "51..12", TEAL, 200), ("OS", "11..9", GREY_D, 80),
        ("G", "8", GREY, 44), ("PS", "7", GREY, 44), ("D", "6", GREY, 44),

```



```

        ("A", "5", GREY, 44), ("PCD", "4", GREY, 54), ("PWT", "3", GREY,
54), ("U/S", "2", GREY, 54), ("R/W", "1", GREY, 54),
        ("P", "0", PURPLE, 40)]
x, y = 20, 64
for name, bit, col, w in fields:
    b.append(box(x, y, w, 46, [name], col, size=10, rx=3))
    b.append(text(x + w / 2, y - 8, bit, LIGHT, 8.5, 600))
    x += w + 2
notes = ["P = present", "R/W = writable", "U/S = user vs kernel",
        "A / D = accessed / dirty (HW-set)",
        "G = global (survives CR3 reload)", "PS = 1 \u2192 huge page here",
        "NX = 1 \u2192 non-executable (data)", "PFN = physical frame number"]
for i, n in enumerate(notes):
    b.append(text(40 + (i % 2) * 470, 150 + (i // 2) * 24, "\u2022 " + n,
        GREY, 10.5, 600, anchor="start"))
write("figures/pte.svg", svg(W, H, "".join(b), "PTE"))

def fig_per_process_tables():
    W, H = 740, 320
    b = [text(W / 2, 26, "Per-process page tables can share a frame",
        GREY, 15, 700)]
    b.append(box(40, 90, 160, 50, ["process A", "CR3 \u2192 PML4 A"], PURPLE,
        size=11, lh=15))
    b.append(box(40, 190, 160, 50, ["process B", "CR3 \u2192 PML4 B"], GREY,
        size=11, lh=15))
    b.append(box(300, 90, 150, 50, ["\u2026 tables \u2026"], GREY_D, size=11))
    b.append(box(300, 190, 150, 50, ["\u2026 tables \u2026"], GREY_D, size=11))
    b.append(box(550, 140, 150, 50, ["frame F7"], TEAL, size=12))
    b.append(arrow(200, 115, 300, 115, GREY, 1.6))
    b.append(arrow(200, 215, 300, 215, GREY, 1.6))
    b.append(arrow(450, 115, 550, 160, GREY, 1.6))
    b.append(arrow(450, 215, 550, 175, GREY, 1.6))
    b.append(text(625, 210, "refcount = 2", TEAL, 10, 700))
    b.append(text(625, 224, "(shared library)", TEAL, 10, 600))
    write("figures/per-process-tables.svg",
        svg(W, H, "".join(b), "Per-process tables"))

def fig_initial_stack():
    W, H = 520, 410
    b = [text(W / 2, 26, "The initial stack the kernel builds", GREY, 15, 700)]
    rows = [("argc", GREY_D, 36), ("argv[] pointers", GREY, 40),
        ("envp[] pointers", GREY, 40), ("auxv[] key/value", PURPLE, 44),
        ("envp strings", GREY_D, 40), ("argv strings", GREY_D, 40),
        ("AT_RANDOM, padding", LIGHT, 40)]
    bx, bw, y = 150, 240, 56
    for name, col, h in rows:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(seg(bx, y, bw, h, name, col, tcol=tc, size=11.5))
        y += h + 5

```

```

b.append(text(bx - 14, 64, "top", LIGHT, 10, 600, anchor="end"))
b.append(text(W / 2, H - 14,
              "getauxval(3) reads auxv; argv/envp point into the strings "
              "below", LIGHT, 10, 500))
write("figures/initial-stack.svg",
      svg(W, H, "".join(b), "Initial stack"))

def fig_call_stack():
    W, H = 560, 470
    b = [text(W / 2, 26, "The call stack (grows down)", GREY, 15, 700)]
    rows = [("args / env / auxv", "set up at execve", GREY_D, 44),
            ("frame: main", "\u2190 %rbp", GREY, 40),
            ("return address", "", PURPLE_D, 32),
            ("frame: foo", "pushed by call", GREY, 40),
            ("return address", "", PURPLE_D, 32),
            ("frame: bar", "", GREY, 40),
            ("unused (mapped)", "\u2190 %rsp", LIGHT, 44),
            ("guard page", "PROT_NONE \u2192 SIGSEGV", RED, 44)]
    bx, bw, y = 160, 270, 52
    for name, sub, col, h in rows:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(seg(bx, y, bw, h, name, col, sub=(sub or None), tcol=tc,
                    size=11.5))
        y += h + 5
    b.append(arrow(bx - 26, 68, bx - 26, y - 22, GREY, 1.6))
    b.append(text(bx - 32, 64, "high", LIGHT, 9, 600, anchor="end"))
    b.append(text(bx - 32, y - 22, "low", LIGHT, 9, 600, anchor="end"))
    write("figures/call-stack.svg",
          svg(W, H, "".join(b), "Call stack"))

def fig_top_chunk():
    W, H = 900, 240
    b = [text(W / 2, 28, "The heap after a while: the top chunk (wilderness)",
              GREY, 15, 700)]
    segs = [("in-use", GREY, 120), ("free", LIGHT, 110), ("in-use", GREY, 120),
            ("in-use", GREY, 120), ("free", LIGHT, 110), ("TOP CHUNK", TEAL,
            230)]
    x, y = 30, 92
    for name, col, w in segs:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(seg(x, y, w, 50, name, col, tcol=tc, size=11))
        x += w + 2
    b.append(arrow(x - 2, y + 74, x - 2, y + 52, GREY, 1.6))
    b.append(text(x - 2, y + 88, "program break (sbrk(0))", GREY, 10, 600,
                  anchor="end"))
    b.append(text(W / 2, y - 12,
                  "only the top chunk can grow via sbrk / mmap", LIGHT, 10.5,
                  600))
    write("figures/top-chunk.svg",

```

```

svg(W, H, "".join(b), "Top chunk"))

def fig_arenas():
    W, H = 820, 320
    b = [text(W / 2, 26, "Arenas: per-thread heaps cut lock contention",
              GREY, 15, 700)]
    cols = [("main arena", "grown by brk", PURPLE, 40),
            ("arena 1", "grown by mmap", GREY, 310),
            ("arena 2", "grown by mmap", GREY, 580)]
    for name, sub, col, x in cols:
        b.append(box(x, 70, 200, 52, ["freelist (bins)"], col, size=11))
        b.append(box(x, 130, 200, 44, ["top chunk"], GREY_D, size=11))
        b.append(text(x + 100, 194, sub, LIGHT, 10, 600))
        b.append(box(x, 214, 200, 44, [name + " lock"], col, size=11))
    b.append(text(W / 2, H - 14,
                  "a thread pins to an arena; many arenas can balloon VSZ far "
                  "past the working set", LIGHT, 10.5, 500))
    write("figures/arenas.svg", svg(W, H, "".join(b), "Arenas"))

def fig_physical_ram():
    W, H = 640, 420
    b = [text(W / 2, 26, "What lives in physical RAM", GREY, 15, 700)]
    rows = [("free pages", "on buddy free lists", GREY_D, 50),
            ("anonymous pages", "heap \u00b7 stack \u00b7 bss \u00b7 anon mmap", TEAL,
56),
            ("file-backed (page cache)", "mmap'd files \u00b7 read() buffers",
            PURPLE, 56),
            ("slab / kernel", "inode & network buffers", GREY, 50),
            ("reserved / kernel text", "", GREY_D, 44)]
    bx, bw, y = 150, 340, 56
    for name, sub, col, h in rows:
        b.append(seg(bx, y, bw, h, name, col, sub=(sub or None), size=11.5))
        y += h + 6
    b.append(text(W / 2, H - 16,
                  "reclaimable cache is why MemAvailable \u226b MemFree",
                  LIGHT, 10.5, 500))
    write("figures/physical-ram.svg",
          svg(W, H, "".join(b), "Physical RAM"))

def fig_tls():
    W, H = 680, 340
    b = [text(W / 2, 26, "Thread-local storage: one copy per thread",
              GREY, 15, 700)]
    for i, (x, lab) in enumerate([(90, "thread A"), (390, "thread B")]):
        b.append(text(x + 100, 60, lab + " TCB", GREY, 12, 700))
        yy = 76
        for name, col in [("dtv pointer", GREY_D), ("per_thread_x", PURPLE),
                          ("per_thread_y", PURPLE)]:

```

```

        b.append(seg(x, yy, 200, 44, name, col, size=11.5))
        yy += 48
    b.append(text(x + 100, yy + 6,
        "%fs in " + ("A" if i == 0 else "B") + " points here",
        TEAL, 10, 600))
b.append(text(W / 2, H - 14,
    "the FS register points at the TCB; each thread's vars are "
    "distinct words", LIGHT, 10, 500))
write("figures/tls.svg", svg(W, H, "".join(b), "TLS"))

def fig_ipc_options():
    W, H = 880, 360
    b = [text(W / 2, 26, "Choosing an IPC mechanism by data shape",
        GREY, 15, 700)]
    b.append(box(340, 54, 200, 44, ["how big is each message?"], PURPLE,
        size=11))
    b.append(text(210, 122, "small (bytes)", GREY, 11, 700))
    for i, s in enumerate(["signals \u00b7 eventfd \u00b7 signalfd", "pipes / FIFO",
        "UNIX sockets", "message queues"]):
        b.append(box(60, 140 + i * 46, 300, 38, [s], GREY, size=11))
    b.append(arrow(400, 98, 210, 140, GREY, 1.4))
    b.append(text(670, 122, "large / streaming", GREY, 11, 700))
    for i, s in enumerate(["mmap + shared memory", "memfd_create",
        "MAP_SHARED file"]):
        b.append(box(520, 140 + i * 46, 300, 38, [s], TEAL, size=11))
    b.append(arrow(480, 98, 670, 140, GREY, 1.4))
    b.append(text(W / 2, H - 14,
        "pass fds with socketpair + SCM_RIGHTS; wake a peer with "
        "eventfd + epoll", LIGHT, 10, 500))
    write("figures/ipc-options.svg",
        svg(W, H, "".join(b), "IPC options"))

def fig_io_uring():
    W, H = 760, 280
    b = [text(W / 2, 26, "io_uring: two mmap'd rings shared with the kernel",
        GREY, 15, 700)]
    b.append(text(180, 64, "process VA", GREY, 12, 700))
    b.append(text(580, 64, "kernel", GREY, 12, 700))
    b.append(box(60, 84, 240, 54, ["SQ ring (mmap)", "submission queue"],
        PURPLE, size=11, lh=15))
    b.append(box(460, 84, 240, 54, ["sq_entries[]"], GREY, size=11))
    b.append(box(60, 164, 240, 54, ["CQ ring (mmap)", "completion queue"],
        TEAL, size=11, lh=15))
    b.append(box(460, 164, 240, 54, ["cq_entries[]"], GREY, size=11))
    b.append(arrow(300, 105, 460, 105, GREY, 1.7))
    b.append(arrow(460, 118, 300, 118, GREY, 1.7))
    b.append(arrow(300, 185, 460, 185, GREY, 1.7))
    b.append(arrow(460, 198, 300, 198, GREY, 1.7))
    b.append(text(380, 98, "shared", LIGHT, 9, 600))

```

```

b.append(text(380, 178, "shared", LIGHT, 9, 600))
b.append(text(W / 2, H - 14,
              "queue requests with no syscall; io_uring_enter submits and "
              "harvests completions", LIGHT, 10, 500))
write("figures/io-uring.svg", svg(W, H, "".join(b), "io_uring"))

def fig_bump():
    W, H = 760, 200
    b = [text(W / 2, 28, "Bump allocator: alloc just advances a pointer",
              GREY, 15, 700)]
    b.append(seg(60, 82, 300, 50, "used", PURPLE, size=12))
    b.append(seg(360, 82, 340, 50, "free", LIGHT, tcol=INK_DARK, size=12))
    b.append(text(62, 150, "base", LIGHT, 10, 600, anchor="start"))
    b.append(arrow(360, 76, 420, 76, TEAL, 1.8))
    b.append(text(390, 66, "bump ptr", TEAL, 10, 700))
    b.append(text(698, 150, "end", LIGHT, 10, 600, anchor="end"))
    b.append(text(W / 2, H - 14,
                  "O(1) alloc, no per-object free \u2014 drop the whole region at "
                  "once", LIGHT, 10.5, 500))
    write("figures/bump.svg",
          svg(W, H, "".join(b), "Bump allocator"))

def fig_pool():
    W, H = 720, 270
    b = [text(W / 2, 26, "Pool allocator: fixed-size slots + freelist",
              GREY, 15, 700)]
    for i in range(4):
        col = GREY if i in (1, 3) else LIGHT
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(box(80 + i * 150, 84, 130, 54, [f"slot {i}"], col, tcol=tc,
                    size=12))
    b.append(path("M145 138 C145 180 445 180 445 138", TEAL, 1.8,
                  arrow_end=True))
    b.append(text(300, 196, "free: 0 \u2192 2 \u2192 NULL (threaded through unused "
                  "slots)", TEAL, 10, 600))
    b.append(text(W / 2, H - 16,
                  "alloc = pop head \u00b7 free = push head \u00b7 both O(1)",
                  LIGHT, 10.5, 500))
    write("figures/pool.svg",
          svg(W, H, "".join(b), "Pool allocator"))

def fig_freelist():
    W, H = 860, 220
    b = [text(W / 2, 28, "Freelist allocator: header + payload chunks",
              GREY, 15, 700)]
    x = 40
    for i in range(3):
        b.append(seg(x, 82, 70, 54, "hdr", PURPLE, size=10))

```

```

    x += 72
    col = GREY if i % 2 == 0 else LIGHT
    tc = WHITE if i % 2 == 0 else INK_DARK
    b.append(seg(x, 82, 180, 54, "payload", col, tc=tc, size=11))
    x += 182
b.append(text(40, 152, "base", LIGHT, 10, 600, anchor="start"))
b.append(text(x - 2, 152, "top", LIGHT, 10, 600, anchor="end"))
b.append(text(W / 2, H - 16,
             "the header holds size + free flag; free(p) reads it at "
             "p - sizeof(header) and coalesces neighbours", LIGHT, 10, 500))
write("figures/freelist.svg",
      svg(W, H, "".join(b), "Freelist allocator"))

def fig_kernel_user():
    W, H = 600, 320
    b = [text(W / 2, 26, "The canonical address split (x86-64)", GREY, 15, 700)]
    bx, bw = 170, 260
    rows = [("kernel space", "CPL0 only \u00b7 in every process", GREY_D, 60,
            "0xFFFFFFFFFFFFFFFF"),
            ("non-canonical hole", "unmapped 47-bit gap", LIGHT, 50,
            "0xFFFF800000000000"),
            ("user space", "your code \u00b7 per process", PURPLE, 70,
            "0x0000800000000000")]
    y = 54
    for name, sub, col, h, a in rows:
        tc = INK_DARK if col == LIGHT else WHITE
        b.append(seg(bx, y, bw, h, name, col, sub=sub, tc=tc, size=12))
        b.append(addr(bx - 12, y + 12, a))
        y += h + 6
    b.append(addr(bx - 12, y + 2, "0x0"))
    b.append(text(W / 2, H - 16,
                 "a syscall switches to ring 0 and the kernel half becomes "
                 "addressable", LIGHT, 10, 500))
    write("figures/kernel-user-split.svg",
          svg(W, H, "".join(b), "Kernel/user split"))

def fig_thread_stacks():
    W, H = 560, 380
    b = [text(W / 2, 26, "Where thread stacks live", GREY, 15, 700)]
    bx, bw = 150, 270
    rows = [("main thread stack", "top of the address space", PURPLE, 54),
            ("mmap region", "", GREY_D, 40),
            ("thread 3 stack", "", GREY, 40),
            ("thread 2 stack", "", GREY, 40),
            ("thread 1 stack", "8 MiB + guard each", GREY, 44),
            ("\u2026 heap, libs below \u2026", "", LIGHT, 40)]
    y = 54
    for name, sub, col, h in rows:
        tc = INK_DARK if col == LIGHT else WHITE

```

```

        b.append(seg(bx, y, bw, h, name, col, sub=(sub or None), tcol=tc,
                    size=11.5))
        y += h + 5
    b.append(text(W / 2, H - 14,
                "each pthread gets its own mmap'd stack with a PROT_NONE "
                "guard page", LIGHT, 10, 500))
    write("figures/thread-stacks.svg",
        svg(W, H, "".join(b), "Thread stacks"))

def fig_pthread_create():
    W, H = 760, 360
    b = [text(W / 2, 26, "pthread_create is a wrapper around clone(2)",
            GREY, 15, 700)]
    b.append(box(280, 54, 200, 44, ["pthread_create(fn)", PURPLE, size=12))
    steps = [("allocate stack", "mmap + guard page", GREY),
            ("allocate TCB", "at the top of the stack", GREY),
            ("set up TLS + signal mask", "", GREY),
            ("clone(CLONE_VM | \u2026)", "shares VM, files, fds, sighand", TEAL)]
    y = 120
    for t, s, col in steps:
        b.append(arrow(380, y - 18, 380, y, GREY, 1.6))
        b.append(box(220, y, 320, 46, [t, s] if s else [t], col, size=11,
                    lh=15))
        y += 60
    b.append(text(W / 2, H - 16,
                "child_tid is cleared on exit, then futex_wake lets "
                "pthread_join return", LIGHT, 10, 500))
    write("figures/pthread-create.svg",
        svg(W, H, "".join(b), "pthread_create"))

# — NASM / assembly-specific figures —————
def fig_register_file():
    W, H = 900, 428
    b = [text(W / 2, 28, "The x86-64 general-purpose register file",
            GREY, 16, 700)]
    names = ["rax", "rbx", "rcx", "rdx", "rsi", "rdi", "rbp", "rsp",
            "r8", "r9", "r10", "r11", "r12", "r13", "r14", "r15"]
    roles = {"rax": "return / accum", "rdi": "arg 1", "rsi": "arg 2",
            "rdx": "arg 3", "rcx": "arg 4", "r8": "arg 5", "r9": "arg 6",
            "rsp": "stack pointer", "rbp": "frame pointer"}
    argset = {"rax", "rdi", "rsi", "rdx", "rcx", "r8", "r9"}
    ptr = {"rsp", "rbp"}
    cw, ch, gx, gy, x0, y0 = 190, 54, 20, 14, 40, 64
    for i, n in enumerate(names):
        r, c = divmod(i, 4)
        x, y = x0 + c * (cw + gx), y0 + r * (ch + gy)
        col = TEAL if n in ptr else (PURPLE if n in argset else GREY)
        b.append(seg(x, y, cw, ch, n, col, sub=roles.get(n, "general"),
                    size=13))

```

```

yb = y0 + 4 * (ch + gy) + 4
b.append(seg(40, yb, 400, 48, "rip", GREY_D,
            sub="next instruction address", size=13))
b.append(seg(460, yb, 400, 48, "rflags", GREY_D,
            sub="status + control flags", size=13))
write("figures/register-file.svg",
      svg(W, H, "".join(b), "Register file"))

def fig_rax_family():
    W, H = 820, 300
    b = [text(W / 2, 28, "One register, four names: RAX / EAX / AX / AH\u00b7AL",
             GREY, 16, 700)]
    x0, full = 60, 700
    b.append(text(x0, 60, "63", LIGHT, 9, 600))
    b.append(text(x0 + full, 60, "0", LIGHT, 9, 600))
    b.append(text(x0 + full / 2, 60, "31", LIGHT, 9, 600))
    b.append(text(x0 + full * 3 / 4, 60, "15", LIGHT, 9, 600))
    b.append(text(x0 + full * 7 / 8, 60, "7", LIGHT, 9, 600))
    b.append(seg(x0, 70, full, 46, "RAX (64-bit)", GREY, size=13))
    b.append(seg(x0 + full / 2, 124, full / 2, 44, "EAX (low 32)", PURPLE,
                size=12))
    b.append(seg(x0 + full * 3 / 4, 176, full / 4, 42, "AX (low 16)", TEAL,
                size=12))
    b.append(seg(x0 + full * 3 / 4, 224, full / 8, 40, "AH", AMBER,
                tcol=INK_DARK, size=11))
    b.append(seg(x0 + full * 7 / 8, 224, full / 8, 40, "AL", AMBER,
                tcol=INK_DARK, size=11))
    b.append(text(W / 2, H - 14,
                 "writing EAX zeroes the upper 32 bits; writing AX / AL leaves "
                 "the rest intact", LIGHT, 10.5, 500))
    write("figures/rax-family.svg", svg(W, H, "".join(b), "RAX family"))

def fig_rflags():
    W, H = 880, 300
    b = [text(W / 2, 28, "RFLAGS \u2014 the status bits that drive branches",
             GREY, 16, 700)]
    bits = [("OF", "11", TEAL), ("DF", "10", GREY), ("IF", "9", GREY),
            ("TF", "8", GREY_D), ("SF", "7", PURPLE), ("ZF", "6", PURPLE),
            ("AF", "4", GREY), ("PF", "2", PURPLE), ("CF", "0", PURPLE)]
    x, cw = 40, 88
    for name, bit, col in bits:
        b.append(box(x, 72, cw, 50, [name], col, size=13))
        b.append(text(x + cw / 2, 62, "bit " + bit, LIGHT, 9, 600))
        x += cw + 2
    notes = ["CF carry / unsigned overflow", "ZF result was zero",
            "SF sign (top bit) of result", "OF signed overflow",
            "PF low-byte parity", "AF BCD adjust carry",
            "DF string direction (0 = up)", "IF interrupts enabled"]
    for i, n in enumerate(notes):

```



```

        b.append(text(60 + (i % 2) * 430, 150 + (i // 2) * 24, "\u2022 " + n,
                     GREY, 11, 600, anchor="start"))
    write("figures/rflags.svg", svg(W, H, "".join(b), "RFLAGS"))

def fig_cmp_jcc():
    W, H = 780, 300
    b = [text(W / 2, 26, "cmp / test set the flags; jcc reads them",
             GREY, 15, 700)]
    b.append(box(60, 78, 250, 60, ["cmp rax, rbx",
                                     "computes rax - rbx,", "throws the result away"], GREY,
                 size=11, lh=16))
    b.append(box(60, 172, 250, 44, ["sets ZF SF CF OF"], PURPLE, size=12))
    b.append(arrow(185, 138, 185, 172, GREY, 1.8))
    b.append(box(470, 74, 270, 68, ["je  \u2192 ZF=1      (equal)",
                                     "jl  \u2192 SF\u22600F      (signed <)",
                                     "jb  \u2192 CF=1      (unsigned <)"], TEAL, size=11, lh=17))
    b.append(box(470, 172, 270, 44, ["jcc reads those flags"], GREY_D,
                                     size=12))
    b.append(arrow(310, 194, 470, 194, GREY, 1.8))
    b.append(text(390, 184, "flags", LIGHT, 9, 600))
    b.append(text(W / 2, H - 16,
                  "test does AND, cmp does SUB \u2014 neither writes its operands, "
                  "they only set flags", LIGHT, 10, 500))
    write("figures/cmp-jcc.svg", svg(W, H, "".join(b), "cmp and jcc"))

def fig_push_pop():
    W, H = 780, 330
    b = [text(W / 2, 26, "push and pop move RSP by 8", GREY, 15, 700)]
    bx, bw = 300, 200
    slots = ["(older frames)", "return address", "saved rbp", "local var",
             "rax \u2190 just pushed"]
    for i, s in enumerate(slots):
        col = PURPLE if "pushed" in s else GREY
        b.append(seg(bx, 70 + i * 44, bw, 40, s, col, size=12))
    ry = 70 + 4 * 44 + 20
    b.append(arrow(bx - 44, ry, bx - 8, ry, TEAL, 2))
    b.append(text(bx - 50, ry, "rsp", TEAL, 12, 700, anchor="end"))
    b.append(text(bx + bw + 16, 90, "high addr", LIGHT, 10, 600,
                  anchor="start"))
    b.append(text(bx + bw + 16, ry, "low addr", LIGHT, 10, 600,
                  anchor="start"))
    b.append(text(W / 2, H - 16,
                  "push: rsp -= 8 then store \u00b7 pop: load then rsp += 8 \u00b7 "
                  "the stack grows downward", LIGHT, 10, 500))
    write("figures/push-pop.svg", svg(W, H, "".join(b), "push and pop"))

def fig_sysv_args():
    W, H = 820, 320

```

```

b = [text(W / 2, 26, "System V AMD64: how arguments are passed",
        GREY, 15, 700)]
for i, r in enumerate(["rdi", "rsi", "rdx", "rcx", "r8", "r9"]):
    b.append(box(40 + i * 128, 74, 116, 52, [r, f"arg {i + 1}"], PURPLE,
        size=12, lh=15))
b.append(text(W / 2, 162, "args 7+ \u2192 pushed on the stack, right to left",
        GREY, 12, 700))
b.append(box(300, 188, 220, 44, ["return value \u2192 rax"], TEAL, size=12))
b.append(text(W / 2, H - 16,
        "rdx can return a second word; for varargs, al holds the "
        "vector-register count", LIGHT, 10, 500))
write("figures/sysv-args.svg", svg(W, H, "".join(b), "SysV arguments"))

def fig_caller_callee():
    W, H = 820, 300
    b = [text(W / 2, 26, "Caller-saved vs callee-saved registers",
        GREY, 15, 700)]
    b.append(text(226, 64, "caller-saved (volatile)", AMBER, 12, 700))
    for i, r in enumerate(["rax", "rcx", "rdx", "rsi", "rdi", "r8", "r9",
        "r10", "r11"]):
        b.append(box(40 + (i % 3) * 128, 80 + (i // 3) * 46, 116, 40, [r],
            AMBER, tcol=INK_DARK, size=12))
    b.append(text(632, 64, "callee-saved (preserved)", TEAL, 12, 700))
    for i, r in enumerate(["rbx", "rbp", "rsp", "r12", "r13", "r14", "r15"]):
        b.append(box(470 + (i % 3) * 116, 80 + (i // 3) * 46, 104, 40, [r],
            TEAL, size=12))
    b.append(text(226, 236, "the callee may clobber these", LIGHT, 10, 600))
    b.append(text(632, 236, "the callee must restore these", LIGHT, 10, 600))
    b.append(text(W / 2, H - 14,
        "need a caller-saved value to survive a call? save it "
        "yourself first", LIGHT, 10, 500))
    write("figures/caller-callee.svg",
        svg(W, H, "".join(b), "Caller/callee saved"))

def fig_addressing():
    W, H = 860, 280
    b = [text(W / 2, 26, "Effective address: [ base + index*scale + disp ]",
        GREY, 15, 700)]
    parts = [("base", "rbx", "any GPR", PURPLE),
        ("index", "rsi", "any GPR but rsp", TEAL),
        ("scale", "\u00d7", "1 2 4 8", AMBER),
        ("disp", "+16", "constant", GREY)]
    x = 60
    for k, (name, ex, note, col) in enumerate(parts):
        w = 160
        tc = INK_DARK if col == AMBER else WHITE
        b.append(box(x, 80, w, 54, [name, ex], col, tcol=tc, size=12, lh=16))
        b.append(text(x + w / 2, 150, note, LIGHT, 10, 600))
        if k < 3:

```

```

        b.append(text(x + w + 20, 107, "+", GREY, 16, 700))
        x += w + 40
    b.append(box(260, 200, 340, 44,
        ["\u2192 one memory operand, computed by the CPU"], GREY_D,
        size=12))
    b.append(text(W / 2, H - 14,
        "mov eax, [rbx + rsi*4 + 16] loads int element rsi of an "
        "array at rbx+16", LIGHT, 10, 500))
    write("figures/addressing.svg", svg(W, H, "".join(b), "Addressing modes"))

def fig_muldiv():
    W, H = 780, 300
    b = [text(W / 2, 26, "mul and div use the RDX:RAX pair", GREY, 15, 700)]
    b.append(text(200, 64, "mul rbx    (unsigned)", PURPLE, 12, 700))
    b.append(box(60, 80, 120, 44, ["rax"], GREY, size=12))
    b.append(text(200, 102, "\u00d7 rbx", GREY, 12, 700))
    b.append(box(250, 80, 300, 44, ["rbx"], GREY, size=12))
    b.append(arrow(205, 124, 305, 158, GREY, 1.8))
    b.append(box(60, 162, 140, 44, ["rdx    (high)"], TEAL, size=11))
    b.append(box(210, 162, 300, 44, ["rax    (low)"], TEAL, size=11))
    b.append(text(575, 64, "div rbx", PURPLE, 12, 700))
    b.append(box(430, 80, 530, 44, ["rdx:rax"], GREY, size=12))
    b.append(text(590, 102, "\u00f7 rbx", GREY, 12, 700))
    b.append(box(620, 80, 720, 44, ["rbx"], GREY, size=12))
    b.append(arrow(575, 124, 725, 158, GREY, 1.8))
    b.append(box(430, 162, 530, 44, ["rax    quotient"], TEAL, size=11))
    b.append(box(580, 162, 720, 44, ["rdx    remainder"], TEAL, size=11))
    b.append(text(W / 2, H - 16,
        "before div set rdx: xor it for unsigned, or cqo to "
        "sign-extend rax", LIGHT, 10, 500))
    write("figures/muldiv.svg", svg(W, H, "".join(b), "mul and div"))

def fig_syscall_path():
    W, H = 780, 320
    b = [text(W / 2, 26, "A Linux syscall: user \u2192 kernel \u2192 back",
        GREY, 15, 700)]
    b.append(box(60, 78, 270, 68, ["set rax = syscall number",
        "args in rdi rsi rdx r10 r8 r9", "execute  syscall"], PURPLE,
        size=10.5, lh=16))
    b.append(box(450, 78, 270, 68, ["CPU enters ring 0 (MSR_LSTAR)",
        "runs the sys_* handler", "no IDT lookup"], GREY, size=10.5,
        lh=16))
    b.append(arrow(330, 100, 450, 100, GREY, 2))
    b.append(text(390, 90, "syscall", LIGHT, 9, 600))
    b.append(box(450, 190, 270, 44, ["result placed in rax"], TEAL, size=12))
    b.append(box(60, 190, 270, 44, ["check rax; negative = -errno"], GREY_D,
        size=11))
    b.append(arrow(450, 212, 330, 212, GREY, 2))
    b.append(text(390, 202, "sysret", LIGHT, 9, 600))

```

```

b.append(arrow(585, 146, 585, 190, GREY, 1.6))
b.append(text(W / 2, H - 16,
             "syscall is far cheaper than the legacy int 0x80 \u2014 it uses "
             "MSRs, not the IDT", LIGHT, 10, 500))
write("figures/syscall-path.svg", svg(W, H, "".join(b), "Syscall path"))

def fig_simd_regs():
    W, H = 820, 320
    b = [text(W / 2, 26, "SIMD registers: XMM \u2286 YMM \u2286 ZMM", GREY, 15, 700)]
    x0 = 60
    b.append(seg(x0, 72, 700, 44, "ZMM0 (512-bit, AVX-512)", GREY, size=12))
    b.append(seg(x0, 120, 350, 42, "YMM0 (256-bit, AVX)", PURPLE, size=12))
    b.append(seg(x0, 166, 175, 40, "XMM0 (128-bit, SSE)", TEAL, size=11))
    b.append(text(x0, 232, "one XMM as 4 packed float32 lanes:", GREY, 11, 700,
                  anchor="start"))
    for i in range(4):
        b.append(box(x0 + i * 92, 246, 88, 40, [f"f{i}"], AMBER, tcol=INK_DARK,
                  size=11))
    b.append(text(W / 2, H - 14,
                 "one instruction (addps) adds all four lanes at once \u2014 "
                 "data-level parallelism", LIGHT, 10, 500))
    write("figures/simd-regs.svg", svg(W, H, "".join(b), "SIMD registers"))

def fig_instr_format():
    W, H = 900, 250
    b = [text(W / 2, 28, "Anatomy of an x86-64 instruction", GREY, 15, 700)]
    parts = [("prefix", "0-4 B", GREY_D), ("REX", "0-1 B", AMBER),
             ("opcode", "1-3 B", PURPLE), ("ModRM", "0-1 B", TEAL),
             ("SIB", "0-1 B", TEAL), ("disp", "0/1/2/4", GREY),
             ("imm", "0/1/2/4/8", GREY)]
    x = 30
    for name, sz, col in parts:
        w = 118
        tc = INK_DARK if col == AMBER else WHITE
        b.append(box(x, 74, w, 54, [name, sz], col, tcol=tc, size=11.5, lh=15))
        x += w + 3
    b.append(text(W / 2, 162,
                 "only the opcode is mandatory; most instructions are "
                 "2\u2013134 bytes", GREY, 11, 600))
    b.append(text(W / 2, H - 16,
                 "mov [rdi+rsi*4+1], eax = REX + opcode + ModRM + SIB + disp8",
                 LIGHT, 10, 500))
    write("figures/instr-format.svg",
          svg(W, H, "".join(b), "Instruction format"))

def fig_rex_byte():
    W, H = 760, 240
    b = [text(W / 2, 28, "The REX prefix byte", GREY, 15, 700)]

```

```

cells = [("0100", "fixed", GREY_D, 150), ("W", "64-bit op", PURPLE, 110),
         ("R", "reg ext", TEAL, 110), ("X", "index ext", TEAL, 110),
         ("B", "r/m ext", TEAL, 110)]

x = 50
for name, sub, col, w in cells:
    b.append(box(x, 78, w, 54, [name, sub], col, size=12, lh=15))
    x += w + 2
b.append(text(W / 2, H - 34,
             "W=1 selects 64-bit operands; R / X / B supply the high bit to "
             "reach r8\u2013r15", LIGHT, 10.5, 500))
write("figures/rex-byte.svg", svg(W, H, "".join(b), "REX byte"))

def fig_modrm():
    W, H = 680, 240
    b = [text(W / 2, 28, "The ModR/M byte", GREY, 15, 700)]
    cells = [("mod (2 bits)", "addressing mode", PURPLE),
             ("reg (3 bits)", "register / opcode ext", TEAL),
             ("r/m (3 bits)", "register or memory", GREY)]
    x = 60
    for head, sub, col in cells:
        b.append(box(x, 78, 180, 60, [head, sub], col, size=11, lh=16))
        x += 188
    b.append(text(W / 2, H - 34,
                 "mod=11 \u2013 register operand; mod=00/01/10 \u2013 memory with "
                 "0/1/4-byte disp", LIGHT, 10.5, 500))
    write("figures/modrm.svg", svg(W, H, "".join(b), "ModRM byte"))

def fig_sib():
    W, H = 680, 240
    b = [text(W / 2, 28, "The SIB byte (scaled-index addressing)",
              GREY, 15, 700)]
    cells = [("scale (2 bits)", "1 2 4 8", AMBER),
             ("index (3 bits)", "index register", TEAL),
             ("base (3 bits)", "base register", GREY)]
    x = 60
    for head, sub, col in cells:
        tc = INK_DARK if col == AMBER else WHITE
        b.append(box(x, 78, 180, 60, [head, sub], col, tc=tc, size=11, lh=16))
        x += 188
    b.append(text(W / 2, H - 34,
                 "present only when r/m=100; encodes [base + index*scale] like "
                 "[rbx + rsi*4]", LIGHT, 10.5, 500))
    write("figures/sib.svg", svg(W, H, "".join(b), "SIB byte"))

def fig_got_plt():
    W, H = 820, 300
    b = [text(W / 2, 26, "Lazy binding through the PLT and GOT",
              GREY, 15, 700)]

```

```

b.append(box(40, 92, 180, 50, ["call printf@plt"], PURPLE, size=12))
b.append(box(300, 92, 200, 50, ["PLT stub", "jmp *[GOT entry]"], GREY,
    size=11, lh=15))
b.append(box(580, 62, 200, 46, ["GOT \u2192 resolver", "(first call only)"],
    AMBER, tcol=INK_DARK, size=11, lh=15))
b.append(box(580, 132, 200, 46, ["GOT \u2192 real printf", "(after resolve)"],
    TEAL, size=11, lh=15))
b.append(arrow(220, 117, 300, 117, GREY, 1.8))
b.append(arrow(500, 110, 580, 85, GREY, 1.6))
b.append(arrow(500, 124, 580, 155, GREY, 1.6))
b.append(text(W / 2, H - 40,
    "the first call jumps to the dynamic linker, which patches the "
    "GOT;", LIGHT, 10.5, 500))
b.append(text(W / 2, H - 22,
    "every later call jumps straight to the resolved address",
    LIGHT, 10.5, 500))
write("figures/got-plt.svg", svg(W, H, "".join(b), "GOT and PLT"))

def fig_elf_layout():
    W, H = 560, 420
    b = [text(W / 2, 26, "ELF file layout", GREY, 15, 700)]
    rows = [("ELF header", "entry point \u00b7 e_phoff", PURPLE, 46),
        ("program headers", "LOAD segments (loader)", TEAL, 44),
        (".text", "machine code", GREY, 38),
        (".rodata", "read-only constants", GREY, 38),
        (".data", "initialised globals", GREY, 38),
        (".bss", "zero-init, no file bytes", GREY_D, 38),
        ("section headers", "symbol / section names", GREY_D, 44)]
    bx, bw, y = 150, 300, 52
    for name, sub, col, h in rows:
        b.append(seg(bx, y, bw, h, name, col, sub=sub, size=11.5))
        y += h + 5
    b.append(text(W / 2, H - 14,
        "the loader reads program headers; the linker reads section "
        "headers", LIGHT, 10, 500))
    write("figures/elf-layout.svg", svg(W, H, "".join(b), "ELF layout"))

def fig_idt():
    W, H = 800, 320
    b = [text(W / 2, 26, "Interrupt dispatch through the IDT", GREY, 15, 700)]
    b.append(box(40, 96, 180, 66, ["event", "exception / IRQ /", "int n"],
        PURPLE, size=11, lh=15))
    b.append(box(300, 92, 190, 76, ["IDT", "[vector] \u2192 handler",
        "+ CPL / gate type"], GREY, size=11, lh=16))
    b.append(box(580, 62, 200, 44, ["#PF handler (14)"], TEAL, size=11))
    b.append(box(580, 122, 200, 44, ["timer IRQ (32)"], TEAL, size=11))
    b.append(box(580, 182, 200, 44, ["int 0x80 (128)"], TEAL, size=11))
    b.append(arrow(220, 129, 300, 129, GREY, 1.8))
    for yy in (84, 144, 204):

```

```

        b.append(arrow(490, 130, 580, yy, GREY, 1.4))
    b.append(text(W / 2, H - 16,
                "the CPU pushes SS:RSP, RFLAGS, CS:RIP, then jumps to the "
                "gated handler", LIGHT, 10, 500))
    write("figures/idt.svg", svg(W, H, "".join(b), "IDT dispatch"))

def fig_context_switch():
    W, H = 840, 300
    b = [text(W / 2, 26, "A context switch swaps register state via the kernel",
              GREY, 15, 700)]
    b.append(box(40, 92, 180, 62, ["process A", "running in CPU"], PURPLE,
                size=11, lh=15))
    b.append(box(300, 64, 230, 50, ["save A's regs \u2192",
                                     "A's kernel stack / TSS"], GREY, size=11, lh=15))
    b.append(box(300, 156, 230, 50, ["load B's regs \u2190",
                                     "B's kernel stack / TSS"], GREY, size=11, lh=15))
    b.append(box(610, 156, 190, 62, ["process B", "runs (switch_to)"], TEAL,
                size=11, lh=15))
    b.append(arrow(220, 118, 300, 92, GREY, 1.6))
    b.append(arrow(415, 114, 415, 156, GREY, 1.8))
    b.append(arrow(530, 182, 610, 186, GREY, 1.6))
    b.append(text(W / 2, H - 16,
                "switching address spaces reloads CR3 and flushes "
                "non-global TLB entries", LIGHT, 10, 500))
    write("figures/context-switch.svg",
          svg(W, H, "".join(b), "Context switch"))

def fig_process_states():
    W, H = 760, 320
    b = [text(W / 2, 26, "Process state machine", GREY, 15, 700)]
    b.append(box(300, 66, 160, 50, ["RUNNING"], PURPLE, size=12))
    b.append(box(80, 182, 160, 50, ["READY"], GREY, size=12))
    b.append(box(520, 182, 160, 50, ["BLOCKED"], AMBER, tcol=INK_DARK,
                size=12))
    b.append(box(300, 256, 160, 46, ["ZOMBIE"], GREY_D, size=12))
    b.append(arrow(300, 108, 190, 182, GREY, 1.5))
    b.append(text(212, 138, "preempt", LIGHT, 9, 600))
    b.append(arrow(210, 182, 320, 112, GREY, 1.5))
    b.append(text(300, 150, "dispatch", LIGHT, 9, 600))
    b.append(arrow(430, 112, 560, 182, GREY, 1.5))
    b.append(text(520, 138, "wait I/O", LIGHT, 9, 600))
    b.append(arrow(520, 214, 240, 214, GREY, 1.5))
    b.append(text(380, 226, "I/O done \u2192 back to READY", LIGHT, 9, 600))
    b.append(arrow(380, 116, 380, 256, GREY, 1.5))
    b.append(text(404, 190, "exit", LIGHT, 9, 600))
    b.append(text(W / 2, H - 12,
                "a blocked task wakes to READY, not straight to RUNNING \u2014 "
                "the scheduler chooses next", LIGHT, 10, 500))
    write("figures/process-states.svg",

```

```

        svg(W, H, "".join(b), "Process states"))

def fig_spinlock():
    W, H = 780, 300
    b = [text(W / 2, 26, "A spinlock is one atomic compare-exchange",
              GREY, 15, 700)]
    card, w, h = code_card(50, 78,
                           [("acquire:", "dim"), (" xor eax, eax    ; want 0",
                                                    "n"), (" mov ecx, 1", "n"),
                              (" lock cmpxchg [lk], ecx", "hi"),
                              (" jnz acquire    ; retry", "n")],
                           "SPIN", GREY_D, LBL_BEFORE)

    b.append(card)
    xr = 50 + w + 56
    b.append(box(xr, 96, 230, 50, ["success \u2192 we own it", "(ZF=1, lock was 0)"],
                TEAL, size=11, lh=15))
    b.append(box(xr, 166, 230, 50, ["fail \u2192 spin again",
                                     "(someone else holds it)"], AMBER, tcol=INK_DARK, size=11,
                                     lh=15))
    b.append(arrow(50 + w + 6, 118, xr, 121, GREY, 1.6))
    b.append(arrow(50 + w + 6, 138, xr, 191, GREY, 1.6))
    b.append(text(W / 2, H - 16,
                  "cmpxchg is atomic under the lock prefix; a futex lets waiters "
                  "sleep instead of burning CPU", LIGHT, 10, 500))
    write("figures/spinlock.svg", svg(W, H, "".join(b), "Spinlock"))

ALL = [
    # reused systems figures (assembly-relevant internals)
    fig_address_space,
    fig_hierarchy, fig_cache_line,
    fig_virtual_physical, fig_va_split, fig_pte, fig_page_walk,
    fig_tlb, fig_page_fault,
    fig_elf_load,
    fig_stack_frame, fig_call_stack,
    fig_malloc_paths, fig_chunk, fig_arenas, fig_brk_vs_mmap,
    fig_fork_cow, fig_signal,
    fig_futex,
    # NASM / assembly-specific
    fig_register_file, fig_rax_family, fig_rflags,
    fig_cmp_jcc, fig_push_pop,
    fig_sysv_args, fig_caller_callee,
    fig_addressing, fig_muldiv,
    fig_syscall_path, fig_simd_regs,
    fig_instr_format, fig_rex_byte, fig_modrm, fig_sib,
    fig_got_plt, fig_elf_layout,
    fig_idt, fig_context_switch, fig_process_states, fig_spinlock,
]

if __name__ == "__main__":

```



```
for fn in ALL:  
    fn()  
print(f"\nDone: {len(ALL)} figures generated.")
```

CHAPTER 18

Topic 1: Setup & First Program

Welcome to your first assembly lesson! Let's get you set up and writing your first NASM program.

Part 1: Installation

Check if NASM is already installed:

```
nasm -version
```

If not installed, install it:

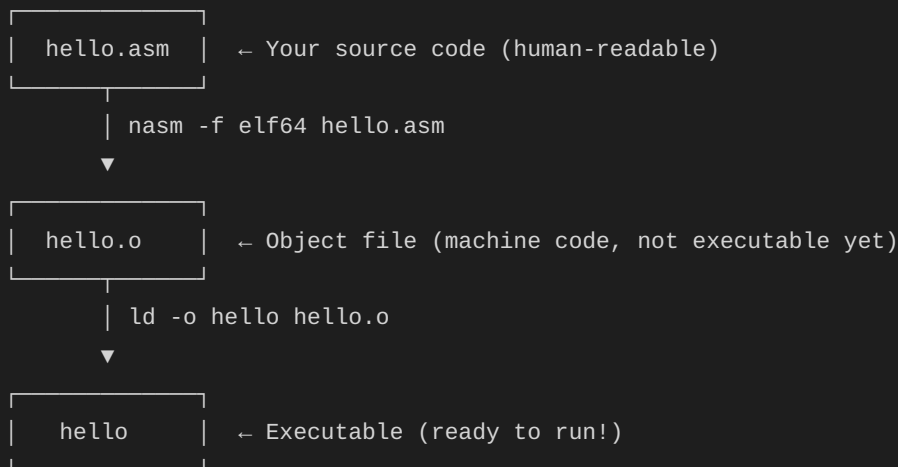
```
# Red Hat/CentOS/Rocky
sudo yum install nasm

# Or if you have dnf
sudo dnf install nasm

# You'll also need a linker (gcc provides ld)
sudo yum install gcc
```

Part 2: The Assembly Workflow

Understanding the build process:



Part 3: Program Structure

NASM programs have **three main sections**:

.data section - Initialized data (variables with values)

```
section .data
    message db "Hello", 0      ; Define byte(s), null-terminated
    number  dd 42               ; Define doubleword (32-bit)
```

.bss section - Uninitialized data (reserved space)

```
section .bss
    buffer resb 64              ; Reserve 64 bytes
    counter resd 1              ; Reserve 1 doubleword
```

.text section - Your actual code

```
section .text
    global _start               ; Entry point for linker

_start:
    ; Your instructions here
    mov eax, 1                  ; Exit syscall
    xor ebx, ebx                ; Return 0
    int 0x80                    ; Call kernel
```

Part 4: Your First Program - Hello World

C Equivalent:

```
#include <unistd.h>

int main() {
    const char *msg = "Hello, Assembly!\n";
    size_t msg_len = 17;

    // Write to stdout
    write(1, msg, msg_len);

    // Exit with code 0
    return 0;
}
```

Create a file called `hello.asm`:

```
; hello.asm - My first NASM program!
; This program prints "Hello, Assembly!" to the console

section .data
    ; Define our message with a newline at the end
    msg db "Hello, Assembly!", 10    ; 10 is newline character '\n'
    msg_len equ $ - msg              ; Calculate length (current pos - start)

section .text
    global _start

_start:
    ; Write message to stdout
    ; syscall: sys_write (rax = 1)
    ; rdi = file descriptor (1 = stdout)
    ; rsi = pointer to message
    ; rdx = message length

    mov rax, 1          ; syscall number for sys_write
    mov rdi, 1          ; file descriptor 1 = stdout
    mov rsi, msg        ; address of message
    mov rdx, msg_len    ; length of message
    syscall             ; make the system call

    ; Exit program
    ; syscall: sys_exit (rax = 60)
    ; rdi = exit code

    mov rax, 60         ; syscall number for sys_exit
    xor rdi, rdi        ; exit code 0 (success)
    syscall             ; make the system call
```

Part 5: Build and Run

Step 1: Assemble (convert .asm to .o)

```
nasm -f elf64 hello.asm -o hello.o
```

- `-f elf64`: output format for 64-bit Linux
- `-o hello.o`: output filename

Step 2: Link (convert .o to executable)

```
ld -o hello hello.o
```

- `-o hello`: output executable name

Step 3: Run

```
./hello
```

You should see:

```
Hello, Assembly!
```

Check exit code:

```
echo $?
```

Should print `0` (success)

Part 6: Understanding the Code

Line-by-Line Breakdown:

```
msg db "Hello, Assembly!", 10
```

- `msg` = label (like a variable name)
- `db` = Define Byte (allocates bytes in memory)
- `10` = ASCII code for newline

```
msg_len equ $ - msg
```

- `equ` = equate (like `#define` in C)
- `$` = current position in memory
- `$ - msg` = length of message

```
mov rax, 1
```

- Move value `1` into register `rax`
- In Linux, syscall number 1 = `write`

```
syscall
```

- Transfers control to the kernel
- Kernel looks at `rax` to know which syscall to execute

Part 7: 32-bit Version (Alternative)

C Equivalent (same as 64-bit):

```
#include <unistd.h>

int main() {
    const char *msg = "Hello from 32-bit!\n";
    write(1, msg, 19);
    return 0;
}
// Note: The C code is identical - the compiler handles 32 vs 64-bit differences
```

If you want to try 32-bit assembly:

```
; hello32.asm - 32-bit version

section .data
    msg db "Hello from 32-bit!", 10
    msg_len equ $ - msg

section .text
    global _start

_start:
    ; sys_write
    mov eax, 4      ; syscall 4 = write (32-bit)
    mov ebx, 1      ; stdout
    mov ecx, msg    ; message address
    mov edx, msg_len ; message length
    int 0x80        ; interrupt (32-bit syscall method)

    ; sys_exit
    mov eax, 1      ; syscall 1 = exit (32-bit)
    xor ebx, ebx    ; exit code 0
    int 0x80
```

Build 32-bit:

```
nasm -f elf32 hello32.asm -o hello32.o
ld -m elf_i386 -o hello32 hello32.o
./hello32
```

x

Part 8: Common Mistakes & Debugging

Forgot `global _start`

```
x ld: warning: cannot find entry symbol _start
```

Fix: Add `global _start` in `.text` section

Wrong format

```
x hello.o: file not recognized: file format not recognized
```

Fix: Use `-f elf64` for 64-bit or `-f elf32` for 32-bit

Segmentation fault

```
Segmentation fault (core dumped)
```

Fix: Check your syscall numbers and register usage

Debug with GDB:

```
nasm -f elf64 -g -F dwarf hello.asm # Add debug info
ld -o hello hello.o
gdb ./hello
```

Practice Exercises

Exercise 1: Modify the Message

Change "Hello, Assembly!" to print your name

Exercise 2: Multiple Prints

Print two different messages (hint: call `sys_write` twice)

Exercise 3: Different Exit Code

Make the program exit with code 42 instead of 0

- Hint: Change the value in `rdi` before the exit syscall
- Verify with `echo $?`

Exercise 4: No Newline

Remove the newline (the `10`) and see what happens to your prompt

Exercise 5: Color Output

Print colored text using ANSI codes:

```
msg db 27, "[31mRed Text!", 27, "[0m", 10 ; 27 = ESC character
```

Quick Reference: Linux 64-bit Syscalls

Syscall	RAX	RDI	RSI	RDX
sys_read	0	fd	buffer	count
sys_write	1	fd	buffer	count
sys_exit	60	exit_code	-	-

File Descriptors:

- 0 = stdin (keyboard input)
- 1 = stdout (console output)
- 2 = stderr (error output)

Knowledge Check

Before moving to Topic 2, make sure you can:

- Assemble and link a NASM program
- Explain the three sections: `.data`, `.bss`, `.text`
- Understand what `syscall` does
- Calculate string length with `$ - label`
- Successfully run your Hello World program

Congratulations! You've written and executed your first assembly program!

Next: [Topic 2: Registers & Data Types](#)

[← Back to Main](#) | [Next Topic →](#)

CHAPTER 19

Topic 2: Registers & Data Types

Welcome to the core of assembly programming! Registers are your fastest storage locations - think of them as variables that live directly in the CPU.

Part 1: What Are Registers?

Registers are small, ultra-fast storage locations built into the CPU.

Speed Comparison:

Location	Access Time	Analogy
Registers	< 1 cycle	Your hands
L1 Cache	~4 cycles	Your desk
RAM	~100 cycles	File room
SSD/HDD	~100,000+	Warehouse

Why use registers?

- Fastest way to store/manipulate data
 - CPU instructions work directly on registers
 - Limited quantity (only 16 general-purpose in x64)
-

Part 2: The x86-64 Register Set

General Purpose Registers (GPRs)

In 64-bit mode, you have **16 general-purpose registers**:

The x86-64 general-purpose register file



ASCII diagram

Original 8 (from 8086 era):

RAX	RBX	RCX	RDY	RSI	RDI	RBP	RSP
-----	-----	-----	-----	-----	-----	-----	-----

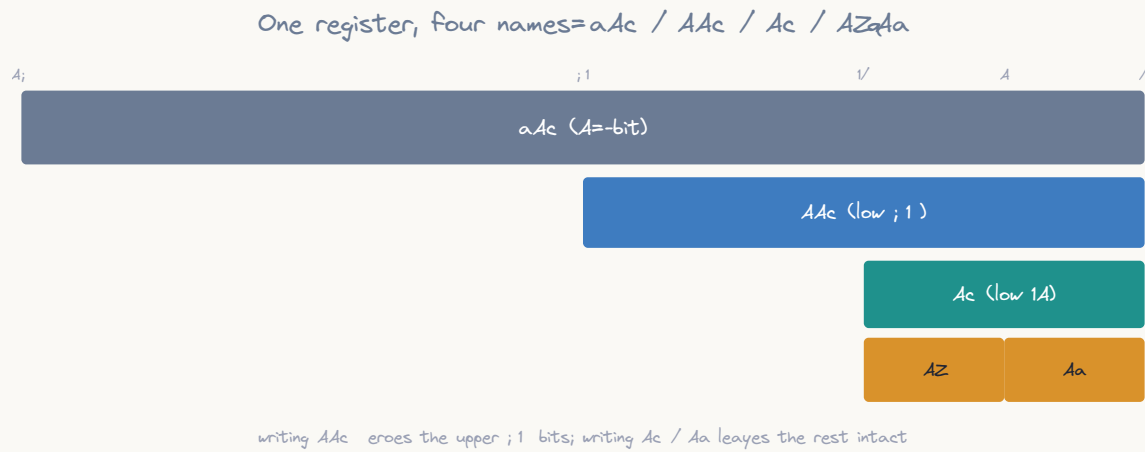
Extended 8 (added in x64):

R8	R9	R10	R11	R12	R13	R14	R15
----	----	-----	-----	-----	-----	-----	-----

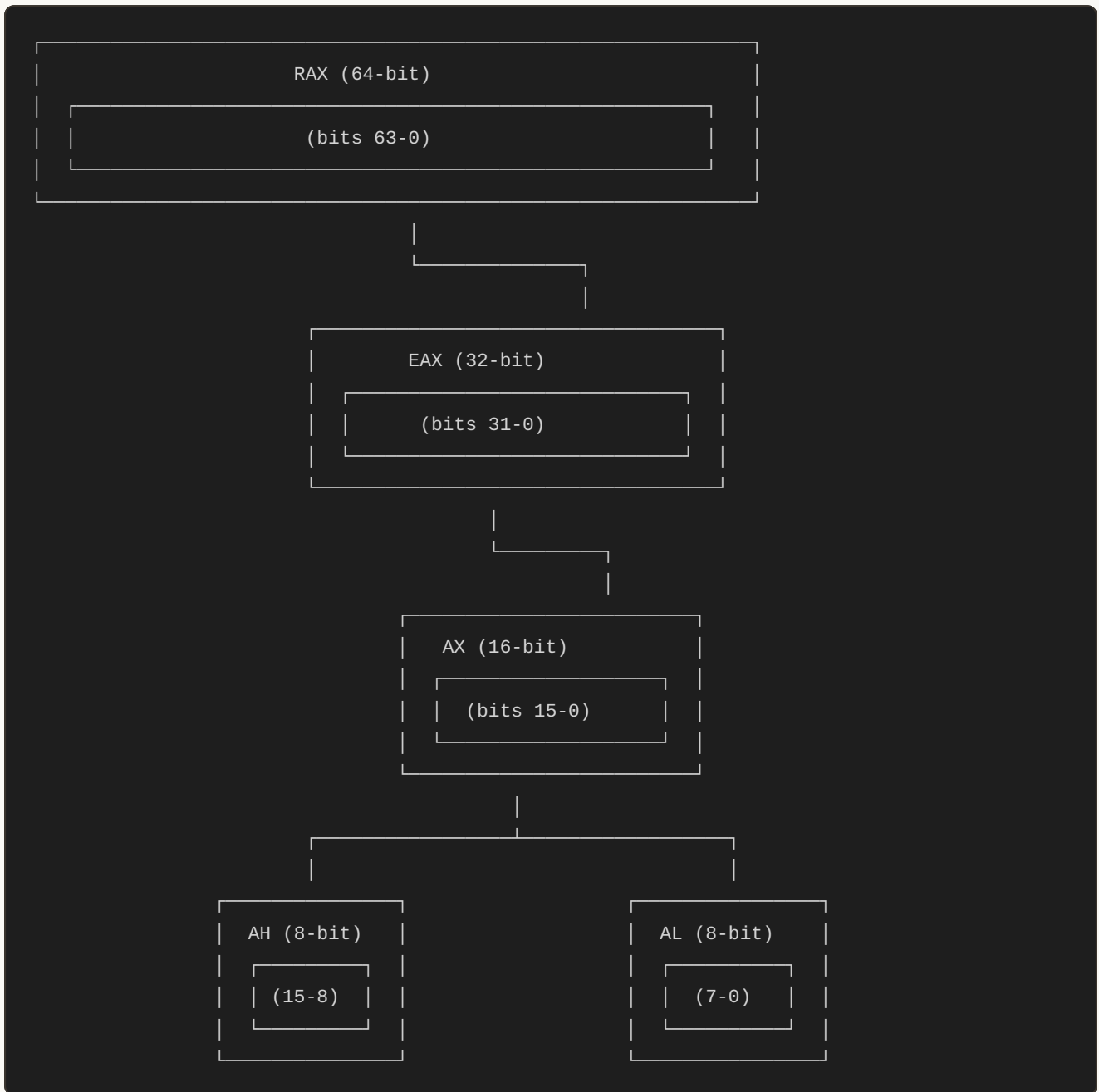
Part 3: Register Sizes - THE KEY CONCEPT!

Each register can be accessed at **different sizes**. Here's the magic:

RAX Register Family Tree



ASCII diagram



All Register Sizes

```

; 64-bit (full register) - "R" prefix
mov rax, 0x123456789ABCDEF0
mov rbx, 0xFFFFFFFFFFFFFFFF
mov rcx, 100

; 32-bit (lower half) - "E" prefix
mov eax, 0x12345678      ; Upper 32 bits of RAX zeroed!
mov ebx, 42
mov ecx, 1000

; 16-bit (lowest 16 bits) - original names
mov ax, 0x1234           ; Doesn't affect upper 48 bits
mov bx, 100
mov cx, 50

; 8-bit (lowest/high byte)
mov al, 0x42             ; Lowest 8 bits of RAX
mov ah, 0x13             ; Bits 8-15 of RAX (only for A,B,C,D)
mov bl, 65               ; ASCII 'A'

```

Complete Register Name Table

64-bit	32-bit	16-bit	8-high	8-low
RAX	EAX	AX	AH	AL
RBX	EBX	BX	BH	BL
RCX	ECX	CX	CH	CL
RDX	EDX	DX	DH	DL
RSI	ESI	SI	-	SIL*
RDI	EDI	DI	-	DIL*
RBP	EBP	BP	-	BPL*
RSP	ESP	SP	-	SPL*
R8	R8D	R8W	-	R8B
R9	R9D	R9W	-	R9B
R10	R10D	R10W	-	R10B
R11	R11D	R11W	-	R11B
R12	R12D	R12W	-	R12B
R13	R13D	R13W	-	R13B
R14	R14D	R14W	-	R14B
R15	R15D	R15W	-	R15B

* SIL, DIL, BPL, SPL available in 64-bit mode only

Part 4: Critical Rule - 32-bit Zeroing

IMPORTANT BEHAVIOR:

```
; Writing to 32-bit register ZEROS upper 32 bits!
mov rax, 0xFFFFFFFFFFFFFFFF ; RAX = 0xFFFFFFFFFFFFFFFF
mov eax, 0x12345678          ; RAX = 0x0000000012345678
                             ; ^^^^^^^^^^ ZEROED!

; Writing to 16-bit or 8-bit PRESERVES upper bits
mov rax, 0xFFFFFFFFFFFFFFFF ; RAX = 0xFFFFFFFFFFFFFFFF
mov ax, 0x1234               ; RAX = 0xFFFFFFFF1234
                             ; ^^^^^^^^^^^^^^^^ PRESERVED

mov rax, 0xFFFFFFFFFFFFFFFF ; RAX = 0xFFFFFFFFFFFFFFFF
mov al, 0x42                 ; RAX = 0xFFFFFFFF42
                             ; ^^^^^^^^^^^^^^^^^^ PRESERVED
```

Why this matters:

```
; Example: Breaking code unintentionally
mov rax, buffer_address ; RAX points to a memory address
mov eax, 5               ; OOPS! Upper 32 bits zeroed
                         ; RAX might now point to invalid memory!
```

Part 5: Register Purposes (Conventions)

While most registers are “general purpose,” they have **traditional roles**:

The Original Eight

```

; RAX - Accumulator (arithmetic, return values)
mov rax, 10
mul rbx                ; Result goes in RAX
; Return value from functions in RAX

; RBX - Base (often saved by functions, general purpose)
mov rbx, array_base
mov al, [rbx + 5]      ; Access array element

; RCX - Counter (loops, string operations)
mov rcx, 10
my_loop:
    ; ... do something ...
    loop my_loop       ; Decrements RCX automatically

; RDX - Data (I/O operations, 128-bit arithmetic)
mul rbx                ; RDX:RAX = RAX * RBX
div rbx                ; Uses RDX:RAX as dividend

; RSI - Source Index (string/memory operations)
mov rsi, source_string
lodsb                  ; Load byte from [RSI] into AL

; RDI - Destination Index (string/memory operations)
mov rdi, dest_string
stosb                  ; Store AL into [RDI]

; RBP - Base Pointer (stack frames, local variables)
push rbp
mov rbp, rsp           ; Standard function prologue
; Access params as [rbp+16], locals as [rbp-4]

; RSP - Stack Pointer (CRITICAL - points to top of stack)
push rax               ; RSP decremented, data stored
pop rbx                ; Data loaded, RSP incremented
; Don't mess with RSP unless you know what you're doing!

```

Calling Convention (System V AMD64 - Linux)

When calling functions, arguments go in specific registers:


```
; Function arguments (in order):
; 1st arg → RDI
; 2nd arg → RSI
; 3rd arg → RDX
; 4th arg → RCX
; 5th arg → R8
; 6th arg → R9
; More args → Stack

; Example: printf(format, arg1, arg2)
mov rdi, format_string      ; 1st arg (format)
mov rsi, 42                 ; 2nd arg
mov rdx, 100                ; 3rd arg
xor rax, rax                ; printf expects RAX=0 (no vector args)
call printf

; Return value always in RAX
```

Part 6: Data Types in NASM

Defining Data

```
section .data
    ; DB - Define Byte (8-bit, 1 byte)
    byte_val    db 0x42                ; Single byte
    char_val    db 'A'                 ; ASCII character
    string_val   db "Hello", 0         ; String with null terminator
    byte_array   db 10, 20, 30, 40     ; Array of bytes

    ; DW - Define Word (16-bit, 2 bytes)
    word_val     dw 0x1234
    word_array    dw 100, 200, 300

    ; DD - Define Doubleword (32-bit, 4 bytes)
    dword_val     dd 0x12345678
    int_val        dd 1000000
    float_val      dd 3.14159          ; 32-bit float

    ; DQ - Define Quadword (64-bit, 8 bytes)
    qword_val     dq 0x123456789ABCDEF0
    long_val       dq 9223372036854775807
    double_val     dq 2.71828182845     ; 64-bit double
    pointer        dq 0                 ; 64-bit pointer

    ; DT - Define Ten Bytes (80-bit, 10 bytes)
    extended       dt 3.14159265358979 ; 80-bit extended precision

    ; Multiple values
    numbers         dd 1, 2, 3, 4, 5    ; 5 doublewords (20 bytes)

    ; TIMES - Repeat
    buffer          times 64 db 0        ; 64 bytes of zeros
    spaces          times 80 db ' '      ; 80 space characters
```

Reserving Uninitialized Data

```
section .bss
    ; RESB - Reserve Bytes
    buffer      resb 256          ; 256 bytes

    ; RESW - Reserve Words
    input       resw 1            ; 1 word (2 bytes)

    ; RESD - Reserve Doublewords
    counter     resd 1            ; 1 dword (4 bytes)
    array       resd 100          ; 100 dwords (400 bytes)

    ; RESQ - Reserve Quadwords
    pointer     resq 1            ; 1 qword (8 bytes)
    big_array   resq 1000         ; 1000 qwords (8000 bytes)
```

Part 7: Practical Examples

Example 1: Working with Different Sizes

C Equivalent:

```
#include <stdint.h>

int main() {
    uint8_t  byte_val  = 0xFF;
    uint16_t word_val  = 0x1234;
    uint32_t dword_val = 0x12345678;
    uint64_t qword_val = 0x123456789ABCDEF0ULL;

    // Load different sizes (C compiler maps these to appropriate registers)
    uint8_t  a8  = byte_val;   // Maps to AL
    uint16_t a16 = word_val;   // Maps to AX
    uint32_t a32 = dword_val;  // Maps to EAX
    uint64_t a64 = qword_val;  // Maps to RAX

    // Note: In C, you work with typed variables.
    // The compiler manages register allocation automatically.

    return 0;
}
```

Assembly:

```

section .data
    byte_val    db 0xFF
    word_val     dw 0x1234
    dword_val    dd 0x12345678
    qword_val    dq 0x123456789ABCDEF0

section .text
    global _start

_start:
    ; Load different sizes
    mov al, [byte_val]      ; AL = 0xFF
    mov ax, [word_val]      ; AX = 0x1234
    mov eax, [dword_val]    ; EAX = 0x12345678
    mov rax, [qword_val]    ; RAX = 0x123456789ABCDEF0

    ; Mix and match
    xor rax, rax            ; RAX = 0
    mov al, 0xFF           ; RAX = 0x00000000000000FF
    mov ah, 0x11           ; RAX = 0x00000000000001FF
    mov ax, 0x2233         ; RAX = 0x0000000000002233
    mov eax, 0x44556677    ; RAX = 0x0000000044556677 (upper cleared!)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: Register Manipulation

C Equivalent (using unions to show bit manipulation):

```

#include <stdint.h>
#include <stdio.h>

// Union allows accessing same memory as different types
union Register {
    uint64_t rax; // 64-bit view
    uint32_t eax; // 32-bit view (low part)
    uint16_t ax;  // 16-bit view (low part)
    struct {
        uint8_t al; // 8-bit low
        uint8_t ah; // 8-bit high
    };
};

int main() {
    union Register reg;

    reg.rax = 0xAAAAAAAAAAAAAAAAULL; // Full 64-bit
    // In real CPU: mov eax zeros upper 32 bits
    // C simulation:
    reg.eax = 0xBBBBBBBB; // Only affects lower 32
    reg.rax &= 0xFFFFFFFF; // Manually zero upper (CPU does auto)

    reg.rax = 0xCCCCCCCCCCCCCCCCULL;
    reg.ax = 0xDDDD; // Only affects lower 16

    return 0;
}

```

Assembly:

```

section .text
    global _start

_start:
    ; Demonstrate register sizes
    mov rax, 0xAAAAAAAAAAAAAAAA    ; RAX = 0xAAAAAAAAAAAAAAAA
    mov eax, 0BBBBBBBB             ; RAX = 0x00000000BBBBBBBB

    mov rax, 0CCCCCCCCCCCCCCCC    ; RAX = 0xCCCCCCCCCCCCCCCC
    mov ax, 0xDDDD                ; RAX = 0xCCCCCCCCCCCCDDDD

    mov rax, 0x1122334455667788    ; RAX = 0x1122334455667788
    mov al, 0x99                   ; RAX = 0x1122334455667799
    mov ah, 0xAA                   ; RAX = 0x112233445566AA99

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 3: Using Register Roles

C Equivalent:

```

#include <stdint.h>

int main() {
    int32_t numbers[] = {5, 10, 15, 20, 25};
    int32_t count = 5;
    int32_t sum = 0;

    // Sum array - compiler typically uses:
    // - accumulator register for sum (RAX)
    // - counter register for index (RCX)
    for (int i = 0; i < count; i++) {
        sum += numbers[i];
    }

    // sum now equals 75
    return 0;
}

```

Assembly:

```

section .data
    numbers dd 5, 10, 15, 20, 25    ; Array of 5 numbers
    count   equ 5

section .bss
    sum resd 1

section .text
    global _start

_start:
    ; Sum array using traditional register roles
    xor rax, rax                    ; RAX = accumulator (sum)
    mov rbx, numbers                ; RBX = base (array address)
    xor rcx, rcx                    ; RCX = counter (index)
    mov rdx, count                  ; RDX = data (loop limit)

sum_loop:
    add eax, [rbx + rcx*4]          ; Add numbers[rcx] to sum
    inc rcx                        ; Increment counter
    cmp rcx, rdx                   ; Compare with limit
    jl sum_loop                    ; Loop if less

    ; Store result
    mov [sum], eax                 ; sum = 75

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

x

Part 8: Common Mistakes

Mistake 1: Size Mismatch

```

section .data
    value dd 0x12345678            ; 32-bit value

section .text
    mov al, [value]                ; ✓ Loads 0x78 (first byte)
    mov ax, [value]                ; ✓ Loads 0x5678 (first word)
    mov eax, [value]               ; ✓ Loads 0x12345678 (full dword)
    mov rax, [value]               ;   Loads 8 bytes (past end of value!)

```

Mistake 2: Forgetting 32-bit Zeroing

x

```
mov rax, 0xFFFFFFFFFFFFFFFF
mov eax, 1                ; RAX = 0x0000000000000001, not 0xFFFFFFFF00000001!
```

Mistake 3: Using Undefined Registers

```
mov r16, 100              ; ERROR: No such register!
mov rxc, 50               ; ERROR: It's RCX, not RxC!
```

Practice Exercises

Exercise 1: Register Explorer

Write a program that:

1. Loads 0xFFFFFFFFFFFFFFFF into RAX
2. Moves 0x11223344 into EAX
3. Print RAX's value (you'll see upper 32 bits are zero)

Exercise 2: Byte Manipulation

```
; Start with RAX = 0
; Set AL = 'H' (0x48)
; Set AH = 'i' (0x69)
; What is the value of AX?
```

Exercise 3: Array Access

Create an array of 5 bytes: `[10, 20, 30, 40, 50]`

- Load the 3rd element (30) into AL using RBX as base register

Exercise 4: Size Detective

```
section .data
    mystery db 0x12, 0x34, 0x56, 0x78

section .text
    mov al, [mystery]      ; AL = ?
    mov ax, [mystery]      ; AX = ?
    mov eax, [mystery]     ; EAX = ?
```


What are the values? (Hint: Little-endian!)

Exercise 5: Data Definition

Define these in `.data` section:

- Your age (1 byte)
- Current year (2 bytes)
- Your favorite number (4 bytes)
- A 10-byte buffer initialized to zeros

Quick Reference Card

REGISTER SIZES			
Size	Suffix	Bits	Example
Byte	B	8	AL, BL, R8B
Word	W	16	AX, BX, R8W
Dword	D	32	EAX, R8D
Qword	(none)	64	RAX, R8

DATA TYPES			
Define	Reserve	Size	Purpose
DB	RESB	1 byte	char, byte
DW	RESW	2 bytes	short
DD	RESD	4 bytes	int, float
DQ	RESQ	8 bytes	long, double

Knowledge Check

Before moving to Topic 3, verify you understand:

- The 16 general-purpose registers
- How to access RAX as EAX, AX, AH, AL
- That 32-bit writes zero the upper 32 bits
- Traditional register roles (RSP, RBP, RCX, etc.)

- DB, DW, DD, DQ data types
 - RESB, RESW, RESD, RESQ for uninitialized data
-

Excellent! You now understand the CPU's register architecture!

Next: Topic 3: Basic Instructions (Coming soon!)

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 20

Topic 3: Basic Instructions

Master the fundamental assembly instructions you'll use every day.

Overview

Assembly instructions are the building blocks of your programs. In this topic, you'll learn the most essential instructions for:

- Moving data around
- Performing arithmetic
- Manipulating bits
- Working with memory addresses

Part 1: The MOV Instruction

MOV is the most fundamental instruction - it copies data from source to destination.

Basic Syntax

```
mov destination, source ; destination = source
```

Important Rules

```
;   ALLOWED combinations:
mov reg, reg      ; Register to register
mov reg, mem      ; Memory to register
mov mem, reg      ; Register to memory
mov reg, imm      ; Immediate to register
mov mem, imm      ; Immediate to memory

;   NOT ALLOWED:
mov mem, mem      ; Cannot move memory to memory directly!
mov seg, seg      ; Cannot move segment to segment
```

MOV Examples

```

section .data
    value dq 42
    buffer resb 64

section .text
    ; Register to register
    mov rax, rbx          ; RAX = RBX
    mov ecx, edx          ; ECX = EDX (32-bit)
    mov al, bl            ; AL = BL (8-bit)

    ; Immediate to register
    mov rax, 100          ; RAX = 100
    mov eax, 0x12345678   ; EAX = 0x12345678
    mov cl, 'A'           ; CL = 65 (ASCII 'A')

    ; Memory to register
    mov rax, [value]      ; RAX = contents of value
    mov al, [buffer]      ; AL = first byte of buffer
    mov ecx, [value + 4]  ; ECX = dword at value+4

    ; Register to memory
    mov [buffer], rax     ; Store RAX at buffer
    mov [value], rbx      ; Store RBX at value

    ; Immediate to memory (must specify size!)
    mov qword [buffer], 42 ; Store 42 as quadword
    mov dword [buffer], 42 ; Store 42 as dword
    mov byte [buffer], 42  ; Store 42 as byte

```

MOV Gotchas

```

; Size ambiguity
mov [buffer], 42      ; ERROR: What size?

; Fix: Specify size
mov byte [buffer], 42 ; Clear: 1 byte
mov qword [buffer], 42 ; Clear: 8 bytes

; Cannot move memory to memory
mov [dest], [source] ; ERROR!

; Fix: Use intermediate register
mov rax, [source]
mov [dest], rax

; Watch 32-bit zeroing!
mov rax, 0xFFFFFFFFFFFF ; RAX = all F's
mov eax, 0x11111111      ; RAX = 0x0000000011111111
                        ; ^^^^^^^^ zeroed!

```

Part 2: Arithmetic Instructions

ADD - Addition

```
add destination, source ; destination = destination + source
```

Examples:

```

; Register arithmetic
mov rax, 10
add rax, 5 ; RAX = 15
add rax, rbx ; RAX = RAX + RBX

; Memory arithmetic
add qword [counter], 1 ; Increment counter in memory
add rax, [value] ; RAX = RAX + [value]

; Sets flags!
mov al, 255
add al, 1 ; AL = 0, Carry Flag (CF) = 1

```

Flags affected: CF, ZF, SF, OF, AF, PF

SUB - Subtraction

```
sub destination, source      ; destination = destination - source
```

Examples:

```
; Basic subtraction
mov rax, 100
sub rax, 30          ; RAX = 70
sub rax, rbx         ; RAX = RAX - RBX

; Detect underflow
mov al, 5
sub al, 10           ; AL = 251 (wraps around), CF = 1

; Compare without storing (common pattern)
mov rax, 10
sub rax, rax         ; RAX = 0, ZF = 1 (zero result)
```

Flags affected: CF, ZF, SF, OF, AF, PF

INC - Increment

```
inc destination          ; destination = destination + 1
```

Examples:

```
; Increment registers
mov rax, 10
inc rax              ; RAX = 11
inc ecx             ; ECX = ECX + 1

; Increment memory
inc qword [counter] ; counter = counter + 1
inc byte [buffer]   ; First byte of buffer + 1

; Loop counter pattern
mov rcx, 0
loop_start:
    ; ... do work ...
    inc rcx
    cmp rcx, 10
    jl loop_start
```

Note: INC is shorter than `add reg, 1` but doesn't affect CF!

Flags affected: ZF, SF, OF, AF, PF (but NOT CF)

DEC - Decrement

```
dec destination      ; destination = destination - 1
```

Examples:

```
; Decrement registers
mov rax, 10
dec rax              ; RAX = 9

; Countdown loop
mov rcx, 10
countdown:
    ; ... do work ...
    dec rcx
    jnz countdown    ; Jump if not zero
```

Flags affected: ZF, SF, OF, AF, PF (but NOT CF)

NEG - Negate (Two's Complement)

```
neg destination      ; destination = -destination
```

Examples:

```
mov rax, 10
neg rax              ; RAX = -10 (0xFFFFFFFFFFFFFFF6)

mov al, 5
neg al               ; AL = -5 (0xFB = 251 unsigned)

; Absolute value pattern
; if (rax < 0) rax = -rax;
test rax, rax        ; Check sign
jns skip_neg         ; Jump if not negative
neg rax
skip_neg:
```

Flags affected: CF, ZF, SF, OF, AF, PF

MUL/IMUL - Multiplication

MUL - Unsigned multiply

IMUL - Signed multiply

Single Operand Form (MUL/IMUL)

```
mul source           ; RDX:RAX = RAX * source (unsigned)
imul source          ; RDX:RAX = RAX * source (signed)
```

Examples:

```
; Unsigned multiply
mov rax, 10
mov rbx, 20
mul rbx           ; RDX:RAX = 200 (result in RAX, RDX = 0)

; 64-bit result
mov rax, 0xFFFFFFFFFFFFFFFF
mov rbx, 2
mul rbx           ; RDX:RAX = 0x1FFFFFFFFFFFFFFFE
                   ; RDX = 0x1, RAX = 0xFFFFFFFFFFFFFFFE

; Signed multiply
mov rax, -5
mov rbx, 3
imul rbx          ; RAX = -15
```

Two Operand Form (IMUL only)

```
imul dest, source   ; dest = dest * source (truncated)
```

Examples:

```
mov rax, 10
imul rax, 5         ; RAX = 50

mov ecx, 100
imul ecx, edx       ; ECX = ECX * EDX
```

Three Operand Form (IMUL only)

```
imul dest, source, imm ; dest = source * immediate
```

Examples:


```
imul rax, rbx, 10      ; RAX = RBX * 10
imul ecx, edx, 100     ; ECX = EDX * 100

; Quick scaling
imul rax, rsi, 4        ; RAX = RSI * 4 (array index)
```

DIV/IDIV - Division

DIV - Unsigned division

IDIV - Signed division

```
div divisor            ; RAX = RDX:RAX / divisor (quotient)
                      ; RDX = RDX:RAX % divisor (remainder)

idiv divisor           ; Same but signed
```

Important: Must set up RDX:RAX correctly before dividing!

Examples:

```
; Unsigned division
mov rax, 100
xor rdx, rdx          ; Clear RDX (upper 64 bits)
mov rbx, 10
div rbx               ; RAX = 10 (quotient), RDX = 0 (remainder)

; Division with remainder
mov rax, 23
xor rdx, rdx
mov rbx, 5
div rbx               ; RAX = 4, RDX = 3 (23 = 5*4 + 3)

; Signed division
mov rax, -20
cqo                  ; Sign-extend RAX into RDX
mov rbx, 3
idiv rbx             ; RAX = -6, RDX = -2

; 32-bit division
mov eax, 100
xor edx, edx          ; Clear EDX
mov ecx, 10
div ecx              ; EAX = 10, EDX = 0
```

Critical: Always clear/set RDX before DIV/IDIV!

```

;   WRONG - RDX has garbage
mov rax, 100
div rbx           ; Divides RDX:RAX (huge number!)

;   CORRECT - Clear RDX for unsigned
mov rax, 100
xor rdx, rdx
div rbx

;   CORRECT - Sign-extend for signed
mov rax, -100
cqo              ; RDX = 0xFFFFFFFFFFFFFFFF if RAX < 0
idiv rbx

```

Part 3: Bitwise Instructions

AND - Bitwise AND

```
and destination, source ; destination = destination & source
```

Truth Table:

```

0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1

```

Examples:

```

; Masking bits
mov rax, 0b11111111
and rax, 0b00001111 ; RAX = 0b00001111 (keep lower 4 bits)

; Check if even (bit 0 = 0)
mov rax, 42
and rax, 1          ; RAX = 0 (even), ZF = 1

; Clear specific bits
mov rax, 0xFF
and rax, 0xF0       ; RAX = 0xF0 (clear lower 4 bits)

; Alignment check (is address 16-byte aligned?)
mov rax, address
and rax, 0xF        ; If RAX = 0, address is aligned

```

Flags affected: ZF, SF, PF (CF and OF cleared)

OR - Bitwise OR

```
or destination, source ; destination = destination | source
```

Truth Table:

```
0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1
```

Examples:

```
; Set specific bits
mov rax, 0b00000000
or rax, 0b00001111 ; RAX = 0b00001111 (set lower 4 bits)

; Combine flags
mov rax, 0x01 ; Flag bit 0
mov rbx, 0x04 ; Flag bit 2
or rax, rbx ; RAX = 0x05 (both flags set)

; Test for zero (common idiom)
or rax, rax ; Sets ZF if RAX = 0 (doesn't change RAX)
jz is_zero
```

Flags affected: ZF, SF, PF (CF and OF cleared)

XOR - Bitwise XOR (Exclusive OR)

```
xor destination, source ; destination = destination ^ source
```

Truth Table:

```
0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0 ← Same bits = 0
```

Examples:

```

; Toggle bits
mov rax, 0b11110000
xor rax, 0b11111111    ; RAX = 0b00001111 (all bits toggled)

; Zero a register (most efficient!)
xor rax, rax           ; RAX = 0 (any value XOR itself = 0)
xor eax, eax           ; EAX = 0 (also zeros upper 32 bits)

; Swap without temp variable!
mov rax, 5
mov rbx, 10
xor rax, rbx           ; RAX = 5 ^ 10
xor rbx, rax           ; RBX = 10 ^ (5 ^ 10) = 5
xor rax, rbx           ; RAX = (5 ^ 10) ^ 5 = 10
; Now: RAX = 10, RBX = 5

; Simple encryption (XOR cipher)
mov al, 'A'            ; AL = 65
xor al, 0x42           ; Encrypt: AL = 39
xor al, 0x42           ; Decrypt: AL = 65 ('A' again!)

```

Flags affected: ZF, SF, PF (CF and OF cleared)

NOT - Bitwise NOT (One's Complement)

```
not destination    ; destination = ~destination
```

Examples:

```

; Invert all bits
mov rax, 0b00001111
not rax              ; RAX = 0b...11110000 (all bits flipped)

mov al, 0x0F
not al              ; AL = 0xF0

; Create mask
mov rax, 0x00FF     ; Mask for lower byte
not rax              ; RAX = 0x...FF00 (inverted mask)

```

Flags affected: None!

TEST - Logical Compare (AND without storing)

```
test operand1, operand2 ; Performs AND, sets flags, doesn't store result
```

Examples:

```
; Check if zero
test rax, rax          ; ZF = 1 if RAX = 0
jz is_zero

; Check specific bit
test rax, 0x01         ; Check bit 0
jnz bit_is_set

; Check if negative (sign bit)
test rax, rax          ; SF = 1 if RAX < 0 (signed)
js is_negative

; Check multiple bits
test al, 0b11110000    ; Check if any of upper 4 bits are set
jnz has_upper_bits
```

Flags affected: ZF, SF, PF (CF and OF cleared)

Why use TEST instead of AND?

- Doesn't modify the operand
- Shorter encoding in some cases
- More readable intent (testing, not modifying)

Part 4: Shift and Rotate Instructions

SHL/SHR - Logical Shifts

```
shl destination, count ; Shift left (multiply by 2^count)
shr destination, count ; Shift right (divide by 2^count, unsigned)
```

Visual:

```

SHL (Shift Left):
Before: 0000 1010 (10)
SHL 1:  0001 0100 (20) ← bit shifted out to CF
SHL 2:  0010 1000 (40)

SHR (Shift Right):
Before: 0000 1010 (10)
SHR 1:  0000 0101 (5)  ← bit shifted out to CF
SHR 2:  0000 0010 (2)

```

Examples:

```

; Multiply by powers of 2
mov rax, 10
shl rax, 1      ; RAX = 20 (10 * 2)
shl rax, 2      ; RAX = 80 (20 * 4)

; Divide by powers of 2 (unsigned)
mov rax, 100
shr rax, 1      ; RAX = 50 (100 / 2)
shr rax, 2      ; RAX = 12 (50 / 4)

; Extract high bits
mov al, 0b11010110
shr al, 4        ; AL = 0b00001101 (upper 4 bits moved down)

; Shift by CL register
mov cl, 3
mov rax, 1
shl rax, cl      ; RAX = 8 (1 << 3)

; Array indexing (multiply by element size)
mov rax, index
shl rax, 3        ; RAX = index * 8 (for 8-byte elements)

```

SAL/SAR - Arithmetic Shifts

```

sal destination, count ; Shift Arithmetic Left (same as SHL)
sar destination, count ; Shift Arithmetic Right (preserves sign bit)

```

SAR preserves the sign bit:

SAR on positive number:

Before: 0000 1010 (10)

SAR 1: 0000 0101 (5)

SAR on negative number:

Before: 1111 0110 (-10 in two's complement)

SAR 1: 1111 1011 (-5) ← sign bit duplicated

Examples:

```
; Signed division by powers of 2
mov rax, -16
sar rax, 2           ; RAX = -4 (preserves sign)

; Compare with SHR (unsigned shift)
mov rax, -16         ; RAX = 0xFFFFFFFFFFFFF0
shr rax, 2           ; RAX = 0x3FFFFFFFFFFFFC (huge positive!)

mov rax, -16
sar rax, 2           ; RAX = 0xFFFFFFFFFFFFFC (correct: -4)
```

ROL/ROR - Rotate (No Bits Lost)

```
rol destination, count ; Rotate left (bits wrap around)
ror destination, count ; Rotate right
```

Visual:

ROL (Rotate Left):

Before: 1010 0001

ROL 1: 0100 0011 ← MSB wraps to LSB

ROR (Rotate Right):

Before: 1010 0001

ROR 1: 1101 0000 ← LSB wraps to MSB

Examples:

```

; Circular rotation
mov al, 0b10000001
rol al, 1           ; AL = 0b00000011
rol al, 1           ; AL = 0b00000110

; Endianness conversion (byte swap)
mov eax, 0x12345678
rol eax, 8          ; EAX = 0x34567812
rol eax, 8          ; EAX = 0x56781234

```

Part 5: LEA - Load Effective Address

LEA is a special instruction that computes an address but doesn't access memory.

```
lea destination, [address] ; destination = address (not contents!)
```

LEA vs MOV

```

section .data
    array dq 10, 20, 30, 40

section .text
    ; MOV loads the VALUE
    mov rax, [array]      ; RAX = 10 (value at array)

    ; LEA loads the ADDRESS
    lea rax, [array]      ; RAX = address of array

    ; Equivalent to:
    mov rax, array        ; Same as LEA for simple labels

```

LEA for Arithmetic (No Memory Access!)

This is where LEA shines - it uses the addressing mode calculator for math:


```

; Traditional arithmetic
mov rax, rbx
add rax, rcx
add rax, 8
; Total: 3 instructions

; With LEA - one instruction!
lea rax, [rbx + rcx + 8]    ; RAX = RBX + RCX + 8

; Multiply by 2, 3, 4, 5, 8, 9
lea rax, [rbx * 2]         ; RAX = RBX * 2
lea rax, [rbx + rbx*2]     ; RAX = RBX * 3
lea rax, [rbx * 4]         ; RAX = RBX * 4
lea rax, [rbx + rbx*4]     ; RAX = RBX * 5
lea rax, [rbx * 8]         ; RAX = RBX * 8
lea rax, [rbx + rbx*8]     ; RAX = RBX * 9

; Complex calculation
lea rax, [rbx + rcx*4 + 100]; RAX = RBX + (RCX * 4) + 100

```

LEA Advantages

```

; 1. No memory access (faster)
lea rax, [rbx + 8]         ; Just arithmetic, no memory read

; 2. Doesn't affect flags (unlike ADD)
add rax, rbx               ; Affects CF, ZF, SF, OF, AF, PF
lea rax, [rax + rbx]       ; Affects NO flags!

; 3. Three-operand arithmetic
add rax, rbx               ; RAX = RAX + RBX (destroys old RAX)
lea rax, [rbx + rcx]       ; RAX = RBX + RCX (preserves RBX and RCX)

; 4. Combined operations
; Traditional:
mov rax, index
shl rax, 3
add rax, base
add rax, 16

; With LEA:
lea rax, [base + index*8 + 16] ; One instruction!

```

Practical LEA Examples

```

; Array element address
; address = base + index * sizeof(element)
lea rax, [array + rcx*8]    ; 8-byte elements

; String offset
lea rsi, [string + 5]      ; Point to 6th character

; Struct member
; struct.field at offset 16
lea rax, [rbx + 16]

; Scale and offset
; result = x * 5 + 10
lea rax, [rdi + rdi*4 + 10] ; x + x*4 + 10 = x*5 + 10

```

Part 6: Comparison Instructions

CMP - Compare (SUB without storing)

```

cmp operand1, operand2    ; Performs SUB, sets flags, doesn't store

```

Examples:

```

; Basic comparison
cmp rax, 10                ; Compare RAX with 10
je equal                   ; Jump if RAX == 10
jne not_equal              ; Jump if RAX != 10
jg greater                 ; Jump if RAX > 10 (signed)
jl less                    ; Jump if RAX < 10 (signed)

; Compare registers
cmp rax, rbx
jge rax_greater_or_equal   ; Jump if RAX >= RBX

; Compare with memory
cmp rax, [value]
je values_match

; Zero check
cmp rax, 0                  ; Check if zero
jz is_zero

; Better: test rax, rax    ; More efficient!

```

How CMP works:

- Computes `operand1 - operand2`
 - Sets flags based on result
 - Doesn't store the result
 - Use conditional jumps after CMP
-

Part 7: Practical Examples

Example 1: Sum Array Elements

C Equivalent:

```
#include <stdint.h>

int main() {
    int32_t numbers[] = {10, 20, 30, 40, 50};
    int32_t count = 5;
    int32_t sum = 0;

    for (int32_t index = 0; index < count; index++) {
        sum += numbers[index];
    }

    // sum = 150
    return 0;
}
```

Assembly:

```

section .data
    numbers dd 10, 20, 30, 40, 50
    count equ 5

section .bss
    sum resd 1

section .text
    global _start

_start:
    xor eax, eax          ; sum = 0
    xor ecx, ecx          ; index = 0

sum_loop:
    add eax, [numbers + rcx*4] ; sum += numbers[index]
    inc ecx                ; index++
    cmp ecx, count         ; index < count?
    jl sum_loop            ; Loop if less

    mov [sum], eax         ; Store result (150)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: Find Maximum

C Equivalent:

```

#include <stdint.h>

int main() {
    int32_t numbers[] = {15, 42, 7, 89, 23};
    int32_t count = 5;
    int32_t max = numbers[0];

    for (int32_t index = 1; index < count; index++) {
        if (numbers[index] > max) {
            max = numbers[index];
        }
    }

    // max = 89
    return 0;
}

```

Assembly:

```

section .data
    numbers dd 15, 42, 7, 89, 23
    count equ 5

section .bss
    max resd 1

section .text
    global _start

_start:
    mov eax, [numbers]      ; max = numbers[0]
    mov ecx, 1              ; index = 1

find_max:
    cmp eax, [numbers + rcx*4] ; max < numbers[index]?
    jge skip_update
    mov eax, [numbers + rcx*4] ; max = numbers[index]

skip_update:
    inc ecx
    cmp ecx, count
    jl find_max

    mov [max], eax          ; Store result (89)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 3: Factorial**C Equivalent:**

```
#include <stdint.h>

int main() {
    int64_t n = 5;
    int64_t result = 1;

    while (n != 0) {
        result *= n;
        n--;
    }

    // result = 120 (5!)
    return result;
}
```

Assembly:

```
; Calculate factorial of 5
section .text
    global _start

_start:
    mov rcx, 5           ; n = 5
    mov rax, 1           ; result = 1

factorial_loop:
    imul rax, rcx        ; result *= n
    dec rcx              ; n--
    jnz factorial_loop   ; Loop if n != 0

    ; RAX now contains 120 (5!)

    ; Exit
    mov rdi, rax         ; Exit with factorial as code
    mov rax, 60
    syscall
```

Example 4: Bit Manipulation**C Equivalent:**

```

#include <stdint.h>

int main() {
    uint64_t value = 0b10110101;

    // Set bit 3
    value |= 0b00001000;           // value = 0b10111101

    // Clear bit 5
    value &= ~0b00100000;          // value = 0b10011101

    // Toggle bit 0
    value ^= 0b00000001;           // value = 0b10011100

    // Test bit 4
    if (value & 0b00010000) {      // Check if bit 4 is set
        // bit4_is_set
    }

    return 0;
}

```

Assembly:

```

section .text
    global _start

_start:
    mov rax, 0b10110101

    ; Set bit 3
    or rax, 0b00001000           ; RAX = 0b10111101

    ; Clear bit 5
    and rax, ~0b00100000         ; RAX = 0b10011101

    ; Toggle bit 0
    xor rax, 0b00000001         ; RAX = 0b10011100

    ; Test bit 4
    test rax, 0b00010000         ; ZF = 0 (bit is set)
    jnz bit4_is_set

bit4_is_set:
    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 5: Using LEA

C Equivalent:

```
#include <stdint.h>

int main() {
    // Calculate: result = x * 5 + y * 3 + 10
    int64_t x = 4;
    int64_t y = 7;

    int64_t result = x * 5 + y * 3 + 10;

    // result = 51
    return result;
}
```

Assembly:

```
section .text
    global _start

_start:
    ; Calculate: result = x * 5 + y * 3 + 10
    ; where x = 4, y = 7

    mov rdi, 4          ; x = 4
    mov rsi, 7          ; y = 7

    ; x * 5 using LEA
    lea rax, [rdi + rdi*4] ; RAX = x * 5 = 20

    ; y * 3 using LEA
    lea rbx, [rsi + rsi*2] ; RBX = y * 3 = 21

    ; Combine and add 10
    lea rax, [rax + rbx + 10] ; RAX = 20 + 21 + 10 = 51

    ; Exit with result
    mov rdi, rax
    mov rax, 60
    syscall
```


Part 8: Performance Tips

Choose the Right Instruction

```

; Zeroing a register:
xor eax, eax      ; Best: 2 bytes, fast
mov eax, 0        ; Worse: 5 bytes, slower

; Incrementing:
inc rax           ; Good: 3 bytes
add rax, 1        ; Longer: 4 bytes
lea rax, [rax + 1] ; Doesn't affect flags, but longer

; Testing for zero:
test rax, rax     ; Best: clear intent
cmp rax, 0        ; Longer encoding
or rax, rax       ; Also good, but TEST is clearer

; Multiplying by constant power of 2:
shl rax, 3        ; Best: RAX * 8
imul rax, 8       ; Slower, longer latency

; Small multiplications:
lea rax, [rax + rax*2] ; Best: RAX * 3
imul rax, 3         ; Slower

```

Avoid False Dependencies

```

; BAD: False dependency
mov eax, [mem1]
add eax, 10
mov eax, [mem2]      ; CPU must wait for previous MOV

; GOOD: Break dependency with XOR
mov eax, [mem1]
add eax, 10
xor eax, eax         ; Breaks dependency chain
mov eax, [mem2]      ; Can start immediately

```

Part 9: Common Patterns

Clear a Register

```
xor rax, rax      ; Preferred
xor eax, eax      ; Even better (shorter, zeros full RAX)
```

Set Register to -1

```
mov rax, -1       ; Works but longer
or rax, -1        ; Same result
xor rax, rax      ; 2 bytes
dec rax           ; 3 bytes total (0 - 1 = -1)
```

Conditional Move

```
; if (rax > rbx) rax = rbx;
cmp rax, rbx
jle skip
mov rax, rbx
skip:

; Better (on modern CPUs): CMOVcc
cmp rax, rbx
cmovg rax, rbx      ; Conditional move if greater
```

Absolute Value

```
; abs(rax)
test rax, rax
jns skip_neg
neg rax
skip_neg:

; Alternative using CDQ tricks (for EAX)
mov edx, eax
sar edx, 31      ; EDX = all 1's if negative, 0 if positive
xor eax, edx
sub eax, edx      ; EAX = abs(EAX)
```

Min/Max

```

; min(rax, rbx) -> rax
cmp rax, rbx
jle already_min
mov rax, rbx
already_min:

; max(rax, rbx) -> rax
cmp rax, rbx
jge already_max
mov rax, rbx
already_max:

```

Practice Exercises

Exercise 1: Basic Operations

Write code to:

1. Set RAX to 100
2. Add 50 to RAX
3. Multiply RAX by 2
4. Subtract 75
5. What's the final value?

Solution

```

mov rax, 100      ; RAX = 100
add rax, 50       ; RAX = 150
shl rax, 1        ; RAX = 300 (multiply by 2)
sub rax, 75       ; RAX = 225

; Final answer: 225

```

Exercise 2: Bit Manipulation

Given AL = 0b10110100:

1. Set bit 0
2. Clear bit 6
3. Toggle bit 3
4. What's the final binary value?

Solution

```

mov al, 0b10110100
or al, 0b00000001      ; Set bit 0: 0b10110101
and al, ~0b01000000    ; Clear bit 6: 0b00110101
xor al, 0b00001000     ; Toggle bit 3: 0b00111101

; Final answer: 0b00111101 (0x3D)

```

Exercise 3: Average of Two Numbers

Calculate average of RAX and RBX (assume they won't overflow).

Solution

```

; Method 1: Simple
mov rcx, rax
add rcx, rbx
shr rcx, 1      ; Divide by 2

; Method 2: Overflow-safe
mov rcx, rax
and rcx, rbx    ; Common bits
xor rax, rbx    ; Differing bits
shr rax, 1      ; Half the difference
add rax, rcx    ; Average

```

Exercise 4: Count Set Bits

Count how many bits are set in AL (population count).

Solution (simple loop)

```

xor rcx, rcx      ; count = 0
mov rbx, 8        ; bits to check

count_loop:
    test al, 1    ; Check lowest bit
    jz skip_count
    inc rcx       ; Count if set
skip_count:
    shr al, 1     ; Next bit
    dec rbx
    jnz count_loop

; RCX contains count

```

Exercise 5: Using LEA

Calculate: $\text{result} = (x * 9) + (y * 5) + 20$ Where $x = \text{RDI}$, $y = \text{RSI}$ Use only LEA instructions for multiplication.

Solution

```

lea rax, [rdi + rdi*8]    ; x * 9
lea rbx, [rsi + rsi*4]    ; y * 5
lea rax, [rax + rbx + 20] ; Combine and add 20
; Final answer in RAX

```

Quick Reference

Data Movement

```

mov dest, src      ; Copy data
lea dest, [addr]   ; Load address (no memory access)
xchg op1, op2      ; Swap

```

Arithmetic

```

add dest, src      ; Addition
sub dest, src      ; Subtraction
inc dest           ; Increment
dec dest           ; Decrement
neg dest           ; Negate
mul src            ; Unsigned multiply
imul src/dest,src  ; Signed multiply
div src            ; Unsigned divide
idiv src           ; Signed divide

```

Bitwise

```

and dest, src      ; Bitwise AND
or dest, src        ; Bitwise OR
xor dest, src       ; Bitwise XOR
not dest           ; Bitwise NOT
test op1, op2       ; AND without storing

```

Shifts

```
shl dest, count    ; Shift left (unsigned)
shr dest, count    ; Shift right (unsigned)
sal dest, count    ; Shift arithmetic left
sar dest, count    ; Shift arithmetic right (preserves sign)
rol dest, count    ; Rotate left
ror dest, count    ; Rotate right
```

Comparison

```
cmp op1, op2      ; Compare (SUB without storing)
```

Knowledge Check

Before moving to Topic 4, verify you understand:

- MOV and its restrictions (no mem-to-mem)
- ADD, SUB, INC, DEC
- MUL/IMUL/DIV/IDIV and setting up RDX:RAX
- AND, OR, XOR, NOT, TEST
- Shifts (SHL, SHR, SAL, SAR, ROL, ROR)
- LEA for arithmetic without memory access
- CMP for comparisons
- When to use which instruction

Excellent! You now know the essential assembly instructions!

Next: Topic 4: Flags & Comparisons (coming soon)

← [Previous Topic](#) | [Back to Main](#) | [Next Topic](#) →

CHAPTER 21

Topic 4: Flags & Comparisons

Understanding the CPU's status flags - the foundation of conditional logic.

Overview

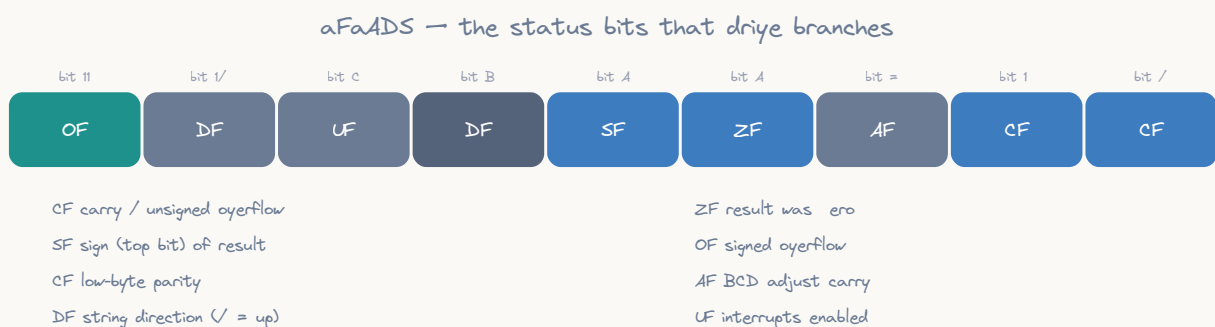
The **FLAGS register** (RFLAGS in 64-bit) is a special register that contains individual bit flags indicating the results of operations. These flags are how the CPU remembers what happened in the last operation, enabling conditional execution.

Why flags matter:

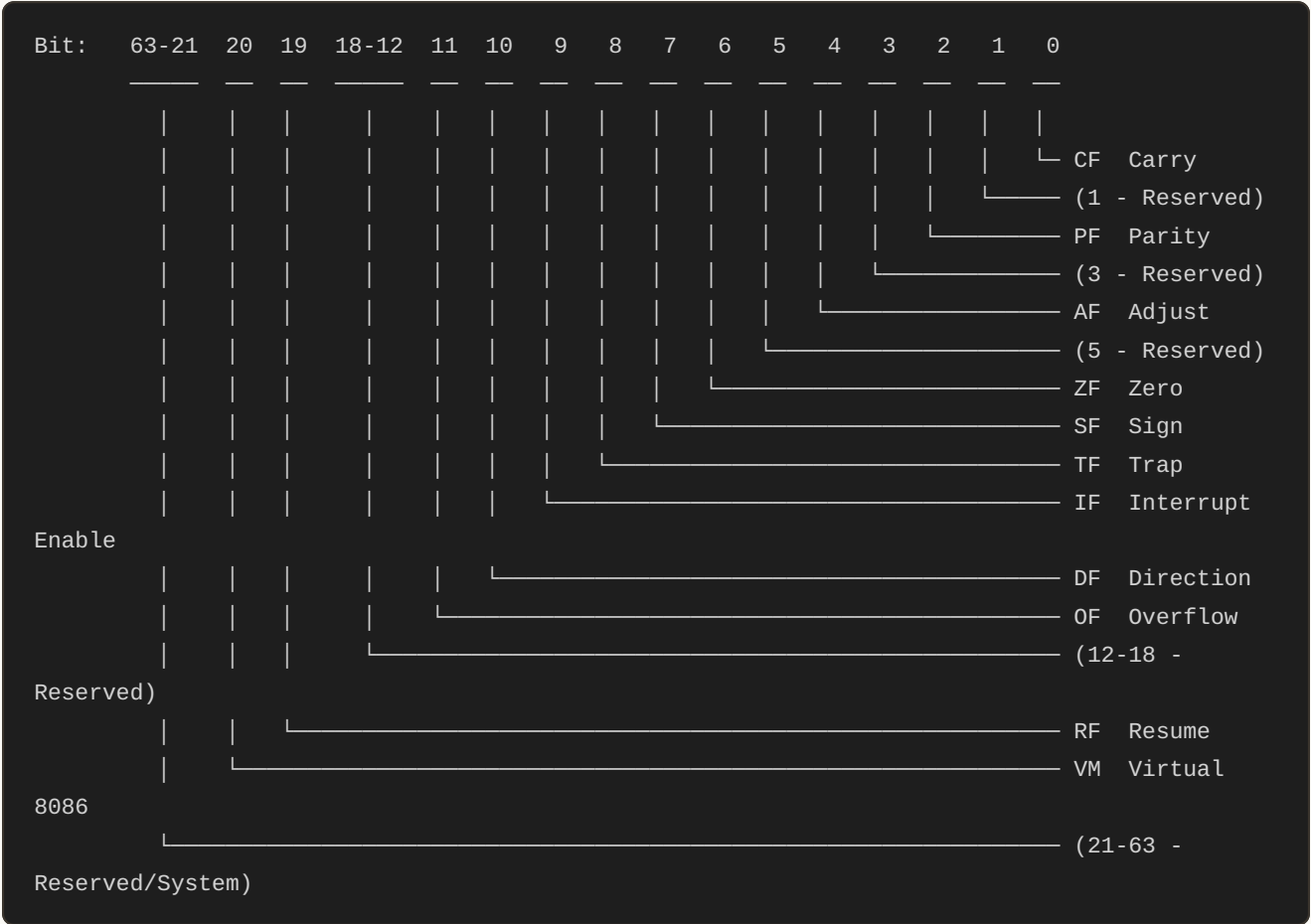
- Enable conditional jumps (if/else logic)
- Detect overflow, carry, and errors
- Control loops and comparisons
- Essential for all control flow

Part 1: The RFLAGS Register

RFLAGS Structure (64-bit)



ASCII diagram



The Six Arithmetic Flags (Most Important)

Flag	Name	Purpose
CF	Carry	Unsigned overflow/borrow
ZF	Zero	Result is zero
SF	Sign	Result is negative (bit 7/15/31/63)
OF	Overflow	Signed overflow
PF	Parity	Even number of 1-bits in low byte
AF	Auxiliary	BCD arithmetic carry (bit 3->4)

Part 2: Individual Flags Explained

CF - Carry Flag (Bit 0)

Set when: Unsigned arithmetic overflow/borrow occurs

Use cases:

- Unsigned comparison (above/below)
- Detecting overflow in addition
- Detecting borrow in subtraction
- Multi-precision arithmetic

Examples:

```

; Addition overflow
mov al, 255
add al, 1           ; AL = 0, CF = 1 (carried out)

; No overflow
mov al, 100
add al, 50          ; AL = 150, CF = 0

; Subtraction borrow
mov al, 5
sub al, 10          ; AL = 251 (wraps), CF = 1 (borrowed)

; Checking if unsigned number is bigger
mov rax, 100
sub rax, 200         ; CF = 1 (100 < 200, borrow occurred)
jc less_than         ; Jump if carry (below)

```

Affected by: ADD, SUB, ADC, SBB, shifts, rotates

Not affected by: INC, DEC

ZF - Zero Flag (Bit 6)

Set when: Result of operation is zero

Use cases:

- Equality testing
- Loop termination
- Null checks

Examples:

```

; Zero result
mov rax, 10
sub rax, 10          ; RAX = 0, ZF = 1

; Non-zero result
mov rax, 10
sub rax, 5           ; RAX = 5, ZF = 0

; Testing for zero
test rax, rax        ; ZF = 1 if RAX = 0
jz is_zero           ; Jump if zero

; Equality check
cmp rax, rbx         ; Compare by subtraction
je equal             ; Jump if equal (ZF = 1)

; Loop until zero
mov rcx, 10
loop_start:
    ; ... do work ...
    dec rcx          ; Decrement
    jnz loop_start   ; Jump if not zero

```

Affected by: Most arithmetic and logical operations

Set by: Any operation producing zero result

SF - Sign Flag (Bit 7)

Set when: Most significant bit (MSB) of result is 1

Use cases:

- Checking if signed number is negative
- Signed comparisons

Examples:

```

; Positive result
mov al, 50
add al, 20          ; AL = 70 (0x46), SF = 0 (bit 7 = 0)

; Negative result (two's complement)
mov al, 10
sub al, 20          ; AL = -10 (0xF6), SF = 1 (bit 7 = 1)

; Check if negative
test rax, rax       ; Sets SF based on MSB
js is_negative      ; Jump if sign (negative)

; Signed comparison
mov rax, -5
cmp rax, 10         ; RAX - 10 = negative
jl less_than        ; Jump if less (SF != OF)

```

Affected by: Most arithmetic and logical operations

Simply: SF = bit 7 (8-bit), bit 15 (16-bit), bit 31 (32-bit), bit 63 (64-bit)

OF - Overflow Flag (Bit 11)

Set when: Signed arithmetic overflow occurs

Use cases:

- Detecting signed overflow
- Signed comparisons

Examples:

```

; Signed overflow (positive + positive = negative)
mov al, 127          ; Maximum signed 8-bit value
add al, 1            ; AL = -128 (0x80), OF = 1, SF = 1

; No overflow
mov al, 100
add al, 20           ; AL = 120, OF = 0

; Signed overflow (negative + negative = positive)
mov al, -128         ; Minimum signed 8-bit value
sub al, 1            ; AL = 127 (0x7F), OF = 1, SF = 0

; Detect overflow
mov ax, 32767        ; Max signed 16-bit
add ax, 1            ; AX = -32768, OF = 1
jo overflow_occurred ; Jump if overflow

```

Key insight: OF tells you if the sign bit is wrong for signed arithmetic

Affected by: ADD, SUB, NEG, arithmetic shifts

Not affected by: Logical operations (AND, OR, XOR)

PF - Parity Flag (Bit 2)

Set when: Low byte of result has even number of 1-bits

Use cases:

- Error detection (rarely used in modern code)
- Serial communication parity checks

Examples:

```
; Even parity (even number of 1-bits)
mov al, 0b00000011      ; Two 1-bits (even)
test al, al              ; PF = 1

; Odd parity
mov al, 0b00000111      ; Three 1-bits (odd)
test al, al              ; PF = 0

; Only low byte matters!
mov ax, 0xFF00
test ax, ax              ; PF = 1 (low byte = 0x00, zero 1-bits = even)
```

Rarely used in typical assembly programming.

AF - Auxiliary Carry Flag (Bit 4)

Set when: Carry/borrow from bit 3 to bit 4 (low nibble to high nibble)

Use cases:

- BCD (Binary Coded Decimal) arithmetic
- Rarely used in modern programming

Examples:

```

; Auxiliary carry
mov al, 0x0F          ; 0000 1111
add al, 0x01          ; 0001 0000, AF = 1 (carry from bit 3->4)

; No auxiliary carry
mov al, 0x07          ; 0000 0111
add al, 0x01          ; 0000 1000, AF = 0

```

Rarely used except in BCD operations.

Part 3: Flags and Instructions

Which Instructions Affect Which Flags?

Instruction	CF	ZF	SF	OF	PF	AF
ADD, SUB	✓	✓	✓	✓	✓	✓
INC, DEC	-	✓	✓	✓	✓	✓
NEG	✓	✓	✓	✓	✓	✓
MUL, IMUL (1op)	✓*	-	?	✓*	?	?
DIV, IDIV	?	?	?	?	?	?
AND, OR, XOR	0	✓	✓	0	✓	?
NOT	-	-	-	-	-	-
TEST	0	✓	✓	0	✓	?
CMP	✓	✓	✓	✓	✓	✓
SHL, SHR	✓	✓	✓	✓*	✓	?
SAL, SAR	✓	✓	✓	✓*	✓	?
ROL, ROR	✓	-	-	✓*	-	-
MOV	-	-	-	-	-	-
LEA	-	-	-	-	-	-
PUSH, POP	-	-	-	-	-	-

Legend:

✓ = Modified - = Not affected

0 = Cleared ? = Undefined

✓* = Special behavior

Important Notes

```
; INC/DEC don't affect CF!
mov al, 255
inc al           ; AL = 0, but CF unchanged!
; This is different from ADD:
mov al, 255
add al, 1        ; AL = 0, CF = 1

; AND/OR/XOR clear CF and OF
mov al, 0xFF
and al, 0x01     ; CF = 0, OF = 0, ZF = 0, SF = 0

; MOV and LEA don't affect any flags
mov rax, rbx     ; Flags unchanged
lea rax, [rbx + 10] ; Flags unchanged
```

Part 4: The CMP Instruction

CMP is specifically designed for comparisons. It performs subtraction but doesn't store the result.

```
cmp operand1, operand2 ; Performs: operand1 - operand2
                        ; Sets flags, doesn't store result
```

How CMP Works

```
; Example: cmp rax, rbx
; Internally does: temp = rax - rbx
; Sets flags based on temp
; Throws away temp (doesn't modify rax or rbx)

cmp rax, 10      ; Compare RAX with 10
; If RAX = 10: ZF = 1 (equal)
; If RAX > 10: SF = 0, ZF = 0 (greater)
; If RAX < 10: SF = 1, ZF = 0 (less)
```

CMP Examples

```

; Equality check
mov rax, 42
cmp rax, 42          ; RAX - 42 = 0, ZF = 1
je equal             ; Jump if equal (ZF = 1)

; Not equal
cmp rax, 100         ; RAX - 100 = -58, ZF = 0
jne not_equal        ; Jump if not equal (ZF = 0)

; Greater than (signed)
mov rax, 50
cmp rax, 10          ; RAX - 10 = 40 (positive)
jg greater           ; Jump if greater (SF = 0F and ZF = 0)

; Less than (signed)
mov rax, -5
cmp rax, 10          ; RAX - 10 = -15 (negative)
jl less              ; Jump if less (SF != 0F)

; Unsigned comparisons
mov rax, 100
cmp rax, 200         ; RAX - 200, CF = 1 (borrow)
jb below             ; Jump if below (CF = 1)

```

Part 5: The TEST Instruction

TEST performs AND but doesn't store the result - only sets flags.

```

test operand1, operand2 ; Performs: operand1 & operand2
                        ; Sets flags, doesn't store result

```

How TEST Works

```

; Example: test rax, rax
; Internally does: temp = rax & rax
; Result is always rax (X & X = X)
; Sets ZF = 1 if rax = 0, ZF = 0 otherwise

```

Common TEST Patterns

```

; Check if zero (most common!)
test rax, rax          ; ZF = 1 if RAX = 0
jz is_zero             ; Jump if zero

; Check specific bit
test al, 0x01          ; Check bit 0
jnz bit0_is_set

test al, 0x80          ; Check bit 7 (sign bit for byte)
jnz bit7_is_set

; Check if negative (for signed numbers)
test rax, rax          ; Sets SF based on MSB
js is_negative         ; Jump if sign flag set

; Check multiple bits
test al, 0b11110000    ; Check if any upper 4 bits are set
jnz has_upper_bits

; Even/odd check
test rax, 1            ; Check bit 0
jz is_even             ; Jump if zero (bit 0 = 0)
jnz is_odd             ; Jump if not zero (bit 0 = 1)

```

TEST vs AND

```

; Using AND (modifies operand)
mov al, 0xFF
and al, 0x0F           ; AL = 0x0F (destroyed original value!)

; Using TEST (doesn't modify)
mov al, 0xFF
test al, 0x0F          ; AL still= 0xFF, flags set
; Now you can use AL again without reloading

```

When to use TEST:

- Checking conditions without modifying data
- More readable intent (testing, not masking)
- Shorter encoding in some cases

When to use AND:

- When you need the result of the AND operation
- Masking/clearing bits

Part 6: Reading Flag Combinations

Conditional jumps test flag combinations. Understanding these is crucial!

Equality (ZF only)

```
cmp rax, rbx
je equal          ; Jump if ZF = 1
jne not_equal     ; Jump if ZF = 0

; Works for both signed and unsigned!
```

Signed Comparisons (SF and OF matter)

```
cmp rax, rbx      ; Compare signed numbers

jg greater         ; Jump if (ZF = 0) AND (SF = OF)
jge greater_equal  ; Jump if (SF = OF)
jl less           ; Jump if (SF != OF)
jle less_equal     ; Jump if (ZF = 1) OR (SF != OF)
```

Why SF and OF?

```
If no overflow (OF = 0):
    SF = 0 means result positive (A > B)
    SF = 1 means result negative (A < B)

If overflow (OF = 1):
    Signs are flipped!
    SF = 0 means result was supposed to be negative (A < B)
    SF = 1 means result was supposed to be positive (A > B)

So: SF != OF means A < B
    SF = OF means A >= B
```

Unsigned Comparisons (CF matters)

```
cmp rax, rbx      ; Compare unsigned numbers

ja above          ; Jump if (CF = 0) AND (ZF = 0)
jae above_equal   ; Jump if (CF = 0)
jb below          ; Jump if (CF = 1)
jbe below_equal   ; Jump if (CF = 1) OR (ZF = 1)
```

Why CF?

```
If borrow occurred (CF = 1): A < B (had to borrow)
If no borrow (CF = 0): A >= B
```

Part 7: Practical Flag Examples

Example 1: Zero Check

```
; Method 1: CMP
cmp rax, 0
je is_zero

; Method 2: TEST (better!)
test rax, rax
jz is_zero

; Method 3: OR (also works)
or rax, rax           ; Doesn't change RAX, sets ZF
jz is_zero
```

Example 2: Range Check ($10 \leq x \leq 20$)

C Equivalent:

```

#include <stdint.h>
#include <stdbool.h>

bool in_range_method1(int64_t x) {
    // Method 1: Two comparisons
    if (x < 10) return false;
    if (x > 20) return false;
    return true; // In range!
}

bool in_range_method2(uint64_t x) {
    // Method 2: Subtract lower bound (unsigned trick)
    uint64_t adjusted = x - 10;
    if (adjusted > 10) return false; // Unsigned comparison
    return true;
}

int main() {
    int64_t value = 15;
    if (in_range_method1(value)) {
        // In range
    }
    return 0;
}

```

Assembly:

```

; Check if RAX is in range [10, 20]

; Method 1: Two comparisons
cmp rax, 10
jle out_of_range      ; RAX < 10
cmp rax, 20
jge out_of_range      ; RAX > 20
; In range!

; Method 2: Subtract lower bound
sub rax, 10           ; Adjust range to [0, 10]
cmp rax, 10
ja out_of_range       ; Unsigned: above 10 means out of range
add rax, 10           ; Restore original value

```

Example 3: Sign Check

C Equivalent:

```

#include <stdint.h>

void check_sign(int64_t value) {
    if (value < 0) {
        // Handle negative
    } else if (value == 0) {
        // Handle zero
    } else {
        // Handle positive (value > 0)
    }
}

int main() {
    int64_t x = -5;
    check_sign(x);
    return 0;
}

```

Assembly:

```

; Check if RAX is negative, zero, or positive

test rax, rax
js negative      ; SF = 1, RAX < 0
jz zero          ; ZF = 1, RAX = 0
; Otherwise RAX > 0

negative:
    ; Handle negative
    jmp done
zero:
    ; Handle zero
    jmp done
; Positive case
done:

```

Example 4: Overflow Detection**C Equivalent:**

```

#include <stdint.h>
#include <stdbool.h>
#include <limits.h>

bool add_with_overflow_check(int16_t a, int16_t b, int16_t *result) {
    int32_t temp = (int32_t)a + (int32_t)b;
    *result = (int16_t)temp;

    // Check if overflow occurred
    if (temp > INT16_MAX || temp < INT16_MIN) {
        return true; // Overflow detected
    }
    return false; // No overflow
}

int main() {
    int16_t result;

    // No overflow
    bool overflow1 = add_with_overflow_check(32000, 1000, &result);
    // overflow1 = true (33000 > 32767)

    // Overflow
    bool overflow2 = add_with_overflow_check(32767, 1, &result);
    // overflow2 = true, result = -32768

    return 0;
}

```

Assembly:

```

; Detect signed overflow in addition
mov ax, 32000
add ax, 1000          ; Result = 33000 (in range)
jo overflow           ; OF = 0, no jump

mov ax, 32767         ; Max signed 16-bit
add ax, 1             ; Result = -32768 (overflow!)
jo overflow           ; OF = 1, jump taken

overflow:
    ; Handle overflow

```

Example 5: Carry Chain (Multi-Precision)**C Equivalent:**

```

#include <stdint.h>

typedef struct {
    uint64_t low;
    uint64_t high;
} uint128_t;

uint128_t add128(uint128_t a, uint128_t b) {
    uint128_t result;

    // Add low 64 bits
    result.low = a.low + b.low;

    // Check if carry occurred (overflow from low addition)
    uint64_t carry = (result.low < a.low) ? 1 : 0;

    // Add high 64 bits + carry
    result.high = a.high + b.high + carry;

    return result;
}

int main() {
    uint128_t num1 = {0xFFFFFFFFFFFFFFFFULL, 0x1234567890ABCDEFULL};
    uint128_t num2 = {0x0000000000000001ULL, 0x0000000000000001ULL};
    uint128_t result = add128(num1, num2);
    // result.low = 0, result.high = 0x1234567890ABCDF1
    return 0;
}

```

Assembly:

```

; Add two 128-bit numbers
; Number1 = RDX:RAX (high:low)
; Number2 = RBX:RCX (high:low)

add rax, rcx          ; Add low 64 bits, may set CF
adc rdx, rbx          ; Add high 64 bits + carry

; Result in RDX:RAX

```

Example 6: Parity Check

```
; Check if byte has even parity
mov al, 0b10110101      ; 5 ones (odd parity)
test al, al              ; Set PF
jp even_parity          ; Jump if PF = 1 (even number of 1s)
; Odd parity

even_parity:
    ; Even parity handling
```

Part 8: Flag Manipulation Instructions

Direct Flag Control

```
; Clear carry flag
clc                      ; CF = 0

; Set carry flag
stc                      ; CF = 1

; Complement carry flag
cmc                      ; CF = !CF

; Clear direction flag (for string operations)
cld                      ; DF = 0 (forward)

; Set direction flag
std                      ; DF = 1 (backward)
```

Save/Restore Flags

```
; Save flags on stack
pushf                    ; Push FLAGS (16-bit)
pushfq                   ; Push RFLAGS (64-bit)

; Restore flags from stack
popf                     ; Pop FLAGS (16-bit)
popfq                    ; Pop RFLAGS (64-bit)

; Example: Preserve flags across operation
pushfq
; ... operations that modify flags ...
popfq                    ; Restore original flags
```

Load Flags

```
; Load AH from FLAGS (legacy, rarely used)
lahf                ; AH = SF:ZF:0:AF:0:PF:1:CF

; Store AH to FLAGS (legacy, rarely used)
sahf                ; FLAGS = AH (some bits)
```

Part 9: Common Patterns and Idioms

Pattern 1: Boolean to Integer

```
; Convert boolean condition to 0/1

; Method 1: Using SETcc
cmp rax, rbx
setg al                ; AL = 1 if RAX > RBX, else AL = 0

; Method 2: Using jumps
xor ecx, ecx          ; ECX = 0
cmp rax, rbx
jle skip
mov ecx, 1
skip:
```

Pattern 2: Max/Min

```
; max(rax, rbx) -> rax
cmp rax, rbx
jge already_max       ; RAX >= RBX, already maximum
mov rax, rbx          ; RBX is bigger
already_max:

; min(rax, rbx) -> rax
cmp rax, rbx
jle already_min       ; RAX <= RBX, already minimum
mov rax, rbx          ; RBX is smaller
already_min:

; Or use CMOVcc (conditional move)
cmp rax, rbx
cmovl rax, rbx         ; If RAX < RBX, RAX = RBX (min)
cmovg rax, rbx         ; If RAX > RBX, RAX = RBX (max)
```


Pattern 3: Abs (Absolute Value)

```
; abs(rax) using flags
test rax, rax
jns already_positive    ; Jump if not sign (positive)
neg rax
already_positive:

; Branchless absolute value (for EAX)
mov edx, eax
sar edx, 31             ; EDX = all 1s if negative, 0 if positive
xor eax, edx
sub eax, edx            ; EAX = abs(EAX)
```

Pattern 4: Conditional Assignment

```
; if (rax > 10) rax = 100; else rax = 0;

cmp rax, 10
jle else_branch
    mov rax, 100
    jmp end_if
else_branch:
    xor rax, rax
end_if:

; Or with conditional move
mov rcx, 100
xor rdx, rdx
cmp rax, 10
cmovg rax, rcx          ; If RAX > 10, RAX = 100
cmovle rax, rdx         ; If RAX <= 10, RAX = 0
```

Part 10: Debugging with Flags

Viewing Flags in GDB

```
# Run program in GDB
gdb ./program

# View all registers including flags
(gdb) info registers

# View just RFLAGS
(gdb) print $eflags

# View flags in binary
(gdb) print/t $eflags

# Common flag abbreviations in GDB output:
# CF PF AF ZF SF TF IF DF OF
```

GDB Flag Display

```
RFLAGS: 0x246 [ PF ZF IF ]

Flags shown if set:
CF - carry
PF - parity
AF - auxiliary
ZF - zero
SF - sign
TF - trap
IF - interrupt enable
DF - direction
OF - overflow
```

Manual Flag Testing

```
section .text
    global _start

_start:
    ; Test various flag conditions

    ; Set CF
    mov al, 255
    add al, 1          ; CF = 1
    ; Now run in debugger and check

    ; Set ZF
    xor rax, rax       ; ZF = 1

    ; Set SF
    mov al, 200
    add al, 100        ; SF = 1 (result = -56 in signed)

    ; Set OF
    mov al, 127
    add al, 1          ; OF = 1 (signed overflow)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Practice Exercises

Exercise 1: Predict Flags

```
mov al, 150
add al, 150
```

What are CF, ZF, SF, OF?

Solution

```

150 + 150 = 300 (0x12C)
AL stores only 8 bits: 0x2C = 44

CF = 1 (unsigned overflow, 300 > 255)
ZF = 0 (result is 44, not zero)
SF = 0 (bit 7 of 0x2C = 0)
OF = 1 (signed: 150+150 should be positive but looks negative in 8-bit)

Wait, let's recalculate:
150 in 8-bit signed = -106
-106 + -106 = -212
In 8-bit: 44 (should be negative but isn't)
OF = 1 (signed overflow)

Actually 150 = 0x96 = 1001 0110 (negative in signed: -106)
300 = 0x12C, truncated to 8-bit = 0x2C = 44 (positive)
OF = 1 because two negatives gave positive

```

Exercise 2: Write Comparison

Write code to check if RAX is in the range [-10, 10] (inclusive, signed).

Solution

```

cmp rax, -10
jl out_of_range      ; RAX < -10
cmp rax, 10
jg out_of_range      ; RAX > 10
; In range [-10, 10]

out_of_range:
; Handle out of range

```

Exercise 3: Flag Logic

After these instructions, which conditional jumps will be taken?

```

mov al, 10
sub al, 10
je target1           ; ?
jg target2           ; ?
js target3           ; ?

```

Solution

```
sub al, 10: AL = 0
```

```
ZF = 1 (result is zero)
```

```
SF = 0 (result is positive/zero)
```

```
OF = 0 (no signed overflow)
```

```
je target1 → TAKEN (ZF = 1)
```

```
jg target2 → NOT TAKEN (ZF = 1, need ZF = 0 for greater)
```

```
js target3 → NOT TAKEN (SF = 0, need SF = 1 for negative)
```

Exercise 4: Implement Sign Function

Implement `sign(x)`: returns -1 if $x < 0$, 0 if $x = 0$, 1 if $x > 0$

Solution

```
; Input: RAX
; Output: RAX = sign(RAX)

test rax, rax
js negative
jz zero
; Positive
mov rax, 1
jmp done

negative:
mov rax, -1
jmp done

zero:
xor rax, rax

done:
```

Exercise 5: Overflow Check

Write code that adds two signed 64-bit numbers and detects overflow.

Solution

```

; Add RAX and RBX, check overflow
add rax, rbx
jo overflow_occurred

; No overflow, continue normally
jmp no_overflow

overflow_occurred:
    ; Handle overflow
    ; Could saturate, return error, etc.

no_overflow:
    ; Result in RAX is valid

```

Quick Reference

The Six Main Flags

CF (Carry)	- Unsigned overflow/borrow
ZF (Zero)	- Result is zero
SF (Sign)	- Result is negative (MSB = 1)
OF (Overflow)	- Signed overflow
PF (Parity)	- Even number of 1-bits in low byte
AF (Aux)	- BCD carry (bit 3-4)

Flag Interpretation

```

; After: cmp rax, rbx (equivalent to rax - rbx)

ZF = 1      → rax == rbx
ZF = 0      → rax != rbx

SF = 0F     → rax >= rbx (signed)
SF != 0F    → rax < rbx (signed)

CF = 0      → rax >= rbx (unsigned)
CF = 1      → rax < rbx (unsigned)

ZF=0 && SF=0F → rax > rbx (signed)
ZF=0 && CF=0  → rax > rbx (unsigned)

```

Common Checks

```
test rax, rax ; Zero check (best)
test rax, 1   ; Check bit 0 (even/odd)
test rax, rax ; Negative check (use js)
cmp rax, rbx  ; Comparison
```

Knowledge Check

Before moving to Topic 5, verify you understand:

- Purpose of each flag (CF, ZF, SF, OF, PF, AF)
- Which instructions affect which flags
- How CMP works (subtraction without storing)
- How TEST works (AND without storing)
- Signed vs unsigned comparisons
- Reading flag combinations
- When to use TEST vs CMP vs AND

Excellent! You now understand CPU flags and comparisons!

Next: [Topic 5: Conditional Jumps](#) (coming soon)

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 22

Topic 5: Conditional Jumps

Master conditional execution - the foundation of if/else logic in assembly.

Overview

Jumps are how you control program flow in assembly. They let you:

- Implement if/else statements
- Create loops
- Handle different cases
- Skip code sections
- Build complex logic

Types of jumps:

- Unconditional jumps (always jump)
- Conditional jumps (jump based on flags)

Part 1: How Jumps Work

The Instruction Pointer

The **RIP** (Instruction Pointer) register always points to the next instruction to execute.

Normal execution (no jumps):

<code>mov rax, 10</code>	← RIP here
<code>add rax, 5</code>	← RIP moves here next
<code>mov rbx, rax</code>	← Then here
<code>ret</code>	← Finally here

With a jump:

<code>mov rax, 10</code>	← RIP here
<code>jmp skip</code>	← Jump changes RIP
<code>add rax, 5</code>	(skipped!)
<code>skip:</code>	← RIP jumps here
<code>mov rbx, rax</code>	

Labels

Labels are markers in your code that jumps can target:

```
section .text
    global _start

_start:                ; Label marking program start
    mov rax, 10
    jmp continue       ; Jump to 'continue' label

    ; This code is skipped
    mov rax, 99

continue:              ; Label marking jump target
    mov rbx, rax        ; RBX = 10 (not 99)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Label rules:

- Can contain letters, numbers, underscore, dot
- Must start with letter, underscore, or dot
- Case-sensitive
- Followed by colon `:` when defined
- No colon when referenced in jumps

Part 2: Unconditional Jumps

JMP - Jump Always

```
jmp label                ; Always jump to label
```

Examples:

```

; Simple jump
jmp end_program
mov rax, 100           ; Skipped!
end_program:
    ; Continues here

; Jump forward
mov rax, 10
jmp skip_error
    mov rax, -1        ; Error code (skipped)
skip_error:
    ; Normal flow

; Jump backward (loop)
start_loop:
    dec rcx
    cmp rcx, 0
    jne start_loop    ; Jump back to start_loop

```

Short vs Near vs Far Jumps

```

; Short jump (2 bytes, offset -128 to +127 bytes)
jmp short nearby_label

; Near jump (5 bytes, offset ±2GB)
jmp near far_label
jmp far_label        ; Default is near

; Far jump (different code segment, rarely used)
jmp far segment:offset

```

Usually: Let NASM choose automatically. It picks the shortest encoding.

Part 3: Conditional Jumps

Conditional jumps check CPU flags and jump only if conditions are met.

Jump Mnemonic Format

J + condition

Examples:

JE = Jump if Equal

JNE = Jump if Not Equal

JG = Jump if Greater

JL = Jump if Less

All Conditional Jumps

cmp / test set the flags; jcc reads them



test does AND, cmp does SUB — neither writes its operands, they only set flags

Mnemonic	Meaning	Condition
JE / JZ	Jump if Equal/Zero	ZF = 1
JNE / JNZ	Jump if Not Equal	ZF = 0
JS	Jump if Sign	SF = 1
JNS	Jump if Not Sign	SF = 0
JC	Jump if Carry	CF = 1
JNC	Jump if Not Carry	CF = 0
JO	Jump if Overflow	OF = 1
JNO	Jump if Not Overflow	OF = 0
JP / JPE	Jump if Parity Even	PF = 1
JNP / JPO	Jump if Parity Odd	PF = 0

Signed Comparison Jumps (After CMP)

Jump	Meaning	Condition
JE	Jump if Equal	ZF = 1
JNE	Jump if Not Equal	ZF = 0
JG	Jump if Greater	ZF = 0 AND SF = 0F
JGE	Jump if ≥	SF = 0F
JL	Jump if Less	SF ≠ 0F
JLE	Jump if ≤	ZF = 1 OR SF ≠ 0F

Unsigned Comparison Jumps (After CMP)

Jump	Meaning	Condition
JE	Jump if Equal	ZF = 1
JNE	Jump if Not Equal	ZF = 0
JA	Jump if Above	CF = 0 AND ZF = 0
JAЕ	Jump if ≥	CF = 0
JB	Jump if Below	CF = 1
JBE	Jump if ≤	CF = 1 OR ZF = 1

Aliases (Same Instruction, Different Name)

```

JE  = JZ      (Equal = Zero)
JNE = JNZ     (Not Equal = Not Zero)
JA  = JNBE    (Above = Not Below or Equal)
JAE = JNB     (Above or Equal = Not Below)
JB  = JNAE    (Below = Not Above or Equal)
JBE = JNA     (Below or Equal = Not Above)
JG  = JNLE    (Greater = Not Less or Equal)
JGE = JNL     (Greater or Equal = Not Less)
JL  = JNGE    (Less = Not Greater or Equal)
JLE = JNG     (Less or Equal = Not Greater)

```

Part 4: Using Conditional Jumps

Basic Pattern

1. Perform comparison (CMP, TEST, or arithmetic)
2. Check flags with conditional jump
3. Execute code based on result

Equality Check

```
; if (rax == 42) goto equal_handler

cmp rax, 42
je equal_handler      ; Jump if equal
; Not equal path
mov rbx, 0
jmp done

equal_handler:
; Equal path
mov rbx, 1

done:
```

Inequality Check

```
; if (rax != 42) goto not_equal

cmp rax, 42
jne not_equal        ; Jump if not equal
; Equal path
mov rbx, 1
jmp done

not_equal:
; Not equal path
mov rbx, 0

done:
```

Greater Than (Signed)

```

; if (rax > 10) goto greater

cmp rax, 10
jg greater          ; Jump if rax > 10 (signed)
; Less than or equal path
mov rbx, 0
jmp done

greater:
; Greater path
mov rbx, 1

done:

```

Less Than (Signed)

```

; if (rax < 0) goto negative

cmp rax, 0
jl negative         ; Jump if rax < 0 (signed)
; Positive or zero path
mov rbx, 0
jmp done

negative:
; Negative path
mov rbx, 1

done:

```

Above (Unsigned)

```

; if (rax > 100) goto above [unsigned comparison]

cmp rax, 100
ja above           ; Jump if rax > 100 (unsigned)
; Below or equal path
mov rbx, 0
jmp done

above:
; Above path
mov rbx, 1

done:

```

Part 5: If/Else in Assembly

Simple If Statement

C Code:

```
if (x == 10) {  
    y = 100;  
}
```

Assembly:

```
; if (rax == 10) rbx = 100  
  
cmp rax, 10  
jne skip_if      ; Jump if not equal (skip the if body)  
    mov rbx, 100  ; If body  
skip_if:  
    ; Continue...
```

If-Else Statement

C Code:

```
if (x > 5) {  
    y = 1;  
} else {  
    y = 0;  
}
```

Assembly:

```
; if (rax > 5) rbx = 1, else rbx = 0  
  
cmp rax, 5  
jle else_branch ; Jump if less or equal  
    ; If true branch  
    mov rbx, 1  
    jmp end_if  
else_branch:  
    ; Else branch  
    mov rbx, 0  
end_if:  
    ; Continue...
```

If-Else If-Else

C Code:

```
if (x < 0) {
    result = -1;
} else if (x > 0) {
    result = 1;
} else {
    result = 0;
}
```

Assembly:

```
; Sign function: sign(rax) -> rbx

cmp rax, 0
jnl negative      ; If x < 0
jg positive       ; Else if x > 0
                  ; Else (x == 0)
mov rbx, 0
jmp done

negative:
mov rbx, -1
jmp done

positive:
mov rbx, 1

done:
; Continue...
```

Nested If Statements

C Code:

```
if (x > 0) {
    if (x < 10) {
        result = 1;
    }
}
```

Assembly:


```

; if (rax > 0 && rax < 10) rbx = 1

cmp rax, 0
jle skip_outer      ; Skip if not > 0
    cmp rax, 10
    jge skip_inner   ; Skip if not < 10
        mov rbx, 1    ; Both conditions true
    skip_inner:
skip_outer:
    ; Continue...

```

Part 6: Loops with Jumps

While Loop

C Code:

```

while (count > 0) {
    sum += count;
    count--;
}

```

Assembly:

```

; while (rcx > 0) { rax += rcx; rcx--; }

while_start:
    cmp rcx, 0
    jle while_end      ; Exit if rcx <= 0

    add rax, rcx        ; sum += count
    dec rcx             ; count--
    jmp while_start     ; Loop back

while_end:
    ; Continue...

```

Do-While Loop

C Code:

```
do {
    sum += count;
    count--;
} while (count > 0);
```

Assembly:

```
; do { rax += rcx; rcx--; } while (rcx > 0)

do_start:
    add rax, rcx      ; sum += count
    dec rcx          ; count--

    cmp rcx, 0
    jg do_start       ; Loop if rcx > 0

; Continue...
```

For Loop

C Code:

```
for (i = 0; i < 10; i++) {
    sum += i;
}
```

Assembly:

```
; for (rcx = 0; rcx < 10; rcx++) { rax += rcx; }

xor rcx, rcx      ; i = 0

for_start:
    cmp rcx, 10
    jge for_end    ; Exit if i >= 10

    add rax, rcx    ; sum += i
    inc rcx         ; i++
    jmp for_start

for_end:
; Continue...
```

Loop with Break

C Code:

```
while (1) {
    if (count == 0) break;
    sum += count;
    count--;
}
```

Assembly:

```
; while (1) { if (rcx == 0) break; rax += rcx; rcx--; }

infinite_loop:
    cmp rcx, 0
    je loop_break      ; Break if count == 0

    add rax, rcx
    dec rcx
    jmp infinite_loop

loop_break:
    ; Continue after loop...
```

Loop with Continue**C Code:**

```
for (i = 0; i < 10; i++) {
    if (i == 5) continue;
    sum += i;
}
```

Assembly:

```

; for (rcx = 0; rcx < 10; rcx++) { if (rcx == 5) continue; rax += rcx; }

xor rcx, rcx

loop_start:
    cmp rcx, 10
    jge loop_end

    cmp rcx, 5
    je continue_point    ; Skip to next iteration

    add rax, rcx          ; sum += i

continue_point:
    inc rcx
    jmp loop_start

loop_end:
    ; Continue...

```

Part 7: Special Loop Instructions

LOOP - Loop with RCX Counter

```

loop label    ; Decrement RCX, jump if RCX != 0

```

Equivalent to:

```

dec rcx
jnz label

```

Example:

```

; Loop 10 times
mov rcx, 10

loop_start:
    ; ... do something ...
    loop loop_start    ; Decrements RCX, loops if RCX != 0

; RCX is now 0

```

Limitations:

- Only works with RCX

- Can't check other conditions
- No way to break early without jumping

When to use:

- Simple counted loops
- When RCX counter is convenient
- Short, simple loop bodies

When not to use:

- Complex loop conditions
- Need to preserve RCX
- Need early exit/break

LOOPcc Variants

```

loope label          ; Loop if equal (ZF = 1) and RCX != 0
loopne label         ; Loop if not equal (ZF = 0) and RCX != 0
loopz label          ; Same as loope
loopnz label         ; Same as loopne

```

Example:

```

; Search for zero in array
mov rcx, array_size
mov rsi, array

search_loop:
    cmp byte [rsi], 0
    loopne search_loop ; Continue if not zero and RCX != 0

; Either found zero or reached end

```

Rarely used in modern code - explicit comparisons are clearer.

Part 8: Practical Examples

Example 1: Find Maximum

C Equivalent:

```
#include <stdint.h>

int main() {
    int32_t numbers[] = {15, 42, 7, 89, 23, 56};
    int32_t count = 6;
    int32_t max = numbers[0];

    for (int32_t i = 1; i < count; i++) {
        if (numbers[i] > max) {
            max = numbers[i];
        }
    }

    // max = 89
    return 0;
}
```

Assembly:

```

section .data
    numbers dd 15, 42, 7, 89, 23, 56
    count equ 6

section .bss
    max resd 1

section .text
    global _start

_start:
    ; Find maximum value in array
    mov eax, [numbers]      ; max = numbers[0]
    mov ecx, 1              ; i = 1

find_max_loop:
    cmp ecx, count
    jge found_max           ; Exit if i >= count

    mov edx, [numbers + rcx*4]
    cmp eax, edx
    jge skip_update         ; Skip if max >= numbers[i]

    mov eax, edx            ; max = numbers[i]

skip_update:
    inc ecx
    jmp find_max_loop

found_max:
    mov [max], eax          ; Store result (89)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: String Length

C Equivalent:

```
#include <stdint.h>

size_t my_strlen(const char *str) {
    size_t length = 0;

    while (str[length] != '\0') {
        length++;
    }

    return length;
}

int main() {
    const char *string = "Hello, World!";
    size_t length = my_strlen(string);
    // length = 13

    return length;
}
```

Assembly:

```
section .data
    string db "Hello, World!", 0

section .text
    global _start

_start:
    ; Calculate string length
    mov rsi, string      ; Pointer to string
    xor rcx, rcx         ; Length = 0

strlen_loop:
    cmp byte [rsi + rcx], 0 ; Check for null terminator
    je strlen_done       ; Exit if found null

    inc rcx              ; Length++
    jmp strlen_loop

strlen_done:
    ; RCX contains length (13)

    ; Exit
    mov rax, 60
    mov rdi, rcx         ; Exit with length as code
    syscall
```


Example 3: Count Set Bits (Population Count)

C Equivalent:

```
#include <stdint.h>

int popcount(uint64_t value) {
    int count = 0;

    while (value != 0) {
        if (value & 1) {
            count++;
        }
        value >>= 1; // Shift right by 1
    }

    return count;
}

int main() {
    uint64_t value = 0b10110101;
    int count = popcount(value);
    // count = 5 (five 1-bits)

    return count;
}
```

Assembly:

```

section .text
    global _start

_start:
    ; Count set bits in RAX
    mov rax, 0b10110101    ; Test value
    xor rcx, rcx           ; count = 0

count_bits_loop:
    test rax, rax
    jz count_done          ; Exit if no bits left

    test rax, 1            ; Check lowest bit
    jz skip_count
    inc rcx                ; Count if set

skip_count:
    shr rax, 1             ; Shift to next bit
    jmp count_bits_loop

count_done:
    ; RCX contains count (5 bits set)

    ; Exit
    mov rax, 60
    mov rdi, rcx
    syscall

```

Example 4: Binary Search

C Equivalent:

```

#include <stdint.h>

int binary_search(int32_t *array, int32_t size, int32_t target) {
    int32_t left = 0;
    int32_t right = size;

    while (left < right) {
        int32_t mid = (left + right) / 2;

        if (array[mid] == target) {
            return mid; // Found at index mid
        } else if (array[mid] < target) {
            left = mid + 1; // Search right half
        } else {
            right = mid; // Search left half
        }
    }

    return -1; // Not found
}

int main() {
    int32_t array[] = {1, 3, 5, 7, 9, 11, 13, 15};
    int32_t size = 8;
    int32_t target = 7;

    int32_t index = binary_search(array, size, target);
    // index = 3 (found at position 3)

    return index;
}

```

Assembly:

```

section .data
    ; Sorted array
    array dd 1, 3, 5, 7, 9, 11, 13, 15
    array_size equ 8
    target dd 7

section .text
    global _start

_start:
    ; Binary search for target
    xor rcx, rcx          ; left = 0
    mov rdx, array_size   ; right = size
    mov edi, [target]     ; value to find

binary_search:
    cmp rcx, rdx
    jge not_found         ; Exit if left >= right

    ; mid = (left + right) / 2
    mov rax, rcx
    add rax, rdx
    shr rax, 1            ; Divide by 2

    ; Compare array[mid] with target
    mov esi, [array + rax*4]
    cmp esi, edi
    je found              ; Found!
    jl search_right       ; array[mid] < target

    ; Search left half
    mov rdx, rax           ; right = mid
    jmp binary_search

search_right:
    ; Search right half
    lea rcx, [rax + 1]     ; left = mid + 1
    jmp binary_search

found:
    ; Found at index RAX
    mov rdi, rax
    jmp exit_program

not_found:
    ; Not found
    mov rdi, -1

exit_program:

```

```
mov rax, 60  
syscall
```

Example 5: FizzBuzz

```

section .data
    fizz db "Fizz", 10
    fizz_len equ $ - fizz
    buzz db "Buzz", 10
    buzz_len equ $ - buzz
    fizzbuzz db "FizzBuzz", 10
    fizzbuzz_len equ $ - fizzbuzz

section .text
    global _start

_start:
    mov rcx, 1                ; counter = 1

fizzbuzz_loop:
    cmp rcx, 100
    jg fizzbuzz_end          ; Exit if counter > 100

    ; Check if divisible by 15
    mov rax, rcx
    xor rdx, rdx
    mov rbx, 15
    div rbx
    test rdx, rdx
    jz print_fizzbuzz        ; Divisible by 15

    ; Check if divisible by 3
    mov rax, rcx
    xor rdx, rdx
    mov rbx, 3
    div rbx
    test rdx, rdx
    jz print_fizz            ; Divisible by 3

    ; Check if divisible by 5
    mov rax, rcx
    xor rdx, rdx
    mov rbx, 5
    div rbx
    test rdx, rdx
    jz print_buzz            ; Divisible by 5

    ; Print number (simplified - just skip)
    jmp next_iteration

print_fizzbuzz:
    mov rax, 1
    mov rdi, 1
    mov rsi, fizzbuzz
    mov rdx, fizzbuzz_len

```

```
    syscall
    jmp next_iteration

print_fizz:
    mov rax, 1
    mov rdi, 1
    mov rsi, fizz
    mov rdx, fizz_len
    syscall
    jmp next_iteration

print_buzz:
    mov rax, 1
    mov rdi, 1
    mov rsi, buzz
    mov rdx, buzz_len
    syscall

next_iteration:
    inc rcx
    jmp fizzbuzz_loop

fizzbuzz_end:
    mov rax, 60
    xor rdi, rdi
    syscall
```


Part 9: Optimization Tips

Invert Conditions to Reduce Jumps

```
; Less efficient (2 jumps in common case)
cmp rax, 10
je equal
mov rbx, 0
jmp done
equal:
mov rbx, 1
done:

; More efficient (1 jump in common case)
cmp rax, 10
jne not_equal
mov rbx, 1
jmp done
not_equal:
mov rbx, 0
done:
```

Place Common Case First

```
; If rax is usually not zero:
test rax, rax
jnz common_case      ; Taken most often (predicted taken)
    ; Rare case
    jmp done
common_case:
    ; Common case
done:
```

Avoid Jumping Over Single Instructions

```
; Inefficient
test rax, rax
jz skip
inc rbx
skip:

; Use conditional move (if appropriate)
test rax, rax
cmovnz rbx, some_value ; Only if rbx = some_value needed
```

Short Jumps When Possible

NASM automatically chooses, but be aware:

```
; Short jump: 2 bytes (offset -128 to +127)  
jmp short nearby  
  
; Near jump: 5 bytes (offset ±2GB)  
jmp near far_away
```

Keep frequently executed loops small to fit in instruction cache.

Part 10: Common Patterns

Pattern 1: Early Exit/Guard Clause

```
; Check precondition and exit early  
test rax, rax  
jz error_null_pointer ; Exit early if invalid  
  
; Normal processing continues...
```

Pattern 2: Switch/Case

```
; switch (rax) { case 0: ..., case 1: ..., default: ... }
```

```
cmp rax, 0  
je case_0  
cmp rax, 1  
je case_1  
cmp rax, 2  
je case_2  
jmp default_case
```

```
case_0:  
    ; Handle case 0  
    jmp end_switch
```

```
case_1:  
    ; Handle case 1  
    jmp end_switch
```

```
case_2:  
    ; Handle case 2  
    jmp end_switch
```

```
default_case:  
    ; Handle default
```

```
end_switch:
```

Pattern 3: Jump Table (Efficient Switch)

```
section .data
    jump_table dq case_0, case_1, case_2

section .text
    ; switch (rax) using jump table
    cmp rax, 2
    ja default_case      ; Out of range

    jmp [jump_table + rax*8]

case_0:
    ; Handle case 0
    jmp done
case_1:
    ; Handle case 1
    jmp done
case_2:
    ; Handle case 2
    jmp done
default_case:
    ; Handle default
done:
```

Pattern 4: State Machine

```

; Simple state machine
state_loop:
    cmp byte [state], 0
    je state_0
    cmp byte [state], 1
    je state_1
    cmp byte [state], 2
    je state_2
    jmp error_invalid_state

state_0:
    ; Process state 0
    ; Update state
    jmp state_loop

state_1:
    ; Process state 1
    jmp state_loop

state_2:
    ; Final state
    jmp exit

error_invalid_state:
    ; Handle error
exit:

```

Practice Exercises

Exercise 1: Absolute Value

Implement `abs(rax)` using conditional jumps.

Solution

```

; abs(rax) -> rax
test rax, rax
jns already_positive    ; Jump if not sign (positive)
neg rax
already_positive:
; RAX now contains absolute value

```

Exercise 2: Clamp Function

Clamp RAX to range [min, max]. If RAX < min, set to min. If RAX > max, set to max. min = 10, max = 100

Solution

```

; Clamp rax to [10, 100]
cmp rax, 10
jge check_max      ; RAX >= 10, check upper bound
mov rax, 10        ; RAX < 10, set to 10
jmp done

check_max:
cmp rax, 100
jle done          ; RAX <= 100, already in range
mov rax, 100      ; RAX > 100, set to 100

done:
; RAX is now clamped to [10, 100]

```

Exercise 3: Count Down Loop

Write a loop that counts from 10 down to 1, summing the values in RAX.

Solution

```

xor rax, rax      ; sum = 0
mov rcx, 10       ; counter = 10

countdown_loop:
    add rax, rcx   ; sum += counter
    dec rcx       ; counter--
    jnz countdown_loop ; Loop if counter != 0

; RAX = 55 (1+2+3+...+10)

```

Exercise 4: Linear Search

Search for value 42 in an array of 10 numbers. Return index in RAX or -1 if not found.

Solution

```

section .data
    array dd 10, 5, 42, 7, 3, 42, 9, 1, 8, 6
    count equ 10

section .text
    xor rcx, rcx          ; index = 0

search_loop:
    cmp rcx, count
    jge not_found        ; Reached end

    cmp dword [array + rcx*4], 42
    je found             ; Found it!

    inc rcx
    jmp search_loop

found:
    mov rax, rcx          ; Return index
    jmp done

not_found:
    mov rax, -1           ; Return -1

done:

```

Exercise 5: Is Prime?

Check if RAX is a prime number. Set RBX = 1 if prime, 0 if not. (Assume RAX >= 2)

Solution

```

; Check if rax is prime
cmp rax, 2
je is_prime          ; 2 is prime

; Check if even
test rax, 1
jz not_prime        ; Even numbers (except 2) not prime

; Check odd divisors from 3 to sqrt(rax)
mov rbx, 3

check_divisor:
    ; Check if rbx * rbx > rax
    mov rcx, rbx
    imul rcx, rcx
    cmp rcx, rax
    jg is_prime      ; No divisors found, is prime

    ; Check if rax % rbx == 0
    mov rdx, rax
    push rax
    mov rax, rdx
    xor rdx, rdx
    div rbx
    test rdx, rdx
    pop rax
    jz not_prime      ; Found divisor, not prime

    add rbx, 2        ; Next odd number
    jmp check_divisor

is_prime:
    mov rbx, 1
    jmp done

not_prime:
    mov rbx, 0
done:

```

Quick Reference

Unconditional Jump

```

jmp label          ; Always jump

```


Equality Jumps

```
je/jz label          ; Jump if equal/zero (ZF=1)
jne/jnz label        ; Jump if not equal/zero (ZF=0)
```

Signed Comparisons

```
jg label            ; Jump if greater (signed)
jge label           ; Jump if greater or equal
jl label            ; Jump if less (signed)
jle label           ; Jump if less or equal
```

Unsigned Comparisons

```
ja label            ; Jump if above (unsigned)
jae label           ; Jump if above or equal
jb label            ; Jump if below (unsigned)
jbe label           ; Jump if below or equal
```

Single Flag Tests

```
js label            ; Jump if sign (negative)
jns label           ; Jump if not sign (positive/zero)
jc label            ; Jump if carry
jnc label           ; Jump if not carry
jo label            ; Jump if overflow
jno label           ; Jump if not overflow
```

Loop Instructions

```
loop label          ; dec rcx; jnz label
loope/loopz label   ; Loop if equal and rcx!=0
loopne/loopnz label ; Loop if not equal and rcx!=0
```

■

Knowledge Check

■

Before moving to Topic 6, verify you understand:

- How RIP controls program flow
- Unconditional jumps (JMP)
- Conditional jumps based on flags
- Signed vs unsigned comparison jumps

- Implementing if/else statements
- Building loops with jumps
- When to use LOOP instruction
- Jump optimization techniques

Excellent! You now master conditional jumps and control flow!

Next: [Topic 6: Loops](#) (coming soon)

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 23

Topic 6: Loops

Master loop patterns and iteration in assembly language.

Overview

Loops are fundamental programming constructs that let you repeat code. In assembly, you build loops using:

- Conditional jumps (from Topic 5)
- Counter registers (usually RCX)
- Comparison instructions
- Special loop instructions

Common loop types:

- Counted loops (repeat N times)
 - Condition-tested loops (while/until)
 - Infinite loops with breaks
 - Nested loops
 - Loop unrolling for performance
-

Part 1: Loop Fundamentals

Basic Loop Structure

Every loop needs three components:

1. Initialization (setup counter/condition)
2. Loop body (code to repeat)
3. Update and test (modify counter, check condition)

Generic Loop Pattern

```

; 1. Initialization
mov rcx, 10          ; Set counter

loop_start:          ; Loop label
; 2. Loop body
; ... do something ...

; 3. Update and test
dec rcx              ; Modify counter
jnz loop_start       ; Jump if not zero

; Continue after loop

```

Part 2: Counted Loops (For Loop)

Count Up (For i = 0 to N)

C Equivalent:

```

for (int i = 0; i < 10; i++) {
    // body
}

```

Assembly:

```

xor rcx, rcx          ; i = 0

loop_start:
cmp rcx, 10
jge loop_end          ; Exit if i >= 10

; Loop body here
; ... use RCX as index ...

inc rcx               ; i++
jmp loop_start

loop_end:

```

Count Down (For i = N to 0)

C Equivalent:

```
for (int i = 10; i > 0; i--) {
    // body
}
```

Assembly:

```
mov rcx, 10          ; i = 10

loop_start:
    ; Loop body here
    ; ... use RCX ...

    dec rcx          ; i--
    jnz loop_start   ; Loop if i != 0

    ; RCX is now 0
```

Why count down is popular:

- Shorter: `dec rcx; jnz` is 2 instructions
- vs count up: `inc rcx; cmp rcx, N; jl` is 3 instructions
- Automatically tests for zero

Example: Sum 1 to 10

```
section .text
global _start

_start:
    xor rax, rax      ; sum = 0
    mov rcx, 10       ; counter = 10

sum_loop:
    add rax, rcx      ; sum += counter
    dec rcx           ; counter--
    jnz sum_loop      ; Loop if counter != 0

    ; RAX = 55 (1+2+3+...+10)

    ; Exit
    mov rdi, rax
    mov rax, 60
    syscall
```

Part 3: Condition-Tested Loops

While Loop

C Equivalent:

```
while (condition) {
    // body
}
```

Assembly:

```
while_start:
    ; Test condition FIRST
    test rax, rax
    jz while_end        ; Exit if condition false

    ; Loop body
    ; ... modify variables ...

    jmp while_start     ; Loop back

while_end:
```

Example: While Not Empty

```
; Process array elements while not zero
section .data
    array dd 5, 10, 0, 15, 20 ; Terminates at first zero

section .text
    mov rsi, array            ; Pointer to array
    xor rax, rax              ; Sum = 0

while_loop:
    cmp dword [rsi], 0        ; Check for zero terminator
    je while_done

    add eax, [rsi]            ; sum += *ptr
    add rsi, 4                ; ptr++
    jmp while_loop

while_done:
    ; RAX = 15 (5 + 10)
```

Do-While Loop

C Equivalent:

```
do {
    // body
} while (condition);
```

Assembly:

```
do_start:
    ; Loop body FIRST
    ; ... code ...

    ; Test condition LAST
    test rax, rax
    jnz do_start          ; Continue if condition true

    ; Exit loop
```

Example: Input Validation

```
; Keep reading until valid input (non-zero)
do_loop:
    ; Read input (simplified)
    mov rax, 0          ; sys_read
    mov rdi, 0           ; stdin
    mov rsi, buffer
    mov rdx, 1
    syscall

    ; Check if valid
    cmp byte [buffer], '0'
    je do_loop          ; Repeat if zero

    ; Valid input received
```

Until Loop (Repeat Until)

Same as do-while but inverted condition:

```

repeat_start:
    ; Loop body
    ; ...

    ; Continue until condition becomes true
    test rax, rax
    jz repeat_start      ; Loop while false

    ; Condition is now true, exit

```

Part 4: The LOOP Instruction

LOOP - Hardware Loop Support

```

loop label      ; Equivalent to: dec rcx; jnz label

```

Automatic behavior:

- Decrements RCX
- Jumps if RCX != 0
- Very compact encoding

Example:

```

    mov rcx, 10      ; Loop 10 times

loop_start:
    ; Loop body
    ; ...
    loop loop_start  ; Decrement RCX, jump if != 0

    ; RCX is now 0

```

LOOP Variants

```

loope/loopz label    ; Loop while equal/zero AND rcx != 0
loopne/loopnz label  ; Loop while not equal/zero AND rcx != 0

```

LOOPE Example:


```

; Find first non-match in strings
mov rcx, string_len
mov rsi, string1
mov rdi, string2

compare_loop:
mov al, [rsi]
cmp al, [rdi]
loopne compare_loop    ; Continue if not equal AND rcx != 0
; Either found match or reached end

```

When to Use LOOP

Use LOOP when:

- Simple counted loop
- RCX is your counter
- No complex exit conditions
- Code clarity matters

Avoid LOOP when:

- Need other exit conditions
- RCX needed for other purposes
- Performance critical (modern CPUs: manual loop may be faster)
- Complex loop logic

Performance Note: On modern CPUs (Intel/AMD), `dec rcx; jnz` is often faster than `loop` due to macro-op fusion.

Part 5: Loop Patterns

Pattern 1: Array Iteration

```
section .data
    array dd 10, 20, 30, 40, 50
    count equ 5

section .text
    mov rsi, array      ; Pointer to array
    xor rcx, rcx        ; Index = 0

array_loop:
    cmp rcx, count
    jge array_done

    ; Access array[rcx]
    mov eax, [rsi + rcx*4]

    ; Process element in EAX
    ; ...

    inc rcx
    jmp array_loop

array_done:
```

Pattern 2: String Traversal

```
section .data
    string db "Hello", 0

section .text
    mov rsi, string
    xor rcx, rcx        ; Length = 0

strlen_loop:
    cmp byte [rsi + rcx], 0 ; Check for null terminator
    je strlen_done

    inc rcx              ; length++
    jmp strlen_loop

strlen_done:
    ; RCX contains string length
```

Pattern 3: Search/Find

```

; Find value in array
section .data
    array dd 5, 10, 15, 20, 25
    count equ 5
    target dd 15

section .text
    mov edi, [target]
    xor rcx, rcx          ; Index = 0

search_loop:
    cmp rcx, count
    jge not_found        ; Searched entire array

    cmp edi, [array + rcx*4]
    je found             ; Match!

    inc rcx
    jmp search_loop

found:
    ; Found at index RCX
    mov rax, rcx
    jmp done

not_found:
    mov rax, -1          ; Not found

done:

```

Pattern 4: Accumulation

```
; Sum array elements
section .data
    numbers dd 1, 2, 3, 4, 5
    count equ 5

section .text
    xor rax, rax           ; sum = 0
    xor rcx, rcx          ; index = 0

sum_loop:
    cmp rcx, count
    jge sum_done

    add eax, [numbers + rcx*4] ; sum += numbers[index]
    inc rcx
    jmp sum_loop

sum_done:
    ; RAX contains sum (15)
```

Pattern 5: Filter/Count

```
; Count even numbers in array
section .data
    array dd 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
    count equ 10

section .text
    xor rax, rax           ; even_count = 0
    xor rcx, rcx           ; index = 0

count_loop:
    cmp rcx, count
    jge count_done

    mov edx, [array + rcx*4]
    test edx, 1            ; Check if odd (bit 0 set)
    jnz skip_count         ; Skip if odd

    inc rax                ; even_count++

skip_count:
    inc rcx
    jmp count_loop

count_done:
    ; RAX contains count of even numbers (5)
```

Part 6: Loop Control

Break (Early Exit)

C Code:

```
for (i = 0; i < 10; i++) {
    if (condition) break;
    // body
}
```

Assembly:

```

    xor rcx, rcx

loop_start:
    cmp rcx, 10
    jge loop_end

    ; Check break condition
    test rax, rax
    jz break_loop      ; Break if condition met

    ; Loop body
    ; ...

    inc rcx
    jmp loop_start

break_loop:
    ; Exited early

loop_end:
    ; Normal or early exit

```

Continue (Skip to Next Iteration)

C Code:

```

for (i = 0; i < 10; i++) {
    if (condition) continue;
    // body
}

```

Assembly:

```
xor rcx, rcx

loop_start:
    cmp rcx, 10
    jge loop_end

    ; Check continue condition
    test rax, rax
    jnz continue_loop      ; Skip body if condition met

    ; Loop body (skipped if continue)
    ; ...

continue_loop:
    inc rcx
    jmp loop_start

loop_end:
```

Multiple Exit Conditions

```

; Loop with multiple ways to exit
xor rcx, rcx

multi_exit_loop:
    ; Exit condition 1: counter limit
    cmp rcx, 100
    jge exit_limit_reached

    ; Exit condition 2: found zero
    cmp dword [array + rcx*4], 0
    je exit_zero_found

    ; Exit condition 3: error flag
    test byte [error_flag], 1
    jnz exit_error

    ; Loop body
    ; ...

    inc rcx
    jmp multi_exit_loop

exit_limit_reached:
    mov rax, 1
    jmp done

exit_zero_found:
    mov rax, 2
    jmp done

exit_error:
    mov rax, -1

done:
    ; RAX indicates exit reason

```

Part 7: Nested Loops

Two-Level Nesting

C Code:


```

for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        // body
    }
}

```

Assembly:

```

    mov rbx, 3                ; rows (outer loop)

outer_loop:
    test rbx, rbx
    jz  outer_done

    mov rcx, 4                ; cols (inner loop)

inner_loop:
    test rcx, rcx
    jz  inner_done

    ; Loop body - access [rbx][rcx]
    ; ...

    dec rcx
    jmp inner_loop

inner_done:
    dec rbx
    jmp outer_loop

outer_done:

```

Matrix Traversal

```

; Process 3x3 matrix
section .data
    matrix dd 1, 2, 3
            dd 4, 5, 6
            dd 7, 8, 9
    rows equ 3
    cols equ 3

section .text
    xor rbx, rbx          ; row = 0

row_loop:
    cmp rbx, rows
    jge matrix_done

    xor rcx, rcx          ; col = 0

col_loop:
    cmp rcx, cols
    jge col_done

    ; Calculate offset: (row * cols + col) * 4
    mov rax, rbx
    imul rax, cols
    add rax, rcx

    ; Access matrix[row][col]
    mov edx, [matrix + rax*4]

    ; Process element in EDX
    ; ...

    inc rcx
    jmp col_loop

col_done:
    inc rbx
    jmp row_loop

matrix_done:

```

Nested Loop with Break

```

; Search 2D array, break on first match
xor rbx, rbx          ; i = 0

outer:
    cmp rbx, rows
    jge not_found_2d

    xor rcx, rcx        ; j = 0

inner:
    cmp rcx, cols
    jge inner_done

    ; Check for match
    mov rax, rbx
    imul rax, cols
    add rax, rcx
    cmp dword [array + rax*4], target
    je found_2d         ; Break both loops

    inc rcx
    jmp inner

inner_done:
    inc rbx
    jmp outer

found_2d:
    ; Found at [rbx][rcx]
    ; ...
    jmp search_done

not_found_2d:
    ; Not found

search_done:

```

Part 8: Loop Optimization

Technique 1: Count Down Instead of Up

```

;    Count up (longer)
xor rcx, rcx
loop_up:
    cmp rcx, 100
    jge loop_up_done
    ; body
    inc rcx
    jmp loop_up
loop_up_done:

;    Count down (shorter)
mov rcx, 100
loop_down:
    ; body
    dec rcx
    jnz loop_down

```

Technique 2: Strength Reduction

Replace expensive operations with cheaper ones:

```

;    Multiplication in loop
mov rcx, 100
loop_mul:
    mov rax, rcx
    imul rax, 4           ; Expensive!
    ; use RAX
    dec rcx
    jnz loop_mul

;    Addition instead
mov rcx, 100
mov rax, 400             ; Start at end
loop_add:
    sub rax, 4           ; Cheaper!
    ; use RAX
    dec rcx
    jnz loop_add

```

Technique 3: Loop Unrolling

Process multiple iterations at once:

```

;   Normal loop (1 element per iteration)
mov rcx, 100
loop_normal:
    mov eax, [array + rcx*4]
    add [sum], eax
    dec rcx
    jnz loop_normal

;   Unrolled (4 elements per iteration)
mov rcx, 25 ; 100/4 iterations
loop_unrolled:
    mov eax, [array + rcx*16] ; element 0
    add eax, [array + rcx*16 + 4] ; element 1
    add eax, [array + rcx*16 + 8] ; element 2
    add eax, [array + rcx*16 + 12] ; element 3
    add [sum], eax
    dec rcx
    jnz loop_unrolled

```

Benefits:

- Fewer jumps
- Better instruction pipeline utilization
- Less loop overhead

Drawbacks:

- More code size
- Need to handle remainder

Technique 4: Pointer Arithmetic

```

;    Index-based (calculates address each time)
xor rcx, rcx
loop_index:
    cmp rcx, count
    jge done
    mov eax, [array + rcx*4] ; Calculate address
    inc rcx
    jmp loop_index

;    Pointer-based (increment pointer)
mov rsi, array
mov rcx, count
loop_pointer:
    mov eax, [rsi]           ; Use pointer directly
    add rsi, 4               ; Advance pointer
    dec rcx
    jnz loop_pointer

```

Technique 5: Eliminate Invariant Code

Move code that doesn't change out of loop:

```

;    Calculation inside loop
mov rcx, 100
loop_invariant:
    mov rax, base_address ; Doesn't change!
    add rax, offset       ; Doesn't change!
    mov edx, [rax + rcx*4]
    dec rcx
    jnz loop_invariant

;    Calculate once before loop
mov rax, base_address
add rax, offset ; Calculate once
mov rcx, 100
loop_optimized:
    mov edx, [rax + rcx*4]
    dec rcx
    jnz loop_optimized

```

Part 9: Advanced Loop Techniques

Technique 1: Duff's Device (Partial Unrolling)

```

; Process array with partial unrolling
mov rcx, count
mov rsi, array

; Calculate remainder
mov rax, rcx
and rax, 3          ; remainder = count % 4
shr rcx, 2          ; iterations = count / 4

; Handle remainder first
jmp [remainder_table + rax*8]

remainder_3:
mov eax, [rsi]
add rsi, 4
; process

remainder_2:
mov eax, [rsi]
add rsi, 4
; process

remainder_1:
mov eax, [rsi]
add rsi, 4
; process

remainder_0:
; Main unrolled loop
test rcx, rcx
jz done

main_loop:
mov eax, [rsi]      ; 1st
add rsi, 4
mov eax, [rsi]      ; 2nd
add rsi, 4
mov eax, [rsi]      ; 3rd
add rsi, 4
mov eax, [rsi]      ; 4th
add rsi, 4

dec rcx
jnz main_loop

done:

```


Technique 2: Sentinel Values

```
; Search without bounds checking
section .data
    array dd 5, 10, 15, 20, 9999 ; Sentinel at end

section .text
    mov edi, 15 ; Search for 15
    mov rsi, array

search_sentinel:
    cmp dword [rsi], edi
    je found_sentinel ; No bounds check needed!
    add rsi, 4
    jmp search_sentinel

found_sentinel:
    ; Check if we found sentinel or actual value
    cmp dword [rsi], 9999
    je not_found_sentinel
    ; Found the value
```

Technique 3: Loop Reversal

```
; Sometimes reversing loop order improves cache performance
; Forward (may cause cache misses in some scenarios)
loop_forward:
    ; Access array[i]
    ; Access other_array[i]

; Backward (better cache locality in some cases)
mov rcx, count
loop_backward:
    dec rcx
    ; Access array[rcx]
    ; Access other_array[rcx]
    test rcx, rcx
    jnz loop_backward
```

Part 10: Practical Examples

Example 1: Bubble Sort

C Equivalent:

```

#include <stdint.h>

void bubble_sort(int32_t *array, int32_t count) {
    for (int32_t i = count - 1; i > 0; i--) {
        for (int32_t j = 0; j < i; j++) {
            if (array[j] > array[j + 1]) {
                // Swap
                int32_t temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}

int main() {
    int32_t array[] = {64, 34, 25, 12, 22, 11, 90};
    int32_t count = 7;

    bubble_sort(array, count);
    // array is now: {11, 12, 22, 25, 34, 64, 90}

    return 0;
}

```

Assembly:

```

section .data
    array dd 64, 34, 25, 12, 22, 11, 90
    count equ 7

section .text
    global _start

_start:
    ; Bubble sort
    mov rbx, count
    dec rbx                ; outer loop: n-1 passes

outer_sort:
    test rbx, rbx
    jz sort_done

    xor rcx, rcx           ; inner loop: compare adjacent

inner_sort:
    cmp rcx, rbx
    jge inner_done

    ; Compare array[rcx] with array[rcx+1]
    mov eax, [array + rcx*4]
    mov edx, [array + rcx*4 + 4]
    cmp eax, edx
    jle no_swap

    ; Swap
    mov [array + rcx*4], edx
    mov [array + rcx*4 + 4], eax

no_swap:
    inc rcx
    jmp inner_sort

inner_done:
    dec rbx
    jmp outer_sort

sort_done:
    ; Array is now sorted

    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: String Copy

C Equivalent:

```
#include <stdint.h>

void my_strcpy(char *dest, const char *source) {
    while (*source != '\0') {
        *dest = *source;
        dest++;
        source++;
    }
    *dest = '\0'; // Copy null terminator
}

int main() {
    const char *source = "Hello, World!";
    char dest[50];

    my_strcpy(dest, source);
    // dest now contains "Hello, World!"

    return 0;
}
```

Assembly:

```

section .data
    source db "Hello, World!", 0

section .bss
    dest resb 50

section .text
    global _start

_start:
    ; strcpy
    mov rsi, source
    mov rdi, dest

strcpy_loop:
    mov al, [rsi]
    mov [rdi], al

    test al, al          ; Check for null terminator
    jz strcpy_done

    inc rsi
    inc rdi
    jmp strcpy_loop

strcpy_done:
    ; String copied

    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 3: Fibonacci Sequence

C Equivalent:

```
#include <stdint.h>

void calculate_fibonacci(int32_t *fib, int32_t count) {
    fib[0] = 0;
    fib[1] = 1;

    for (int32_t i = 2; i < count; i++) {
        fib[i] = fib[i - 1] + fib[i - 2];
    }
}

int main() {
    int32_t fibonacci[10];

    calculate_fibonacci(fibonacci, 10);
    // fibonacci array contains: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

    return 0;
}
```

Assembly:

```

; Calculate first 10 Fibonacci numbers
section .bss
    fibonacci resd 10

section .text
    global _start

_start:
    ; fib[0] = 0, fib[1] = 1
    mov dword [fibonacci], 0
    mov dword [fibonacci + 4], 1

    mov rcx, 2          ; Start from index 2

fib_loop:
    cmp rcx, 10
    jge fib_done

    ; fib[n] = fib[n-1] + fib[n-2]
    mov eax, [fibonacci + rcx*4 - 4] ; fib[n-1]
    add eax, [fibonacci + rcx*4 - 8] ; + fib[n-2]
    mov [fibonacci + rcx*4], eax

    inc rcx
    jmp fib_loop

fib_done:
    ; fibonacci array contains: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 4: Prime Sieve (Sieve of Eratosthenes)


```

section .bss
    sieve resb 101          ; Sieve for numbers 0-100

section .text
    global _start

_start:
    ; Initialize sieve (1 = prime, 0 = composite)
    mov rdi, sieve
    mov rcx, 101
    mov al, 1
    rep stosb               ; Fill with 1s

    ; 0 and 1 are not prime
    mov byte [sieve], 0
    mov byte [sieve + 1], 0

    ; Sieve algorithm
    mov rbx, 2              ; Start with 2

sieve_outer:
    cmp rbx, 10             ; Only need to check up to sqrt(100)
    jg sieve_done

    ; Skip if already marked composite
    cmp byte [sieve + rbx], 0
    je sieve_next_outer

    ; Mark multiples of rbx as composite
    mov rcx, rbx
    add rcx, rbx             ; Start at 2*rbx

sieve_inner:
    cmp rcx, 100
    jg sieve_inner_done

    mov byte [sieve + rcx], 0 ; Mark as composite
    add rcx, rbx             ; Next multiple
    jmp sieve_inner

sieve_inner_done:
sieve_next_outer:
    inc rbx
    jmp sieve_outer

sieve_done:
    ; sieve array now contains 1 for primes, 0 for composites

    mov rax, 60

```

```
xor rdi, rdi  
syscall
```

Example 5: Matrix Multiplication

```

; Multiply two 2x2 matrices
section .data
    matrixA dd 1, 2
             dd 3, 4
    matrixB dd 5, 6
             dd 7, 8

section .bss
    result resd 4          ; 2x2 result matrix

section .text
    global _start

_start:
    xor rbx, rbx           ; row = 0

mat_row:
    cmp rbx, 2
    jge mat_done

    xor rcx, rcx           ; col = 0

mat_col:
    cmp rcx, 2
    jge mat_col_done

    ; Calculate result[row][col] = sum of A[row][k] * B[k][col]
    xor eax, eax           ; sum = 0
    xor rdx, rdx           ; k = 0

mat_inner:
    cmp rdx, 2
    jge mat_inner_done

    ; Get A[row][k]
    mov rsi, rbx
    shl rsi, 1             ; row * 2
    add rsi, rdx
    mov edi, [matrixA + rsi*4]

    ; Get B[k][col]
    mov rsi, rdx
    shl rsi, 1             ; k * 2
    add rsi, rcx
    imul edi, [matrixB + rsi*4]

    ; sum += A[row][k] * B[k][col]
    add eax, edi

    inc rdx

```

```

        jmp mat_inner

mat_inner_done:
    ; Store result[row][col]
    mov rsi, rbx
    shl rsi, 1
    add rsi, rcx
    mov [result + rsi*4], eax

    inc rcx
    jmp mat_col

mat_col_done:
    inc rbx
    jmp mat_row

mat_done:
    ; result now contains: 19 22
    ;                      43 50

    mov rax, 60
    xor rdi, rdi
    syscall

```

Practice Exercises

Exercise 1: Factorial

Calculate factorial of 5 using a loop.

Solution

```

mov rax, 1          ; result = 1
mov rcx, 5          ; n = 5

factorial_loop:
    imul rax, rcx    ; result *= n
    dec rcx          ; n--
    jnz factorial_loop

; RAX = 120 (5!)

```

Exercise 2: Reverse Array

Reverse an array in place.

Solution

```

section .data
    array dd 1, 2, 3, 4, 5
    count equ 5

section .text
    xor rcx, rcx          ; left = 0
    mov rdx, count
    dec rdx              ; right = count - 1

reverse_loop:
    cmp rcx, rdx
    jge reverse_done

    ; Swap array[left] with array[right]
    mov eax, [array + rcx*4]
    mov ebx, [array + rdx*4]
    mov [array + rcx*4], ebx
    mov [array + rdx*4], eax

    inc rcx              ; left++
    dec rdx              ; right--
    jmp reverse_loop

reverse_done:
    ; array is now: 5, 4, 3, 2, 1

```

Exercise 3: Count Vowels

Count vowels (a,e,i,o,u) in a string.

Solution

```

section .data
    string db "Hello World", 0

section .text
    mov rsi, string
    xor rax, rax           ; count = 0

vowel_loop:
    mov cl, [rsi]
    test cl, cl
    jz vowel_done

    ; Check lowercase vowels
    cmp cl, 'a'
    je is_vowel
    cmp cl, 'e'
    je is_vowel
    cmp cl, 'i'
    je is_vowel
    cmp cl, 'o'
    je is_vowel
    cmp cl, 'u'
    je is_vowel
    jmp next_char

is_vowel:
    inc rax

next_char:
    inc rsi
    jmp vowel_loop

vowel_done:
    ; RAX = 3 (e, o, o)

```

Exercise 4: GCD (Greatest Common Divisor)

Implement Euclid's algorithm with a loop.

Solution

```

; GCD(48, 18) using Euclidean algorithm
mov rax, 48
mov rbx, 18

gcd_loop:
    test rbx, rbx
    jz gcd_done

    ; temp = a % b
    xor rdx, rdx
    div rbx

    ; a = b, b = temp
    mov rax, rbx
    mov rbx, rdx
    jmp gcd_loop

gcd_done:
; RAX = 6 (GCD of 48 and 18)

```

Exercise 5: Sum of Digits

Calculate sum of digits of a number.

Solution

```

; Sum digits of 12345
mov rax, 12345
xor rbx, rbx          ; sum = 0

digit_loop:
    test rax, rax
    jz digit_done

    ; Get last digit
    xor rdx, rdx
    mov rcx, 10
    div rcx            ; RAX = number / 10, RDX = digit

    add rbx, rdx        ; sum += digit
    jmp digit_loop

digit_done:
; RBX = 15 (1+2+3+4+5)

```


Quick Reference

Basic Loop Patterns

```

; Count down (most efficient)
mov rcx, N
loop_down:
    ; body
    dec rcx
    jnz loop_down

; Count up
xor rcx, rcx
loop_up:
    cmp rcx, N
    jge done
    ; body
    inc rcx
    jmp loop_up
done:

; While
while_start:
    test condition
    jz while_end
    ; body
    jmp while_start
while_end:

; Do-while
do_start:
    ; body
    test condition
    jnz do_start

```

Loop Instructions

```

loop label           ; dec rcx; jnz label
loope label          ; Loop if ZF=1 and RCX!=0
loopne label         ; Loop if ZF=0 and RCX!=0

```

Common Optimizations

- Count down instead of up
- Use pointer arithmetic
- Unroll loops for performance

- Move invariant code outside loop
- Use strength reduction

Knowledge Check

Before moving to Topic 7, verify you understand:

- Counted loops (for equivalent)
- Condition-tested loops (while/do-while)
- LOOP instruction and variants
- Loop control (break/continue)
- Nested loops
- Loop optimization techniques
- Common loop patterns

Excellent! You now master loops in assembly!

Next: [Topic 7: Stack Operations](#) (coming soon)

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 24

Topic 7: The Stack

Master the stack - your program's temporary workspace for function calls, local variables, and more.

Overview

The **stack** is a special region of memory used for:

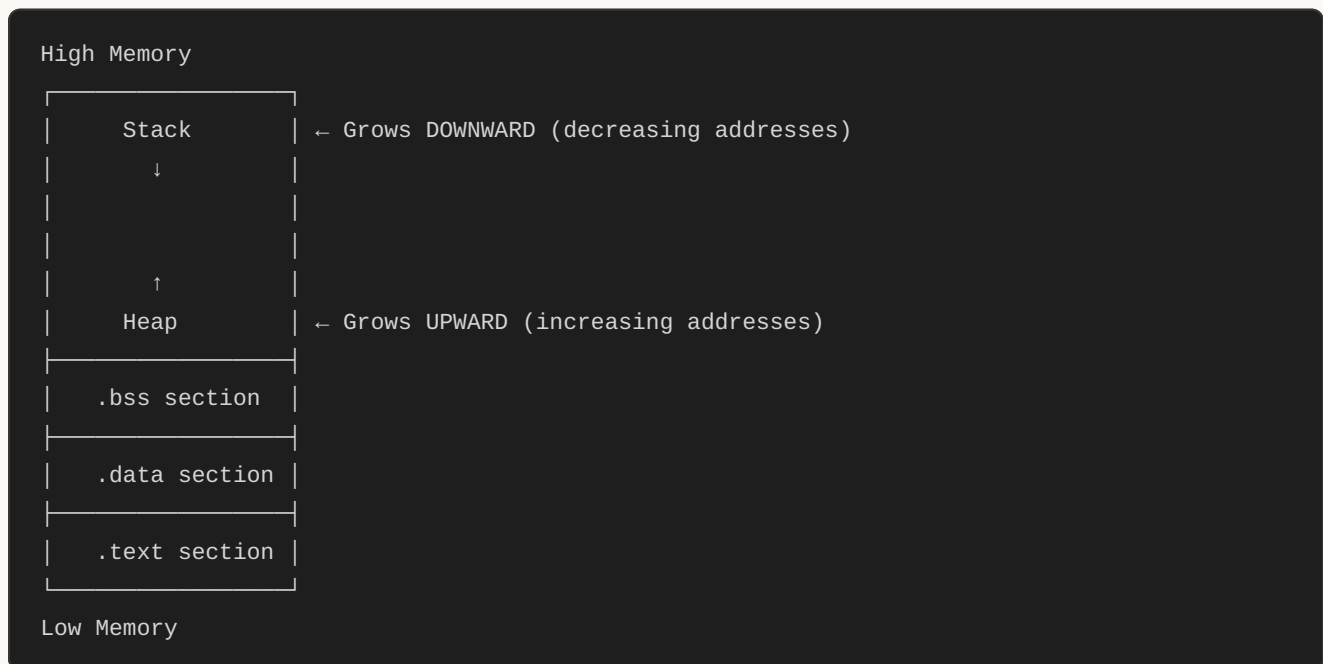
- Temporary storage (push/pop)
- Function calls and returns
- Local variables
- Preserving register values
- Passing function arguments (in 32-bit)

Part 1: What is the Stack?

The Stack Memory Region



ASCII diagram



Key Characteristics

- **LIFO:** Last In, First Out (like a stack of plates)
- **Grows downward:** Adding items decreases RSP
- **Automatic management:** CPU tracks top with RSP register
- **Fast access:** Directly in CPU's L1 cache
- **Limited size:** Typically 1-8 MB (can overflow!)

Part 2: Stack Pointer (RSP)

The RSP Register

```

; RSP = Register Stack Pointer
; Always points to the TOP of the stack (most recently pushed item)

; Example stack state:
; RSP → 0x7fffffff0000: [0x42] ← Top of stack
;      0x7fffffff0008: [0x10]
;      0x7fffffff0010: [0x99]

```

Critical Rules

```
; 1. NEVER manually modify RSP unless you know what you're doing
mov rsp, rax           ; DANGEROUS! Can crash your program

; 2. Keep stack aligned to 16 bytes (System V ABI requirement)
; 3. Balance every PUSH with a POP (or stack adjustment)
```

Part 3: PUSH Instruction

Syntax

```
push operand           ; Push operand onto stack
```

What PUSH Does (Step-by-Step)

```
push rax

; Equivalent to:
sub rsp, 8           ; 1. Decrease stack pointer by 8 bytes
mov [rsp], rax       ; 2. Store RAX at new top of stack
```

Visual Example

C Equivalent:

```
// Conceptual stack implementation
#define STACK_SIZE 1024
uint64_t stack[STACK_SIZE];
int stack_pointer = STACK_SIZE; // Points to next free slot

void push(uint64_t value) {
    stack_pointer--;           // Grow downward
    stack[stack_pointer] = value;
}
```

Assembly:

```

; Before: RSP = 0x1000
push rax          ; RAX = 0x42

; After:  RSP = 0x0FF8 (0x1000 - 8)
;         [0x0FF8] = 0x42

```

Stack Memory:

```

0x0FF8: [0x42] ← RSP (new top)
0x1000: [old data]

```

PUSH Examples

```

; Push register
push rax
push rbx
push rcx

; Push immediate (sign-extended)
push 42
push 0x100

; Push memory
push qword [var]
push qword [rax + 8]

; Push 32-bit (decreases RSP by 8, but only stores 4 bytes - padding added)
push eax          ; Actually pushes 8 bytes (upper 4 are sign-extended)

```

Part 4: POP Instruction

Syntax

```

pop operand          ; Pop from stack into operand

```

What POP Does (Step-by-Step)

```

pop rbx

; Equivalent to:
mov rbx, [rsp]      ; 1. Load value from top of stack
add rsp, 8          ; 2. Increase stack pointer by 8 bytes

```

Visual Example

C Equivalent:

```
uint64_t pop() {
    uint64_t value = stack[stack_pointer];
    stack_pointer++;    // Shrink upward
    return value;
}
```

Assembly:

```
; Before: RSP = 0x0FF8
;         [0x0FF8] = 0x42
pop rbx

; After:  RSP = 0x1000 (0x0FF8 + 8)
;         RBX = 0x42

Stack Memory:
0x0FF8: [0x42] ← Old data (not erased, just ignored)
0x1000: [...] ← RSP (new top)
```

POP Examples

```
; Pop to register
pop rax
pop rbx

; Pop and discard (common for cleanup)
add rsp, 8          ; More efficient than: pop rax (when value not needed)

; Cannot pop to immediate
pop 42              ; ERROR!

; Can pop to memory
pop qword [var]
```

Part 5: Common Stack Patterns

Pattern 1: Temporary Storage

C Equivalent:

```
int compute(int a, int b) {
    int temp_rax = a + b; // Save intermediate result
    int result = temp_rax * 2;
    return result;
}
```

Assembly:

```
; Need to use RAX but want to preserve its value
push rax ; Save RAX
; ... do work that modifies RAX ...
mov rax, [value]
add rax, 100
pop rax ; Restore original RAX
```

Pattern 2: Register Preservation**C Equivalent:**

```
void my_function() {
    // C compiler automatically saves registers that need preservation
    int saved_rbx = /* current rbx value */;
    // ... function body ...
    // Restore rbx before return
}
```

Assembly:

```
my_function:
    ; Save callee-saved registers
    push rbx
    push r12
    push r13

    ; Function body (can freely use RBX, R12, R13)
    mov rbx, 100
    ; ...

    ; Restore registers in REVERSE order
    pop r13
    pop r12
    pop rbx
    ret
```

Pattern 3: Swap Using Stack**C Equivalent:**


```
void swap_with_stack(int *a, int *b) {
    int temp1 = *a;
    int temp2 = *b;
    *a = temp2;
    *b = temp1;
}
```

Assembly:

```
; Swap RAX and RBX using stack
push rax           ; Stack: [rax]
push rbx           ; Stack: [rbx, rax]
pop rax            ; RAX = old RBX, Stack: [rax]
pop rbx            ; RBX = old RAX, Stack: []
```

Pattern 4: Multiple Register Save/Restore**C Equivalent:**

```
void preserve_all_registers() {
    // Save all general-purpose registers
    uint64_t saved[16];
    // ... assembly handles this with push/pop ...
}
```

Assembly:

```
; Save all volatile registers before syscall
push rax
push rcx
push rdx
push rsi
push rdi
push r8
push r9
push r10
push r11

; ... make syscall ...

; Restore in reverse order
pop r11
pop r10
pop r9
pop r8
pop rdi
pop rsi
pop rdx
pop rcx
pop rax
```

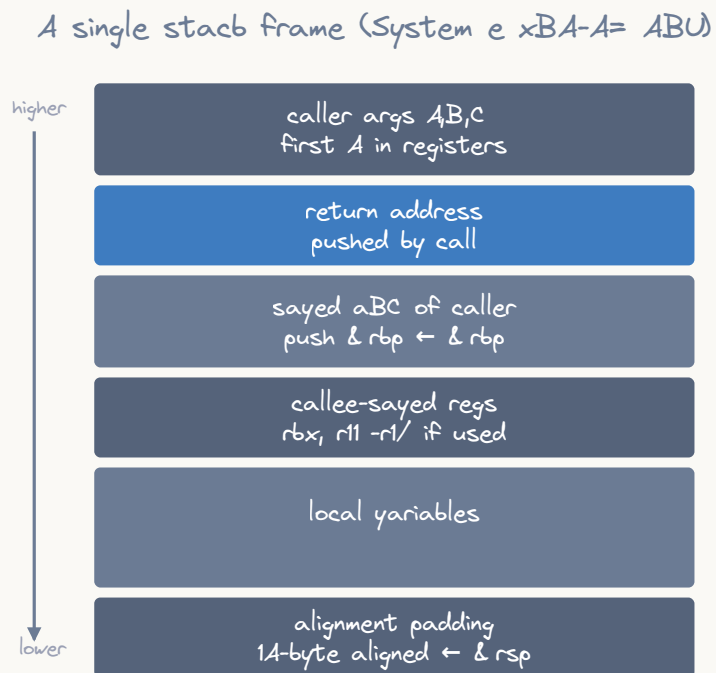
Part 6: Stack Frames

What is a Stack Frame?

A **stack frame** is a section of the stack dedicated to a single function call, containing:

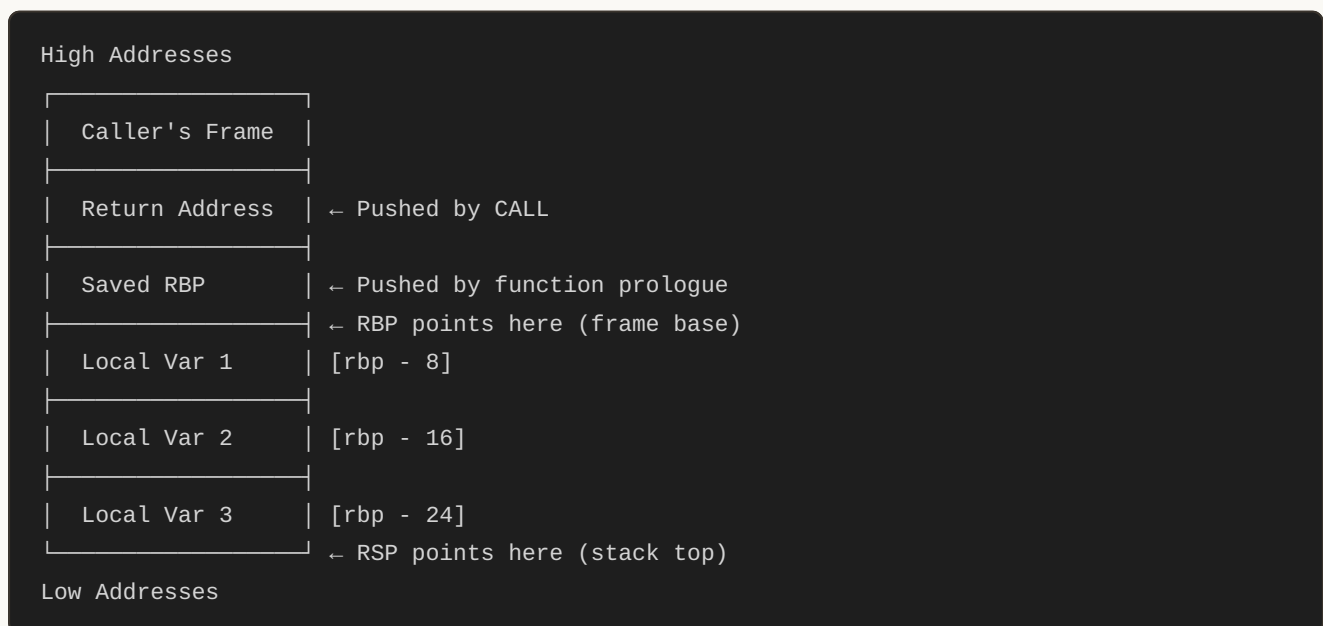
- Return address
- Saved registers
- Local variables
- Function parameters (in 32-bit)

Stack Frame Layout



stack grows downward; args in rdi,rsi,rdx,rcx,r8,r9; return in rax

ASCII diagram



Function Prologue (Setup)

C Equivalent:

```
void my_function() {
    // Compiler generates prologue automatically
    // Saves frame pointer, allocates locals
    int local1, local2, local3;
    // ... function body ...
}
```

Assembly:

```
my_function:
    ; === PROLOGUE ===
    push rbp                ; Save caller's base pointer
    mov rbp, rsp            ; Set up new frame base
    sub rsp, 32             ; Allocate space for local variables

    ; === FUNCTION BODY ===
    ; Access locals relative to RBP:
    mov qword [rbp - 8], 100 ; local1 = 100
    mov qword [rbp - 16], 200 ; local2 = 200
    mov qword [rbp - 24], 300 ; local3 = 300

    ; === EPILOGUE ===
    mov rsp, rbp            ; Deallocate locals (restore RSP)
    pop rbp                 ; Restore caller's base pointer
    ret                     ; Return to caller
```

Alternative: ENTER and LEAVE

Assembly:

```
my_function:
    ; ENTER creates stack frame
    enter 32, 0              ; Equivalent to: push rbp; mov rbp, rsp; sub rsp, 32

    ; Function body
    mov qword [rbp - 8], 42

    ; LEAVE destroys stack frame
    leave                    ; Equivalent to: mov rsp, rbp; pop rbp
    ret

; Note: ENTER/LEAVE are slower than manual prologue/epilogue!
```

Part 7: Calling Conventions (System V AMD64)

Register Usage Rules

```
| CALLER-SAVED (Volatile) |  
| Caller must save these before calling |  
| RAX, RCX, RDX, RSI, RDI, R8-R11 |
```

```
| CALLEE-SAVED (Non-volatile) |  
| Function must preserve these |  
| RBX, RBP, R12-R15 |
```

```
| SPECIAL |  
| RSP: Stack pointer (always callee-saved) |
```

Function Call Example

C Equivalent:

```
int add_three(int a, int b, int c) {  
    return a + b + c;  
}  
  
int main() {  
    int result = add_three(10, 20, 30);  
    return result;  
}
```

Assembly:

```

add_three:
    ; Arguments arrive in: RDI=a, RSI=b, RDX=c
    ; No need to save caller-saved regs (we don't call anyone)

    mov rax, rdi        ; RAX = a
    add rax, rsi        ; RAX = a + b
    add rax, rdx        ; RAX = a + b + c
    ret                ; Return value in RAX

main:
    ; Set up arguments
    mov rdi, 10         ; 1st arg
    mov rsi, 20         ; 2nd arg
    mov rdx, 30         ; 3rd arg

    call add_three      ; Call function
    ; Result in RAX = 60

    ; Exit
    mov rdi, rax
    mov rax, 60
    syscall

```

Part 8: Stack Alignment

16-Byte Alignment Requirement

The Rule: RSP must be aligned to 16 bytes at function call boundaries.

```

; Before CALL: RSP must be 16-byte aligned
; After CALL: RSP is misaligned (CALL pushes 8-byte return address)
; Inside function: Prologue must restore alignment

main:
    ; RSP is 16-byte aligned here

    push rbp            ; RSP now misaligned (-8)
    mov rbp, rsp
    sub rsp, 16         ; Allocate 16 bytes (now aligned again!)

    call other_func     ; RSP must be 16-aligned before this!

    leave
    ret

```

Why Alignment Matters

C Equivalent (conceptual):

```
// SSE/AVX instructions require aligned memory
#include <xmmintrin.h>

void example() {
    __m128 vec; // Must be 16-byte aligned
    // If stack is misaligned, this crashes!
}
```

Assembly:

```
; Aligned load (fast)
movaps xmm0, [rsp] ; Works if RSP is 16-aligned
                  ; CRASHES if misaligned!

; Unaligned load (slower but safe)
movups xmm0, [rsp] ; Always works (slower)
```

Part 9: Practical Examples

Example 1: Factorial with Stack

C Equivalent:

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int main() {
    int result = factorial(5);
    return result; // 120
}
```

Assembly:

```

section .text
    global factorial

factorial:
    ; RDI = n
    push rbp
    mov rbp, rsp
    push rbx                ; Save RBX (callee-saved)

    ; Base case: if (n <= 1) return 1
    cmp rdi, 1
    jg recursive_case
    mov rax, 1              ; Return 1
    jmp factorial_done

recursive_case:
    mov rbx, rdi            ; Save n in RBX
    dec rdi                 ; n - 1
    call factorial          ; factorial(n - 1)
    imul rax, rbx           ; n * factorial(n-1)

factorial_done:
    pop rbx                 ; Restore RBX
    pop rbp
    ret

```

Example 2: Nested Function Calls

C Equivalent:

```

int func_a(int x) {
    return x * 2;
}

int func_b(int x) {
    int temp = func_a(x);
    return temp + 10;
}

int main() {
    int result = func_b(5); // result = 20
    return 0;
}

```

Assembly:


```

section .text
    global main

func_a:
    ; RDI = x
    mov rax, rdi
    shl rax, 1          ; x * 2
    ret

func_b:
    ; RDI = x
    push rbp
    mov rbp, rsp
    sub rsp, 16         ; Align stack

    ; RDI already contains x
    call func_a         ; temp = func_a(x)
    add rax, 10         ; temp + 10

    leave
    ret

main:
    mov rdi, 5
    call func_b         ; RAX = 20

    mov rdi, rax
    mov rax, 60
    syscall

```

Example 3: Local Variables

C Equivalent:

```

int compute() {
    int a = 10;
    int b = 20;
    int c = 30;
    int sum = a + b + c;
    return sum;
}

int main() {
    int result = compute();
    return result;
}

```

Assembly:

```

section .text
    global compute

compute:
    push rbp
    mov rbp, rsp
    sub rsp, 32          ; Allocate 4 local variables (aligned)

    ; int a = 10
    mov dword [rbp - 4], 10

    ; int b = 20
    mov dword [rbp - 8], 20

    ; int c = 30
    mov dword [rbp - 12], 30

    ; int sum = a + b + c
    mov eax, [rbp - 4]    ; EAX = a
    add eax, [rbp - 8]    ; EAX = a + b
    add eax, [rbp - 12]   ; EAX = a + b + c
    mov [rbp - 16], eax   ; sum = a + b + c

    ; return sum
    mov eax, [rbp - 16]

    leave
    ret

```

x

Part 10: Stack Pitfalls

Mistake 1: Unbalanced Stack

x

```

; BAD: More pushes than pops
my_func:
    push rax
    push rbx
    pop rax          ; RBX never popped!
    ret              ; Returns to WRONG address!

```

x

Mistake 2: Stack Overflow

C Equivalent:

```
void infinite_recursion() {
    infinite_recursion(); // Stack overflow!
}
```

Assembly:

```
; Infinite recursion - will crash
bad_func:
    call bad_func      ; Each call uses stack space
    ret               ; Never reached
```

Mistake 3: Misalignment

```
; BAD: Calling function with misaligned stack
main:
    push rax           ; RSP now misaligned
    call my_func       ; May crash if my_func uses SSE
    pop rax
```

Fix: Proper Alignment

```
; GOOD: Maintain alignment
main:
    push rax
    sub rsp, 8         ; Dummy push to re-align
    call my_func       ; RSP is 16-aligned
    add rsp, 8
    pop rax
```

Part 11: Stack Debugging

Viewing the Stack

```
; In GDB:
; x/16gx $rsp      - View 16 quadwords from stack
; info frame       - Show current stack frame
; backtrace        - Show call stack
```

Common Issues

Issue	Symptom	Solution
Stack overflow	Segmentation fault	Reduce recursion depth
Unbalanced push/pop	Return to wrong address	Count push/pop pairs
Misalignment	Bus error / SSE crash	Use <code>sub rsp, 8</code>
Stack corruption	Random crashes	Check buffer overflows

Practice Exercises

Exercise 1: Stack Calculator

Implement a stack-based calculator:

```
; push 10
; push 20
; add      ; Pop two values, push sum
; push 5
; mul      ; Pop two values, push product
; pop      ; Get result
```

Exercise 2: String Reverse

Reverse a string using the stack:

```
; Push each character onto stack
; Pop them back (reversed order)
```

Exercise 3: Fibonacci with Stack

Implement recursive Fibonacci preserving all registers correctly.

Quick Reference

Stack Operations

```
push rax      ; Push RAX onto stack (RSP -= 8)
pop  rbx      ; Pop from stack into RBX (RSP += 8)
```

Stack Frame Setup

```
; Prologue
push rbp
mov rbp, rsp
sub rsp, N      ; N must maintain 16-byte alignment

; Epilogue
mov rsp, rbp
pop rbp
ret
```

Calling Convention (Linux x64)

```
Arguments: RDI, RSI, RDX, RCX, R8, R9, then stack
Return: RAX
• Caller-saved: RAX, RCX, RDX, RSI, RDI, R8-R11
Callee-saved: RBX, RBP, R12-R15
```

Knowledge Check

Before moving to Topic 8, verify you understand:

- Stack grows downward (toward lower addresses)
- RSP always points to top of stack
- PUSH decrements RSP, POP increments RSP
- Stack frames store local variables and return addresses
- 16-byte alignment requirement for function calls
- Caller-saved vs callee-saved registers
- Function prologue and epilogue

Excellent! You now understand the stack and how functions use it!

Next: Topic 8: Functions & Procedures (Advanced function techniques)

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 25

Topic 8: Functions & Procedures

Master function creation, calling conventions, and advanced procedural programming techniques.

Overview

Functions (also called procedures or subroutines) are the building blocks of modular code. In this topic, you'll learn:

- CALL and RET instructions
- System V AMD64 calling convention (Linux)
- Parameter passing and return values
- Linking with external code
- Advanced function techniques

Part 1: CALL and RET Instructions

CALL Instruction

```
call label           ; Call function at label
call register        ; Call function at address in register
call [memory]        ; Call function at address in memory
```

What CALL Does:

```
call my_function

; Equivalent to:
push rip + 5          ; Push return address (address of next instruction)
jmp my_function       ; Jump to function
```

RET Instruction

```
ret                  ; Return from function
ret N                ; Return and pop N bytes from stack (rare in 64-bit)
```

What RET Does:

```
ret

; Equivalent to:
pop rip           ; Pop return address and jump to it
```

Simple Example

C Equivalent:

```
void greet() {
    // Function body
}

int main() {
    greet(); // Call function
    return 0;
}
```

Assembly:

```
section .text
    global _start

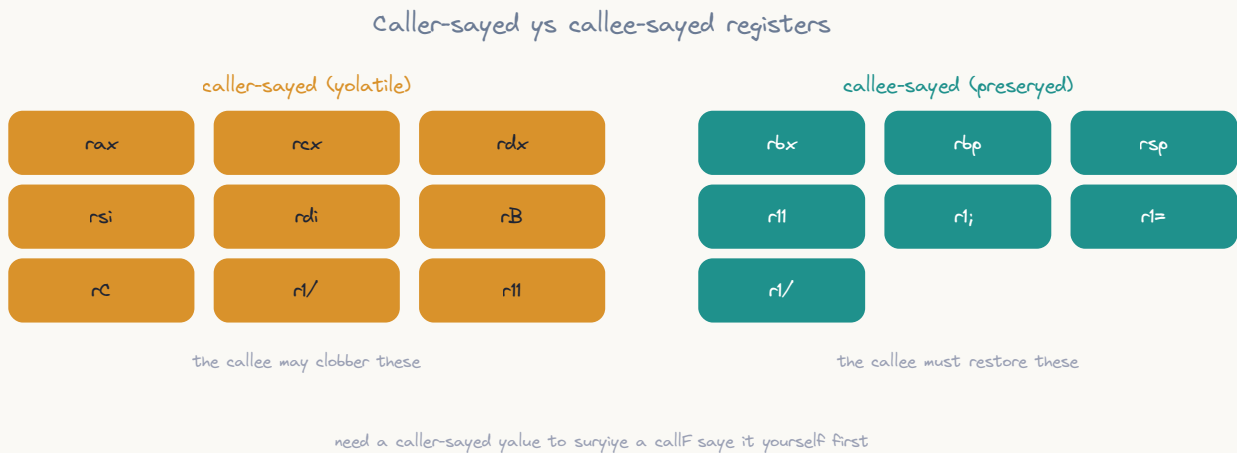
greet:
    ; Function body
    ; ... do something ...
    ret                ; Return to caller

_start:
    call greet         ; Call function
    ; Execution continues here after greet() returns

    mov rax, 60
    xor rdi, rdi
    syscall
```

Part 2: System V AMD64 Calling Convention

Register Usage



ASCII diagram

```

FUNCTION ARGUMENTS (in order)
1st: RDI    2nd: RSI    3rd: RDX
4th: RCX    5th: R8     6th: R9
7+: Stack (right-to-left)

```

```

RETURN VALUE
RAX (integer/pointer)
RDX:RAX (128-bit values)
XMM0 (floating point)

```

```

CALLER-SAVE (Volatile)
Function can modify without saving
RAX, RCX, RDX, RSI, RDI, R8-R11

```

```

CALLEE-SAVE (Non-volatile)
Function must preserve (save and restore)
RBX, RBP, R12-R15
RSP (always!)

```


Complete Function Template

C Equivalent:

```
int my_function(int arg1, int arg2, int arg3) {
    // Compiler handles:
    // - Saving callee-saved registers
    // - Setting up stack frame
    // - Accessing arguments
    // - Returning value in RAX

    return arg1 + arg2 + arg3;
}
```

Assembly:

```
my_function:
    ; === PROLOGUE ===
    push rbp                ; Save frame pointer
    mov rbp, rsp            ; Set up new frame
    push rbx                ; Save callee-saved registers
    push r12
    push r13
    sub rsp, 8              ; Align stack to 16 bytes

    ; === FUNCTION BODY ===
    ; Arguments arrive in: RDI, RSI, RDX, RCX, R8, R9
    ; Use them or save them if needed

    mov rax, rdi            ; arg1
    add rax, rsi            ; + arg2
    add rax, rdx            ; + arg3

    ; === EPILOGUE ===
    add rsp, 8              ; Deallocate locals
    pop r13                 ; Restore in reverse order
    pop r12
    pop rbx
    pop rbp
    ret                    ; Return (value in RAX)
```

Part 3: Parameter Passing

Example 1: Six or Fewer Parameters

C Equivalent:

```

int add_six(int a, int b, int c, int d, int e, int f) {
    return a + b + c + d + e + f;
}

int main() {
    int result = add_six(1, 2, 3, 4, 5, 6);
    return result; // 21
}

```

Assembly:

```

section .text
    global add_six

add_six:
    ; Arguments: RDI=a, RSI=b, RDX=c, RCX=d, R8=e, R9=f
    mov rax, rdi
    add rax, rsi
    add rax, rdx
    add rax, rcx
    add rax, r8
    add rax, r9
    ret

main:
    mov rdi, 1           ; arg1
    mov rsi, 2           ; arg2
    mov rdx, 3           ; arg3
    mov rcx, 4           ; arg4
    mov r8, 5            ; arg5
    mov r9, 6            ; arg6
    call add_six         ; RAX = 21

    mov rdi, rax
    mov rax, 60
    syscall

```

Example 2: More Than Six Parameters**C Equivalent:**

```
int add_many(int a, int b, int c, int d, int e, int f, int g, int h) {  
    return a + b + c + d + e + f + g + h;  
}  
  
int main() {  
    int result = add_many(1, 2, 3, 4, 5, 6, 7, 8);  
    return result; // 36  
}
```

Assembly:

```

section .text
    global add_many

add_many:
    ; RDI=a, RSI=b, RDX=c, RCX=d, R8=e, R9=f
    ; [rbp + 16]=g, [rbp + 24]=h (on stack)

    push rbp
    mov rbp, rsp

    mov rax, rdi          ; a
    add rax, rsi          ; + b
    add rax, rdx          ; + c
    add rax, rcx          ; + d
    add rax, r8           ; + e
    add rax, r9           ; + f
    add rax, [rbp + 16]   ; + g (7th arg on stack)
    add rax, [rbp + 24]   ; + h (8th arg on stack)

    pop rbp
    ret

main:
    ; First 6 args in registers
    mov rdi, 1
    mov rsi, 2
    mov rdx, 3
    mov rcx, 4
    mov r8, 5
    mov r9, 6

    ; 7th and 8th args on stack (push in reverse order!)
    push 8                ; 8th arg
    push 7                ; 7th arg

    call add_many          ; RAX = 36

    add rsp, 16            ; Clean up stack (2 args × 8 bytes)

    mov rdi, rax
    mov rax, 60
    syscall

```

Stack Layout for 7+ Arguments

Before CALL add_many:

8 (arg 8)	← RSP + 8
7 (arg 7)	← RSP

After CALL (inside function):

8 (arg 8)	← RBP + 24
7 (arg 7)	← RBP + 16
Return Addr	← RBP + 8
Saved RBP	← RBP, RSP

Part 4: Return Values

Single Integer Return

C Equivalent:

```
int get_value() {
    return 42;
}
```

Assembly:

```
get_value:
    mov rax, 42
    ret
```

Large Value Return (128-bit)

C Equivalent:

```
#include <stdint.h>

typedef struct {
    uint64_t low;
    uint64_t high;
} uint128_t;

uint128_t get_large_value() {
    uint128_t result = {0xFFFFFFFFFFFFFFFF, 0x123456789ABCDEF0};
    return result;
}
```

Assembly:

```
get_large_value:
    ; Return 128-bit value in RDX:RAX
    mov rax, 0xFFFFFFFFFFFFFFFF    ; Low 64 bits
    mov rdx, 0x123456789ABCDEF0    ; High 64 bits
    ret
```

Pointer Return**C Equivalent:**

```
char* get_message() {
    static char msg[] = "Hello!";
    return msg;
}
```

Assembly:

```
section .data
    message db "Hello!", 0

section .text
get_message:
    mov rax, message    ; Return pointer in RAX
    ret
```

Boolean Return**C Equivalent:**

```
#include <stdbool.h>

bool is_even(int n) {
    return (n & 1) == 0;
}
```

Assembly:

```
is_even:
    ; RDI = n
    mov rax, rdi
    and rax, 1           ; n & 1
    xor rax, 1           ; Return 1 if even, 0 if odd
    ret                 ; Or: setz al to get 0/1
```

Part 5: Local Variables

Allocating Local Variables

C Equivalent:

```
int compute(int x, int y) {
    int a = x + y;
    int b = x - y;
    int c = a * b;
    return c;
}
```

Assembly:

```

compute:
    ; RDI = x, RSI = y
    push rbp
    mov rbp, rsp
    sub rsp, 32                ; Allocate space for locals (aligned)

    ; int a = x + y
    mov eax, edi
    add eax, esi
    mov [rbp - 4], eax         ; a

    ; int b = x - y
    mov eax, edi
    sub eax, esi
    mov [rbp - 8], eax         ; b

    ; int c = a * b
    mov eax, [rbp - 4]
    imul eax, [rbp - 8]
    mov [rbp - 12], eax        ; c

    ; return c
    mov eax, [rbp - 12]

    leave
    ret

```

Optimized Version (Registers Only)

Assembly:

```

compute:
    ; RDI = x, RSI = y

    ; a = x + y
    lea eax, [rdi + rsi]      ; a in EAX

    ; b = x - y
    mov ecx, edi
    sub ecx, esi              ; b in ECX

    ; c = a * b
    imul eax, ecx             ; c in EAX

    ret                      ; Return c

```


Part 6: Recursive Functions

Example: Fibonacci

C Equivalent:

```
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int result = fibonacci(10); // 55
    return 0;
}
```

Assembly:

```

section .text
    global fibonacci

fibonacci:
    ; RDI = n
    push rbp
    mov rbp, rsp
    push rbx                ; Save RBX (will use it)
    push r12                ; Save R12 (will use it)

    ; Base case: if (n <= 1) return n
    cmp rdi, 1
    jg recursive_case
    mov rax, rdi             ; return n
    jmp fib_done

recursive_case:
    mov rbx, rdi             ; Save n

    ; Calculate fib(n-1)
    dec rdi                  ; n - 1
    call fibonacci           ; RAX = fib(n-1)
    mov r12, rax             ; Save result

    ; Calculate fib(n-2)
    lea rdi, [rbx - 2]       ; n - 2
    call fibonacci           ; RAX = fib(n-2)

    ; Return fib(n-1) + fib(n-2)
    add rax, r12

fib_done:
    pop r12
    pop rbx
    pop rbp
    ret

main:
    mov rdi, 10
    call fibonacci           ; RAX = 55

    mov rdi, rax
    mov rax, 60
    syscall

```

Tail Recursion Optimization

C Equivalent:

```
// Tail-recursive factorial
int factorial_helper(int n, int acc) {
    if (n <= 1) return acc;
    return factorial_helper(n - 1, n * acc); // Tail call
}

int factorial(int n) {
    return factorial_helper(n, 1);
}
```

Assembly:

```
factorial_helper:
    ; RDI = n, RSI = acc

tail_recursion_loop:
    cmp rdi, 1
    jle base_case

    ; acc = n * acc
    imul rsi, rdi

    ; n = n - 1
    dec rdi

    ; Tail call optimization: JMP instead of CALL
    jmp tail_recursion_loop    ; No stack growth!

base_case:
    mov rax, rsi                ; return acc
    ret

factorial:
    ; RDI = n
    mov rsi, 1                  ; acc = 1
    jmp factorial_helper        ; Tail call
```

Part 7: Function Pointers

Calling Through Function Pointer

C Equivalent:

```
typedef int (*operation_t)(int, int);

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

int apply(operation_t op, int a, int b) {
    return op(a, b);
}

int main() {
    int result1 = apply(add, 10, 5); // 15
    int result2 = apply(sub, 10, 5); // 5
    return 0;
}
```

Assembly:

```

section .text
    global add, sub, apply

add:
    ; RDI = a, RSI = b
    lea rax, [rdi + rsi]
    ret

sub:
    ; RDI = a, RSI = b
    mov rax, rdi
    sub rax, rsi
    ret

apply:
    ; RDI = function pointer, RSI = a, RDX = b
    push rbp
    mov rbp, rsp

    ; Move arguments
    mov rcx, rdi          ; Save function pointer
    mov rdi, rsi          ; arg1 = a
    mov rsi, rdx          ; arg2 = b

    ; Call through pointer
    call rcx              ; call [function pointer]

    pop rbp
    ret

main:
    ; apply(add, 10, 5)
    mov rdi, add          ; Function pointer
    mov rsi, 10           ; a
    mov rdx, 5            ; b
    call apply            ; RAX = 15

    ; apply(sub, 10, 5)
    mov rdi, sub
    mov rsi, 10
    mov rdx, 5
    call apply            ; RAX = 5

    xor rdi, rdi
    mov rax, 60
    syscall

```

Part 8: Variadic Functions

Variable Arguments (C-style)

C Equivalent:

```
#include <stdarg.h>

int sum_all(int count, ...) {
    va_list args;
    va_start(args, count);

    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += va_arg(args, int);
    }

    va_end(args);
    return sum;
}

int main() {
    int result = sum_all(4, 10, 20, 30, 40); // 100
    return 0;
}
```

Assembly:

```

section .text
    global sum_all

sum_all:
    ; RDI = count, RSI, RDX, RCX, R8, R9 = first 5 args
    ; More args on stack

    push rbp
    mov rbp, rsp

    xor rax, rax                ; sum = 0
    test rdi, rdi
    jz done                    ; if count == 0, return

    mov r10, rdi                ; counter

    ; First arg
    add rax, rsi
    dec r10
    jz done

    ; Second arg
    add rax, rdx
    dec r10
    jz done

    ; Third arg
    add rax, rcx
    dec r10
    jz done

    ; Fourth arg
    add rax, r8
    dec r10
    jz done

    ; Fifth arg
    add rax, r9
    dec r10
    jz done

    ; Remaining args on stack
    lea r11, [rbp + 16]        ; Start of stack args

sum_stack_args:
    add rax, [r11]
    add r11, 8
    dec r10
    jnz sum_stack_args

```

```

done:
    pop rbp
    ret

main:
    mov rdi, 4           ; count
    mov rsi, 10          ; arg1
    mov rdx, 20          ; arg2
    mov rcx, 30          ; arg3
    mov r8, 40           ; arg4
    call sum_all         ; RAX = 100

    mov rdi, rax
    mov rax, 60
    syscall

```

Part 9: Interfacing with C

Calling C Functions from Assembly

C code (mylib.c):

```

#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

void print_number(int n) {
    printf("Number: %d\n", n);
}

```

Assembly (main.asm):


```

section .text
    extern add, print_number
    global main

main:
    push rbp
    mov rbp, rsp

    ; Call add(10, 20)
    mov rdi, 10
    mov rsi, 20
    call add                ; RAX = 30

    ; Call print_number(result)
    mov rdi, rax
    call print_number        ; Prints "Number: 30"

    xor eax, eax            ; return 0
    pop rbp
    ret

```

Build:

```

gcc -c mylib.c -o mylib.o
nasm -f elf64 main.asm -o main.o
gcc mylib.o main.o -o program -no-pie
./program

```

Calling Assembly from C**Assembly (functions.asm):**

```

section .text
    global multiply, power

multiply:
    ; int multiply(int a, int b)
    ; RDI = a, RSI = b
    mov rax, rdi
    imul rax, rsi
    ret

power:
    ; int power(int base, int exp)
    ; RDI = base, RSI = exp
    mov rax, 1
    test rsi, rsi
    jz power_done

power_loop:
    imul rax, rdi
    dec rsi
    jnz power_loop

power_done:
    ret

```

C code (main.c):

```

#include <stdio.h>

// Declare assembly functions
extern int multiply(int a, int b);
extern int power(int base, int exp);

int main() {
    int result1 = multiply(6, 7);           // 42
    int result2 = power(2, 10);            // 1024

    printf("6 * 7 = %d\n", result1);
    printf("2^10 = %d\n", result2);

    return 0;
}

```

Build:

```
nasm -f elf64 functions.asm -o functions.o
gcc main.c functions.o -o program
./program
```

Part 10: Advanced Techniques

Inline Assembly in C

C with inline assembly:

```
#include <stdio.h>

int add_asm(int a, int b) {
    int result;
    __asm__ __volatile__ (
        "add %1, %2;"
        "mov %2, %0;"
        : "=r" (result)      // Output
        : "r" (a), "r" (b)   // Inputs
    );
    return result;
}

int main() {
    printf("Sum: %d\n", add_asm(10, 20));
    return 0;
}
```

Position-Independent Code (PIC)

Assembly:

```

section .data
    value dq 42

section .text
    global get_value

get_value:
    ; Non-PIC (absolute addressing)
    mov rax, [value]          ; Won't work in shared library
    ret

get_value_pic:
    ; PIC (RIP-relative addressing)
    mov rax, [rel value]      ; Works in shared library
    ret

```

Computed Goto (Jump Table)

C Equivalent:

```

int dispatch(int op, int a, int b) {
    switch(op) {
        case 0: return a + b;
        case 1: return a - b;
        case 2: return a * b;
        case 3: return a / b;
        default: return 0;
    }
}

```

Assembly:

```

section .data
    jump_table dq op_add, op_sub, op_mul, op_div

section .text
dispatch:
    ; RDI = op, RSI = a, RDX = b

    ; Bounds check
    cmp rdi, 3
    ja invalid_op

    ; Jump through table
    mov rax, [jump_table + rdi*8]
    jmp rax

op_add:
    lea rax, [rsi + rdx]
    ret

op_sub:
    mov rax, rsi
    sub rax, rdx
    ret

op_mul:
    mov rax, rsi
    imul rax, rdx
    ret

op_div:
    mov rax, rsi
    xor rdx, rdx
    div rdx                ; Note: should handle div by zero
    ret

invalid_op:
    xor rax, rax
    ret

```

Practice Exercises

Exercise 1: String Length Function

Implement `strlen` that counts characters until null terminator:

```
size_t strlen(const char *str);
```

Exercise 2: Array Sum Function

```
int array_sum(int *arr, int count);
```

Exercise 3: GCD (Greatest Common Divisor)

Implement Euclidean algorithm recursively:

```
int gcd(int a, int b);
```

Exercise 4: Callback Function

Implement a function that takes a callback:

```
void for_each(int *arr, int count, void (*callback)(int));
```

Quick Reference

Function Template

```
my_function:
    push rbp
    mov rbp, rsp
    push rbx                ; Save callee-saved regs
    sub rsp, N              ; Allocate locals (keep aligned!)

    ; Function body

    add rsp, N
    pop rbx
    pop rbp
    ret
```

Argument Registers (Linux x64)

```
1st: RDI    2nd: RSI    3rd: RDX
4th: RCX    5th: R8     6th: R9
7+: Stack (accessed via RBP)
```

Return Value

```
• RAX = integer/pointer return
  RDX:RAX = 128-bit return
```

Knowledge Check

Before moving to Topic 9, verify you understand:

- CALL pushes return address and jumps
- RET pops return address and returns
- First 6 args in RDI, RSI, RDX, RCX, R8, R9
- Return value in RAX
- Callee must save RBX, RBP, R12-R15
- Stack must be 16-byte aligned before CALL
- Function prologue/epilogue pattern

Excellent! You now understand functions and calling conventions!

Next: Topic 9: String Operations (Advanced string manipulation)

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 26

Topic 9: Calling Conventions

Master the rules for passing parameters and calling functions across different platforms and ABIs.

Overview

A **calling convention** is a standardized protocol that defines:

- How arguments are passed to functions
- How return values are returned
- Which registers must be preserved
- Who cleans up the stack (caller vs callee)
- Stack alignment requirements

Different conventions exist for different architectures, operating systems, and languages.

Part 1: Why Calling Conventions Matter

The Problem

```
; Without conventions, this is ambiguous:
call my_function

; Questions:
; - Where are the arguments?
; - Where does the result go?
; - Which registers can the function modify?
; - Who cleans the stack?
```

The Solution

Calling conventions provide standard answers so code from different sources can work together.

Benefits:

- Different compilers produce compatible code
- Assembly can call C functions (and vice versa)
- Libraries can be shared
- Predictable behavior for debugging

Part 2: System V AMD64 ABI (Linux/Unix x86-64)

This is the **standard** for **Linux**, **macOS**, **BSD** on 64-bit systems.

Parameter Passing

First 6 integer/pointer arguments go in registers:

System e Ab DA==how arguments are passed



args A → pushed on the stack, right to left



rdx can return a second word; for varargs, rdi holds the vector-register count

ASCII diagram

Arg 1	Arg 2	Arg 3	Arg 4	Arg 5	Arg 6
RDI	RSI	RDX	RCX	R8	R9

Additional arguments (7th, 8th, ...) go on the **stack** (right-to-left).

Floating-point arguments use **XMM0-XMM7** (8 registers).

Return Values

```
Integer/Pointer: RAX
Floating-point:  XMM0
128-bit:         RDX:RAX (high:low)
```

Register Usage

Register	Usage
RAX	Return value, temp, syscalls
RBX, RBP	Callee-saved (must preserve)
R12, R13, R14, R15	Callee-saved (must preserve)
RDI, RSI, RDX,	Argument passing, temp (caller-saved)
RCX, R8, R9	
R10, R11	Temp, caller-saved
RSP	Stack pointer (must preserve)
RIP	Instruction pointer (hardware)

Stack Alignment

RSP must be 16-byte aligned before `call` instruction.

- `call` pushes 8-byte return address → RSP becomes misaligned by 8
- Function prologue must realign if needed

Part 3: System V Examples

Example 1: Simple Function Call

C Equivalent:

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 10);
    return result;
}
```

Assembly:

```

section .text
    global main

; int add(int a, int b)
add:
    ; a is in EDI, b is in ESI (32-bit ints use lower 32 bits)
    lea eax, [rdi + rsi]      ; result = a + b
    ret                      ; return in EAX

main:
    ; Call add(5, 10)
    mov edi, 5                ; arg1 = 5
    mov esi, 10               ; arg2 = 10
    call add

    ; RAX now contains 15
    mov rdi, rax              ; exit code = result
    mov rax, 60               ; sys_exit
    syscall

```

Example 2: Six Arguments

C Equivalent:

```

long sum6(long a, long b, long c, long d, long e, long f) {
    return a + b + c + d + e + f;
}

int main() {
    long result = sum6(1, 2, 3, 4, 5, 6);
    return result;
}

```

Assembly:

```

section .text
    global main

; long sum6(long a, long b, long c, long d, long e, long f)
sum6:
    ; a=RDI, b=RSI, c=RDY, d=RCX, e=R8, f=R9
    mov rax, rdi
    add rax, rsi
    add rax, rdx
    add rax, rcx
    add rax, r8
    add rax, r9
    ret                ; return sum in RAX

main:
    mov rdi, 1          ; arg1
    mov rsi, 2          ; arg2
    mov rdx, 3          ; arg3
    mov rcx, 4          ; arg4
    mov r8, 5           ; arg5
    mov r9, 6           ; arg6
    call sum6

    ; RAX = 21
    mov rdi, rax
    mov rax, 60
    syscall

```

Example 3: More Than Six Arguments (Stack)

C Equivalent:

```

long sum8(long a, long b, long c, long d, long e, long f, long g, long h) {
    return a + b + c + d + e + f + g + h;
}

int main() {
    long result = sum8(1, 2, 3, 4, 5, 6, 7, 8);
    return result;
}

```

Assembly:

```

section .text
    global main

; long sum8(long a, long b, long c, long d, long e, long f, long g, long h)
sum8:
    ; Args 1-6 in registers: RDI, RSI, RDX, RCX, R8, R9
    ; Args 7-8 on stack: [rsp+8] = g, [rsp+16] = h

    mov rax, rdi          ; sum = a
    add rax, rsi          ; sum += b
    add rax, rdx          ; sum += c
    add rax, rcx          ; sum += d
    add rax, r8           ; sum += e
    add rax, r9           ; sum += f
    add rax, [rsp + 8]    ; sum += g (7th arg)
    add rax, [rsp + 16]   ; sum += h (8th arg)
    ret

main:
    ; Push args 8, 7 (right-to-left) for stack alignment
    push 8                ; arg8 (h)
    push 7                ; arg7 (g)

    ; Register args 1-6
    mov rdi, 1            ; arg1
    mov rsi, 2            ; arg2
    mov rdx, 3            ; arg3
    mov rcx, 4            ; arg4
    mov r8, 5             ; arg5
    mov r9, 6             ; arg6
    call sum8

    ; Clean up stack (2 pushes × 8 bytes = 16 bytes)
    add rsp, 16

    ; RAX = 36
    mov rdi, rax
    mov rax, 60
    syscall

```

Example 4: Callee-Saved Registers

C Equivalent:

```

int helper(int x) {
    // Uses registers that must be preserved
    int temp1 = x * 2;    // Uses RBX
    int temp2 = x * 3;    // Uses R12
    return temp1 + temp2;
}

int main() {
    int result = helper(10);
    return result;
}

```

Assembly:

```

section .text
    global main

; int helper(int x)
helper:
    ; Must save callee-saved registers before using them
    push rbx
    push r12

    ; x is in EDI
    mov ebx, edi
    shl ebx, 1                ; temp1 = x * 2 (in RBX)

    lea r12d, [rdi + rdi*2]    ; temp2 = x * 3 (in R12)

    lea eax, [rbx + r12]       ; result = temp1 + temp2

    ; Restore callee-saved registers
    pop r12
    pop rbx
    ret

main:
    mov edi, 10
    call helper

    ; RAX = 50
    mov rdi, rax
    mov rax, 60
    syscall

```

Part 4: Microsoft x64 Calling Convention (Windows)

Parameter Passing

First 4 arguments go in registers (different from System V!):

Arg 1	Arg 2	Arg 3	Arg 4
RCX	RDX	R8	R9

All additional arguments go on the stack.

Shadow Space: Caller must allocate 32 bytes (4×8) on stack for register args (even if not used).

Return Values

Integer/Pointer: RAX
Floating-point: XMM0

Register Usage

Register	Usage
RAX	Return value, temp
RBX, RBP, RDI, RSI, R12-R15	Callee-saved (must preserve)
RCX, RDX, R8, R9	Argument passing, temp
R10, R11	Temp, caller-saved
RSP	Stack pointer

Stack Alignment

RSP must be **16-byte aligned** before `call`.

Part 5: 32-bit Conventions (cdecl, stdcall, fastcall)

cdecl (C Declaration)

Default for C on 32-bit x86.

```

; int add(int a, int b)
; C: add(5, 10)

; Caller:
push 10                ; arg2 (right-to-left)
push 5                 ; arg1
call add
add esp, 8             ; Caller cleans stack (2 args × 4 bytes)

; Callee:
add:
    push ebp
    mov ebp, esp
    mov eax, [ebp + 8]  ; arg1
    add eax, [ebp + 12] ; arg2
    pop ebp
    ret                ; No stack cleanup

```

Characteristics:

- Arguments pushed **right-to-left**
- **Caller** cleans the stack
- Allows variadic functions (printf)

stdcall (Standard Call)

Used by Windows API.

```

; int add(int a, int b)

; Caller:
push 10
push 5
call add
; Note: NO stack cleanup by caller

; Callee:
add:
    push ebp
    mov ebp, esp
    mov eax, [ebp + 8]
    add eax, [ebp + 12]
    pop ebp
    ret 8                ; Callee cleans stack (return and pop 8 bytes)

```


Characteristics:

- Arguments pushed **right-to-left**
 - **Callee** cleans the stack
 - Cannot do variadic functions
 - Slightly more efficient (smaller code)
-

fastcall

Uses registers for first few arguments.

```
; int add(int a, int b)

; Caller:
mov ecx, 5           ; arg1 in ECX
mov edx, 10          ; arg2 in EDX
call add

; Callee:
add:
    lea eax, [ecx + edx] ; result = arg1 + arg2
    ret
```

Characteristics:

- First 2 args in **ECX, EDX**
 - Remaining args on stack
 - Faster (no memory access for first 2 args)
-

Part 6: Comparison Table

Convention	Architecture	Arg Order	Stack Cleanup	Registers
System V AMD64	x64 Linux/Mac	Reg then Stk	Caller	RDI RSI RDX RCX R8 R9
Microsoft x64	x64 Windows	Reg then Stk	Caller (shadow!)	RCX RDX R8 R9
cdecl	x86 (32-bit)	Right-to-left on stack	Caller	None
stdcall	x86 (32-bit) Windows	Right-to-left on stack	Callee	None
fastcall	x86 (32-bit)	ECX, EDX, then stack	Callee	ECX, EDX

Part 7: Practical Example - Mixed Convention

Calling C from Assembly (System V)

C Code (mylib.c):

```
#include <stdio.h>

int multiply(int a, int b) {
    return a * b;
}

void print_result(int result) {
    printf("Result: %d\n", result);
}
```

Assembly (main.asm):

```

section .text
    extern multiply
    extern print_result
    global main

main:
    ; Call multiply(6, 7)
    mov edi, 6           ; arg1 = 6
    mov esi, 7           ; arg2 = 7
    call multiply        ; result in EAX

    ; Call print_result(result)
    mov edi, eax         ; arg1 = result
    call print_result

    ; Exit
    xor eax, eax         ; return 0
    ret

```

Compile and Link:

```

nasm -f elf64 main.asm -o main.o
gcc -c mylib.c -o mylib.o
gcc main.o mylib.o -o program
./program
# Output: Result: 42

```

Part 8: Stack Frame Layout

System V AMD64 Stack Frame

Higher Addresses

Arg 8	← [rbp + 24]
Arg 7	← [rbp + 16]
Return Address	← [rbp + 8] (pushed by call)
Saved RBP	← RBP (current frame base)
Local Var 1	← [rbp - 8]
Local Var 2	← [rbp - 16]
...	
Saved Regs	(if needed)
Alignment	(padding to 16 bytes)
	← RSP (stack top)

Lower Addresses

x

Part 9: Common Mistakes

Mistake 1: Wrong Argument Order

```
; WRONG - System V uses RDI, RSI, not RCX, RDX
mov rcx, 5           ; Wrong register!
mov rdx, 10
call my_function

; CORRECT
mov rdi, 5           ; ✓ RDI for 1st arg (System V)
mov rsi, 10          ; ✓ RSI for 2nd arg
call my_function
```

Mistake 2: Not Preserving Callee-Saved Registers

```

; WRONG
my_function:
    mov rbx, rdi          ; Destroyed RBX without saving!
    ; ... use rbx ...
    ret

; CORRECT
my_function:
    push rbx              ; ✓ Save RBX
    mov rbx, rdi
    ; ... use rbx ...
    pop rbx               ; ✓ Restore RBX
    ret

```

Mistake 3: Forgetting Stack Cleanup

```

; cdecl convention - caller cleans
push 10
push 5
call add
; MISSING: add esp, 8

; CORRECT
push 10
push 5
call add
add esp, 8          ; ✓ Caller cleans stack

```

Mistake 4: Stack Misalignment

```

; WRONG - RSP not 16-byte aligned
main:
    push rax              ; RSP -= 8 (now misaligned!)
    call function         ; Function expects aligned stack

; CORRECT
main:
    push rax              ; RSP -= 8
    sub rsp, 8            ; Align to 16 bytes
    call function         ; ✓ Stack aligned
    add rsp, 8
    pop rax

```

Part 10: Debugging Tips

Check Calling Convention

```
# Check how GCC compiles a function
gcc -S -O2 test.c -o test.s
cat test.s

# Check what convention is used
objdump -d program | less
```

GDB Tips

```
gdb ./program

# View registers before call
(gdb) info registers

# Check stack
(gdb) x/10xg $rsp

# Step into function
(gdb) si

# View arguments (System V)
(gdb) print $rdi
(gdb) print $rsi
```

Practice Exercises

Exercise 1: Convert to Assembly

Write assembly for this C function using System V:

```
int average(int a, int b, int c) {
    return (a + b + c) / 3;
}
```

Solution

```

; int average(int a, int b, int c)
average:
    ; a=EDI, b=ESI, c=EDX
    lea eax, [rdi + rsi]    ; sum = a + b
    add eax, edx            ; sum += c

    mov ecx, 3
    cdq                    ; Sign-extend EAX to EDX:EAX
    idiv ecx                ; EAX = sum / 3

    ret

```

Exercise 2: Call from Assembly

Write assembly that calls this C function:

```
int max(int x, int y);
```

Solution

```

section .text
extern max
global main

main:
    mov edi, 42            ; arg1 = 42
    mov esi, 17            ; arg2 = 17
    call max

    ; RAX contains max(42, 17) = 42
    mov rdi, rax           ; exit code
    mov rax, 60
    syscall

```

Exercise 3: Fix the Bug

What's wrong with this code?

```

my_function:
    mov r12, rdi           ; Use R12 for something
    ; ... more code ...
    ret

```

Solution

R12 is callee-saved and must be preserved!

```
my_function:
    push r12           ; Save R12
    mov r12, rdi
    ; ... more code ...
    pop r12           ; Restore R12
    ret
```

Quick Reference Card

System V AMD64 (Linux/macOS)

```
Args (int):  RDI, RSI, RDX, RCX, R8, R9, then stack
Return:      RAX
Callee-save: RBX, RBP, R12-R15
Alignment:   16-byte before call
```

Microsoft x64 (Windows)

```
Args (int):  RCX, RDX, R8, R9, then stack
Return:      RAX
Shadow:      32 bytes
Callee-save: RBX, RBP, RDI, RSI, R12-R15
Alignment:   16-byte before call
```

32-bit cdecl

```
Args:        Stack (right-to-left)
Return:      EAX
Cleanup:     Caller
Callee-save: EBX, ESI, EDI, EBP
```

Knowledge Check

Before moving to Topic 10, verify you understand:

- What a calling convention is and why it matters
- System V AMD64 parameter passing (RDI, RSI, RDX, RCX, R8, R9)
- Microsoft x64 differences (RCX, RDX, R8, R9 and shadow space)
- Callee-saved vs caller-saved registers
- Stack alignment requirements (16-byte)

- How to call C functions from assembly
 - cdecl vs stdcall vs fastcall (32-bit)
-

Excellent! You now understand calling conventions!

Next: Topic 10: Procedures

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 27

Topic 10: Procedures

Master the `call` and `ret` instructions, and learn to write reusable procedures including recursive functions.

Overview

A **procedure** (also called a **subroutine** or **function**) is a reusable block of code that can be called from different places in your program.

Key Concepts:

- `call` instruction - jumps to procedure and saves return address
- `ret` instruction - returns to caller
- Recursion - procedures that call themselves
- Modularity - breaking programs into manageable pieces

Part 1: The CALL Instruction

What `call` Does

```
call procedure_name
```

Internally, `call` performs two operations:

```
; call is equivalent to:  
push return_address      ; Save where to return to  
jmp procedure_name       ; Jump to procedure
```

Step-by-step:

1. Push the address of the **next instruction** onto the stack
2. Jump to the procedure's address

Example: Simple Call

C Equivalent:

```

void greet() {
    // Print greeting (conceptual)
}

int main() {
    greet();
    // Execution continues here after greet() returns
    return 0;
}

```

Assembly:

```

section .text
    global _start

_start:
    call greet          ; Call the procedure
                        ; Execution continues here after greet returns

    mov rax, 60
    xor rdi, rdi
    syscall

greet:
    ; Do something here
    ret                ; Return to caller

```

What happens:

```

Before call:
RSP → [some_value]
RIP = address of "call greet"

During call:
RSP → [return_address]    ← RSP decremented by 8
RIP = address of greet

After ret:
RSP → [some_value]        ← RSP incremented by 8
RIP = return_address (instruction after call)

```

Part 2: The RET Instruction

What `ret` Does

```
ret
```

Internally, `ret` performs:

```
; ret is equivalent to:
pop rip                ; Pop return address into RIP
```

Step-by-step:

1. Pop the return address from the stack
 2. Jump to that address (continue execution after `call`)
-

RET Variants

```
; Standard return
ret

; Return and pop N bytes from stack (32-bit stdcall)
ret 16                ; Pop return address, then add 16 to ESP

; Far return (rarely used, different code segment)
retf
```

Part 3: Simple Procedures

Example 1: Procedure with No Arguments

C Equivalent:

```
void print_newline() {
    write(1, "\n", 1);
}

int main() {
    print_newline();
    print_newline();
    return 0;
}
```

Assembly:

```

section .data
    newline db 10

section .text
    global _start

print_newline:
    mov rax, 1          ; sys_write
    mov rdi, 1          ; stdout
    mov rsi, newline    ; "\n"
    mov rdx, 1          ; length
    syscall
    ret

_start:
    call print_newline
    call print_newline

    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: Procedure with Arguments (System V)**C Equivalent:**

```

int square(int x) {
    return x * x;
}

int main() {
    int result = square(5);
    return result;
}

```

Assembly:

```

section .text
    global _start

; int square(int x)
; Argument: x in EDI
; Return: result in EAX
square:
    mov eax, edi        ; EAX = x
    imul eax, edi       ; EAX = x * x
    ret

_start:
    mov edi, 5          ; arg: x = 5
    call square
    ; EAX now contains 25

    mov rdi, rax        ; exit code = result
    mov rax, 60
    syscall

```

Example 3: Procedure with Return Value

C Equivalent:

```

int add_three(int a, int b, int c) {
    return a + b + c;
}

int main() {
    int sum = add_three(10, 20, 30);
    return sum;
}

```

Assembly:

```

section .text
    global _start

; int add_three(int a, int b, int c)
; Args: a=EDI, b=ESI, c=EDX
; Return: EAX
add_three:
    lea eax, [rdi + rsi]    ; sum = a + b
    add eax, edx            ; sum += c
    ret

_start:
    mov edi, 10             ; a = 10
    mov esi, 20             ; b = 20
    mov edx, 30             ; c = 30
    call add_three
    ; EAX = 60

    mov rdi, rax
    mov rax, 60
    syscall

```

Part 4: Preserving Registers

Callee-Saved Registers

When a procedure uses callee-saved registers (RBX, RBP, R12-R15), it **must** preserve them.

C Equivalent:

```

int complex_calc(int x) {
    int temp1 = x * 2;    // Might use RBX
    int temp2 = x + 10;   // Might use R12
    return temp1 + temp2;
}

```

Assembly:

```

; int complex_calc(int x)
complex_calc:
    ; Save callee-saved registers we'll use
    push rbx
    push r12

    ; x is in EDI
    mov ebx, edi
    shl ebx, 1          ; temp1 = x * 2 (in RBX)

    lea r12d, [rdi + 10] ; temp2 = x + 10 (in R12)

    lea eax, [rbx + r12] ; result = temp1 + temp2

    ; Restore callee-saved registers (reverse order!)
    pop r12
    pop rbx
    ret

```

Why it matters:

```

main:
    mov rbx, 100          ; RBX has important value
    call complex_calc
    ; RBX still contains 100 (procedure preserved it)

```

Part 5: Local Variables

Allocating Space on Stack

C Equivalent:

```

int calculate() {
    int local1 = 10;
    int local2 = 20;
    int result = local1 + local2;
    return result;
}

```

Assembly:


```

; int calculate()
calculate:
    push rbp
    mov rbp, rsp
    sub rsp, 16          ; Allocate 16 bytes for locals

    ; local1 at [rbp-4], local2 at [rbp-8]
    mov dword [rbp - 4], 10    ; local1 = 10
    mov dword [rbp - 8], 20    ; local2 = 20

    ; result = local1 + local2
    mov eax, [rbp - 4]
    add eax, [rbp - 8]

    mov rsp, rbp          ; Deallocate locals
    pop rbp
    ret

```

Part 6: Recursion

What is Recursion?

A procedure that **calls itself** to solve a problem by breaking it into smaller instances.

Example 1: Factorial

C Equivalent:

```

int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    int result = factorial(5); // result = 120
    return result;
}

```

Assembly:

```

section .text
    global _start

; int factorial(int n)
; Arg: n in EDI
; Return: EAX
factorial:
    ; Base case: if n <= 1, return 1
    cmp edi, 1
    jg recursive_case

    mov eax, 1          ; return 1
    ret

recursive_case:
    ; Save n on stack (RDI is caller-saved, might be clobbered)
    push rdi

    ; Recursive call: factorial(n-1)
    dec edi             ; n-1
    call factorial      ; EAX = factorial(n-1)

    ; Restore n
    pop rdi

    ; return n * factorial(n-1)
    imul eax, edi       ; EAX = n * factorial(n-1)
    ret

_start:
    mov edi, 5          ; factorial(5)
    call factorial
    ; EAX = 120

    mov rdi, rax
    mov rax, 60
    syscall

```

How it works:

```

factorial(5)
├─ 5 * factorial(4)
│  ├─ 4 * factorial(3)
│  │  ├─ 3 * factorial(2)
│  │  │  ├─ 2 * factorial(1)
│  │  │  │  └─ returns 1
│  │  │  └─ returns 2 * 1 = 2
│  │  └─ returns 3 * 2 = 6
│  └─ returns 4 * 6 = 24
└─ returns 5 * 24 = 120

```

Example 2: Fibonacci

C Equivalent:

```

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int result = fibonacci(6); // result = 8
    return result;
}

```

Assembly:

```

section .text
    global _start

; int fibonacci(int n)
; Arg: n in EDI
; Return: EAX
fibonacci:
    ; Base case: if n <= 1, return n
    cmp edi, 1
    jg recursive_case

    mov eax, edi          ; return n (0 or 1)
    ret

recursive_case:
    push rbx              ; Save RBX (callee-saved)
    push rdi              ; Save n

    ; fib(n-1)
    dec edi
    call fibonacci
    mov ebx, eax          ; RBX = fib(n-1)

    ; fib(n-2)
    pop rdi               ; Restore n
    sub edi, 2
    call fibonacci        ; EAX = fib(n-2)

    ; return fib(n-1) + fib(n-2)
    add eax, ebx

    pop rbx               ; Restore RBX
    ret

_start:
    mov edi, 6            ; fibonacci(6)
    call fibonacci
    ; EAX = 8 (sequence: 0,1,1,2,3,5,8)

    mov rdi, rax
    mov rax, 60
    syscall

```

Example 3: String Length (Recursive)

C Equivalent:

```
int strlen_recursive(const char *str) {
    if (*str == '\0') {
        return 0;
    }
    return 1 + strlen_recursive(str + 1);
}

int main() {
    const char *text = "Hello";
    int len = strlen_recursive(text); // len = 5
    return len;
}
```

Assembly:

```

section .data
    text db "Hello", 0

section .text
    global _start

; int strlen_recursive(const char *str)
; Arg: str in RDI
; Return: EAX
strlen_recursive:
    ; Base case: if (*str == 0), return 0
    cmp byte [rdi], 0
    jne recursive_case

    xor eax, eax          ; return 0
    ret

recursive_case:
    push rdi              ; Save str

    inc rdi               ; str + 1
    call strlen_recursive ; EAX = strlen(str+1)

    pop rdi               ; Restore str (not actually needed)

    inc eax               ; return 1 + strlen(str+1)
    ret

_start:
    mov rdi, text         ; arg = "Hello"
    call strlen_recursive
    ; EAX = 5

    mov rdi, rax
    mov rax, 60
    syscall

```

Part 7: Nested Calls

Procedures can call other procedures:

C Equivalent:

```
int helper(int x) {  
    return x * 2;  
}  
  
int calculate(int a, int b) {  
    int temp = helper(a);  
    return temp + b;  
}  
  
int main() {  
    int result = calculate(5, 10); // result = 20  
    return result;  
}
```

Assembly:

```

section .text
    global _start

; int helper(int x)
helper:
    lea eax, [rdi + rdi]    ; return x * 2
    ret

; int calculate(int a, int b)
; Args: a=EDI, b=ESI
calculate:
    push rbx                ; Save RBX (we'll use it for b)
    mov ebx, esi            ; Save b in RBX

    ; temp = helper(a)
    ; EDI already has a
    call helper             ; EAX = helper(a)

    ; return temp + b
    add eax, ebx

    pop rbx
    ret

_start:
    mov edi, 5              ; a = 5
    mov esi, 10             ; b = 10
    call calculate
    ; EAX = 20

    mov rdi, rax
    mov rax, 60
    syscall

```

Part 8: Leaf vs Non-Leaf Procedures

Leaf Procedure

A procedure that **doesn't call other procedures** (and doesn't need a stack frame).

```

; Leaf procedure - simple and fast
add_numbers:
    lea eax, [rdi + rsi]
    ret                ; No prologue/epilogue needed

```


Non-Leaf Procedure

A procedure that **calls other procedures** (needs to save return address protection).

```
; Non-leaf procedure
outer:
    push rbp
    mov rbp, rsp

    call inner          ; Calls another procedure

    mov rsp, rbp
    pop rbp
    ret
```

Part 9: Tail Call Optimization

When the last thing a procedure does is call another procedure, you can optimize by replacing `call` + `ret` with just `jmp`.

Without optimization:

```
wrapper:
    ; Setup...
    call actual_work
    ret          ; Immediately return after call
```

With tail call optimization:

```
wrapper:
    ; Setup...
    jmp actual_work          ; Jump instead of call
                             ; actual_work will return directly to our caller
```

C Example:

```
int factorial_tail(int n, int accumulator) {
    if (n <= 1) {
        return accumulator;
    }
    return factorial_tail(n - 1, n * accumulator); // Tail call
}
```

Assembly (optimized):

```

; int factorial_tail(int n, int accumulator)
; Args: n=EDI, accumulator=ESI
factorial_tail:
    cmp edi, 1
    jle base_case

    ; Tail call: factorial_tail(n-1, n*accumulator)
    imul esi, edi        ; accumulator *= n
    dec edi              ; n--
    jmp factorial_tail    ; Tail call (no call/ret overhead!)

base_case:
    mov eax, esi
    ret

```

Part 10: Common Patterns

Pattern 1: Standard Prologue/Epilogue

```

procedure:
    ; Prologue
    push rbp
    mov rbp, rsp
    sub rsp, N            ; Allocate N bytes for locals

    ; Body
    ; ... procedure code ...

    ; Epilogue
    mov rsp, rbp          ; or: add rsp, N
    pop rbp
    ret

```

Pattern 2: Minimal (Leaf Procedure)

```

leaf_procedure:
    ; No prologue needed

    ; Body (doesn't call others, no locals)
    ; ... simple computation ...

    ; Return
    ret

```

Pattern 3: Save/Restore Callee-Saved

```
procedure:
    push rbx
    push r12
    push r13

    ; Use RBX, R12, R13

    pop r13
    pop r12
    pop rbx
    ret
```

Part 11: Debugging Procedures

GDB Commands

```
gdb ./program

# Set breakpoint at procedure
(gdb) break my_procedure

# Step into call
(gdb) step

# Step over call
(gdb) next

# View stack frames
(gdb) backtrace

# View current frame
(gdb) frame

# View arguments
(gdb) info args

# View locals
(gdb) info locals

# View return address
(gdb) x/xg $rsp
```

Practice Exercises

Exercise 1: Write a Procedure

Write a procedure that returns the maximum of two integers.

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Solution

```
; int max(int a, int b)
; Args: a=EDI, b=ESI
; Return: EAX
max:
    cmp edi, esi
    jg return_a

    mov eax, esi          ; return b
    ret

return_a:
    mov eax, edi          ; return a
    ret
```

Exercise 2: Recursive Sum

Write a recursive procedure that sums integers from 1 to n.

```
int sum_to_n(int n) {
    if (n <= 0) return 0;
    return n + sum_to_n(n - 1);
}
```

Solution

```

; int sum_to_n(int n)
; Arg: n in EDI
; Return: EAX
sum_to_n:
    test edi, edi
    jle base_case

    push rdi                ; Save n
    dec edi
    call sum_to_n           ; EAX = sum_to_n(n-1)
    pop rdi

    add eax, edi            ; return n + sum_to_n(n-1)
    ret

base_case:
    xor eax, eax
    ret

```

Exercise 3: Fix the Bug

What's wrong with this recursive procedure?

```

countdown:
    test edi, edi
    jz done

    dec edi
    call countdown          ; BUG!

done:
    ret

```

Solution

The procedure doesn't preserve RDI, which is caller-saved. After the recursive call, RDI might be modified.

```

countdown:
    test edi, edi
    jz done

    push rdi                ; ✓ Save RDI
    dec edi
    call countdown
    pop rdi                 ; ✓ Restore (though not needed here)

done:
    ret

```

Quick Reference

Call/Ret Mechanics

```

call proc    ; push rip; jmp proc
ret          ; pop rip
ret N        ; pop rip; add rsp, N (32-bit)

```

Standard Pattern

```

procedure:
    push rbp
    mov rbp, rsp
    sub rsp, 16    ; locals
    push rbx       ; save regs

    ; ... body ...

    pop rbx        ; restore
    mov rsp, rbp
    pop rbp
    ret

```

Recursion Checklist

- Base case (termination condition)
- Save arguments if needed
- Recursive call with modified argument
- Combine result properly
- Restore saved values

Knowledge Check

Before moving to Topic 11, verify you understand:

- What `call` and `ret` do internally
 - How the stack stores return addresses
 - Writing procedures with arguments and return values
 - Preserving callee-saved registers
 - Allocating local variables on the stack
 - Recursive procedures and base cases
 - Tail call optimization
 - Leaf vs non-leaf procedures
-

Excellent! You now understand procedures!

Next: Topic 11: Memory Addressing Modes

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 28

Topic 11: Memory Addressing Modes

Master all x86-64 addressing modes for flexible and efficient memory access.

Overview

Memory addressing modes determine how the CPU calculates the address of data in memory. Understanding these is crucial for:

- Efficient array access
- Data structure manipulation
- Performance optimization
- Reading compiler output

Part 1: The Square Brackets []

Square brackets mean: “Go to this address and get/put the value there”

Without brackets = the address itself

With brackets = the value at that address

```
mov rax, array      ; RAX = address of array
mov rax, [array]    ; RAX = value at array
```

Part 2: All Addressing Modes

Effective address = base (index & scale (disp)



→ one memory operand, computed by the CCU

mov eax, dbx (rsi & 14) loads int element rsi of an array at rbx(14)

1. Immediate (No Memory Access)

C Equivalent:

```
int x = 42;           // Constant value
```

Assembly:

```
mov rax, 42           ; RAX = 42 (no memory access)
mov eax, 0x1234        ; EAX = 0x1234
```

2. Register Direct

C Equivalent:

```
int a = b;           // Register to register
```

Assembly:

```
mov rax, rbx          ; RAX = RBX (no memory access)
mov eax, edx          ; EAX = EDX
```

3. Memory Direct (Absolute Address)

C Equivalent:

```
int x = *(int*)0x1000; // Read from absolute address
```

Assembly:

```
mov rax, [0x1000]      ; RAX = value at address 0x1000
mov [0x2000], rbx      ; Store RBX at address 0x2000
```

Usage: Rarely used (except for memory-mapped I/O).

4. Register Indirect

C Equivalent:

```
int *ptr = &value;
int x = *ptr;           // Dereference pointer
```

Assembly:

```
mov rbx, array          ; RBX = address
mov rax, [rbx]           ; RAX = value at address in RBX
mov [rbx], rcx           ; Store RCX at address in RBX
```

5. Register + Displacement

C Equivalent:

```
struct Point {
    int x;                // offset 0
    int y;                // offset 4
};
struct Point *p;
int y_value = p->y;      // Access field at offset
```

Assembly:

```
; RBX points to struct
mov eax, [rbx + 0]       ; Access x (offset 0)
mov eax, [rbx + 4]       ; Access y (offset 4)
mov eax, [rbx + 8]       ; Access next field

; Array access
mov rax, [array + 16]    ; Access element 16 bytes into array
```

6. Base + Index

C Equivalent:

```
int array[10];
int index = 5;
int value = array[index]; // Dynamic index
```

Assembly:

```

mov rbx, array      ; base = array address
mov rcx, 5           ; index = 5
mov eax, [rbx + rcx] ; value = array[index]

```

7. Base + Index*Scale (SIB - Scale-Index-Base)

Most powerful addressing mode!

Format: `[base + index*scale + displacement]`

Where:

- `base` = any register (RBX, RSI, etc.)
- `index` = any register except RSP
- `scale` = 1, 2, 4, or 8
- `displacement` = constant offset (-2^{31} to $2^{31}-1$)

C Equivalent:

```

int array[100];
int index = 10;
int value = array[index]; // array + index * sizeof(int)

```

Assembly:

```

; Access int array (4 bytes per element)
mov rbx, array      ; base
mov rcx, 10          ; index
mov eax, [rbx + rcx*4] ; value = array[index]
                     ; Address = base + index*4

; Access long array (8 bytes per element)
mov rax, [rbx + rcx*8] ; Address = base + index*8

; With displacement
mov eax, [rbx + rcx*4 + 16] ; Skip first 4 elements (16 bytes)

```

Part 3: Scale Factor Examples

```

; Scale = 1 (byte arrays, char)
mov al, [rsi + rdi*1]      ; char array[index]

; Scale = 2 (word arrays, short)
mov ax, [rsi + rdi*2]      ; short array[index]

; Scale = 4 (dword arrays, int, float)
mov eax, [rsi + rdi*4]     ; int array[index]

; Scale = 8 (qword arrays, long, double, pointers)
mov rax, [rsi + rdi*8]     ; long array[index]

```

Part 4: RIP-Relative Addressing (x86-64)

Position-independent code - address relative to current instruction.

C Equivalent:

```

static int global_var = 42;
int x = global_var;          // Compiler uses RIP-relative

```

Assembly:

```

section .data
    value dq 100

section .text
    ; RIP-relative (x86-64 only)
    mov rax, [rel value]      ; Address = RIP + offset to value

    ; Explicit form (NASM)
    mov rax, [value wrt ..gotpc]

    * ; Default in 64-bit
    . mov rax, [value]         ; NASM uses RIP-relative by default

```

Why use it:

- Code can be loaded at any address (PIE/PIC)
- More compact encoding
- Required for shared libraries

Part 5: Practical Examples

Example 1: Array Iteration

C Equivalent:

```
int sum_array(int *array, int count) {
    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += array[i];
    }
    return sum;
}
```

Assembly (Method 1: Index):

```
; int sum_array(int *array, int count)
; Args: array=RDI, count=ESI
sum_array:
    xor eax, eax          ; sum = 0
    xor ecx, ecx          ; i = 0

loop:
    cmp ecx, esi
    jge done

    add eax, [rdi + rcx*4] ; sum += array[i] (SIB addressing!)
    inc ecx
    jmp loop

done:
    ret
```

Assembly (Method 2: Pointer):

```

sum_array:
    xor eax, eax          ; sum = 0
    lea rdx, [rdi + rsi*4] ; end = array + count*4

loop:
    cmp rdi, rdx
    jge done

    add eax, [rdi]         ; sum += *ptr (register indirect!)
    add rdi, 4             ; ptr++
    jmp loop

done:
    ret

```

Example 2: Struct Access

C Equivalent:

```

struct Person {
    int age;           // offset 0
    int height;        // offset 4
    long id;           // offset 8
};

int get_height(struct Person *p) {
    return p->height;
}

```

Assembly:

```

; int get_height(struct Person *p)
; Arg: p in RDI
get_height:
    mov eax, [rdi + 4]    ; return p->height (displacement!)
    ret

```

Example 3: 2D Array Access

C Equivalent:

```
int matrix[ROWS][COLS];
int get_element(int row, int col) {
    return matrix[row][col];
}
```

Assembly:

```
; Assume COLS = 10, each element is 4 bytes
; Address = base + (row * COLS + col) * 4

section .data
    matrix times 100 dd 0 ; 10x10 matrix

section .text
; int get_element(int row, int col)
; Args: row=EDI, col=ESI
get_element:
    ; Calculate: row * COLS (10)
    imul edi, 10 ; row * COLS

    ; Calculate: (row * COLS + col) * 4
    add edi, esi ; row * COLS + col

    mov rbx, matrix
    mov eax, [rbx + rdi*4] ; matrix[row][col] (SIB!)
    ret
```

Example 4: String Operations**C Equivalent:**

```
char *strcpy(char *dest, const char *src) {
    char *orig_dest = dest;
    while (*src != '\0') {
        *dest++ = *src++;
    }
    *dest = '\0';
    return orig_dest;
}
```

Assembly:

```

; char *strcpy(char *dest, const char *src)
; Args: dest=RDI, src=RSI
strcpy:
    mov rax, rdi          ; Save orig_dest

loop:
    mov cl, [rsi]         ; Load byte from src (register indirect!)
    mov [rdi], cl         ; Store to dest

    test cl, cl           ; Check if null
    jz done

    inc rsi               ; src++
    inc rdi               ; dest++
    jmp loop

done:
    ret

```

Part 6: Address Calculation Examples

Calculate Effective Address (LEA)

LEA doesn't access memory - it just calculates the address!

C Equivalent:

```

int *ptr = &array[index]; // Get address, not value
int offset = index * 4 + 10; // Calculate without memory access

```

Assembly:

```

; Get address (doesn't dereference)
lea rax, [rbx + rcx*4]      ; RAX = rbx + rcx*4 (no memory access!)

; Use for arithmetic
lea rax, [rdi + rdi*4]      ; RAX = rdi * 5 (1 + 4)
lea rax, [rdi + rdi*8]      ; RAX = rdi * 9 (1 + 8)
lea rax, [rdi + rsi + 10]   ; RAX = rdi + rsi + 10

```

Part 7: Size Specifiers

When size is ambiguous, use size specifiers:


```

; Ambiguous
mov [rbx], 42           ; ERROR: What size?

; Clear
mov byte [rbx], 42      ; ✓ 1 byte
mov word [rbx], 42      ; ✓ 2 bytes
mov dword [rbx], 42     ; ✓ 4 bytes
mov qword [rbx], 42     ; ✓ 8 bytes

; Not ambiguous (size inferred from register)
mov [rbx], al           ; ✓ 1 byte (AL is 8-bit)
mov [rbx], ax           ; ✓ 2 bytes (AX is 16-bit)
mov [rbx], eax          ; ✓ 4 bytes (EAX is 32-bit)
mov [rbx], rax          ; ✓ 8 bytes (RAX is 64-bit)

```

Part 8: Common Patterns

Pattern 1: Array Element Access

```

; array[index]
mov eax, [array + rcx*4] ; For int array (4 bytes)
mov rax, [array + rcx*8] ; For long/pointer array (8 bytes)

```

Pattern 2: Struct Field Access

```

; struct->field
mov eax, [rbx + OFFSET] ; Access field at offset

```

Pattern 3: Pointer Arithmetic

```

; ptr++
add rdi, 4 ; Advance by 4 bytes (int*)
add rdi, 8 ; Advance by 8 bytes (long*)

```

Pattern 4: Array of Structs

```

; array[index].field
; Address = base + index*sizeof(struct) + field_offset
mov eax, [rbx + rcx*16 + 4] ; Assuming 16-byte structs, field at offset 4

```

Part 9: Performance Considerations

Cache-Friendly Access

```
; ✓ GOOD: Sequential access (cache-friendly)
mov rcx, 0
loop:
    mov eax, [array + rcx*4]
    ; Process eax
    inc rcx
    cmp rcx, 100
    jl loop

; BAD: Random access (cache-unfriendly)
mov rcx, 0
loop:
    ; Random index calculation
    mov eax, [array + random_index*4]
```

Alignment

```
; Aligned access (faster)
section .data
    align 16
    aligned_array dq 1, 2, 3, 4

; Misaligned access (slower, may cause crashes on some CPUs)
mov rax, [rbx + 1] ; Misaligned 64-bit read
```

Practice Exercises

Exercise 1: Array Sum

Calculate the sum of a double array (8-byte elements).

Solution

```

; double sum_doubles(double *array, int count)
; Args: array=RDI, count=ESI
sum_doubles:
    xorpd xmm0, xmm0      ; sum = 0.0
    xor ecx, ecx          ; i = 0

loop:
    cmp ecx, esi
    jge done

    addsd xmm0, [rdi + rcx*8] ; sum += array[i] (scale=8!)
    inc ecx
    jmp loop

done:
    ; Result in XMM0
    ret

```

Exercise 2: Reverse Array

Reverse an array in place.

Solution

```

; void reverse(int *array, int count)
; Args: array=RDI, count=ESI
reverse:
    xor ecx, ecx          ; left = 0
    lea edx, [esi - 1]    ; right = count - 1

loop:
    cmp ecx, edx
    jge done

    ; Swap array[left] and array[right]
    mov eax, [rdi + rcx*4]
    mov ebx, [rdi + rdx*4]
    mov [rdi + rcx*4], ebx
    mov [rdi + rdx*4], eax

    inc ecx              ; left++
    dec edx              ; right--
    jmp loop

done:
    ret

```

Quick Reference

Addressing Modes

```
[register]           ; Register indirect
[register + disp]    ; Register + displacement
[base + index]       ; Base + index
[base + index*scale] ; SIB
[base + index*scale + disp] ; Full SIB
[rel label]          ; RIP-relative
```

Scale Values

```
1 → byte/char
2 → word/short
4 → dword/int/float
8 → qword/long/double/pointer
```

Knowledge Check

- All 7 addressing modes and when to use each
- SIB addressing (Scale-Index-Base)
- Scale factors for different data types
- RIP-relative addressing and why it matters
- LEA for address calculation
- Size specifiers (byte/word/dword/qword)
- Performance implications of different modes

Excellent! You've mastered memory addressing!

Next: Topic 12: Arrays & Strings

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 29

Topic 12: Arrays & Strings

Master array manipulation and string operations in assembly, including powerful string instructions.

Overview

Arrays and strings are fundamental data structures. Assembly provides:

- Direct memory manipulation for arrays
- Specialized string instructions (REP prefix)
- Highly optimized bulk operations
- Fine-grained control over data processing

Part 1: Arrays in Assembly

Declaring Arrays

C Equivalent:

```
int numbers[] = {10, 20, 30, 40, 50};
char name[] = "Hello";
int buffer[100]; // Uninitialized
```

Assembly:

```
section .data
    ; Initialized arrays
    numbers dd 10, 20, 30, 40, 50      ; 5 ints (20 bytes)
    floats  dd 1.0, 2.5, 3.14          ; 3 floats
    name     db "Hello", 0              ; String (6 bytes including null)
    bytes    db 0x41, 0x42, 0x43        ; 3 bytes

section .bss
    ; Uninitialized arrays
    buffer   resb 100                   ; 100 bytes
    int_arr  resd 50                     ; 50 ints (200 bytes)
    long_arr resq 20                     ; 20 longs (160 bytes)
```

Accessing Array Elements

C Equivalent:

```
int get_element(int *array, int index) {
    return array[index];
}
```

Assembly:

```
; int get_element(int *array, int index)
; Args: array=RDI, index=ESI
get_element:
    mov eax, [rdi + rsi*4]    ; array[index], scale=4 for int
    ret
```

Part 2: Common Array Operations

Example 1: Sum Array

C Equivalent:

```
int sum(int *arr, int size) {
    int total = 0;
    for (int i = 0; i < size; i++) {
        total += arr[i];
    }
    return total;
}
```

Assembly:

```
sum:
    xor eax, eax            ; total = 0
    xor ecx, ecx            ; i = 0
loop:
    cmp ecx, esi
    jge done
    add eax, [rdi + rcx*4]
    inc ecx
    jmp loop
done:
    ret
```

Example 2: Find Maximum

C Equivalent:

```
int find_max(int *arr, int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
```

Assembly:

```
find_max:
    mov eax, [rdi]           ; max = arr[0]
    mov ecx, 1               ; i = 1
loop:
    cmp ecx, esi
    jge done
    cmp eax, [rdi + rcx*4]
    jge skip
    mov eax, [rdi + rcx*4]    ; max = arr[i]
skip:
    inc ecx
    jmp loop
done:
    ret
```

Part 3: String Basics

Strings in Assembly

Strings are **byte arrays** terminated by null (0).

```
section .data
    msg db "Hello", 0        ; 6 bytes: 'H', 'e', 'l', 'l', 'o', 0
    msg_len equ $ - msg      ; Length = 6
```

Manual String Length

C Equivalent:

```

size_t strlen(const char *str) {
    size_t len = 0;
    while (str[len] != '\0') {
        len++;
    }
    return len;
}

```

Assembly:

```

my_strlen:
    xor eax, eax          ; len = 0
loop:
    cmp byte [rdi + rax], 0 ; Check for null
    je done
    inc rax
    jmp loop
done:
    ret

```

Part 4: String Instructions

x86 provides specialized string instructions that operate on memory blocks.

Direction Flag (DF)

Controls direction of string operations:

- **DF = 0**: Forward (increment addresses) - use **cld**
- **DF = 1**: Backward (decrement addresses) - use **std**

```

cld          ; Clear DF (forward direction)
std          ; Set DF (backward direction)

```


String Instructions Overview

Instr	Operation
MOVSB	Move byte: [RDI] = [RSI], RSI++, RDI++
MOVSW	Move word (2 bytes)
MOVSD	Move dword (4 bytes)
MOVSQ	Move qword (8 bytes)
LODSB	Load byte: AL = [RSI], RSI++
LODSW	Load word: AX = [RSI], RSI += 2
LODSD	Load dword: EAX = [RSI], RSI += 4
LODSQ	Load qword: RAX = [RSI], RSI += 8
STOSB	Store byte: [RDI] = AL, RDI++
STOSW	Store word: [RDI] = AX, RDI += 2
STOSD	Store dword: [RDI] = EAX, RDI += 4
STOSQ	Store qword: [RDI] = RAX, RDI += 8
SCASB	Scan byte: Compare AL with [RDI], RDI++
SCASW	Scan word
SCASD	Scan dword
SCASQ	Scan qword
CMPSB	Compare byte: [RSI] with [RDI], RSI++, RDI++
CMPSW	Compare word
CMPSD	Compare dword
CMPSQ	Compare qword

REP Prefix - Repeat

Repeats string instruction RCX times:

```
rep movsb          ; Repeat MOVSB RCX times
rep stosb          ; Repeat STOSB RCX times
```

Conditional repeats:

```
repe cmpsb         ; Repeat while equal (and RCX > 0)
repne scasb        ; Repeat while not equal (and RCX > 0)
```

Part 5: String Operations Examples

Example 1: Memory Copy (memcpy)

C Equivalent:

```
void *memcpy(void *dest, const void *src, size_t n) {
    char *d = dest;
    const char *s = src;
    while (n--) {
        *d++ = *s++;
    }
    return dest;
}
```

Assembly (Manual):

```
my_memcpy:
    mov rax, rdi                ; Save dest
    test rdx, rdx
    jz done
loop:
    mov cl, [rsi]
    mov [rdi], cl
    inc rsi
    inc rdi
    dec rdx
    jnz loop
done:
    ret
```

Assembly (Optimized with REP):

```
my_memcpy:
    mov rax, rdi                ; Save dest
    mov rcx, rdx                ; Count
    cld                        ; Forward direction
    rep movsb                   ; Copy RCX bytes from RSI to RDI
    ret
```

Example 2: Memory Set (memset)

C Equivalent:

```
void *memset(void *s, int c, size_t n) {
    unsigned char *p = s;
    while (n--) {
        *p++ = (unsigned char)c;
    }
    return s;
}
```

Assembly:

```
my_memset:
    mov rax, rdi          ; Save dest
    mov al, sil           ; Value to set (2nd arg, low byte)
    mov rcx, rdx          ; Count
    cld
    rep stosb             ; Set RCX bytes at RDI to AL
    ret
```

Example 3: String Copy (strcpy)**C Equivalent:**

```
char *strcpy(char *dest, const char *src) {
    char *orig = dest;
    while ((*dest++ = *src++) != '\0');
    return orig;
}
```

Assembly:

```
my_strcpy:
    mov rax, rdi          ; Save dest
loop:
    lodsb                 ; AL = [RSI++]
    stosb                 ; [RDI++] = AL
    test al, al
    jnz loop
    ret
```

Example 4: String Length (strlen)**C Equivalent:**

```
size_t strlen(const char *s) {
    size_t len = 0;
    while (s[len]) len++;
    return len;
}
```

Assembly (with SCASB):

```
my_strlen:
    mov rax, rdi                ; Save start
    xor al, al                  ; Search for null (0)
    mov rcx, -1                 ; Max count (search forever)
    cld
    repne scasb                 ; Scan until AL found or RCX=0
    ; RDI now points one past null
    mov rax, rdi
    sub rax, [rsp+8]            ; Original rdi (from caller)
    dec rax                     ; Don't count null
    ret

; Simpler version:
my_strlen_simple:
    xor rax, rax
loop:
    cmp byte [rdi + rax], 0
    je done
    inc rax
    jmp loop
done:
    ret
```

Example 5: String Compare (strcmp)

C Equivalent:

```
int strcmp(const char *s1, const char *s2) {
    while (*s1 && (*s1 == *s2)) {
        s1++;
        s2++;
    }
    return *(unsigned char*)s1 - *(unsigned char*)s2;
}
```

Assembly:

```

my_strcmp:
loop:
    mov al, [rdi]
    mov cl, [rsi]
    cmp al, cl
    jne not_equal
    test al, al           ; Check if null
    jz equal
    inc rdi
    inc rsi
    jmp loop

equal:
    xor eax, eax         ; Return 0
    ret

not_equal:
    movzx eax, al
    movzx ecx, cl
    sub eax, ecx
    ret

```

Part 6: Multi-Dimensional Arrays

C Equivalent:

```

int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

int get(int row, int col) {
    return matrix[row][col];
}

```

Assembly:

```

section .data
    matrix dd 1,2,3,4, 5,6,7,8, 9,10,11,12
    COLS equ 4

section .text
; int get(int row, int col)
; Args: row=EDI, col=ESI
get:
    imul edi, COLS           ; row * COLS
    add edi, esi             ; row * COLS + col
    mov rbx, matrix
    mov eax, [rbx + rdi*4]    ; matrix[row][col]
    ret

```

Practice Exercises

Exercise 1: Reverse String

```

void reverse(char *str) {
    int len = strlen(str);
    for (int i = 0; i < len/2; i++) {
        char temp = str[i];
        str[i] = str[len-1-i];
        str[len-1-i] = temp;
    }
}

```

Solution

```

reverse:
    push rbx
    mov rbx, rdi                ; str
    call my_strlen              ; RAX = len
    xor rcx, rcx                ; i = 0
    lea rdx, [rax - 1]          ; j = len - 1
loop:
    cmp rcx, rdx
    jge done
    ; Swap str[i] and str[j]
    mov al, [rbx + rcx]
    mov ah, [rbx + rdx]
    mov [rbx + rcx], ah
    mov [rbx + rdx], al
    inc rcx
    dec rdx
    jmp loop
done:
    pop rbx
    ret

```

Quick Reference

```

; String Instructions
cld                ; Clear direction flag (forward)
std                ; Set direction flag (backward)
rep movsb          ; Copy RCX bytes: RSI → RDI
rep stosb          ; Fill RCX bytes: AL → RDI
repne scasb        ; Find AL in [RDI], max RCX bytes

; Array Access
mov eax, [array + rcx*4] ; array[index] (int)
mov rax, [array + rcx*8] ; array[index] (long/ptr)
lea rax, [base + index*8 + off] ; Calculate address

```

Great! You've mastered arrays and strings!

Next: Topic 13: Multiplication & Division

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 30

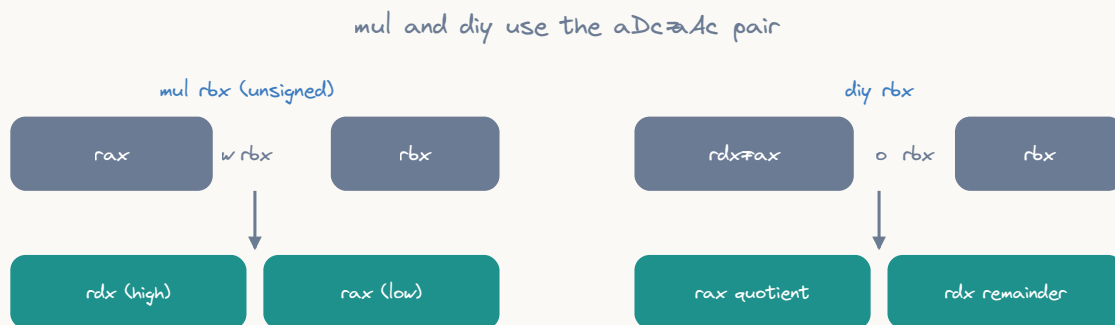
Topic 13: Multiplication and Division

Overview

Multiplication and division are more complex operations in assembly than addition and subtraction. They involve multiple registers, can produce results larger than the input operands, and have special handling for signed vs unsigned operations.

```
// C equivalent concepts we'll explore:
int product = a * b;
int quotient = a / b;
int remainder = a % b;
unsigned long long wide = (unsigned long long)a * b; // 64-bit * 64-bit = 128-bit
```

Multiplication Instructions



before div set `rdx=xor` it for unsigned, or `cqo` to sign-extend `rax`

MUL - Unsigned Multiplication

```
; C equivalent:
; unsigned long long result = rax * operand;
; // Upper 64 bits go to RDX, lower 64 bits go to RAX

mul operand    ; RDX:RAX = RAX * operand (unsigned)
```

Key Points:

- **Operand sizes:** byte (8-bit), word (16-bit), dword (32-bit), qword (64-bit)
- **Implicit operands:** Always uses `AL/AX/EAX/RAX` as one input

- **Result:** Double-width result (upper half in `DX/EDX/RDX`, lower in `AX/EAX/RAX`)

Size Variants:

Instruction	Operation	Result Location
<code>mul r/m8</code>	<code>AX = AL * r/m8</code>	AH:AL (upper:lower)
<code>mul r/m16</code>	<code>DX:AX = AX * r/m16</code>	DX:AX
<code>mul r/m32</code>	<code>EDX:EAX = EAX * r/m32</code>	EDX:EAX
<code>mul r/m64</code>	<code>RDX:RAX = RAX * r/m64</code>	RDX:RAX

Flags Affected:

- **CF/OF:** Set if upper half of result is non-zero (overflow occurred)
- **SF, ZF, AF, PF:** Undefined

Example: 64-bit × 64-bit = 128-bit

```
section .data
    a dq 0x123456789ABCDEF0
    b dq 0x0FEDCBA987654321
    result_low dq 0
    result_high dq 0

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned __int128 result = (unsigned __int128)a * (unsigned __int128)b;
    ; unsigned long long result_high = result >> 64;
    ; unsigned long long result_low = result & 0xFFFFFFFFFFFFFFFF;

    mov rax, [a]          ; RAX = first operand
    mul qword [b]          ; RDX:RAX = RAX * [b]
    mov [result_low], rax  ; Store lower 64 bits
    mov [result_high], rdx ; Store upper 64 bits

    ; Result = 0x0121FA00AD77D9B0:2B00A5AAF1C5DF10

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

IMUL - Signed Multiplication

IMUL has three forms with different capabilities:

Form 1: Single Operand (like MUL)

```
; C equivalent:
; long long result = (long long)rax * (long long)operand;

imul operand    ; RDX:RAX = RAX * operand (signed)
```

Same behavior as **MUL** but treats operands as signed:

Instruction	Operation
<code>imul r/m8</code>	<code>AX = AL * r/m8</code>
<code>imul r/m16</code>	<code>DX:AX = AX * r/m16</code>
<code>imul r/m32</code>	<code>EDX:EAX = EAX * r/m32</code>
<code>imul r/m64</code>	<code>RDX:RAX = RAX * r/m64</code>

Form 2: Two Operands

```
; C equivalent:
; dest = dest * src;

imul dest, src    ; dest = dest * src (signed, truncated to dest size)
```

- Result only in destination register (no double-width)
- **CF/OF** set if result doesn't fit in destination
- Most commonly used form for normal arithmetic

Example:

```

section .data
    x dd -15
    y dd 7

section .text
    ; C equivalent:
    ; int result = x * y;  // -15 * 7 = -105

    mov eax, [x]          ; EAX = -15
    imul eax, [y]          ; EAX = -15 * 7 = -105
    ; Result: EAX = -105 (0xFFFFFFFF97)

```

Form 3: Three Operands

```

; C equivalent:
; dest = src1 * immediate;

imul dest, src, immediate ; dest = src * immediate

```

Example:

```

; C equivalent:
; int result = x * 10;

mov eax, [x]
imul ebx, eax, 10 ; EBX = EAX * 10

```

Complete Multiplication Example: Computing Area

```
section .data
    width dd 1920
    height dd 1080
    area dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int area = width * height; // 1920 * 1080 = 2,073,600

    mov eax, [width]      ; EAX = 1920
    imul eax, [height]    ; EAX = 1920 * 1080 = 2,073,600
    mov [area], eax       ; Store result

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Division Instructions

DIV - Unsigned Division

```
; C equivalent:
; unsigned long long dividend = (rdx << 64) | rax;
; rax = dividend / divisor; // quotient
; rdx = dividend % divisor; // remainder

div divisor
```

Key Points:

- **Dividend:** Double-width in **DX:AX/EDX:EAX/RDX:RAX**
- **Divisor:** Single operand (register or memory)
- **Quotient:** Stored in **AL/AX/EAX/RAX**
- **Remainder:** Stored in **AH/DX/EDX/RDX**

Size Variants:

Instruction	Dividend	Divisor	Quotient	Remainder
<code>div r/m8</code>	AX	r/m8	AL	AH
<code>div r/m16</code>	DX:AX	r/m16	AX	DX
<code>div r/m32</code>	EDX:EAX	r/m32	EAX	EDX
<code>div r/m64</code>	RDX:RAX	r/m64	RAX	RDX

Flags Affected:

- All arithmetic flags are **undefined** after division

Important: Zero the Upper Half!

```

; C equivalent:
; unsigned int quotient = dividend / divisor;

; WRONG - upper half contains garbage!
mov eax, [dividend]
div dword [divisor]      ; May cause #DE (divide error exception)!

; CORRECT - zero the upper half first
mov eax, [dividend]
xor edx, edx             ; Clear EDX (upper half)
div dword [divisor]      ; Now safe

```

Example: Simple Division

```

section .data
    dividend dd 100
    divisor dd 7
    quotient dd 0
    remainder dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned int quotient = dividend / divisor;    // 100 / 7 = 14
    ; unsigned int remainder = dividend % divisor;   // 100 % 7 = 2

    mov eax, [dividend]    ; EAX = 100
    xor edx, edx           ; Clear EDX (no upper 32 bits)
    div dword [divisor]    ; EAX = quotient (14), EDX = remainder (2)

    mov [quotient], eax    ; Store quotient
    mov [remainder], edx  ; Store remainder

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

IDIV - Signed Division

```

; C equivalent:
; long long dividend = (rdx << 64) | rax; // Sign-extended!
; rax = dividend / divisor; // quotient
; rdx = dividend % divisor; // remainder

idiv divisor

```

Same operand structure as `DIV`, but treats operands as **signed** integers.

Key Difference: Sign Extension!

For signed division, you must **sign-extend** the dividend:

```

; C equivalent:
; int quotient = dividend / divisor;

; 32-bit division
mov eax, [dividend]    ; EAX = dividend (32-bit signed)
cdq                    ; Sign-extend EAX into EDX:EAX
idiv dword [divisor]    ; Signed division

; 64-bit division
mov rax, [dividend]    ; RAX = dividend (64-bit signed)
cqo                    ; Sign-extend RAX into RDX:RAX
idiv qword [divisor]    ; Signed division

```

Sign Extension Instructions:

Instruction	Operation	C Equivalent
<code>cbw</code>	AX = sign_extend(AL)	<code>short x = (char)al;</code>
<code>cwd</code>	DX:AX = sign_extend(AX)	<code>int x = (short)ax;</code>
<code>cdq</code>	EDX:EAX = sign_extend(EAX)	<code>long long x = (int)eax;</code>
<code>cqo</code>	RDX:RAX = sign_extend(RAX)	<code>__int128 x = (long)rax;</code>

Example: Signed Division

```
section .data
    dividend dd -100
    divisor dd 7
    quotient dd 0
    remainder dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int quotient = dividend / divisor;    // -100 / 7 = -14
    ; int remainder = dividend % divisor;   // -100 % 7 = -2

    mov eax, [dividend]    ; EAX = -100
    cdq                   ; EDX:EAX = sign-extend(-100)
    idiv dword [divisor]   ; EAX = -14, EDX = -2

    mov [quotient], eax    ; Store quotient
    mov [remainder], edx  ; Store remainder

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```


Practical Applications

Example 1: Computing Average

```

section .data
    numbers dd 15, 23, 42, 8, 31
    count dd 5
    average dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int sum = 0;
    ; for (int i = 0; i < count; ++i) {
    ;     sum += numbers[i];
    ; }
    ; int average = sum / count;

    ; Sum the array
    xor eax, eax          ; sum = 0
    xor ecx, ecx          ; i = 0
sum_loop:
    cmp ecx, [count]
    jge sum_done
    add eax, [numbers + ecx*4]
    inc ecx
    jmp sum_loop

sum_done:
    ; EAX now contains sum (119)

    ; Divide by count
    xor edx, edx          ; Clear upper half
    div dword [count]      ; EAX = 119 / 5 = 23
    mov [average], eax

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 2: Integer Square Root (Newton's Method)

```

section .data
    n dd 144          ; Find sqrt(144)
    result dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int isqrt(int n) {
    ;     if (n == 0) return 0;
    ;     int x = n;
    ;     while (1) {
    ;         int x_new = (x + n / x) / 2;
    ;         if (x_new >= x) return x;
    ;         x = x_new;
    ;     }
    ; }

    mov ebx, [n]          ; n
    test ebx, ebx
    jz done               ; if n == 0, result = 0

    mov eax, ebx          ; x = n (initial guess)

sqrt_loop:
    ; x_new = (x + n/x) / 2
    mov ecx, eax          ; Save x

    mov eax, ebx          ; EAX = n
    xor edx, edx
    div ecx               ; EAX = n / x

    add eax, ecx          ; EAX = x + n/x
    shr eax, 1            ; EAX = (x + n/x) / 2 = x_new

    cmp eax, ecx          ; if x_new >= x
    jge sqrt_done

    jmp sqrt_loop

sqrt_done:
    mov eax, ecx          ; result = x

done:
    mov [result], eax     ; Store result (12)

    ; Exit
    mov rax, 60

```

```
xor rdi, rdi  
syscall
```

Example 3: Convert Decimal to ASCII

```

section .data
    number dd 12345
    buffer times 12 db 0    ; Room for digits + newline + null

section .text
global _start

_start:
    ; C equivalent:
    ; char buffer[12];
    ; int n = number;
    ; int i = 0;
    ; do {
    ;     buffer[i++] = '0' + (n % 10);
    ;     n /= 10;
    ; } while (n > 0);
    ; // Reverse buffer

    mov eax, [number]        ; EAX = number to convert
    lea rsi, [buffer + 11]   ; RSI = end of buffer
    mov byte [rsi], 10       ; Newline
    dec rsi

convert_loop:
    xor edx, edx             ; Clear remainder
    mov ecx, 10
    div ecx                  ; EAX = quotient, EDX = remainder

    add dl, '0'              ; Convert digit to ASCII
    mov [rsi], dl            ; Store digit
    dec rsi

    test eax, eax            ; More digits?
    jnz convert_loop

    ; Now print the string
    inc rsi                  ; Move back to first digit
    mov rax, 1               ; sys_write
    mov rdi, 1               ; stdout
    lea rdx, [buffer + 12]
    sub rdx, rsi             ; Length = end - start
    syscall

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Example 4: 64-bit × 64-bit = 128-bit Multiplication

```

section .data
    ; Multiply two 64-bit numbers to get 128-bit result
    a dq 0xFFFFFFFFFFFFFFFF      ; Max 64-bit value
    b dq 0xFFFFFFFFFFFFFFFF      ; Max 64-bit value
    result_low dq 0
    result_high dq 0

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned __int128 result = (unsigned __int128)a * (unsigned __int128)b;
    ; result_low = (unsigned long long)result;
    ; result_high = (unsigned long long)(result >> 64);

    mov rax, [a]                ; RAX = first operand
    mul qword [b]                ; RDX:RAX = RAX * [b]
    ; Result: RDX = 0xFFFFFFFFFFFFFFFE, RAX = 0x0000000000000001
    ; Because: 0xFFFF...FFFF * 0xFFFF...FFFF = 0xFFFE...0001

    mov [result_low], rax
    mov [result_high], rdx

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Common Pitfalls and Best Practices

1. Division by Zero

```

; C equivalent:
; int safe_divide(int dividend, int divisor) {
;     if (divisor == 0) return -1; // Error
;     return dividend / divisor;
; }

mov eax, [dividend]
mov ebx, [divisor]

test ebx, ebx      ; Check if divisor is zero
jz division_error  ; Handle error

xor edx, edx
div ebx            ; Safe to divide

division_error:
    ; Handle error (set result to -1, etc.)
    mov eax, -1

```

2. Forgetting to Clear/Extend Upper Half

```

; WRONG (unsigned):
mov eax, [dividend]
div dword [divisor]      ; EDX contains garbage!

; CORRECT (unsigned):
mov eax, [dividend]
xor edx, edx             ; Clear EDX
div dword [divisor]

; WRONG (signed):
mov eax, [dividend]
idiv dword [divisor]     ; EDX contains garbage!

; CORRECT (signed):
mov eax, [dividend]
cdq                     ; Sign-extend EAX into EDX:EAX
idiv dword [divisor]

```

3. Overflow Detection

```
; C equivalent:
; bool multiply_overflows(int a, int b) {
;     long long result = (long long)a * b;
;     return result > INT_MAX || result < INT_MIN;
; }

mov eax, [a]
imul eax, [b]      ; EAX = a * b
jo overflow_occurred ; Jump if overflow flag set

overflow_occurred:
    ; Handle overflow
```

4. Using the Right Instruction

```
// Unsigned multiplication → MUL
unsigned int a, b, product;
product = a * b;

// Signed multiplication → IMUL
int a, b, product;
product = a * b;

// Unsigned division → DIV
unsigned int quotient = a / b;

// Signed division → IDIV
int quotient = a / b;
```

Optimization Tips

1. Multiply by Powers of 2: Use Shifts

```
; C equivalent:
; int result = x * 8;

; Slow:
imul eax, [x], 8

; Fast:
mov eax, [x]
shl eax, 3      ; x * 8 = x << 3
```


2. Divide by Powers of 2: Use Shifts

```

; C equivalent (unsigned):
; unsigned int result = x / 4;

; Slow:
mov eax, [x]
xor edx, edx
mov ecx, 4
div ecx

; Fast:
mov eax, [x]
shr eax, 2          ; x / 4 = x >> 2

```

Note: Signed division by power of 2 requires rounding toward zero:

```

; C equivalent (signed):
; int result = x / 4;

mov eax, [x]
cdq          ; Sign extend
and edx, 3   ; Add (divisor - 1) if negative
add eax, edx  ; Adjust for rounding
sar eax, 2   ; Arithmetic shift right

```

3. Multiply by Constants: LEA Trick

```

; C equivalent:
; int result = x * 5;

; Using IMUL:
imul eax, [x], 5

; Using LEA (faster):
mov eax, [x]
lea eax, [rax + rax*4] ; EAX = x + x*4 = x*5

```

4. Reciprocal Multiplication

For repeated division by the same constant, use reciprocal multiplication:

```

; C equivalent:
; // Instead of: result = x / 10
; // Use: result = (x * 0xCCCCCCCD) >> 35

mov eax, [x]
mov edx, 0xCCCCCCCD    ; Reciprocal of 10
mul edx                ; EDX:EAX = x * reciprocal
shr edx, 3              ; Shift by (35 - 32) = 3
; EDX now contains x / 10

```

Performance Characteristics

Operation	Latency	Throughput	Notes
<code>imul r32, r32</code>	3 cycles	1/cycle	Two-operand form
<code>imul r64, r64</code>	3 cycles	1/cycle	Two-operand form
<code>mul r32</code>	3 cycles	1/cycle	One-operand form
<code>mul r64</code>	3 cycles	1/cycle	One-operand form
<code>div r32</code>	14-23 cycles	6 cycles	Very slow!
<code>div r64</code>	32-95 cycles	20-40 cycles	Extremely slow!
<code>idiv r32</code>	17-26 cycles	6 cycles	Very slow!
<code>idiv r64</code>	39-103 cycles	20-40 cycles	Extremely slow!

Key Takeaways:

- Multiplication is relatively fast (3 cycles)
- Division is **very slow** (20-100 cycles)
- Avoid division in tight loops when possible
- Use shifts, LEA, and reciprocal multiplication as alternatives

Practice Exercises

Exercise 1: Factorial

Write a function to calculate `n!` (factorial of n).

```

; C equivalent:
; unsigned long long factorial(int n) {
;     unsigned long long result = 1;
;     for (int i = 2; i <= n; ++i) {
;         result *= i;
;     }
;     return result;
; }

section .data
    n dd 10
    result dq 0

section .text
global _start

_start:
    ; Your code here
    ; Calculate factorial of n and store in result

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Solution

```

section .data
    n dd 10
    result dq 0

section .text
global _start

_start:
    mov rax, 1          ; result = 1
    mov ecx, 2          ; i = 2

factorial_loop:
    cmp ecx, [n]
    jg factorial_done

    imul rax, rcx        ; result *= i
    inc ecx
    jmp factorial_loop

factorial_done:
    mov [result], rax    ; result = 3,628,800

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Exercise 2: GCD (Greatest Common Divisor)

Implement Euclid's algorithm to find GCD of two numbers.

```

; C equivalent:
; int gcd(int a, int b) {
;     while (b != 0) {
;         int temp = b;
;         b = a % b;
;         a = temp;
;     }
;     return a;
; }

section .data
    a dd 48
    b dd 18
    result dd 0

section .text
global _start

_start:
    ; Your code here
    ; Calculate GCD and store in result

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Solution

```

section .data
    a dd 48
    b dd 18
    result dd 0

section .text
global _start

_start:
    mov eax, [a]           ; EAX = a
    mov ebx, [b]           ; EBX = b

gcd_loop:
    test ebx, ebx          ; while (b != 0)
    jz gcd_done

    xor edx, edx           ; Clear remainder
    div ebx                ; EDX = a % b

    mov eax, ebx           ; a = b
    mov ebx, edx           ; b = remainder
    jmp gcd_loop

gcd_done:
    mov [result], eax      ; result = 6

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Exercise 3: Check if Number is Prime

```

; C equivalent:
; bool is_prime(int n) {
;     if (n <= 1) return false;
;     if (n <= 3) return true;
;     if (n % 2 == 0 || n % 3 == 0) return false;
;     for (int i = 5; i * i <= n; i += 6) {
;         if (n % i == 0 || n % (i + 2) == 0)
;             return false;
;     }
;     return true;
; }

section .data
    n dd 97
    is_prime db 0          ; 1 = prime, 0 = not prime

section .text
global _start

_start:
    ; Your code here

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Summary

Multiplication

- **MUL**: Unsigned, double-width result in **DX:AX/EDX:EAX/RDX:RAX**
- **IMUL** (1-operand): Signed, double-width result
- **IMUL** (2-operand): Signed, single register result (most common)
- **IMUL** (3-operand): Signed with immediate value

Division

- **DIV**: Unsigned, requires zeroing upper half (**xor edx, edx**)
- **IDIV**: Signed, requires sign-extending (**cdq** or **cqo**)
- **Quotient**: In **AL/AX/EAX/RAX**
- **Remainder**: In **AH/DX/EDX/RDX**

Key Concepts

1. Always prepare upper register before division
2. Check for division by zero
3. Use appropriate signed/unsigned instruction
4. Division is slow - optimize when possible
5. CF/OF indicate multiplication overflow

Optimization

- Use **shifts** for powers of 2
- Use **LEA** for simple multiplications (3x, 5x, 9x)
- Use **reciprocal multiplication** for repeated divisions
- Avoid division in hot loops

Next Topic

In **Topic 14: Shifts & Rotates (Advanced)**, we'll dive deeper into bit manipulation with shifts, rotates, and their applications in optimization, bitfields, and low-level programming.

Preview:

- Logical vs arithmetic shifts
- Rotate operations
- Multi-precision arithmetic
- Bit extraction and manipulation
- Performance optimization techniques

CHAPTER 31

Topic 14: Shifts and Rotates (Advanced)

Overview

Shift and rotate instructions are fundamental bit manipulation operations. They're used for multiplication/division by powers of 2, bit extraction, packing/unpacking data, and many low-level optimizations.

```
// C equivalent concepts:
x << 2;    // Shift left (multiply by 4)
x >> 2;    // Shift right (divide by 4)
x >>> 2;   // Logical right shift (unsigned)
// Rotates have no direct C equivalent
```

Logical Shifts

SHL / SAL - Shift Left

Shifts bits to the left, filling with zeros from the right. Both **SHL** and **SAL** are identical.

```
; C equivalent:
; x = x << count;

shl dest, count    ; dest <<= count
sal dest, count    ; Same as SHL
```

What happens:

1. Bits shift left by **count** positions
2. Leftmost bits fall into the **Carry Flag (CF)**
3. Rightmost bits filled with **0**
4. Effectively multiplies by 2^{count} (if no overflow)

```
Before: 01011010 (0x5A = 90)
SHL 1:  10110100 (0xB4 = 180)  CF=0
SHL 1:  01101000 (0x68 = 104)  CF=1 (bit fell off)
```

Syntax variants:

```

; C equivalent:
; x = x << 1;
; y = y << 3;
; z = z << cl;

shl rax, 1      ; Shift by 1 (most efficient)
shl rbx, 3      ; Shift by immediate (0-63)
shl rcx, cl     ; Shift by CL register (variable)

```

Flags:

- **CF**: Last bit shifted out
- **OF**: Set if sign bit changed (only for count=1)
- **SF, ZF, PF**: Set according to result

Example: Multiply by 8

```

section .data
    x dd 15
    result dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int result = x * 8; // 15 * 8 = 120

    mov eax, [x]
    shl eax, 3      ; Multiply by 2^3 = 8
    mov [result], eax ; result = 120

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

SHR - Logical Shift Right

Shifts bits to the right, filling with zeros from the left. Used for **unsigned** division by powers of 2.

```

; C equivalent (unsigned):
; x = x >> count;

shr dest, count ; dest >>= count (unsigned)

```

What happens:

1. Bits shift right by `count` positions
2. Rightmost bits fall into the **Carry Flag (CF)**
3. Leftmost bits filled with **0**
4. Effectively divides by 2^{count} (unsigned)

```
Before: 10110100 (0xB4 = 180)
SHR 1:  01011010 (0x5A = 90)   CF=0
SHR 1:  00101101 (0x2D = 45)   CF=0
```

Example: Divide by 4 (Unsigned)

```
section .data
    x dd 100
    result dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned int result = x / 4; // 100 / 4 = 25

    mov eax, [x]
    shr eax, 2           ; Divide by 2^2 = 4
    mov [result], eax    ; result = 25

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Arithmetic Shift

SAR - Arithmetic Shift Right

Like `SHR`, but preserves the **sign bit** (leftmost bit). Used for **signed** division by powers of 2.

```
; C equivalent (signed):
; x = x >> count; // For signed integers

sar dest, count    ; dest >>= count (signed)
```

What happens:

1. Bits shift right by `count` positions
2. Rightmost bits fall into the **Carry Flag (CF)**
3. Leftmost bits filled with **original sign bit**
4. Effectively divides by 2^{count} (signed, rounds toward $-\infty$)

Positive: 01011010 (0x5A = 90)

SAR 1: 00101101 (0x2D = 45) CF=0 (sign bit = 0)

Negative: 10110100 (0xB4 = -76 in signed)

SAR 1: 11011010 (0xDA = -38) CF=0 (sign bit = 1, preserved)

Example: Signed Division

```
section .data
    x dd -100
    result dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int result = x / 4; // -100 / 4 = -25

    mov eax, [x]
    sar eax, 2          ; Signed divide by 4
    mov [result], eax   ; result = -25

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Comparison: SHR vs SAR

```
section .data
    unsigned_val dd 0x80000000 ; 2,147,483,648 (unsigned)
    signed_val   dd 0x80000000 ; -2,147,483,648 (signed)

section .text
    ; Unsigned division by 2
    ; C equivalent:
    ; unsigned int result = 0x80000000U >> 1; // 1,073,741,824
    mov eax, [unsigned_val]
    shr eax, 1                ; EAX = 0x40000000 (1,073,741,824)

    ; Signed division by 2
    ; C equivalent:
    ; int result = ((int)0x80000000) >> 1; // -1,073,741,824
    mov eax, [signed_val]
    sar eax, 1                ; EAX = 0xC0000000 (-1,073,741,824)
```

Rotate Instructions

Rotate instructions shift bits in a circular manner - bits that fall off one end reappear at the other.

ROL - Rotate Left

```
; No direct C equivalent
; Conceptually: (x << count) | (x >> (SIZE - count))

rol dest, count    ; Rotate left
```

What happens:

1. Bits rotate left by `count` positions
2. Bits that fall off the left reappear on the right
3. Last bit rotated out copied to **CF**

```
Before: 10110100
ROL 1:  01101001 CF=1 (leftmost bit wraps to right)
ROL 1:  11010010 CF=0
```

Example: Rotate for Bit Inspection

```

section .data
    x dd 0xABCD1234

section .text
global _start

_start:
    ; C equivalent (conceptual):
    ; // Inspect each nibble (4 bits) by rotating
    ; for (int i = 0; i < 8; ++i) {
    ;     int nibble = x & 0xF;
    ;     x = (x >> 4) | (x << 28); // Rotate right by 4
    ; }

    mov eax, [x]
    mov ecx, 8          ; 8 nibbles in 32 bits

inspect_loop:
    rol eax, 4          ; Rotate left by 4 (next nibble to bottom)
    ; Bottom 4 bits now contain next nibble
    loop inspect_loop

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

ROR - Rotate Right

```

; No direct C equivalent
; Conceptually: (x >> count) | (x << (SIZE - count))

ror dest, count      ; Rotate right

```

What happens:

1. Bits rotate right by `count` positions
2. Bits that fall off the right reappear on the left
3. Last bit rotated out copied to **CF**

```

Before: 10110100
ROR 1:  01011010  CF=0  (rightmost bit wraps to left)
ROR 1:  00101101  CF=0

```

RCL - Rotate Through Carry Left

Like `ROL`, but **includes the Carry Flag** in the rotation.

```
rcl dest, count    ; Rotate through carry left
```

What happens:

1. Bits rotate left through CF
2. CF becomes the rightmost bit
3. Leftmost bit becomes new CF

```
CF=1, Value: 10110100
RCL 1:  CF=1, Value: 01101001  (CF -> bit 0, bit 7 -> CF)
```

Use case: Multi-precision arithmetic (shifting 128-bit, 256-bit values)

RCR - Rotate Through Carry Right

Like **ROR**, but **includes the Carry Flag** in the rotation.

```
rcr dest, count    ; Rotate through carry right
```

Example: 128-bit Right Shift

```

section .data
    ; 128-bit value: high:low
    value_high dq 0x0123456789ABCDEF
    value_low  dq 0xFEDCBA9876543210

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned __int128 value = ((__int128)value_high << 64) | value_low;
    ; value >>= 1; // Shift entire 128-bit value right by 1

    ; Shift high qword right, bit 0 goes to CF
    mov rax, [value_high]
    shr rax, 1                ; Shift high part, bit 0 -> CF
    mov [value_high], rax

    ; Rotate low qword right through carry
    mov rax, [value_low]
    rcr rax, 1                ; CF -> bit 63, bit 0 -> CF
    mov [value_low], rax

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Double-Precision Shifts

SHLD - Shift Left Double

Shifts the destination left, filling from the source register.

```

; C equivalent (conceptual):
; dest = (dest << count) | (src >> (SIZE - count));

shld dest, src, count    ; Shift dest left, fill from src

```

Example: Extract Middle Bits


```

section .data
    high dd 0xABCD0000
    low  dd 0x00001234

section .text
global _start

_start:
    ; C equivalent:
    ; // Extract bits 16-47 from a 64-bit value (high:low)
    ; unsigned long long combined = ((unsigned long long)high << 32) | low;
    ; unsigned int middle = (combined >> 16) & 0xFFFFFFFF;

    mov eax, [high]
    mov edx, [low]
    shld eax, edx, 16    ; EAX = (high << 16) | (low >> 16)
    ; EAX now contains 0x0000ABCD

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

SHRD - Shift Right Double

Shifts the destination right, filling from the source register.

```

; C equivalent (conceptual):
; dest = (dest >> count) | (src << (SIZE - count));

shrd dest, src, count    ; Shift dest right, fill from src

```

Example: 64-bit Rotate Right

```

; C equivalent:
; // Rotate a 64-bit value right by 8 bits
; unsigned long long rotate_right_64(unsigned long long x, int count) {
;     return (x >> count) | (x << (64 - count));
; }

mov rax, [value]          ; RAX = 64-bit value
mov rdx, rax              ; Copy to RDX
shrd rax, rdx, 8          ; RAX = (RAX >> 8) | (RDX << 56)
; RAX now rotated right by 8

```

Practical Applications

Application 1: Bit Extraction

```
section .data
    ; Extract bits 12-19 from a 32-bit value
    value dd 0x12345678

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned int extract = (value >> 12) & 0xFF;

    mov eax, [value]    ; EAX = 0x12345678
    shr eax, 12         ; EAX = 0x00012345
    and eax, 0xFF       ; EAX = 0x00000045
    ; Extracted bits 12-19 (value = 0x45 = 69)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall
```

Application 2: Bit Field Packing

```

section .data
    ; Pack three values: A (5 bits), B (11 bits), C (16 bits)
    a dd 0x1F          ; 5 bits: 11111
    b dd 0x7FF         ; 11 bits: 11111111111
    c dd 0xFFFF        ; 16 bits: 1111111111111111
    packed dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; struct Packed {
    ;     unsigned a : 5;    // bits 0-4
    ;     unsigned b : 11;   // bits 5-15
    ;     unsigned c : 16;   // bits 16-31
    ; };
    ; unsigned int packed = (a & 0x1F) | ((b & 0x7FF) << 5) | ((c & 0xFFFF) << 16);

    ; Pack A (bits 0-4)
    mov eax, [a]
    and eax, 0x1F      ; Mask to 5 bits

    ; Pack B (bits 5-15)
    mov ebx, [b]
    and ebx, 0x7FF     ; Mask to 11 bits
    shl ebx, 5         ; Shift to position
    or eax, ebx

    ; Pack C (bits 16-31)
    mov ebx, [c]
    and ebx, 0xFFFF    ; Mask to 16 bits
    shl ebx, 16         ; Shift to position
    or eax, ebx

    mov [packed], eax  ; Result: 0xFFFFFFFF

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Application 3: Fast Power of 2 Check

```

section .data
    x dd 64                ; Is 64 a power of 2? (Yes)

section .text
global _start

_start:
    ; C equivalent:
    ; bool is_power_of_2(unsigned int x) {
    ;     return x != 0 && (x & (x - 1)) == 0;
    ; }
    ; // Power of 2 has exactly one bit set

    mov eax, [x]
    test eax, eax          ; Check if zero
    jz not_power_of_2

    mov ebx, eax
    dec ebx                ; EBX = x - 1
    and ebx, eax            ; EBX = x & (x - 1)
    jnz not_power_of_2

    ; Is power of 2
    mov edi, 1
    jmp done

not_power_of_2:
    xor edi, edi

done:
    ; EDI = 1 (true, 64 is power of 2)

    ; Exit
    mov rax, 60
    syscall

```

Application 4: Byte Swapping (Endianness Conversion)

```

section .data
    value dd 0x12345678      ; Little-endian
    swapped dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned int swap32(unsigned int x) {
    ;     return ((x & 0xFF000000) >> 24) |
    ;         ((x & 0x00FF0000) >> 8) |
    ;         ((x & 0x0000FF00) << 8) |
    ;         ((x & 0x000000FF) << 24);
    ; }

    mov eax, [value]        ; EAX = 0x12345678

    ; Modern CPUs have BSWAP instruction:
    bswap eax                ; EAX = 0x78563412

    ; Manual method:
    ; mov eax, [value]
    ; rol eax, 8             ; 0x34567812
    ; mov ebx, eax
    ; and eax, 0x00FF00FF    ; 0x00560012
    ; and ebx, 0xFF00FF00    ; 0x34007800
    ; shr ebx, 8             ; 0x00340078
    ; shl eax, 8             ; 0x56001200
    ; or eax, ebx            ; 0x56341278 (wrong!)

    mov [swapped], eax      ; Result: 0x78563412 (big-endian)

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Application 5: Count Leading Zeros (CLZ)

```

section .data
    x dd 0x00001234      ; 16 leading zeros
    clz dd 0

section .text
global _start

_start:
    ; C equivalent:
    ; int count_leading_zeros(unsigned int x) {
    ;     if (x == 0) return 32;
    ;     int count = 0;
    ;     while ((x & 0x80000000) == 0) {
    ;         count++;
    ;         x <<= 1;
    ;     }
    ;     return count;
    ; }

    mov eax, [x]
    test eax, eax
    jz all_zeros

    xor ecx, ecx          ; count = 0

clz_loop:
    test eax, 0x80000000
    jnz clz_done
    shl eax, 1
    inc ecx
    jmp clz_loop

all_zeros:
    mov ecx, 32

clz_done:
    mov [clz], ecx        ; Result: 16

    ; Modern alternative: BSR (Bit Scan Reverse)
    mov eax, [x]
    bsr ecx, eax          ; ECX = position of highest bit set
    xor ecx, 31           ; Convert to leading zeros
    ; ECX = 16

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Application 6: Alignment Check

```

section .data
    address dq 0x1000      ; 4KB-aligned
    align_mask dq 0xFFF    ; 4KB - 1

section .text
global _start

_start:
    ; C equivalent:
    ; bool is_aligned(void* addr, size_t alignment) {
    ;     return ((uintptr_t)addr & (alignment - 1)) == 0;
    ; }

    mov rax, [address]
    and rax, [align_mask]
    test rax, rax
    jz is_aligned

    ; Not aligned
    xor edi, edi
    jmp done

is_aligned:
    mov edi, 1

done:
    ; EDI = 1 (aligned)

    ; Exit
    mov rax, 60
    syscall

```

Advanced: Barrel Shifter Emulation

Some processors have hardware barrel shifters that can shift by any amount in one cycle. Here's how to emulate efficient shifting:


```

section .data
    value dq 0x123456789ABCDEF0
    shift_count db 17

section .text
global _start

_start:
    ; C equivalent:
    ; unsigned long long result = value << shift_count;

    movzx ecx, byte [shift_count]    ; CL = shift count
    mov rax, [value]
    shl rax, cl                      ; Variable shift

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Optimization Techniques

1. Multiply/Divide by Powers of 2

```

; C equivalent:
; int x = 100;
; int a = x * 16;
; int b = x / 8;

; Slow:
imul eax, [x], 16
mov edx, [x]
mov ecx, 8
xor ebx, ebx
div ecx

; Fast:
mov eax, [x]
shl eax, 4          ; x * 16
mov edx, [x]
shr edx, 3          ; x / 8

```

2. Extracting Specific Bits

```

; C equivalent:
; unsigned int extract_bits_10_15(unsigned int x) {
;     return (x >> 10) & 0x3F; // Extract 6 bits
; }

; Method 1: Shift then mask
mov eax, [x]
shr eax, 10
and eax, 0x3F

; Method 2: Mask then shift (if other bits needed)
mov eax, [x]
and eax, 0xFC00          ; Mask bits 10-15
shr eax, 10

```

3. Clearing Lower Bits (Alignment)

```

; C equivalent:
; void* align_down(void* ptr, size_t alignment) {
;     return (void*)((uintptr_t)ptr & ~(alignment - 1));
; }

; Align pointer to 16-byte boundary
and rax, ~0xF          ; Clear lower 4 bits

; Align to 4KB (0x1000)
and rax, ~0xFFF        ; Clear lower 12 bits

```

4. Fast Modulo Powers of 2

```

; C equivalent:
; unsigned int mod = x % 16;

; Slow:
mov eax, [x]
xor edx, edx
mov ecx, 16
div ecx          ; EDX = remainder

; Fast:
mov eax, [x]
and eax, 0xF     ; x & (16-1) = x % 16

```

Performance Characteristics

Instruction	Latency	Throughput	Notes
<code>shl/shr/sar r, 1</code>	1 cycle	0.5/cycle	Immediate count = 1
<code>shl/shr/sar r, imm</code>	1 cycle	0.5/cycle	Immediate count
<code>shl/shr/sar r, cl</code>	1-3 cycles	2/cycle	Variable count
<code>rol/ror r, imm</code>	1 cycle	0.5/cycle	Fast on modern CPUs
<code>rcl/rcr r, 1</code>	3 cycles	1/cycle	Slower due to CF dependency
<code>shld/shrd</code>	3 cycles	0.5/cycle	Complex operation
<code>bswap</code>	1 cycle	0.5/cycle	Very fast byte swap

Common Patterns and Idioms

Pattern 1: Test Specific Bit

```
; C equivalent:
; bool is_bit_set(unsigned int x, int bit) {
;     return (x & (1 << bit)) != 0;
; }

mov eax, [x]
bt eax, 5      ; Test bit 5, result in CF
jc bit_is_set

; Alternative with shift:
mov eax, [x]
shr eax, 5      ; Shift bit 5 to bit 0
and eax, 1      ; Isolate bit
```

Pattern 2: Set/Clear Specific Bit

```

; C equivalent:
; x |= (1 << bit); // Set bit
; x &= ~(1 << bit); // Clear bit

; Set bit 7
bts dword [x], 7 ; Bit Test and Set

; Clear bit 7
btr dword [x], 7 ; Bit Test and Reset

; Alternative:
mov eax, [x]
or eax, 0x80 ; Set bit 7
and eax, ~0x80 ; Clear bit 7

```

Pattern 3: Sign Extension

```

; C equivalent:
; int extend = (char)byte_val; // Sign-extend 8-bit to 32-bit

; 8-bit to 32-bit
movsx eax, byte [val]

; Manual with shifts:
movzx eax, byte [val]
shl eax, 24
sar eax, 24 ; Sign bit fills from left

```

Practice Exercises

Exercise 1: Rotate Bits

Write code to rotate a 32-bit value left by an arbitrary amount.

```

section .data
    value dd 0x12345678
    count dd 12
    result dd 0

section .text
global _start

_start:
    ; Your code here: rotate value left by count
    ; Store in result

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Solution

```

section .data
    value dd 0x12345678
    count dd 12
    result dd 0

section .text
global _start

_start:
    mov eax, [value]
    mov ecx, [count]
    rol eax, cl
    mov [result], eax    ; Result: 0x45678123

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Exercise 2: Count Set Bits (Population Count)

Count the number of 1-bits in a 32-bit value.

```

; C equivalent:
; int popcount(unsigned int x) {
;     int count = 0;
;     while (x) {
;         count += x & 1;
;         x >>= 1;
;     }
;     return count;
; }

```

```

section .data
    value dd 0x12345678
    count dd 0

```

```

section .text
global _start

```

```

_start:
    ; Your code here

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Solution

```

section .data
    value dd 0x12345678      ; Binary: 00010010001101000101011001111000
    count dd 0

section .text
global _start

_start:
    mov eax, [value]
    xor ecx, ecx             ; count = 0

popcount_loop:
    test eax, eax
    jz popcount_done

    mov edx, eax
    and edx, 1               ; Check lowest bit
    add ecx, edx              ; Add to count
    shr eax, 1               ; Shift right
    jmp popcount_loop

popcount_done:
    mov [count], ecx         ; Result: 13

    ; Modern alternative: POPCNT instruction
    ; popcnt ecx, [value]

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Exercise 3: Reverse Bits

Reverse the order of bits in a 32-bit value.

```

; C equivalent:
; unsigned int reverse_bits(unsigned int x) {
;     unsigned int result = 0;
;     for (int i = 0; i < 32; ++i) {
;         result = (result << 1) | (x & 1);
;         x >>= 1;
;     }
;     return result;
; }

```

```

section .data
    value dd 0x12345678
    reversed dd 0

```

```

section .text
global _start

```

```

_start:
    ; Your code here

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Solution


```

section .data
    value dd 0x12345678
    reversed dd 0

section .text
global _start

_start:
    mov eax, [value]          ; Source
    xor ebx, ebx              ; Result = 0
    mov ecx, 32               ; Bit count

reverse_loop:
    shl ebx, 1                ; Make room for next bit
    shr eax, 1                ; Get lowest bit in CF
    adc ebx, 0                ; Add CF to result
    loop reverse_loop

    mov [reversed], ebx       ; Result: 0x1E6A2C48

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

```

Summary

Shift Instructions

- **SHL/SAL**: Shift left (multiply by 2^n), fill with 0
- **SHR**: Logical shift right (unsigned divide by 2^n), fill with 0
- **SAR**: Arithmetic shift right (signed divide by 2^n), fill with sign bit

Rotate Instructions

- **ROL/ROR**: Rotate left/right (circular)
- **RCL/RCR**: Rotate through carry (9-bit, 17-bit, 33-bit, 65-bit rotation)

Double Precision

- **SHLD/SHRD**: Shift from two registers

Key Concepts

1. Shifts are fast alternatives to multiply/divide by powers of 2

2. Logical shifts treat values as unsigned
3. Arithmetic shifts preserve sign bit
4. Rotates have no C equivalent but useful for bit manipulation
5. Carry flag captures bits that “fall off”

Common Uses

- Fast multiplication/division
 - Bit field extraction/insertion
 - Alignment checks and adjustments
 - Endianness conversion
 - Multi-precision arithmetic
 - Bit manipulation algorithms
-

Next Topic

In **Topic 15: Macros & Directives**, we'll learn about NASM's powerful macro system and assembler directives that help write cleaner, more maintainable assembly code.

Preview:

- Defining constants and symbols
- Text substitution macros
- Multi-line macros with parameters
- Conditional assembly
- File inclusion
- Repetition and loops at assembly time

CHAPTER 32

Topic 15: Macros & Directives

Master NASM's powerful preprocessor for code generation, conditional assembly, and creating reusable code templates.

Overview

Macros and **directives** let you:

- Create reusable code templates
- Conditional compilation
- Organize large projects
- Generate repetitive code automatically
- Build domain-specific assembly "languages"

Part 1: Simple Macros

%define - Simple Text Replacement

Like `#define` in C.

C Equivalent:

```
#define BUFFER_SIZE 1024
#define MAX(a,b) ((a) > (b) ? (a) : (b))
```

NASM:

```
%define BUFFER_SIZE 1024
%define SYSCALL_WRITE 1
%define SYSCALL_EXIT 60

section .bss
    buffer resb BUFFER_SIZE           ; Expands to: resb 1024

section .text
    mov rax, SYSCALL_WRITE           ; Expands to: mov rax, 1
```

%assign - Numeric Constants

Can be redefined (unlike `%define`).

```
%assign COUNTER 0
%assign COUNTER COUNTER+1      ; COUNTER = 1
%assign COUNTER COUNTER+1      ; COUNTER = 2
```

Part 2: Multi-Line Macros

Basic Macro Syntax

```
%macro macro_name parameter_count
    ; Macro body
    ; %1, %2, %3... refer to parameters
%endmacro
```

Example 1: Print Macro

C Equivalent:

```
#define PRINT_STR(msg, len) \
    write(1, msg, len)
```

NASM:

```
%macro print_string 2          ; 2 parameters
    mov rax, 1                 ; sys_write
    mov rdi, 1                 ; stdout
    mov rsi, %1                ; message (1st parameter)
    mov rdx, %2                ; length (2nd parameter)
    syscall
%endmacro

; Usage:
section .data
    msg db "Hello!", 10
    len equ $ - msg

section .text
    print_string msg, len      ; Expands to mov/syscall sequence
```

Example 2: Exit Macro

```
%macro exit 1                                ; 1 parameter: exit code
    mov rax, 60
    mov rdi, %1
    syscall
%endmacro

; Usage:
exit 0                                ; Exit with code 0
exit 42                               ; Exit with code 42
```

Example 3: Function Prologue/Epilogue

C Equivalent:

```
// Every function does this:
void func() {
    // Prologue
    // ... body ...
    // Epilogue
}
```

NASM:

```
%macro prologue 0
    push rbp
    mov rbp, rsp
%endmacro

%macro epilogue 0
    mov rsp, rbp
    pop rbp
    ret
%endmacro

my_function:
    prologue
    ; Function body here
    epilogue
```

Example 4: Save/Restore Registers

```
%macro pushall 0
    push rax
    push rbx
    push rcx
    push rdx
    push rsi
    push rdi
%endmacro

%macro popall 0
    pop rdi
    pop rsi
    pop rdx
    pop rcx
    pop rbx
    pop rax
%endmacro

my_function:
    pushall
    ; Use all registers freely
    popall
    ret
```

Part 3: Macro Parameters

Parameter Types

```
%macro demo 3
    mov rax, %1          ; 1st parameter
    mov rbx, %2          ; 2nd parameter
    mov rcx, %3          ; 3rd parameter
%endmacro

demo 10, 20, 30         ; Expands to 3 mov instructions
```

Default Parameters

```
%macro greet 1-2 "World"           ; 1-2 params, default 2nd = "World"
    section .data
        %%msg db %1, " ", %2, 10
        %%len equ $ - %%msg
    section .text
        print_string %%msg, %%len
%endmacro

greet "Hello"                       ; Uses default: "Hello World"
greet "Hi", "There"                 ; Custom: "Hi There"
```

Variable Number of Parameters

```
%macro sum 1-*                      ; 1 or more parameters
    xor rax, rax
    %rep %0                          ; %0 = number of parameters
        add rax, %1
        %rotate 1                   ; Shift parameters
    %endrep
%endmacro

sum 10                              ; RAX = 10
sum 10, 20, 30                     ; RAX = 60
```

Part 4: Local Labels

Prevent label conflicts when macro is used multiple times.

```
%macro my_abs 1                    ; Absolute value
    test %1, %1
    jns %%positive                 ; %%label creates unique label
    neg %1
%%positive:
%endmacro

my_abs rax                         ; Creates label like ..@1.positive
my_abs rbx                         ; Creates different label ..@2.positive
```

Part 5: Conditional Assembly

%if Directives

```
%define DEBUG 1

%if DEBUG
    ; Debug code included
    mov rdi, error_msg
    call print
%endif

%ifdef FEATURE_X
    ; Include if FEATURE_X is defined
    call feature_x_init
%endif

%ifndef FEATURE_Y
    ; Include if FEATURE_Y is NOT defined
    call fallback_init
%endif
```

Compile-Time Conditions

```
%if BUFFER_SIZE > 1024
    %warning "Buffer size is very large!"
%endif

%if CPU_BITS == 64
    ; 64-bit specific code
    mov rax, large_value
%else
    ; 32-bit specific code
    mov eax, small_value
%endif
```


%rep - Repetition

```
; Generate 10 nops
%rep 10
    nop
%endrep

; Unroll loop
%assign i 0
%rep 5
    mov eax, [array + i*4]
    add ebx, eax
    %assign i i+1
%endrep
```

Part 6: File Inclusion

%include

```
; macros.inc
%macro print 2
    mov rax, 1
    mov rdi, 1
    mov rsi, %1
    mov rdx, %2
    syscall
%endmacro

; main.asm
%include "macros.inc"

section .text
    global _start
_start:
    print msg, len
    exit 0
```

%pathsearch

```
%pathsearch syscalls "syscalls.inc"
%include syscalls ; Search in include paths
```

Part 7: Advanced Macro Techniques

Macro Overloading (by parameter count)

```
%macro print 1                                ; 1 parameter version
    ; Assume null-terminated string
    call strlen
    print_string %1, rax
%endmacro

%macro print 2                                ; 2 parameter version
    print_string %1, %2
%endmacro

print msg                                     ; Calls 1-param version
print msg, 10                                ; Calls 2-param version
```

String Manipulation

```
%define CONCAT(a,b) a %+ b

CONCAT(my_, function):                       ; Expands to: my_function:
    ret

%define STRINGIFY(x) `x`
%define CREATE_MSG(name) db STRINGIFY(name), 0

section .data
    CREATE_MSG>Hello)                       ; Expands to: db "Hello", 0
```

Part 8: Practical Macro Library

System Call Macros

```
%macro sys_write 2                                ; fd, message, length
    mov rax, 1
    mov rdi, %1
    mov rsi, %2
    syscall
%endmacro

%macro sys_exit 1                                  ; exit_code
    mov rax, 60
    mov rdi, %1
    syscall
%endmacro

%macro sys_open 3                                  ; filename, flags, mode
    mov rax, 2
    mov rdi, %1
    mov rsi, %2
    mov rdx, %3
    syscall
%endmacro
```

Debug Macros

```
%ifdef DEBUG
    %macro dbg_print 2
        pushall
        print_string %1, %2
        popall
    %endmacro
%else
    %macro dbg_print 2
        ; Empty in release build
    %endmacro
%endif
```

Alignment Macros

```
%macro align_stack 0
    ; Ensure 16-byte alignment
    mov rbp, rsp
    and rsp, -16
%endmacro

%macro restore_stack 0
    mov rsp, rbp
%endmacro
```

Part 9: Complete Example

```

; config.inc
#define VERSION "1.0.0"
#define BUFFER_SIZE 4096
#define DEBUG 1

; macros.inc
%macro print 2
    mov rax, 1
    mov rdi, 1
    mov rsi, %1
    mov rdx, %2
    syscall
%endmacro

%macro exit 1
    mov rax, 60
    mov rdi, %1
    syscall
%endmacro

#ifdef DEBUG
    %macro debug 1
        print %1, debug_msg_len
    %endmacro
%else
    %macro debug 1
        ; No-op in release
    %endmacro
#endif

; main.asm
#include "config.inc"
#include "macros.inc"

section .data
    version db "Version: " %+ VERSION, 10
    version_len equ $ - version

#ifdef DEBUG
    debug_msg db "[DEBUG] Program started", 10
    debug_msg_len equ $ - debug_msg
#endif

section .text
    global _start

_start:
    print version, version_len
    debug debug_msg

```

```
; Main program logic
```

```
exit 0
```

Part 10: Best Practices

DO:

- Use macros for repeated patterns
- Use local labels (`%%label`)
- Document macro parameters
- Use meaningful macro names
- Group related macros in include files

DON'T:

- Overuse macros (readability!)
- Create overly complex macros
- Forget side effects (register usage)
- Use macros for single-use code
- Ignore macro expansion size

Practice Exercises

Exercise 1: Create a MIN/MAX Macro

Solution

```

%macro min 3                                ; dest, a, b
    mov %1, %2
    cmp %1, %3
    jle %%done
    mov %1, %3
%%done:
%endmacro

%macro max 3
    mov %1, %2
    cmp %1, %3
    jge %%done
    mov %1, %3
%%done:
%endmacro

; Usage:
min rax, rbx, rcx                        ; RAX = min(RBX, RCX)
max rdx, rsi, rdi                        ; RDX = max(RSI, RDI)

```

Quick Reference

```

; Defines
%define NAME value
%assign COUNT 0

; Macros
%macro name param_count
    ; body with %1, %2, %3...
%endmacro

; Conditionals
%if condition
%elif condition
%else
%endif

; Loops
%rep count
%endrep

; Include
#include "file.inc"

```

Excellent! You've mastered macros and directives!

Next: Topic 16: Linux System Calls

[← Previous Topic](#) | [Back to Main](#) | [Next Topic →](#)

CHAPTER 33

Topic 16: Linux System Calls

Overview

System calls (syscalls) are the interface between user-space programs and the kernel. They allow programs to request services from the operating system: file I/O, process management, memory allocation, network operations, and more.

```
// C equivalent - syscalls are typically wrapped in libc functions:
#include <unistd.h>
#include <sys/syscall.h>

// High-level wrapper:
write(1, "Hello\n", 6);

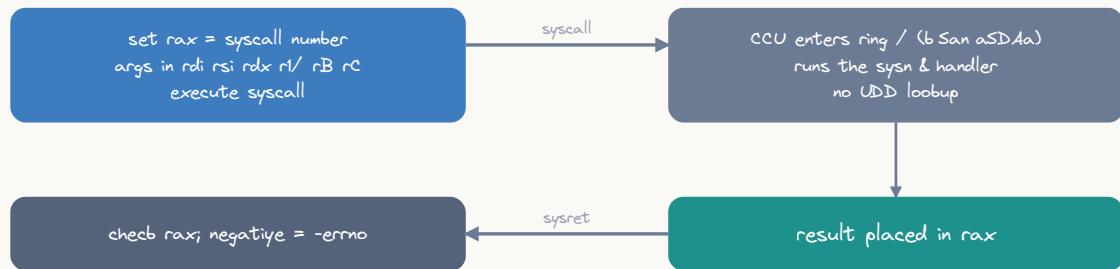
// Low-level syscall:
syscall(SYS_write, 1, "Hello\n", 6);
```

Syscall Mechanism

How Syscalls Work

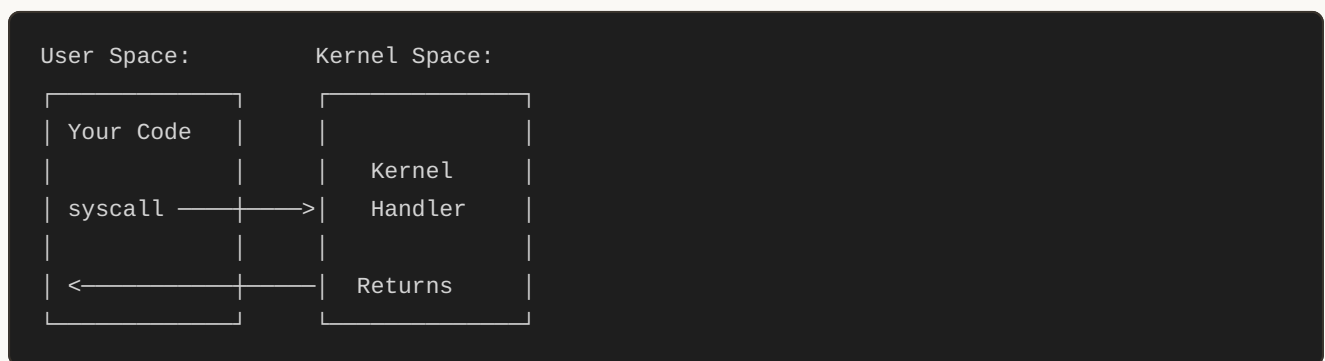
1. **User program** sets up syscall number and arguments in registers
2. **Executes** `syscall` instruction (64-bit) or `int 0x80` (32-bit)
3. **CPU switches** from user mode to kernel mode
4. **Kernel** validates parameters, performs the operation
5. **Return value** placed in RAX/EAX
6. **CPU switches** back to user mode
7. **Program continues** execution

A linux syscall = user → kernel → back



syscall is far cheaper than the legacy int /x86/ — it uses bSAs, not the UDD

ASCII diagram



64-bit Linux Syscall Convention (x86-64)

```

; C equivalent:
; ssize_t write(int fd, const void *buf, size_t count);
; syscall(SYS_write, fd, buf, count);

mov rax, 1          ; Syscall number (SYS_write)
mov rdi, fd         ; Argument 1: file descriptor
mov rsi, buffer     ; Argument 2: buffer address
mov rdx, count      ; Argument 3: byte count
syscall            ; Execute syscall
; RAX now contains return value (bytes written, or -errno on error)
  
```

Register Mapping:

Argument	Register	Notes
Syscall #	RAX	Set before syscall
Arg 1	RDI	
Arg 2	RSI	
Arg 3	RDX	
Arg 4	R10	(NOT RCX!)
Arg 5	R8	
Arg 6	R9	
Return	RAX	Positive = success, negative = -errno

Why R10 instead of RCX? The `syscall` instruction uses RCX to save the return address (RIP), so R10 is used for the 4th argument instead.

32-bit Linux Syscall Convention (i386)

```

; C equivalent:
; ssize_t write(int fd, const void *buf, size_t count);

mov eax, 4          ; Syscall number (sys_write = 4 in 32-bit)
mov ebx, fd         ; Argument 1
mov ecx, buffer     ; Argument 2
mov edx, count      ; Argument 3
int 0x80            ; Execute syscall (32-bit method)
; EAX contains return value

```

Register Mapping (32-bit):

Argument	Register
Syscall #	EAX
Arg 1	EBX
Arg 2	ECX
Arg 3	EDX
Arg 4	ESI
Arg 5	EDI
Arg 6	EBP
Return	EAX

Common System Calls

File I/O Syscalls

read - Read from File Descriptor

```

; C equivalent:
; ssize_t read(int fd, void *buf, size_t count);

section .bss
    buffer resb 1024

section .text
    mov rax, 0          ; sys_read
    mov rdi, 0          ; stdin (fd = 0)
    lea rsi, [buffer]   ; Buffer address
    mov rdx, 1024       ; Max bytes to read
    syscall
    ; RAX = number of bytes read, or -1 on error

```

write - Write to File Descriptor

```

; C equivalent:
; ssize_t write(int fd, const void *buf, size_t count);

section .data
    msg db "Hello, World!", 10      ; Message with newline
    msg_len equ $ - msg

section .text
    mov rax, 1                     ; sys_write
    mov rdi, 1                     ; stdout (fd = 1)
    lea rsi, [msg]                 ; Message address
    mov rdx, msg_len               ; Message length
    syscall
    ; RAX = number of bytes written

```

open - Open File

```

; C equivalent:
; int open(const char *pathname, int flags, mode_t mode);
; // flags: O_RDONLY=0, O_WRONLY=1, O_RDWR=2, O_CREAT=64, O_TRUNC=512
; // mode: 0644 (rw-r--r--) = 0x1A4

section .data
    filename db "output.txt", 0

section .text
    mov rax, 2                     ; sys_open
    lea rdi, [filename]           ; Pathname
    mov rsi, 0x41                  ; O_WRONLY | O_CREAT (0x01 | 0x40)
    mov rdx, 0o644                ; Mode: rw-r--r--
    syscall
    ; RAX = file descriptor (or -1 on error)

```

Common Flags:

Flag	Value	Description
O_RDONLY	0	Read only
O_WRONLY	1	Write only
O_RDWR	2	Read and write
O_CREAT	0x40	Create file if doesn't exist
O_TRUNC	0x200	Truncate to 0 length
O_APPEND	0x400	Append mode
O_EXCL	0x80	Fail if file exists (with O_CREAT)

`close` - Close File Descriptor

```

; C equivalent:
; int close(int fd);

mov rax, 3           ; sys_close
mov rdi, [file_fd]   ; File descriptor to close
syscall
; RAX = 0 on success, -1 on error

```

Complete File I/O Example


```

section .data
    filename db "test.txt", 0
    content db "Hello from assembly!", 10
    content_len equ $ - content

section .bss
    fd resq 1          ; File descriptor

section .text
global _start

_start:
    ; C equivalent:
    ; int fd = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    ; if (fd < 0) exit(1);
    ; write(fd, content, content_len);
    ; close(fd);
    ; exit(0);

    ; Open file
    mov rax, 2          ; sys_open
    lea rdi, [filename]
    mov rsi, 0x241      ; O_WRONLY | O_CREAT | O_TRUNC
    mov rdx, 0o644
    syscall

    test rax, rax
    js error            ; Jump if negative (error)
    mov [fd], rax       ; Save fd

    ; Write to file
    mov rax, 1          ; sys_write
    mov rdi, [fd]
    lea rsi, [content]
    mov rdx, content_len
    syscall

    ; Close file
    mov rax, 3          ; sys_close
    mov rdi, [fd]
    syscall

    ; Exit success
    mov rax, 60         ; sys_exit
    xor rdi, rdi        ; Exit code 0
    syscall

error:
    mov rax, 60

```

```
mov rdi, 1          ; Exit code 1
syscall
```

Process Management

`exit` - Terminate Process

```
; C equivalent:
; void exit(int status);

mov rax, 60          ; sys_exit
mov rdi, 0           ; Exit code (0 = success)
syscall
```

`fork` - Create Child Process

```
; C equivalent:
; pid_t fork(void);
; // Returns: 0 in child, child PID in parent, -1 on error

mov rax, 57          ; sys_fork
syscall

test rax, rax
jz child_code        ; RAX = 0, we're in child
js error             ; RAX < 0, error occurred
; RAX > 0, we're in parent, RAX = child PID

parent_code:
    ; Parent-specific code
    jmp done

child_code:
    ; Child-specific code

done:
    ; Continue...
```

execve - Execute Program

```

; C equivalent:
; int execve(const char *pathname, char *const argv[], char *const envp[]);

section .data
    prog db "/bin/ls", 0
    arg0 db "/bin/ls", 0
    arg1 db "-la", 0
    argv dq arg0, arg1, 0      ; NULL-terminated array
    envp dq 0                  ; Empty environment

section .text
    mov rax, 59                ; sys_execve
    lea rdi, [prog]            ; Program path
    lea rsi, [argv]            ; Arguments array
    lea rdx, [envp]            ; Environment array
    syscall
    ; If execve returns, it failed

```

wait4 - Wait for Process

```

; C equivalent:
; pid_t wait4(pid_t pid, int *status, int options, struct rusage *rusage);

section .bss
    status resd 1

section .text
    mov rax, 61                ; sys_wait4
    mov rdi, -1                ; Wait for any child
    lea rsi, [status]          ; Status location
    xor rdx, rdx                ; Options = 0
    xor r10, r10               ; rusage = NULL
    syscall
    ; RAX = PID of terminated child

```

Memory Management

brk - Change Data Segment Size

```

; C equivalent:
; int brk(void *addr);
; // Returns: 0 on success, -1 on error

mov rax, 12          ; sys_brk
mov rdi, 0           ; Query current break
syscall
; RAX = current program break (end of heap)

; Extend heap by 4096 bytes
add rax, 4096
mov rdi, rax
mov rax, 12
syscall

```

mmap - Memory Mapping

```

; C equivalent:
; void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
; // PROT_READ=1, PROT_WRITE=2, PROT_EXEC=4
; // MAP_PRIVATE=2, MAP_ANONYMOUS=0x20

mov rax, 9           ; sys_mmap
xor rdi, rdi         ; addr = NULL (kernel chooses)
mov rsi, 4096        ; length = 4096 bytes
mov rdx, 3           ; prot = PROT_READ | PROT_WRITE
mov r10, 0x22        ; flags = MAP_PRIVATE | MAP_ANONYMOUS
mov r8, -1           ; fd = -1 (not file-backed)
xor r9, r9           ; offset = 0
syscall
; RAX = address of mapped region, or -errno on error

```

munmap - Unmap Memory

```

; C equivalent:
; int munmap(void *addr, size_t length);

mov rax, 11          ; sys_munmap
mov rdi, [mapped_addr] ; Address to unmap
mov rsi, 4096        ; Length
syscall
; RAX = 0 on success

```

Time and Date

`time` - Get Current Time

```

; C equivalent:
; time_t time(time_t *tloc);

section .bss
    current_time resq 1

section .text
    mov rax, 201          ; sys_time
    lea rdi, [current_time] ; Output location (or 0 for RAX only)
    syscall
    ; RAX = seconds since epoch (1970-01-01 00:00:00 UTC)

```

`clock_gettime` - High-Resolution Time

```

; C equivalent:
; int clock_gettime(clockid_t clk_id, struct timespec *tp);
; // CLOCK_REALTIME = 0, CLOCK_MONOTONIC = 1

section .bss
    timespec:
        tv_sec resq 1      ; Seconds
        tv_nsec resq 1     ; Nanoseconds

section .text
    mov rax, 228           ; sys_clock_gettime
    mov rdi, 1             ; CLOCK_MONOTONIC
    lea rsi, [timespec]
    syscall
    ; RAX = 0 on success

```

nanosleep - Sleep for Duration

```

; C equivalent:
; int nanosleep(const struct timespec *req, struct timespec *rem);

section .data
    sleep_time:
        dq 1                ; 1 second
        dq 500000000        ; 500 million nanoseconds (0.5 sec)
                                ; Total: 1.5 seconds

section .text
    mov rax, 35              ; sys_nanosleep
    lea rdi, [sleep_time]    ; Requested time
    xor rsi, rsi              ; Remaining time (NULL)
    syscall
    ; RAX = 0 if completed, -1 if interrupted

```

Process Information

getpid - Get Process ID

```

; C equivalent:
; pid_t getpid(void);

mov rax, 39                  ; sys_getpid
syscall
; RAX = process ID

```

getuid - Get User ID

```

; C equivalent:
; uid_t getuid(void);

mov rax, 102                 ; sys_getuid
syscall
; RAX = user ID

```

uname - Get System Information

```

; C equivalent:
; int uname(struct utsname *buf);

section .bss
    utsname:
        sysname resb 65
        nodename resb 65
        release resb 65
        version resb 65
        machine resb 65
        domainname resb 65

section .text
    mov rax, 63          ; sys_uname
    lea rdi, [utsname]
    syscall
    ; RAX = 0 on success
    ; utsname now contains: "Linux", hostname, kernel version, etc.

```

Advanced I/O

ioctl - Device Control

```

; C equivalent:
; int ioctl(int fd, unsigned long request, ...);
; // Example: Get terminal size

section .bss
    winsize:
        ws_row resw 1
        ws_col resw 1
        ws_xpixel resw 1
        ws_ypixel resw 1

section .text
    mov rax, 16          ; sys_ioctl
    mov rdi, 1           ; stdout
    mov rsi, 0x5413      ; TIOCGWINSZ (get window size)
    lea rdx, [winsize]
    syscall
    ; winsize.ws_row and ws_col now contain terminal dimensions

```

select - I/O Multiplexing

```

; C equivalent:
; int select(int nfd, fd_set *readfds, fd_set *writefds,
;           fd_set *exceptfds, struct timeval *timeout);

section .bss
    readfds resb 128      ; fd_set structure

section .data
    timeout:
        dq 5              ; 5 seconds
        dq 0              ; 0 microseconds

section .text
    ; Setup: FD_ZERO and FD_SET for stdin (fd 0)
    lea rdi, [readfds]
    mov rcx, 16            ; 128 bytes / 8
    xor rax, rax
    rep stosq              ; Zero out fd_set

    mov byte [readfds], 1  ; Set bit 0 (stdin)

    ; Call select
    mov rax, 23            ; sys_select
    mov rdi, 1             ; nfd = highest fd + 1
    lea rsi, [readfds]     ; read fds
    xor rdx, rdx           ; write fds = NULL
    xor r10, r10           ; except fds = NULL
    lea r8, [timeout]      ; timeout
    syscall
    ; RAX = number of ready fds, 0 = timeout, -1 = error

```


Practical Example: Echo Server Skeleton

```

section .data
    bind_addr:
        sa_family dw 2           ; AF_INET
        sa_port dw 0x5000        ; Port 80 (network byte order)
        sa_addr dd 0             ; INADDR_ANY
        sa_zero times 8 db 0

    backlog equ 5

section .bss
    sockfd resq 1
    clientfd resq 1
    buffer resb 1024

section .text
global _start

_start:
    ; C equivalent:
    ; int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    ; bind(sockfd, (struct sockaddr*)&bind_addr, sizeof(bind_addr));
    ; listen(sockfd, backlog);
    ; while (1) {
    ;     int clientfd = accept(sockfd, NULL, NULL);
    ;     // Read and echo data
    ;     close(clientfd);
    ; }

    ; Create socket
    mov rax, 41           ; sys_socket
    mov rdi, 2            ; AF_INET
    mov rsi, 1            ; SOCK_STREAM
    xor rdx, rdx          ; protocol = 0
    syscall
    test rax, rax
    js exit_error
    mov [sockfd], rax

    ; Bind
    mov rax, 49           ; sys_bind
    mov rdi, [sockfd]
    lea rsi, [bind_addr]
    mov rdx, 16           ; addrlen
    syscall
    test rax, rax
    js exit_error

    ; Listen
    mov rax, 50           ; sys_listen
    mov rdi, [sockfd]

```

```

    mov rsi, backlog
    syscall
    test rax, rax
    js exit_error

accept_loop:
    ; Accept
    mov rax, 43          ; sys_accept
    mov rdi, [sockfd]
    xor rsi, rsi         ; addr = NULL
    xor rdx, rdx         ; addrlen = NULL
    syscall
    test rax, rax
    js accept_loop       ; Ignore errors, continue
    mov [clientfd], rax

    ; Read from client
    mov rax, 0           ; sys_read
    mov rdi, [clientfd]
    lea rsi, [buffer]
    mov rdx, 1024
    syscall
    test rax, rax
    jle close_client     ; EOF or error
    mov r12, rax         ; Save bytes read

    ; Echo back
    mov rax, 1           ; sys_write
    mov rdi, [clientfd]
    lea rsi, [buffer]
    mov rdx, r12         ; Bytes to write
    syscall

close_client:
    ; Close client connection
    mov rax, 3           ; sys_close
    mov rdi, [clientfd]
    syscall

    jmp accept_loop

exit_error:
    mov rax, 60
    mov rdi, 1
    syscall

```

Error Handling

Syscalls return negative errno values on error in RAX.

```

; C equivalent:
; int fd = open("file.txt", O_RDONLY);
; if (fd < 0) {
;     // Handle error
;     if (errno == ENOENT) { /* file not found */ }
; }

mov rax, 2          ; sys_open
lea rdi, [filename]
mov rsi, 0          ; O_RDONLY
syscall

test rax, rax
js handle_error     ; Jump if negative (error)

; Success path
mov [fd], rax
jmp continue

handle_error:
    neg rax          ; Make positive (errno value)
    cmp rax, 2       ; ENOENT = 2 (file not found)
    je file_not_found
    cmp rax, 13      ; EACCES = 13 (permission denied)
    je permission_denied
    ; ... other error cases

```

Common Error Codes:

Code	Name	Description
1	EPERM	Operation not permitted
2	ENOENT	No such file or directory
9	EBADF	Bad file descriptor
11	EAGAIN	Resource temporarily unavailable
12	ENOMEM	Out of memory
13	EACCES	Permission denied
14	EFAULT	Bad address
22	EINVAL	Invalid argument
32	EPIPE	Broken pipe

Performance Considerations

Syscall Overhead

- **Context switch:** ~100-1000 cycles (user → kernel → user)
- **Validation:** Kernel must validate all parameters
- **Batching:** Combine operations when possible

```

; C equivalent:
; // SLOW: Many syscalls
; for (int i = 0; i < 1000; ++i) {
;     write(fd, &data[i], 1); // 1000 syscalls!
; }
;
; // FAST: Single syscall
; write(fd, data, 1000); // 1 syscall

; Bad: 1000 separate write syscalls
mov rcx, 1000
write_loop:
    mov rax, 1
    mov rdi, [fd]
    lea rsi, [data + rcx - 1]
    mov rdx, 1
    syscall
    loop write_loop

; Good: Single write
mov rax, 1
mov rdi, [fd]
lea rsi, [data]
mov rdx, 1000
syscall

```

Avoiding Syscalls

Use buffering and batch operations:

```

; C equivalent:
; // Use buffered I/O
; FILE *f = fopen("file.txt", "w");
; for (int i = 0; i < 1000; ++i) {
;     fprintf(f, "%d\n", i); // Buffered internally
; }
; fclose(f); // Flush buffer

section .bss
    buffer resb 4096
    buf_pos resq 1

section .text
    ; Write to buffer, flush only when full
    ; (Implementation omitted for brevity)

```

Summary

Key Concepts

1. **Syscalls** are the interface to the kernel
2. Use `syscall` instruction (64-bit) or `int 0x80` (32-bit)
3. Arguments passed in **registers** (RAX, RDI, RSI, RDX, R10, R8, R9)
4. Return value in **RAX** (negative = error)
5. Syscalls are **expensive** - minimize when possible

Common Syscalls

- **File I/O**: read (0), write (1), open (2), close (3)
- **Process**: exit (60), fork (57), execve (59), wait4 (61)
- **Memory**: brk (12), mmap (9), munmap (11)
- **Time**: time (201), clock_gettime (228), nanosleep (35)
- **Network**: socket (41), bind (49), listen (50), accept (43)

Best Practices

1. Always check return values for errors
2. Use appropriate flags and permissions
3. Close file descriptors when done
4. Batch operations to reduce syscall count
5. Handle interrupted syscalls (EINTR)

Practice Exercises

Exercise 1: Cat Command

Implement a simple `cat` program that reads a file and writes to stdout.

Exercise 2: Copy Command

Implement `cp` that copies one file to another.

Exercise 3: Process Information

Write a program that prints its PID, UID, and current time.

Next Topic

In **Topic 17: Interfacing with C**, we'll learn how to call assembly from C and vice versa, link with C libraries, and use C's standard library functions from assembly.

Preview:

- Calling conventions review
- Linking with C code
- Using libc functions
- Creating reusable assembly libraries
- Inline assembly in C

CHAPTER 34

Topic 17: Interfacing with C

Overview

Assembly code rarely exists in isolation. Most real-world assembly programming involves interfacing with C code - either calling C functions from assembly or calling assembly routines from C. This topic covers how to bridge these two worlds.

```
// C calling assembly:
extern long my_asm_function(long a, long b);
int main() {
    long result = my_asm_function(10, 20);
    return 0;
}
```

```
; Assembly implementing function for C:
global my_asm_function
my_asm_function:
    mov rax, rdi        ; First argument
    add rax, rsi        ; Add second argument
    ret                ; Return RAX to C
```

Why Interface with C?

Advantages

1. **Access to C standard library:** printf, malloc, FILE I/O, etc.
2. **Portability:** C handles platform differences
3. **Development speed:** Use C for most code, assembly for hot paths
4. **Maintainability:** Mix high-level C with performance-critical assembly
5. **Ecosystem:** Link with existing C libraries

Common Use Cases

- Optimize performance-critical functions
- Access CPU-specific instructions (SIMD, crypto)
- Low-level hardware control
- Inline assembly in C for small snippets
- Creating reusable assembly libraries

Calling Conventions (Review)

System V AMD64 ABI (Linux, macOS, Unix)

Function Parameters:

Argument	Register	Type
1st	RDI	Integer/Pointer
2nd	RSI	Integer/Pointer
3rd	RDX	Integer/Pointer
4th	RCX	Integer/Pointer
5th	R8	Integer/Pointer
6th	R9	Integer/Pointer
7th+	Stack	Pushed right-to-left

Floating-Point Parameters:

Argument	Register
1st-8th	XMM0-XMM7
9th+	Stack

Return Values:

- Integer/Pointer: **RAX** (64-bit), **RDX:RAX** (128-bit)
- Floating-point: **XMM0** (float/double), **XMM1:XMM0** (long double)

Caller-Saved (Volatile): RAX, RCX, RDX, RSI, RDI, R8-R11, XMM0-XMM15

Callee-Saved (Non-Volatile): RBX, RBP, R12-R15

Stack Alignment: 16-byte aligned before `call` instruction

Assembly Function Called from C

Example 1: Simple Addition

```
; add.asm - Simple function to add two numbers

section .text
global add_numbers      ; Make function visible to C

; C prototype:
; long add_numbers(long a, long b);

add_numbers:
    ; RDI = a (first parameter)
    ; RSI = b (second parameter)

    mov rax, rdi          ; RAX = a
    add rax, rsi          ; RAX = a + b
    ret                  ; Return RAX to caller
```

```
// main.c - C code calling assembly function

#include <stdio.h>

// Declare external assembly function
extern long add_numbers(long a, long b);

int main() {
    long result = add_numbers(10, 20);
    printf("Result: %ld\n", result); // Output: Result: 30
    return 0;
}
```

Compile and Link:

```
nasm -f elf64 add.asm -o add.o
gcc main.c add.o -o program
./program
```

Example 2: Function with Multiple Parameters

```

; compute.asm - More complex calculation

section .text
global compute

; C prototype:
; long compute(long a, long b, long c, long d, long e, long f);
; Returns: (a + b) * (c - d) + e / f

compute:
    ; RDI = a, RSI = b, RDX = c, RCX = d, R8 = e, R9 = f

    push rbx                ; Save callee-saved register

    ; Compute a + b
    mov rax, rdi
    add rax, rsi            ; RAX = a + b

    ; Compute c - d
    mov rbx, rdx
    sub rbx, rcx           ; RBX = c - d

    ; Multiply (a + b) * (c - d)
    imul rax, rbx          ; RAX = (a + b) * (c - d)

    ; Save intermediate result
    mov rbx, rax

    ; Compute e / f
    mov rax, r8            ; RAX = e
    cqo                   ; Sign-extend for division
    idiv r9                ; RAX = e / f

    ; Add to intermediate result
    add rax, rbx           ; RAX = (a+b)*(c-d) + e/f

    pop rbx               ; Restore callee-saved register
    ret

```

```
// main.c

#include <stdio.h>

extern long compute(long a, long b, long c, long d, long e, long f);

int main() {
    // (10 + 20) * (50 - 30) + 100 / 10
    // = 30 * 20 + 10
    // = 600 + 10 = 610
    long result = compute(10, 20, 50, 30, 100, 10);
    printf("Result: %ld\n", result); // Output: Result: 610
    return 0;
}
```

Example 3: String Length (Working with Pointers)

```
; strlen.asm - Calculate string length

section .text
global asm_strlen

; C prototype:
; size_t asm_strlen(const char *str);

asm_strlen:
    ; RDI = str (pointer to null-terminated string)

    xor rax, rax          ; length = 0

.loop:
    cmp byte [rdi + rax], 0 ; Check for null terminator
    je .done
    inc rax                ; length++
    jmp .loop

.done:
    ret                  ; Return length in RAX
```

```
// main.c

#include <stdio.h>
#include <string.h>

extern size_t asm_strlen(const char *str);

int main() {
    const char *str = "Hello, Assembly!";

    size_t len1 = strlen(str);    // C standard library
    size_t len2 = asm_strlen(str); // Our assembly version

    printf("strlen:    %zu\n", len1);
    printf("asm_strlen: %zu\n", len2);

    return 0;
}
```

C Function Called from Assembly

Example 1: Calling printf

```
; hello.asm - Call printf from assembly

section .data
    format db "Hello, %s! The answer is %d", 10, 0
    name db "World", 0
    answer dd 42

section .text
extern printf           ; Declare external C function
global main            ; Entry point for C runtime

main:
    ; C equivalent:
    ; printf("Hello, %s! The answer is %d\n", "World", 42);

    push rbp            ; Set up stack frame
    mov rbp, rsp

    ; Prepare arguments for printf
    ; RDI = format string
    ; RSI = first %s argument
    ; RDX = first %d argument

    lea rdi, [format]   ; 1st arg: format string
    lea rsi, [name]     ; 2nd arg: "World"
    mov edx, [answer]   ; 3rd arg: 42

    xor rax, rax        ; RAX = 0 (no FP args in XMM registers)
    call printf

    ; Return 0 from main
    xor eax, eax

    leave               ; Restore stack frame
    ret
```

Compile:

```
nasm -f elf64 hello.asm -o hello.o
gcc hello.o -o hello -no-pie # -no-pie for simpler addressing
./hello
```

Important: Set `RAX` to the number of floating-point arguments passed in XMM registers (0 if none).

Example 2: Calling malloc and free

```

; memory.asm - Dynamic memory allocation

section .data
    format db "Allocated at: %p", 10, 0

section .text
extern malloc
extern free
extern printf
global main

main:
    push rbp
    mov rbp, rsp
    push rbx                ; Save callee-saved register

    ; C equivalent:
    ; void *ptr = malloc(1024);
    ; printf("Allocated at: %p\n", ptr);
    ; free(ptr);

    ; Allocate 1024 bytes
    mov rdi, 1024            ; Size argument
    call malloc
    mov rbx, rax             ; Save pointer in callee-saved RBX

    ; Print the address
    lea rdi, [format]
    mov rsi, rbx             ; Pointer as argument
    xor rax, rax
    call printf

    ; Free memory
    mov rdi, rbx
    call free

    ; Return 0
    xor eax, eax

    pop rbx
    leave
    ret

```


Example 3: Reading User Input

```
; input.asm - Read and echo user input

section .data
    prompt db "Enter your name: ", 0
    format db "Hello, %s!", 10, 0

section .bss
    buffer resb 100

section .text
extern printf
extern fgets
extern stdin
global main

main:
    push rbp
    mov rbp, rsp

    ; C equivalent:
    ; printf("Enter your name: ");
    ; fgets(buffer, sizeof(buffer), stdin);
    ; printf("Hello, %s!", buffer);

    ; Print prompt
    lea rdi, [prompt]
    xor rax, rax
    call printf

    ; Read input: char *fgets(char *s, int size, FILE *stream)
    lea rdi, [buffer]      ; Buffer
    mov rsi, 100           ; Size
    mov rdx, [stdin]       ; FILE *stdin (from libc)
    call fgets

    ; Print greeting
    lea rdi, [format]
    lea rsi, [buffer]
    xor rax, rax
    call printf

    xor eax, eax
    leave
    ret
```

Note: `stdin` is a global variable from libc, accessed via `[stdin]` (not `stdin` directly).

Stack Alignment

The System V ABI requires the stack to be **16-byte aligned** before a `call` instruction. Misalignment can cause crashes (especially with SSE instructions).

Problem Example

```
main:
    push rbp                ; RSP now misaligned (8-byte offset)
    mov rbp, rsp

    call printf             ; CRASH! Stack not 16-byte aligned
```

Solution 1: Dummy Push

```
main:
    push rbp                ; RSP -= 8 (misaligned)
    mov rbp, rsp
    push rax                ; Dummy push to realign RSP

    call printf             ; OK: stack 16-byte aligned

    leave
    ret
```

Solution 2: Sub RSP

```
main:
    push rbp
    mov rbp, rsp
    sub rsp, 8              ; Align stack

    call printf

    leave
    ret
```

Solution 3: Calculate Alignment

```
main:
    push rbp
    mov rbp, rsp

    ; Save registers we need
    push rbx
    push r12
    ; ... more pushes

    ; Align stack
    and rsp, -16      ; Round down to 16-byte boundary

    ; ... function calls ...

    leave
    ret
```

Passing Structures

Example: Returning Small Struct

```
// C code
typedef struct {
    long x;
    long y;
} Point;

Point create_point(long x, long y);
```

```
; point.asm

section .text
global create_point

; Small structs (≤16 bytes) returned in RAX, RDX

create_point:
    ; RDI = x, RSI = y
    mov rax, rdi      ; Return x in RAX
    mov rdx, rsi      ; Return y in RDX
    ret
```

```
// main.c

#include <stdio.h>

typedef struct {
    long x;
    long y;
} Point;

extern Point create_point(long x, long y);

int main() {
    Point p = create_point(10, 20);
    printf("Point: (%ld, %ld)\n", p.x, p.y);
    return 0;
}
```

Example: Passing Large Struct

```
// Large structs (>16 bytes) passed by pointer

typedef struct {
    long a, b, c, d;
} BigStruct;

long sum_struct(const BigStruct *s);
```

```
; struct.asm

section .text
global sum_struct

sum_struct:
    ; RDI = pointer to BigStruct

    mov rax, [rdi]          ; s->a
    add rax, [rdi + 8]      ; s->b
    add rax, [rdi + 16]     ; s->c
    add rax, [rdi + 24]     ; s->d
    ret
```

Inline Assembly in C

For small snippets, you can embed assembly directly in C using GCC's extended asm syntax.

Basic Syntax

```
asm("assembly code");
```

Extended Syntax

```
asm("assembly code"
    : output operands
    : input operands
    : clobbered registers);
```

Example 1: Read CPU Timestamp Counter

```
#include <stdio.h>
#include <stdint.h>

uint64_t rdtsc() {
    uint32_t low, high;
    asm volatile("rdtsc" : "=a"(low), "=d"(high));
    return ((uint64_t)high << 32) | low;
}

int main() {
    uint64_t start = rdtsc();

    // Do some work
    for (int i = 0; i < 1000000; ++i);

    uint64_t end = rdtsc();
    printf("Cycles: %lu\n", end - start);
    return 0;
}
```

Example 2: Fast Multiply

```
#include <stdio.h>

long fast_multiply(long a, long b) {
    long result;
    asm("imul %1, %2"
        : "=r"(result)      // Output: result = any register
        : "r"(a), "r"(b)    // Inputs: a and b in registers
        );
    return result;
}

int main() {
    printf("%ld\n", fast_multiply(10, 20)); // 200
    return 0;
}
```

Example 3: Atomic Operations

```
#include <stdio.h>

int atomic_increment(int *ptr) {
    int result;
    asm volatile(
        "lock xadd %0, %1"
        : "=r"(result), "+m"(*ptr)
        : "0"(1)
        : "memory"
    );
    return result;
}

int main() {
    int counter = 0;
    printf("Old: %d\n", atomic_increment(&counter)); // 0
    printf("New: %d\n", counter); // 1
    return 0;
}
```

Constraints:

- `"r"` - Any register
- `"a"`, `"b"`, `"c"`, `"d"` - RAX, RBX, RCX, RDX
- `"m"` - Memory operand
- `"i"` - Immediate constant
- `"=r"` - Output register (write-only)

- `"r"` - Input/output register (read/write)
- `"0"`, `"1"` - Same as operand 0, 1

Clobbers:

- `"memory"` - Memory was modified (prevents reordering)
- `"cc"` - Condition codes (flags) were modified

Creating a Reusable Library

Example: Math Library

mathlib.asm:

```

section .text

; long factorial(long n);
global factorial
factorial:
    cmp rdi, 1
    jle .base_case

    push rbx
    mov rbx, rdi        ; Save n
    dec rdi
    call factorial      ; factorial(n-1)
    imul rax, rbx        ; n * factorial(n-1)
    pop rbx
    ret

.base_case:
    mov rax, 1
    ret

; long power(long base, long exp);
global power
power:
    mov rax, 1          ; result = 1
    test rsi, rsi
    jz .done            ; exp == 0

.loop:
    imul rax, rdi        ; result *= base
    dec rsi
    jnz .loop

.done:
    ret

; long gcd(long a, long b);
global gcd
gcd:
    test rsi, rsi
    jz .done            ; b == 0, return a

    mov rax, rdi        ; rax = a
    xor rdx, rdx
    div rsi              ; rdx = a % b

    mov rdi, rsi        ; a = b
    mov rsi, rdx        ; b = remainder
    jmp gcd             ; tail recursion

.done:

```



```
mov rax, rdi
ret
```

mathlib.h:

```
#ifndef MATHLIB_H
#define MATHLIB_H

long factorial(long n);
long power(long base, long exp);
long gcd(long a, long b);

#endif
```

test.c:

```
#include <stdio.h>
#include "mathlib.h"

int main() {
    printf("factorial(5) = %ld\n", factorial(5));    // 120
    printf("power(2, 10) = %ld\n", power(2, 10));   // 1024
    printf("gcd(48, 18) = %ld\n", gcd(48, 18));     // 6
    return 0;
}
```

Build:

```
nasm -f elf64 mathlib.asm -o mathlib.o
gcc -c test.c -o test.o
gcc test.o mathlib.o -o test
./test
```

Create Static Library:

```
nasm -f elf64 mathlib.asm -o mathlib.o
ar rcs libmathlib.a mathlib.o
gcc test.c -L. -lmathlib -o test
```

Debugging Mixed C/Assembly

Using GDB

```
# Compile with debug symbols
nasm -f elf64 -g -F dwarf mathlib.asm -o mathlib.o
gcc -g test.c mathlib.o -o test

# Debug
gdb ./test
```

GDB Commands:

```
(gdb) break main
(gdb) run
(gdb) stepi           # Step one instruction
(gdb) info registers  # Show all registers
(gdb) x/10i $rip      # Disassemble next 10 instructions
(gdb) layout asm      # Show assembly in TUI mode
(gdb) layout regs      # Show registers in TUI mode
```

Printing from Assembly (Debug)

```

section .data
    debug_fmt db "Debug: RAX=%ld, RBX=%ld", 10, 0

section .text
extern printf

; ... in your function ...
push rax
push rbx

; Save registers that printf uses
push rdi
push rsi
push rdx

lea rdi, [debug_fmt]
mov rsi, rax
mov rdx, rbx
xor rax, rax
call printf

pop rdx
pop rsi
pop rdi
pop rbx
pop rax

```

Common Pitfalls

1. Stack Misalignment

```

// WRONG: Misaligned stack
void asm_func() {
    asm("call printf"); // May crash!
}

// CORRECT: Ensure alignment
void asm_func() {
    asm("sub $8, %%rsp"); // Align
    asm("call printf");
    asm("add $8, %%rsp"); // Restore
}

```

2. Not Saving Callee-Saved Registers

```
; WRONG: Corrupting RBX
my_func:
    mov rbx, rdi        ; Use RBX
    ; ... use rbx ...
    ret                ; BUG: didn't restore RBX!

; CORRECT:
my_func:
    push rbx
    mov rbx, rdi
    ; ... use rbx ...
    pop rbx
    ret
```

3. Forgetting RAX for varargs

```
; WRONG: Calling printf with float
mov rdi, format
movsd xmm0, [value]
call printf            ; BUG: RAX not set!

; CORRECT:
mov rdi, format
movsd xmm0, [value]
mov rax, 1             ; 1 floating-point arg in XMM
call printf
```

4. Wrong Parameter Order

```
// C: long subtract(long a, long b) { return a - b; }
```

```
; WRONG: Swapped parameters
subtract:
    mov rax, rsi        ; rsi is b, not a!
    sub rax, rdi
    ret

; CORRECT:
subtract:
    mov rax, rdi        ; First param (a) in RDI
    sub rax, rsi        ; Subtract second param (b)
    ret
```

Summary

Key Concepts

1. **System V ABI:** Parameters in RDI, RSI, RDX, RCX, R8, R9; return in RAX
2. **Stack alignment:** Must be 16-byte aligned before `call`
3. **Callee-saved:** Must preserve RBX, RBP, R12-R15
4. **Caller-saved:** RAX, RCX, RDX, RSI, RDI, R8-R11 can be clobbered
5. **Small structs:** Returned in RAX:RDX
6. **Large structs:** Passed/returned by pointer

Workflow

1. Write assembly with `global` for exported functions
2. Declare `extern` for C functions you call
3. Use proper calling convention
4. Compile: `nasm -f elf64 file.asm -o file.o`
5. Link: `gcc main.c file.o -o program`

Best Practices

1. Always preserve callee-saved registers
2. Align stack before calling C functions
3. Set RAX for varargs functions
4. Use inline asm for small snippets
5. Create libraries for reusable assembly code
6. Test thoroughly - calling convention bugs are subtle

Practice Exercises

Exercise 1: Implement memcpy

Write an assembly function with C prototype:

```
void *memcpy(void *dest, const void *src, size_t n);
```

Exercise 2: Binary Search

Implement binary search in assembly, callable from C:

```
int binary_search(const int *array, int size, int target);
```

Exercise 3: SIMD Sum

Use SSE to sum an array of floats:

```
float sum_array(const float *array, int count);
```

Next Topic

In **Topic 18: SIMD Instructions**, we'll explore Single Instruction Multiple Data operations using SSE and AVX, enabling massive parallelism for data processing.

Preview:

- SSE/AVX registers (XMM, YMM, ZMM)
- Packed operations on multiple data elements
- Vectorized array processing
- Common SIMD patterns
- Performance gains from SIMD

CHAPTER 35

Topic 18: SIMD Instructions (SSE/AVX)

Overview

SIMD (Single Instruction, Multiple Data) allows processing multiple data elements simultaneously with a single instruction. Modern x86-64 processors support several SIMD instruction sets: SSE, SSE2, SSE3, SSSE3, SSE4, AVX, AVX2, and AVX-512.

```
// C equivalent - scalar processing:
for (int i = 0; i < 4; ++i) {
    result[i] = a[i] + b[i]; // 4 separate additions
}

// SIMD - vectorized processing:
// One instruction processes 4 floats at once!
// result_vector = a_vector + b_vector;
```

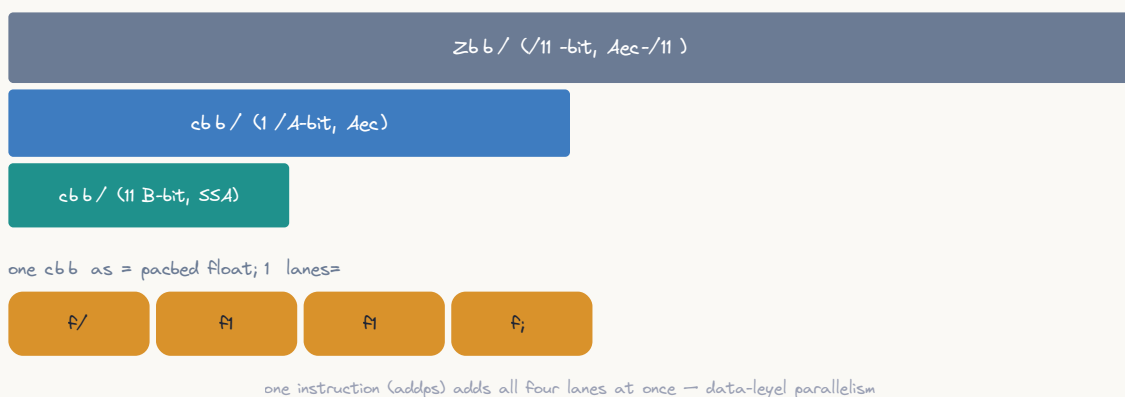
Performance Gain: Up to **4x-16x** speedup for compatible workloads!

SIMD Registers

XMM Registers (SSE): 128-bit

16 registers: **XMM0-XMM15**

SubD registers = cbb $\subseteq$ cbb $\subseteq$ Zbb



ASCII diagram

```

XMM0: [127 ----- 64][63 ----- 0]
      | 64-bit   || 64-bit   |
      | 4x float   || 2x double  |
      | 4x int32    || 2x int64   |

```

Data Layouts:

Type	Count	Each Element
float	4	32-bit single-precision
double	2	64-bit double-precision
int8	16	8-bit integer
int16	8	16-bit integer
int32	4	32-bit integer
int64	2	64-bit integer

YMM Registers (AVX): 256-bit

16 registers: **YMM0-YMM15** (extend XMM0-XMM15)

```

YMM0: [255----192][191----128][127----64][63-----0]
      | 64-bit || 64-bit || 64-bit || 64-bit |
      |      8x float      || 4x double      |

```

ZMM Registers (AVX-512): 512-bit

32 registers: **ZMM0-ZMM31** (extend YMM0-YMM31)

Note: We'll focus on SSE and AVX, as they're most widely supported.

SSE Instructions

Moving Data

```

; C equivalent:
; float a[4] = {1.0, 2.0, 3.0, 4.0};
; float b[4];
; memcpy(b, a, sizeof(a));

section .data
    align 16
    floats dd 1.0, 2.0, 3.0, 4.0

section .bss
    align 16
    result resd 4

section .text
    ; Load 4 floats at once
    movaps xmm0, [floats] ; Aligned load (faster)

    ; Store 4 floats at once
    movaps [result], xmm0 ; Aligned store

    ; Unaligned variants (slower but flexible)
    movups xmm1, [floats] ; Unaligned load
    movups [result], xmm1 ; Unaligned store

```

Alignment: SSE requires **16-byte alignment** for best performance. Use `align 16` directive.

Arithmetic Operations

Packed Single-Precision (PS): 4x float

```

; C equivalent:
; for (int i = 0; i < 4; ++i) {
;     c[i] = a[i] + b[i];
;     d[i] = a[i] - b[i];
;     e[i] = a[i] * b[i];
;     f[i] = a[i] / b[i];
; }

section .data
    align 16
    a dd 1.0, 2.0, 3.0, 4.0
    b dd 5.0, 6.0, 7.0, 8.0

section .bss
    align 16
    c resd 4
    d resd 4
    e resd 4
    f resd 4

section .text
    movaps xmm0, [a]          ; Load a
    movaps xmm1, [b]          ; Load b

    movaps xmm2, xmm0
    addps xmm2, xmm1          ; xmm2 = a + b (4 additions)
    movaps [c], xmm2

    movaps xmm2, xmm0
    subps xmm2, xmm1          ; xmm2 = a - b (4 subtractions)
    movaps [d], xmm2

    movaps xmm2, xmm0
    mulps xmm2, xmm1          ; xmm2 = a * b (4 multiplications)
    movaps [e], xmm2

    movaps xmm2, xmm0
    divps xmm2, xmm1          ; xmm2 = a / b (4 divisions)
    movaps [f], xmm2

```

Packed Double-Precision (PD): 2x double

```

; C equivalent:
; for (int i = 0; i < 2; ++i) {
;     c[i] = a[i] + b[i];
; }

section .data
    align 16
    a dq 1.5, 2.5
    b dq 3.5, 4.5

section .text
    movapd xmm0, [a]          ; Load 2 doubles
    movapd xmm1, [b]
    addpd  xmm0, xmm1         ; Add 2 doubles at once

```

Scalar Operations (SS/SD): Single element

```

; C equivalent:
; float result = a + b;

section .data
    a dd 1.5
    b dd 2.5

section .text
    movss xmm0, [a]          ; Load single float
    addss  xmm0, [b]          ; Add single float
    ; xmm0[31:0] = result, xmm0[127:32] unchanged

```

Comparison Operations

```

; C equivalent:
; for (int i = 0; i < 4; ++i) {
;     mask[i] = (a[i] < b[i]) ? 0xFFFFFFFF : 0x00000000;
; }

movaps xmm0, [a]
movaps xmm1, [b]
cmpltps xmm0, xmm1          ; Compare less-than
; xmm0[i] = 0xFFFFFFFF if a[i] < b[i], else 0x00000000

```

Comparison Predicates:

Instruction	Condition	Description
<code>cmpeqps</code>	<code>==</code>	Equal
<code>cmpltps</code>	<code><</code>	Less than
<code>cmpleps</code>	<code><=</code>	Less or equal
<code>cmpneqps</code>	<code>!=</code>	Not equal
<code>cmpnltps</code>	<code>>=</code>	Not less than
<code>cmpnleps</code>	<code>></code>	Not less or equal

Logical Operations

```
; C equivalent (bitwise):
; for (int i = 0; i < 4; ++i) {
;     result[i] = a[i] & b[i]; // Bitwise AND
; }
```

```
movaps xmm0, [a]
andps  xmm0, [b]           ; Bitwise AND
orps   xmm1, [c]           ; Bitwise OR
xorps  xmm2, [d]           ; Bitwise XOR
andnps xmm3, [e]           ; Bitwise AND NOT
```

Practical Example: Array Addition

```

; add_arrays.asm - Add two float arrays using SSE

section .text
global add_arrays_simd

; C prototype:
; void add_arrays_simd(float *result, const float *a, const float *b, size_t count);
; Assumes count is multiple of 4, arrays are 16-byte aligned

add_arrays_simd:
    ; RDI = result, RSI = a, RDX = b, RCX = count

    xor rax, rax          ; i = 0

.loop:
    cmp rax, rcx
    jge .done

    ; Load 4 floats from a
    movaps xmm0, [rsi + rax*4]

    ; Add 4 floats from b
    addps xmm0, [rdx + rax*4]

    ; Store 4 results
    movaps [rdi + rax*4], xmm0

    add rax, 4             ; i += 4 (processed 4 elements)
    jmp .loop

.done:
    ret

```

```

// test.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern void add_arrays_simd(float *result, const float *a, const float *b, size_t count);

// Scalar version for comparison
void add_arrays_scalar(float *result, const float *a, const float *b, size_t count) {
    for (size_t i = 0; i < count; ++i) {
        result[i] = a[i] + b[i];
    }
}

int main() {
    const size_t count = 8;

    // Allocate aligned memory
    float *a = aligned_alloc(16, count * sizeof(float));
    float *b = aligned_alloc(16, count * sizeof(float));
    float *result = aligned_alloc(16, count * sizeof(float));

    // Initialize
    for (size_t i = 0; i < count; ++i) {
        a[i] = i + 1.0f;
        b[i] = (i + 1.0f) * 10.0f;
    }

    // SIMD version
    add_arrays_simd(result, a, b, count);

    printf("SIMD results:\n");
    for (size_t i = 0; i < count; ++i) {
        printf("%.1f ", result[i]);
    }
    printf("\n");

    free(a);
    free(b);
    free(result);
    return 0;
}

```

Advanced SIMD Techniques

Horizontal Operations

```

; C equivalent:
; float sum = a[0] + a[1] + a[2] + a[3];

section .data
    align 16
    values dd 1.0, 2.0, 3.0, 4.0

section .text
    movaps xmm0, [values]    ; [1, 2, 3, 4]

    ; Method 1: Hadd (SSE3)
    haddps xmm0, xmm0        ; [1+2, 3+4, 1+2, 3+4] = [3, 7, 3, 7]
    haddps xmm0, xmm0        ; [3+7, 3+7, 3+7, 3+7] = [10, 10, 10, 10]
    ; xmm0[0] now contains sum

    ; Method 2: Manual shuffling (faster)
    movaps xmm1, xmm0        ; Copy
    shufps xmm1, xmm1, 0xB1   ; Swap adjacent pairs
    addps  xmm0, xmm1         ; Add pairs
    movaps xmm1, xmm0
    shufps xmm1, xmm1, 0x4E   ; Swap high/low halves
    addps  xmm0, xmm1         ; Final sum in all elements

```

Complete Array Sum Example


```

; sum_array.asm - Sum array of floats using SSE

section .text
global sum_array_simd

; C prototype:
; float sum_array_simd(const float *array, size_t count);

sum_array_simd:
    ; RDI = array, RSI = count

    xorps xmm0, xmm0      ; accumulator = 0
    xor rax, rax           ; i = 0

    ; Process 4 elements at a time
    mov rcx, rsi
    shr rcx, 2             ; count / 4
    jz .remainder

.loop:
    addps xmm0, [rdi + rax*4] ; Add 4 floats to accumulator
    add rax, 4
    dec rcx
    jnz .loop

.remainder:
    ; Handle leftover elements (count % 4)
    mov rcx, rsi
    and rcx, 3
    jz .horizontal_sum

.remainder_loop:
    addss xmm0, [rdi + rax*4]
    inc rax
    dec rcx
    jnz .remainder_loop

.horizontal_sum:
    ; Sum the 4 values in xmm0 into xmm0[0]
    movaps xmm1, xmm0
    shufps xmm1, xmm1, 0xB1      ; Swap adjacent
    addps xmm0, xmm1
    movaps xmm1, xmm0
    shufps xmm1, xmm1, 0x4E      ; Swap high/low
    addps xmm0, xmm1

    ; Result in xmm0[0] (XMM0 is return register for float)
    ret

```

Shuffling and Permutation

```
; C equivalent:
; float result[4] = {a[2], a[0], a[3], a[1]};

movaps xmm0, [a]           ; [a0, a1, a2, a3]
shufps xmm0, xmm0, 0x9C    ; Shuffle: [a2, a0, a3, a1]
; Immediate 0x9C = 10 01 11 00 (binary)
;                a2 a0 a3 a1 (indices)
```

Shuffle Control Byte:

- Bits [1:0]: Select from source for element 0
- Bits [3:2]: Select from source for element 1
- Bits [5:4]: Select from source for element 2
- Bits [7:6]: Select from source for element 3

Broadcast (Splat)

```
; C equivalent:
; float result[4] = {x, x, x, x};

section .data
    x dd 5.0

section .text
    movss xmm0, [x]        ; Load single value
    shufps xmm0, xmm0, 0    ; Broadcast to all elements
    ; xmm0 = [5.0, 5.0, 5.0, 5.0]
```

AVX Instructions

AVX extends SSE with 256-bit operations and non-destructive operations.

Non-Destructive Operations

```
; SSE (destructive):
movaps xmm0, [a]
addps xmm0, [b]           ; xmm0 = xmm0 + [b] (xmm0 overwritten)

; AVX (non-destructive):
vmovaps ymm0, [a]
vaddps ymm2, ymm0, [b]    ; ymm2 = ymm0 + [b] (ymm0 preserved!)
```

256-bit Operations

```

; C equivalent:
; for (int i = 0; i < 8; ++i) {
;     result[i] = a[i] + b[i]; // 8 floats at once!
; }

section .data
    align 32                ; AVX requires 32-byte alignment
    a dd 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0
    b dd 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0

section .bss
    align 32
    result resd 8

section .text
    vmovaps ymm0, [a]        ; Load 8 floats
    vaddps ymm0, ymm0, [b]    ; Add 8 floats
    vmovaps [result], ymm0    ; Store 8 floats

    ; IMPORTANT: Clean up AVX state
    vzeroupper                ; Clear upper 128 bits of YMM registers
    ret

```

Note: Always use `vzeroupper` or `vzeroall` before transitioning back to non-AVX code to avoid performance penalties.

Complete Real-World Example: Dot Product

```

; dot_product.asm - Compute dot product using SSE

section .text
global dot_product_simd

; C prototype:
; float dot_product_simd(const float *a, const float *b, size_t count);
; Assumes count is multiple of 4

dot_product_simd:
    ; RDI = a, RSI = b, RDX = count

    xorps xmm0, xmm0      ; sum = 0
    xor rax, rax           ; i = 0

.loop:
    cmp rax, rdx
    jge .horizontal_sum

    ; Load 4 elements from each array
    movaps xmm1, [rdi + rax*4]
    movaps xmm2, [rsi + rax*4]

    ; Multiply element-wise
    mulps xmm1, xmm2

    ; Add to accumulator
    addps xmm0, xmm1

    add rax, 4
    jmp .loop

.horizontal_sum:
    ; Sum the 4 partial sums in xmm0
    movaps xmm1, xmm0
    shufps xmm1, xmm1, 0xB1      ; [1, 0, 3, 2]
    addps xmm0, xmm1             ; [0+1, 1+0, 2+3, 3+2]

    movaps xmm1, xmm0
    shufps xmm1, xmm1, 0x4E      ; [2, 3, 0, 1]
    addps xmm0, xmm1             ; [sum, sum, sum, sum]

    ; Result in xmm0[0]
    ret

```

```

// test_dot.c

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

extern float dot_product_simd(const float *a, const float *b, size_t count);

float dot_product_scalar(const float *a, const float *b, size_t count) {
    float sum = 0.0f;
    for (size_t i = 0; i < count; ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}

int main() {
    const size_t count = 1024 * 1024; // 1 million elements

    float *a = aligned_alloc(16, count * sizeof(float));
    float *b = aligned_alloc(16, count * sizeof(float));

    // Initialize
    for (size_t i = 0; i < count; ++i) {
        a[i] = 1.0f;
        b[i] = 2.0f;
    }

    // Benchmark scalar
    clock_t start = clock();
    float result_scalar = dot_product_scalar(a, b, count);
    clock_t end = clock();
    double time_scalar = (double)(end - start) / CLOCKS_PER_SEC;

    // Benchmark SIMD
    start = clock();
    float result_simd = dot_product_simd(a, b, count);
    end = clock();
    double time_simd = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Scalar: %.2f (%.6f seconds)\n", result_scalar, time_scalar);
    printf("SIMD:   %.2f (%.6f seconds)\n", result_simd, time_simd);
    printf("Speedup: %.2fx\n", time_scalar / time_simd);

    free(a);
    free(b);
    return 0;
}

```

Integer SIMD Operations

```

; C equivalent:
; for (int i = 0; i < 4; ++i) {
;     result[i] = a[i] + b[i]; // 32-bit integers
; }

section .data
    align 16
    a dd 1, 2, 3, 4
    b dd 5, 6, 7, 8

section .bss
    align 16
    result resd 4

section .text
    movdqa xmm0, [a]      ; Load 4 int32s (aligned)
    paddb xmm0, [b]      ; Add 4 int32s
    movdqa [result], xmm0 ; Store 4 int32s

```

Integer Operations:

Instruction	Description
<code>paddb/w/d/q</code>	Add packed bytes/words/dwords/qwords
<code>psubb/w/d/q</code>	Subtract
<code>pmullw/d</code>	Multiply low (words/dwords)
<code>pmulhw/uw</code>	Multiply high (signed/unsigned words)
<code>pand/or/pxor</code>	Logical AND/OR/XOR
<code>psllw/d/q</code>	Shift left logical
<code>psrlw/d/q</code>	Shift right logical
<code>psraw/d</code>	Shift right arithmetic
<code>pcmpeqb/w/d</code>	Compare equal
<code>pcmpgtb/w/d</code>	Compare greater than

Performance Considerations

When to Use SIMD

Good candidates:

- Array operations (add, multiply, etc.)
- Vector math (graphics, physics)
- Signal processing (audio, video)
- Matrix operations
- Image processing
- Data parallel algorithms

Poor candidates:

- Irregular control flow (many branches)
- Data dependencies between iterations
- Non-contiguous memory access
- Small datasets (overhead dominates)

Alignment

```
; Aligned (fast):
section .data
    align 16
    data dd 1.0, 2.0, 3.0, 4.0

movaps xmm0, [data]    ; Fast aligned load

; Unaligned (slower):
movups xmm0, [data]    ; Slower unaligned load
```

Best Practice: Always align data to 16/32 bytes when possible.

Typical Speedups

Operation	SSE Speedup	AVX Speedup
Float add/sub	3-4x	6-8x
Float multiply	3-4x	6-8x
Float divide	2-3x	4-6x
Int32 add	3-4x	6-8x
Dot product	3-4x	6-8x

Actual speedup depends on:

- Memory bandwidth
- Data alignment
- Loop overhead
- CPU architecture

Common Patterns

Pattern 1: Conditional Selection

```

; C equivalent:
; for (int i = 0; i < 4; ++i) {
;     result[i] = (a[i] > b[i]) ? a[i] : b[i]; // Max
; }

movaps xmm0, [a]
maxps xmm0, [b]          ; Per-element maximum
movaps [result], xmm0

; Manual with blending:
movaps xmm0, [a]
movaps xmm1, [b]
cmpgtps xmm2, xmm0, xmm1  ; mask = (a > b)
andps xmm0, xmm2          ; a & mask
andnps xmm2, xmm1         ; b & ~mask
orps xmm0, xmm2           ; result = (a & mask) | (b & ~mask)

```


Pattern 2: AoS to SoA Conversion

```
// Array of Structures (AoS) - cache unfriendly for SIMD
struct Point { float x, y, z; };
Point points[1000];

// Structure of Arrays (SoA) - SIMD friendly
float x[1000], y[1000], z[1000];
```

```
; Process SoA data with SIMD
movaps xmm0, [x]      ; Load 4 x values
movaps xmm1, [y]      ; Load 4 y values
movaps xmm2, [z]      ; Load 4 z values
; Process 4 points in parallel!
```

Pattern 3: Fused Multiply-Add (FMA)

```
; C equivalent:
; for (int i = 0; i < 4; ++i) {
;     result[i] = a[i] * b[i] + c[i];
; }

; Without FMA (SSE):
movaps xmm0, [a]
mulps xmm0, [b]
addps xmm0, [c]

; With FMA (AVX2+):
vmovaps ymm0, [a]
vfmadd231ps ymm0, [b], [c] ; ymm0 = (b * c) + ymm0
```

Summary

Key Concepts

1. **SIMD**: Process multiple data elements with one instruction
2. **SSE**: 128-bit (4x float, 2x double, 4x int32)
3. **AVX**: 256-bit (8x float, 4x double, 8x int32)
4. **Alignment**: 16-byte for SSE, 32-byte for AVX
5. **Speedup**: Typically 3-8x for vectorizable code

Common Instructions

- **Load/Store**: `movaps`, `movups`, `movdqa`, `movdqu`

- **Arithmetic:** `addps`, `subps`, `mulps`, `divps`
- **Comparison:** `cmpltps`, `cmpeqps`, etc.
- **Logical:** `andps`, `orps`, `xorps`
- **Shuffle:** `shufps`, `unpcklps`, `unpckhps`

Best Practices

1. Align data to 16/32 bytes
2. Process 4/8 elements at a time
3. Handle remainder elements separately
4. Use non-destructive AVX when possible
5. Call `vzeroupper` after AVX code
6. Profile to verify actual speedup

Limitations

1. Requires contiguous memory
2. Benefits diminish with branching
3. Data dependencies reduce parallelism
4. Alignment requirements
5. CPU feature detection needed

Practice Exercises

Exercise 1: Max Element

Implement a function to find the maximum element in a float array using SSE.

Exercise 2: Vector Normalization

Normalize a 3D vector array (x, y, z) using SIMD.

Exercise 3: Matrix Multiplication

Implement 4x4 matrix multiplication using SSE.

Next Topic

In **Topic 19: Performance & Optimization**, we'll explore advanced optimization techniques, profiling, and how to write the fastest possible assembly code.

Preview:

- CPU pipeline and instruction-level parallelism
- Cache optimization
- Branch prediction
- Loop unrolling
- Register allocation strategies
- Profiling and benchmarking

CHAPTER 36

Topic 19: Performance and Optimization

Overview

Writing correct assembly is one thing; writing *fast* assembly is another. This topic covers techniques to maximize performance by understanding CPU architecture, instruction timing, memory hierarchy, and compiler-level optimizations.

```
// Correct but slow:
for (int i = 0; i < n; ++i) {
    result += array[i]; // Memory access every iteration
}

// Fast:
int temp = 0;
for (int i = 0; i < n; ++i) {
    temp += array[i]; // Register accumulation
}
result = temp;
```

CPU Architecture Fundamentals

Pipeline Stages

Modern CPUs use instruction pipelining to execute multiple instructions simultaneously:

Cycle:	1	2	3	4	5	6	7	8
Inst1:	F	D	E	M	W			
Inst2:		F	D	E	M	W		
Inst3:			F	D	E	M	W	
Inst4:				F	D	E	M	W

F = Fetch, D = Decode, E = Execute, M = Memory, W = Write-back

Throughput: Up to 1 instruction per cycle (ideal)

Latency: Individual instruction takes multiple cycles

Superscalar Execution

Modern CPUs can execute multiple instructions per cycle:

```

Port 0: ALU, FPU
Port 1: ALU, FPU
Port 2: Load (Memory Read)
Port 3: Load (Memory Read)
Port 4: Store (Memory Write)
Port 5: ALU, Branch

```

Example: These can execute in parallel:

```

mov rax, [data]      ; Port 2 (load)
add rbx, rcx         ; Port 0 or 1 (ALU)
imul rdx, rsi        ; Port 1 (multiply)

```

Instruction-Level Parallelism (ILP): CPU finds independent instructions and executes them simultaneously.

Instruction Timing

Latency vs Throughput

Latency: Cycles until result is available

Throughput: How often instruction can be issued

Example (typical Intel/AMD):

Instruction	Latency	Throughput	Ports
<code>mov r, r</code>	0-1	0.25/cycle	0,1,5,6
<code>add r, r</code>	1	0.25/cycle	0,1,5,6
<code>imul r, r</code>	3	1/cycle	1
<code>div r64</code>	36-95	21-74/cycle	0
<code>movaps xmm, m</code>	5-7	0.5/cycle	2,3
<code>addps xmm, xmm</code>	3-4	0.5/cycle	0,1

Key Insight: Focus on **throughput** for loops, **latency** for dependency chains.

Dependency Chains

```

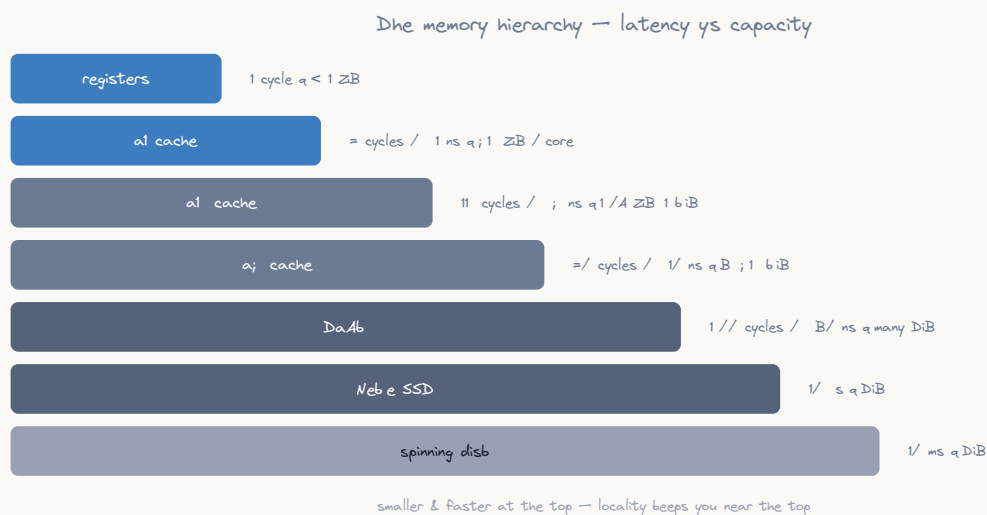
; C equivalent:
; int a = 1;
; a = a + 1; // Depends on previous
; a = a + 1; // Depends on previous
; a = a + 1; // Depends on previous

; BAD: Serial dependency chain (latency = 3 cycles)
mov eax, 1
add eax, 1      ; Waits for previous
add eax, 1      ; Waits for previous
add eax, 1      ; Waits for previous
; Total: ~3 cycles

; GOOD: Independent operations (throughput = 1 cycle)
mov eax, 1
mov ebx, 1
mov ecx, 1
add eax, 1      ; All execute in parallel
add ebx, 1
add ecx, 1
; Total: ~1 cycle

```

Memory Hierarchy



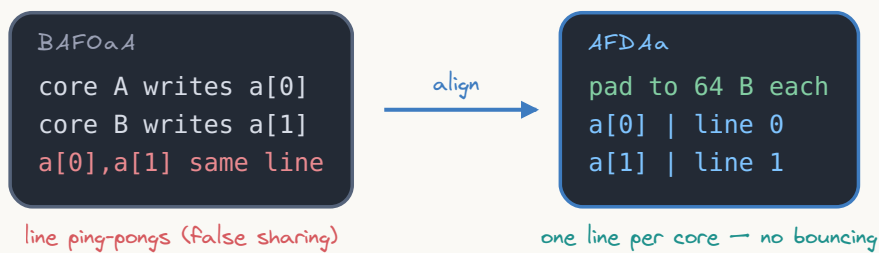
Cache Levels

CPU Registers:	< 1 cycle	~1 KB
L1 Cache:	4-5 cycles	32-64 KB per core
L2 Cache:	12-15 cycles	256-512 KB per core
L3 Cache:	40-75 cycles	4-32 MB shared
RAM:	200+ cycles	Gigabytes

Performance Impact: Cache miss can be **50-200x slower** than cache hit!

Cache Lines

Cache line = 4= bytes (the unit of coherence)



- **Size:** 64 bytes (typical)
- **Prefetching:** CPU loads entire cache line, not just requested byte
- **Alignment:** Misaligned access may span two cache lines

```
; C equivalent:
; struct Data { int value; char padding[60]; }; // 64 bytes
; Data array[100]; // Each element = one cache line

section .bss
    align 64
    data resb 6400      ; 100 x 64-byte structures

; Access aligned to cache line - FAST
mov eax, [data]
mov eax, [data + 64]
mov eax, [data + 128]

; Misaligned - may span cache lines - SLOWER
mov eax, [data + 1]
```

False Sharing

```
// BAD: Two threads writing to same cache line
struct {
    int counter1; // Thread 1 writes here
    int counter2; // Thread 2 writes here (same cache line!)
} shared;

// GOOD: Separate cache lines
struct {
    int counter1;
    char padding[60];
    int counter2;
} shared;
```

Optimization Techniques

1. Loop Unrolling

Process multiple iterations in one loop iteration to reduce overhead.


```

; C equivalent:
; int sum = 0;
; for (int i = 0; i < n; ++i) {
;     sum += array[i];
; }

; Original loop
xor eax, eax          ; sum = 0
xor ecx, ecx          ; i = 0
loop_original:
    add eax, [array + rcx*4]
    inc ecx
    cmp ecx, [n]
    jl loop_original
; Overhead: cmp, jl every iteration

; Unrolled by 4
xor eax, eax
xor ecx, ecx
mov edx, [n]
shr edx, 2             ; n / 4
loop_unrolled:
    add eax, [array + rcx*4]
    add eax, [array + rcx*4 + 4]
    add eax, [array + rcx*4 + 8]
    add eax, [array + rcx*4 + 12]
    add ecx, 4
    dec edx
    jnz loop_unrolled
; Overhead: dec, jnz every 4 iterations (4x less!)

```

Benefits:

- Reduced branch overhead
- More instruction-level parallelism
- Better CPU utilization

Drawback: Increased code size

2. Multiple Accumulators

Break dependency chains by using multiple independent accumulators.

```

; C equivalent:
; int sum = 0;
; for (int i = 0; i < n; ++i) {
;     sum += array[i];
; }

; Single accumulator (dependency chain)
xor eax, eax
xor ecx, ecx
loop_single:
    add eax, [array + rcx*4]    ; Each add depends on previous
    inc ecx
    cmp ecx, [n]
    jl loop_single
; Latency-bound!

; Four accumulators (parallel)
xor eax, eax    ; sum0
xor ebx, ebx    ; sum1
xor edx, edx    ; sum2
xor esi, esi    ; sum3
xor ecx, ecx
loop_parallel:
    add eax, [array + rcx*4]
    add ebx, [array + rcx*4 + 4]
    add edx, [array + rcx*4 + 8]
    add esi, [array + rcx*4 + 12]
    add ecx, 4
    cmp ecx, [n]
    jl loop_parallel

; Combine accumulators
add eax, ebx
add edx, esi
add eax, edx
; Much faster! Throughput-bound instead of latency-bound

```

3. Software Pipelining

Overlap operations from different iterations.

```

; C equivalent:
; for (int i = 0; i < n; ++i) {
;     int temp = expensive_operation(array[i]);
;     result[i] = process(temp);
; }

; Original (serial)
loop_original:
    mov eax, [array + rcx*4]
    ; expensive_operation (10 cycles latency)
    imul eax, eax
    imul eax, eax
    ; process
    add eax, 1
    mov [result + rcx*4], eax
    inc rcx
    cmp ecx, [n]
    jl loop_original

; Pipelined (overlap operations)
mov eax, [array]          ; Preload first
imul eax, eax
loop_pipelined:
    imul eax, eax          ; Finish expensive op for i
    mov ebx, [array + rcx*4 + 4] ; Start load for i+1
    add eax, 1             ; Process for i
    imul ebx, ebx          ; Start expensive op for i+1
    mov [result + rcx*4], eax
    mov eax, ebx           ; Move i+1 result to working register
    inc rcx
    cmp ecx, [n]
    jl loop_pipelined

```

4. Strength Reduction

Replace expensive operations with cheaper equivalents.

```

; C equivalent:
; for (int i = 0; i < n; ++i) {
;     result[i] = array[i] * 10;
; }

; SLOW: Multiplication every iteration
loop_mul:
    mov eax, [array + rcx*4]
    imul eax, 10                ; 3 cycles latency
    mov [result + rcx*4], eax
    inc rcx
    cmp ecx, [n]
    jl loop_mul

; FAST: LEA trick
loop_lea:
    mov eax, [array + rcx*4]
    lea eax, [rax + rax*4]      ; eax = rax + rax*4 = rax*5
    add eax, eax                ; eax = eax*2 = rax*10 (1 cycle!)
    mov [result + rcx*4], eax
    inc rcx
    cmp ecx, [n]
    jl loop_lea

```

Common Strength Reductions:

Original	Optimized	Savings
<code>x * 2</code>	<code>add x, x</code> or <code>shl x, 1</code>	2 cycles
<code>x / 2</code> (unsigned)	<code>shr x, 1</code>	20+ cycles
<code>x % powerof2</code>	<code>and x, (powerof2-1)</code>	20+ cycles
<code>x * 10</code>	<code>lea rax, [rax+rax*4]</code> + <code>add rax, rax</code>	2 cycles

5. Induction Variable Elimination

Compute addresses directly instead of using index.

```

; C equivalent:
; for (int i = 0; i < n; ++i) {
;     result[i] = array[i] + 1;
; }

; With index
xor ecx, ecx
loop_index:
    mov eax, [array + rcx*4]    ; Compute address every time
    add eax, 1
    mov [result + rcx*4], eax  ; Compute address every time
    inc ecx
    cmp ecx, [n]
    jl loop_index

; With pointer
lea rsi, [array]
lea rdi, [result]
mov ecx, [n]
loop_pointer:
    mov eax, [rsi]              ; Direct dereference
    add eax, 1
    mov [rdi], eax              ; Direct dereference
    add rsi, 4                  ; Increment pointer
    add rdi, 4
    dec ecx
    jnz loop_pointer
; Fewer address calculations!

```

Branch Optimization

Branch Prediction

Modern CPUs predict branch outcomes to avoid pipeline stalls.

Branch Misprediction Penalty: 15-20 cycles!

```

; C equivalent:
; int count = 0;
; for (int i = 0; i < n; ++i) {
;     if (array[i] > threshold) { // Hard to predict!
;         count++;
;     }
; }

; BAD: Unpredictable branch
xor eax, eax           ; count
xor ecx, ecx
loop_branch:
    mov edx, [array + rcx*4]
    cmp edx, [threshold]
    jle skip           ; 50% misprediction if data is random
    inc eax
skip:
    inc ecx
    cmp ecx, [n]
    jl loop_branch

```

Branch-Free Code (Predication)

```

; C equivalent (branchless):
; count += (array[i] > threshold);

; GOOD: Branchless with conditional move
xor eax, eax
xor ecx, ecx
loop_branchless:
    mov edx, [array + rcx*4]
    xor ebx, ebx          ; temp = 0
    cmp edx, [threshold]
    setg bl               ; bl = 1 if edx > threshold, else 0
    add eax, ebx          ; No branch!
    inc ecx
    cmp ecx, [n]
    jl loop_branchless

; Alternative with CMOV
loop_cmov:
    mov edx, [array + rcx*4]
    mov ebx, eax
    inc ebx               ; ebx = count + 1
    cmp edx, [threshold]
    cmovg eax, ebx        ; count = (edx > threshold) ? ebx : count
    inc ecx
    cmp ecx, [n]
    jl loop_cmov

```

Loop-Invariant Code Motion

Move calculations that don't change out of the loop.

```

; C equivalent:
; int factor = x * 2 + 5;
; for (int i = 0; i < n; ++i) {
;     result[i] = array[i] * factor;
; }

; BAD: Recomputing factor every iteration
loop_bad:
    mov eax, [x]
    shl eax, 1
    add eax, 5                ; Computed every iteration!
    imul eax, [array + rcx*4]
    mov [result + rcx*4], eax
    inc rcx
    cmp ecx, [n]
    jl loop_bad

; GOOD: Compute factor once
mov eax, [x]
shl eax, 1
add eax, 5
mov ebx, eax                ; Save factor

loop_good:
    mov eax, [array + rcx*4]
    imul eax, ebx            ; Use saved factor
    mov [result + rcx*4], eax
    inc rcx
    cmp ecx, [n]
    jl loop_good

```


Data Alignment

Alignment Requirements

```
; SLOW: Misaligned data
section .data
    x db 0
    data dq 0x123456789ABCDEF0 ; Misaligned!

; FAST: Aligned data
section .data
    align 8
    data dq 0x123456789ABCDEF0 ; 8-byte aligned

; CRITICAL for SIMD
section .data
    align 16
    floats dd 1.0, 2.0, 3.0, 4.0 ; Must be 16-byte aligned for movaps
```

Performance Impact:

- Misaligned scalar access: 0-10% slower
- Misaligned SIMD access: Can be **2-3x slower** or **crash!**

Structure Padding

```
// BAD: Lots of padding, poor cache utilization
struct Data {
    char a;          // 1 byte
    // 7 bytes padding
    double b;        // 8 bytes (needs 8-byte alignment)
    char c;          // 1 byte
    // 7 bytes padding
}; // Total: 24 bytes (13 bytes wasted!)

// GOOD: Reordered to minimize padding
struct Data {
    double b;        // 8 bytes
    char a;          // 1 byte
    char c;          // 1 byte
    // 6 bytes padding (end)
}; // Total: 16 bytes (6 bytes wasted)
```

Complete Optimization Example

Problem: Compute average of large array

```

; Original - unoptimized
; C equivalent:
; double average = 0;
; for (int i = 0; i < n; ++i) {
;     average += array[i];
; }
; average /= n;

average_slow:
    xorpd xmm0, xmm0      ; sum = 0.0
    xor ecx, ecx          ; i = 0
.loop:
    addsd xmm0, [rdi + rcx*8] ; Scalar add (2 doubles per cycle max)
    inc ecx
    cmp ecx, esi
    jl .loop

    cvtsi2sd xmm1, esi     ; Convert n to double
    divsd xmm0, xmm1      ; Divide (very slow!)
    ret

```

```

; Optimized version
; Uses: SIMD, unrolling, multiple accumulators

average_fast:
    ; Check if we have enough elements for SIMD
    cmp esi, 8
    jl .scalar_fallback

    ; Initialize 4 accumulators for parallel accumulation
    xorpd xmm0, xmm0      ; sum0
    xorpd xmm1, xmm1      ; sum1
    xorpd xmm2, xmm2      ; sum2
    xorpd xmm3, xmm3      ; sum3

    xor ecx, ecx
    mov edx, esi
    shr edx, 3             ; Process 8 doubles per iteration
    jz .remainder

.loop_simd:
    ; Process 8 doubles = 4 * 2 doubles
    addpd xmm0, [rdi + rcx*8]      ; Add 2 doubles
    addpd xmm1, [rdi + rcx*8 + 16] ; Add 2 doubles
    addpd xmm2, [rdi + rcx*8 + 32] ; Add 2 doubles
    addpd xmm3, [rdi + rcx*8 + 48] ; Add 2 doubles
    add ecx, 8
    dec edx
    jnz .loop_simd

.remainder:
    ; Handle remaining elements (n % 8)
    mov edx, esi
    and edx, 7
    jz .horizontal_sum

.remainder_loop:
    addsd xmm0, [rdi + rcx*8]
    inc ecx
    dec edx
    jnz .remainder_loop

.horizontal_sum:
    ; Combine the 4 accumulators
    addpd xmm0, xmm1      ; xmm0 = sum0+sum1
    addpd xmm2, xmm3      ; xmm2 = sum2+sum3
    addpd xmm0, xmm2      ; xmm0 = all sums

    ; Horizontal add within xmm0
    movapd xmm1, xmm0
    unpckhpd xmm1, xmm0   ; Extract high double

```

```

    addsd xmm0, xmm1      ; Final sum

    ; Divide by n (use multiplication by reciprocal if n is constant!)
    cvtsi2sd xmm1, esi
    divsd xmm0, xmm1
    ret

.scalar_fallback:
    ; Handle small arrays
    xorpd xmm0, xmm0
    xor ecx, ecx
.scalar_loop:
    addsd xmm0, [rdi + rcx*8]
    inc ecx
    cmp ecx, esi
    jl .scalar_loop
    cvtsi2sd xmm1, esi
    divsd xmm0, xmm1
    ret

```

Optimizations Applied:

1. ✓ SIMD (4x parallelism with packed doubles)
2. ✓ Loop unrolling (8 elements per iteration)
3. ✓ Multiple accumulators (break dependency chains)
4. ✓ Separate remainder handling
5. ✓ Scalar fallback for small arrays

Expected Speedup: 10-15x over naive version!

Profiling and Measurement

CPU Cycle Counting

```
section .text
global benchmark_function

benchmark_function:
    ; Save registers
    push rbx

    ; Read timestamp counter (before)
    rdtsc                ; EDX:EAX = cycle count
    shl rdx, 32
    or rax, rdx           ; RAX = 64-bit cycle count
    mov rbx, rax          ; Save start time

    ; *** Code to benchmark ***
    ; ...

    ; Read timestamp counter (after)
    rdtsc
    shl rdx, 32
    or rax, rdx

    ; Calculate elapsed cycles
    sub rax, rbx          ; RAX = elapsed cycles

    pop rbx
    ret
```

Using `perf` (Linux)

```
# Count cycles, instructions, cache misses
perf stat -e cycles,instructions,cache-misses ./program

# Profile with sampling
perf record -g ./program
perf report

# Analyze specific events
perf stat -e L1-dcache-loads,L1-dcache-load-misses ./program
```

Optimization Checklist

Algorithm Level

- ☐ Use optimal algorithm ($O(n \log n)$ vs $O(n^2)$)
- ☐ Minimize memory allocations
- ☐ Cache reusable computations

Data Structure Level

- ☐ Align data structures
- ☐ Use cache-friendly layouts (SoA vs AoS)
- ☐ Pack structures to minimize padding
- ☐ Consider false sharing in multithreaded code

Loop Level

- ☐ Unroll loops (2x, 4x, 8x)
- ☐ Use multiple accumulators
- ☐ Move invariant code out of loop
- ☐ Eliminate induction variables
- ☐ Use SIMD when possible

Instruction Level

- ☐ Use faster instructions (LEA, shifts vs multiply/divide)
- ☐ Minimize dependency chains
- ☐ Use conditional moves instead of branches
- ☐ Align loop targets
- ☐ Avoid partial register stalls

Memory Level

- ☐ Prefetch data before use
- ☐ Access memory sequentially
- ☐

Minimize cache misses



Use appropriate data alignment

Common Pitfalls

1. Premature Optimization

```
// DON'T optimize before profiling!  
// 90% of time is often in 10% of code  
  
// Profile first:  
// - Find the hot spots  
// - Optimize those  
// - Measure improvement
```

2. Over-Optimization

```
; Sometimes "optimized" code is slower!  
  
; "Optimized" but slower (more instructions, worse for branch prediction)  
xor eax, eax  
cmp [value], 0  
setg al  
add [result], eax  
  
; Simpler and faster (well-predicted branch)  
cmp [value], 0  
jle .skip  
inc dword [result]  
.skip:
```

3. Ignoring the Compiler

```
// Modern compilers are VERY good  
// Sometimes C is faster than hand-written assembly!  
  
// Let the compiler optimize:  
int sum = 0;  
for (int i = 0; i < n; ++i) {  
    sum += array[i]; // Compiler may auto-vectorize this!  
}
```

Summary

Key Principles

1. **Profile first** - Optimize hot paths only
2. **Algorithm beats microoptimization** - $O(n)$ vs $O(n^2)$ matters more
3. **Understand hardware** - Cache, pipeline, branch prediction
4. **Break dependencies** - Use multiple accumulators
5. **Use SIMD** - 4-16x speedup for parallel data

Optimization Priorities

1. **Algorithm** (100x+ gains possible)
2. **Data structures** (10x+ gains)
3. **Loop optimizations** (2-5x gains)
4. **SIMD** (4-16x gains)
5. **Instruction selection** (1.5-2x gains)
6. **Micro-optimizations** (5-20% gains)

Tools

- **perf** - CPU performance counters
- **valgrind/cachegrind** - Cache profiling
- **gprof** - Function-level profiling
- **RDTSC** - Cycle-accurate timing
- **VTune** - Intel's profiler (commercial)

Practice Exercises

Exercise 1: Matrix Multiplication

Optimize 4x4 matrix multiplication using SIMD and unrolling.

Exercise 2: String Length

Write the fastest possible `strlen` implementation.

Exercise 3: Sorting

Implement optimized quicksort with:

- Insertion sort for small arrays

- Three-way partitioning
 - Tail recursion elimination
-

Next Topic

In **Topic 20: Debugging & Tools**, we'll explore essential debugging techniques, disassembly analysis, and tools for working with assembly code.

Preview:

- GDB debugging
- Disassembly with objdump
- strace for syscall tracing
- Memory debugging with valgrind
- Reading crash dumps
- Reverse engineering basics

CHAPTER 37

Topic 20: Debugging and Tools

Overview

Debugging assembly code requires specialized tools and techniques. This topic covers essential debugging workflows, tools for analysis, and strategies for finding and fixing bugs in low-level code.

```
# Essential toolkit:
gdb          # The GNU debugger
objdump      # Disassemble binaries
strace       # Trace system calls
valgrind     # Memory debugging
readelf      # Examine ELF files
hexdump      # View binary data
```

GDB - The GNU Debugger

Basic GDB Workflow

```
# Compile with debug symbols
nasm -f elf64 -g -F dwarf program.asm -o program.o
ld program.o -o program

# Or with C:
gcc -g program.c -o program

# Start GDB
gdb ./program
```

Essential GDB Commands

```

# Starting and stopping
(gdb) run [args]           # Start program
(gdb) run < input.txt      # Start with input redirection
(gdb) start                # Start and break at main
(gdb) quit                 # Exit GDB

# Breakpoints
(gdb) break main           # Break at function
(gdb) break *0x401000      # Break at address
(gdb) break program.asm:42 # Break at source line
(gdb) info breakpoints     # List breakpoints
(gdb) delete 1             # Delete breakpoint 1
(gdb) disable 2            # Disable breakpoint 2
(gdb) enable 2             # Enable breakpoint 2

# Execution control
(gdb) continue             # Continue execution
(gdb) next                 # Next source line (step over calls)
(gdb) step                 # Step into function calls
(gdb) stepi                # Single instruction step
(gdb) nexti                # Next instruction (step over calls)
(gdb) finish               # Run until current function returns
(gdb) until *0x401050      # Run until address

# Examining state
(gdb) info registers       # Show all registers
(gdb) info registers rax rbx # Show specific registers
(gdb) print $rax           # Print register value
(gdb) print /x $rax        # Print in hex
(gdb) print /t $rax        # Print in binary
(gdb) print /d $rax        # Print in decimal

# Memory examination
(gdb) x/10x $rsp           # Examine 10 hex values at RSP
(gdb) x/10i $rip           # Disassemble 10 instructions at RIP
(gdb) x/s 0x404000         # Display string at address
(gdb) x/4gx $rsp           # Examine 4 giant (8-byte) hex values

# Memory modification
(gdb) set $rax = 0          # Set register
(gdb) set *(int*)0x404000 = 42 # Write to memory

# Watchpoints (break on memory change)
(gdb) watch *0x404000      # Break when memory changes
(gdb) watch $rax           # Break when register changes
(gdb) rwatch *0x404000    # Break on read
(gdb) awatch *0x404000    # Break on read or write

# Stack examination
(gdb) backtrace            # Show call stack

```

```
(gdb) frame 2           # Switch to frame 2
(gdb) info frame        # Show current frame info

# Disassembly
(gdb) disassemble main  # Disassemble function
(gdb) disassemble /r main # With raw bytes
(gdb) set disassembly-flavor intel # Use Intel syntax
```

TUI Mode (Text User Interface)

```
(gdb) tui enable        # Enable TUI
(gdb) layout asm        # Show assembly
(gdb) layout regs       # Show registers
(gdb) layout split      # Show source and assembly
(gdb) focus cmd         # Focus command window
(gdb) refresh           # Redraw screen

# Or start with:
gdb -tui ./program
```

Example Debugging Session

```
; buggy.asm - Contains an off-by-one error
section .data
    array dd 1, 2, 3, 4, 5
    count equ 5

section .text
global _start

_start:
    xor eax, eax        ; sum = 0
    xor ecx, ecx        ; i = 0

loop:
    cmp ecx, count
    jge done
    add eax, [array + rcx*4]
    inc ecx
    jmp loop            ; BUG: Should be conditional

done:
    ; Exit
    mov rdi, rax
    mov rax, 60
    syscall
```

Debugging session:

```
$ gdb ./buggy
(gdb) break _start
(gdb) run

# Step through
(gdb) display /x $eax      # Always show EAX
(gdb) display /x $ecx      # Always show ECX

(gdb) stepi                # Execute xor eax, eax
# EAX = 0

(gdb) stepi                # Execute xor ecx, ecx
# ECX = 0

(gdb) stepi                # Execute cmp ecx, count
(gdb) stepi                # Execute jge done (not taken)
(gdb) stepi                # Execute add eax, [array]
# EAX = 1, ECX = 0

(gdb) stepi                # Execute inc ecx
# ECX = 1

(gdb) stepi                # Execute jmp loop (UNCONDITIONAL!)
# Notice: We always jump back, ECX never reaches count!

# Found the bug: jmp should be replaced with conditional logic
# or the loop structure needs fixing
```

Objdump - Disassembly Tool

Basic Usage

```
# Disassemble entire program
objdump -d program

# Intel syntax
objdump -d -M intel program

# With source code interleaved
objdump -d -S program

# Specific section
objdump -d -j .text program

# Show all headers
objdump -x program

# Full disassembly with symbols
objdump -D -M intel program > disassembly.txt
```

Example Output

```
$ objdump -d -M intel program

program:      file format elf64-x86-64

Disassembly of section .text:

0000000000401000 <_start>:
 401000:  48 31 c0                xor     rax,rax
 401003:  48 31 c9                xor     rcx,rcx
 401006:  48 39 0d f3 0f 00 00    cmp     QWORD PTR [rip+0xff3],rcx
 40100d:  7e 0e                  jle     40101d <done>

000000000040100f <loop>:
 40100f:  48 03 04 8d e0 0f 40    add     rax,QWORD PTR [rcx*4+0x400fe0]
 401016:  00
 401017:  48 ff c1                inc     rcx
 40101a:  eb ed                  jmp     401009 <loop+0xfffffffffffffffffa>

000000000040101c <done>:
 40101c:  48 89 c7                mov     rdi,rax
 40101f:  48 c7 c0 3c 00 00 00    mov     rax,0x3c
 401026:  0f 05                  syscall
```

Analysis:

- Addresses (column 1)
- Machine code bytes (column 2)
- Assembly instructions (column 3)

Strace - System Call Tracer

Basic Usage

```
# Trace all syscalls
strace ./program

# Show syscall stats
strace -c ./program

# Filter specific syscalls
strace -e trace=open,read,write ./program

# Show only file operations
strace -e trace=file ./program

# Show timestamps
strace -t ./program

# Show time spent in each syscall
strace -T ./program

# Attach to running process
strace -p <pid>
```

Example Output

```
$ strace ./hello

execve("./hello", ["./hello"], 0x7fff...) = 0
write(1, "Hello, World!\n", 14)           = 14
exit_group(0)                             = ?
+++ exited with 0 +++
```

Use cases:

- Debug file access issues
- Find missing files
- Trace network operations

- Identify performance bottlenecks (slow syscalls)
- Understand program behavior

Valgrind - Memory Debugger

Memcheck - Memory Error Detection

```
# Basic memory check
valgrind --leak-check=full ./program

# With detailed allocation tracking
valgrind --leak-check=full --track-origins=yes ./program

# Show reachable leaks too
valgrind --leak-check=full --show-reachable=yes ./program
```

Common Errors Detected

```
// C example with errors (for demonstration)

int main() {
    int *ptr = malloc(100 * sizeof(int));

    // 1. Invalid write (beyond allocated memory)
    ptr[100] = 42; // Valgrind: Invalid write of size 4

    // 2. Invalid read
    int x = ptr[100]; // Valgrind: Invalid read of size 4

    // 3. Memory leak (forgot to free)
    return 0; // Valgrind: 400 bytes in 1 blocks definitely lost
}
```

Valgrind output:

```
==12345== Invalid write of size 4
==12345==    at 0x401234: main (test.c:5)
==12345==    Address 0x5204190 is 0 bytes after a block of size 400 alloc'd

==12345== LEAK SUMMARY:
==12345==    definitely lost: 400 bytes in 1 blocks
```

Cachegrind - Cache Profiler

```
# Profile cache usage
valgrind --tool=cachegrind ./program

# View results
cg_annotate cachegrind.out.<pid>

# Show specific function
cg_annotate --show=my_function cachegrind.out.<pid>
```

Callgrind - Call Graph Profiler

```
# Generate call graph
valgrind --tool=callgrind ./program

# Visualize with kcachegrind
kcachegrind callgrind.out.<pid>
```

Readelf - ELF File Analysis

Common Commands

```
# Show ELF header
readelf -h program

# Show section headers
readelf -S program

# Show program headers (segments)
readelf -l program

# Show symbol table
readelf -s program

# Show dynamic section
readelf -d program

# Show relocations
readelf -r program
```

Example: Analyzing Sections

```
$ readelf -S program
```

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info Align	
[1]	.text	PROGBITS	0000000000401000	00001000
	00000000000001a0	0000000000000000	AX 0 0	16
[2]	.rodata	PROGBITS	00000000004011a0	000011a0
	0000000000000020	0000000000000000	A 0 0	8
[3]	.data	PROGBITS	00000000004021c0	000021c0
	0000000000000010	0000000000000000	WA 0 0	8
[4]	.bss	NOBITS	00000000004021d0	000021d0
	0000000000000008	0000000000000000	WA 0 0	8

Key:

A = Allocate, X = Execute, W = Write

Hexdump - Binary Data Viewer

Viewing Binary Files

```
# Standard hex dump
hexdump -C program.o

# 16-bit values
hexdump -e '16/2 "%04x " "\n"' file.bin

# Show strings in binary
strings program

# Search for specific byte sequence
hexdump -C file.bin | grep "48 89 e5"
```

Example Output

```
$ hexdump -C program | head -20

00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF.....|
00000010  02 00 3e 00 01 00 00 00  00 10 40 00 00 00 00 00  |..>.....@...|
00000020  40 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |@.....|
...
```

Common Debugging Scenarios

Scenario 1: Segmentation Fault

```
$ gdb ./program
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x0000000000401015 in loop ()

(gdb) info registers
rax            0x5            5
rbx            0x0            0
rcx            0x6            6  <-- Suspicious value
...

(gdb) x/10i $rip-10
=> 0x401015 <loop+6>:  mov    eax,DWORD PTR [rbx+rcx*4]

# Problem:RBX is 0 (null pointer)!
# Accessing [0 + 6*4] = [24] -> SEGFAULT

(gdb) backtrace
#0  0x401015 in loop ()
#1  0x401008 in _start ()

# Debug: Check where RBX should have been initialized
```

Scenario 2: Infinite Loop

```
(gdb) run
^C # Ctrl+C to interrupt

Program received signal SIGINT, Interrupt.
0x0000000000401018 in loop ()

(gdb) disassemble
=> 0x401018 <loop+9>:      jmp     0x40100f <loop>

(gdb) print $rcx
$1 = 1000000 # Very high value - loop counter not terminating

(gdb) break loop
(gdb) commands
> print $rcx
> continue
> end

(gdb) run
# Watch RCX values to see if it ever reaches exit condition
```

Scenario 3: Wrong Result

```
(gdb) break done
(gdb) run
Breakpoint 1, done ()

(gdb) print /d $eax
$1 = 120 # Expected 15

# Work backwards
(gdb) break loop
(gdb) run
(gdb) print $eax
$2 = 0 # Initial value correct

(gdb) continue # After first iteration
$3 = 1

(gdb) continue # After second iteration
$4 = 2

# Ah! Should be cumulative sum (1, 3, 6, 10, 15)
# but we're getting indices (1, 2, 3, 4, 5)
# Bug: Adding ECX instead of array value!
```

Advanced Debugging Techniques

Conditional Breakpoints

```
# Break only when condition is true
(gdb) break loop if $rcx == 5

# Break after N hits
(gdb) break loop
(gdb) ignore 1 100  # Ignore breakpoint 1 for 100 hits

# Complex condition
(gdb) break *0x401020 if $rax > 1000 && $rbx != 0
```

Scripting GDB

```
# commands.gdb
break _start
commands
  silent
  printf "RAX: %llx\n", $rax
  continue
end

run
```

```
# Run with script
gdb -x commands.gdb ./program
```

Core Dumps

```
# Enable core dumps
ulimit -c unlimited

# Run program (it crashes)
./program
Segmentation fault (core dumped)

# Analyze core dump
gdb ./program core
(gdb) backtrace
(gdb) info registers
(gdb) x/10i $rip
```

Remote Debugging

```
# On target machine:
gdbserver :1234 ./program

# On development machine:
gdb ./program
(gdb) target remote target_ip:1234
(gdb) continue
```

Reverse Engineering Basics

Static Analysis

```
# Find entry point
readelf -h program | grep Entry
Entry point address:          0x401000

# Find interesting strings
strings program
strings -n 8 program # Min length 8

# Extract sections
objcopy --dump-section .text=text.bin program
```

Dynamic Analysis

```
# Trace execution
ltrace ./program # Trace library calls
strace ./program # Trace syscalls

# Attach to running process
gdb -p $(pidof program)
```

Patching Binaries

```
# Using hex editor
hexedit program

# Or with dd
echo -ne '\x90\x90' | dd of=program bs=1 seek=$((0x1234)) conv=notrunc

# Or with objcopy
objcopy --update-section .text=patched.bin program program_patched
```

Best Practices

Development Workflow

1. **Write incrementally** - Test small pieces
2. **Use assertions** - Validate assumptions
3. **Add debug output** - Print key values
4. **Version control** - Commit working states
5. **Document assumptions** - Comment tricky code

Debug Build vs Release Build

```
# Debug build
debug: CFLAGS = -g -O0 -DDEBUG
debug: program

# Release build
release: CFLAGS = -O3 -DNDEBUG
release: program
```


Defensive Programming

```
; Check for null pointer
test rdi, rdi
jz .error_null_pointer

; Verify array bounds
cmp rcx, [array_size]
jae .error_out_of_bounds

; Validate syscall return
test rax, rax
js .error_syscall_failed
```

Debugging Checklist

Before Debugging

- ☐ Can you reproduce the bug?
- ☐ Do you have debug symbols?
- ☐ Is the code under version control?
- ☐ Do you have a minimal test case?

During Debugging

- ☐ Read error messages carefully
- ☐ Check return values
- ☐ Verify assumptions
- ☐ Use the right tool (gdb vs valgrind vs strace)
- ☐ Take notes on what you try

Common Bug Locations

- ☐ Off-by-one errors in loops
- ☐ Uninitialized registers/memory
- ☐ Wrong register size (RAX vs EAX vs AX vs AL)
- ☐ Incorrect calling convention
- ☐

Stack misalignment

□

Signed vs unsigned comparison

□

Endianness issues

□

Buffer overflows

Tool Summary

Tool	Purpose	When to Use
GDB	Interactive debugging	Logic errors, crashes
Objdump	Disassembly	Understanding compiled code
Strace	Syscall tracing	File I/O, network issues
Valgrind	Memory debugging	Leaks, invalid accesses
Readelf	ELF analysis	Understanding binary structure
Hexdump	Binary viewing	Low-level data inspection
Ltrace	Library call tracing	Understanding library usage
Perf	Performance profiling	Finding bottlenecks

Practice Exercises

Exercise 1: Fix the Bugs

Given a buggy assembly program, use GDB to find and fix:

1. Segmentation fault
2. Incorrect calculation
3. Memory leak

Exercise 2: Reverse Engineering

Disassemble a simple program and determine:

1. What it does
2. Its input/output
3. Algorithm used

Exercise 3: Performance Analysis

Use `perf` to profile a program and identify:

1. Hottest function
 2. Cache miss rate
 3. Branch misprediction rate
-

Summary

Essential Skills

1. **GDB proficiency** - Step through code, examine state
2. **Disassembly reading** - Understand compiler output
3. **Tool selection** - Right tool for the problem
4. **Systematic approach** - Reproduce, isolate, fix, verify

Key Tools

- **GDB**: Interactive debugging
- **Strace**: System call tracing
- **Valgrind**: Memory debugging
- **Objdump**: Static analysis

Debugging Mindset

1. Understand what the code *should* do
2. Observe what it *actually* does
3. Form hypotheses about the cause
4. Test hypotheses systematically
5. Fix root cause, not symptoms

Resources

- GDB manual: `info gdb`
 - Intel manuals: software.intel.com
 - Linux syscall reference: man7.org
 - Stack Overflow: [assembly debugging questions](#)
-

Congratulations!

You've completed the comprehensive NASM x86-64 assembly programming tutorial! You now have the knowledge to:

- Write efficient assembly code
- Interface with C and operating systems
- Optimize for performance
- Debug complex low-level issues
- Understand how computers work at the lowest level

Next steps:

- Practice writing assembly programs
- Contribute to open-source projects
- Explore compiler output for your C code
- Study CPU architecture in depth
- Build your own tools and libraries

Happy hacking!

CHAPTER 38

Topic 21: Memory Allocation Internals

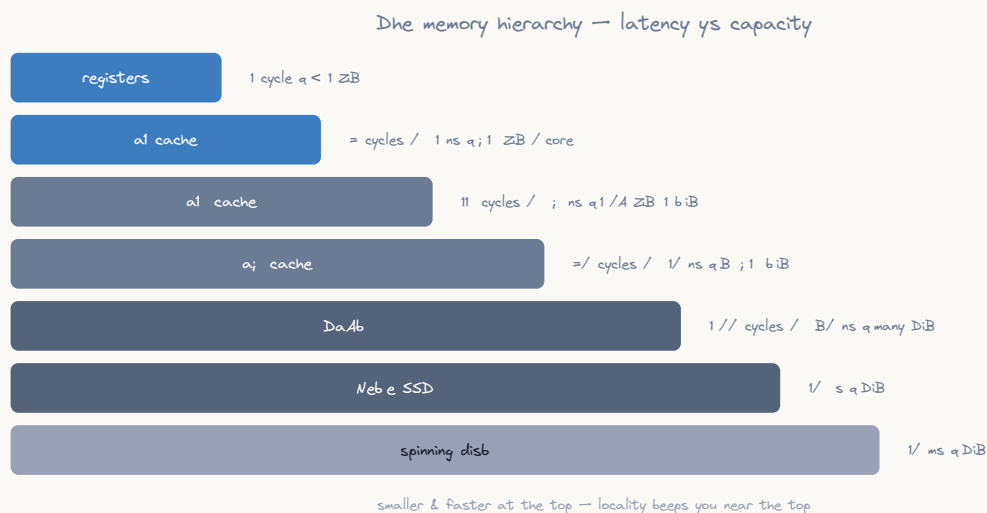
Overview

When you call `malloc()` in C, something magical seems to happen — memory appears. But under the hood, `malloc` is just user-space code that manages a pool of memory obtained from the kernel through syscalls. This topic dissects the entire memory allocation stack, from the high-level C API down to the kernel's page allocator, all explained at the assembly level.

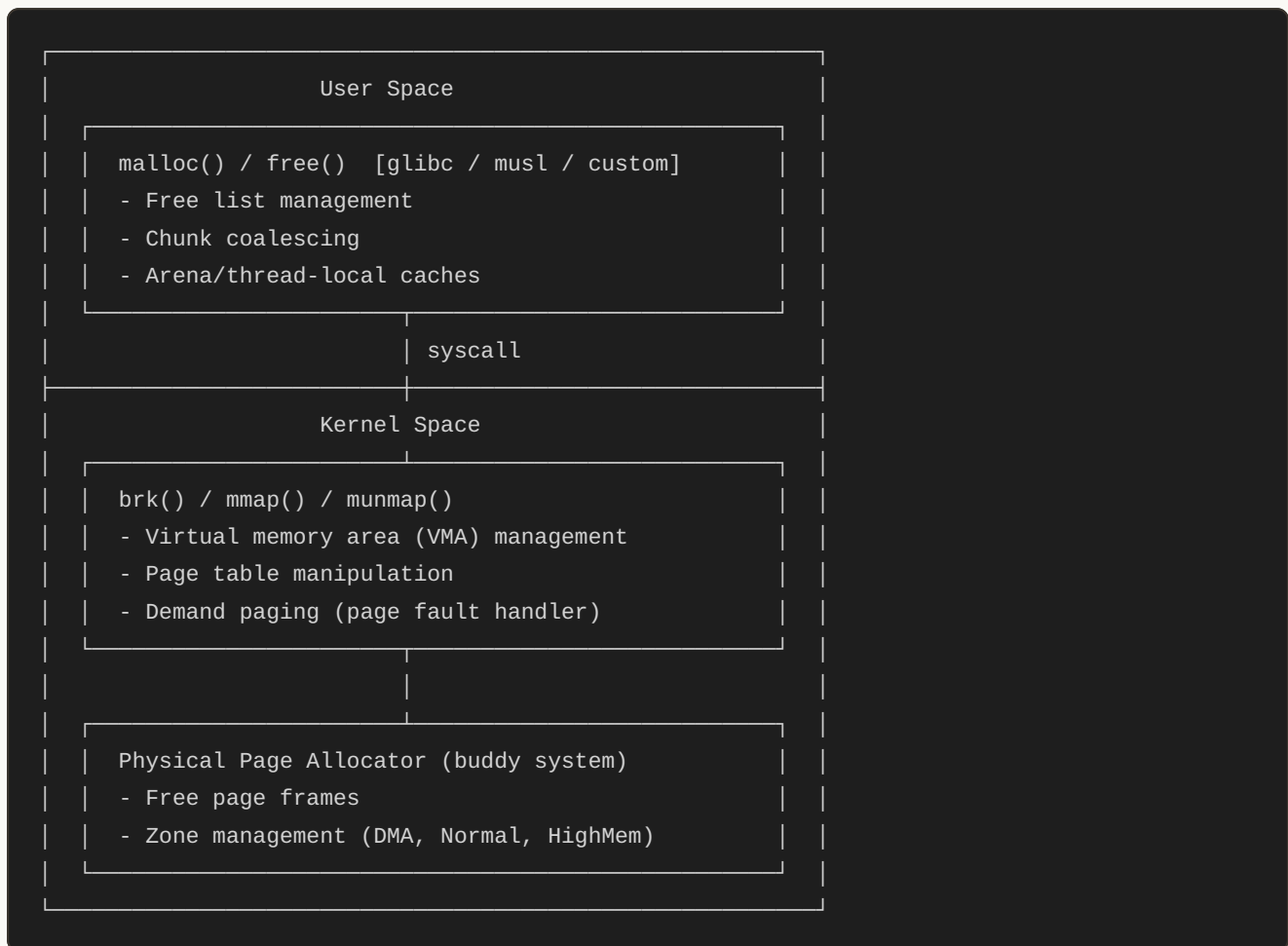
```
// What you write in C:
void *ptr = malloc(64);
free(ptr);

// What actually happens (simplified):
// 1. malloc checks its internal free list
// 2. If no suitable chunk: calls brk() or mmap() syscall
// 3. Kernel maps physical pages to virtual address space
// 4. malloc carves a chunk from the mapped region
// 5. Returns pointer to usable area (after metadata header)
```

The Memory Hierarchy



ASCII diagram

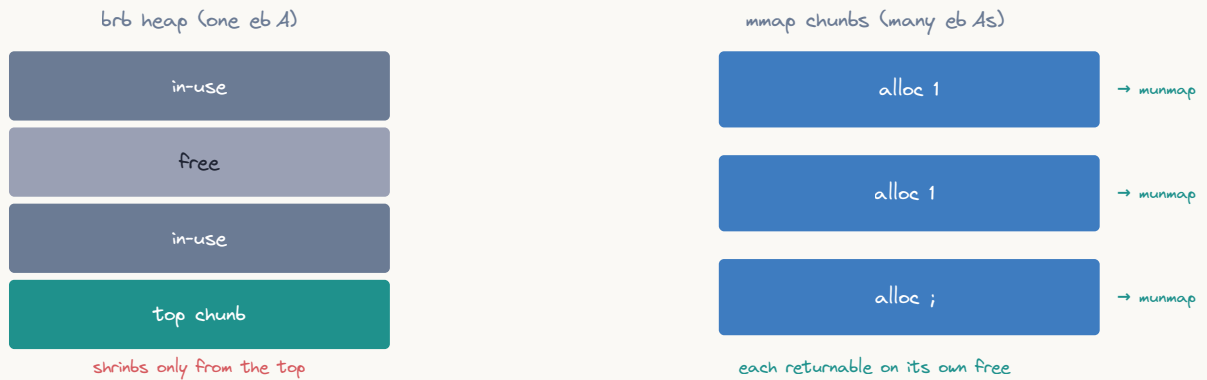


Part 1: `brk()` — The Program Break

What is the Program Break?

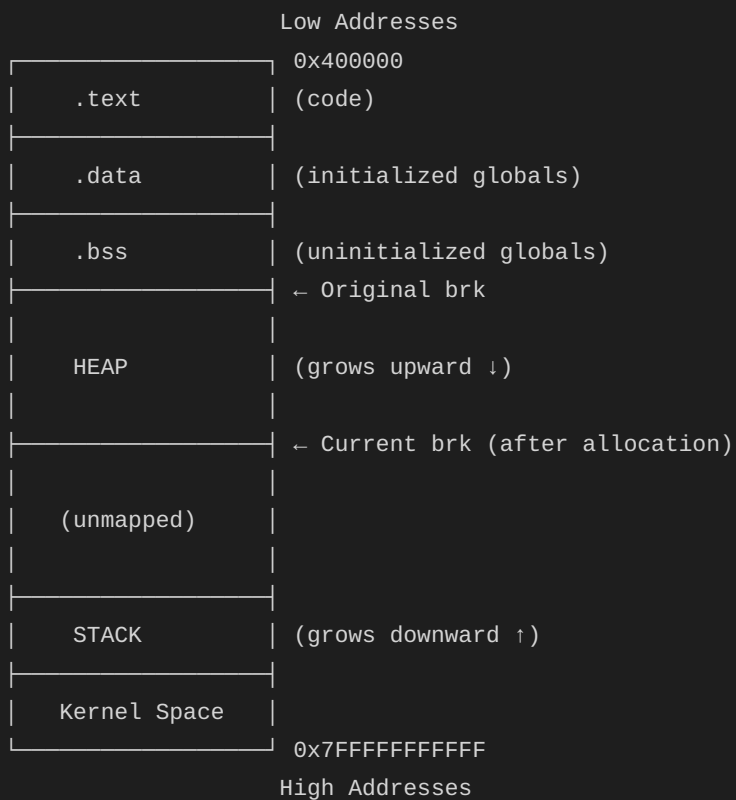
The program break is the boundary between the heap and unused virtual address space. Moving it up “allocates” more virtual address space for the heap.

Why ptmalloc uses both brk and mmap



ASCII diagram

Virtual Memory Layout:



brk() Syscall in Assembly


```

; C equivalent:
; int brk(void *addr);           // set program break
; void *sbrk(intptr_t increment); // increment program break

section .data
    alloc_msg db "Allocated memory at: 0x", 0
    newline   db 10

section .bss
    heap_start resq 1
    heap_end   resq 1

section .text
    global _start

_start:
    ; Get current program break (brk(0) returns current break)
    mov rax, 12          ; sys_brk
    xor rdi, rdi         ; addr = 0 (query current break)
    syscall
    mov [heap_start], rax ; Save initial break address
    mov [heap_end], rax

    ; Allocate 4096 bytes by moving break up
    mov rdi, rax
    add rdi, 4096        ; New break = current + 4096
    mov rax, 12          ; sys_brk
    syscall

    ; Check if allocation succeeded
    cmp rax, [heap_end]  ; If rax == old break, it failed
    je .alloc_failed

    mov [heap_end], rax  ; Save new break

    ; Now [heap_start] to [heap_end] is our heap
    ; Write something to our allocated memory
    mov rdi, [heap_start]
    mov byte [rdi], 'H'
    mov byte [rdi+1], 'i'

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

.alloc_failed:
    mov rax, 60

```

```
mov rdi, 1
syscall
```

sbrk() Implementation in Assembly

```
; sbrk equivalent - increment the program break by N bytes
; Input: RDI = number of bytes to allocate
; Output: RAX = pointer to start of new memory (old break)
; On error: RAX = -1
sbrk:
    push rbx
    push rdi                ; Save requested size

    ; Get current break
    mov rax, 12             ; sys_brk
    push rdi                ; Save size
    xor rdi, rdi            ; Query current break
    syscall
    mov rbx, rax            ; RBX = old break (return value)

    ; Set new break
    pop rdi                 ; Restore requested size
    pop rdi                 ; Restore from stack
    add rdi, rbx             ; New break = old break + size
    mov rax, 12             ; sys_brk
    syscall

    ; Check success: new break should equal requested address
    cmp rax, rbx            ; If unchanged, allocation failed
    je .sbrk_fail

    mov rax, rbx            ; Return old break (start of new memory)
    pop rbx
    ret

.sbrk_fail:
    mov rax, -1
    pop rbx
    ret
```

Part 2: mmap() — Memory Mapped Allocation

Why mmap() for Large Allocations?

`brk()` has a critical limitation: you can only free memory at the top of the heap. If you `brk()` up by 1MB, use it, then want to free it, you can only do so if nothing was allocated above it. `mmap()` solves this by allocating independent virtual memory regions.

Rule of thumb in glibc:

- Allocations < 128KB (MMAP_THRESHOLD): Use `brk()`
- Allocations >= 128KB: Use `mmap()`

mmap() Syscall in Assembly

```

; C equivalent:
; void *mmap(void *addr, size_t length, int prot,
;           int flags, int fd, off_t offset);

; Syscall number: 9 (sys_mmap)
; Arguments:
;   RDI = addr (NULL for kernel to choose)
;   RSI = length (bytes)
;   RDX = prot (PROT_READ=1, PROT_WRITE=2, PROT_EXEC=4)
;   R10 = flags (MAP_PRIVATE=2, MAP_ANONYMOUS=32)
;   R8  = fd (-1 for anonymous mapping)
;   R9  = offset (0 for anonymous mapping)

section .text
    global _start

_start:
    ; Allocate 1 page (4096 bytes) of anonymous memory
    mov rax, 9                ; sys_mmap
    xor rdi, rdi              ; addr = NULL (kernel chooses)
    mov rsi, 4096             ; length = 4096 bytes (1 page)
    mov rdx, 3                ; prot = PROT_READ | PROT_WRITE
    mov r10, 34               ; flags = MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1                ; fd = -1 (not file-backed)
    xor r9, r9                ; offset = 0
    syscall

    ; Check for error (returns -errno on failure)
    cmp rax, -4096            ; Check if return is in error range
    ja .mmap_failed           ; Unsigned comparison catches -1 to -4095

    ; RAX now points to our mapped memory
    mov rdi, rax              ; Save pointer

    ; Use the memory
    mov qword [rdi], 0xDEADBEEF
    mov qword [rdi+8], 0xCAFEFEBABE

    ; Free the memory with munmap
    ; mov rdi, rdi            ; addr (already in RDI)
    mov rsi, 4096             ; length
    mov rax, 11               ; sys_munmap
    syscall

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

.mmap_failed:

```

```

mov rax, 60
mov rdi, 1
syscall

```

Allocating Executable Memory (JIT Compilation)

```

; Allocate memory that can hold executable code (like a JIT compiler)
; prot = PROT_READ | PROT_WRITE | PROT_EXEC = 7
; WARNING: Most systems have W^X policy; allocate RW, write code, then mprotect to RX

section .text
    global _start

_start:
    ; Step 1: Allocate RW memory
    mov rax, 9                ; sys_mmap
    xor rdi, rdi              ; addr = NULL
    mov rsi, 4096              ; 1 page
    mov rdx, 3                 ; PROT_READ | PROT_WRITE
    mov r10, 34                ; MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall
    mov r12, rax               ; Save pointer in callee-saved register

    ; Step 2: Write machine code into the buffer
    ; We'll write: mov eax, 42; ret (returns 42)
    mov byte [r12], 0xB8       ; mov eax, imm32
    mov dword [r12+1], 42      ; immediate value 42
    mov byte [r12+5], 0xC3     ; ret

    ; Step 3: Change protection to RX (remove write, add execute)
    mov rax, 10                ; sys_mprotect
    mov rdi, r12                ; addr
    mov rsi, 4096               ; length
    mov rdx, 5                  ; PROT_READ | PROT_EXEC
    syscall

    ; Step 4: Call our generated code
    call r12                    ; Call the JIT'd function
    ; RAX now contains 42

    ; Step 5: Exit with the returned value
    mov rdi, rax                ; Exit code = 42
    mov rax, 60
    syscall

```

Part 3: How malloc() Actually Works

The Chunk Structure

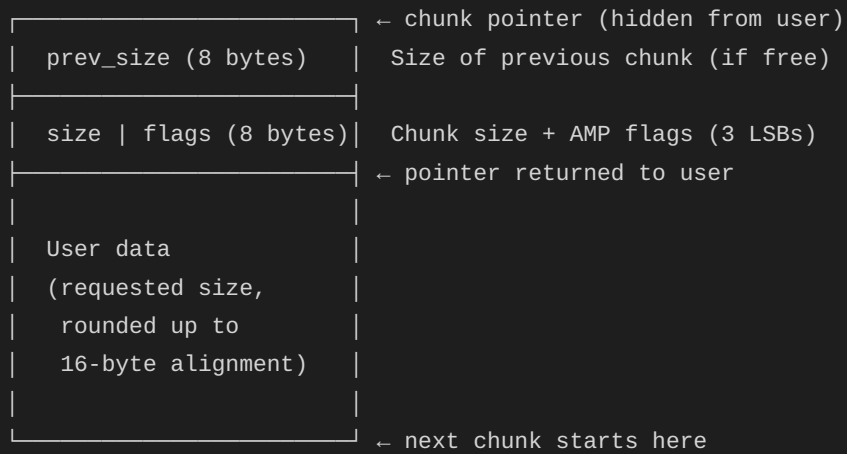
Every `malloc` implementation wraps each allocation in a chunk with metadata. Here's what glibc's `malloc` (ptmalloc2) uses:



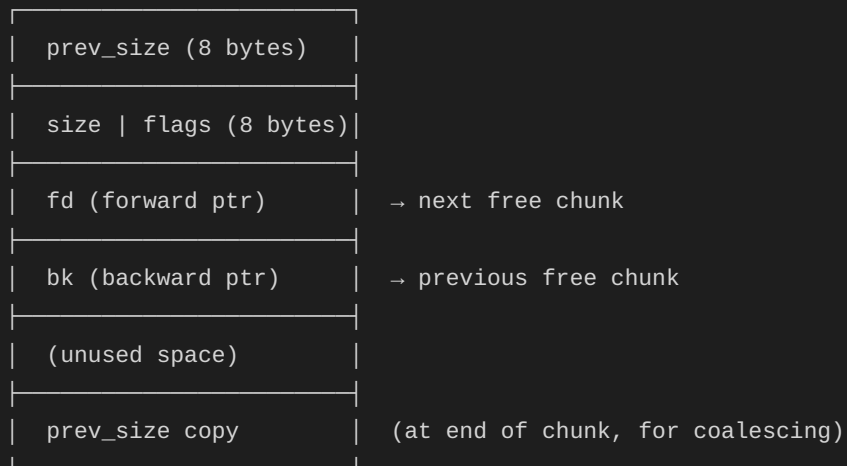
14 B of metadata per allocation; free chunks store fd/bb inside the old payload

ASCII diagram

Allocated chunk:



Free chunk:



Flags in size field (3 least significant bits):

Bit 0 (P): PREV_INUSE - previous chunk is allocated

Bit 1 (M): IS_MMAPPED - chunk was allocated via mmap

Bit 2 (A): NON_MAIN_ARENA - chunk belongs to non-main arena

Implementing a Simple malloc in Assembly

```

; Minimal malloc/free implementation using brk()
; Supports: allocation, free, first-fit free list

section .bss
    heap_base    resq 1      ; Start of our heap
    heap_top     resq 1      ; Current top of heap
    free_list    resq 1      ; Head of free list (singly-linked)

section .text
    global my_malloc
    global my_free
    global malloc_init

; Chunk header structure (16 bytes):
;   [0..7] = size (including header) | flags
;   [8..15] = next_free (only when free; overlaps with user data)
HEADER_SIZE equ 16
FLAG_FREE    equ 1          ; Bit 0: chunk is free
SIZE_MASK    equ ~7         ; Mask off flag bits

; Initialize the allocator
; Call once before using malloc/free
malloc_init:
    push rbx
    ; Get initial program break
    mov rax, 12              ; sys_brk
    xor rdi, rdi
    syscall
    mov [heap_base], rax
    mov [heap_top], rax
    mov qword [free_list], 0 ; Empty free list
    pop rbx
    ret

; my_malloc(size_t size) -> void*
; Input: RDI = requested size
; Output: RAX = pointer to usable memory, or 0 on failure
my_malloc:
    push rbx
    push r12
    push r13

    ; Round up size to 16-byte alignment + header
    add rdi, HEADER_SIZE + 15
    and rdi, ~15             ; Align to 16 bytes
    mov r12, rdi             ; R12 = total chunk size needed

    ; Search free list (first-fit)
    mov rax, [free_list]     ; Current free chunk
    xor rbx, rbx             ; Previous chunk (for unlinking)

```

```

.search_free:
    test rax, rax
    jz .no_free_chunk      ; End of free list

    ; Get chunk size (mask off flags)
    mov rcx, [rax]         ; Load size|flags
    mov rdx, rcx
    and rdx, SIZE_MASK     ; Actual size

    ; Is this chunk big enough?
    cmp rdx, r12
    jge .found_free

    ; Move to next free chunk
    mov rbx, rax           ; prev = current
    mov rax, [rax + 8]     ; current = current->next_free
    jmp .search_free

.found_free:
    ; Unlink from free list
    mov rcx, [rax + 8]     ; next_free of found chunk
    test rbx, rbx
    jz .unlink_head
    mov [rbx + 8], rcx     ; prev->next_free = found->next_free
    jmp .mark_used

.unlink_head:
    mov [free_list], rcx   ; free_list = found->next_free

.mark_used:
    ; Clear free flag, keep size
    mov rcx, [rax]
    and rcx, ~FLAG_FREE    ; Clear free bit
    mov [rax], rcx

    ; Return pointer past header
    add rax, HEADER_SIZE
    pop r13
    pop r12
    pop rbx
    ret

.no_free_chunk:
    ; Extend heap using brk()
    mov rax, 12            ; sys_brk
    xor rdi, rdi           ; Get current break
    syscall
    mov r13, rax           ; R13 = current break (chunk start)

    ; Set new break
    lea rdi, [rax + r12]   ; New break = current + chunk_size

```

```

mov rax, 12          ; sys_brk
syscall

; Verify success
lea rcx, [r13 + r12]
cmp rax, rcx
jl .alloc_fail      ; brk didn't move enough

; Write chunk header
mov [r13], r12       ; size (no flags set = allocated)
mov [heap_top], rax  ; Update heap top

; Return pointer past header
lea rax, [r13 + HEADER_SIZE]
pop r13
pop r12
pop rbx
ret

.alloc_fail:
xor rax, rax        ; Return NULL
pop r13
pop r12
pop rbx
ret

; my_free(void *ptr)
; Input: RDI = pointer previously returned by my_malloc
my_free:
test rdi, rdi
jz .free_done       ; free(NULL) is a no-op

; Get chunk header (ptr - HEADER_SIZE)
sub rdi, HEADER_SIZE

; Set free flag
mov rax, [rdi]
or rax, FLAG_FREE
mov [rdi], rax

; Prepend to free list
mov rax, [free_list]
mov [rdi + 8], rax   ; chunk->next_free = old head
mov [free_list], rdi ; free_list = chunk

.free_done:
ret

```

How glibc malloc Decides: brk vs mmap

```

; Pseudocode of glibc's malloc decision in assembly form:
;
; if (size >= MMAP_THRESHOLD) {      ; typically 128KB
;     return mmap_alloc(size);
; }
; if (free_list has suitable chunk) {
;     return reuse_chunk();
; }
; return brk_alloc(size);

; This shows the mmap path for large allocations:
mmap_alloc:
    ; Input: RDI = size needed
    push rbx
    mov rbx, rdi

    ; Add space for mmap header (stores size for munmap)
    add rbx, 32          ; 16 header + 16 alignment padding
    add rbx, 4095
    and rbx, ~4095      ; Round up to page boundary

    ; mmap anonymous memory
    mov rax, 9           ; sys_mmap
    xor rdi, rdi         ; addr = NULL
    mov rsi, rbx         ; length (page-aligned)
    mov rdx, 3           ; PROT_READ | PROT_WRITE
    mov r10, 34          ; MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall

    cmp rax, -4096
    ja .mmap_fail

    ; Store metadata: total mapped size and IS_MMAPPED flag
    mov qword [rax], 0    ; prev_size = 0
    mov rcx, rbx
    or rcx, 2            ; Set IS_MMAPPED flag (bit 1)
    mov [rax + 8], rcx    ; size | IS_MMAPPED

    ; Return usable pointer
    add rax, 16
    pop rbx
    ret

.mmap_fail:
    xor rax, rax
    pop rbx
    ret

```

```

; Free an mmap'd chunk: just munmap it
mmap_free:
    ; Input: RDI = user pointer
    sub rdi, 16          ; Back to chunk start
    mov rsi, [rdi + 8]   ; Get size
    and rsi, SIZE_MASK   ; Mask off flags
    mov rax, 11          ; sys_munmap
    syscall
    ret

```

Part 4: Free List Strategies

Understanding Fragmentation

After many malloc/free cycles:

Heap: [USED 32][FREE 16][USED 64][FREE 48][USED 16][FREE 24][USED 128]

```

      ↓           ↓           ↓
free_list → [16 bytes] → [48 bytes] → [24 bytes] → NULL

```

Problem: Request for 80 bytes FAILS even though total free = 88 bytes!

This is "external fragmentation"

Bin Organization (glibc approach)


```

; glibc organizes free chunks into bins by size:
;
; Fast bins (sizes 16-80, step 16):
;   Single-linked LIFO lists, never coalesced
;   bin[0] = 16-byte chunks
;   bin[1] = 32-byte chunks
;   bin[2] = 48-byte chunks
;   bin[3] = 64-byte chunks
;   bin[4] = 80-byte chunks
;
; Small bins (sizes 96-512, step 16):
;   Doubly-linked FIFO lists
;
; Large bins (sizes > 512):
;   Sorted by size, doubly-linked
;
; Unsorted bin:
;   Recently freed chunks awaiting sorting

; Fast bin lookup (given a size):
; bin_index = (size >> 4) - 1
; For size=32: index = (32 >> 4) - 1 = 1

section .bss
    fastbins resq 5          ; 5 fast bin heads (16,32,48,64,80)

; Fast bin free (O(1)):
; Chunks go to head of appropriate bin
fastbin_free:
    ; RDI = chunk pointer (with header)
    mov rax, [rdi]           ; Get chunk size
    and rax, SIZE_MASK
    shr rax, 4
    dec rax                  ; bin index
    lea rcx, [fastbins]

    ; Push onto bin's free list (single-linked stack)
    mov rdx, [rcx + rax*8] ; Old head
    mov [rdi + 8], rdx      ; chunk->next = old head
    mov [rcx + rax*8], rdi ; bin[i] = chunk
    ret

; Fast bin malloc (O(1)):
fastbin_malloc:
    ; RDI = requested size (already aligned)
    mov rax, rdi
    shr rax, 4
    dec rax                  ; bin index
    lea rcx, [fastbins]

```

```
; Pop from bin's free list
mov rdx, [rcx + rax*8] ; Head of bin
test rdx, rdx
jz .fastbin_empty      ; No cached chunks

mov rsi, [rdx + 8]      ; next = head->next
mov [rcx + rax*8], rsi ; bin[i] = next

; Return chunk (clear free flag if needed)
lea rax, [rdx + HEADER_SIZE]
ret

.fastbin_empty:
xor rax, rax            ; Fall through to slower path
ret
```

Chunk Coalescing (Merging Adjacent Free Chunks)

```

; When free() is called, check if adjacent chunks are also free
; and merge them to reduce fragmentation

; Given: RDI = chunk being freed
coalesce:
    push rbx
    push r12
    mov r12, rdi

    ; Check if NEXT chunk is free
    mov rax, [r12]          ; Our size
    and rax, SIZE_MASK
    lea rbx, [r12 + rax]    ; Next chunk address

    ; Verify next chunk is within heap bounds
    cmp rbx, [heap_top]
    jge .check_prev

    mov rcx, [rbx]          ; Next chunk's size|flags
    test rcx, FLAG_FREE     ; Is it free?
    jz .check_prev

    ; Merge with next chunk: our_size += next_size
    and rcx, SIZE_MASK
    mov rax, [r12]
    and rax, SIZE_MASK
    add rax, rcx
    or rax, FLAG_FREE      ; Keep free flag
    mov [r12], rax         ; Update our size

    ; Remove next chunk from free list
    ; (simplified: would need to search and unlink)

.check_prev:
    ; Check if PREVIOUS chunk is free (using prev_size field)
    ; Bit 0 (PREV_INUSE) of our size tells us
    mov rax, [r12]
    test rax, 1            ; PREV_INUSE flag
    jnz .done              ; Previous is in use, can't merge

    ; Previous is free: merge backward
    ; prev_size field tells us previous chunk's size
    mov rcx, [r12 - 8]     ; prev_size (stored at end of prev chunk)
    sub r12, rcx           ; Back up to previous chunk start

    ; Combine sizes
    mov rax, [r12]
    and rax, SIZE_MASK
    mov rdx, [r12]         ; Reload (size of prev)
    and rdx, SIZE_MASK

```

```

    add rax, rdx          ; Combined size
    or  rax, FLAG_FREE
    mov [r12], rax

.done:
    mov rax, r12          ; Return merged chunk address
    pop r12
    pop rbx
    ret

```

Part 5: What Happens at Page Fault

When `brk()` or `mmap()` returns successfully, the kernel hasn't actually allocated physical memory yet. It only updates the page tables to mark the virtual addresses as valid. Physical pages are allocated on **first access** via the page fault mechanism.

Timeline of a malloc'd byte being used:

1. `malloc(64)` returns `0x55555576a2a0`
 - `brk()` expanded virtual address space
 - Page table entry: Valid=0, Present=0
 - NO physical memory used yet!
2. `mov byte [0x55555576a2a0], 'A'` ← First write
 - CPU walks page table, finds page not present
 - TRIGGERS PAGE FAULT (exception #14)
 - Kernel page fault handler:
 - a. Checks VMA: is this address valid? YES (within `brk` range)
 - b. Allocates a physical page (4KB) from buddy allocator
 - c. Zeros the page (security: don't leak other process data)
 - d. Updates page table: PTE points to physical page, Present=1
 - e. Returns to user code – instruction is RETRIED
 - Write succeeds on retry
3. `mov byte [0x55555576a2a1], 'B'` ← Same page, no fault
 - Page already present, direct access

```

; Demonstrating demand paging:
; Allocate 1MB but only touch 1 page – only 4KB physical memory used

section .text
    global _start

_start:
    ; Allocate 1MB (256 pages)
    mov rax, 9                ; sys_mmap
    xor rdi, rdi
    mov rsi, 1048576          ; 1MB
    mov rdx, 3                ; PROT_READ | PROT_WRITE
    mov r10, 34               ; MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall
    mov r12, rax              ; Save base

    ; Only touch first byte – only 1 physical page allocated
    mov byte [r12], 42

    ; The other 255 pages remain un-faulted
    ; Physical memory used: ~4KB, not 1MB!

    ; Now touch last page – second page fault, second physical page
    mov byte [r12 + 1048575], 99

    ; Physical memory used: ~8KB (2 pages)

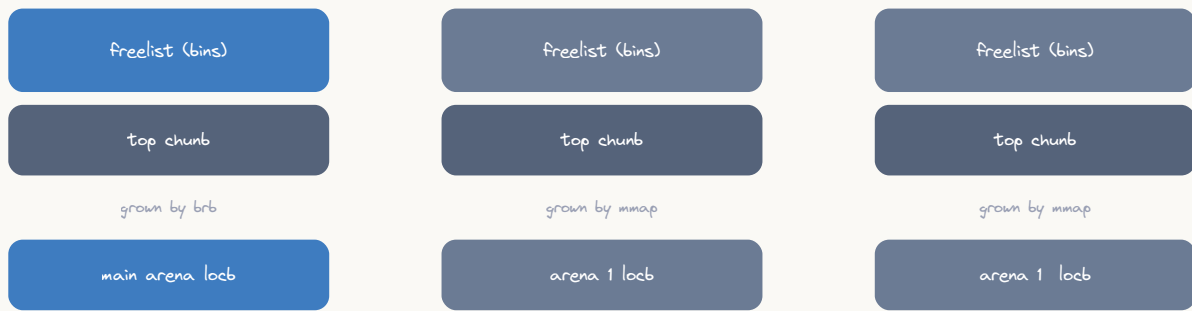
    mov rax, 60
    xor rdi, rdi
    syscall

```

Part 6: Thread-Local Allocation (Arenas)

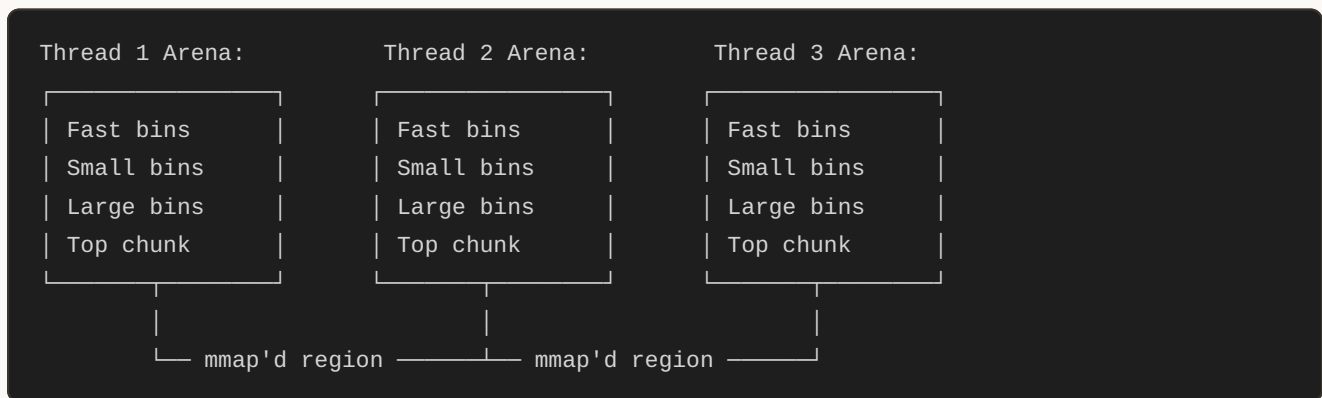
In multi-threaded programs, a single lock on the heap would be a bottleneck. Modern allocators use per-thread arenas:

Arenas=per-thread heaps cut lock contention



a thread pins to an arena; many arenas can balloon eSZ far past the working set

ASCII diagram



```
; Per-thread allocation using thread-local storage (TLS)
; Each thread has its own cache of free chunks

; Using the arch_prctl syscall to access thread-local data:
; or using FS segment register (Linux TLS convention)

; Read TLS base:
; mov rax, [fs:0] ; TLS base address

; Thread-local free list (conceptual):
; struct thread_cache {
;     void *free_list[NUM_BINS];
;     size_t cached_count;
; };

; Accessing thread-local allocator cache:
get_thread_cache:
    ; The FS register points to thread control block (TCB)
    ; TLS variables are at negative offsets from FS base
    mov rax, [fs:-8]      ; Our thread_cache pointer
    ret
```

Part 7: Memory Alignment

Why Alignment Matters

```

; Aligned access: address is multiple of data size
mov rax, [rbp-8]          ; Address ending in 0 or 8: aligned QWORD

; Unaligned access: address NOT multiple of data size
mov rax, [rbp-5]          ; Address ending in 5: unaligned!
; Works on x86 but may be:
;   - Split across cache lines (2x memory access)
;   - Split across pages (very expensive, may fault on some CPUs)

; malloc guarantees 16-byte alignment (glibc on 64-bit):
; Every returned pointer is a multiple of 16
; This ensures SSE/AVX instructions work:
movaps xmm0, [rax]        ; REQUIRES 16-byte alignment (faults otherwise)
movups xmm0, [rax]        ; Unaligned version (slower but safe)

; Custom aligned allocation using mmap:
aligned_alloc_page:
    ; Allocate on page boundary (4096-byte alignment)
    mov rax, 9
    xor rdi, rdi           ; NULL = kernel chooses page-aligned addr
    mov rsi, 4096
    mov rdx, 3             ; RW
    mov r10, 34            ; PRIVATE | ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall
    ; RAX is guaranteed page-aligned (multiple of 4096)
    ret

```

posix_memalign / aligned_alloc Implementation

```

; Allocate with custom alignment
; Input: RDI = alignment (must be power of 2), RSI = size
; Output: RAX = aligned pointer
my_aligned_alloc:
    push rbx
    push r12
    mov r12, rdi        ; alignment
    mov rbx, rsi        ; size

    ; Allocate extra space: size + alignment + header
    lea rdi, [rbx + r12 + HEADER_SIZE]
    call my_malloc      ; Get unaligned block
    test rax, rax
    jz .aligned_fail

    ; Find aligned address within the block
    mov rcx, rax
    add rcx, r12
    dec rcx
    neg r12              ; Create alignment mask
    and rcx, r12         ; Round up to alignment

    ; Store original pointer just before aligned address (for free)
    mov [rcx - 8], rax   ; Stash real pointer

    mov rax, rcx
    pop r12
    pop rbx
    ret

.aligned_fail:
    xor rax, rax
    pop r12
    pop rbx
    ret

```

Part 8: Debugging Memory Issues

Detecting Use-After-Free

```

; Secure free: overwrite memory to catch use-after-free
secure_free:
    ; RDI = pointer to free
    test rdi, rdi
    jz .done

    ; Get chunk size from header
    mov rax, [rdi - HEADER_SIZE]
    and rax, SIZE_MASK
    sub rax, HEADER_SIZE    ; Usable size

    ; Fill with poison pattern (0xDEADBEEF...)
    push rdi
    push rcx
    mov rcx, rax
    shr rcx, 3              ; Count in qwords
    mov rax, 0xDEADBEEFDEADBEEF
    rep stosq               ; Fill with poison
    pop rcx
    pop rdi

    ; Now actually free
    call my_free
.done:
    ret

; Detecting double-free:
safe_free:
    ; RDI = pointer
    test rdi, rdi
    jz .ok

    sub rdi, HEADER_SIZE
    mov rax, [rdi]
    test rax, FLAG_FREE     ; Already free?
    jnz .double_free        ; BUG! Double free detected

    ; Normal free path
    add rdi, HEADER_SIZE
    call my_free
.ok:
    ret

.double_free:
    ; Print error and abort
    mov rax, 1              ; sys_write
    mov rdi, 2              ; stderr
    lea rsi, [rel .errmsg]
    mov rdx, .errlen
    syscall

```

```
mov rax, 60
mov rdi, 134      ; SIGABRT exit code
syscall

section .rodata
.errmsg: db "*** DOUBLE FREE DETECTED ***", 10
.errlen equ $ - .errmsg
```

Memory Layout Inspection with /proc/self/maps

```

; Read our own memory map (equivalent to cat /proc/self/maps)
section .data
    maps_path db "/proc/self/maps", 0

section .bss
    maps_buf resb 4096

section .text
print_memory_map:
    ; Open /proc/self/maps
    mov rax, 2                ; sys_open
    lea rdi, [rel maps_path]
    xor rsi, rsi              ; 0_RDONLY
    xor rdx, rdx
    syscall
    mov r12, rax              ; fd

    ; Read and print
.read_loop:
    mov rax, 0                ; sys_read
    mov rdi, r12
    lea rsi, [rel maps_buf]
    mov rdx, 4096
    syscall
    test rax, rax
    jle .read_done

    ; Write to stdout
    mov rdx, rax              ; bytes read
    mov rax, 1                ; sys_write
    mov rdi, 1                ; stdout
    lea rsi, [rel maps_buf]
    syscall
    jmp .read_loop

.read_done:
    ; Close file
    mov rax, 3                ; sys_close
    mov rdi, r12
    syscall
    ret

```

Exercises

1. **Basic brk allocator:** Write a program that allocates 3 separate buffers using `brk()`, writes strings to each, prints them all, then exits.

2. **mmap allocator:** Allocate 10 pages with mmap, use only pages 0, 4, and 9. Use `strace` to observe page faults.
3. **Free list malloc:** Extend the simple malloc implementation to support chunk splitting (when a free chunk is much larger than requested, split it).
4. **Memory pool:** Implement a fixed-size object pool (all allocations are same size) — this is what kernel slab allocators do.
5. **Fragmentation demo:** Write a program that alternately allocates and frees chunks of different sizes, then attempts a large allocation that fails due to fragmentation. Demonstrate how coalescing fixes it.

Key Takeaways

Concept	Assembly Reality
<code>malloc(n)</code>	Check free list → brk/mmap → return chunk+16
<code>free(p)</code>	Add chunk to free list (maybe coalesce)
Small allocs	brk() extends heap linearly
Large allocs	mmap() creates independent mapping
Demand paging	Physical pages allocated on first access (page fault)
Alignment	malloc returns 16-byte aligned; page boundary = 4096
Thread safety	Per-thread arenas avoid lock contention

Next Topic

[Topic 22: Process Internals](#) → — How fork(), exec(), and process creation work at the assembly level.

CHAPTER 39

Topic 22: Process Internals

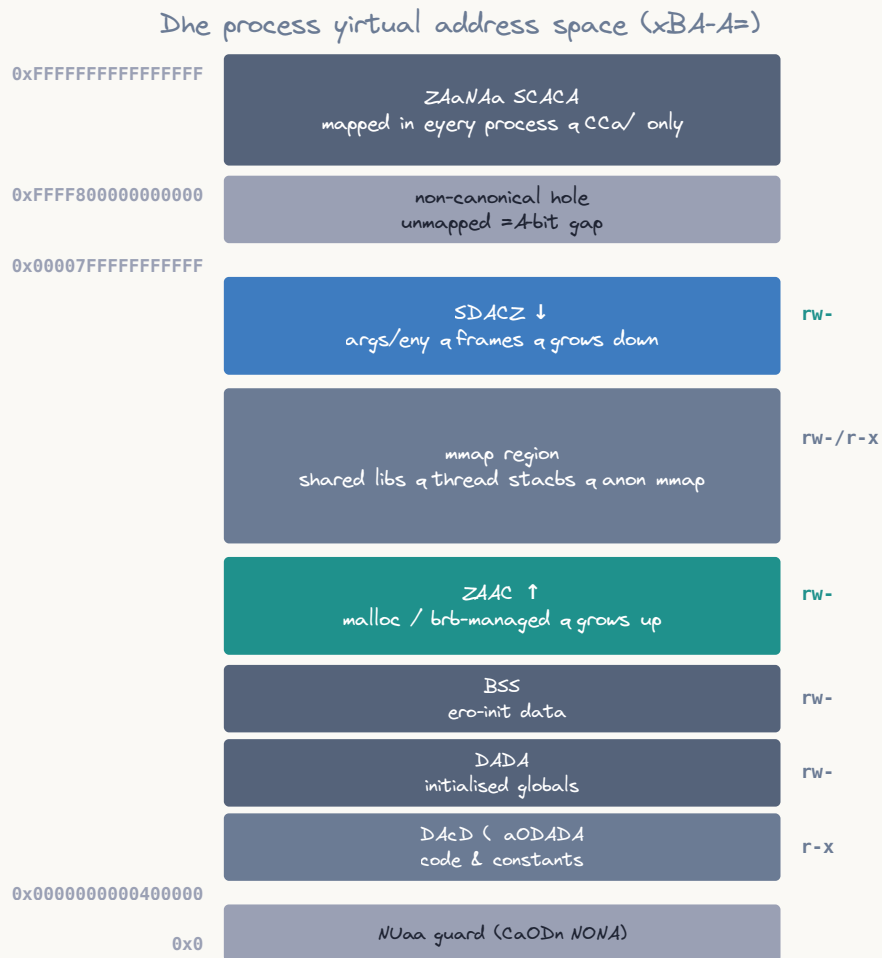
Overview

Every running program on Linux is a **process** — an isolated instance with its own virtual address space, file descriptors, and execution state. This topic examines how processes are created, how `fork()` duplicates a process, how `execve()` replaces a process image, and how the kernel manages all of this — explained entirely at the assembly level.

```
// What happens in C:
pid_t pid = fork();
if (pid == 0) {
    // Child process
    execve("/bin/ls", argv, envp);
} else {
    // Parent process
    wait(&status);
}

// Under the hood:
// 1. fork() → sys_clone/sys_fork → kernel duplicates process descriptor
// 2. Copy-on-write page tables are set up (no actual memory copy!)
// 3. Child gets PID 0 return, parent gets child's PID
// 4. execve() → kernel loads new ELF binary, replaces address space
// 5. wait() → parent sleeps until child changes state
```

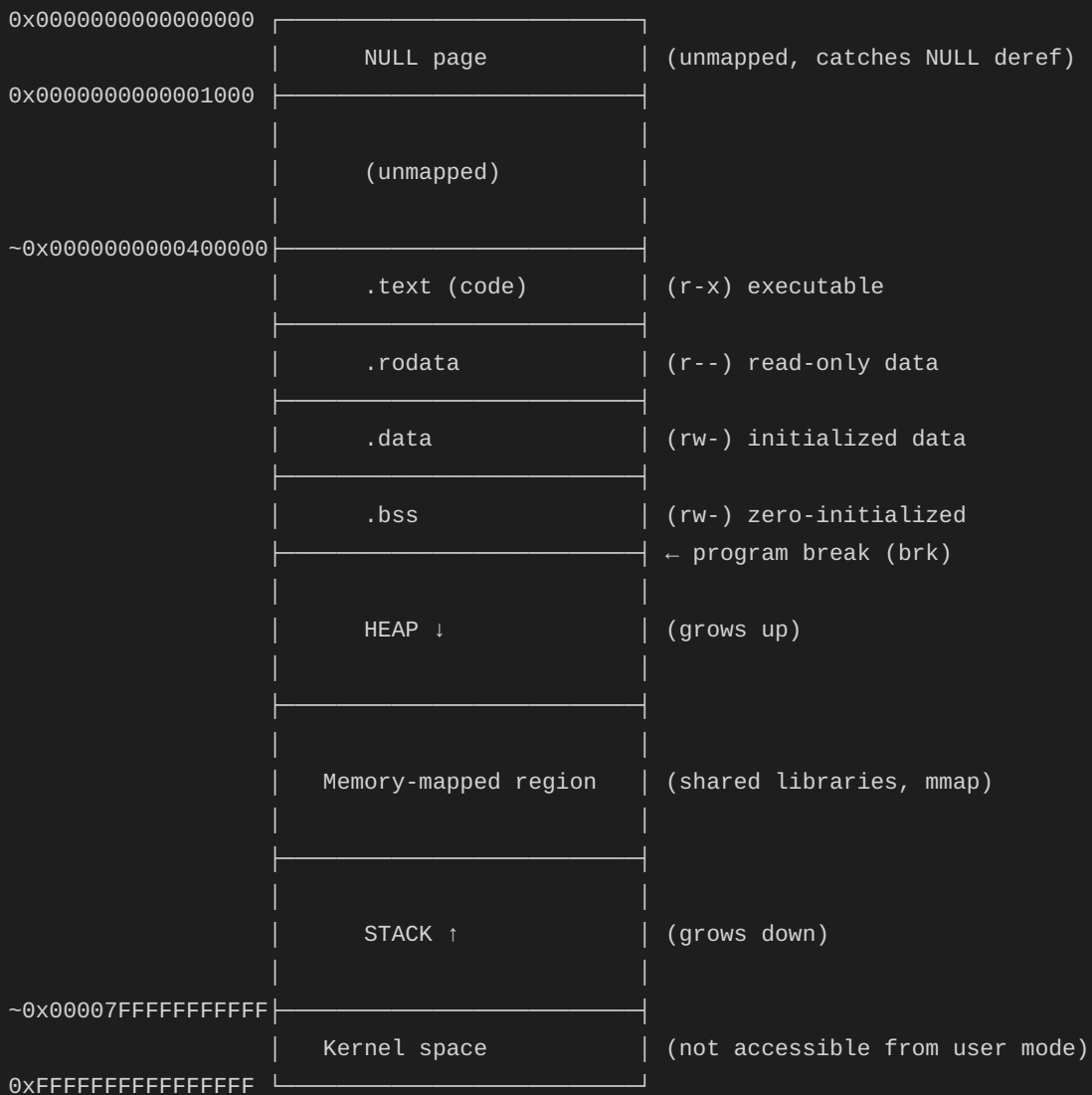

Part 1: Process Memory Layout



virtual ≠ physical & memory is la y & the b b U maps pages to frames

ASCII diagram

Process Virtual Address Space (x86-64 Linux):



Examining Our Own Process

```

section .text
    global _start

_start:
    ; Get our PID
    mov rax, 39          ; sys_getpid
    syscall
    mov r12, rax         ; Save PID

    ; Get parent PID
    mov rax, 110         ; sys_getppid
    syscall
    mov r13, rax         ; Save PPID

    ; Get user ID
    mov rax, 102         ; sys_getuid
    syscall
    mov r14, rax         ; Save UID

    ; Print PID (convert to string first)
    mov rdi, r12
    call print_number

    mov rax, 60
    xor rdi, rdi
    syscall

; Convert integer to string and print
; Input: RDI = number to print
print_number:
    push rbp
    mov rbp, rsp
    sub rsp, 32          ; Buffer on stack

    lea rsi, [rbp-1]      ; Point to end of buffer
    mov byte [rsi], 10     ; Newline
    mov rcx, 1            ; Length counter

    mov rax, rdi          ; Number to convert
    mov r8, 10            ; Divisor

.digit_loop:
    xor rdx, rdx
    div r8                ; RAX = quotient, RDX = remainder
    add dl, '0'           ; Convert to ASCII
    dec rsi
    mov [rsi], dl
    inc rcx
    test rax, rax
    jnz .digit_loop

```

```
; Write the string
mov rax, 1           ; sys_write
mov rdi, 1           ; stdout
mov rdx, rcx         ; length
syscall

leave
ret
```

Part 2: fork() — Process Duplication

The fork() Syscall

```
; fork() creates an exact copy of the current process
; Returns: child PID to parent, 0 to child, -1 on error
```

```
section .data
```

```
parent_msg db "I am the parent. Child PID: "
parent_len equ $ - parent_msg
child_msg db "I am the child!", 10
child_len equ $ - child_msg
newline db 10
```

```
section .text
```

```
global _start
```

```
_start:
```

```
; Fork!
mov rax, 57 ; sys_fork
syscall

; After fork, both parent and child execute here
; RAX = 0 in child, > 0 (child PID) in parent

test rax, rax
jz .child ; RAX == 0 → we're the child
js .fork_error ; RAX < 0 → error
```

```
.parent:
```

```
; We're the parent, RAX = child's PID
mov r12, rax ; Save child PID

; Print parent message
mov rax, 1
mov rdi, 1
lea rsi, [rel parent_msg]
mov rdx, parent_len
syscall

; Print child PID
mov rdi, r12
call print_number

; Wait for child to finish
mov rax, 61 ; sys_wait4
mov rdi, -1 ; Wait for any child
xor rsi, rsi ; status = NULL
xor rdx, rdx ; options = 0
xor r10, r10 ; rusage = NULL
syscall

; Exit parent
mov rax, 60
```

```

    xor rdi, rdi
    syscall

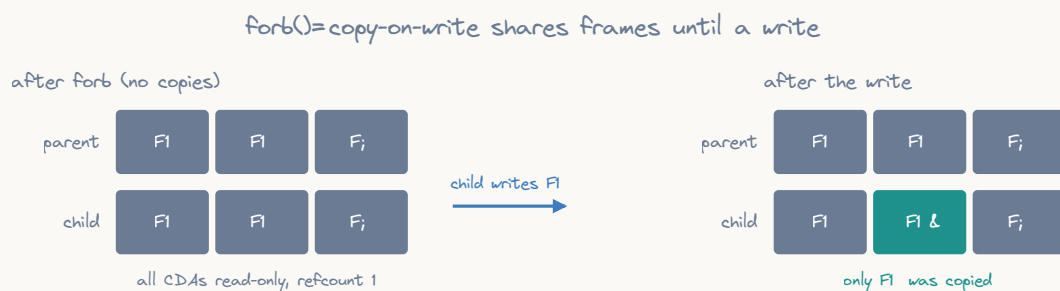
.child:
    ; We're the child
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel child_msg]
    mov rdx, child_len
    syscall

    ; Exit child
    mov rax, 60
    xor rdi, rdi
    syscall

.fork_error:
    mov rax, 60
    mov rdi, 1
    syscall

```

What fork() Actually Does in the Kernel



ASCII diagram

Before fork():

Parent Process:

PID: 1000
PPID: 999
Registers:
RAX = (syscall)
RSP = 0x7fff...
RIP = next_inst
Page Tables:
VA→PA mapping
(read/write)
File Descriptors:
0: stdin
1: stdout
2: stderr

COW →

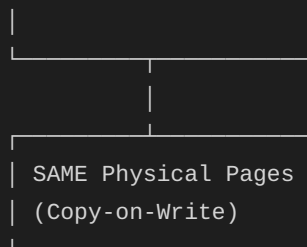
After fork():

Parent Process:

PID: 1000
PPID: 999
Registers:
RAX = 1001
RSP = 0x7fff...
RIP = next_inst
Page Tables:
VA→PA (R/O)
Same physical
pages!
FDS: 0,1,2,...
(same as before)

Child Process:

PID: 1001
PPID: 1000
Registers:
RAX = 0
RSP = 0x7fff...
RIP = next_inst
Page Tables:
VA→PA (R/O)
Same physical
pages!
FDS: 0,1,2,...
(duplicated)



Copy-on-Write (COW) Mechanism

```

; Copy-on-Write demonstration:
; After fork, parent and child share physical pages
; Pages are marked read-only in both
; On first WRITE, a page fault occurs and the kernel copies the page

section .data
    shared_data dq 42          ; This data is shared after fork (COW)

section .text
    global _start

_start:
    mov rax, 57                ; sys_fork
    syscall
    test rax, rax
    jz .child

.parent:
    ; Parent modifies shared_data
    ; This triggers a COW page fault:
    ; 1. CPU tries to write to read-only page
    ; 2. Page fault → kernel checks: is this a COW page? YES
    ; 3. Kernel allocates new physical page
    ; 4. Copies content from original page
    ; 5. Updates parent's page table to point to new page (now RW)
    ; 6. Returns to user code, write succeeds
    mov qword [rel shared_data], 100    ; ← COW fault here!

    ; Wait for child
    mov rax, 61
    mov rdi, -1
    xor rsi, rsi
    xor rdx, rdx
    xor r10, r10
    syscall

    mov rax, 60
    xor rdi, rdi
    syscall

.child:
    ; Child reads shared_data – no COW fault (read is fine)
    mov rax, [rel shared_data] ; Still reads 42!

    ; Child modifies – triggers its OWN COW fault
    mov qword [rel shared_data], 200    ; ← COW fault here!
    ; Now child has its own private copy of this page

    mov rax, 60

```

```
xor rdi, rdi  
syscall
```

Part 3: clone() — The Real Process Creation

fork() is Actually clone()

On modern Linux, `fork()` is implemented as a wrapper around `clone()`:

```

; clone() is the underlying syscall for fork(), vfork(), and pthread_create()
;
; long clone(unsigned long flags, void *stack,
;           int *parent_tid, int *child_tid,
;           unsigned long tls);
;
; Syscall number: 56 (sys_clone)
; RDI = flags
; RSI = child_stack (NULL = share parent's stack → fork behavior)
; RDX = parent_tid pointer
; R10 = child_tid pointer
; R8 = TLS descriptor

; fork() equivalent using clone():
fork_via_clone:
    mov rax, 56          ; sys_clone
    mov rdi, 17          ; SIGCHLD (= flags for basic fork)
    xor rsi, rsi         ; child_stack = NULL (use parent's)
    xor rdx, rdx         ; parent_tid = NULL
    xor r10, r10         ; child_tid = NULL
    xor r8, r8           ; tls = NULL
    syscall
    ret

; Creating a thread (shares everything except stack):
; CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD |
; CLONE_SYSVSEM | CLONE_SETTLS | CLONE_PARENT_SETTID | CLONE_CHILD_CLEARTID
CLONE_THREAD_FLAGS equ 0x3D0F00 ; Combined thread flags

create_thread:
    ; Input: RDI = function pointer, RSI = argument
    push rbx
    push r12
    push r13
    mov r12, rdi         ; Save function
    mov r13, rsi         ; Save argument

    ; Allocate stack for new thread (8KB)
    mov rax, 9           ; sys_mmap
    xor rdi, rdi
    mov rsi, 8192        ; 8KB stack
    mov rdx, 3           ; PROT_READ | PROT_WRITE
    mov r10, 34          ; MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall
    mov rbx, rax         ; Stack base

    ; Stack grows down, so point to top
    lea rsi, [rbx + 8192 - 8]

```

```
; Put function address and arg on new stack  
; (child will pop these after clone returns)  
mov [rsi], r12      ; function pointer  
mov [rsi - 8], r13   ; argument  
sub rsi, 16  
  
; Clone with thread flags  
mov rax, 56         ; sys_clone  
mov rdi, CLONE_THREAD_FLAGS  
; RSI already = child stack top  
xor rdx, rdx  
xor r10, r10  
xor r8, r8  
syscall  
  
; In parent: RAX = child TID  
; In child: RAX = 0, starts executing at return address on new stack  
  
pop r13  
pop r12  
pop rbx  
ret
```

Part 4: `execve()` — Replacing the Process Image

How `execve()` Works

```

; execve() replaces the ENTIRE process image with a new program
; The current code, data, heap, and stack are all destroyed
; Only PID and open file descriptors (without CLOEXEC) survive

; long execve(const char *filename, char *const argv[],
;             char *const envp[]);

section .data
    prog_path db "/bin/ls", 0
    arg0      db "ls", 0
    arg1      db "-la", 0
    arg2      db "/tmp", 0

; argv array (array of pointers, NULL-terminated)
    argv      dq arg0, arg1, arg2, 0

; envp array (array of "KEY=VALUE" strings, NULL-terminated)
    env0      db "PATH=/usr/bin:/bin", 0
    env1      db "HOME=/root", 0
    envp      dq env0, env1, 0

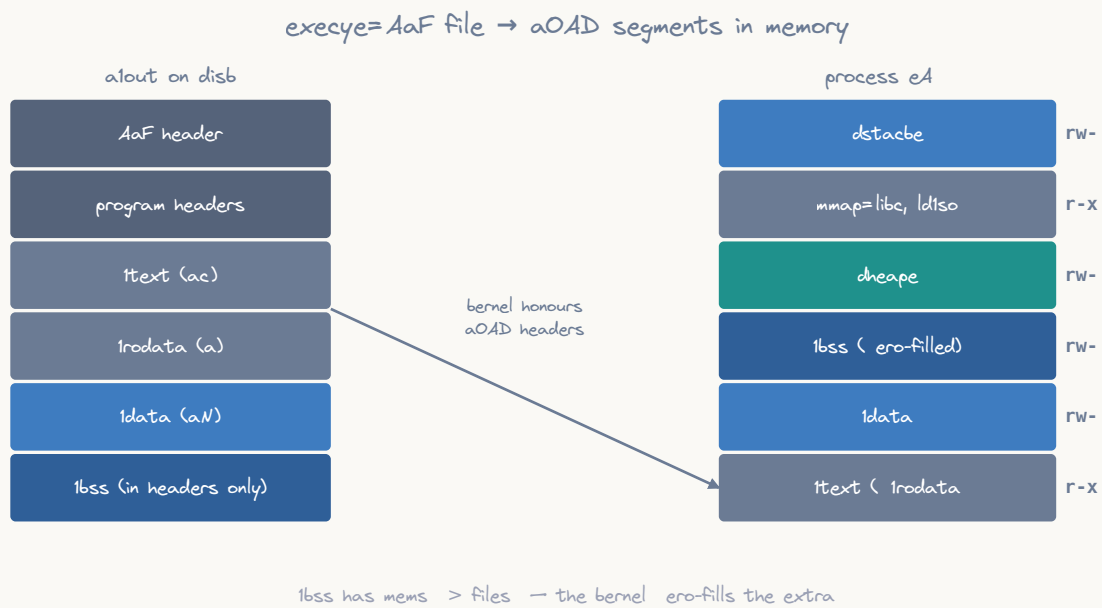
section .text
    global _start

_start:
    ; Execute /bin/ls -la /tmp
    mov rax, 59          ; sys_execve
    lea rdi, [rel prog_path] ; filename
    lea rsi, [rel argv]    ; argv[]
    lea rdx, [rel envp]    ; envp[]
    syscall

; If execve returns, it FAILED (only returns on error)
; RAX = -errno
    neg rax              ; Make positive errno
    mov rdi, rax
    mov rax, 60          ; sys_exit
    syscall

```

What the Kernel Does During `execve()`



ASCII diagram


```
execve("/bin/ls", argv, envp) kernel path:
```

1. Open the file, read ELF header

```
| Check: is it executable? |
| Check: do we have permission? |
| Read: ELF magic (7f 45 4c 46) |
| Parse: program headers (PT_LOAD) |
```

2. Destroy old address space

```
| Unmap all user pages |
| Release old page tables |
| Close CLOEXEC file descriptors |
| Reset signal handlers to default |
```

3. Set up new address space

```
| Map .text segment (R-X) |
| Map .data segment (RW-) |
| Map .bss (zero pages, RW-) |
| Map interpreter (ld-linux.so) |
| Set up new stack: |
|   [top of stack] |
|   platform string |
|   random bytes (16 for AT_RANDOM) |
|   environment strings |
|   argument strings |
|   padding |
|   auxv[] (auxiliary vector) |
|   NULL |
|   envp[n]=NULL |
|   envp[1] |
|   envp[0] |
|   NULL |
|   argv[argc-1] |
|   ... |
|   argv[0] |
|   argc |
|   [bottom = initial RSP] |
```

4. Set registers and jump

```
| RSP = top of new stack |
| RIP = entry point (from ELF) |
| All other registers = 0 |
```

```
| Return to user mode |
```

The Initial Stack After `execve`

```

; When _start runs after execve, the stack looks like this:
; RSP → [argc]           (8 bytes)
;     [argv[0]]          (pointer to first arg string)
;     [argv[1]]          (pointer to second arg string)
;     ...
;     [NULL]             (argv terminator)
;     [envp[0]]          (pointer to first env string)
;     [envp[1]]          (pointer to second env string)
;     ...
;     [NULL]             (envp terminator)
;     [auxv[0].type]      (auxiliary vector entry)
;     [auxv[0].value]
;     ...
;     [AT_NULL, 0]        (auxv terminator)

```

```

section .text
    global _start

```

```

_start:
    ; Read argc
    mov rdi, [rsp]           ; argc

    ; Read argv[0] (program name)
    mov rsi, [rsp + 8]       ; argv[0] pointer

    ; Find envp (skip past argv + NULL)
    lea rbx, [rsp + 8]       ; Start of argv array
    mov rcx, rdi             ; argc
    inc rcx                  ; +1 for NULL terminator
    lea rbx, [rbx + rcx*8]    ; rbx = &envp[0]

    ; Print each environment variable
.print_env:
    mov rsi, [rbx]
    test rsi, rsi
    jz .done_env

    ; Calculate string length
    mov rdi, rsi
    xor rcx, rcx
.strlen:
    cmp byte [rdi + rcx], 0
    je .got_len
    inc rcx
    jmp .strlen
.got_len:
    ; Print it
    mov rax, 1               ; sys_write
    mov rdi, 1               ; stdout
    mov rdx, rcx             ; length

```

```
syscall

; Newline
push 10
mov rax, 1
mov rdi, 1
mov rsi, rsp
mov rdx, 1
syscall
pop rax

add rbx, 8          ; Next envp entry
jmp .print_env

.done_env:
mov rax, 60
xor rdi, rdi
syscall
```

Auxiliary Vector (auxv)

```

; The auxiliary vector provides information from the kernel:
; AT_PHDR (3)      = Program headers address
; AT_PHENT (4)     = Size of program header entry
; AT_PHNUM (5)     = Number of program headers
; AT_PAGESZ (6)    = System page size (usually 4096)
; AT_BASE (7)     = Interpreter base address
; AT_ENTRY (9)     = Program entry point
; AT_UID (11)      = Real user ID
; AT_EUID (12)     = Effective user ID
; AT_GID (13)      = Real group ID
; AT_EGID (14)     = Effective group ID
; AT_RANDOM (25)   = Address of 16 random bytes
; AT_NULL (0)      = End of auxv

; Reading auxiliary vector:
read_auxv:
    ; After envp, find auxv
    ; (assumes RBX points past envp NULL)
    add rbx, 8                ; Skip envp NULL terminator

.auxv_loop:
    mov rax, [rbx]           ; type
    test rax, rax
    jz .auxv_done            ; AT_NULL = end

    mov rcx, [rbx + 8]       ; value

    ; Check for specific entries
    cmp rax, 6               ; AT_PAGESZ?
    je .got_pagesize
    cmp rax, 25              ; AT_RANDOM?
    je .got_random

    add rbx, 16              ; Next entry (type + value)
    jmp .auxv_loop

.got_pagesize:
    mov [page_size], rcx
    add rbx, 16
    jmp .auxv_loop

.got_random:
    mov [random_addr], rcx
    add rbx, 16
    jmp .auxv_loop

.auxv_done:
    ret

section .bss

```

```
page_size    resq 1  
random_addr  resq 1
```

Part 5: fork + exec Pattern (Spawning a Program)

```

; The standard Unix pattern: fork() then execve()
; Parent creates child, child becomes new program

section .data
    shell      db "/bin/sh", 0
    sh_arg0    db "sh", 0
    sh_arg1    db "-c", 0
    sh_arg2    db "echo Hello from child process! PID=$$", 0
    sh_argv    dq sh_arg0, sh_arg1, sh_arg2, 0
    sh_envp    dq 0

    wait_msg   db "Child exited with status: "
    wait_len   equ $ - wait_msg

section .bss
    child_status resd 1

section .text
    global _start

_start:
    ; Fork
    mov rax, 57          ; sys_fork
    syscall
    test rax, rax
    jz .child
    js .error

.parent:
    mov r12, rax         ; Save child PID

    ; Wait for child
    mov rax, 61          ; sys_wait4
    mov rdi, r12         ; Wait for specific child
    lea rsi, [rel child_status]
    xor rdx, rdx         ; options = 0
    xor r10, r10         ; rusage = NULL
    syscall

    ; Extract exit status: bits 15-8 of status word
    mov eax, [rel child_status]
    shr eax, 8
    and eax, 0xFF        ; Exit code

    ; Print exit status
    mov rdi, 1
    lea rsi, [rel wait_msg]
    mov rdx, wait_len
    mov rax, 1
    syscall

```

```
    ; (would print the number here)
    mov rax, 60
    xor rdi, rdi
    syscall

.child:
    ; Replace ourselves with /bin/sh
    mov rax, 59          ; sys_execve
    lea rdi, [rel shell]
    lea rsi, [rel sh_argv]
    lea rdx, [rel sh_envp]
    syscall

    ; Only reached on execve failure
    mov rax, 60
    mov rdi, 127         ; Convention for exec failure
    syscall

.error:
    mov rax, 60
    mov rdi, 1
    syscall
```

Part 6: Process Signals

```

; Signals are asynchronous notifications delivered to processes
; Common signals:
;   SIGTERM (15) - polite termination request
;   SIGKILL (9)  - forced kill (cannot be caught)
;   SIGSEGV (11) - segmentation fault
;   SIGCHLD (17) - child process status change
;   SIGUSR1 (10) - user-defined signal 1

; Sending a signal:
; long kill(pid_t pid, int sig);
send_signal:
    mov rax, 62          ; sys_kill
    ; RDI = pid (already set by caller)
    ; RSI = signal number (already set by caller)
    syscall
    ret

; Installing a signal handler:
; We need the rt_sigaction syscall (13)
; struct sigaction {
;     void (*sa_handler)(int);    // or sa_sigaction for SA_SIGINFO
;     unsigned long sa_flags;
;     void (*sa_restorer)(void); // internal, set by libc
;     sigset_t sa_mask;          // signals to block during handler
; };

section .bss
    old_action resb 152      ; sizeof(struct sigaction) on x86-64

section .data
    caught_msg db "Signal caught!", 10
    caught_len equ $ - caught_msg

; Our sigaction structure
align 8
new_action:
    dq signal_handler        ; sa_handler
    dq 0x04000000            ; sa_flags = SA_RESTORER
    dq sig_restorer          ; sa_restorer
    times 16 db 0            ; sa_mask (128 bytes / 8 = 16 qwords... actually 128 bytes)

section .text
    global _start

; Signal handler – called asynchronously by kernel
signal_handler:
    ; WARNING: only async-signal-safe operations here!
    ; Cannot use malloc, printf, etc.
    push rax
    push rdi

```

```

push rsi
push rdx

mov rax, 1
mov rdi, 1
lea rsi, [rel caught_msg]
mov rdx, caught_len
syscall

pop rdx
pop rsi
pop rdi
pop rax
ret

; Signal restorer (required by kernel for return from signal handler)
sig_restorer:
    mov rax, 15          ; sys_rt_sigreturn
    syscall

_start:
    ; Install handler for SIGUSR1 (signal 10)
    mov rax, 13          ; sys_rt_sigaction
    mov rdi, 10          ; SIGUSR1
    lea rsi, [rel new_action]
    lea rdx, [rel old_action]
    mov r10, 8           ; sigsetsize
    syscall

    ; Send SIGUSR1 to ourselves
    mov rax, 39          ; sys_getpid
    syscall
    mov rdi, rax         ; pid = self
    mov rsi, 10          ; sig = SIGUSR1
    mov rax, 62          ; sys_kill
    syscall

    ; Signal handler will execute before we get here
    ; (or between these instructions)

    mov rax, 60
    xor rdi, rdi
    syscall

```

Part 7: Process Groups and Sessions

```

; Process hierarchy:
; Session → Process Groups → Processes
;
; Session Leader: typically the login shell
; Process Group: related processes (e.g., a pipeline: ls | grep | wc)
; Foreground Group: receives terminal input
; Background Group: runs without terminal input

; Get process group ID
get_pgid:
    mov rax, 121          ; sys_getpgid
    xor rdi, rdi          ; pid=0 means current process
    syscall
    ret                  ; RAX = PGID

; Set process group (used by shells for job control)
set_pgid:
    mov rax, 109          ; sys_setpgid
    ; RDI = pid (0 = self)
    ; RSI = pgid (0 = use pid as new pgid)
    syscall
    ret

; Create new session (setsid) – used by daemons
create_session:
    mov rax, 112          ; sys_setsid
    syscall
    ; RAX = new session ID (= our PID), or -errno
    ret

; Implementing a simple daemon:
daemonize:
    ; Step 1: fork and parent exits
    mov rax, 57
    syscall
    test rax, rax
    jnz .parent_exit      ; Parent exits
    ; (child continues)

    ; Step 2: Create new session
    mov rax, 112          ; setsid
    syscall

    ; Step 3: fork again (prevents reacquiring terminal)
    mov rax, 57
    syscall
    test rax, rax
    jnz .parent_exit

    ; Step 4: Close stdin/stdout/stderr

```



```
mov rax, 3          ; sys_close
xor rdi, rdi        ; fd 0
syscall
mov rax, 3
mov rdi, 1          ; fd 1
syscall
mov rax, 3
mov rdi, 2          ; fd 2
syscall

; Step 5: Change working directory to /
mov rax, 80         ; sys_chdir
lea rdi, [rel .root]
syscall

; Now we're a proper daemon
ret

.parent_exit:
mov rax, 60
xor rdi, rdi
syscall

section .rodata
.root: db "/", 0
```

Part 8: Pipes — Inter-Process Communication

```

; pipe() creates a unidirectional data channel
; Used for parent-child communication and shell pipelines

section .bss
    pipe_fds resq 1          ; [read_fd, write_fd] (2 × 4 bytes)
    buffer   resb 256

section .data
    message db "Hello through the pipe!", 10
    msg_len equ $ - message

section .text
    global _start

_start:
    ; Create pipe
    mov rax, 22                ; sys_pipe
    lea rdi, [rel pipe_fds]
    syscall
    test rax, rax
    js .error

    ; pipe_fds[0] = read end, pipe_fds[1] = write end
    mov r12d, dword [rel pipe_fds] ; Read FD
    mov r13d, dword [rel pipe_fds + 4] ; Write FD

    ; Fork
    mov rax, 57
    syscall
    test rax, rax
    jz .child

.parent:
    ; Parent writes to pipe
    ; Close read end (parent only writes)
    mov rax, 3                ; sys_close
    mov edi, r12d
    syscall

    ; Write message
    mov rax, 1                ; sys_write
    mov edi, r13d             ; Write to pipe write-end
    lea rsi, [rel message]
    mov rdx, msg_len
    syscall

    ; Close write end (signals EOF to reader)
    mov rax, 3
    mov edi, r13d
    syscall

```

```

    ; Wait for child
    mov rax, 61
    mov rdi, -1
    xor rsi, rsi
    xor rdx, rdx
    xor r10, r10
    syscall

    mov rax, 60
    xor rdi, rdi
    syscall

.child:
    ; Child reads from pipe
    ; Close write end (child only reads)
    mov rax, 3
    mov edi, r13d
    syscall

    ; Read from pipe
    mov rax, 0                ; sys_read
    mov edi, r12d            ; Read from pipe read-end
    lea rsi, [rel buffer]
    mov rdx, 256
    syscall
    mov r14, rax              ; Bytes read

    ; Print what we read to stdout
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel buffer]
    mov rdx, r14
    syscall

    ; Close read end
    mov rax, 3
    mov edi, r12d
    syscall

    mov rax, 60
    xor rdi, rdi
    syscall

.error:
    mov rax, 60
    mov rdi, 1
    syscall

```

Implementing Shell Pipeline: `ls | wc -l`

```

; Implementing: ls | wc -l
; Parent creates pipe, forks twice:
;   Child 1: exec("ls"), stdout → pipe write end
;   Child 2: exec("wc", "-l"), stdin → pipe read end

```

```

section .data
    ls_path    db "/bin/ls", 0
    ls_arg0    db "ls", 0
    ls_argv    dq ls_arg0, 0

    wc_path    db "/usr/bin/wc", 0
    wc_arg0    db "wc", 0
    wc_arg1    db "-l", 0
    wc_argv    dq wc_arg0, wc_arg1, 0

```

```

    null_envp dq 0

```

```

section .bss
    pipe_fds2 resd 2

```

```

section .text
    global _start

```

```

_start:
    ; Create pipe
    mov rax, 22
    lea rdi, [rel pipe_fds2]
    syscall

    mov r12d, [rel pipe_fds2]      ; Read end
    mov r13d, [rel pipe_fds2 + 4] ; Write end

    ; Fork first child (ls)
    mov rax, 57
    syscall
    test rax, rax
    jz .child_ls

    ; Fork second child (wc)
    mov rax, 57
    syscall
    test rax, rax
    jz .child_wc

    ; Parent: close both pipe ends
    mov rax, 3
    mov edi, r12d
    syscall
    mov rax, 3
    mov edi, r13d

```

```

syscall

; Wait for both children
mov rax, 61
mov rdi, -1
xor rsi, rsi
xor rdx, rdx
xor r10, r10
syscall
mov rax, 61
mov rdi, -1
xor rsi, rsi
xor rdx, rdx
xor r10, r10
syscall

mov rax, 60
xor rdi, rdi
syscall

.child_ls:
; Redirect stdout to pipe write end
mov rax, 33          ; sys_dup2
mov edi, r13d        ; oldfd = pipe write
mov esi, 1           ; newfd = stdout
syscall

; Close both original pipe fds
mov rax, 3
mov edi, r12d
syscall
mov rax, 3
mov edi, r13d
syscall

; exec ls
mov rax, 59
lea rdi, [rel ls_path]
lea rsi, [rel ls_argv]
lea rdx, [rel null_envp]
syscall
; Exit on failure
mov rax, 60
mov rdi, 1
syscall

.child_wc:
; Redirect stdin to pipe read end
mov rax, 33          ; sys_dup2
mov edi, r12d        ; oldfd = pipe read
xor esi, esi         ; newfd = stdin

```

```

syscall

; Close both original pipe fds
mov rax, 3
mov edi, r12d
syscall
mov rax, 3
mov edi, r13d
syscall

; exec wc -l
mov rax, 59
lea rdi, [rel wc_path]
lea rsi, [rel wc_argv]
lea rdx, [rel null_envp]
syscall
; Exit on failure
mov rax, 60
mov rdi, 1
syscall

```

Exercises

1. **Fork bomb guard:** Write a program that forks, but the child checks if it's been forked more than 3 times (pass a counter via shared memory or argument) and stops.
 2. **Mini shell:** Implement a program that reads a command from stdin, forks, and exec's it. Handle exit status reporting.
 3. **Pipe chain:** Implement a 3-stage pipeline in assembly (e.g., `cat file | sort | uniq`).
 4. **Process tree:** Fork multiple children, have each report its PID and PPID, demonstrating the process hierarchy.
 5. **Daemon:** Write a proper daemon that detaches from the terminal and writes periodic timestamps to a log file.
-

Key Takeaways

Concept	Assembly Reality
fork()	sys_clone(SIGCHLD, NULL, ...) → kernel copies task_struct
COW	Pages shared read-only; write triggers fault → kernel copies page
execve()	Destroys address space, loads ELF, sets up stack, jumps to entry
wait()	Parent sleeps in kernel until child's state changes
Signals	Kernel modifies process stack to call handler on next return to userspace
Pipes	Kernel buffer (64KB default); read blocks when empty, write blocks when full
_start stack	[argc][argv...][NULL][envp...][NULL][auxv...][AT_NULL]

Next Topic

[Topic 23: Virtual Memory & Paging →](#) — How the CPU translates virtual addresses to physical, page tables, TLB, and page fault handling.

CHAPTER 40

Topic 23: Virtual Memory & Paging

Overview

Virtual memory is the fundamental abstraction that makes modern operating systems possible. Every process believes it has the entire address space to itself, but the CPU's Memory Management Unit (MMU) transparently translates virtual addresses to physical addresses using **page tables**. This topic explains the entire translation mechanism at the hardware level and shows how the OS manages it.

```
// Every pointer in your program is a VIRTUAL address:
int *p = malloc(sizeof(int)); // p = 0x55555576a2a0 (virtual)
*p = 42;
// The CPU translates 0x55555576a2a0 → some physical address
// (e.g., 0x000000001F3A02A0) transparently

// You NEVER see physical addresses in user space
// Only the kernel manages the virtual → physical mapping
```

Part 1: Why Virtual Memory?

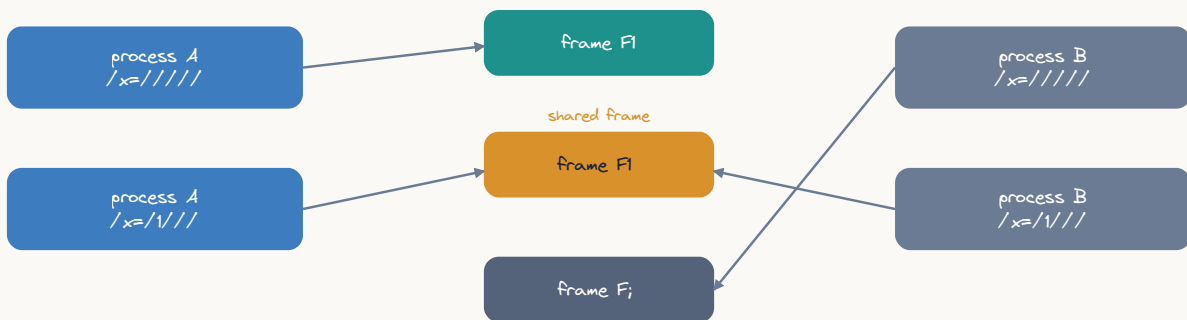
Problems Without Virtual Memory

```
Without virtual memory (DOS era):
- Programs must know their load address at compile time
- Two programs cannot use the same address
- One buggy program can corrupt any other program's memory
- No memory protection whatsoever
- Physical RAM is the only limit

With virtual memory:
- Every process has isolated address space (0x0 to 0x7FFFFFFFFFFF)
- Protection: process A cannot access process B's memory
- Overcommit: allocate more virtual memory than physical RAM exists
- Demand paging: only load pages actually accessed
- Shared libraries: one physical copy, mapped into many processes
- Memory-mapped files: access files as if they were arrays
```

The Translation

virtual $\neq$ physical = same address, different frames



per-process page tables map identical virtual addresses to distinct (or shared) frames

ASCII diagram

Virtual Address (48-bit on x86-64)

```
0x00007FFFF7A2C000
```

MMU (Hardware)
Page Table Walk

Physical Address (52-bit max)

```
0x000000001F3A0000
```

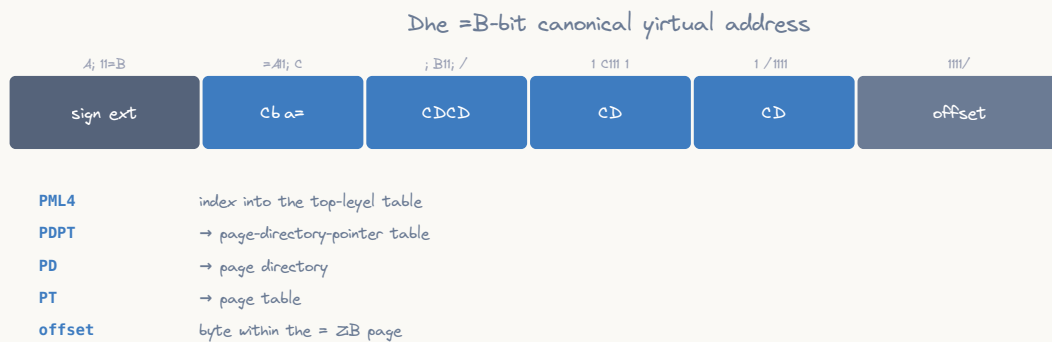
The MMU does this for EVERY memory access:

- Every instruction fetch
- Every data read
- Every data write

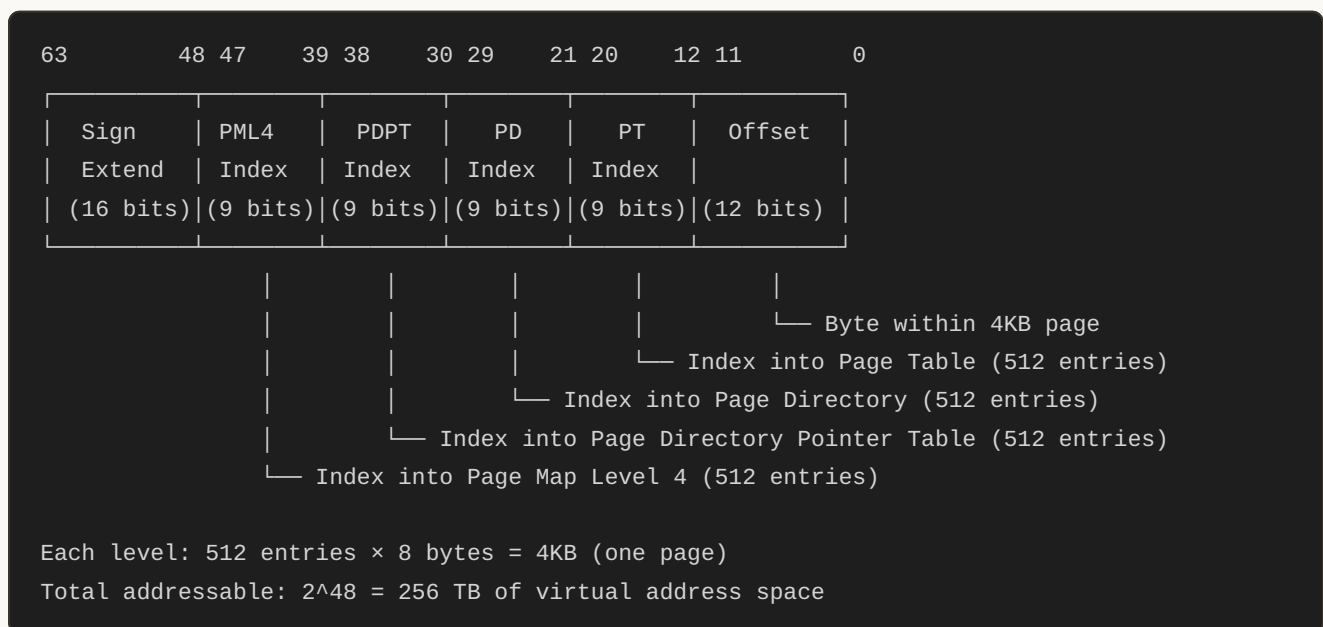
At full CPU speed (with TLB cache hits)

Part 2: Page Table Structure (x86-64, 4-Level)

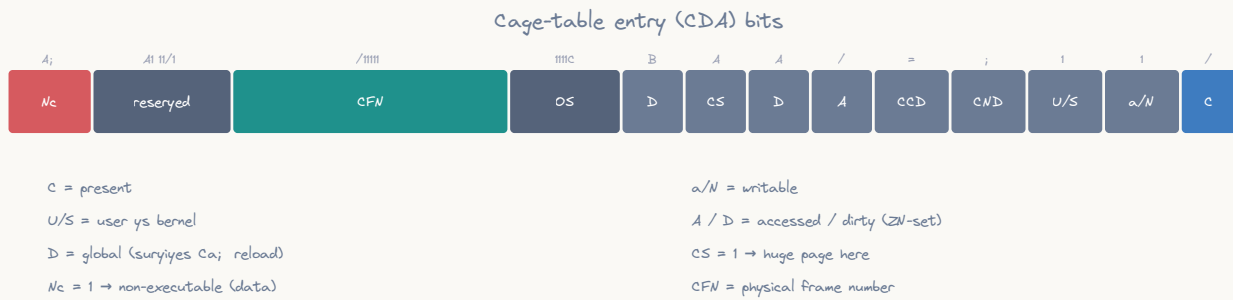
Virtual Address Breakdown



ASCII diagram



Page Table Entry (PTE) Format



ASCII diagram

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
NX	Avail	Physical Page Number				Avail	G	D	A			U	R	P	
		(40 bits = up to 52-bit					S		C	T	/	/	/	/	
		physical address)							D	W	S	W			

Bit	Name	Meaning
0	Present (P)	Page is in physical memory (1=yes, 0=page fault)
1	Read/Write	0=read-only, 1=read-write
2	User/Super	0=kernel only, 1=user accessible
3	Write-Through	Page-level write-through caching
4	Cache Disable	Disable caching for this page
5	Accessed (A)	Set by CPU on any access (used by page replacement)
6	Dirty (D)	Set by CPU on write (page table level only)
7	Page Size (S)	1=large page (2MB at PD level, 1GB at PDPT level)
8	Global (G)	Don't flush from TLB on CR3 switch
11:9	Available	OS can use these bits
51:12	Phys Addr	Physical address of next level / actual page
63	NX (No Execute)	1=page cannot contain executable code

Manual Page Table Walk (What the CPU Does)

```

; This is what the MMU hardware does for every memory access:
; (You can't execute this in user mode – CR3 is privileged)
; This is for educational understanding only.

; Given: virtual address in RAX
; Goal: find physical address

; Step 0: CR3 contains the physical address of PML4 table
; mov rbx, cr3          ; (privileged! kernel only)
;                        ; RBX = physical address of PML4

; Step 1: Extract PML4 index (bits 47-39)
; mov rcx, rax
; shr rcx, 39
; and rcx, 0x1FF        ; 9-bit index (0-511)
; ; PML4 entry at: [CR3 + RCX*8]
; mov rdx, [rbx + rcx*8] ; Read PML4 entry
; ; Check Present bit
; test rdx, 1
; jz .page_fault        ; Not present!
; ; Extract physical address of PDPT
; and rdx, 0x000FFFFFFFFF000 ; Mask to get physical page address
; mov rbx, rdx

; Step 2: Extract PDPT index (bits 38-30)
; mov rcx, rax
; shr rcx, 30
; and rcx, 0x1FF
; mov rdx, [rbx + rcx*8] ; Read PDPT entry
; test rdx, 1
; jz .page_fault
; test rdx, 0x80        ; Check PS bit (1GB page?)
; jnz .huge_page_1gb
; and rdx, 0x000FFFFFFFFF000
; mov rbx, rdx

; Step 3: Extract PD index (bits 29-21)
; mov rcx, rax
; shr rcx, 21
; and rcx, 0x1FF
; mov rdx, [rbx + rcx*8] ; Read PD entry
; test rdx, 1
; jz .page_fault
; test rdx, 0x80        ; Check PS bit (2MB page?)
; jnz .huge_page_2mb
; and rdx, 0x000FFFFFFFFF000
; mov rbx, rdx

; Step 4: Extract PT index (bits 20-12)
; mov rcx, rax

```

```

; shr rcx, 12
; and rcx, 0x1FF
; mov rdx, [rbx + rcx*8] ; Read PT entry (final level)
; test rdx, 1
; jz .page_fault
; ; Extract physical page address
; and rdx, 0x000FFFFFFFFF000
;
; Step 5: Add page offset (bits 11-0)
; mov rcx, rax
; and rcx, 0xFFF          ; 12-bit offset
; or rdx, rcx             ; Physical address = page base + offset
; ; RDX now contains the physical address!

```

Visualizing a Complete Translation

Example: Translating virtual address 0x00007FFFF7A2C123

Binary: 0000000000000000 0|11111111 111111011 110100010 110000010 0100100011

Split into fields:

```

PML4 Index = 0b01111111 = 255
PDPT Index = 0b111111011 = 507
PD Index   = 0b110100010 = 418
PT Index   = 0b110000010 = 386 (0x182)
Offset     = 0b000100100011 = 0x123

```

Walk:

```

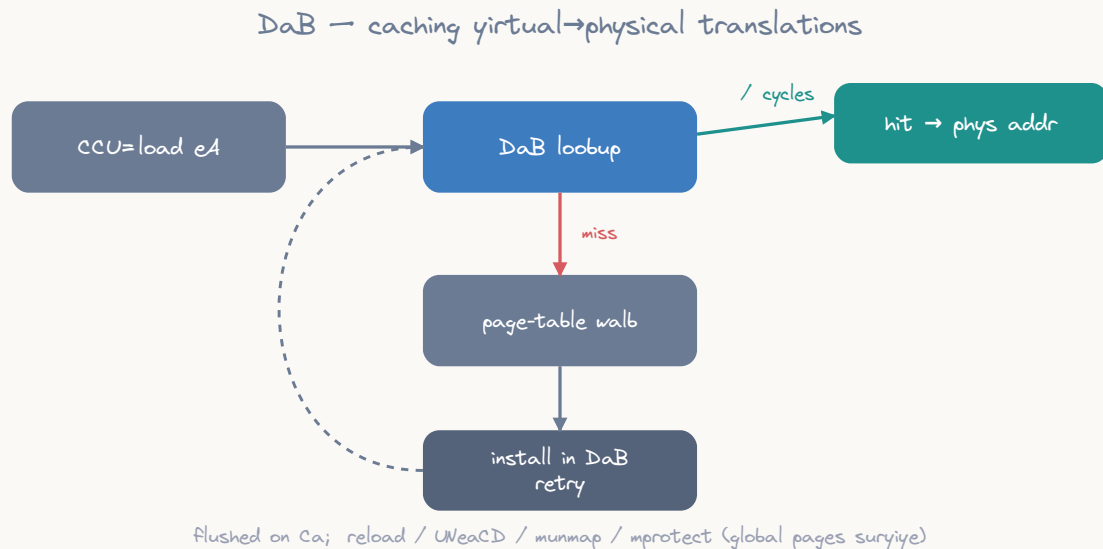
CR3 → PML4 physical base (e.g., 0x1000)
PML4[255] → PDPT physical base (e.g., 0x5000)
PDPT[507] → PD physical base (e.g., 0x9000)
PD[418]   → PT physical base (e.g., 0xD000)
PT[386]   → Physical page (e.g., 0x1F3A0000)
Final:    → 0x1F3A0000 + 0x123 = 0x1F3A0123

```

Total memory accesses for one translation: 4 (without TLB!)

Part 3: The TLB (Translation Lookaside Buffer)

Why the TLB is Critical



ASCII diagram

Without TLB:

Every memory access requires 4 additional memory accesses (page walk)
 Effective memory access time: $5 \times \text{memory_latency} \approx 500\text{ns}$

With TLB (hit):

Translation cached in hardware
 Effective memory access time: $1 \times \text{memory_latency} \approx 100\text{ns}$

TLB hit rate is typically 99%+ for most workloads

Typical TLB sizes (modern CPU):

L1 iTLB: 64 entries (4KB pages) + 8 entries (2MB)
L1 dTLB: 64 entries (4KB pages) + 32 entries (2MB)
L2 sTLB: 1536 entries (4KB + 2MB, unified)

TLB entry:

Virtual Page Num (tag)	Physical Page Frame (translation result)	Flags RWXUGD

TLB Flush Events

```

; TLB must be flushed when page tables change:
;
; 1. Context switch (new CR3 value)
;   - mov cr3, rax → flushes entire TLB (except Global pages)
;   - This is why context switches are expensive!
;
; 2. Single page invalidation:
;   - invlpg [address] → flushes one TLB entry
;   - Used when unmapping/protecting a single page
;
; 3. PCID (Process Context ID) optimization:
;   - Tag TLB entries with process ID
;   - CR3 switch doesn't flush if PCID matches
;   - Reduces context switch overhead significantly

; Kernel code to flush a single TLB entry:
; invlpg [rdi]          ; Invalidate page containing address in RDI

; Effect on user-space performance:
; - Many mmap/munmap calls → many TLB flushes → slowdown
; - Large pages (2MB/1GB) → fewer TLB entries needed → better coverage
; - Accessing memory across many pages → TLB misses → performance wall

```

Demonstrating TLB Effects in User Space

```

; Benchmark: sequential vs stride access patterns
; Sequential: stays within TLB coverage
; Large stride: causes TLB misses

section .bss
    ; Allocate 64MB buffer (16384 pages)
    align 4096
    big_buffer resb 67108864

section .text
    global _start

_start:
    ; Pattern 1: Sequential access (TLB-friendly)
    ; Each page is accessed after the previous one
    ; TLB can prefetch/predict next pages
    lea rdi, [rel big_buffer]
    mov rcx, 67108864 / 8 ; 8M qword accesses
    xor rax, rax

.seq_loop:
    add rax, [rdi]
    add rdi, 8 ; Sequential: next qword
    dec rcx
    jnz .seq_loop

    ; Pattern 2: Page-stride access (TLB-hostile)
    ; Jump 4096 bytes (1 page) each iteration
    ; Each access hits a different page → TLB miss
    lea rdi, [rel big_buffer]
    mov rcx, 16384 ; Visit each page once
    xor rax, rax

.stride_loop:
    add rax, [rdi]
    add rdi, 4096 ; Stride = page size
    dec rcx
    jnz .stride_loop

    ; Pattern 2 will be MUCH slower due to TLB misses
    ; (measurable with rdtsc or perf stat)

    mov rax, 60
    xor rdi, rdi
    syscall

```

Part 4: Page Sizes

4KB Pages (Standard)

Standard 4KB page:

- Offset field: 12 bits ($2^{12} = 4096$)
- 4-level page table walk
- Good for: general purpose, fine-grained protection
- Downside: many TLB entries needed for large data

2MB Huge Pages

2MB page (eliminates one level of translation):

- Offset field: 21 bits ($2^{21} = 2\text{MB}$)
- 3-level page table walk (PML4 → PDPT → PD → done)
- Page Directory entry has PS=1 (Page Size bit)
- 512× fewer TLB entries needed for same coverage!

Use cases: databases, HPC, large in-memory structures

1GB Gigantic Pages

1GB page (eliminates two levels):

- Offset field: 30 bits ($2^{30} = 1\text{GB}$)
- 2-level page table walk (PML4 → PDPT → done)
- PDPT entry has PS=1

Use cases: scientific computing, huge databases

Using Huge Pages from Assembly

```

; Request 2MB huge pages via mmap

MAP_HUGETLB equ 0x40000
MAP_HUGE_2MB equ (21 << 26) ; log2(2MB) = 21, shifted to bits 31-26

section .text
    global _start

_start:
    ; Allocate 2MB with huge page
    mov rax, 9 ; sys_mmap
    xor rdi, rdi ; addr = NULL
    mov rsi, 2097152 ; length = 2MB (must be multiple of page size)
    mov rdx, 3 ; PROT_READ | PROT_WRITE
    mov r10, 34 | MAP_HUGETLB | MAP_HUGE_2MB ; MAP_PRIVATE|ANON|HUGETLB
    mov r8, -1
    xor r9, r9
    syscall

    cmp rax, -4096
    ja .no_hugepages ; Might fail if not configured

    ; Use the huge page memory
    mov r12, rax
    mov qword [r12], 42 ; This single page fault maps 2MB!

    ; Only ONE TLB entry covers all 2MB
    ; Compare: 4KB pages would need 512 TLB entries!

    ; Cleanup
    mov rdi, r12
    mov rsi, 2097152
    mov rax, 11 ; sys_munmap
    syscall

    mov rax, 60
    xor rdi, rdi
    syscall

.no_hugepages:
    ; Huge pages not available, fall back to regular pages
    mov rax, 9
    xor rdi, rdi
    mov rsi, 2097152
    mov rdx, 3
    mov r10, 34 ; MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall
    ; Continue with regular pages...

```

```

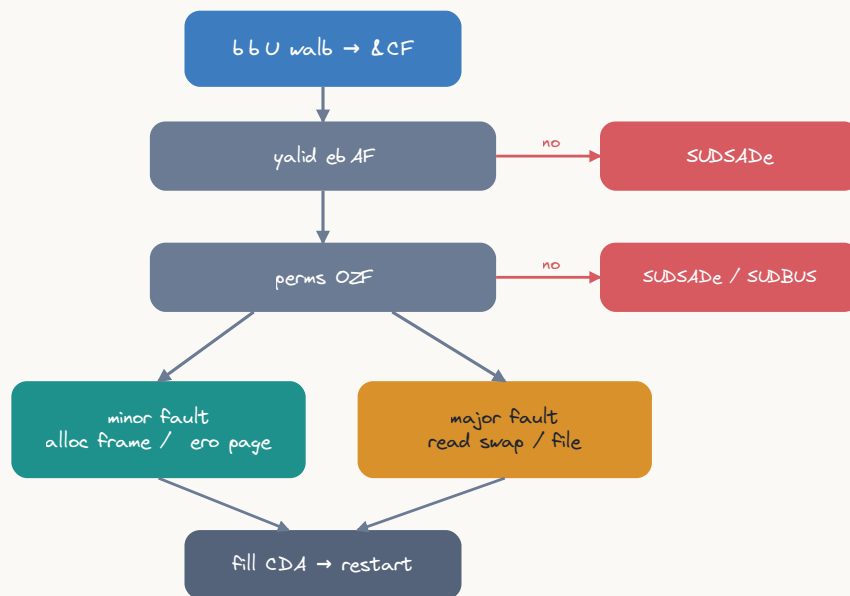
mov rax, 60
xor rdi, rdi
syscall

```

Part 5: Page Faults in Detail

Types of Page Faults

Cage-fault handling (&CF → don't page fault)



ASCII diagram

Page Fault (Exception #14) occurs when:

1. Page Not Present (P=0) - page not in physical memory
2. Protection Violation - access violates permissions
3. Reserved Bit Set - malformed page table entry

CPU pushes error code with fault details:

Bit 0 (P): 0=not-present, 1=protection	
Bit 1 (W): 0=read access, 1=write	
Bit 2 (U): 0=kernel mode, 1=user mode	
Bit 3 (R): 1=reserved bit violation	
Bit 4 (I): 1=instruction fetch	

CR2 register contains the faulting virtual address.

Kernel Page Fault Handler (Conceptual)

```
page_fault_handler(error_code, fault_addr = CR2):
```

1. Find VMA containing fault_addr
 - vma = find_vma(current->mm, fault_addr)
 - if (no VMA): send SIGSEGV (segfault!)
2. Check permissions against VMA flags
 - if (write && !(vma->flags & VM_WRITE)): SIGSEGV
 - if (exec && !(vma->flags & VM_EXEC)): SIGSEGV
3. Handle the fault by type:
 - a. Demand paging (first access to mmap'd region):
 - Allocate physical page
 - Zero it (anonymous) or read from file (file-backed)
 - Install PTE: present=1, permissions from VMA
 - Return to user code (instruction retried)
 - b. Copy-on-Write (write to COW page):
 - Allocate new physical page
 - Copy content from shared page
 - Update PTE to point to new page with write permission
 - Decrement reference count on old page
 - Return to user code (write succeeds on retry)
 - c. Swap-in (page was swapped to disk):
 - Read page from swap device
 - Allocate physical page
 - Copy data from swap to new page
 - Install PTE
 - Return to user code
 - d. Stack growth (fault just below stack VMA):
 - Expand stack VMA downward
 - Allocate page
 - Install PTE
 - Return to user code

Triggering and Observing Page Faults

```

; This program deliberately triggers different types of page faults
; Run with: strace -e trace=fault ./program (or perf stat)

section .text
    global _start

_start:
    ; 1. Demand page fault (first access to mmap'd memory)
    mov rax, 9
    xor rdi, rdi
    mov rsi, 4096
    mov rdx, 3                ; PROT_READ | PROT_WRITE
    mov r10, 34               ; MAP_PRIVATE | MAP_ANONYMOUS
    mov r8, -1
    xor r9, r9
    syscall
    mov r12, rax

    ; Page fault #1: first write to fresh page
    mov byte [r12], 'X'      ; ← Minor page fault here!

    ; 2. No fault: same page, already present
    mov byte [r12 + 100], 'Y' ; ← No fault, page already mapped

    ; 3. Another demand fault: touch a different page
    mov byte [r12 + 4096], 'Z' ; Wait—this is beyond our mapping!
    ; Actually this would SIGSEGV because we only mapped 4096 bytes
    ; Let's map more:
    mov rax, 9
    xor rdi, rdi
    mov rsi, 8192            ; Map 2 pages
    mov rdx, 3
    mov r10, 34
    mov r8, -1
    xor r9, r9
    syscall
    mov r13, rax

    mov byte [r13], 'A'      ; ← Page fault (page 1)
    mov byte [r13 + 4096], 'B' ; ← Page fault (page 2)
    mov byte [r13 + 1], 'C'  ; ← No fault (page 1 already mapped)

    ; 4. Stack page fault (stack grows on demand too!)
    ; The kernel maps a few pages for initial stack
    ; Deep recursion causes stack growth faults
    sub rsp, 65536           ; Jump 16 pages down the stack
    mov byte [rsp], 'S'      ; ← Stack growth page fault!
    add rsp, 65536           ; Restore

    ; Cleanup

```

```
mov rdi, r12
mov rsi, 4096
mov rax, 11
syscall
mov rdi, r13
mov rsi, 8192
mov rax, 11
syscall

mov rax, 60
xor rdi, rdi
syscall
```

Part 6: Memory Protection

mprotect() — Changing Page Permissions

```

; mprotect() changes the protection on existing pages
; Syscall 10: mprotect(addr, len, prot)
; addr MUST be page-aligned

section .text
    global _start

_start:
    ; Allocate a page with RW permission
    mov rax, 9
    xor rdi, rdi
    mov rsi, 4096
    mov rdx, 3                ; PROT_READ | PROT_WRITE
    mov r10, 34
    mov r8, -1
    xor r9, r9
    syscall
    mov r12, rax

    ; Write some data
    mov qword [r12], 0xDEADCAFE

    ; Make it read-only
    mov rax, 10                ; sys_mprotect
    mov rdi, r12                ; addr (page-aligned)
    mov rsi, 4096                ; length
    mov rdx, 1                ; PROT_READ only
    syscall

    ; This read works fine:
    mov rax, [r12]                ; OK

    ; This write would trigger SIGSEGV:
    ; mov qword [r12], 0    ; ← SIGSEGV! Protection violation!

    ; Make it executable (for JIT code)
    mov rax, 10                ; sys_mprotect
    mov rdi, r12
    mov rsi, 4096
    mov rdx, 5                ; PROT_READ | PROT_EXEC (no write!)
    syscall

    ; Page is now executable but not writable (W^X security)

    mov rax, 60
    xor rdi, rdi
    syscall

```

Guard Pages (Stack Overflow Detection)

```

; Guard pages: PROT_NONE pages that trigger SIGSEGV on access
; Used to detect stack overflow and buffer overruns

; How the kernel implements stack guards:
;
; High address ┌───────────────────┐
;               │ Stack (RW)        │ ← RSP starts here
;               │                   │
;               │ ↓ grows down      │
;               └───────────────────┘
;               │ GUARD PAGE        │ ← PROT_NONE (access = SIGSEGV)
;               │ (no permissions) │
;               └───────────────────┘
;               │ More stack        │ ← Kernel grows stack past guard
;               │ (not yet mapped)  │   on controlled page faults
; Low address  └───────────────────┘

; Implementing our own guard page:
create_guarded_buffer:
    ; Allocate 3 pages: [guard][data][guard]
    mov rax, 9
    xor rdi, rdi
    mov rsi, 12288          ; 3 pages
    mov rdx, 3              ; PROT_READ | PROT_WRITE
    mov r10, 34
    mov r8, -1
    xor r9, r9
    syscall
    mov r12, rax             ; Base address

    ; Set first page as guard (PROT_NONE)
    mov rax, 10
    mov rdi, r12
    mov rsi, 4096
    xor rdx, rdx            ; PROT_NONE
    syscall

    ; Set last page as guard (PROT_NONE)
    mov rax, 10
    lea rdi, [r12 + 8192]
    mov rsi, 4096
    xor rdx, rdx            ; PROT_NONE
    syscall

    ; Return pointer to middle page (usable data)
    lea rax, [r12 + 4096]
    ret

    ; Any underflow (write before buffer) hits first guard → SIGSEGV
    ; Any overflow (write past buffer) hits last guard → SIGSEGV

```


Part 7: Shared Memory

Processes Sharing Physical Pages

```

; Two processes can share memory via mmap with MAP_SHARED
; or using the shmget/shmat system (System V) or memfd_create

; Method 1: mmap MAP_SHARED on a file (or memfd)
; Both processes mmap the same file → same physical pages

; Create anonymous shared memory (parent-child):
section .bss
    shared_ptr resq 1

section .text
    global _start

_start:
    ; Create shared anonymous mapping
    ; MAP_SHARED means fork'd child shares the SAME physical pages
    ; (no Copy-on-Write!)
    mov rax, 9                ; sys_mmap
    xor rdi, rdi
    mov rsi, 4096
    mov rdx, 3                ; PROT_READ | PROT_WRITE
    mov r10, 33               ; MAP_SHARED | MAP_ANONYMOUS (0x21)
    mov r8, -1
    xor r9, r9
    syscall
    mov [shared_ptr], rax
    mov r12, rax

    ; Write initial value
    mov qword [r12], 0

    ; Fork
    mov rax, 57
    syscall
    test rax, rax
    jz .child

.parent:
    ; Wait a bit for child to write (naive synchronization)
    mov rax, 35                ; sys_nanosleep
    lea rdi, [rel .timespec]
    xor rsi, rsi
    syscall

    ; Read value written by child (SAME physical page!)
    mov rax, [r12]             ; Should be 42 (written by child)
    mov rdi, rax
    mov rax, 60
    syscall

```

```

.child:
    ; Write to shared memory
    mov qword [r12], 42    ; Parent will see this!
    mov rax, 60
    xor rdi, rdi
    syscall

section .rodata
.timespec:
    dq 0                ; seconds
    dq 1000000000       ; nanoseconds (100ms)

```

Part 8: Kernel Virtual Address Space

x86-64 Linux kernel memory map (5-level paging disabled):

User space: 0x0000000000000000 – 0x00007FFFFFFFFFFF (128 TB)
[per-process, different page tables]

Hole: 0x0000800000000000 – 0xFFFF7FFFFFFFFFFF
[non-canonical addresses, access = #GP fault]

Kernel space: 0xFFFF800000000000 – 0xFFFFFFFFFFFFFFFF (128 TB)
[same mapping in all processes]

Kernel sub-regions:

0xFFFF800000000000: Direct mapping of all physical memory
(kernel can access any phys addr by adding offset)
0xFFFFa00000000000: vmalloc region (non-contiguous kernel allocs)
0xFFFFe00000000000: kernel text (.text, .data, .rodata)
0xFFFFFFFF80000000: modules
0xFFFFFFFF600000: vsyscall page (legacy, deprecated)

Why kernel is in EVERY process's page tables:

- Syscalls don't need to switch page tables
- Exception handlers available immediately
- User/Supervisor bit prevents user-mode access
- KPTI (Meltdown mitigation) unmaps most kernel pages from user view

Part 9: ASLR (Address Space Layout Randomization)

```

; ASLR randomizes memory layout to prevent exploits:
; - Stack: random base address
; - mmap: random base address
; - Heap (brk): small random offset
; - PIE executables: random load address

; Observe ASLR by printing stack address across runs:
section .data
    msg db "Stack RSP = 0x"
    msg_len equ $ - msg
    hex_chars db "0123456789abcdef"

section .bss
    hex_buf resb 16

section .text
    global _start

_start:
    ; Print prefix
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel msg]
    mov rdx, msg_len
    syscall

    ; Convert RSP to hex string
    mov rax, rsp
    lea rdi, [rel hex_buf]
    mov rcx, 16          ; 16 hex digits for 64-bit value

.hex_loop:
    dec rcx
    mov rdx, rax
    and rdx, 0xF          ; Low nibble
    lea rsi, [rel hex_chars]
    mov dl, [rsi + rdx]    ; Hex character
    mov [rdi + rcx], dl
    shr rax, 4
    test rcx, rcx
    jnz .hex_loop

    ; Print hex value
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel hex_buf]
    mov rdx, 16
    syscall

    ; Newline

```

```
push 10
mov rax, 1
mov rdi, 1
mov rsi, rsp
mov rdx, 1
syscall
pop rax

; Each run shows a different address!
; Disable ASLR: echo 0 > /proc/sys/kernel/randomize_va_space

mov rax, 60
xor rdi, rdi
syscall
```

Part 10: Memory-Mapped Files

```

; Map a file into memory – access it like an array
; Changes to memory are written back to the file (MAP_SHARED)
; or kept private (MAP_PRIVATE, COW semantics)

section .data
    filename db "testfile.txt", 0

section .text
    global _start

_start:
    ; Open file
    mov rax, 2          ; sys_open
    lea rdi, [rel filename]
    mov rsi, 2          ; O_RDWR
    mov rdx, 0644o      ; mode (for creation)
    syscall
    mov r12, rax        ; fd

    ; Get file size with fstat
    sub rsp, 144        ; struct stat buffer
    mov rax, 5          ; sys_fstat
    mov rdi, r12
    mov rsi, rsp
    syscall
    mov r13, [rsp + 48] ; st_size field offset
    add rsp, 144

    ; Memory-map the file
    mov rax, 9          ; sys_mmap
    xor rdi, rdi        ; addr = NULL
    mov rsi, r13        ; length = file size
    mov rdx, 3          ; PROT_READ | PROT_WRITE
    mov r10, 1          ; MAP_SHARED (writes go to file)
    mov r8, r12         ; fd
    xor r9, r9          ; offset = 0
    syscall
    mov r14, rax        ; Mapped address

    ; Now the entire file is accessible as memory!
    ; Reading file[0]:
    mov al, [r14]       ; First byte of file (no read() syscall!)

    ; Modifying the file:
    mov byte [r14], 'H' ; Writes directly to file!
    mov byte [r14+1], 'i'

    ; Sync changes to disk
    mov rax, 26         ; sys_msync
    mov rdi, r14

```

```

mov rsi, r13
mov rdx, 4          ; MS_SYNC
syscall

; Unmap
mov rax, 11         ; sys_munmap
mov rdi, r14
mov rsi, r13
syscall

; Close file
mov rax, 3
mov rdi, r12
syscall

mov rax, 60
xor rdi, rdi
syscall

```

Exercises

1. **Page counter:** Write a program that allocates N pages with mmap, touches them one at a time, and uses the `perf` tool to count page faults.
 2. **TLB benchmark:** Compare access time for sequential vs random access patterns across a large array. Use `rdtsc` to measure.
 3. **Self-modifying code:** Allocate RW memory, write machine code into it, mprotect to RX, then call it. Make the generated code compute factorial.
 4. **Shared memory counter:** Two processes increment a shared counter. Observe the race condition, then fix it with atomic operations (next topic: synchronization).
 5. **Memory map dump:** Write a program that reads and parses `/proc/self/maps`, printing each region's address range, permissions, and mapped file.
-

Key Takeaways

Concept	Hardware/OS Reality
Virtual Address	48-bit, split into 4×9 -bit indexes + 12-bit offset
Page Table	4 levels, each is a 4KB page with 512×8 -byte entries
TLB	Hardware cache of recent translations; miss = 4 memory accesses
Page Fault	CPU exception #14; kernel handles by mapping physical page
Huge Pages	2MB/1GB pages reduce TLB pressure for large data
COW	Pages shared read-only; write fault → kernel copies page
mprotect	Changes PTE permission bits; takes effect immediately
ASLR	Kernel randomizes mmap/stack/heap base addresses
Shared Memory	Multiple PTEs (different processes) point to same physical page

Next Topic

[Topic 24: I/O Internals](#) → — How write(), read(), and printf() work from syscall to hardware.

CHAPTER 41

Topic 24: I/O Internals — How Print and Read Actually Work

Overview

When you call `write(1, "Hello\n", 6)`, the string doesn't teleport to your screen. It travels through layers of kernel buffering, device drivers, terminal emulators, and finally hardware. This topic traces the complete path of a byte from your assembly `syscall` instruction to pixels on screen, and similarly for keyboard input reaching your `read()` call.

```
// The journey of printf("Hello\n"):
// 1. printf() formats string into user-space buffer (stdio)
// 2. Buffer flush triggers write() syscall
// 3. Kernel copies data to kernel buffer (page cache / pipe buffer)
// 4. Kernel wakes up the terminal device driver
// 5. Driver sends bytes to PTY (pseudo-terminal)
// 6. Terminal emulator reads from PTY master
// 7. Terminal emulator renders glyphs to screen (GPU/framebuffer)
```

Part 1: The write() Syscall Path

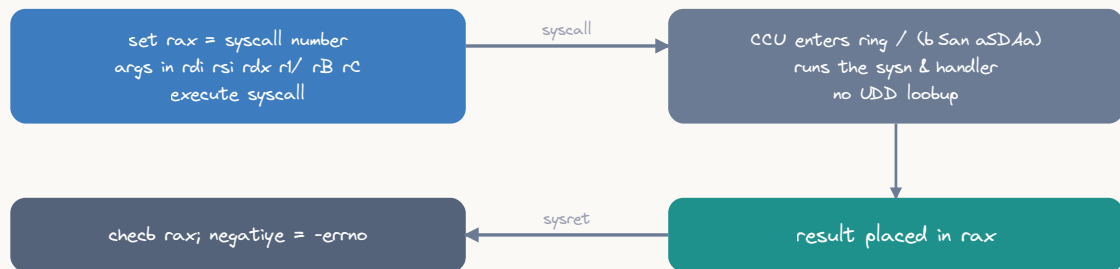
User Space → Kernel

```
; Every write starts here:
mov rax, 1           ; sys_write
mov rdi, 1           ; fd = STDOUT (file descriptor 1)
lea rsi, [rel msg]   ; buf = address of data
mov rdx, 6           ; count = number of bytes
syscall              ; ← Transition to kernel mode
; RAX = bytes written (or -errno)

section .rodata
msg db "Hello", 10
```

What Happens Inside `syscall`

A linux `syscall` = user → kernel → back



`syscall` is far cheaper than the legacy `int /xB/` — it uses `bSan`, not the UDD

ASCII diagram

CPU executes SYSCALL instruction:

- | 1. Save user RIP → RCX (return address)
- | 2. Save user RFLAGS → R11
- | 3. Load kernel RIP from MSR_LSTAR (IA32_LSTAR = 0xC0000082)
- | 4. Load kernel CS/SS from MSR_STAR
- | 5. Mask RFLAGS with MSR_SFmask (disable interrupts)
- | 6. Set CPL = 0 (kernel privilege level)
- | 7. Continue execution at kernel entry point

Kernel syscall entry (`entry_SYSCALL_64`):

- | 1. Switch to kernel stack (from TSS.RSP0)
- | 2. Save all user registers on kernel stack (`pt_regs`)
- | 3. Look up `RAX` in `sys_call_table[]`
- | 4. Call `sys_write(fd=RDI, buf=RSI, count=RDY)`
- | 5. Store return value in `RAX` slot of saved registers
- | 6. Restore user registers from kernel stack
- | 7. Execute `SYSRET` (return to user mode)

Inside sys_write()

```
sys_write(unsigned int fd, const char *buf, size_t count):
```

1. Get file struct from fd table:

```
file = current->files->fd_array[fd]
```

2. Validate:

- Is fd valid? ($0 \leq fd < \text{max_fds}$)
- Is file open for writing? (`file->f_mode & FMODE_WRITE`)
- Is buf in user-accessible memory? (`access_ok(buf, count)`)

3. Copy from user space to kernel:

```
copy_from_user(kernel_buf, user_buf, count)
```

(Can't use user pointer directly – page might be swapped out!)

4. Call file operation:

```
file->f_op->write(file, kernel_buf, count, &pos)
```

For a terminal (stdout):

→ `tty_write()` → line discipline → driver → PTY/UART

For a regular file:

→ `ext4_file_write()` → page cache → disk I/O

For a pipe:

→ `pipe_write()` → pipe buffer (wakes up reader)

For a socket:

→ `sock_sendmsg()` → TCP/IP stack → NIC driver

5. Return bytes written (or `-EFAULT`, `-EINVAL`, `-EIO...`)

Part 2: File Descriptors Explained

The fd Table

Process task_struct:

```
[
  ...
  | files ————— |
  | ...
]
```

→ files_struct:

```
[
  | fd_array[0] → file (stdin) |
  | fd_array[1] → file (stdout) |
  | fd_array[2] → file (stderr) |
  | fd_array[3] → file (opened...) |
  | ... |
]
```

|
▼

struct file:

```
[
  | f_op → file_operations { |
  |   .read = tty_read, |
  |   .write = tty_write, |
  |   .poll = tty_poll, |
  |   ... |
  | } |
  | f_pos (current offset) |
  | f_flags (O_RDONLY, O_NONBLOCK..) |
  | f_mode (FMODE_READ|FMODE_WRITE) |
  | inode → actual file/device |
]
```

File Descriptor Operations in Assembly

```

; Standard file descriptors:
; 0 = stdin  (keyboard input)
; 1 = stdout (terminal output)
; 2 = stderr (error output)

; Open a file and get a new fd:
section .data
    filepath db "/tmp/output.txt", 0

section .text
open_file:
    mov rax, 2          ; sys_open
    lea rdi, [rel filepath]
    mov rsi, 01020      ; O_CREAT | O_WRONLY (octal)
    mov rdx, 06440      ; mode: rw-r--r--
    syscall
    ; RAX = new fd (e.g., 3) or -errno
    ret

; Duplicate a file descriptor (used for redirection):
; dup2(old_fd, new_fd) – makes new_fd point to same file as old_fd
redirect_stdout:
    ; RDI = fd of our file
    mov rax, 33         ; sys_dup2
    ; RDI = oldfd (already set)
    mov rsi, 1          ; newfd = stdout
    syscall
    ; Now all writes to fd 1 (stdout) go to our file!
    ret

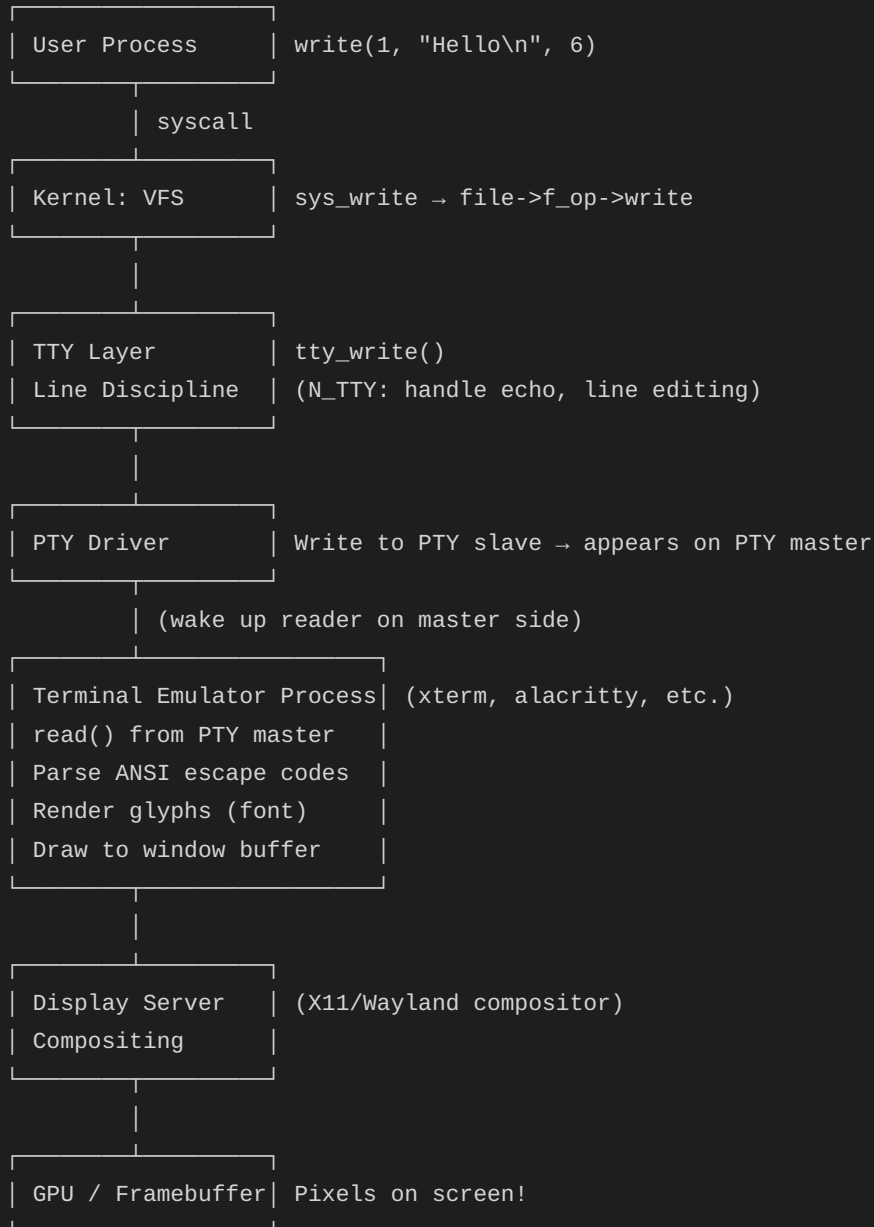
; After redirect_stdout(file_fd):
; mov rax, 1; mov rdi, 1; ... syscall → writes to FILE, not terminal!

```


Part 3: Terminal I/O — The Full Path

From write() to Pixels

Your assembly program writes "Hello\n":



Line Discipline (Canonical vs Raw Mode)

```

; The line discipline processes terminal I/O:
;
; Canonical mode (default, "cooked"):
;   - Input buffered until Enter is pressed
;   - Backspace works (line editing)
;   - Ctrl+C sends SIGINT
;   - Ctrl+D sends EOF
;
; Raw mode (used by vim, tmux, games):
;   - Each keypress delivered immediately
;   - No echo, no signals, no editing
;   - Application handles everything

; Setting raw mode requires ioctl on the terminal:
; ioctl(fd, TCSETS, &termios_struct)
; This is complex — here's the assembly:

section .bss
    termios resb 60          ; struct termios (Linux x86-64)

section .text
; Get current terminal settings
get_termios:
    mov rax, 16              ; sys_ioctl
    xor rdi, rdi             ; fd = stdin
    mov rsi, 0x5401          ; TCGETS
    lea rdx, [rel termios]
    syscall
    ret

; Set terminal to raw mode
set_raw_mode:
    call get_termios

    ; Modify termios fields:
    ; c_lflag: disable ICANON (canonical) and ECHO
    ; c_lflag is at offset 12 in struct termios
    mov eax, [rel termios + 12]
    and eax, ~(0x02 | 0x08)   ; Clear ICANON (0x02) and ECHO (0x08)
    mov [rel termios + 12], eax

    ; c_cc[VMIN] = 1 (minimum chars for read)
    ; c_cc[VTIME] = 0 (no timeout)
    ; c_cc is at offset 17
    mov byte [rel termios + 17 + 6], 1 ; VMIN
    mov byte [rel termios + 17 + 5], 0 ; VTIME

    ; Apply new settings
    mov rax, 16              ; sys_ioctl
    xor rdi, rdi             ; fd = stdin

```

```
    mov rsi, 0x5402      ; TCSETS
    lea rdx, [rel _termios]
    syscall
    ret

; Restore terminal (should be called before exit!)
restore_termios:
    mov rax, 16
    xor rdi, rdi
    mov rsi, 0x5402      ; TCSETS
    lea rdx, [rel _termios] ; Original settings (saved at start)
    syscall
    ret

; Read single keypress in raw mode:
read_key:
    sub rsp, 8
    mov rax, 0           ; sys_read
    xor rdi, rdi         ; fd = stdin
    mov rsi, rsp         ; buffer (on stack)
    mov rdx, 1           ; read 1 byte
    syscall
    movzx eax, byte [rsp] ; Return the character
    add rsp, 8
    ret
```

Part 4: Buffered I/O (Why printf Doesn't Write Immediately)

User-Space Buffering (libc stdio)

`printf("Hello")` does NOT immediately call `write()`!

```
| stdio buffer (typically 4096 or 8192 bytes) |
|                                           |
| [ H | e | l | l | o |   |   |   | ... (4090 bytes free) ] |
|                                           |
|           ↑ |
|         buf_ptr |
|                                           |
| Buffer is flushed (write() called) when: |
| 1. Buffer is full (4096 bytes accumulated) |
| 2. Newline '\n' encountered (line-buffered mode, for ttys) |
| 3. fflush(stdout) called explicitly |
| 4. Program exits normally (atexit handlers) |
| 5. Input is requested (stdin read forces stdout flush) |
```

Buffering modes:

```
_IOFBF (full buffering): Flush only when buffer full (files)
_IOLBF (line buffering): Flush on newline (terminal stdout)
_IONBF (no buffering):   Every write goes to kernel (stderr)
```

Implementing Our Own Buffered Writer

```

; A buffered write implementation in assembly
; Accumulates bytes, flushes to kernel in chunks

section .bss
    out_buf    resb 4096    ; 4KB output buffer
    out_pos    resq 1       ; Current position in buffer

section .text

; Initialize buffer
buf_init:
    mov qword [out_pos], 0
    ret

; Write a single byte to buffer
; Input: DIL = byte to write
buf_putchar:
    mov rcx, [out_pos]
    mov [out_buf + rcx], dil
    inc rcx
    mov [out_pos], rcx

    ; Flush if buffer full
    cmp rcx, 4096
    jge buf_flush
    ret

; Write string to buffer
; Input: RSI = string pointer, RDX = length
buf_write:
    push rbx
    push r12
    push r13
    mov r12, rsi        ; String
    mov r13, rdx        ; Length
    xor rbx, rbx        ; Index

.write_loop:
    cmp rbx, r13
    jge .write_done
    movzx edi, byte [r12 + rbx]
    call buf_putchar
    inc rbx
    jmp .write_loop

.write_done:
    pop r13
    pop r12
    pop rbx
    ret

```

```

; Flush buffer to kernel (actual syscall)
buf_flush:
    push rax
    push rdi
    push rsi
    push rdx

    mov rdx, [out_pos]    ; Bytes to write
    test rdx, rdx
    jz .flush_done        ; Nothing to flush

    mov rax, 1             ; sys_write
    mov rdi, 1             ; stdout
    lea rsi, [rel out_buf]
    syscall

    mov qword [out_pos], 0 ; Reset buffer

.flush_done:
    pop rdx
    pop rsi
    pop rdi
    pop rax
    ret

; Write a number (decimal) to buffer
; Input: RDI = number
buf_print_number:
    push rbp
    mov rbp, rsp
    sub rsp, 24            ; Digit buffer

    mov rax, rdi
    lea rcx, [rbp - 1]     ; End of buffer
    mov r8, 10
    xor r9, r9             ; Digit count

.num_loop:
    xor rdx, rdx
    div r8
    add dl, '0'
    mov [rcx], dl
    dec rcx
    inc r9
    test rax, rax
    jnz .num_loop

    ; Write digits to output buffer
    inc rcx                ; Point to first digit
    mov rsi, rcx

```



```

mov rdx, r9
call buf_write

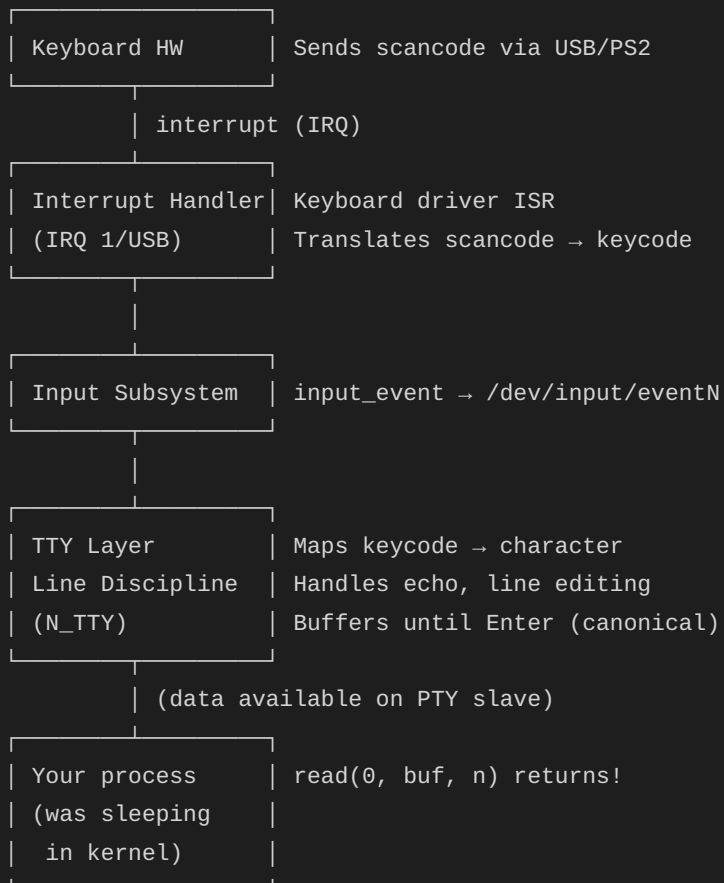
leave
ret

```

Part 5: The read() Syscall Path

From Keyboard to Your Buffer

User presses a key:



Blocking vs Non-Blocking read()

```

; Blocking read (default):
; Process sleeps until data is available
blocking_read:
    mov rax, 0                ; sys_read
    xor rdi, rdi              ; fd = stdin
    lea rsi, [rel buffer]
    mov rdx, 256              ; max bytes
    syscall
    ; Process was ASLEEP here if no data was ready
    ; Kernel woke us up when data arrived
    ; RAX = bytes read, or 0 = EOF, or -errno
    ret

; Non-blocking read:
; Returns immediately with -EAGAIN if no data
nonblocking_read:
    ; First, set O_NONBLOCK on stdin
    mov rax, 72               ; sys_fcntl
    xor rdi, rdi              ; fd = stdin
    mov rsi, 3                ; F_GETFL (get flags)
    syscall
    mov r12, rax              ; Save current flags

    mov rax, 72               ; sys_fcntl
    xor rdi, rdi              ; fd = stdin
    mov rsi, 4                ; F_SETFL (set flags)
    lea rdx, [r12 + 2048]     ; Add O_NONBLOCK (0x800)
    syscall

    ; Now read won't block
    mov rax, 0
    xor rdi, rdi
    lea rsi, [rel buffer]
    mov rdx, 256
    syscall
    ; RAX = bytes read, or -11 (-EAGAIN = no data available)

    ; Restore original flags
    push rax
    mov rax, 72
    xor rdi, rdi
    mov rsi, 4
    mov rdx, r12              ; Original flags
    syscall
    pop rax
    ret

section .bss
    buffer resb 256

```

poll/select — Waiting for Multiple FDs

```

; poll() - wait for activity on multiple file descriptors
; Useful for: servers handling multiple connections,
;           programs reading both stdin and a pipe/socket

; struct pollfd {
;     int    fd;           // offset 0, size 4
;     short events;        // offset 4, size 2 (what we want)
;     short revents;       // offset 6, size 2 (what happened)
; };
; POLLIN = 1 (data available for reading)

section .bss
    poll_fds resb 16          ; Space for 2 pollfd structs

section .text
; Wait for stdin to be readable (with timeout)
poll_stdin:
    ; Set up pollfd for stdin
    lea rdi, [rel poll_fds]
    mov dword [rdi], 0        ; fd = 0 (stdin)
    mov word [rdi + 4], 1     ; events = POLLIN
    mov word [rdi + 6], 0     ; revents = 0

    ; Call poll
    mov rax, 7                ; sys_poll
    ; RDI already points to pollfd array
    mov rsi, 1                ; nfds = 1
    mov rdx, 5000             ; timeout = 5000ms (5 seconds)
    syscall

    ; RAX = number of fds ready, 0 = timeout, -1 = error
    test rax, rax
    jz .timeout
    js .error

    ; Check revents
    lea rdi, [rel poll_fds]
    test word [rdi + 6], 1     ; POLLIN set?
    jz .not_readable

    ; Data is available! Read it
    mov rax, 0
    xor rdi, rdi
    lea rsi, [rel buffer]
    mov rdx, 256
    syscall
    ret

.timeout:
    ; No data after 5 seconds

```

```
    xor rax, rax
    ret
.error:
.not_readable:
    mov rax, -1
    ret
```

Part 6: Implementing printf-like Formatting

```

; A minimal printf implementation in assembly
; Supports: %d (integer), %s (string), %x (hex), %c (char), %% (literal %)

; Calling convention: format string in RDI, arguments in RSI, RDX, RCX, R8, R9
; (additional arguments on stack – we'll support up to 5 args)

section .bss
    printf_buf resb 4096
    printf_pos resq 1

section .text
    global my_printf

; Input: RDI = format string, variable args in RSI, RDX, RCX, R8, R9
my_printf:
    push rbp
    mov rbp, rsp
    push rbx
    push r12
    push r13
    push r14
    push r15

    mov r12, rdi          ; Format string
    ; Store arguments in array for indexed access
    mov [rbp-48], rsi      ; arg[0]
    mov [rbp-56], rdx      ; arg[1]
    mov [rbp-64], rcx      ; arg[2]
    mov [rbp-72], r8       ; arg[3]
    mov [rbp-80], r9       ; arg[4]
    sub rsp, 48

    xor r13, r13          ; Argument index
    mov qword [printf_pos], 0

.fmt_loop:
    movzx eax, byte [r12]
    test al, al
    jz .fmt_done

    cmp al, '%'
    je .format_spec

    ; Regular character – output it
    mov dil, al
    call emit_char
    inc r12
    jmp .fmt_loop

.format_spec:

```



```

    inc r12                ; Skip '%'
    movzx eax, byte [r12]

    cmp al, 'd'
    je .fmt_int
    cmp al, 's'
    je .fmt_str
    cmp al, 'x'
    je .fmt_hex
    cmp al, 'c'
    je .fmt_char
    cmp al, '%'
    je .fmt_percent
    ; Unknown format — output as-is
    mov dil, '%'
    call emit_char
    jmp .fmt_loop

.fmt_int:
    ; Print integer argument
    mov rax, [rbp-48 + r13*8] ; Get next argument
    inc r13
    call emit_decimal
    inc r12
    jmp .fmt_loop

.fmt_str:
    ; Print string argument
    mov rsi, [rbp-48 + r13*8]
    inc r13
.fmt_str_loop:
    movzx eax, byte [rsi]
    test al, al
    jz .str_done
    mov dil, al
    call emit_char
    inc rsi
    jmp .str_loop
.str_done:
    inc r12
    jmp .fmt_loop

.fmt_hex:
    ; Print hex argument
    mov rax, [rbp-48 + r13*8]
    inc r13
    call emit_hex
    inc r12
    jmp .fmt_loop

.fmt_char:

```

```

    ; Print character argument
    mov rax, [rbp-48 + r13*8]
    inc r13
    mov dil, al
    call emit_char
    inc r12
    jmp .fmt_loop

.fmt_percent:
    mov dil, '%'
    call emit_char
    inc r12
    jmp .fmt_loop

.fmt_done:
    ; Flush the buffer
    mov rdx, [printf_pos]
    test rdx, rdx
    jz .no_flush
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel printf_buf]
    syscall
.no_flush:
    add rsp, 48
    pop r15
    pop r14
    pop r13
    pop r12
    pop rbx
    pop rbp
    ret

; Emit single character to buffer
emit_char:
    mov rcx, [printf_pos]
    mov [printf_buf + rcx], dil
    inc qword [printf_pos]
    ret

; Emit decimal number
; Input: RAX = number
emit_decimal:
    push rbx
    test rax, rax
    jns .positive
    ; Negative number
    push rax
    mov dil, '-'
    call emit_char
    pop rax

```

```

    neg rax
.positive:
    mov rbx, rsp
    sub rsp, 24
    mov r8, 10
    xor ecx, ecx          ; Digit count
.dec_loop:
    xor rdx, rdx
    div r8
    add dl, '0'
    push rdx              ; Push digit (we'll reverse)
    inc ecx
    test rax, rax
    jnz .dec_loop

    ; Pop digits in correct order
.dec_output:
    pop rax
    mov dil, al
    call emit_char
    dec ecx
    jnz .dec_output

    mov rsp, rbx
    pop rbx
    ret

; Emit hex number
; Input: RAX = number
emit_hex:
    push rbx
    mov rbx, rsp
    sub rsp, 24
    xor ecx, ecx

    test rax, rax
    jnz .hex_loop
    mov dil, '0'
    call emit_char
    jmp .hex_done

.hex_loop:
    test rax, rax
    jz .hex_output
    mov rdx, rax
    and rdx, 0xF
    shr rax, 4
    cmp dl, 10
    jl .hex_digit
    add dl, 'a' - 10
    jmp .hex_push

```

```
.hex_digit:
    add dl, '0'
.hex_push:
    push rdx
    inc ecx
    jmp .hex_loop

.hex_output:
    pop rax
    mov dil, al
    call emit_char
    dec ecx
    jnz .hex_output

.hex_done:
    mov rsp, rbx
    pop rbx
    ret
```

Part 7: Standard I/O Streams and Redirection

```

; How shell redirection works at the fd level:
;
; $ ./program > output.txt
; Shell does: open("output.txt") → fd 3
;             dup2(3, 1) → fd 1 now points to output.txt
;             close(3)
;             execve("./program")
;             → program writes to fd 1 (thinks it's terminal, but it's a file!)
;
; $ ./program 2>&1
; Shell does: dup2(1, 2) → fd 2 (stderr) now points to same as fd 1 (stdout)
;
; $ cat file | ./program
; Shell creates pipe:
;   pipe() → read_fd, write_fd
;   fork cat: dup2(write_fd, 1); exec("cat", "file")
;   fork program: dup2(read_fd, 0); exec("./program")

; Implementing output redirection ourselves:
section .data
    outfile db "/tmp/redirected.txt", 0
    test_msg db "This goes to a file!", 10
    test_len equ $ - test_msg

section .text
    global _start

_start:
    ; Open output file
    mov rax, 2          ; sys_open
    lea rdi, [rel outfile]
    mov rsi, 01010      ; O_CREAT | O_WRONLY
    mov rdx, 0644o
    syscall
    mov r12, rax        ; New fd (e.g., 3)

    ; Redirect stdout to file
    mov rax, 33         ; sys_dup2
    mov rdi, r12         ; oldfd = file fd
    mov rsi, 1          ; newfd = stdout
    syscall

    ; Close original fd (stdout is now the file)
    mov rax, 3
    mov rdi, r12
    syscall

    ; This write goes to the file, not terminal!
    mov rax, 1
    mov rdi, 1          ; Still "stdout" but now it's the file

```

```
lea rsi, [rel test_msg]
mov rdx, test_len
syscall

mov rax, 60
xor rdi, rdi
syscall
```

Part 8: ANSI Escape Sequences

```

; Terminal emulators interpret special byte sequences for:
; - Cursor movement
; - Colors
; - Screen clearing
; - Text formatting
; These are NOT handled by the kernel – they pass through as data
; The terminal emulator (xterm, alacritty) interprets them

section .data
    ; Clear screen
    clear_seq db 27, "[2J", 27, "[H"
    clear_len equ $ - clear_seq

    ; Colors
    red      db 27, "[31m"
    red_len  equ $ - red
    green    db 27, "[32m"
    green_len equ $ - green
    reset    db 27, "[0m"
    reset_len equ $ - reset

    ; Bold
    bold db 27, "[1m"
    bold_len equ $ - bold

    ; Cursor positioning: ESC[row;colH
    pos db 27, "[10;20H"      ; Move to row 10, col 20
    pos_len equ $ - pos

    hello db "Hello, World!", 10
    hello_len equ $ - hello

section .text
    global _start

_start:
    ; Clear screen
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel clear_seq]
    mov rdx, clear_len
    syscall

    ; Print in red
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel red]
    mov rdx, red_len
    syscall

```

```

mov rax, 1
mov rdi, 1
lea rsi, [rel hello]
mov rdx, hello_len
syscall

; Reset color
mov rax, 1
mov rdi, 1
lea rsi, [rel reset]
mov rdx, reset_len
syscall

; Move cursor and print in green bold
mov rax, 1
mov rdi, 1
lea rsi, [rel pos]
mov rdx, pos_len
syscall

mov rax, 1
mov rdi, 1
lea rsi, [rel bold]
mov rdx, bold_len
syscall

mov rax, 1
mov rdi, 1
lea rsi, [rel green]
mov rdx, green_len
syscall

mov rax, 1
mov rdi, 1
lea rsi, [rel hello]
mov rdx, hello_len
syscall

mov rax, 1
mov rdi, 1
lea rsi, [rel reset]
mov rdx, reset_len
syscall

mov rax, 60
xor rdi, rdi
syscall

```

Part 9: Direct Hardware I/O (Framebuffer)

```

; On Linux, you can write directly to the framebuffer device
; /dev/fb0 – bypasses the terminal entirely!
; Each pixel is typically 4 bytes (BGRA)

section .data
    fb_path db "/dev/fb0", 0

section .text
    global _start

_start:
    ; Open framebuffer
    mov rax, 2
    lea rdi, [rel fb_path]
    mov rsi, 1                ; O_WRONLY
    xor rdx, rdx
    syscall
    cmp rax, 0
    jl .no_fb
    mov r12, rax              ; fb fd

    ; mmap the framebuffer into our address space
    ; Assume 1920x1080x4 = 8,294,400 bytes
    mov rax, 9
    xor rdi, rdi
    mov rsi, 8294400          ; Framebuffer size
    mov rdx, 3                ; PROT_READ | PROT_WRITE
    mov r10, 1                ; MAP_SHARED (changes visible on screen!)
    mov r8, r12                ; fd = framebuffer
    xor r9, r9                ; offset = 0
    syscall
    mov r13, rax              ; Framebuffer memory pointer

    ; Draw a red rectangle (100x100 pixels at position 50,50)
    ; Pixel format: BGRA (Blue, Green, Red, Alpha)
    mov r14, 50                ; Start Y
    mov r15, 50                ; Start X

.draw_y:
    cmp r14, 150              ; End Y
    jge .draw_done

    mov rbx, r15                ; Reset X
.draw_x:
    cmp rbx, 150              ; End X
    jge .next_row

    ; Calculate pixel offset: (y * width + x) * 4
    mov rax, r14
    imul rax, 1920             ; Width

```

```
add rax, rbx
shl rax, 2           ; × 4 bytes per pixel

; Write red pixel (BGRA = 0x00, 0x00, 0xFF, 0xFF)
mov dword [r13 + rax], 0xFF0000FF ; ABGR format

inc rbx
jmp .draw_x

.next_row:
inc r14
jmp .draw_y

.draw_done:
; Unmap and close
mov rdi, r13
mov rsi, 8294400
mov rax, 11
syscall
mov rax, 3
mov rdi, r12
syscall

.no_fb:
mov rax, 60
xor rdi, rdi
syscall
```

Part 10: writev() — Scatter/Gather I/O

```

; writev() writes multiple buffers in a single syscall
; Avoids multiple syscall overhead and copying into a single buffer
; Kernel assembles the output from multiple source buffers

; struct iovec {
;     void *iov_base;    // Pointer to data
;     size_t iov_len;    // Length of data
; };

section .data
    header db "=== Result ===", 10
    h_len  equ $ - header
    value  db "42", 10
    v_len  equ $ - value
    footer db "===== ", 10
    f_len  equ $ - footer

; Array of iovec structures
iov:
    dq header      ; iov[0].iov_base
    dq h_len       ; iov[0].iov_len
    dq value       ; iov[1].iov_base
    dq v_len       ; iov[1].iov_len
    dq footer      ; iov[2].iov_base
    dq f_len       ; iov[2].iov_len

section .text
    global _start

_start:
    ; Single syscall writes all three buffers atomically
    mov rax, 20      ; sys_writev
    mov rdi, 1       ; fd = stdout
    lea rsi, [rel iov] ; iovec array
    mov rdx, 3       ; iovcnt = 3 buffers
    syscall
    ; RAX = total bytes written

    mov rax, 60
    xor rdi, rdi
    syscall

```

Exercises

- 1. Buffered output:** Implement a program that writes 1 million numbers using your buffered writer. Compare `strace` output (number of write syscalls) with/without buffering.
- 2. Raw terminal:** Put the terminal in raw mode, read individual keypresses, and display their ASCII/hex values. Handle Ctrl+Q to quit (restore terminal first!).
- 3. Simple cat:** Implement `cat` — read from stdin (or file argument), write to stdout, using a 4096-byte buffer.
- 4. Shell redirection:** Write a program that redirects its own stdout to a file, prints something, then restores stdout and prints to terminal.
- 5. ANSI animation:** Create a simple animation using ANSI escape sequences (e.g., a bouncing ball or a progress bar).

Key Takeaways

Concept	What Actually Happens
<code>write(1, buf, n)</code>	syscall → kernel copies buf → VFS → device driver → hardware
<code>read(0, buf, n)</code>	Process sleeps → keyboard IRQ → driver → TTY → kernel copies → process wakes
File descriptor	Index into per-process table → points to kernel file struct
<code>printf</code> buffering	libc accumulates in user buffer → flushes on <code>\n</code> or full buffer
Terminal output	Bytes travel through: kernel TTY → PTY → terminal emulator → GPU
Raw mode	<code>ioctl(TCSETS)</code> changes line discipline behavior
Redirection	<code>dup2(file_fd, 1)</code> makes stdout point to file
<code>writew</code>	Single syscall writes from multiple buffers

Next Topic

Topic 25: ELF Binary Format → — How your assembled code becomes an executable: ELF structure, sections, segments, linking, and loading.

CHAPTER 42

Topic 25: ELF Binary Format

Overview

Every executable you create with NASM is an **ELF** (Executable and Linkable Format) file. This topic dissects the ELF format completely — showing how your assembly source becomes bytes on disk, how the linker combines object files, and how the kernel's ELF loader maps your program into memory and starts execution.

Your workflow:

```
hello.asm →(nasm)→ hello.o →(ld)→ hello
```

Source	Object file	Executable
(text)	(ELF relocatable)	(ELF executable)

What's inside each:

```
hello.o:  ELF header + section headers + .text + .data + .symtab + .rela.text
hello:    ELF header + program headers + segments (.text+.rodata, .data+.bss)
```


Part 1: ELF File Structure

High-Level Layout

AaF file layout



the loader reads program headers; the linker reads section headers

ASCII diagram

ELF Executable:

	Offset 0
ELF Header (64 bytes)	Magic number, type, entry point, etc.
Program Header Table (array of Phdr entries)	Describes segments for loading (used by kernel/loader at runtime)
Segment 1: .text + .rodata	(LOAD, R-X)
Segment 2: .data + .bss	(LOAD, RW-)
Section Header Table (array of Shdr entries)	Describes sections for linking/debugging (used by linker, debugger, objdump)
.symtab (symbol table)	Function/variable names and addresses
.strtab (string table)	Null-terminated name strings
.shstrtab	Section name strings

Key distinction:

Sections: logical units (.text, .data, .bss) – for linking/debugging

Segments: loadable chunks – for execution (kernel only reads these)

One segment typically contains multiple sections

The ELF Header

```

; The ELF header is the first 64 bytes of any ELF file (x86-64)
; You can craft one manually to make a minimal executable:

; ELF header structure (Elf64_Ehdr):
; e_ident[16]    Magic + class + endian + version + OS/ABI
; e_type         ET_EXEC=2, ET_DYN=3, ET_REL=1
; e_machine      EM_X86_64 = 0x3E
; e_version      EV_CURRENT = 1
; e_entry        Virtual address of entry point (_start)
; e_phoff        Offset to program header table
; e_shoff        Offset to section header table
; e_flags        Processor-specific flags (0 for x86-64)
; e_ehsize       Size of ELF header (64 bytes)
; e_phentsize    Size of one program header entry (56 bytes)
; e_phnum        Number of program headers
; e_shentsize    Size of one section header entry (64 bytes)
; e_shnum        Number of section headers
; e_shstrndx     Index of .shstrtab section header

; Minimal ELF executable (no linker, no sections – just raw bytes!)
; This is a valid Linux x86-64 executable:

```

```

BITS 64

```

```

ORG 0x400000          ; Load address

```

```

; === ELF Header (64 bytes) ===

```

```

elf_header:
    db 0x7F, "ELF"          ; e_ident[0..3]: magic number
    db 2                    ; e_ident[4]: ELFCLASS64
    db 1                    ; e_ident[5]: ELFDATA2LSB (little-endian)
    db 1                    ; e_ident[6]: EV_CURRENT
    db 0                    ; e_ident[7]: ELFOSABI_NONE
    dq 0                    ; e_ident[8..15]: padding
    dw 2                    ; e_type: ET_EXEC (executable)
    dw 0x3E                 ; e_machine: EM_X86_64
    dd 1                    ; e_version: EV_CURRENT
    dq _start                ; e_entry: entry point address
    dq phdr - $$             ; e_phoff: program header offset
    dq 0                    ; e_shoff: no section headers
    dd 0                    ; e_flags
    dw 64                   ; e_ehsize: ELF header size
    dw 56                   ; e_phentsize: program header entry size
    dw 1                    ; e_phnum: one program header
    dw 64                   ; e_shentsize
    dw 0                    ; e_shnum: no section headers
    dw 0                    ; e_shstrndx

```

```

; === Program Header (56 bytes) ===

```

```

phdr:
    dd 1                    ; p_type: PT_LOAD

```

```

    dd 5                ; p_flags: PF_R | PF_X (read + execute)
    dq 0                ; p_offset: start of file
    dq 0x400000         ; p_vaddr: virtual address
    dq 0x400000         ; p_paddr: physical address (ignored)
    dq file_size        ; p_filesz: size in file
    dq file_size        ; p_memsz: size in memory
    dq 0x200000         ; p_align: alignment

; === Code starts here ===
_start:
    ; Write "Hi\n"
    mov rax, 1          ; sys_write
    mov rdi, 1          ; stdout
    lea rsi, [rel message]
    mov rdx, 3          ; length
    syscall

    ; Exit
    mov rax, 60
    xor rdi, rdi
    syscall

message:
    db "Hi", 10

file_size equ $ - $$
; Total file size: ~170 bytes! (smallest valid ELF executable)

```

Part 2: Object Files (.o) — Relocatable ELF

What NASM Produces

```
$ nasm -f elf64 hello.asm -o hello.o
```

```
$ readelf -h hello.o
```

ELF Header:

Type: REL (Relocatable file) ← Not executable yet!

Entry: 0x0 ← No entry point

```
$ readelf -S hello.o
```

Section Headers:

[0]	NULL		
[1]	.text	PROGBITS	ALLOC EXEC ← Your code
[2]	.data	PROGBITS	ALLOC WRITE ← Initialized data
[3]	.bss	NOBITS	ALLOC WRITE ← Uninitialized data
[4]	.symtab	SYMTAB	← Symbol table
[5]	.strtab	STRTAB	← String table
[6]	.rela.text	RELA	← Relocation entries
[7]	.shstrtab	STRTAB	← Section name strings

Relocations — Why Object Files Need the Linker

```

; In hello.o, addresses are NOT final:
;
; section .data
;     msg db "Hello", 10
;
; section .text
;     global _start
; _start:
;     mov rax, 1
;     mov rdi, 1
;     mov rsi, msg      ← Address of msg is UNKNOWN at assembly time!
;     mov rdx, 6
;     syscall
;
; NASM emits a relocation entry:
;     "At offset X in .text, patch in the address of symbol 'msg'"
;
; Relocation entry (Elf64_Rela):
;     r_offset:  offset in section where fix is needed
;     r_info:    symbol index + relocation type
;     r_addend:  adjustment value
;
; When the linker assigns final addresses, it:
;     1. Places .text at 0x401000, .data at 0x402000
;     2. Finds msg is at 0x402000
;     3. Patches the mov instruction with the correct address
;
; Common relocation types for x86-64:
;     R_X86_64_64      – absolute 64-bit address
;     R_X86_64_PC32    – PC-relative 32-bit (for RIP-relative addressing)
;     R_X86_64_PLT32   – PLT entry for function calls
;     R_X86_64_GOTPCREL – GOT entry for global data access

```

Examining Relocations

```

; Example: calling an external function from assembly

; In mylib.asm:
section .text
    global add_numbers
add_numbers:
    mov rax, rdi
    add rax, rsi
    ret

; In main.asm:
section .text
    extern add_numbers      ; Defined elsewhere
    global _start
_start:
    mov rdi, 10
    mov rsi, 20
    call add_numbers        ; ← Generates R_X86_64_PLT32 relocation
    ; Linker patches the call target with actual address

    mov rdi, rax
    mov rax, 60
    syscall

; Build:
; nasm -f elf64 mylib.asm -o mylib.o
; nasm -f elf64 main.asm -o main.o
; ld main.o mylib.o -o program
;
; readelf -r main.o shows:
;   Offset      Type           Symbol
;   0x0000010   R_X86_64_PLT32   add_numbers - 4
;
; The linker resolves this to the actual address of add_numbers

```


Part 3: The Linker (ld) — Combining Object Files

What the Linker Does

Input: main.o, lib.o, libc.a, libm.so

Output: executable (or shared library)

Steps:

1. Symbol Resolution

- Collect all symbol definitions and references
- Match each reference to exactly one definition
- Error if: undefined symbol, multiple definitions

2. Section Merging

- Combine all .text sections into one .text segment
- Combine all .data sections into one .data segment
- Apply linker script rules for ordering

3. Address Assignment

- Assign virtual addresses to each section
- Default text starts at 0x401000 (Linux x86-64)
- Data follows text, aligned to page boundary

4. Relocation

- For each relocation entry: compute final address
- Patch the instruction bytes with resolved addresses

5. Output

- Write ELF header with entry point
- Write program headers for kernel loader
- Write merged sections
- Write section headers (for debugging)

Linker Script (How Addresses Are Assigned)

```

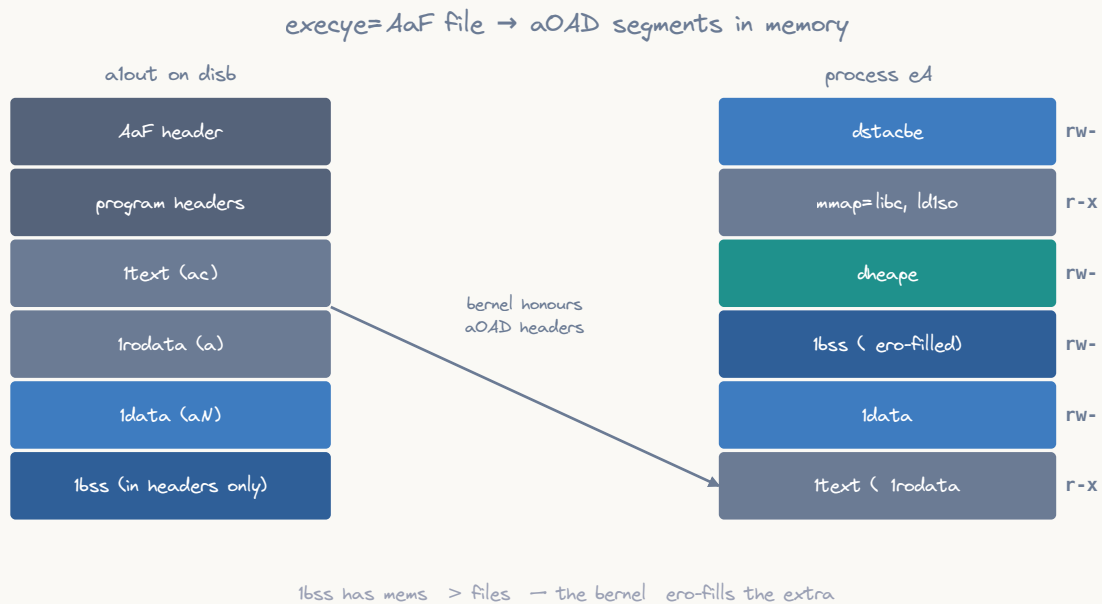
; Default linker script places sections at these addresses:
; (simplified view of `ld --verbose` output)
;
; SECTIONS {
;   . = 0x400000 + SIZEOF_HEADERS;
;   .text : { *(.text) }
;   .rodata : { *(.rodata) }
;   . = ALIGN(0x1000);          /* Page-align data segment */
;   .data : { *(.data) }
;   .bss : { *(.bss) }
; }
;
; Custom linker script for embedded/OS development:
; ld -T my_linker.ld main.o -o kernel.bin

; Example: placing code at a specific address (for a bootloader)
; linker.ld:
; ENTRY(_start)
; SECTIONS {
;   . = 0x7C00;                /* BIOS loads bootsector here */
;   .text : { *(.text) }
;   .data : { *(.data) }
;   . = 0x7C00 + 510;
;   .sig : { SHORT(0xAA55) } /* Boot signature */
; }

```

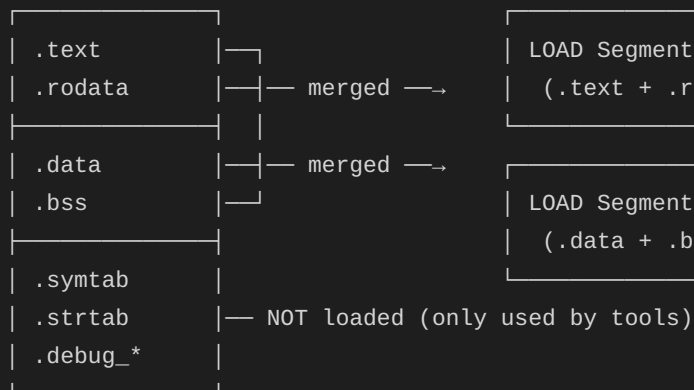
Part 4: Program Headers — Runtime Loading

Segments vs Sections

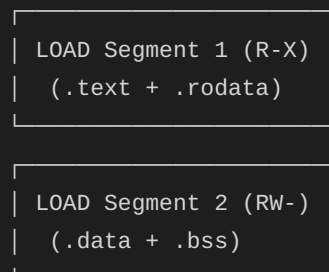


ASCII diagram

Linking (sections):



Loading (segments):



The kernel only reads PROGRAM HEADERS (segments).

Section headers are optional for execution!

(You can strip them: `strip --strip-all ./program`)

Program Header Types

```

; Common program header types:
;
; PT_LOAD (1):      Loadable segment – kernel mmmaps this into memory
; PT_DYNAMIC (2):   Dynamic linking information (.dynamic section)
; PT_INTERP (3):    Path to dynamic linker ("/lib64/ld-linux-x86-64.so.2")
; PT_NOTE (4):      Auxiliary information (build ID, etc.)
; PT_GNU_STACK (0x6474e551): Stack executability flag
; PT_GNU_RELRO (0x6474e552): Read-only after relocation

; readelf -l output for a typical static executable:
;
; Program Headers:
;   Type      Offset      VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
;   LOAD      0x0000000  0x0000000000400000  0x0000000000400000  0x000200 0x000200 R    0x1000
;   LOAD      0x001000  0x0000000000401000  0x0000000000401000  0x000035 0x000035 R E  0x1000
;   LOAD      0x002000  0x0000000000402000  0x0000000000402000  0x000010 0x000018 RW  0x1000
;
;                                     ↑
;                                     MemSiz > FileSiz = BSS (zero-filled)

```

Part 5: Dynamic Linking

How Shared Libraries Work

Static linking:

- All code copied into executable at link time
- Large executable, no external dependencies

Dynamic linking:

- Only stubs included; actual code loaded at runtime
- Small executable, depends on .so files being present
- Allows library updates without recompiling

At runtime:

1. Kernel loads your program
2. Kernel sees PT_INTERP → loads dynamic linker (ld-linux-x86-64.so.2)
3. Dynamic linker reads PT_DYNAMIC segment
4. Loads required shared libraries (.so files)
5. Performs relocations (patches GOT/PLT entries)
6. Calls your `_start` (or `__libc_start_main` for C programs)

The GOT and PLT

allow binding through the GOT and GOT



the first call jumps to the dynamic linker, which patches the GOT;
every later call jumps straight to the resolved address

ASCII diagram

GOT (Global Offset Table):

Array of pointers to external symbols

Initially points to PLT resolver stub

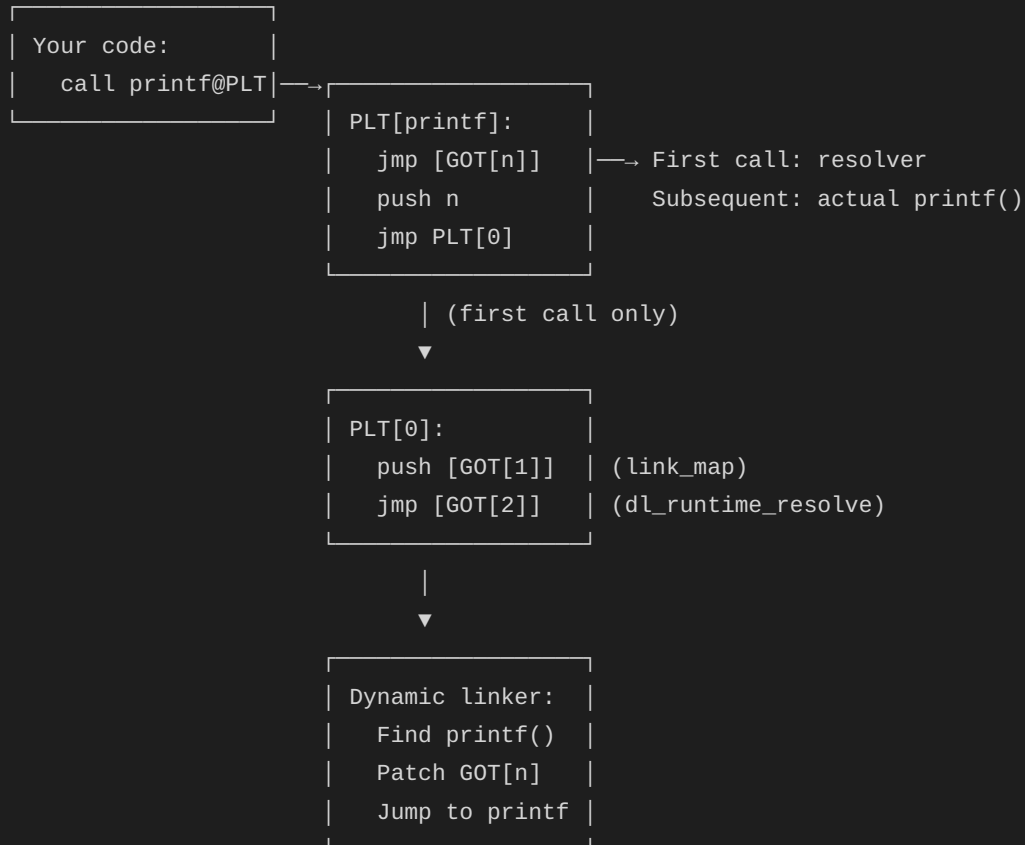
After first call: patched to point to actual function

PLT (Procedure Linkage Table):

Stub code for each external function

First call: jumps to resolver (slow path)

Subsequent calls: jumps directly via GOT (fast path)



Calling Shared Library Functions from Assembly

```

; Calling printf from assembly (dynamic linking)
; Build: nasm -f elf64 prog.asm && ld -dynamic-linker /lib64/ld-linux-x86-64.so.2 -lc
prog.o -o prog

section .data
    fmt db "Value: %d", 10, 0

section .text
    extern printf
    extern exit
    global _start

_start:
    ; Align stack to 16 bytes before calling C functions
    and rsp, -16

    ; Call printf(fmt, 42)
    lea rdi, [rel fmt]      ; Format string
    mov rsi, 42             ; Value
    xor eax, eax            ; Number of vector args (0 for integer-only)
    call printf wrt ..plt   ; Call through PLT (NASM syntax for PLT call)

    ; Exit
    xor edi, edi
    call exit wrt ..plt

```

Position-Independent Code (PIC/PIE)

```

; Position-Independent Executables (PIE) are loaded at random addresses (ASLR)
; All memory references must be RIP-relative

; Build PIE: nasm -f elf64 prog.asm && ld -pie -dynamic-linker ... prog.o -o prog

; PIE requirements:
; 1. All data access via RIP-relative addressing (lea rdi, [rel msg])
; 2. All external calls through PLT (call func wrt ..plt)
; 3. Global data access through GOT (for shared libraries)

section .data
    msg db "Hello from PIE!", 10, 0

section .text
    global _start

_start:
    ; RIP-relative addressing works regardless of load address
    lea rsi, [rel msg]      ; This works at ANY load address
    ; NOT: mov rsi, msg      ; This would be an absolute address (wrong for PIE!)

    mov rax, 1
    mov rdi, 1
    mov rdx, 16
    syscall

    mov rax, 60
    xor rdi, rdi
    syscall

```


Part 6: How the Kernel Loads an ELF

The `execve()` Loading Sequence

When kernel processes `execve()` for an ELF file:

1. Read first 4 bytes: check magic (0x7F "ELF")
2. Parse ELF header:
 - Verify `e_type` (must be `ET_EXEC` or `ET_DYN`)
 - Verify `e_machine` (must be `EM_X86_64`)
 - Read `e_entry` (entry point)
 - Read `e_phoff`, `e_phnum` (program headers)
3. For each `PT_LOAD` program header:
 - Calculate mapping parameters:
 - `file_offset = p_offset`
 - `map_addr = p_vaddr` (or random base for PIE)
 - `file_size = p_filesz`
 - `mem_size = p_memsz`
 - `permissions = p_flags` (R/W/X)
 - `mmap` the segment:
 - `mmap(map_addr, file_size, prot, MAP_PRIVATE|MAP_FIXED, fd, file_offset)`
 - Zero BSS portion (`memsz > filesz`):
 - `memset(map_addr + filesz, 0, memsz - filesz)`
4. If `PT_INTERP` present (dynamically linked):
 - Read interpreter path ("`/lib64/ld-linux-x86-64.so.2`")
 - Load interpreter ELF into memory
 - Set entry point to interpreter's entry (not program's!)
 - Interpreter will eventually jump to program's entry
5. Set up initial stack:
 - [See Topic 22 for stack layout details]
 - Push auxiliary vector, environment, arguments
6. Set registers and jump:
 - `RSP` = top of new stack
 - `RIP` = `e_entry` (or interpreter entry for dynamic)
 - All other registers = 0

Part 7: Symbol Tables

```

; Symbol table entries tell tools about functions and variables:
;
; Elf64_Sym:
;   st_name:  offset into .strtab (name string)
;   st_info:  type (FUNC/OBJECT/NOTYPE) + binding (LOCAL/GLOBAL/WEAK)
;   st_other: visibility (DEFAULT/HIDDEN/PROTECTED)
;   st_shndx: section index where symbol is defined
;   st_value: address (or offset in relocatable file)
;   st_size:  size of symbol (function size, variable size)

; Making symbols visible/hidden:
section .text
    global public_func      ; Visible to linker (GLOBAL binding)
    global _start

; Local label (not in symbol table by default):
.local_helper:
    ret

; Global function:
public_func:
    ; ... code ...
    ret

; Weak symbol (can be overridden by a strong definition):
; global weak_func:function weak
; weak_func:
;     ret

_start:
    call public_func
    mov rax, 60
    xor rdi, rdi
    syscall

; readelf -s program shows:
;   Num  Value             Size Type  Bind  Vis      Name
;   --  -
;   1    0x401000           0    FUNC  GLOBAL DEFAULT _start
;   2    0x401020          16    FUNC  GLOBAL DEFAULT public_func

```

Part 8: Debugging Information (DWARF)

```

; DWARF debug info maps machine code back to source:
; - Line number tables (.debug_line)
; - Variable locations (.debug_info)
; - Type information (.debug_abbrev, .debug_types)
;
; Build with debug info:
; nasm -f elf64 -g -F dwarf prog.asm -o prog.o
; ld -g prog.o -o prog
;
; Now GDB can show source lines and variable values!

; NASM debug directives:
section .text
    global _start

_start:
    ; GDB will show these source lines
    mov rdi, 42
    call do_work

    mov rdi, rax
    mov rax, 60
    syscall

do_work:
    mov rax, rdi
    imul rax, rax    ; Square the input
    ret

; With -g -F dwarf, NASM emits:
; .debug_info    - compilation unit, types
; .debug_line    - line number → address mapping
; .debug_abbrev  - abbreviation tables
;
; GDB can then:
; (gdb) list      → show source
; (gdb) break _start → set breakpoint by name
; (gdb) next      → step by source line

```

Part 9: ELF Inspection Tools

```

; Essential tools for examining ELF files:

; readelf - comprehensive ELF analysis
; $ readelf -h prog          # ELF header
; $ readelf -S prog          # Section headers
; $ readelf -l prog          # Program headers (segments)
; $ readelf -s prog          # Symbol table
; $ readelf -r prog.o        # Relocations
; $ readelf -d prog          # Dynamic section
; $ readelf -n prog          # Notes (build ID)
; $ readelf -a prog          # Everything

; objdump - disassembly and headers
; $ objdump -d prog          # Disassemble .text
; $ objdump -D prog          # Disassemble all sections
; $ objdump -t prog          # Symbol table
; $ objdump -r prog.o        # Relocations
; $ objdump -x prog          # All headers

; nm - symbol listing
; $ nm prog                  # List symbols (address, type, name)
; $ nm -D prog                # Dynamic symbols only

; hexdump - raw bytes
; $ hexdump -C prog | head   # View ELF magic and header bytes
; $ xxd prog | head

; file - identify file type
; $ file prog
; prog: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
;       statically linked, not stripped

; strip - remove symbols (smaller binary)
; $ strip --strip-all prog # Remove all symbols
; $ strip --strip-debug prog # Remove only debug info

; size - section sizes
; $ size prog
;   text    data    bss    dec    hex filename
;    53      10      8     71     47 prog

```

Part 10: Self-Modifying ELF Tricks

```

; Writing a program that modifies its own ELF file on disk
; (Educational – demonstrates ELF structure understanding)

section .data
    self_path db "/proc/self/exe", 0
    ; Note: /proc/self/exe is a symlink to our own executable

section .bss
    elf_header_buf resb 64 ; Buffer for reading our own ELF header

section .text
    global _start

_start:
    ; Read our own ELF header from /proc/self/exe
    ; (Can't write to it – it's the running executable)
    ; But we can READ and understand our own structure

    ; Open ourselves
    mov rax, 2 ; sys_open
    lea rdi, [rel self_path]
    xor rsi, rsi ; 0_RDONLY
    xor rdx, rdx
    syscall
    mov r12, rax

    ; Read ELF header
    mov rax, 0 ; sys_read
    mov rdi, r12
    lea rsi, [rel elf_header_buf]
    mov rdx, 64 ; ELF header size
    syscall

    ; Parse: verify magic
    lea rdi, [rel elf_header_buf]
    cmp byte [rdi], 0x7F
    jne .not_elf
    cmp byte [rdi+1], 'E'
    jne .not_elf
    cmp byte [rdi+2], 'L'
    jne .not_elf
    cmp byte [rdi+3], 'F'
    jne .not_elf

    ; Read entry point (offset 24 in ELF64 header)
    mov rax, [rdi + 24] ; e_entry
    ; RAX now contains our own entry point address!

    ; Read number of program headers (offset 56, 2 bytes)
    movzx ecx, word [rdi + 56] ; e_phnum

```

```

; Close
mov rax, 3
mov rdi, r12
syscall

; Exit with number of program headers as status
mov rax, 60
mov rdi, rcx
syscall

.not_elf:
mov rax, 60
mov rdi, 1
syscall

```

Exercises

1. **Minimal ELF:** Create the smallest possible valid ELF executable (aim for under 200 bytes) that prints "Hi\n" and exits. Write the ELF header manually in NASM.
 2. **ELF parser:** Write an assembly program that reads an ELF file, parses its header, and prints: type, architecture, entry point, and number of segments.
 3. **Symbol resolver:** Given an object file, read its symbol table and print all global function symbols with their addresses.
 4. **Two-file linking:** Create two .asm files that call each other's functions. Observe the relocation entries in the .o files and verify they're resolved in the final executable.
 5. **Shared library:** Create a simple `.so` shared library in assembly, then write a program that calls it through PLT/GOT.
-

Key Takeaways

Concept	Reality
ELF Header	64 bytes at offset 0; magic(7F ELF), type, entry point, table offsets
Sections	Logical divisions (.text/.data/.bss) for linker and debugger
Segments	Loadable chunks for kernel; each maps to a memory region
Relocations	"Patch address X with symbol Y's final address"
Linker	Resolves symbols, assigns addresses, patches relocations
GOT/PLT	Indirection for dynamic linking; lazy binding on first call
PIE/ASLR	RIP-relative addressing enables random load addresses
Program loading	Kernel mmap's segments, sets up stack, jumps to entry

Next Topic

Topic 26: Interrupts & Exceptions → — How the CPU handles hardware interrupts, software exceptions, and the interrupt descriptor table.

CHAPTER 43

Topic 26: Interrupts & Exceptions

Overview

Interrupts are the mechanism by which hardware devices and software errors communicate with the CPU. They temporarily suspend normal execution to handle urgent events — from keyboard presses to division by zero errors. This topic explains the complete interrupt architecture of x86-64: the Interrupt Descriptor Table (IDT), exception handling, hardware IRQs, and how the `syscall` instruction relates to (and replaced) the old `int 0x80` mechanism.

```
// Interrupts affect your assembly code in several ways:  
// 1. Your program can be interrupted at ANY instruction boundary  
// 2. Segfaults, divide-by-zero → CPU generates exceptions  
// 3. The 'syscall' instruction is a controlled interrupt to kernel  
// 4. Signal handlers are delivered via interrupt-like mechanisms  
  
// You don't directly handle interrupts in user-mode programs,  
// but understanding them explains:  
// - Why segfaults happen  
// - How system calls work internally  
// - Why NMI/timer interrupts preempt your code  
// - How debugger breakpoints work (INT 3)
```

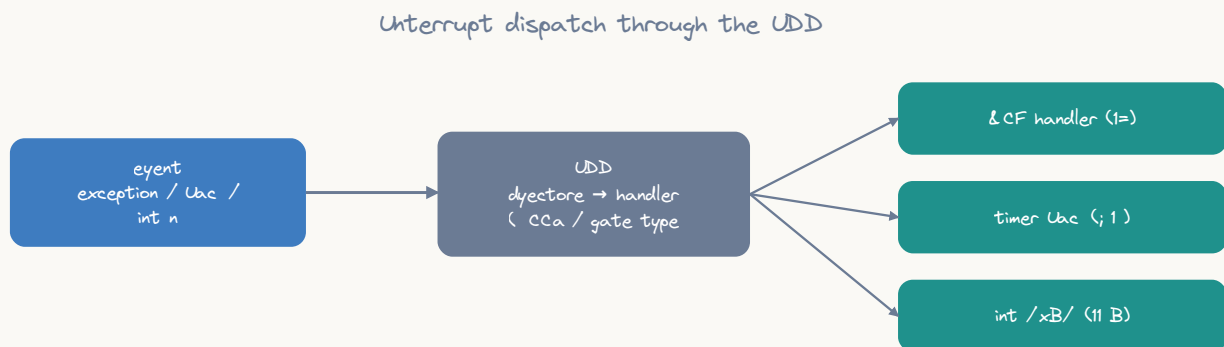
Part 1: Interrupt Types

x86-64 Interrupt Classification:

INTERRUPTS	
Synchronous (caused by CPU)	Asynchronous (external events)
Exceptions:	Hardware Interrupts (IRQs):
Faults: #DE Divide Error #PF Page Fault #GP General Prot. (instruction retried after fix)	Maskable (INTR pin): IRQ0 Timer IRQ1 Keyboard IRQ8 RTC IRQ12 Mouse IRQ14 Disk (can be disabled via CLI)
Traps: #DB Debug #BP Breakpoint #OF Overflow (next instruction after trap)	Non-Maskable (NMI): Hardware failures Watchdog timeout (CANNOT be disabled)
Aborts: #DF Double Fault #MC Machine Check (unrecoverable)	Software Interrupts: INT n (explicit interrupt) INT 3 (breakpoint) INT 0x80 (old Linux syscall) SYSCALL (modern fast syscall)

Part 2: The Interrupt Descriptor Table (IDT)

IDT Structure



the CPU pushes SS=CS, RAX=CS, CS=0, then jumps to the gated handler

ASCII diagram

The IDT maps interrupt/exception numbers (0-255) to handler addresses:

IDT Register (IDTR):

Limit (16 bits)	Base Address (64 bits)
-----------------	------------------------

(loaded with LIDT instruction by kernel at boot)

IDT Entry (Gate Descriptor, 16 bytes on x86-64):

Bytes 0-1:	Offset low (bits 15:0 of handler address)
Bytes 2-3:	Segment Selector (code segment, usually 0x08)
Byte 4:	IST (Interrupt Stack Table, bits 2:0)
Byte 5:	Type (0xE=Interrupt Gate, 0xF=Trap Gate) + DPL (ring level) + Present bit
Bytes 6-7:	Offset mid (bits 31:16)
Bytes 8-11:	Offset high (bits 63:32)
Bytes 12-15:	Reserved (must be 0)

Interrupt Gate vs Trap Gate:

Interrupt Gate: automatically clears IF (disables further interrupts)

Trap Gate: keeps IF unchanged (interrupts remain enabled)

Exceptions (like #PF): use Trap Gate (kernel needs interrupts for I/O)

Hardware IRQs: use Interrupt Gate (prevent interrupt nesting)

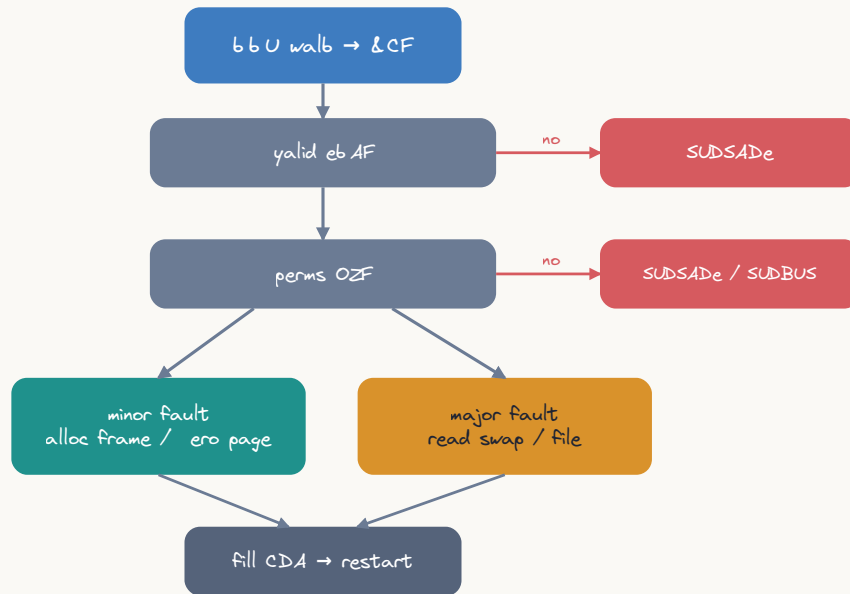
IDT Entries for x86-64 Exceptions

Vector	Name	Type	Error Code?
0	#DE Divide Error	Fault	No
1	#DB Debug	Fault/Trap	No
2	NMI Non-Maskable Int	Interrupt	No
3	#BP Breakpoint	Trap	No
4	#OF Overflow	Trap	No
5	#BR Bound Range	Fault	No
6	#UD Invalid Opcode	Fault	No
7	#NM Device Not Avail	Fault	No
8	#DF Double Fault	Abort	Yes (always 0)
9	(reserved)		
10	#TS Invalid TSS	Fault	Yes
11	#NP Segment Not Present	Fault	Yes
12	#SS Stack Segment Fault	Fault	Yes
13	#GP General Protection	Fault	Yes
14	#PF Page Fault	Fault	Yes
15	(reserved)		
16	#MF x87 FP Exception	Fault	No
17	#AC Alignment Check	Fault	Yes
18	#MC Machine Check	Abort	No
19	#XM SIMD FP Exception	Fault	No
20	#VE Virtualization	Fault	No
21-31	(reserved by Intel)		
32-255	Available for OS use (IRQs, syscalls, etc.)		

Part 3: What Happens During an Exception

Page Fault (#PF, Vector 14) — Step by Step

Page-fault handling (&CF → don't page fault)



Your code: `mov rax, [0x0]` ← NULL pointer dereference!

CPU Page Fault Sequence:

1. MMU detects page not present (P=0 in PTE for address 0x0)
2. CPU pushes to KERNEL stack (found via TSS.RSP0):

SS	(user stack seg)	RSP+40
RSP	(user stack ptr)	RSP+32
RFLAGS	(saved flags)	RSP+24
CS	(user code seg)	RSP+16
RIP	(faulting instr)	RSP+8 ← points to mov!
Error Code (page fault)		RSP+0
3. CPU stores faulting address in CR2
CR2 = 0x0000000000000000
4. CPU loads handler address from IDT[14]
RIP = `page_fault_handler` (kernel function)
5. CPU clears IF if Interrupt Gate (keeps for Trap Gate)
6. CPU changes to ring 0 (kernel mode)
7. Execution continues at kernel's `page_fault_handler`

Error code for #PF:

- Bit 0: P - 0=not present, 1=protection violation
- Bit 1: W/R - 0=read, 1=write caused fault
- Bit 2: U/S - 0=kernel mode, 1=user mode
- Bit 3: RSVD - 1=reserved bits set in PTE
- Bit 4: I/D - 1=instruction fetch

For NULL deref: `error_code = 0b00100 = 4`
(user mode, read access, page not present)

General Protection Fault (#GP, Vector 13)

```

; Common causes of #GP:
; 1. Accessing kernel memory from user mode
; 2. Writing to read-only segment
; 3. Exceeding segment limit
; 4. Invalid system instruction in user mode

; This triggers #GP in user mode:
section .text
    global _start
_start:
    ; Attempt to read kernel memory → #GP → SIGSEGV
    ; mov rax, [0xFFFF800000000000] ; kernel address!

    ; Attempt privileged instruction → #GP → SIGILL or SIGSEGV
    ; cli                      ; clear interrupt flag (ring 0 only!)
    ; hlt                      ; halt CPU (ring 0 only!)
    ; mov cr3, rax             ; write control register (ring 0 only!)
    ; lgdt [rdi]               ; load GDT (ring 0 only!)

    ; The kernel's #GP handler will:
    ; 1. Check if fault occurred in user mode (error code bit 2)
    ; 2. If user mode: deliver SIGSEGV to the process
    ; 3. If kernel mode: oops/panic!

    mov rax, 60
    xor rdi, rdi
    syscall

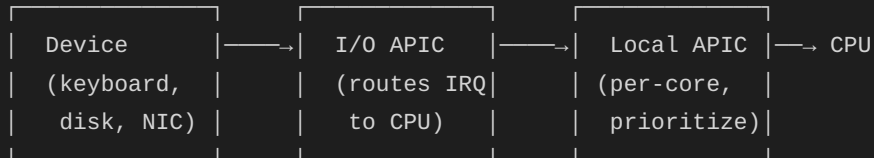
```

Part 4: Hardware Interrupts (IRQs)

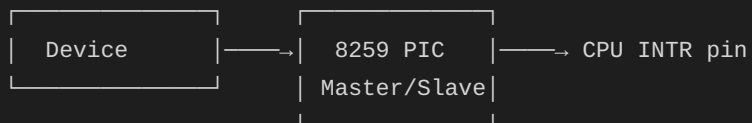
Interrupt Delivery Path

Hardware device → Interrupt Controller → CPU → IDT → Handler

Modern path (x2APIC):



Legacy path (8259 PIC):



IRQ → IDT vector mapping (typical Linux):

```

IRQ 0  (Timer)    → Vector 32
IRQ 1  (Keyboard) → Vector 33
IRQ 8  (RTC)      → Vector 40
IRQ 14 (IDE disk) → Vector 46
(Vectors 0-31 reserved for CPU exceptions)
  
```


Timer Interrupt (IRQ 0) — How Preemption Works

Timer fires every ~1ms (HZ=1000 or tickless):

1. CPU is executing your user-mode code
2. Timer hardware fires IRQ 0
3. CPU:
 - Finishes current instruction
 - Saves state to kernel stack (SS, RSP, RFLAGS, CS, RIP)
 - Loads IDT[32] handler address
 - Switches to ring 0
4. Kernel timer handler:
 - Update jiffies (time accounting)
 - Check if current process used up its time slice
 - If yes: set TIF_NEED_RESCHED flag
 - Return from interrupt (IRET)
5. On return path to user mode, kernel checks TIF_NEED_RESCHED:
 - If set: call schedule() → context switch to another process!
 - Your code is now paused until rescheduled

This is INVOLUNTARY preemption:

Your code has NO say in when it's interrupted

The OS guarantees fair CPU sharing among processes

Part 5: Debugger Breakpoints (INT 3)

```

; INT 3 (0xCC) is a single-byte instruction that triggers exception #BP
; Debuggers use it to implement breakpoints:
;
; 1. GDB reads the byte at breakpoint address
; 2. GDB replaces it with 0xCC (INT 3)
; 3. When CPU executes 0xCC → trap #BP → kernel delivers SIGTRAP
; 4. GDB catches SIGTRAP, restores original byte
; 5. User inspects state, then GDB single-steps past, re-inserts 0xCC

section .text
    global _start

_start:
    ; Insert our own breakpoint (for debugging without GDB)
    ; This will cause SIGTRAP → program stops (or core dump if no debugger)
    int 3                ; 0xCC – breakpoint trap!

    ; If debugger is attached, execution resumes here after user continues

    ; Self-debugging technique: set up SIGTRAP handler
    ; (see signal handling in Topic 22)

    mov rax, 60
    xor rdi, rdi
    syscall

; How single-stepping works (TRAP flag):
; The CPU has a TF (Trap Flag) in RFLAGS
; When TF=1, CPU generates #DB exception after EVERY instruction
; GDB sets TF via ptrace to implement "step" command
;
; pushfq                ; Push RFLAGS
; or qword [rsp], 0x100 ; Set TF (bit 8)
; popfq                 ; Pop modified RFLAGS
; ; Next instruction will trigger #DB trap!
; nop                   ; ← #DB exception fires here

```

Hardware Breakpoints (Debug Registers)

```

; x86-64 has 4 debug address registers (DR0-DR3)
; and control register DR7 for configuring them
; These enable breakpoints WITHOUT modifying code!

; DR0-DR3: breakpoint addresses (up to 4 simultaneous)
; DR6: debug status (which breakpoint triggered)
; DR7: debug control (enable/disable, condition, length)

; Types of hardware breakpoints:
;   Execute: break when RIP reaches address (like INT 3, but no code change)
;   Write:    break when address is WRITTEN (watchpoint)
;   Read/Write: break on any access (data breakpoint)
;   I/O:      break on port access (privileged)

; In user mode, you can't set DR registers directly
; (they're privileged – only kernel/debugger via ptrace)
;
; GDB's "watch" command uses hardware breakpoints:
;   (gdb) watch my_variable
;   → ptrace sets DR0 = &my_variable, DR7 = write-break
;   → CPU triggers #DB on any write to that address
;   → Kernel delivers SIGTRAP to debugger
;   → GDB shows "Watchpoint hit: my_variable changed"

; Using ptrace to set debug registers (from a debugger process):
; ptrace(PTRACE_POKEUSER, child_pid, offsetof(user, u_debugreg[0]), addr)
; ptrace(PTRACE_POKEUSER, child_pid, offsetof(user, u_debugreg[7]), ctrl)

```

Part 6: The syscall Instruction vs INT 0x80

Legacy: INT 0x80 (32-bit Linux)

```

; INT 0x80 uses the IDT mechanism:
; 1. CPU looks up IDT[0x80]
; 2. Saves SS, RSP, RFLAGS, CS, RIP to kernel stack
; 3. Loads handler from IDT entry
; 4. Handler dispatches based on EAX (syscall number)
; 5. Returns via IRET

; Slow because:
; - Full privilege level checks
; - IDT lookup
; - Stack switching via TSS
; - IRET is complex (loads 5 values from stack)

mov eax, 4          ; sys_write (32-bit number!)
mov ebx, 1          ; fd
mov ecx, msg        ; buffer
mov edx, len        ; count
int 0x80            ; Trigger software interrupt 0x80
; ~200-400 cycles for the transition

```

Modern: SYSCALL/SYSRET (64-bit)

```

; SYSCALL is a special instruction (not via IDT):
; 1. Saves RIP in RCX, RFLAGS in R11
; 2. Loads kernel RIP from MSR_LSTAR
; 3. Loads kernel CS from MSR_STAR
; 4. Masks RFLAGS with MSR_FMASK
; 5. NO stack switch (kernel must do it manually)
; 6. NO privilege checks (hardcoded ring 0 transition)
;
; Much faster: ~50-100 cycles for the transition

; SYSRET (kernel → user mode):
; 1. Loads RIP from RCX
; 2. Loads RFLAGS from R11
; 3. Loads user CS/SS from MSR_STAR
; 4. Sets CPL = 3 (user mode)
;
; Even faster than IRET: ~30-50 cycles

mov rax, 1          ; sys_write (64-bit number)
mov rdi, 1          ; fd
lea rsi, [rel msg]  ; buffer
mov rdx, len        ; count
syscall             ; Fast path to kernel
; RCX and R11 are clobbered!
; (RCX = return RIP, R11 = saved RFLAGS)

```

Why RCX and R11 are Clobbered

```

; IMPORTANT: After syscall, RCX and R11 contain kernel data:
;   RCX = the return address (old RIP)
;   R11 = the old RFLAGS
;
; If you need RCX/R11 across a syscall, save them first!

section .text
_start:
    mov rcx, 42          ; Important value in RCX
    mov r11, 99          ; Important value in R11

    ; Save before syscall
    push rcx
    push r11

    mov rax, 1
    mov rdi, 1
    lea rsi, [rel msg]
    mov rdx, 5
    syscall              ; Destroys RCX and R11!

    ; Restore after syscall
    pop r11              ; R11 = 99 again
    pop rcx              ; RCX = 42 again

    mov rax, 60
    xor rdi, rdi
    syscall

section .rodata
    msg db "Hello"

```

Part 7: vDSO — Virtual Dynamically-linked Shared Object

```

; Some "syscalls" don't actually enter the kernel!
; The vDSO is a kernel-mapped shared library in every process's address space
;
; Functions in vDSO (execute entirely in user mode):
;   clock_gettime()  - reads time without syscall
;   gettimeofday()  - reads time without syscall
;   time()           - reads time without syscall
;   getcpu()         - reads CPU number without syscall
;
; How it works:
; 1. Kernel maps a special page into every process (visible in /proc/self/maps)
; 2. The page contains optimized code that reads kernel-maintained data
; 3. Kernel updates a shared memory region with current time, CPU info, etc.
; 4. User-mode code reads this shared region directly (no syscall needed!)
;
; Result: clock_gettime() takes ~20ns instead of ~100ns
;
; Finding the vDSO:
; - In auxiliary vector (AT_SYSINFO_EHDR, type 33)
; - Or by parsing /proc/self/maps (look for [vdso])

; The vDSO page appears at a random address (ASLR):
; $ cat /proc/self/maps | grep vdso
; 7ffd1b9f4000-7ffd1b9f6000 r-xp 00000000 00:00 0 [vdso]
;
; $ objdump -d /path/to/extracted/vdso.so
; Shows user-mode implementations of clock_gettime, etc.

```

Part 8: Interrupt Latency and Real-Time

```

; Interrupt latency = time from hardware event to handler execution
;
; Sources of latency:
; 1. Current instruction must complete (can be slow for DIV, REP)
; 2. CPU pipeline flush
; 3. Mode switch overhead
; 4. Handler prologue (save registers)
; 5. If interrupts were disabled (CLI): must wait for STI
;
; Typical latency on modern x86-64:
;   Best case: ~1µs (interrupt already enabled, short instruction)
;   Worst case: ~100µs (interrupts disabled, long critical section)
;
; Critical for:
;   Audio: must deliver samples every ~5ms (44.1kHz × buffer size)
;   Network: packet processing at 10Gbps = ~67ns per packet
;   Storage: NVMe completion in ~10µs

; CLI/STI – Disable/Enable interrupts (kernel only!):
; cli                                ; Clear Interrupt Flag (IF=0)
;                                   ; Maskable interrupts are now blocked!
;                                   ; ... critical section ...
; sti                                ; Set Interrupt Flag (IF=1)
;                                   ; Interrupts can fire again

; In user mode, CLI/STI trigger #GP (privilege violation)
; User programs can't disable interrupts – only the kernel can
; This ensures the timer interrupt can always preempt user code

```


Part 9: Exception Handling in Practice

Catching SIGSEGV (from Assembly)

```

; When your code triggers an exception (e.g., page fault on invalid address):
; 1. CPU generates exception → kernel handler
; 2. Kernel checks: is this a valid fault? (VMA lookup)
; 3. If invalid: deliver signal to process
;    - SIGSEGV for memory access violations
;    - SIGFPE for division errors
;    - SIGILL for illegal instructions
;    - SIGBUS for bus errors (alignment on some archs)
;
; You CAN catch these signals and recover!

section .data
    segv_msg db "Caught segfault! Recovered gracefully.", 10
    segv_len equ $ - segv_msg

    align 8
    sa_struct:
        dq segv_handler      ; sa_handler
        dq 0x04000004         ; sa_flags = SA_RESTORER | SA_SIGINFO
        dq sa_restorer        ; sa_restorer
        times 16 db 0         ; sa_mask

    jmp_buf resb 200          ; Save point for recovery

section .text
    global _start

sa_restorer:
    mov rax, 15               ; sys_rt_sigreturn
    syscall

segv_handler:
    ; Signal handler for SIGSEGV
    ; At this point, we can't just return (would re-execute faulting instruction!)
    ; Instead, modify the saved RIP to skip past the bad instruction
    ;
    ; RDI = signal number
    ; RSI = siginfo_t*
    ; RDX = ucontext_t* (contains saved registers)

    ; ucontext_t layout (simplified):
    ; [rdx+0x28..]: mcontext_t.gregs[] (register file)
    ; REG_RIP is at index 16 in gregs array
    ; Each greg is 8 bytes, offset = 0x28 + 16*8 = 0xA8

    ; Skip the faulting instruction (advance RIP past it)
    ; mov instruction is typically 7 bytes for mov rax, [addr]
    add qword [rdx + 0xA8], 7 ; Skip past faulting instruction

    ; Print message

```

```

push rax
push rdi
push rsi
push rdx
mov rax, 1
mov rdi, 1
lea rsi, [rel segv_msg]
mov rdx, segv_len
syscall
pop rdx
pop rsi
pop rdi
pop rax
ret

_start:
; Install SIGSEGV handler
mov rax, 13          ; sys_rt_sigaction
mov rdi, 11          ; SIGSEGV
lea rsi, [rel sa_struct]
xor rdx, rdx         ; old_act = NULL
mov r10, 8           ; sigsetsize
syscall

; Deliberately trigger segfault
xor rax, rax
mov rax, [rax]       ; Read from NULL! → SIGSEGV
; Handler advances RIP, execution continues here

; We survived the segfault!
mov rax, 60
xor rdi, rdi         ; Exit success
syscall

```

Division by Zero Handler

```

; Integer division by zero triggers #DE (exception 0)
; Kernel delivers SIGFPE to the process

section .data
    fpe_msg db "Division by zero caught!", 10
    fpe_len equ $ - fpe_msg

    align 8
    fpe_sa:
        dq fpe_handler
        dq 0x04000004          ; SA_RESTORER | SA_SIGINFO
        dq fpe_restorer
        times 16 db 0

section .text
    global _start

fpe_restorer:
    mov rax, 15
    syscall

fpe_handler:
    ; Skip the DIV instruction (~2 bytes for div ecx)
    add qword [rdi + 0xA8], 2

    ; Set RAX to a safe value (so code can continue)
    ; mcontext gregs: RAX is at index 13, offset = 0x28 + 13*8 = 0x90
    mov qword [rdi + 0x90], 0 ; Set recovered RAX = 0

    mov rax, 1
    mov rdi, 2                ; stderr
    lea rsi, [rel fpe_msg]
    mov rdx, fpe_len
    syscall
    ret

_start:
    ; Install SIGFPE handler
    mov rax, 13
    mov rdi, 8                ; SIGFPE
    lea rsi, [rel fpe_sa]
    xor rdx, rdx
    mov r10, 8
    syscall

    ; Trigger division by zero
    mov eax, 42
    xor ecx, ecx              ; ECX = 0
    div ecx                   ; 42 / 0 → #DE → SIGFPE!
    ; Handler skips past div, sets RAX=0, continues here

```

```
; Survived!  
mov rax, 60  
xor rdi, rdi  
syscall
```

Part 10: Interrupt Descriptor Table Setup (Kernel Context)

```

; This section shows how the kernel sets up the IDT at boot
; You can't run this in user mode, but understanding it
; completes the picture of how interrupts work

; NOTE: This is x86-64 kernel-mode code (for educational purposes)
; Setting up a single IDT entry:

; %macro SET_IDT_ENTRY 3    ; vector, handler_addr, type_attr
;     ; Calculate entry address: IDT_BASE + vector * 16
;     mov rdi, IDT_BASE
;     add rdi, %1 * 16
;
;     ; Write the 16-byte gate descriptor:
;     mov rax, %2          ; Handler address
;
;     ; Bytes 0-1: offset low
;     mov word [rdi], ax
;
;     ; Bytes 2-3: code segment selector
;     mov word [rdi+2], 0x08 ; Kernel code segment
;
;     ; Byte 4: IST (0 = don't use IST)
;     mov byte [rdi+4], 0
;
;     ; Byte 5: type + DPL + present
;     mov byte [rdi+5], %3    ; e.g., 0x8E = present, DPL=0, interrupt gate
;
;     ; Bytes 6-7: offset mid
;     shr rax, 16
;     mov word [rdi+6], ax
;
;     ; Bytes 8-11: offset high
;     shr rax, 16
;     mov dword [rdi+8], eax
;
;     ; Bytes 12-15: reserved
;     mov dword [rdi+12], 0
; %endmacro

; Example entries:
; SET_IDT_ENTRY 0, divide_error_handler, 0x8F    ; #DE (Trap gate)
; SET_IDT_ENTRY 3, breakpoint_handler, 0xEF      ; #BP (Trap gate, DPL=3!)
; SET_IDT_ENTRY 14, page_fault_handler, 0x8E     ; #PF (Interrupt gate)
; SET_IDT_ENTRY 32, timer_handler, 0x8E         ; Timer IRQ

; Note: #BP (breakpoint) has DPL=3, meaning user-mode code CAN trigger it
; with INT 3. All other exceptions have DPL=0 (only kernel can INT them).

; Load the IDT:
; lidt [idtr_descriptor]

```



```

; idtr_descriptor:
;     dw IDT_SIZE - 1      ; Limit
;     dq IDT_BASE          ; Base address

```

Exercises

- Exception explorer:** Write a program that installs handlers for SIGSEGV, SIGFPE, and SIGILL, then deliberately triggers each one. Print which exception was caught.
- Breakpoint insertion:** Write a simple debugger that `fork()`s, uses `ptrace` to insert INT 3 at a specific address in the child, and catches the resulting SIGTRAP.
- Timing interrupts:** Use `rdtsc` to measure how long a tight loop iteration takes. Run it many times and observe the occasional spike caused by timer interrupts preempting your code.
- Signal vs exception:** Compare the behavior of `kill(getpid(), SIGSEGV)` (signal delivery) vs actually dereferencing NULL (CPU exception). How does the handler's ucontext differ?
- vDSO discovery:** Parse the auxiliary vector to find AT_SYSINFO_EHDR, dump the vDSO ELF, and disassemble `clock_gettime` to see how it reads time without a syscall.

Key Takeaways

Concept	Hardware Reality
IDT	256-entry table mapping vectors to handler addresses
Exception	Synchronous: caused by instruction (fault/trap/abort)
Interrupt	Asynchronous: caused by hardware device (IRQ)
Page Fault	#PF (vector 14): CPU pushes error code + saves RIP to kernel stack
INT 3	Single-byte (0xCC) breakpoint trap; debuggers patch code with it
SYSCALL	Not an interrupt! Uses MSRs for fast ring transition
Timer IRQ	Fires every ~1ms; enables preemptive multitasking
CLI/STI	Kernel-only instructions to mask/unmask IRQs
Signal delivery	Kernel modifies user stack to call signal handler on return

Next Topic

[Topic 27: Context Switching →](#) — How the OS saves one process's state and restores another's, enabling multitasking.

CHAPTER 44

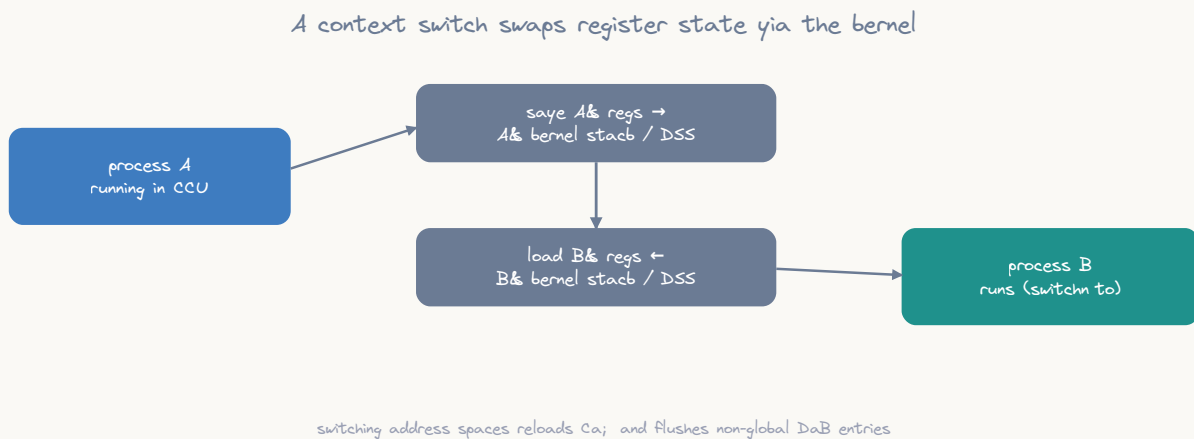
Topic 27: Context Switching & Scheduling

Overview

When the OS switches from running Process A to Process B, it must save every piece of A's execution state and restore B's — all without either process noticing. This is a **context switch**, and it's the foundation of multitasking. This topic explains exactly what state is saved, how the kernel performs the switch, and how the scheduler decides which process runs next.

```
// From the process's perspective:
// - It was executing instruction N
// - "Instantly" it's executing instruction N+1
// - But between those two instructions, the OS may have:
//   1. Saved all our registers to memory
//   2. Ran a completely different process for 10ms
//   3. Restored our registers from memory
//   4. Resumed us exactly where we left off
// We never know the difference!
```

Part 1: What Is Process Context?



ASCII diagram

Complete process context (everything that defines execution state):

```
| CPU Registers (saved/restored on context switch): |
|   General Purpose: RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP |
|               R8-R15 |
|   Instruction Pointer: RIP |
|   Flags: RFLAGS |
|   Segment Registers: CS, DS, ES, FS, GS, SS |
|   FPU/SSE/AVX: x87 FPU state, XMM0-XMM15, YMM0-YMM15 |
|               MXCSR, FPU control/status words |
|   Debug Registers: DR0-DR3, DR6, DR7 |
| |
| Memory State (switched via CR3): |
|   Page Tables (entire virtual address space mapping) |
|   TLB entries (flushed on CR3 change, unless PCID) |
| |
| Kernel State: |
|   Kernel stack (each process has its own) |
|   task_struct (process descriptor in kernel memory) |
|   File descriptor table, signal masks, credentials... |
```

What ISN'T switched:

- Kernel code and data (same in all processes)
- Hardware state (I/O APIC, PCI devices)
- CPU caches (L1/L2/L3) – NOT flushed, but effectively cold
- Branch predictor state – NOT flushed, can cause slowdown

Part 2: The Context Switch in Detail

Voluntary Switch (Process Blocks on I/O)

Process A calls `read(fd, buf, 4096)`:

1. A enters kernel via `syscall`
2. Kernel finds: data not ready (e.g., waiting for disk)
3. Kernel marks A as `TASK_INTERRUPTIBLE` (sleeping)
4. Kernel calls `schedule()`

`schedule()`:

- | |
|---|
| 1. Pick next process to run (scheduler algorithm) |
| <code>next = pick_next_task()</code> → selects Process B |
| 2. <code>context_switch(A, B)</code> : |
| a. <code>switch_mm(A->mm, B->mm)</code> : |
| - Load B's page tables: <code>mov cr3, B->pgd</code> |
| - TLB flush happens (or PCID avoids it) |
| b. <code>switch_to(A, B)</code> : |
| - Save A's kernel registers on A's kernel stack |
| - Switch kernel stack pointer to B's kernel stack |
| - Restore B's kernel registers from B's kernel stack |
| - <code>RET</code> returns to wherever B was in the kernel! |
| 3. Now executing as Process B in kernel mode |
| 4. B's previous kernel path completes |
| 5. <code>SYSRET</code> back to B's user-mode code |

Involuntary Switch (Timer Preemption)

Process A is running in user mode (no `syscall`):

1. Timer interrupt fires (`IRQ 0` → IDT vector 32)
2. CPU saves A's user state on A's kernel stack:
 `[SS, RSP, RFLAGS, CS, RIP]` – pushed by hardware
3. Timer handler runs:
 - Update timekeeping
 - Decrement A's time slice
 - If time slice expired: set `TIF_NEED_RESCHED`
4. On return from interrupt, kernel checks `TIF_NEED_RESCHED`:
 - If set: call `schedule()` instead of returning to user mode
5. `schedule()` → `context_switch(A, B)` [same as above]
6. A is suspended mid-instruction (from A's perspective)

Part 3: The switch_to Macro (Assembly)

Simplified Linux switch_to (x86-64)

```

; This is kernel code – the actual assembly that swaps processes
; Simplified from arch/x86/kernel/process_64.c / entry_64.S

; switch_to(prev, next):
; Saves prev's callee-saved registers, switches stack, restores next's

; Input:  RDI = prev task_struct, RSI = next task_struct
; The task_struct contains a 'thread' field with saved SP and other state

; Offsets in task_struct (simplified):
THREAD_SP    equ 0x08      ; Saved kernel RSP
THREAD_FLAGS equ 0x10      ; Thread flags

__switch_to_asm:
    ; Save prev's callee-saved registers on prev's kernel stack
    push rbp
    push rbx
    push r12
    push r13
    push r14
    push r15

    ; Save prev's kernel stack pointer
    mov [rdi + THREAD_SP], rsp    ; prev->thread.sp = RSP

    ; Load next's kernel stack pointer
    mov rsp, [rsi + THREAD_SP]    ; RSP = next->thread.sp

    ; Restore next's callee-saved registers from next's kernel stack
    pop r15
    pop r14
    pop r13
    pop r12
    pop rbx
    pop rbp

    ; Return to wherever next was when it was switched out
    ; (the return address is on next's kernel stack)
    ret

; That's it! The RET pops next's saved RIP and continues
; next's kernel execution path, which eventually returns to user mode

```

What's on Each Process's Kernel Stack

Process A's kernel stack (when A is switched OUT):

High address	
User SS	← Pushed by hardware (interrupt/syscall entry)
User RSP	
User RFLAGS	
User CS	
User RIP	

... kernel syscall path ...	← Whatever kernel functions were executing

Return address (to schedule())	← Where switch_to should return
RBP (saved by switch_to)	← Callee-saved registers
RBX	
R12	
R13	
R14	
R15	

	← RSP saved in A->thread.sp

When A is selected again:

1. Its RSP is restored from A->thread.sp
2. Pop R15-RBP restores its callee-saved registers
3. RET returns to schedule()'s caller
4. Kernel path continues (finishes syscall/interrupt handling)
5. SYSRET/IRET returns to user mode with A's saved user registers

Part 4: FPU/SSE/AVX State Switching

Lazy FPU Context Switching

FPU/SSE state is LARGE (512 bytes for FXSAVE, up to 2KB+ for AVX-512)
Saving/restoring it on every context switch would be expensive.

Optimization: Lazy FPU switching

1. On context switch: set CR0.TS bit (Task Switched)
2. DON'T save FPU state yet
3. If the new process uses an FPU instruction:
 - CPU generates #NM exception (Device Not Available)
 - Kernel handler: save old process's FPU, load new process's FPU
 - Clear TS bit, retry instruction

Modern approach: Eager FPU switching (Linux ≥ 4.2)

- Always save/restore FPU state on context switch
- Uses XSAVE/XRSTOR (handles SSE, AVX, AVX-512 in one shot)
- Simpler and actually faster on modern CPUs (no #NM overhead)

```
; FPU state save/restore instructions:

; FXSAVE/FXRSTOR: Save/restore x87 FPU + SSE state (512 bytes)
; fxsave [rdi]           ; Save FPU+SSE state to memory
; fxrstor [rsi]          ; Restore FPU+SSE state from memory

; XSAVE/XRSTOR: Save/restore extended state (FPU+SSE+AVX+...)
; The mask in EDX:EAX selects which components to save:
; Bit 0: x87 FPU
; Bit 1: SSE (XMM registers)
; Bit 2: AVX (upper YMM)
; Bit 5: AVX-512 opmask
; Bit 6: AVX-512 upper ZMM (ZMM0-15 upper)
; Bit 7: AVX-512 ZMM16-31

; Save all supported state:
; mov eax, 0xFFFFFFFF
; mov edx, 0xFFFFFFFF
; xsave [rdi]           ; Save to 'rdi' (must be 64-byte aligned)

; Restore:
; xrstor [rsi]          ; Restore from 'rsi'

; On context switch, the kernel does approximately:
; xsave [prev_task + FPU_OFFSET] ; Save current FPU state
; xrstor [next_task + FPU_OFFSET] ; Load next task's FPU state
```


Part 5: Thread Context Switching

Threads vs Processes

Process switch:

- Save all registers
- Switch page tables (CR3)
- Flush TLB (expensive!)
- Switch kernel stack
- All caches effectively cold

Thread switch (same process):

- Save all registers (SAME)
- NO page table switch!
- NO TLB flush!
- Switch kernel stack (SAME)
- Caches stay warm!

Thread switch is MUCH cheaper because:

- Same address space → same page tables → no CR3 write
- No TLB flush → cached translations still valid
- Shared memory → cache lines stay hot
- Cost: ~1-2μs (vs ~3-5μs for full process switch)

Thread Local Storage (TLS)

```

; Each thread has its own TLS area, accessed via FS segment register
; The kernel sets FS base for each thread during context switch

; arch_prctl(ARCH_SET_FS, addr) - set FS base
; arch_prctl(ARCH_GET_FS, &addr) - get FS base

section .bss
    my_tls resb 4096          ; Thread-local storage area

section .text
; Set up TLS for current thread:
setup_tls:
    mov rax, 158              ; sys_arch_prctl
    mov rdi, 0x1002           ; ARCH_SET_FS
    lea rsi, [rel my_tls]
    syscall
    ret

; Access thread-local variable:
; Variable at offset 0 in TLS:
get_thread_var:
    mov rax, [fs:0]           ; Read from FS-relative address
    ret

set_thread_var:
    mov [fs:0], rdi           ; Write to FS-relative address
    ret

; The kernel's context switch includes:
; wrmsrl(MSR_FS_BASE, next->thread.fsbase)
; This changes what FS:0 points to for the new thread

```

Part 6: Scheduler Algorithms

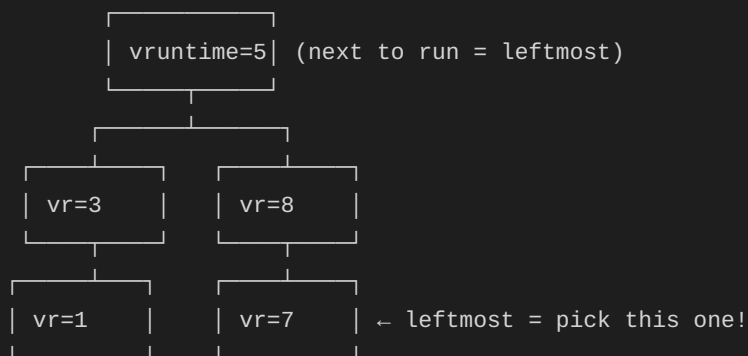
CFS (Completely Fair Scheduler)

Linux's default scheduler (CFS) uses a red-black tree of processes sorted by "virtual runtime" (vruntime).

Key concept: vruntime

- Each process accumulates vruntime as it runs
- Process with LOWEST vruntime is selected next
- Higher-priority processes accumulate vruntime SLOWER
- Result: all processes get "fair" share of CPU time

Red-Black Tree (sorted by vruntime):



Time slice calculation:

$$\text{slice} = (\text{scheduling_period} / \text{nr_running}) * (\text{nice_weight} / \text{total_weight})$$

Default scheduling_period: 6ms (if ≤ 8 runnable tasks)

For nice 0 (default): weight = 1024

For nice -20 (highest): weight = 88761

For nice 19 (lowest): weight = 15

Schedule Decision Points

```

; The scheduler is invoked at these points:

; 1. Voluntary: process blocks on I/O/lock
;   read() → no data → mark SLEEPING → schedule()

; 2. Timer tick: TIF_NEED_RESCHED set
;   Timer IRQ → check timeslice → set flag → on return: schedule()

; 3. Wake up: higher-priority task becomes runnable
;   I/O completion → wake up blocked task → if higher priority: preempt

; 4. Yield: process voluntarily gives up CPU
;   sched_yield syscall:
yield_cpu:
    mov rax, 24          ; sys_sched_yield
    syscall
    ; We might be switched out and back, or just continue
    ret

; 5. Process exit: must schedule another process
;   exit() → do_exit() → schedule() (never returns)

```

Part 7: Measuring Context Switch Cost

```

; Benchmark context switch overhead using pipe ping-pong:
; Parent and child alternate sending one byte through a pipe
; Each send+receive requires 2 context switches

section .bss
    pipe_r2w resd 2      ; Pipe: parent reads, child writes
    pipe_w2r resd 2      ; Pipe: parent writes, child reads
    byte_buf resb 1
    time_start resq 1
    time_end resq 1

section .data
    iterations equ 100000

section .text
    global _start

_start:
    ; Create two pipes
    mov rax, 22           ; sys_pipe
    lea rdi, [rel pipe_r2w]
    syscall
    mov rax, 22
    lea rdi, [rel pipe_w2r]
    syscall

    ; Fork
    mov rax, 57
    syscall
    test rax, rax
    jz .child

.parent:
    ; Close unused ends
    mov rax, 3
    mov edi, [rel pipe_r2w + 4] ; Close write end of r2w
    syscall
    mov rax, 3
    mov edi, [rel pipe_w2r]     ; Close read end of w2r
    syscall

    ; Get start time
    ; Use clock_gettime for high precision
    sub rsp, 16
    mov rax, 228               ; sys_clock_gettime
    mov rdi, 1                 ; CLOCK_MONOTONIC
    mov rsi, rsp               ; timespec buffer
    syscall
    mov rax, [rsp]             ; seconds
    imul rax, 1000000000

```

```

    add rax, [rsp + 8]      ; + nanoseconds
    mov [time_start], rax

    ; Ping-pong loop
    mov ecx, iterations
.parent_loop:
    push rcx

    ; Write one byte (triggers child to wake)
    mov rax, 1              ; sys_write
    mov edi, [rel pipe_w2r + 4] ; Write end
    lea rsi, [rel byte_buf]
    mov rdx, 1
    syscall

    ; Read one byte (blocks until child writes)
    mov rax, 0              ; sys_read
    mov edi, [rel pipe_r2w] ; Read end
    lea rsi, [rel byte_buf]
    mov rdx, 1
    syscall

    pop rcx
    dec ecx
    jnz .parent_loop

    ; Get end time
    mov rax, 228
    mov rdi, 1
    mov rsi, rsp
    syscall
    mov rax, [rsp]
    imul rax, 1000000000
    add rax, [rsp + 8]
    mov [time_end], rax
    add rsp, 16

    ; Calculate: (end - start) / (iterations * 2) = ns per context switch
    mov rax, [time_end]
    sub rax, [time_start]
    ; RAX = total nanoseconds
    ; Divide by (iterations * 2) for per-switch time
    xor rdx, rdx
    mov rcx, iterations * 2
    div rcx
    ; RAX = nanoseconds per context switch (typically 1000-5000)

    ; Wait for child and exit
    mov rax, 61
    mov rdi, -1
    xor rsi, rsi

```

```

xor rdx, rdx
xor r10, r10
syscall

mov rdi, rax          ; Exit with ns/switch as code (truncated)
mov rax, 60
syscall

.child:
    ; Close unused ends
    mov rax, 3
    mov edi, [rel pipe_r2w]    ; Close read end of r2w
    syscall
    mov rax, 3
    mov edi, [rel pipe_w2r + 4] ; Close write end of w2r
    syscall

    mov ecx, iterations
.child_loop:
    push rcx

    ; Read one byte (blocks until parent writes)
    mov rax, 0
    mov edi, [rel pipe_w2r]    ; Read end
    lea rsi, [rel byte_buf]
    mov rdx, 1
    syscall

    ; Write one byte back
    mov rax, 1
    mov edi, [rel pipe_r2w + 4] ; Write end
    lea rsi, [rel byte_buf]
    mov rdx, 1
    syscall

    pop rcx
    dec ecx
    jnz .child_loop

    mov rax, 60
    xor rdi, rdi
    syscall

```


Part 8: CPU Affinity and NUMA

```

; CPU affinity: pin a process to specific CPU cores
; Avoids migration overhead (cache cold start on new core)

; sched_setaffinity(pid, cpusetsize, mask):
; The mask is a bitmask where bit N = allowed on CPU N

section .bss
    cpu_mask resb 128      ; CPU affinity mask (supports up to 1024 CPUs)

section .text
; Pin current process to CPU 0 only:
pin_to_cpu0:
    ; Set bit 0 in mask (only CPU 0 allowed)
    lea rdi, [rel cpu_mask]
    mov qword [rdi], 1     ; Bit 0 = CPU 0

    mov rax, 203           ; sys_sched_setaffinity
    xor rdi, rdi           ; pid = 0 (self)
    mov rsi, 128           ; cpusetsize
    lea rdx, [rel cpu_mask]
    syscall
    ret

; Pin to CPUs 0 and 2:
pin_to_cpu02:
    lea rdi, [rel cpu_mask]
    mov qword [rdi], 5     ; Binary: 101 = CPUs 0 and 2

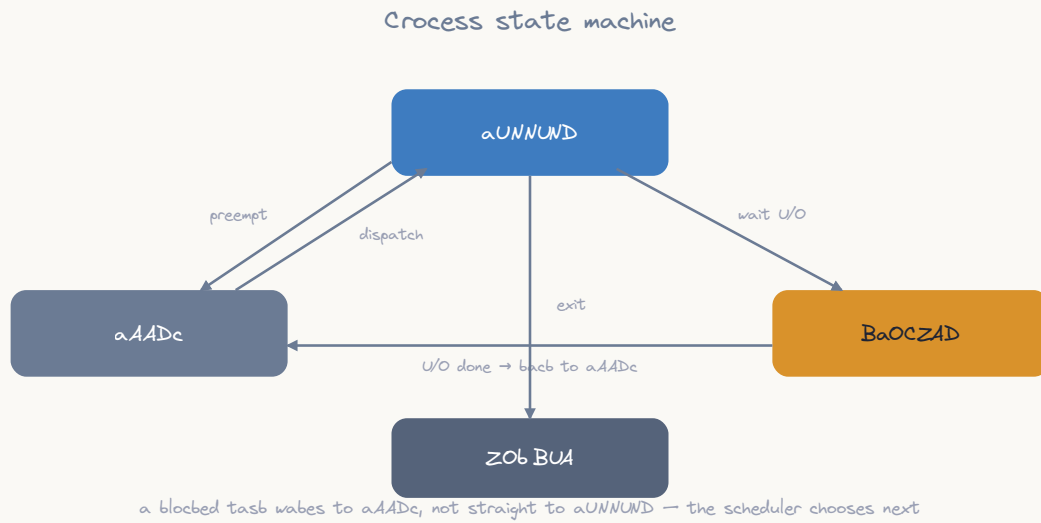
    mov rax, 203
    xor rdi, rdi
    mov rsi, 128
    lea rdx, [rel cpu_mask]
    syscall
    ret

; Get current CPU number (without syscall, using vDSO):
; Or via cpuid instruction:
get_current_cpu:
    ; rdtscp also returns CPU ID in ECX on modern CPUs
    rdtscp                 ; RAX:RDX = timestamp, ECX = CPU_ID
    mov eax, ecx           ; Return CPU number
    ret

; NUMA awareness:
; On NUMA systems, memory access time depends on which CPU accesses it
; Memory local to CPU 0 is faster for CPU 0 than for CPU 1
;
; sys_mbind, sys_set_mempolicy – control memory placement
; sys_migrate_pages – move pages between NUMA nodes

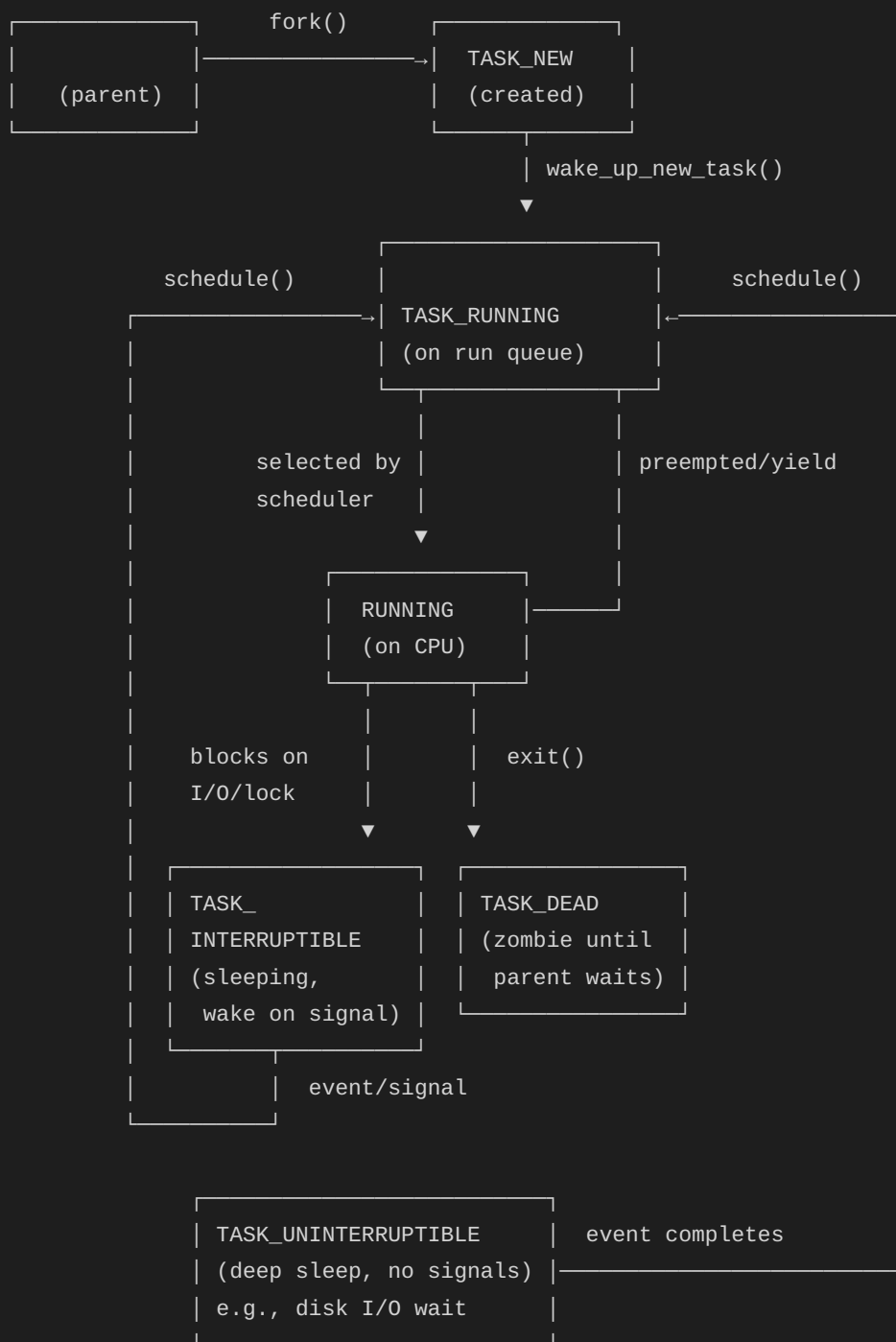
```

Part 9: Process States



ASCII diagram

Linux Process States:



```

; Observing process states from assembly:
; Read /proc/self/status for our own state
; Read /proc/[pid]/stat for another process's state

; State characters in /proc/[pid]/stat:
;   R = Running
;   S = Sleeping (interruptible)
;   D = Disk sleep (uninterruptible)
;   Z = Zombie
;   T = Stopped (by signal or debugger)
;   t = Tracing stop
;   X = Dead

section .data
    stat_path db "/proc/self/stat", 0

section .bss
    stat_buf resb 512

section .text
read_own_state:
    mov rax, 2
    lea rdi, [rel stat_path]
    xor rsi, rsi
    xor rdx, rdx
    syscall
    mov r12, rax

    mov rax, 0
    mov rdi, r12
    lea rsi, [rel stat_buf]
    mov rdx, 512
    syscall

    mov rax, 3
    mov rdi, r12
    syscall

; Parse: find state character (3rd field)
; Format: pid (comm) state ...
; Find the ')' then skip space
    lea rdi, [rel stat_buf]
.find_paren:
    cmp byte [rdi], ')'
    je .found
    inc rdi
    jmp .find_paren
.found:
    add rdi, 2           ; Skip ')' '

```

```
movzx eax, byte [rdi] ; State character (R, S, D, etc.)  
ret
```

Part 10: Coroutines — User-Space Context Switching

```

; You can implement your OWN context switching in user space!
; This is how coroutines, green threads, and async runtimes work
; (Go goroutines, Python asyncio, Rust async)

; Coroutine context (what we need to save):
struc coroutine
    .rsp resq 1          ; Saved stack pointer
    .stack resq 1        ; Stack base (for cleanup)
    .stack_size resq 1    ; Stack size
    .finished resb 1      ; Is this coroutine done?
endstruc

section .bss
    ; Two coroutines
    coro_a resb coroutine_size
    coro_b resb coroutine_size
    current_coro resq 1    ; Pointer to current coroutine

    ; Stacks for coroutines (8KB each)
    align 16
    stack_a resb 8192
    stack_b resb 8192

section .data
    msg_a db "Coroutine A running", 10
    msg_a_len equ $ - msg_a
    msg_b db "Coroutine B running", 10
    msg_b_len equ $ - msg_b
    msg_done db "Both coroutines finished!", 10
    msg_done_len equ $ - msg_done

section .text
    global _start

; Switch from current coroutine to target
; Input: RDI = target coroutine context pointer
; This is our user-space "context switch"!
coro_switch:
    ; Save current state
    mov rax, [current_coro]

    ; Save callee-saved registers on current stack
    push rbp
    push rbx
    push r12
    push r13
    push r14
    push r15

    ; Save current RSP

```



```

mov [rax + coroutine.rsp], rsp

; Switch to target
mov [current_coro], rdi
mov rsp, [rdi + coroutine.rsp]

; Restore target's registers
pop r15
pop r14
pop r13
pop r12
pop rbx
pop rbp

ret                ; "Returns" into target's execution!

; Coroutine A's function
coro_a_func:
    ; Print 3 times, yielding to B each time
    mov ecx, 3
.loop:
    push rcx
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel msg_a]
    mov rdx, msg_a_len
    syscall

    ; Yield to coroutine B
    lea rdi, [rel coro_b]
    call coro_switch

    pop rcx
    dec ecx
    jnz .loop

    ; Mark finished and switch back
    mov rax, [current_coro]
    mov byte [rax + coroutine.finished], 1
    lea rdi, [rel coro_b] ; Switch to B (which will notice A is done)
    call coro_switch
    ; Never returns

; Coroutine B's function
coro_b_func:
    mov ecx, 3
.loop:
    push rcx
    mov rax, 1
    mov rdi, 1
    lea rsi, [rel msg_b]

```

```

mov rdx, msg_b_len
syscall

; Yield to coroutine A
lea rdi, [rel coro_a]
call coro_switch

pop rcx
dec ecx
jnz .loop

; Mark finished and return to main
mov rax, [current_coro]
mov byte [rax + coroutine.finished], 1
lea rdi, [rel coro_a] ; Switch back (main is using coro_a's context)
call coro_switch

_start:
; Initialize coroutine A
lea rax, [rel stack_a + 8192 - 8] ; Top of stack A
mov qword [rax], coro_a_func ; "Return address" = entry point
sub rax, 48 ; Space for 6 saved registers
; Initialize saved registers to 0
mov qword [rax], 0 ; R15
mov qword [rax+8], 0 ; R14
mov qword [rax+16], 0 ; R13
mov qword [rax+24], 0 ; R12
mov qword [rax+32], 0 ; RBX
mov qword [rax+40], 0 ; RBP
mov [coro_a + coroutine.rsp], rax
mov byte [coro_a + coroutine.finished], 0

; Initialize coroutine B
lea rax, [rel stack_b + 8192 - 8]
mov qword [rax], coro_b_func
sub rax, 48
mov qword [rax], 0
mov qword [rax+8], 0
mov qword [rax+16], 0
mov qword [rax+24], 0
mov qword [rax+32], 0
mov qword [rax+40], 0
mov [coro_b + coroutine.rsp], rax
mov byte [coro_b + coroutine.finished], 0

; Use main's stack as a pseudo-coroutine for returning
; Start by running coroutine A
; (We need a "main" context to switch back to)
; Store our current context in coro_a temporarily for the initial switch

; Actually, let's set current to a dummy and switch to A

```

```

; Simpler: just call coro_a_func after setting up switch infrastructure

; Set current coroutine (main uses coro_a's slot initially for bookkeeping)
lea rax, [rel coro_a]
mov [current_coro], rax

; Start coroutine A (first switch – enters coro_a_func)
lea rdi, [rel coro_a]
call coro_switch

; When all coroutines finish, we end up back here
mov rax, 1
mov rdi, 1
lea rsi, [rel msg_done]
mov rdx, msg_done_len
syscall

mov rax, 60
xor rdi, rdi
syscall

```

Exercises

- Context switch timer:** Use the pipe ping-pong technique to measure context switch time on your system. Compare with `perf sched latency`.
 - CPU affinity:** Pin a process to CPU 0, measure loop performance. Then let it float across CPUs and measure again. Observe the difference.
 - User-space threads:** Extend the coroutine example to support N coroutines (use an array and round-robin scheduling).
 - Priority experiment:** Fork two children — one with `nice -20` (highest priority) and one with `nice 19` (lowest). Have both increment counters in a tight loop for 1 second. Compare final counts.
 - State observer:** Write a program that forks, has the child loop through different states (running, sleeping, stopped), while the parent reads `/proc/child_pid/stat` and reports state transitions.
-

Key Takeaways

Concept	What Happens
Context switch	Save registers + switch stack + switch page tables
Voluntary switch	Process blocks (I/O, lock) → calls schedule()
Involuntary	Timer interrupt → set NEED_RESCHED → schedule() on return
switch_to()	Push callee-saved regs, swap RSP, pop next's regs, RET
FPU state	XSAVE/XRSTOR handles all FP/SIMD registers (~2KB)
Thread switch	Same as process but NO CR3 change (much faster)
CFS scheduler	Red-black tree sorted by vruntime; pick lowest
Coroutines	User-space switch: save/restore RSP + callee-saved regs
Cost	Process switch: ~3-5μs, Thread switch: ~1-2μs

Next Topic

[Topic 28: Synchronization Primitives](#) → — Atomic operations, spinlocks, mutexes, and the futex syscall at the assembly level.

CHAPTER 45

Topic 28: Synchronization Primitives

Overview

When multiple threads or processes share memory, they need synchronization to prevent data races. This topic covers the complete hierarchy of synchronization mechanisms — from hardware atomic instructions up through OS-assisted mutexes — all at the assembly level.

```
// The fundamental problem:
// Thread 1: counter++    →  load counter; add 1; store counter
// Thread 2: counter++    →  load counter; add 1; store counter
//
// Without synchronization, both might load the same value,
// both add 1, and both store – result: incremented only once!
// This is a RACE CONDITION.

// Solutions (from lowest to highest level):
// 1. Atomic instructions (LOCK prefix, CMPXCHG)
// 2. Spinlocks (busy-wait using atomics)
// 3. Futex (fast userspace mutex – hybrid: atomic + kernel wait)
// 4. Mutexes (pthread_mutex = futex wrapper)
// 5. Read-write locks
// 6. Semaphores
// 7. Condition variables
```

Part 1: Memory Ordering and the Problem

Why Simple Load/Store Isn't Enough

CPU cores have store buffers and can reorder memory operations!

x86-64 memory model (TSO – Total Store Order):

- ✓ Loads are NOT reordered with other loads
- ✓ Stores are NOT reordered with other stores
- ✓ Loads are NOT reordered with older stores to same address
- ✗ Loads CAN be reordered with older stores to DIFFERENT addresses
- ✗ Stores can be delayed in store buffer (other CPUs see stale data)

Problem example (store buffer):

Initially: $x = 0$, $y = 0$

CPU 0:	CPU 1:
mov [x], 1	mov [y], 1
mov rax, [y]	mov rbx, [x]

Possible result: $rax = 0$, $rbx = 0$ (!)

Both stores in store buffers, not yet visible to other CPU!

Solution: memory barriers (MFENCE, LOCK prefix)

Memory Barriers

```
; x86-64 memory barrier instructions:

; MFENCE: Full memory barrier (all prior stores visible before subsequent loads)
mfence                                ; Drains store buffer, prevents all reordering

; SFENCE: Store barrier (all prior stores visible before subsequent stores)
sfence                                ; Rarely needed on x86 (stores already ordered)

; LFENCE: Load barrier (all prior loads complete before subsequent loads)
lfence                                ; Serializes loads (useful for rdtsc ordering)

; LOCK prefix: atomic operation + full barrier
; Every LOCK'd instruction acts as a full memory barrier
lock add qword [counter], 1 ; Atomic increment + memory barrier

; In practice on x86-64:
; - LOCK prefix is your primary synchronization tool
; - MFENCE is for special cases (e.g., non-temporal stores + normal loads)
; - Most "memory barrier" concerns from other architectures are handled
;   automatically by x86's strong memory model (TSO)
```


Part 2: Atomic Instructions

The LOCK Prefix


```

; LOCK prefix makes a read-modify-write operation ATOMIC
; No other CPU can access the cache line between the read and write
; Also serves as a full memory barrier

section .data
    align 8
    counter dq 0          ; Shared counter

section .text

; Atomic increment (thread-safe counter++)
atomic_increment:
    lock inc qword [rel counter]
    ret

; Atomic add (thread-safe counter += value)
; Input: RDI = value to add
atomic_add:
    lock add qword [rel counter], rdi
    ret

; Atomic exchange (swap register and memory atomically)
; Input: RDI = new value
; Output: RAX = old value
atomic_exchange:
    mov rax, rdi
    xchg [rel counter], rax    ; XCHG is implicitly LOCK'd!
    ret

; Atomic fetch-and-add (get old value while adding)
; Input: RDI = value to add
; Output: RAX = old value (before add)
atomic_fetch_add:
    mov rax, rdi
    lock xadd [rel counter], rax ; RAX = old value, [counter] = old + rax
    ret

; Atomic OR (set bits atomically)
atomic_or:
    lock or qword [rel counter], rdi
    ret

; Atomic AND (clear bits atomically)
atomic_and:
    lock and qword [rel counter], rdi
    ret

```

CMPXCHG — Compare and Swap (CAS)

```

; CMPXCHG: The fundamental building block of lock-free algorithms
;
; Semantics:
;   if ([mem] == RAX) {
;       [mem] = new_value;
;       ZF = 1; // Success
;   } else {
;       RAX = [mem]; // Load actual value
;       ZF = 0; // Failure
;   }
;
; Used to implement: mutexes, lock-free stacks, lock-free queues, etc.

; Compare-and-swap wrapper
; Input: RDI = address, RSI = expected, RDX = new value
; Output: RAX = actual old value, ZF = success
cas:
    mov rax, rsi           ; Expected value in RAX
    lock cmpxchg [rdi], rdx ; if *RDI == RAX: *RDI = RDX
    ret                   ; RAX = old value, ZF = success/fail

; Atomic increment using CAS (retry loop):
cas_increment:
    mov rax, [rel counter] ; Load current value
.retry:
    lea rdx, [rax + 1]      ; New value = old + 1
    lock cmpxchg [rel counter], rdx
    jnz .retry             ; If failed (another thread changed it), retry
    ; RAX = the value we successfully incremented from
    ret

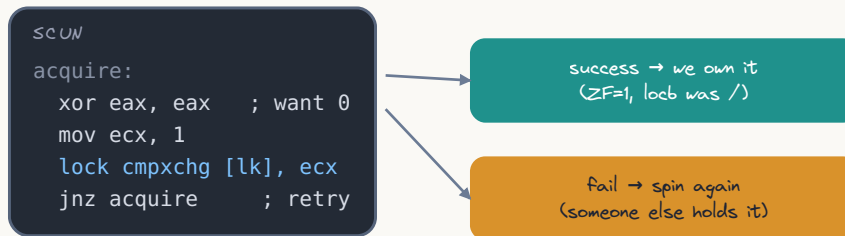
; 128-bit CAS (CMPXCHG16B) — for lock-free data structures
; Atomically compares and swaps 16 bytes
; Input: RDI = address of 16-byte value
;       RCX:RBX = new value (RCX=high, RBX=low)
;       RDX:RAX = expected value (RDX=high, RAX=low)
cas_128:
    lock cmpxchg16b [rdi] ; Compare RDX:RAX with [RDI], swap with RCX:RBX
    ret

```

Part 3: Spinlocks

Simple Spinlock (Test-and-Set)

A spinlock is one atomic compare-exchange



`cmpxchg` is atomic under the `lock` prefix; a `futex` lets waiters sleep instead of burning CPU

```

; A spinlock is the simplest mutex: busy-wait until lock is free
; Good for: very short critical sections, kernel context (can't sleep)
; Bad for: long critical sections (wastes CPU cycles spinning)

section .data
  align 8
  spinlock dq 0          ; 0 = unlocked, 1 = locked

section .text

; Acquire spinlock (busy-wait)
spin_lock:
  mov rax, 1
.try:
  xchg [rel spinlock], rax ; Atomically swap 1 into lock
  test rax, rax            ; Was it 0 (unlocked)?
  jnz .spin               ; No → it was already locked, spin
  ret                    ; Yes → we acquired it!

.spin:
  pause                  ; CPU hint: we're spinning (saves power, avoids
                        ; memory order violation pipeline flush)
  cmp qword [rel spinlock], 0 ; Check without LOCK (reduces bus traffic)
  jne .spin              ; Still locked? Keep waiting
  jmp .try               ; Looks free! Try to acquire again

; Release spinlock
spin_unlock:
  mov qword [rel spinlock], 0 ; Simple store (x86 store ordering is sufficient)
  ret                      ; No need for LOCK prefix on release (on x86)
  
```

Ticket Spinlock (Fair Ordering)

```

; Problem with simple spinlock: no guarantee of fairness
; Thread that just released might immediately re-acquire!
;
; Ticket lock: serves threads in FIFO order (like a bakery number system)

section .data
    align 8
    ticket_next dq 0      ; Next ticket to be served
    ticket_tail dq 0      ; Next ticket to be dispensed

section .text

; Acquire ticket lock
ticket_lock:
    ; Take a ticket (atomic increment of tail)
    mov rax, 1
    lock xadd [rel ticket_tail], rax ; RAX = our ticket number
    ; Now wait until our ticket is being served
.wait:
    cmp [rel ticket_next], rax
    je .acquired
    pause
    jmp .wait
.acquired:
    ret

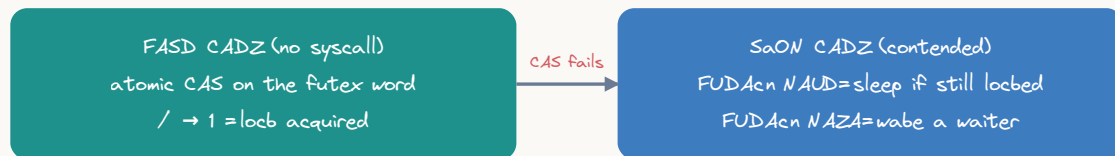
; Release ticket lock
ticket_unlock:
    ; Serve next ticket
    lock inc qword [rel ticket_next]
    ret

```

Part 4: The Futex Syscall

What is a Futex?

Futex=fast userspace path, kernel only on contention



word=0=free, 1=locked, 1=locked\waiters & kernel buys waiters by physical address

Futex = Fast Userspace muTEX

The problem with spinlocks: waste CPU while waiting

The problem with kernel mutexes: syscall overhead even when uncontended

Futex solution: HYBRID approach

- Uncontended case (common): pure user-space atomic operation (no syscall!)
- Contended case (rare): kernel puts thread to sleep

Performance comparison:

Spinlock uncontended:	~10ns (atomic exchange)
Futex uncontended:	~10ns (atomic CAS, no syscall)
Spinlock contended:	burns CPU (100% utilization while waiting)
Futex contended:	~1μs (syscall + sleep, 0% CPU while waiting)
Kernel mutex always:	~100ns (syscall even when uncontended)

Futex Operations

```

; futex(uint32_t *uaddr, int futex_op, uint32_t val,
;      const struct timespec *timeout, uint32_t *uaddr2, uint32_t val3)
;
; Syscall number: 202 (sys_futex)
;
; Key operations:
;   FUTEX_WAIT (0): if *uaddr == val, sleep until woken
;   FUTEX_WAKE (1): wake up at most val threads waiting on uaddr

section .data
    align 4
    futex_var dd 0          ; 0 = unlocked, 1 = locked (no waiters)
                           ; 2 = locked (has waiters)

section .text

; FUTEX_WAIT: sleep until *addr != val
; Input: RDI = addr, ESI = expected_val
futex_wait:
    mov rax, 202            ; sys_futex
    ; RDI = uaddr (already set)
    mov esi, esi            ; Clear upper ESI (val)
    mov edx, esi            ; val = expected value
    mov esi, 0              ; op = FUTEX_WAIT
    xor r10, r10            ; timeout = NULL (wait forever)
    xor r8, r8              ; uaddr2 = NULL
    xor r9, r9              ; val3 = 0
    ; Correction: let me redo the argument mapping
    ; RDI = uaddr, RSI = op, RDX = val, R10 = timeout
    mov rax, 202
    ; rdi already = address
    xor esi, esi            ; FUTEX_WAIT = 0
    mov edx, edx            ; val (expected value, passed in original ESI)
    xor r10, r10            ; timeout = NULL
    syscall
    ; Returns 0 on success, -EAGAIN if *uaddr != val, -EINTR if signaled
    ret

; FUTEX_WAKE: wake up waiting threads
; Input: RDI = addr, ESI = max_threads_to_wake
futex_wake:
    mov rax, 202            ; sys_futex
    ; RDI = uaddr (already set)
    mov edx, esi            ; val = number to wake
    mov esi, 1              ; op = FUTEX_WAKE
    syscall
    ; Returns number of threads woken
    ret

```

Implementing a Mutex with Futex


```

; This is essentially what pthread_mutex does internally:
; Three states:
; 0 = unlocked
; 1 = locked, no waiters
; 2 = locked, there are waiters (must wake on unlock)

section .data
    align 4
    mutex dd 0

section .text

; mutex_lock()
mutex_lock:
    ; Fast path: try to change 0 → 1 (uncontended case)
    xor eax, eax           ; Expected: 0 (unlocked)
    mov ecx, 1             ; Desired: 1 (locked, no waiters)
    lock cmpxchg dword [rel mutex], ecx
    je .locked             ; Got it! (ZF=1 means CAS succeeded)

    ; Slow path: lock is contended
    ; Change state to 2 (locked with waiters) and sleep
.wait_loop:
    ; If state is already 2, or we can change 1→2
    mov eax, 2
    xchg dword [rel mutex], eax ; Set to 2, get old value
    test eax, eax
    jz .locked             ; Was 0? We got lucky, locked now!

    ; Sleep until woken (FUTEX_WAIT with expected value 2)
    mov rax, 202           ; sys_futex
    lea rdi, [rel mutex]   ; uaddr
    xor esi, esi           ; FUTEX_WAIT
    mov edx, 2             ; val = 2 (expected value)
    xor r10, r10           ; timeout = NULL
    syscall

    ; Woke up! But might not have the lock – try again
    mov eax, 2
    xchg dword [rel mutex], eax
    test eax, eax
    jnz .wait_loop         ; Still locked? Back to sleep

.locked:
    ret

; mutex_unlock()
mutex_unlock:
    ; Atomically decrement: if it was 1→0, done (no waiters)
    mov eax, 1

```

```
lock xadd dword [rel mutex], eax ; Atomic fetch-and-subtract...
; Actually let's use a different approach:

; Better approach: exchange with 0
xor eax, eax
xchg dword [rel mutex], eax ; Unlock, get old state

cmp eax, 2 ; Were there waiters?
jne .no_waiters ; No waiters (state was 1)

; Wake one waiter
mov rax, 202 ; sys_futex
lea rdi, [rel mutex] ; uaddr
mov esi, 1 ; FUTEX_WAKE
mov edx, 1 ; Wake 1 thread
syscall

.no_waiters:
ret
```

Part 5: Lock-Free Data Structures

Lock-Free Stack (Treiber Stack)

```

; A stack where push and pop are atomic – no locks needed!
; Uses CMPXCHG to atomically update the head pointer

struc node
    .next resq 1          ; Pointer to next node
    .data resq 1          ; Data payload
endstruc

section .data
    align 8
    stack_head dq 0       ; Head of lock-free stack (NULL = empty)

section .text

; Push a node onto the lock-free stack
; Input: RDI = pointer to node (node.data already set)
lockfree_push:
    ; Load current head
    mov rax, [rel stack_head]
.retry:
    ; Set new node's next to current head
    mov [rdi + node.next], rax
    ; CAS: if head still == rax, set head = new node
    lock cmpxchg [rel stack_head], rdi
    jnz .retry           ; Head changed! Reload and retry
    ret

; Pop a node from the lock-free stack
; Output: RAX = pointer to popped node (or NULL if empty)
lockfree_pop:
    mov rax, [rel stack_head]
.retry:
    test rax, rax
    jz .empty            ; Stack is empty
    ; Read next pointer (will become new head)
    mov rdx, [rax + node.next]
    ; CAS: if head still == rax, set head = rax->next
    lock cmpxchg [rel stack_head], rdx
    jnz .retry           ; Head changed! Reload and retry
    ret                  ; RAX = popped node
.empty:
    xor eax, eax         ; Return NULL
    ret

; ABA problem warning:
; Between our load and CAS, another thread might:
;   pop A, pop B, push A back
; Our CAS sees head==A and succeeds, but the stack structure changed!
; Solution: use 128-bit CAS (CMPXCHG16B) with a counter to detect this:
;   head = {pointer, version_counter}

```

```

; On every push/pop, increment counter
; CMPXCHG16B compares both pointer AND counter

```

Lock-Free Counter with Exponential Backoff

```

; When many threads contend on one atomic variable,
; CAS retries can cause "livelock" (everyone failing continuously)
; Solution: exponential backoff

section .data
    align 8
    shared_counter dq 0

section .text

; Increment with backoff on contention
atomic_inc_backoff:
    push rbx
    mov ebx, 1                ; Initial backoff = 1 iteration

.retry:
    mov rax, [rel shared_counter]
    lea rdx, [rax + 1]
    lock cmpxchg [rel shared_counter], rdx
    je .done                 ; Success!

    ; CAS failed – backoff before retrying
    mov ecx, ebx

.backoff:
    pause                    ; CPU-friendly spin
    dec ecx
    jnz .backoff

    ; Double backoff (cap at 1024)
    shl ebx, 1
    cmp ebx, 1024
    jle .retry
    mov ebx, 1024
    jmp .retry

.done:
    pop rbx
    ret

```

Part 6: Read-Write Lock

```

; Read-write lock: many readers OR one writer (not both)
; Readers don't block each other (great for read-heavy workloads)
;
; Implementation using a single atomic value:
; state > 0: number of active readers
; state = 0: unlocked
; state = -1: writer holds lock

section .data
    align 8
    rwlock dq 0          ; 0=free, >0=readers, -1=writer

section .text

; Acquire read lock
read_lock:
    mov rax, [rel rwlock]
.retry:
    cmp rax, -1
    je .wait_reader      ; Writer active, can't read

    ; Try to increment reader count
    lea rdx, [rax + 1]    ; New value = old + 1
    lock cmpxchg [rel rwlock], rdx
    jne .retry           ; Someone else changed it, retry (RAX updated)
    ret

.wait_reader:
    pause
    mov rax, [rel rwlock]
    jmp .retry

; Release read lock
read_unlock:
    lock dec qword [rel rwlock] ; Decrement reader count
    ret

; Acquire write lock
write_lock:
    xor eax, eax          ; Expected: 0 (completely free)
    mov rdx, -1           ; Desired: -1 (writer)
.retry:
    lock cmpxchg [rel rwlock], rdx
    je .got_write         ; Was 0? Now -1, we have it!

    ; Not free - wait
    pause
    xor eax, eax
    jmp .retry

```

```
.got_write:
    ret

; Release write lock
write_unlock:
    mov qword [rel rwlock], 0 ; Simple store (was -1, set to 0)
    ret
```

Part 7: Semaphores

```

; Semaphore: counter that allows N concurrent accesses
; sem_wait: decrement (block if would go negative)
; sem_post: increment (wake one waiter)

section .data
    align 4
    semaphore dd 3          ; Allow up to 3 concurrent accesses

section .text

; sem_wait (decrement, potentially blocking)
sem_wait:
.retry:
    mov eax, [rel semaphore]
    test eax, eax
    jle .block              ; Counter is 0 or negative, must block

    ; Try to decrement
    lea edx, [eax - 1]
    lock cmpxchg dword [rel semaphore], edx
    jne .retry              ; Changed, retry
    ret                    ; Decrement successfully

.block:
    ; Use futex to sleep
    mov rax, 202            ; sys_futex
    lea rdi, [rel semaphore]
    xor esi, esi            ; FUTEX_WAIT
    xor edx, edx            ; val = 0 (expected)
    xor r10, r10            ; no timeout
    syscall

    jmp .retry              ; Woke up, try again

; sem_post (increment, potentially waking)
sem_post:
    lock inc dword [rel semaphore]

    ; Wake one waiter
    mov rax, 202
    lea rdi, [rel semaphore]
    mov esi, 1              ; FUTEX_WAKE
    mov edx, 1              ; Wake 1
    syscall
    ret

```

Part 8: Condition Variables

```

; Condition variable: wait until a condition is true
; Always used with a mutex:
;   lock(mutex)
;   while (!condition)
;       cond_wait(&cond, &mutex) // atomically unlock + sleep
;   // condition is true here, mutex is held
;   unlock(mutex)
;
; Another thread signals:
;   lock(mutex)
;   set_condition_true()
;   cond_signal(&cond) // wake one waiter
;   unlock(mutex)

section .data
    align 4
    cond_seq dd 0           ; Sequence number (incremented on signal)
    cond_mutex dd 0         ; Associated mutex

    ; Shared state
    data_ready dd 0         ; Our condition: is data ready?
    shared_data dq 0        ; The data itself

section .text

; Wait on condition variable
; Must hold cond_mutex on entry!
; Input: RDI = pointer to cond_seq, RSI = pointer to mutex
cond_wait:
    push r12
    push r13
    mov r12, rdi           ; Save cond address
    mov r13, rsi           ; Save mutex address

    ; Read current sequence number
    mov eax, [r12]         ; seq before sleep
    mov r14d, eax

    ; Release mutex (so other threads can make condition true)
    mov rdi, r13
    call mutex_unlock

    ; Sleep until sequence number changes (FUTEX_WAIT)
    mov rax, 202           ; sys_futex
    mov rdi, r12           ; uaddr = &cond_seq
    xor esi, esi           ; FUTEX_WAIT
    mov edx, r14d          ; val = seq number we saw
    xor r10, r10           ; no timeout
    syscall

```

```

; Woke up! Re-acquire mutex before returning
mov rdi, r13
call mutex_lock

pop r13
pop r12
ret

; Signal condition (wake one waiter)
; Input: RDI = pointer to cond_seq
cond_signal:
; Increment sequence number (so FUTEX_WAIT sees change)
lock inc dword [rdi]

; Wake one waiter
mov rax, 202
; rdi already = address
mov esi, 1 ; FUTEX_WAKE
mov edx, 1 ; wake 1
syscall
ret

; Broadcast (wake all waiters)
cond_broadcast:
lock inc dword [rdi]

mov rax, 202
mov esi, 1 ; FUTEX_WAKE
mov edx, 0x7FFFFFFF ; Wake all (MAX_INT)
syscall
ret

```

Part 9: Producer-Consumer with Ring Buffer

```

; Lock-free single-producer single-consumer ring buffer
; (No locks needed when there's exactly one reader and one writer!)
; Uses memory ordering guarantees of x86-64

RING_SIZE equ 1024          ; Must be power of 2
RING_MASK equ RING_SIZE - 1

section .bss
    align 64                ; Cache line alignment to avoid false sharing
    ring_buf resq RING_SIZE ; The buffer

    align 64
    ring_head resq 1         ; Write position (only producer modifies)

    align 64                ; Separate cache line!
    ring_tail resq 1        ; Read position (only consumer modifies)

section .text

; Producer: write one item
; Input: RDI = value to write
; Output: RAX = 0 on success, -1 if full
ring_push:
    mov rax, [rel ring_head]
    mov rcx, rax
    inc rcx
    and rcx, RING_MASK      ; Wrap around

    ; Check if full
    cmp rcx, [rel ring_tail]
    je .full                ; Would wrap into tail → buffer full!

    ; Write data
    mov [rel ring_buf + rax*8], rdi

    ; Update head (store barrier implicit on x86 for stores)
    ; Consumer will see the data write BEFORE the head update
    ; because x86 doesn't reorder store-store
    mov [rel ring_head], rcx

    xor eax, eax            ; Return success
    ret

.full:
    mov rax, -1
    ret

; Consumer: read one item
; Output: RAX = value, or -1 if empty
ring_pop:

```

```
mov rax, [rel ring_tail]
cmp rax, [rel ring_head]
je .empty           ; Tail == Head → buffer empty

; Read data
mov rdx, [rel ring_buf + rax*8]

; Update tail
inc rax
and rax, RING_MASK
mov [rel ring_tail], rax

mov rax, rdx        ; Return the value
ret

.empty:
mov rax, -1
ret
```

Part 10: Practical Example — Thread-Safe Counter

```

; Complete working example: multiple threads incrementing a shared counter
; Uses clone() to create threads, atomic operations for synchronization

section .data
    align 64
    counter dq 0          ; Shared counter
    barrier dd 0          ; Startup barrier

    num_threads equ 4
    increments_per_thread equ 1000000
    expected_total equ num_threads * increments_per_thread

    result_msg db "Final counter: "
    result_len equ $ - result_msg
    newline db 10
    expected_msg db "Expected:      "
    expected_len equ $ - expected_msg

section .bss
    thread_stacks resb num_threads * 65536 ; 64KB stack per thread

section .text
    global _start

; Thread function: increment counter N times
thread_func:
    ; Wait for all threads to be created (barrier)
.wait_barrier:
    pause
    cmp dword [rel barrier], num_threads
    jl .wait_barrier

    ; Increment shared counter
    mov ecx, increments_per_thread
.inc_loop:
    lock inc qword [rel counter]
    dec ecx
    jnz .inc_loop

    ; Exit thread
    mov rax, 60          ; sys_exit (exits just this thread with CLONE_THREAD)
    xor rdi, rdi
    syscall

_start:
    ; Create threads
    mov r12, num_threads
    xor r13, r13          ; Thread index

.create_loop:

```

```

    cmp r13, r12
    jge .threads_created

    ; Calculate stack top for this thread
    lea rax, [rel thread_stacks]
    mov rcx, r13
    inc rcx
    imul rcx, 65536      ; Stack size
    add rax, rcx         ; Top of this thread's stack
    sub rax, 8           ; Alignment

    ; Clone (create thread)
    mov rax, 56          ; sys_clone
    ; flags: CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND |
    ;         CLONE_THREAD | CLONE_SYSVSEM
    mov rdi, 0x00010F00  ; Simplified thread flags
    or rdi, 17           ; | SIGCHLD
    mov rsi, rax         ; child_stack = stack top
    xor rdx, rdx         ; parent_tid
    xor r10, r10         ; child_tid
    xor r8, r8           ; tls

    ; Put thread entry point on child stack
    mov qword [rsi], thread_func
    sub rsi, 8

    mov rax, 56
    syscall

    test rax, rax
    js .clone_error

    ; Signal barrier
    lock inc dword [rel barrier]

    inc r13
    jmp .create_loop

.threads_created:
    ; Signal that all threads are created
    lock inc dword [rel barrier] ; Extra increment for main

    ; Wait for threads to finish (simple spin on counter)
    ; In production, use futex or proper join mechanism
.wait_done:
    pause
    cmp qword [rel counter], expected_total
    jl .wait_done

    ; Small delay to let threads exit
    mov rax, 35

```

```

sub rsp, 16
mov qword [rsp], 0      ; 0 seconds
mov qword [rsp+8], 500000000 ; 50ms
mov rdi, rsp
xor rsi, rsi
syscall
add rsp, 16

; Print result
mov rax, 1
mov rdi, 1
lea rsi, [rel result_msg]
mov rdx, result_len
syscall

; Print counter value
mov rdi, [rel counter]
call print_number

; Print expected
mov rax, 1
mov rdi, 1
lea rsi, [rel expected_msg]
mov rdx, expected_len
syscall

mov rdi, expected_total
call print_number

; Exit
mov rax, 60
xor rdi, rdi
syscall

.clone_error:
mov rax, 60
mov rdi, 1
syscall

; Print number followed by newline
print_number:
push rbp
mov rbp, rsp
sub rsp, 32

mov rax, rdi
lea rsi, [rbp - 1]
mov byte [rsi], 10 ; Newline
mov rcx, 1
mov r8, 10

```

```

.digits:
    xor rdx, rdx
    div r8
    add dl, '0'
    dec rsi
    mov [rsi], dl
    inc rcx
    test rax, rax
    jnz .digits

    mov rax, 1
    mov rdi, 1
    mov rdx, rcx
    syscall

    leave
    ret

```

Exercises

1. **Race condition demo:** Create a program with two threads incrementing a counter WITHOUT atomics. Run it many times and observe the incorrect final count.
 2. **Spinlock vs Futex benchmark:** Implement both and measure throughput under low contention (2 threads) and high contention (8 threads) scenarios.
 3. **Lock-free queue:** Implement a multi-producer, multi-consumer lock-free queue using CMPXCHG. Handle the ABA problem with CMPXCHG16B.
 4. **Readers-writer benchmark:** Create a workload with 90% reads and 10% writes. Compare throughput of a mutex vs a read-write lock.
 5. **Producer-consumer:** Implement a bounded producer-consumer using the ring buffer. Create 2 producers and 2 consumers. Measure throughput.
-

Key Takeaways

Concept	Assembly Implementation
Atomic increment	<code>lock inc qword [addr]</code>
Compare-and-swap	<code>lock cmpxchg [addr], new</code> (expected in RAX)
Memory barrier	<code>mfence</code> or any LOCK'd instruction
Spinlock	XCHG loop with PAUSE instruction
Futex (fast path)	Atomic CAS — no syscall needed!
Futex (slow path)	<code>sys_futex(FUTEX_WAIT)</code> → kernel sleeps thread
Lock-free stack	CAS on head pointer (watch for ABA problem)
SPSC ring buffer	No atomics needed! (x86 TSO handles ordering)
False sharing	Separate variables on different 64-byte cache lines

Further Reading

- Intel SDM Vol. 3A, Chapter 8: Multiple-Processor Management
- `man 2 futex` — Linux futex interface
- “The Art of Multiprocessor Programming” by Herlihy & Shavit
- Linux kernel source: `kernel/futex.c`, `kernel/locking/`